

The Semware Editor Professional Manual

TSE Pro v4.40TD

© SemWare Corporation.
All rights reserved worldwide.

Visit www.semware.com for lots of TSE info and additional macros, or to join the mailing list to exchange info with TSE users worldwide.

This manual was generated on 05 July 12 from TSE's interactive Help system with version 2.4.4 of Carlo Hogeveen's Hlp2txt macro.

Characters with an ascii code below 32 and above 127 were converted to readable alternatives. You will especially notice this in box drawings. The result is not as pretty as the original, but quite readable.

This html manual was tested with Internet Explorer 6.

Index

[MAXPATH](#)

Subtopic of: [Pre-Defined Constants](#)

[#define](#)

Subtopic of: [Conditional Compilation Directives](#)

[#else](#)

Subtopic of: [Conditional Compilation Directives](#)

[#endif](#)

Subtopic of: [Conditional Compilation Directives](#)

[#if](#)

Subtopic of: [Conditional Compilation Directives](#)
[#ifdef](#)

Subtopic of: [Conditional Compilation Directives](#)
[#ifexist](#)

Subtopic of: [Conditional Compilation Directives](#)
[#ifndef](#)

Subtopic of: [Conditional Compilation Directives](#)
[#ifnexist](#)

Subtopic of: [Conditional Compilation Directives](#)

--- A ---

[A Walk-Through of Window Commands](#)

[AbandonEditor\(\)](#)

[AbandonFile\(\)](#)

[About](#)

[Abs\(\)](#)

[Accessing Sub-Menu Commands and Options](#)

[Accessing the Pop-Up Ascii Chart](#)

[Activating Editing Modes](#)

Subtopic of: [Working in the Editor](#)

[Activating the Pull-Down Menu](#)

[AddAutoLoadMacro\(\)](#)

[AddFFInfoToBuffer\(\)](#)

[AddFileToRing\(\)](#)

[AddHistoryStr\(\)](#)

[Adding a Variable](#)

Subtopic of: [Some Simple Macro Examples](#)

[Adding an External Macro to the Potpourri](#)

Subtopic of: [Customization Using Sal Compiled Macros](#)

[Adding Files to the Ring: Specifying and Opening Files](#)

[Adding Macros to the User-Interface \(UI\) Macro File](#)

Subtopic of: [Customization Using Sal Compiled Macros](#)

[Adding More Functionality](#)

Subtopic of: [Customization Using Sal Compiled Macros](#)

[Adding or Changing Key Assignments](#)

Subtopic of: [Customization Using Sal Compiled Macros](#)

[Adding Your Own Commands](#)

Subtopic of: [Customization Using Sal Compiled Macros](#)

[Additional Options Available at Startup](#)

Subtopic of: [Starting an Editing Session](#)

[Additional Regular Expression Examples](#)

Subtopic of: [Searching With Regular Expressions](#)

[AddLine\(\)](#)

[Addr\(\)](#)

[AddTrailingSlash\(\)](#)

[AdjPtr\(\)](#)

[Affects](#)

Subtopic of: [Editor Variables](#)

[Alarm\(\)](#)

[Alternate Methods for Expanding a Template](#)

Subtopic of: [Template Mode](#)

[Alternative Modes For Viewing Files and](#)

Subtopic of: [Statements](#)

[Appendix A: Technical Specifications](#)

[Appendix B: Command Implementation](#)

[Appendix C: Commands By Category](#)

[Appendix C: Standard Key Assignments](#)

[Appendix D: Variables By Category](#)

[Appendix E: Command Implementation](#)

[Appendix F: Standard Configuration Settings](#)

[Appendixes](#)

[Arguments - Passing and Retrieving Parameters](#)

Subtopic of: [Macro Procedures](#)

[Arithmetic Operators](#)

Subtopic of: [Statements](#)

[Asc\(\)](#)

[Ask\(\)](#)

[AskFilename\(\)](#)

[AskNumeric\(\)](#)

[Assigning a Macro to a Key](#)

Subtopic of: [Some Simple Macro Examples](#)

[Assigning Macros and Commands to Keys](#)

[Associating a SyntaxHilite Mapping Set with a Group of Files](#)

Subtopic of: [Syntaxhilite Mode](#)

[Attr](#)

[Augment Load Dir](#)

Subtopic of: [Command-Line Options](#)

[Augmenting the Editor Load Directory](#)

[AutoIndent](#)

[Autoindent Mode](#)

[autoload file](#)

Subtopic of: [Startup Processing](#)

--- B ---

[BackSpace\(\)](#)

[BackupExt](#)

[BackupPath](#)

[Basic Concepts](#)

[Basic Find and Replace Options](#)

[Basic Operation of Template Mode](#)

Subtopic of: [Template Mode](#)

[Beep](#)

[BegFile\(\)](#)

[BegLine\(\)](#)

[BegLineTog\(\)](#)

[BegWindow\(\)](#)

[BegWord\(\)](#)

[Behavior of WrapPara within a Block](#)

Subtopic of: [Commands to Format Text](#)

[Binary Load](#)

Subtopic of: [Command-Line Options](#)

[Binary Mode](#)

Subtopic of: [Alternative Modes For Viewing Files](#)

[BinaryMode\(\)](#)

[Bitwise Logical Operators](#)

Subtopic of: [Statements](#)

[Block Operations](#)

[Block-Marking Commands](#)

Subtopic of: [Marking and Manipulating a Block of Text](#)

[BlockAttr](#)

Subtopic of: [COLOR Set](#)

[BlockBegCol](#)

[BlockBegLine](#)

[BlockEndCol](#)

[BlockEndLine](#)

[BlockId](#)

[Bookmark Commands](#)

[Bookmark Shortcut](#)

[Bookmarks](#)

[Boolean Expressions](#)

Subtopic of: [Statements](#)

[Break](#)

[break statement](#)

Subtopic of: [Statements](#)

[BreakHookChain\(\)](#)

[Breaking Out of Run-Away Macros](#)

[Browse Load](#)

Subtopic of: [Command-Line Options](#)

[Browse Mode](#)

Subtopic of: [Alternative Modes For Viewing Files](#)

[BrowseMode\(\)](#)

[BufferFlags](#)

[Buffering Screen Output](#)

Subtopic of: [Video Output Commands](#)

[Buffers](#)

[BufferType\(\)](#)

[BufferVideo\(\)](#)

[BuildPickBufferEx\(\)](#)

[by](#)

Subtopic of: [Statements](#)

--- C ---

[C Mode](#)

Subtopic of: [Special Programming-Language Modes](#)

[case](#)

Subtopic of: [Statements](#)

[CenterFinds](#)

[ChainCmd\(\)](#)

[ChangeCurrFilename\(\)](#)

[ChangedFilesExist\(\)](#)

[ChangeShellPrompt](#)

[Changing Position by Multiple Characters or Lines](#)

Subtopic of: [Changing Position in a File](#)

[Changing Position by One Character or Line](#)

Subtopic of: [Changing Position in a File](#)

[Changing Position in a File](#)

[Changing Settings for Editing Options Using IConfig](#)

Subtopic of: [Simple Customization from Within the Editor](#)

[Changing the Case of Characters in a Block](#)

Subtopic of: [Marking and Manipulating a Block of Text](#)

[Changing the HelpLines](#)

Subtopic of: [Customization Using Sal Compiled Macros](#)

[Changing Window Size and Orientation](#)

[ChDir\(\)](#)

[CheckDefaultExt\(\)](#)

[Checking Your Spelling](#)

[ChooseFont\(\)](#)

[Chr\(\)](#)

[ChrSet\(\)](#)

[ClearBit\(\)](#)

[ClearBufferDaTmAttr\(\)](#)

[Clearing Portions of the Screen](#)

Subtopic of: [Video Output Commands](#)

[ClearUndoRedoList\(\)](#)

[ClipboardId](#)

[ClipBoards](#)

Subtopic of: [Manipulating Blocks Via Clipboards](#)

[ClosePrint\(\)](#)

[CloseWindow\(\)](#)

[Closing Editing Windows](#)

[ClrEol\(\)](#)

[ClrScr\(\)](#)

[CmpiStr\(\)](#)

[Color\(\)](#)

[COLOR Set](#)

[COLOR VARIABLES](#)

[Command Prompts](#)

[Command Prompts and Keydefs](#)

[Command Prompts and Picklists](#)

[Command-Line Options](#)

[Command/Format Options](#)

Subtopic of: [Configuration Options Available Interactively](#)

[Commands and Features That Allow Recovery of Text](#)

[Commands to Add Text](#)

[Commands to Copy Text](#)

[Commands to Delete Text](#)

[Commands to Exit from the Editor](#)

Subtopic of: [Saving and Closing Files, and Exiting from the Editor](#)

[Commands to Format Text](#)

[Commands to Move Text](#)

[Commands to Save and/or Close Files](#)

Subtopic of: [Saving and Closing Files, and Exiting from the Editor](#)

[Comments](#)

[Compiling Macro Source](#)

Subtopic of: [Introduction to External Compiled Macros](#)

[Compiling Macros](#)

[Compiling Macros \(Overview\)](#)

[Compiling Macros from Inside the Editor](#)

Subtopic of: [Compiling Macros \(Overview\)](#)

[Compiling Macros from Outside the Editor](#)

Subtopic of: [Compiling Macros \(Overview\)](#)

[Compiling Programs from Within the Editor](#)

[Concatenation Operators](#)

Subtopic of: [Statements](#)

[Conditional Compilation Directives](#)

[Conditional Execution ... if and iif](#)

Subtopic of: [Statements](#)

[Config](#)

Subtopic of: [External Macros Vs. the User-Interface Macro](#)

[Configuration Options Available Interactively](#)

[Configure SyntaxHilite Mapping Sets](#)

Subtopic of: [Configuration Options Available Interactively](#)

[Configuring the Editor Interactively](#)

[constant](#)

Subtopic of: [Constants](#)

[Constant Expressions](#)

Subtopic of: [Statements](#)

[Constants](#)

[Context-Sensitive Help](#)

Subtopic of: [The Helpsystem](#)

[Control Statements](#)

Subtopic of: [Statements](#)

[Copy\(\)](#)

[CopyAppend\(\)](#)

[CopyAppendToWinClip\(\)](#)

[CopyBlock\(\)](#)

[CopyFile\(\)](#)

[CopyToWinClip\(\)](#)

[CreateBuffer\(\)](#)

[CreateTempBuffer\(\)](#)

[Creating a Custom User-Interface \(UI\) Macro File](#)

Subtopic of: [Customization Using Sal Compiled Macros](#)

[Creating a Menu](#)

Subtopic of: [Some Simple Macro Examples](#)

[Creating a TSEStart Macro File](#)

Subtopic of: [Customization Using Sal Compiled Macros](#)

[Creating an External Compiled Macro](#)

Subtopic of: [Customization Using Sal Compiled Macros](#)

[Creating and Using Compiled Macros](#)

Subtopic of: [Introduction to External Compiled Macros](#)

[Creating and Using Macros](#)

[Creating Buffers](#)

Subtopic of: [Buffers](#)

[Creating Keyboard Macros](#)

Subtopic of: [Simple Customization from Within the Editor](#)

[Creating Macros](#)

[Creating Macros \(Overview\)](#)

[Creating Templates/Abbreviations](#)

Subtopic of: [Simple Customization from Within the Editor](#)

[Creating View Port Windows](#)

Subtopic of: [Video Output Commands](#)

[Creating Your Own Template](#)

Subtopic of: [Template Mode](#)

[CReturn\(\)](#)

[CurrChar\(\)](#)

[CurrCol\(\)](#)

[CurrDir\(\)](#)

[CurrExt\(\)](#)

[CurrFilename\(\)](#)

[CurrHistoryList](#)

[CurrLine\(\)](#)

[CurrLineLen\(\)](#)

[CurrMacroFilename\(\)](#)

[CurrPos\(\)](#)

[CurrRow\(\)](#)

[CurrVideoMode](#)

[CurrWinBorderAttr](#)

Subtopic of: [COLOR Set](#)

[CurrWinBorderType](#)

[CurrXoffset\(\)](#)

[Cursor](#)

[CursorAttr](#)

Subtopic of: [COLOR Set](#)

[CursorInBlockAttr](#)

Subtopic of: [COLOR Set](#)

[CursorTabWidth](#)

[Customization Using Sal Compiled Macros](#)

[Customizing the Editor](#)

[Cut\(\)](#)

[CutAppend\(\)](#)

[CutAppendToWinClip\(\)](#)

[CutToWinClip\(\)](#)

--- D ---

[Datadef](#)

[DateFormat](#)

[DateSeparator](#)

[Debugging Compiled Macros](#)

Subtopic of: [Introduction to External Compiled Macros](#)

[Debugging Macros](#)

[Declaring Global Integers](#)

Subtopic of: [Variables](#)

[Declaring Global Strings](#)

Subtopic of: [Variables](#)

[Declaring Local Integers](#)

Subtopic of: [Variables](#)

[Declaring Local Strings](#)

Subtopic of: [Variables](#)

[Default](#)

Subtopic of: [Editor Variables](#)

[DefaultExt](#)

[Defining a Menubar](#)

Subtopic of: [Menus](#)

[Defining a Pull-Down or Pop-Up Menu](#)

Subtopic of: [Menus](#)

[DelAllBookMarks\(\)](#)

[DelAnyChar\(\)](#)

[DelAnyCharOrBlock\(\)](#)

[DelAutoLoadMacro\(\)](#)

[Delay\(\)](#)

[DelBlock\(\)](#)

[DelBookmark\(\)](#)

[DelBufferVar\(\)](#)

[DelChar\(\)](#)

[DelCharOrBlock\(\)](#)

[Deleting an Existing Template](#)

Subtopic of: [Template Mode](#)

[DelGlobalVar\(\)](#)

[DelHistory\(\)](#)

[DelHistoryStr\(\)](#)

[DelLeftWord\(\)](#)

[DelLine\(\)](#)

[DelRightWord\(\)](#)

[DelStr\(\)](#)

[DelToEol\(\)](#)

[DelWindow\(\)](#)

[DetabOnLoad](#)

[Directive tokens](#)

Subtopic of: [Syntaxhilite Mode](#)

[Directive1Attr](#)

Subtopic of: [COLOR Set](#)

[Directive2Attr](#)

Subtopic of: [COLOR Set](#)

[Directive3Attr](#)

Subtopic of: [COLOR Set](#)

[Disable\(\)](#)

[Display/Color Options](#)

Subtopic of: [Configuration Options Available Interactively](#)

[DisplayBoxed](#)

[DisplayMode\(\)](#)

[DistanceToTab\(\)](#)

[DLL Parameters and Return Values](#)

Subtopic of: [Macro Procedures](#)

[do](#)

Subtopic of: [Statements](#)

[Dos\(\)](#)

[DosCmdLine](#)

[DosIOResult\(\)](#)

[Down\(\)](#)

[downto](#)

Subtopic of: [Statements](#)

[DrawBox\(\)](#)

[Drawing Lines in the Text](#)

[DupLine\(\)](#)

--- E ---

[EditAutoLoadList\(\)](#)

[EditBuffer\(\)](#)

[EditFile\(\)](#)

[Editing Modes](#)

[Editing Multiple Files and Displaying Multiple Windows](#)

Subtopic of: [Working in the Editor](#)

[Editor Commands](#)

[Editor Files and Directories](#)

[Editor Startup Processing](#)

[Editor Variables](#)

[EDITOR VERSION](#)

Subtopic of: [Pre-Defined Constants](#)

[EditThisFile\(\)](#)

[Effects of Expandtabs](#)

[Element Classes Recognized by SyntaxHilite Mode](#)

Subtopic of: [Syntaxhilite Mode](#)

[else](#)

Subtopic of: [Statements](#)

[elseif](#)

Subtopic of: [Statements](#)

[EmptyBuffer\(\)](#)

[EmptyWinClip\(\)](#)

[Enable\(\)](#)

[end](#)

Subtopic of: [Macro Procedures](#)

[endcase](#)

Subtopic of: [Statements](#)

[EndConfig](#)

Subtopic of: [External Macros Vs. the User-Interface Macro](#)

[enddo](#)

Subtopic of: [Statements](#)

[EndFile\(\)](#)

[endfor](#)

Subtopic of: [Statements](#)

[endif](#)

Subtopic of: [Statements](#)

[EndLine\(\)](#)

[EndLineTog\(\)](#)

[endloop](#)

Subtopic of: [Statements](#)

[EndProcess\(\)](#)

[endwhile](#)

Subtopic of: [Statements](#)

[EndWindow\(\)](#)

[EndWord\(\)](#)

[EntabCurrLine\(\)](#)

[EntabOnSave](#)

[EOFMarkerAttr](#)

Subtopic of: [COLOR Set](#)

[EOFTYPE](#)

[EOLType](#)

[EquateEnhancedKbd](#)

[EquiStr\(\)](#)

[EraseDiskFile\(\)](#)

[Escape\(\)](#)

[Event Macros](#)

[Event Macros Defined by the User](#)

Subtopic of: [Event Macros](#)

[Event Macros with Predefined Names](#)

Subtopic of: [Event Macros](#)

[Examples](#)

Subtopic of: [Editor Commands](#)

[ExecHook\(\)](#)

[ExecLoadedMacro\(\)](#)

[ExecMacro\(\)](#)

[ExecScrapMacro\(\)](#)

[Execute Macro](#)

Subtopic of: [Command-Line Options](#)

[Executing Commands](#)

Subtopic of: [Working in the Editor](#)

[Executing External Commands from Within the Editor](#)

[Executing Macros \(Overview\)](#)

[Executing Supplemental Macros from the Potpourri](#)

[ExistBufferVar\(\)](#)

[ExistGlobalVar\(\)](#)

[Exit\(\)](#)

[Exiting from the Menu System](#)

[ExpandPath\(\)](#)

[ExpandTabs](#)

[ExpandTabsToSpaces\(\)](#)

[Expressions](#)

Subtopic of: [Statements](#)

[Expressions in Control Statements](#)

Subtopic of: [Statements](#)

[Extending a Block](#)

Subtopic of: [Marking and Manipulating a Block of Text](#)

[External Macro Files](#)

Subtopic of: [External Macros Vs. the User-Interface Macro](#)

[External Macros Vs. the User-Interface Macro](#)

--- F ---

[FALSE](#)

Subtopic of: [Pre-Defined Constants](#)

[fclose\(\)](#)

[fCreate\(\)](#)

[fDup\(\)](#)

[fDup2\(\)](#)

[FFAttribute\(\)](#)

[FFDate\(\)](#)

[FFDateStr\(\)](#)

[FFName\(\)](#)

[FFSize\(\)](#)

[FFTime\(\)](#)

[FFTimeStr\(\)](#)

[File PickList Settings](#)

Subtopic of: [Configuration Options Available Interactively](#)

[FileChanged\(\)](#)

[FileExists\(\)](#)

[FileLocking](#)

[Files Needed By Iconfig](#)

[Files Needed By Spellcheck](#)

[Files Needed by the HelpSystem](#)

Subtopic of: [The Helpsystem](#)

[FillBlock\(\)](#)

[Find\(\)](#)

[FindFileClose\(\)](#)

[FindFirstFile\(\)](#)

[FindHistoryStr\(\)](#)

[FindNextFile\(\)](#)

[FindOptions](#)

[FindThisFile\(\)](#)

[Flip\(\)](#)

[Flow of Control](#)

Subtopic of: [Some Simple Macro Examples](#)

[FlushProfile\(\)](#)

[FontFlags](#)

[FontName](#)

[FontSize](#)

[fOpen\(\)](#)

[for](#)

Subtopic of: [Statements](#)

[Format\(\)](#)

[Formatting Print Output](#)

[forward](#)

Subtopic of: [Macro Procedures](#)

[Forward Procedures](#)

Subtopic of: [Macro Procedures](#)

[fRead\(\)](#)

[fReadFile\(\)](#)

[fSeek\(\)](#)

[Full Configuration](#)

Subtopic of: [Methods For Interactively Configuring the Editor](#)

[FullWindow\(\)](#)

[Fully Customizing the Editor](#)

[Function Call Statements](#)

Subtopic of: [Statements](#)

[fWrite\(\)](#)

[fWriteFile\(\)](#)

--- G ---

[General Use of the Mouse](#)

[GetBit\(\)](#)

[GetBookMarkInfo\(\)](#)

[GetBufferDaTmAttr\(\)](#)

[GetBufferId\(\)](#)

[GetBufferInt\(\)](#)

[GetBufferStr\(\)](#)

[GetCharWidthHeight\(\)](#)

[GetClipboardBlockType\(\)](#)

[GetClockTicks\(\)](#)

[GetColorTableValue\(\)](#)

[GetDate\(\)](#)

[GetDateStr\(\)](#)

[GetDir\(\)](#)

[GetDrive\(\)](#)

[GetEnvStr\(\)](#)

[GetFileToken\(\)](#)

[GetFont\(\)](#)

[GetFoundText\(\)](#)

[GetFreeHistory\(\)](#)

[GetGlobalInt\(\)](#)

[GetGlobalStr\(\)](#)

[GetHistoryStr\(\)](#)

[GetHookState\(\)](#)

[GetKey\(\)](#)

[GetKeyFlags\(\)](#)

[GetMarkedText\(\)](#)

[GetMaxRowsCols\(\)](#)

[GetNextProfileItem\(\)](#)

[GetNextProfileSectionName\(\)](#)

[GetPositionInfo\(\)](#)

[GetProfileInt\(\)](#)

[GetProfileStr\(\)](#)

[GetStr\(\)](#)

[GetStrAttr\(\)](#)

[GetStrAttrXY\(\)](#)
[GetStrFromWinClip\(\)](#)
[GetStrXY\(\)](#)
[GetSynFilename\(\)](#)
[GetSynLanguageType\(\)](#)
[GetSynMultiLnDlmt\(\)](#)
[GetSynToEOL\(\)](#)
[GetText\(\)](#)
[GetTime\(\)](#)
[GetTimeStr\(\)](#)
[Getting Help](#)

Subtopic of: [Working in the Editor](#)

[Getting Help in the Editor](#)
[Getting User Input](#)
[GetToken\(\)](#)
[GetWheelScrollLines\(\)](#)
[GetWindowTitle\(\)](#)
[GetWinHandle\(\)](#)
[GetWord\(\)](#)
[Global vs. Local Variables](#)

Subtopic of: [Macro Procedures](#)

[GlobalUnDelete\(\)](#)
[Go To Line/Column Number](#)

Subtopic of: [Command-Line Options](#)

[goto](#)

Subtopic of: [Statements](#)

[GotoBlockBegin\(\)](#)
[GotoBlockBeginCol\(\)](#)
[GotoBlockEnd\(\)](#)
[GotoBlockEndCol\(\)](#)
[GotoBufferId\(\)](#)
[GotoColumn\(\)](#)
[GotoLine\(\)](#)
[GotoMark\(\)](#)
[GotoMouseCursor\(\)](#)
[GotoPos\(\)](#)
[GotoRow\(\)](#)
[GotoWindow\(\)](#)
[GotoXoffset\(\)](#)
[GotoXY\(\)](#)
[GUIStartupFlags](#)

--- H ---

[halt](#)

Subtopic of: [Statements](#)

[Help\(\)](#)

[HelpBoldAttr](#)

Subtopic of: [COLOR Set](#)

[Helpdef](#)

[HelpInfoAttr](#)

Subtopic of: [COLOR Set](#)

[HelpItalicsAttr](#)

Subtopic of: [COLOR Set](#)

[HelpLine bars](#)

Subtopic of: [The Customizable Helpline](#)

[HelpLineDelay](#)

[Helplines](#)

[HelpLinkAttr](#)

Subtopic of: [COLOR Set](#)

[HelpSelectAttr](#)

Subtopic of: [COLOR Set](#)

[HelpSystem Color Variables](#)

Subtopic of: [COLOR Set](#)

[HelpTextAttr](#)

Subtopic of: [COLOR Set](#)

[Hex Load](#)

Subtopic of: [Command-Line Options](#)

[Hex Mode](#)

Subtopic of: [Alternative Modes For Viewing Files](#)

[HexEdit\(\)](#)

[HiByte\(\)](#)

[HideMouse\(\)](#)

[HiLiteAttr](#)

Subtopic of: [COLOR Set](#)

[HiLiteFoundText\(\)](#)

[History Buffer for Prompt Boxes](#)

Subtopic of: [Command Prompts](#)

[HiWord\(\)](#)

[Hook\(\)](#)

[HWindow\(\)](#)

--- I ---

[Identifiers](#)

[IdleTime](#)

[if](#)

Subtopic of: [Statements](#)

[iif](#)

Subtopic of: [Statements](#)

[in](#)

Subtopic of: [Statements](#)

[include](#)

Subtopic of: [Include Files](#)

[Include Files](#)

[IncompleteQuoteAttr](#)

Subtopic of: [COLOR Set](#)

[Incrementalsearch Feature](#)

[Indexing Operators](#)

Subtopic of: [Statements](#)

[InitSynhiCurrFile\(\)](#)

[Insert](#)

[Insert Mode](#)

[InsertCursorSize](#)

[InsertData\(\)](#)

[InsertFile\(\)](#)

[Inserting the Date and Time Into the Text](#)

[InsertLine\(\)](#)

[InsertLineBlocksAbove](#)

[InsertText\(\)](#)

[InsStr\(\)](#)

[Installation of the Semware Editor Professional](#)

[Installation Steps](#)

[Installing a User-Interface \(UI\) Macro File](#)

Subtopic of: [Customization Using Sal Compiled Macros](#)

[Integer](#)

Subtopic of: [Variables](#)

[Integer Variables](#)

Subtopic of: [Variables](#)

[Introduction](#)

[Introduction to External Compiled Macros](#)

[is32BitApp\(\)](#)

[isAlpha\(\)](#)

[isAlphaNum\(\)](#)

[isAutoLoaded\(\)](#)

[isBlockInCurrFile\(\)](#)

[isBlockMarked\(\)](#)

[isBookMarkSet\(\)](#)

[isCharDevice\(\)](#)

[isCurrLineInBlock\(\)](#)

[isCursorInBlock\(\)](#)

[isDigit\(\)](#)

[isFullScreen\(\)](#)

[isGUI\(\)](#)

[isHexDigit\(\)](#)

[isKeyAssigned\(\)](#)

[isLower\(\)](#)

[isMacroLoaded\(\)](#)

[isTrailingSlash\(\)](#)

[isTypeableKey\(\)](#)

[isUpper\(\)](#)

[isWhite\(\)](#)

[isWildPath\(\)](#)

[isWinClipAvailable\(\)](#)

[isWord\(\)](#)

[isZoomed\(\)](#)

[Iterative Statements ... while, repeat, for, do, and loop](#)

Subtopic of: [Statements](#)

--- J ---

[JoinLine\(\)](#)

[Jump Statement ... goto](#)

Subtopic of: [Statements](#)

--- K ---

[KbdMacroRecording](#)

[KbdMacroRunning](#)

[KbdPath](#)

[KeepUndoBeyondSave](#)

[KeepWinOnTop](#)

[Key](#)

[Key Assignments Displayed on the Menu](#)

[Keyboard Macros](#)

[Keyboard/Mouse Options](#)

Subtopic of: [Configuration Options Available Interactively](#)

[KeyCode\(\)](#)

[Keydef](#)

[KeyName\(\)](#)

[KeyPressed\(\)](#)

[Keyword tokens](#)

Subtopic of: [Syntaxhilite Mode](#)

[Keyword Wordset](#)

Subtopic of: [Syntaxhilite Mode](#)

[KeyWords1Attr](#)

Subtopic of: [COLOR Set](#)

[KeyWords2Attr](#)

Subtopic of: [COLOR Set](#)

[KeyWords3Attr](#)

Subtopic of: [COLOR Set](#)

[KeyWords4Attr](#)

Subtopic of: [COLOR Set](#)

[KeyWords5Attr](#)

Subtopic of: [COLOR Set](#)

[KeyWords6Attr](#)

Subtopic of: [COLOR Set](#)

[KeyWords7Attr](#)

Subtopic of: [COLOR Set](#)

[KeyWords8Attr](#)

Subtopic of: [COLOR Set](#)

[KeyWords9Attr](#)

Subtopic of: [COLOR Set](#)

[KillBlock\(\)](#)

[KillFile\(\)](#)

[KillLine\(\)](#)

[KillLocation\(\)](#)

[KillMax](#)

[KillPosition\(\)](#)

[KillToEol\(\)](#)

--- L ---

[LastKey](#)

[LastMouseKey](#)

[LastMouseX](#)

[LastMouseY](#)

[lDos\(\)](#)

[Left\(\)](#)

[LeftMargin](#)

[LeftStr\(\)](#)

[Length\(\)](#)

[lFind\(\)](#)

[Limits](#)

Subtopic of: [Editor Variables](#)

[LineDraw\(\)](#)

[LineDrawChar](#)

[LineDrawing](#)

[LineDrawType](#)

[LineNumbersAttr](#)

Subtopic of: [COLOR Set](#)

[LineTypeMenu\(\)](#)

[LinkSynFile\(\)](#)

[List\(\)](#)

[ListFooter\(\)](#)

[ListHeader\(\)](#)

[Listing All Occurrences of a Specified Text String](#)

[Literal\(\)](#)

[lList\(\)](#)

[Load All](#)

Subtopic of: [Command-Line Options](#)

[load directory](#)

Subtopic of: [Augmenting the Editor Load Directory](#)

[Load Keyboard Macro](#)

Subtopic of: [Command-Line Options](#)

[Load Macro](#)

Subtopic of: [Command-Line Options](#)

[LoadBuffer\(\)](#)

[LoadDir\(\)](#)

[LoadHistory\(\)](#)

[Loading and Executing Compiled Macros](#)

Subtopic of: [Introduction to External Compiled Macros](#)

[Loading Keyboard Macros Saved in a Previous Session](#)

Subtopic of: [Keyboard Macros](#)

[Loading, Processing, and Saving Files](#)

[Loading/Executing Macros](#)

[LoadKeyMacro\(\)](#)

[LoadMacro\(\)](#)

[LoadProfileSection\(\)](#)

[LoadProfileSectionNames\(\)](#)

[LoadStartupMacros\(\)](#)

[LoadSynhiAssoc\(\)](#)

[LoadWildFromDOS](#)

[LoadWildFromInside](#)

[LoByte\(\)](#)

[LockCurrentFile\(\)](#)

[Locking Files](#)

[LogDrive\(\)](#)

[Logical Operators](#)

Subtopic of: [Statements](#)

[LongestLineInBuffer\(\)](#)

[loop](#)

Subtopic of: [Statements](#)

[Lower\(\)](#)

[LoWord\(\)](#)

[lRead\(\)](#)

[lReadNumeric\(\)](#)

[lRepeatFind\(\)](#)

[lReplace\(\)](#)

[lShowEntryScreen\(\)](#)

[LTrim\(\)](#)

[lVersion\(\)](#)

--- M ---

[macro file search order](#)

Subtopic of: [Introduction to External Compiled Macros](#)

[Macro File Search Order and the Tsepath Variable](#)

[Macro Language Syntax](#)

[Macro Procedures](#)

[Macro Source Files Vs. Macro Object Files](#)

[Macro Syntax](#)

[MacroCmdLine](#)

[Macros](#)

[MacroStackAvail\(\)](#)

[Main](#)

Subtopic of: [Macro Procedures](#)

[Main macro](#)

Subtopic of: [Event Macros](#)

[Maintaining a Macro Autoload File](#)

Subtopic of: [Customization Using Sal Compiled Macros](#)

[MakeBackups](#)

[MakeTempName\(\)](#)

[Making Changes to the Pull-Down Menu](#)

Subtopic of: [Customization Using Sal Compiled Macros](#)

[Managing Multiple Files and Maintaining the File Ring](#)

[Manipulating Blocks Via Clipboards](#)

[Manipulating Text](#)

[Manipulating Text in a Block](#)

Subtopic of: [Marking and Manipulating a Block of Text](#)

[Manipulating Text in Buffers](#)

Subtopic of: [Buffers](#)

[Manipulating the Video Output Position and the Visible Cursor](#)

Subtopic of: [Video Output Commands](#)

[Mapping SyntaxHilite Tokens to Colors & Setting Other Options](#)

Subtopic of: [Syntaxhilite Mode](#)

[Mark\(\)](#)

[MarkBlockBegin\(\)](#)

[MarkBlockEnd\(\)](#)

[MarkChar\(\)](#)

[MarkColumn\(\)](#)

[MarkFoundText\(\)](#)

[Marking](#)

[Marking a Block](#)

[Marking and Manipulating a Block of Text](#)

[MarkLine\(\)](#)

[MarkStream\(\)](#)

[MarkToEOL\(\)](#)

[MarkWord\(\)](#)

[MatchFilename\(\)](#)

[Max\(\)](#)

[MaxHistoryPerList](#)

[MaxHistorySize](#)

[Maximum closure](#)

Subtopic of: [Searching With Regular Expressions](#)

[MAXINT](#)

Subtopic of: [Pre-Defined Constants](#)

[MAXLINELEN](#)

Subtopic of: [Pre-Defined Constants](#)

[MaxRecentFiles](#)

[MAXSTRINGLEN](#)

Subtopic of: [Pre-Defined Constants](#)

[Mega-Conditional Statement ... case](#)

Subtopic of: [Statements](#)

[menu](#)

Subtopic of: [Menus](#)

[Menu/HelpLine Color Variables](#)

Subtopic of: [COLOR Set](#)

[menubar](#)

Subtopic of: [Menus](#)

[MenuBarAttr](#)

Subtopic of: [COLOR Set](#)

[MenuGrayAttr](#)

Subtopic of: [COLOR Set](#)

[MenuKey\(\)](#)

[MenuOption\(\)](#)

[Menus](#)

[MenuSelectAttr](#)

Subtopic of: [COLOR Set](#)

[MenuSelectGrayAttr](#)

Subtopic of: [COLOR Set](#)

[MenuSelectLtrAttr](#)

Subtopic of: [COLOR Set](#)

[MenuStr\(\)](#)

[MenuTextAttr](#)

Subtopic of: [COLOR Set](#)

[MenuTextLtrAttr](#)

Subtopic of: [COLOR Set](#)

[Message\(\)](#)

[Methods For Interactively Configuring the Editor](#)

[Min\(\)](#)

[Minimum closure](#)

Subtopic of: [Searching With Regular Expressions](#)

[Minimum/Maximum Closure](#)

Subtopic of: [Searching With Regular Expressions](#)

[MININT](#)

Subtopic of: [Pre-Defined Constants](#)

[Miscellaneous Questions](#)

[MkDir\(\)](#)

[mod](#)

Subtopic of: [Statements](#)

[Modifying the Key-Assignment Help Screen](#)

Subtopic of: [Customization Using Sal Compiled Macros](#)

[Mouse Equipment](#)

[Mouse Symbols and Scrollbars](#)

[MouseEnabled](#)

[MouseHoldTime](#)

[MouseHotSpot\(\)](#)

[MouseKey](#)

[MouseKeyHeld\(\)](#)

[MouseMarking\(\)](#)

[MouseRepeatDelay](#)

[MouseStatus\(\)](#)

[MouseWindowId\(\)](#)

[MouseX](#)

[MouseY](#)

[Mousing around the Editor](#)

[MoveBlock\(\)](#)

[MoveFile\(\)](#)

[Moving Around a File](#)

Subtopic of: [Working in the Editor](#)

[Moving Between Files in the Ring](#)

[Moving Through Text in a File](#)

[Moving to the Beginning or End of a Block](#)

Subtopic of: [Marking and Manipulating a Block of Text](#)

[MsgAttr](#)

Subtopic of: [COLOR Set](#)

[MsgBox\(\)](#)

[MsgBoxBuff\(\)](#)

[MsgBoxEx\(\)](#)

[MsgLevel](#)

[Multi-line delimiter tokens](#)

Subtopic of: [Syntaxhilite Mode](#)

[MultiLnDlmt1Attr](#)

Subtopic of: [COLOR Set](#)

[MultiLnDlmt2Attr](#)

Subtopic of: [COLOR Set](#)

[MultiLnDlmt3Attr](#)

Subtopic of: [COLOR Set](#)

--- N ---

[Named ClipBoards](#)

Subtopic of: [Manipulating Blocks Via Clipboards](#)

[Navigating through the HelpSystem](#)

Subtopic of: [The Helpsystem](#)

[NewFile\(\)](#)

[NextChar\(\)](#)

[NextFile\(\)](#)

[NextWindow\(\)](#)

[NoOp\(\)](#)

[not](#)

Subtopic of: [Statements](#)

[Number tokens](#)

Subtopic of: [Syntaxhilite Mode](#)

[NumberAttr](#)

Subtopic of: [COLOR Set](#)

[Numbers](#)

[Numeric Expressions](#)

Subtopic of: [Statements](#)

[NumFiles\(\)](#)

[NumFileTokens\(\)](#)

[NumHistoryItems\(\)](#)

[NumLines\(\)](#)

[NumTokens\(\)](#)

[NumWindows\(\)](#)

--- 0 ---

[Obtaining a File PickList within a Prompt Box](#)

Subtopic of: [Command Prompts](#)

[Obtaining HelpSystem Information](#)

Subtopic of: [The Helpsystem](#)

[Obtaining Useful Information Within a Macro](#)

Subtopic of: [Some Simple Macro Examples](#)

[OFF](#)

Subtopic of: [Pre-Defined Constants](#)

[Ofs\(\)](#)

[ON](#)

Subtopic of: [Pre-Defined Constants](#)

[OneWindow\(\)](#)

[Opening and Closing a Block](#)

Subtopic of: [Marking and Manipulating a Block of Text](#)

[Opening Windows](#)

[Optional Parameters](#)

Subtopic of: [Editor Commands](#)

[or](#)

Subtopic of: [Statements](#)

[Other Color Variables](#)

Subtopic of: [COLOR Set](#)

[Other Special-Purpose Search Features](#)

[Other SyntaxHilite Options](#)

Subtopic of: [Syntaxhilite Mode](#)

[OtherWinBorderAttr](#)

Subtopic of: [COLOR Set](#)

[OtherWinBorderType](#)

[otherwise](#)

Subtopic of: [Statements](#)

[Overview of the Editor](#)

[Overview of the Macro Programming Language](#)

[OverwriteCursorSize](#)

--- P ---

[PageDown\(\)](#)

[PageUp\(\)](#)

[ParaEndStyle](#)

[Passing and Retrieving Parameters by Reference](#)

Subtopic of: [Macro Procedures](#)

[Passing and Retrieving Parameters by Value](#)

Subtopic of: [Macro Procedures](#)

[Paste\(\)](#)

[PasteFromWinClip\(\)](#)

[PasteReplace\(\)](#)

[PasteReplaceFromWinClip\(\)](#)

[PasteUnDelete\(\)](#)

[PBAttribute\(\)](#)

[PBDate\(\)](#)

[PBDateStr\(\)](#)

[PBName\(\)](#)

[PBSize\(\)](#)

[PBTime\(\)](#)

[PBTimeStr\(\)](#)

[PeekByte\(\)](#)

[PeekLong\(\)](#)

[PeekWord\(\)](#)

[Performing Editing Tasks](#)

Subtopic of: [Working in the Editor](#)

[Performing Other File-Processing Operations](#)

[PersistentHistory](#)

[PersistentRecentFiles](#)

[Pick from List](#)

Subtopic of: [Command-Line Options](#)

[PickDrive\(\)](#)

[PickFile\(\)](#)

[PickFileChangesDir](#)

[PickFileFlags](#)

[PickFileSortOrder](#)

[PlaceMark\(\)](#)

[PokeByte\(\)](#)

[PokeLong\(\)](#)

[PokeWord\(\)](#)

[PopBlock\(\)](#)

[PopLocation\(\)](#)

[PopPosition\(\)](#)

[PopWinClose\(\)](#)

[PopWinCols](#)

[PopWinOpen\(\)](#)

[PopWinRows](#)

[PopWinX1](#)

[PopWinY1](#)

[Pos\(\)](#)

[PosFirstNonWhite\(\)](#)

[Positioning the Cursor and Scrolling Text](#)

[PosLastNonWhite\(\)](#)

[Potpourri](#)

Subtopic of: [Executing Supplemental Macros from the Potpourri](#)

[Pre-Defined Constants](#)

[Precedence and Order of Evaluation in Expressions](#)

Subtopic of: [Statements](#)

[PressKey\(\)](#)

[PrevChar\(\)](#)

[PrevFile\(\)](#)

[PrevHelp\(\)](#)

[PrevPosition\(\)](#)

[PrevWindow\(\)](#)

[Print Output Options](#)

Subtopic of: [Configuration Options Available Interactively](#)

[PrintBlock\(\)](#)

[PrintBotMargin](#)

[PrintChar\(\)](#)

[PrintCopies](#)

[PrinterFontFlags](#)

[PrinterFontName](#)

[PrinterFontSize](#)

[PrintFile\(\)](#)

[PrintFirstPage](#)

[PrintFooter](#)

[PrintHeader](#)

[Printing from the Editor](#)

[PrintLastPage](#)

[PrintLeftMargin](#)

[PrintLineNumbers](#)

[PrintLineSpacing](#)

[PrintLinesPerPage](#)

[PrintPause](#)

[PrintRightMargin](#)

[PrintTopMargin](#)

[PrintUseScreenFont](#)

[Private macros](#)

Subtopic of: [External Macros Vs. the User-Interface Macro proc](#)

Subtopic of: [Macro Procedures Process\(\)](#)

[Process\(\) and Keydefs](#)

[ProcessHotSpot\(\)](#)

[Prompt Box](#)

Subtopic of: [Command Prompts](#)

[Prompting the User](#)

Subtopic of: [Some Simple Macro Examples](#)

[PromptKeys](#)

Subtopic of: [Command Prompts and Keydefs](#)

[ProtectedSaves](#)

[Ptr\(\)](#)

[Public](#)

Subtopic of: [Macro Procedures](#)

[Public macros](#)

Subtopic of: [External Macros Vs. the User-Interface Macro](#)

[Pull-Down Menu Categories](#)

[PurgeKeyMacro\(\)](#)

[PurgeMacro\(\)](#)

[PurgeSynhi\(\)](#)

[Purging Compiled Macros](#)

Subtopic of: [Introduction to External Compiled Macros](#)

[Purging Keyboard Macros from the Current Session](#)

Subtopic of: [Keyboard Macros](#)

[Purging Macros](#)

[PushBlock\(\)](#)

[PushKey\(\)](#)

[PushKeyStr\(\)](#)

[PushLocation\(\)](#)

[PushPosition\(\)](#)

[PutAttr\(\)](#)

[PutAttrXY\(\)](#)

[PutChar\(\)](#)

[PutCharH\(\)](#)

[PutCharHXY\(\)](#)

[PutCharV\(\)](#)

[PutCharXY\(\)](#)

[PutCtrStr\(\)](#)

[PutHelpLine\(\)](#)

[PutLine\(\)](#)

[PutLineXY\(\)](#)

[PutNCharAttr\(\)](#)

[PutOemStrXY\(\)](#)

[PutStr\(\)](#)

[PutStrAttr\(\)](#)
[PutStrAttrXY\(\)](#)
[PutStrEOL\(\)](#)
[PutStrEOLXY\(\)](#)
[PutStrXY\(\)](#)

--- Q ---

[Query\(\)](#)
[QueryEditState\(\)](#)
[QuickHelp\(\)](#)
[QuitFile\(\)](#)
[QuitToPrompt](#)
[Quote delimiter tokens](#)
 Subtopic of: [Syntaxhilite Mode](#)
[Quote1Attr](#)
 Subtopic of: [COLOR Set](#)
[Quote2Attr](#)
 Subtopic of: [COLOR Set](#)
[Quote3Attr](#)
 Subtopic of: [COLOR Set](#)
[QuotePath\(\)](#)

--- R ---

[Re-Playing Keyboard Macros](#)
 Subtopic of: [Keyboard Macros](#)
[Read\(\)](#)
[Reading from the Screen](#)
 Subtopic of: [Video Output Commands](#)
[ReadNumeric\(\)](#)
[Recording Keyboard Macros](#)
 Subtopic of: [Keyboard Macros](#)
[RecordKeyMacro\(\)](#)
[Redo\(\)](#)
[RedoCount\(\)](#)
[Referencing and Switching to Buffers](#)
 Subtopic of: [Buffers](#)
[Referencing Global Integers](#)
 Subtopic of: [Variables](#)
[Referencing Global Strings](#)
 Subtopic of: [Variables](#)
[Referencing Local Integers](#)
 Subtopic of: [Variables](#)
[Referencing Local Strings](#)
 Subtopic of: [Variables](#)
[Regular Expression Operators](#)

Subtopic of: [Searching With Regular Expressions](#)
[Relational Operators for Numeric Expressions](#)

Subtopic of: [Statements](#)
[Relational Operators for String Expressions](#)

Subtopic of: [Statements](#)

[RemoveProfileItem\(\)](#)
[RemoveProfileSection\(\)](#)
[RemoveTrailingSlash\(\)](#)
[RemoveTrailingWhite](#)
[RemoveUnloadedFiles\(\)](#)
[RenameAndSaveFile\(\)](#)
[RenameDiskFile\(\)](#)
[repeat](#)

Subtopic of: [Statements](#)
[RepeatCmd\(\)](#)
[RepeatFind\(\)](#)
[Repeating a Character, Command, Or Macro](#)
[Replace\(\)](#)
[ReplaceFile\(\)](#)
[ReplaceOptions](#)
[ReplaceSynFile\(\)](#)
[Reserved Words](#)
[ResizeFont\(\)](#)
[ResizeWindow\(\)](#)
[Restore State](#)

Subtopic of: [Command-Line Options](#)
[RestoreCursorLine\(\)](#)
[Restoring the Current Line](#)

Subtopic of: [Commands and Features That Allow Recovery of Text](#)
[Retrieving Text from Buffers](#)

Subtopic of: [Buffers](#)
[return](#)

Subtopic of: [Statements](#)
[Return Values](#)

Subtopic of: [Editor Commands](#)
[ReturnEqNextLine](#)
[Returning Values](#)

Subtopic of: [Macro Procedures](#)
[Right\(\)](#)
[RightMargin](#)
[RightStr\(\)](#)
[Rmdir\(\)](#)
[RollDown\(\)](#)
[RollLeft\(\)](#)
[RollRight\(\)](#)
[RollUp\(\)](#)

[RTrim\(\)](#)
[Running Spellcheck](#)

--- S ---

[SaveAllAndExit\(\)](#)
[SaveAllFiles\(\)](#)
[SaveAndQuitFile\(\)](#)
[SaveAs\(\)](#)
[SaveBlock\(\)](#)
[SaveFile\(\)](#)
[SaveHistory\(\)](#)
[SaveKeyMacro\(\)](#)
[SaveSettings\(\)](#)
[SaveState](#)
[Saving and Closing Files, and Exiting from the Editor](#)
[Saving and Restoring State Information](#)
[Saving Configuration Settings](#)
[Saving Files and Exiting from the Editor](#)
[Saving Keyboard Macros for Use in a Later Session](#)

Subtopic of: [Keyboard Macros](#)

[Saving Print Options For Future Sessions](#)
[Saving/Restoring Entries in the History Buffer](#)

Subtopic of: [Saving/Restoring the Current Editing Session](#)

[Saving/Restoring Entries in the Recent Files List](#)

Subtopic of: [Saving/Restoring the Current Editing Session](#)

[Saving/Restoring the Current Editing Session](#)

[Saving/Restoring the Editing Environment](#)

Subtopic of: [Saving/Restoring the Current Editing Session](#)

[Scope](#)

Subtopic of: [Macro Procedures](#)

[ScreenCols](#)

[ScreenRows](#)

[ScrollDown\(\)](#)

[Scrolling Forward or Backward in a File](#)

Subtopic of: [Scrolling Text](#)

[Scrolling Text](#)

[Scrolling the Current Line](#)

Subtopic of: [Scrolling Text](#)

[ScrollLeft\(\)](#)

[ScrollRight\(\)](#)

[ScrollToBottom\(\)](#)

[ScrollToCenter\(\)](#)

[ScrollToRow\(\)](#)

[ScrollToTop\(\)](#)

[ScrollUp\(\)](#)

[Search Features: Finding and Replacing Text](#)

[Search SubDirs](#)

Subtopic of: [Command-Line Options](#)

[SearchHelp\(\)](#)

[Searching](#)

[Searching With Regular Expressions](#)

[SearchPath\(\)](#)

[Seg\(\)](#)

[Selecting Colors for SyntaxHilite Classes](#)

Subtopic of: [Syntaxhilite Mode](#)

[Selecting Files to Edit](#)

[Selecting the Tab Type](#)

[Selecting the Video Mode](#)

[Selection Menu](#)

Subtopic of: [Command Prompts](#)

[SelfInsert\(\)](#)

[Sending Output and Commands to the Printer](#)

[Session-Global Variables](#)

[Set\(\)](#)

[Set Colors](#)

Subtopic of: [Configuration Options Available Interactively](#)

[SetBit\(\)](#)

[SetBufferInt\(\)](#)

[SetBufferStr\(\)](#)

[SetColorTableValue\(\)](#)

[SetCursorOff\(\)](#)

[SetCursorOn\(\)](#)

[SetFileAttr\(\)](#)

[SetFont\(\)](#)

[SetGlobalInt\(\)](#)

[SetGlobalStr\(\)](#)

[SetHookState\(\)](#)

[SetMenuBar\(\)](#)

[SetPrinterFont\(\)](#)

[Setting Print Options](#)

[Setting Tab Stops](#)

[Setting the Option to Display SyntaxHilite Colors](#)

Subtopic of: [Syntaxhilite Mode](#)

[SetUndoOff\(\)](#)

[SetUndoOn\(\)](#)

[SetVideoRowsCols\(\)](#)

[SetWindowHeight\(\)](#)

[SetWindowTitle\(\)](#)

[SetWindowWidth\(\)](#)

[Shell\(\)](#)

[ShiftText\(\)](#)

[shl](#)

Subtopic of: [Statements](#)

[ShowEntryScreen\(\)](#)

[ShowEOFMarker](#)

[ShowHelpLine](#)

[ShowLineNumbers](#)

[ShowMainMenu](#)

[ShowMouse\(\)](#)

[ShowStatusLine](#)

[ShowSyntaxHilite](#)

[shr](#)

Subtopic of: [Statements](#)

[SignOn\(\)](#)

[Simple Customization from Within the Editor](#)

[Single Instance of the Editor Off](#)

Subtopic of: [Command-Line Options](#)

[Single Instance of the Editor On](#)

Subtopic of: [Command-Line Options](#)

[Single-line delimiter tokens](#)

Subtopic of: [Syntaxhilite Mode](#)

[SingleInstance](#)

[SingleLnDlmt1Attr](#)

Subtopic of: [COLOR Set](#)

[SingleLnDlmt2Attr](#)

Subtopic of: [COLOR Set](#)

[SingleLnDlmt3Attr](#)

Subtopic of: [COLOR Set](#)

[SizeOf\(\)](#)

[Smart Tabs](#)

Subtopic of: [Setting Tab Stops](#)

[SmartIndent Mode](#)

Subtopic of: [Special Programming-Language Modes](#)

[Soft and Hard Tabs](#)

Subtopic of: [Setting Tab Stops](#)

[Some Simple Macro Examples](#)

[Sort\(\)](#)

[Sorting Lines of Text](#)

[Sound\(\)](#)

[Special Commands and Features](#)

[Special Handling For Certain Types of Text](#)

[Special Programming-Language Modes](#)

[SpecialEffects](#)

[Speedsearch Feature](#)

[SplitLine\(\)](#)

[SplitPath\(\)](#)

[Splitting the Screen Into Windows](#)

[SqueezePath\(\)](#)

[StartedFromDosPrompt\(\)](#)

[Starting an Editing Session](#)

[Starting Simple](#)

Subtopic of: [Customization Using Sal Compiled Macros](#)

[Starting the Editor](#)

[Starting the Editor from the Command Line](#)

[Starting the Editor from the Desktop](#)

[Starting the Editor With No Command-Line Parameters](#)

[StartPgm\(\)](#)

[Startup Processing](#)

[StartupFlags](#)

[StartupVideoMode](#)

[Statements](#)

[StatusLine Prompt](#)

Subtopic of: [Command Prompts](#)

[StatusLineAtTop](#)

[StatusLineAttr](#)

Subtopic of: [COLOR Set](#)

[StatusLineFillChar](#)

[StatusLineRow](#)

[StatusLineUpdating](#)

[Storing Text in Buffers](#)

Subtopic of: [Buffers](#)

[Str\(\)](#)

[StrCount\(\)](#)

[StrFind\(\)](#)

[String](#)

Subtopic of: [Variables](#)

[String Expressions](#)

Subtopic of: [Statements](#)

[String Length](#)

Subtopic of: [Editor Commands](#)

[String Slices](#)

Subtopic of: [Statements](#)

[String Variables](#)

Subtopic of: [Variables](#)

[Strings](#)

[StrReplace\(\)](#)

[Sub-Menu Items that Change an Editing Option](#)

Subtopic of: [Accessing Sub-Menu Commands and Options](#)

[Sub-Menu Items that Execute a Command](#)

Subtopic of: [Accessing Sub-Menu Commands and Options](#)

[SubStr\(\)](#)

[Summary List of Regular Expression Operators](#)

Subtopic of: [Searching With Regular Expressions](#)

[Summary of Mouse Operations](#)

[Summary of New/Enhanced Features in the Semware Editor Professional V4.0](#)
[Summary of New/Enhanced Features in the Semware Editor Professional V4.2](#)
[Summary of New/Enhanced Features in the Semware Editor Professional V4.4](#)
[Summary of New/Enhanced Features in the Semware Editor Professional/32 V2.6](#)
[Summary of New/Enhanced Features in the Semware Editor Professional/32 V2.8](#)
[Summary of New/Enhanced Features in the Semware Editor Professional/32 V3.0](#)

[SwapPosition\(\)](#)

[Switching Between Windows](#)

[Synchronized Scrolling in Windows](#)

[Syntax](#)

Subtopic of: [Editor Commands](#)

[SyntaxHilite Color Variables](#)

Subtopic of: [COLOR Set](#)

[Syntaxhilite Mode](#)

[System ClipBoard](#)

Subtopic of: [Manipulating Blocks Via Clipboards](#)

[System/File Options](#)

Subtopic of: [Configuration Options Available Interactively](#)

--- T ---

[Tab/Margin Options](#)

Subtopic of: [Configuration Options Available Interactively](#)

[Table Of Contents](#)

[TabLeft\(\)](#)

[TabRight\(\)](#)

[TabSet\(\)](#)

[TabShiftsBlock](#)

[TabType](#)

[TabWidth](#)

[Template Mode](#)

[TemplateExpansion](#)

[TerminateEvent\(\)](#)

[Terminating the Editor](#)

[Termination Statements ... break, return, and halt](#)

Subtopic of: [Statements](#)

[Text Color Variables](#)

Subtopic of: [COLOR Set](#)

[Text Entry/Block Options](#)

Subtopic of: [Configuration Options Available Interactively](#)

[TextAttr](#)

Subtopic of: [COLOR Set](#)

[The AutoLoadList Command](#)

Subtopic of: [Introduction to External Compiled Macros](#)

[The ClipBoard Copy Commands](#)

Subtopic of: [Manipulating Blocks Via Clipboards](#)

[The ClipBoard CopyAppend Commands](#)

Subtopic of: [Manipulating Blocks Via Clipboards](#)
[The ClipBoard Cut Commands](#)

Subtopic of: [Manipulating Blocks Via Clipboards](#)
[The ClipBoard CutAppend Commands](#)

Subtopic of: [Manipulating Blocks Via Clipboards](#)
[The ClipBoard Empty Command](#)

Subtopic of: [Manipulating Blocks Via Clipboards](#)
[The ClipBoard Paste Commands](#)

Subtopic of: [Manipulating Blocks Via Clipboards](#)
[The ClipBoard PasteOver Commands](#)

Subtopic of: [Manipulating Blocks Via Clipboards](#)
[The Customizable Helpline](#)
[The Deletion Buffer: A Safety Net](#)

Subtopic of: [Commands and Features That Allow Recovery of Text](#)
[The Editor Design](#)
[The Editor Features](#)
[The Editor Manuals](#)
[The Effective Left Margin](#)

Subtopic of: [Autoindent Mode](#)
[The End-of-File Marker](#)
[The ExecMacro Command](#)

Subtopic of: [Introduction to External Compiled Macros](#)
[The Helpline](#)
[The Helpsystem](#)
[The LoadMacro Command](#)

Subtopic of: [Introduction to External Compiled Macros](#)
[The Macro Reference Guide](#)

Subtopic of: [The Editor Manuals](#)
[The Microsoft Windows Clipboard Commands](#)

Subtopic of: [Manipulating Blocks Via Clipboards](#)
[The Pull-Down Menu System](#)
[The Screen Layout](#)
[The Screen Layout With Multiple Windows](#)
[The Statusline](#)
[The TSELOAD.DAT File](#)

Subtopic of: [Startup Processing](#)
[The TSEStart Macro](#)

Subtopic of: [Startup Processing](#)
[The User's Guide](#)

Subtopic of: [The Editor Manuals](#)
[The User-Defined Word List](#)
[The User-Interface Macro File \(TSE.UI\)](#)

Subtopic of: [External Macros Vs. the User-Interface Macro](#)
[TimeFormat](#)
[times](#)

Subtopic of: [Statements](#)

[TimeSeparator](#)

[to](#)

Subtopic of: [Statements](#)

[To-EOL delimiter tokens](#)

Subtopic of: [Syntaxhilite Mode](#)

[ToEOL1Attr](#)

Subtopic of: [COLOR Set](#)

[ToEOL2Attr](#)

Subtopic of: [COLOR Set](#)

[ToEOL3Attr](#)

Subtopic of: [COLOR Set](#)

[Toggle\(\)](#)

[ToggleInsert\(\)](#)

[TrackMouseCursor\(\)](#)

[Transparency](#)

[Trim\(\)](#)

[TRUE](#)

Subtopic of: [Pre-Defined Constants](#)

[TSELOAD.DAT](#)

Subtopic of: [Startup Processing](#)

[TSELOADDIR](#)

Subtopic of: [Augmenting the Editor Load Directory](#)

[TSEPath](#)

[TSEStart Macro](#)

Subtopic of: [Startup Processing](#)

[TSESTART.MAC](#)

Subtopic of: [Startup Processing](#)

[Type Conversions within Expressions](#)

Subtopic of: [Statements](#)

--- U ---

[UnBufferVideo\(\)](#)

[UnDelete\(\)](#)

[Understanding User-Interface \(UI\) Macro Files](#)

Subtopic of: [Customization Using Sal Compiled Macros](#)

[Undo\(\)](#)

[UndoCount\(\)](#)

[UndoMode\(\)](#)

[UnHook\(\)](#)

[UnLockCurrentFile\(\)](#)

[UnMarkAfterPaste](#)

[UnMarkBlock\(\)](#)

[Unmarking a Block](#)

Subtopic of: [Marking and Manipulating a Block of Text](#)

[until](#)

Subtopic of: [Statements](#)

[Up\(\)](#)

[UpdateBufferDaTmAttr\(\)](#)

[UpdateDisplay\(\)](#)

[UpdateDisplayFlags\(\)](#)

[Upper\(\)](#)

[UseCommonDialogs](#)

[UseCurrLineIfNoBlock](#)

[Using a Mouse](#)

Subtopic of: [Working in the Editor](#)

[Using Bookmarks](#)

[Using Command Lines in Macros](#)

[Using DLLs](#)

Subtopic of: [Macro Procedures](#)

[Using Keyboard Macros](#)

Subtopic of: [Working in the Editor](#)

[Using Macro Language Features](#)

[Using Macros](#)

[Using Physical Tabs: Hard](#)

[Using Picklists](#)

[Using Positional Tabs: Soft, Variable, and Smart](#)

[Using Tabs](#)

[Using the Pop-Up Mouse Menu](#)

[Using the Pull-Down Menu System](#)

[Using Undo/Redo](#)

Subtopic of: [Commands and Features That Allow Recovery of Text](#)

--- V ---

[Val\(\)](#)

[Values](#)

Subtopic of: [Editor Variables](#)

[var](#)

Subtopic of: [Macro Procedures](#)

[Variable Assignment Statements](#)

Subtopic of: [Statements](#)

[Variable Tabs](#)

Subtopic of: [Setting Tab Stops](#)

[Variables](#)

[VarTabs](#)

[Version\(\)](#)

[VersionStr\(\)](#)

[VGotoXY\(\)](#)

[VGotoXYAbs\(\)](#)

[VHomeCursor\(\)](#)

[Video Output Commands](#)

[ViewFinds\(\)](#)

[ViewFindsId](#)

[VWhereX\(\)](#)

[VWhereY\(\)](#)

[VWindow\(\)](#)

--- W ---

[WaitForKeyPressed\(\)](#)

[WaitForMouseEvent\(\)](#)

[Warn\(\)](#)

[What Happens when a File Is Saved](#)

Subtopic of: [Saving and Closing Files, and Exiting from the Editor when](#)

Subtopic of: [Statements](#)

[WhenLoaded macro](#)

Subtopic of: [Event Macros](#)

[WhenPurged macro](#)

Subtopic of: [Event Macros](#)

[WhereX\(\)](#)

[WhereXAbs\(\)](#)

[WhereY\(\)](#)

[WhereYAbs\(\)](#)

[WhichOS\(\)](#)

[while](#)

Subtopic of: [Statements](#)

[White Characters](#)

[WIN32](#)

Subtopic of: [Pre-Defined Constants](#)

[Window\(\)](#)

[Window Border Variables](#)

Subtopic of: [COLOR Set](#)

[WindowCols](#)

[WindowFooter\(\)](#)

[WindowId\(\)](#)

[WindowRows](#)

[Windows](#)

[WindowX1](#)

[WindowY1](#)

[WordLeft\(\)](#)

[WordRight\(\)](#)

[WordSet](#)

[WordWrap](#)

[Wordwrap Mode](#)

[Working in the Editor](#)

[Working With Windows](#)

[WrapLine\(\)](#)

[WrapPara\(\)](#)

[Write\(\)](#)

[WriteLine\(\)](#)[WriteProfileInt\(\)](#)[WriteProfileStr\(\)](#)[Writing Configuration Settings to a Settings File](#)[Writing to the Screen](#)Subtopic of: [Video Output Commands](#)

--- X ---

[X1](#)

--- Y ---

[Y1](#)[YesNo\(\)](#)

--- Z ---

[ZeroBasedTab](#)[ZoomWindow\(\)](#)

A Walk-Through of Window Commands

This section walks you through a simple editing session, using window commands described above. This guided tour will familiarize you with some of the commands available for creating and working within

[windows](#). To begin, load a file into the editor. Now open a horizontal window by pressing <Ctrl O H> (or select "Horizontal" under the Window Menu). You now have two horizontal windows on the screen. The bottom window is the current window and contains the cursor. This is window 2. Notice the color and style of the current window divider line. Open a vertical window in one of the horizontal windows by pressing <Ctrl O V> (or select "Vertical" under the Window Menu). Window 2 splits vertically into two windows, creating a third window. Window 3 becomes the current window. Move between windows by pressing <Alt> and the number key that corresponds to the window desired. For example, press <Alt 1> to move to window 1. Be sure to use the number keys located above the alphabetic keys. Next, close a window by using the CloseWindow command. For example, press <Alt 3> to go to window 3, and then press <Ctrl O C> (or select "Close" under the Window Menu). This closes window 3 and makes window 2 the current window. Re-open window 3 by pressing <Ctrl O V> (or by selecting "Vertical" under the Window [Menu](#)). Resize window 3 using the ResizeWindow command. To do this, press <Ctrl O R> (or select "Resize" under the Window Menu) while in window 3. The editor prompts for the edge to be resized. Press <CursorLeft> to select the left window edge. A resizing bar appears. Press <CursorRight> and <CursorLeft> to move the resizing bar left and right to adjust the left edge of the window. Press <Enter> when complete, to position the left edge of the window according to the final placement of the resizing bar. Close a window using the DelWindow command. For example, in window 3, press <Ctrl O D>, then press <CursorLeft> to delete the left window. This closes window 2, and window 3 expands to fill the area vacated by window 2. Windows 1 and 3 remain open on the screen. Zoom window 1 to temporarily make that window fill the screen. To do this, press <Alt 1> to go to window 1, and then press <Ctrl O Z> (or select "Zoom" under the Window Menu). Window 1 expands to fill the screen. Unless you are [using a mouse](#), the window identifier changes from 1 to Z. To restore windows 1 and 3 to the screen, press <Ctrl O Z> again (or select "Zoom" under the Window Menu). Close all [windows](#) except window 1. You can do this from window 1 by pressing <Ctrl O O> (or select "One" under the Window Menu). Window 3 is removed, and window 1 expands to fill the screen.

AbandonEditor

Terminates the editor without any prompt, discarding all unsaved changes to files.

Syntax: [AbandonEditor](#)([[INTEGER](#) error_level]) - error_level is the return code passed to the calling program (usually the operating system). Returns: Nothing. Notes: This command exits the editor immediately, WITH NO WARNING AND NO REGARD FOR UNSAVED [BUFFERS](#). For this reason, this command is intended for use only in macros (as opposed to a stand-alone command) where this type of action might be necessary. To save all files before exiting, use one of the following commands: [SaveAllFiles](#)(), [SaveAllAndExit](#)(). The [QuitToPrompt](#) variable has no effect on this command. **Examples:** The following macro is an example of an alternative to the editor's [Exit](#)() command. If [buffers](#) have been changed, instead of prompting on each changed buffer, it gives the user the option of abandoning all [buffers](#) (by selecting "y"), writing all [buffers](#) (by selecting "w"), or by returning to the editor (by selecting "n" or <escape>). `proc mEmacsExit() integer start_file = GetBufferId(), n = NumFiles() + (BufferType() <> _NORMAL_), changed = 0 // traverse the ring of files, to see if one is modified while n if FileChanged() changed = changed + 1 endif n = n - 1 NextFile(_DONT_LOAD_) endwhile // return to where we were GotoBufferId(start_file) if changed == 0 AbandonEditor() else Message(changed, " buffer(s) have not been saved. " + "Exit [ynw]?") loop case GetKey() when <escape>, <n>, <shift n> UpdateDisplay() break // break out of loop when <y>, <shift y> AbandonEditor() // never returns break when <w>, <shift w> SaveAllAndExit() // never returns break endcase endloop endif end See Also: SaveAllFiles(), SaveAllAndExit(), Exit()`

AbandonFile

Removes the current or specified buffer from the editor, without any prompt, discarding all unsaved changes.

Syntax: [INTEGER](#) [AbandonFile](#)([[INTEGER](#) buffer_id]) - buffer_id is the id number of the buffer to be abandoned. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command discards the current or optionally-specified buffer immediately, with no warning and regardless of changes made to the buffer. For this reason, this command is intended for use only in macros (as opposed to a stand-alone command) where this type of action might be necessary. If a buffer_id is specified and the buffer does not exist in the editor's Ring of files, zero will be returned; otherwise, non-zero will be returned. When the last edited file is abandoned from a macro, control is not returned to the editor until the macro finishes operation. If control is returned to the editor, and the total number of files loaded is 0 (for example, [NumFiles](#)() == 0), the editor will terminate. The [QuitToPrompt](#) variable has no effect on this command. **Examples:** See the [YesNo](#)() command for an example. See Also: [FileChanged](#)(), [QuitFile](#)(), [SaveAndQuitFile](#)(), [EmptyBuffer](#)()

About

Displays information about the current version of the editor.

Abs

Returns the unsigned value of an integer expression.

Syntax: [INTEGER Abs](#)([INTEGER](#) i) - i is the integer expression whose absolute value is to be returned. Returns: The absolute value of the passed integer expression. **Examples:** i = [Abs](#)(-3) // i is 3 i = [Abs](#)(3) // i is 3 i = [Abs](#)(0) // i is 0 j = 5 i = [Abs](#)(j - 15) // i is 10 See Also: [Min](#)(), [Max](#)()

Accessing Sub-Menu Commands and Options

To display a sub-menu of commands or options for a particular main Menu category, select the desired category. Several methods are available to accomplish this:

- From the main Menu, press <CursorRight> or <CursorLeft> to move the cursor bar directly over the desired main Menu category. Then press either <Enter> or <CursorDown> to display the sub-menu for that category.
- Or, from the main Menu, press the highlighted character, or "quick-key," of the desired Menu category to automatically display the sub-menu for that category.
- From within a sub-menu, you can move to another sub-menu without going back to the main Menu. Simply press <CursorRight> or <CursorLeft> to move across the sub-menus until you are in the sub-menu of the desired category. Or, just press <Alt> plus the quick-key of the desired main

[Menu](#) category. - For the File and Text Menu categories, Menu hotkeys are available to directly access the sub-menus while editing. Press <Alt F> or <Alt T>, respectively, to activate the pull-down Menu and display the associated sub-menu. Within a sub-menu, you can select an item to execute a command or set an option. Just press <CursorDown> or <CursorUp> to move the cursor bar over the desired item. Then press <Enter> to execute or process that item.

Alternatively, you can press the quick-key (the highlighted character) for an item to select that item and immediately execute or process it. Certain sub-menu items may be disabled at various times. For example, if no block is currently marked in any file, then "Copy" under the Block menu will not have any effect; that is, it is temporarily disabled while no block is marked. Such disabled items are "grayed" in the sub-menu list to indicate that they currently have no effect. No action will occur if a "grayed" item is selected. Here is an example of the Menu showing the File sub-menu.

```
+-----+ |*File* Block Text Search Window Clip Macro Print Util
Options Help | ++-----+-----+ |*New*****| | Open...
<Alt E> | | Insert... <Alt R> | | File Manager... | | Change Dir... | +-----+ | Next <Alt N> | | Previous
<Alt P> | | List Open... <Alt O> | | List Recent... <Ctrl O> | +----- Current File -----+ | Save <Ctrl K S> | | Save As... | |
Save & Close <Ctrl K X> | | Close <Ctrl K Q> | | Change Name... <Alt O> | | Show File Info | +----- All Files -----+
| Save All | | Save All & Exit | | Exit <Alt X> | +-----+ Here is an example of the Menu showing the
Options sub-menu. +-----+ | File Block Text Search Window Clip
Macro Print Util *Options* Help | +-----+-----+ | AutoIndent*****
[***On*] | | WordWrap [ Off] | | Right Margin [ 72] | | Left Margin [ 0] | +-----+ | Tab Type [ Soft] |
| Tab Width [ 8] | +-----+ | Read Only [Off] | | Backups [ On] | | Save State on Exit [ Off] |
+-----+ | Full Configuration > | | Save Current Settings > | +-----+ The sub-menus
```

display items that either execute a command (such as "Open" under the File Menu), or that allow you to specify or toggle an editing option (such as "[AutoIndent](#)" under the Options Menu). Certain sub-menu commands are followed by ">" to indicate that the command displays a selection menu of choices (such as "[Full Configuration](#)" under the Options Menu). Other commands are followed by "..." to indicate that execution of the command either displays a prompt box

or StatusLine prompt requesting additional information needed to complete the command (such as "Open" under the File Menu); or displays a PickList of entries that you can view and select from (such as "List Open" under the File Menu). (For more information on prompts, selection menus, and PickLists, see the chapter on "[Command Prompts and PickLists](#).")

[Sub-Menu Items that Execute a Command](#) ----- If the item executes a command, execution begins once the item is selected. If the editor needs additional input to continue processing a command, it displays a prompt box, selection menu, or PickList. Execution of the command continues when you provide the necessary information or make the desired selection. You can press <Escape> within a prompt, selection menu, or PickList to cancel the operation. Upon completion or cancellation of the command, the editing session resumes and [the pull-down Menu system](#) is no longer active. For example: - Selecting "Save All" under the File Menu causes the editor to immediately save all files in memory. The editor then deactivates the pull-down Menu and resumes the editing session. - Selecting "Open" under the File Menu causes the EditFile command to be processed. The editor first displays the following prompt box to request the filename(s) to be opened:

```
+-----+ |File(s) to edit: [current path] | | |
```

+-----+ Once you type the desired filename(s) and press <Enter>, the editor processes the indicated file(s). The pull-down Menu is then deactivated, and the cursor is placed in the first newly-opened file, ready for editing [to](#) resume. - Selecting "List Open" under the File Menu causes a PickList to be displayed that provides a list of all files that are open in the current session. To select a file to edit, move the cursor bar over the desired file and press <Enter>. The command resumes execution based on the selection made. The editor then deactivates the pull-down Menu and resumes the editing session.

[Sub-Menu Items that Change an Editing Option](#) ----- If the sub-menu item toggles or changes an editing option, that option is followed by an indication of its current setting enclosed within []. If the option has only two states, such as ON and OFF, the state is immediately toggled when you select that option. If the option has more than two possible settings, or allows a user-defined value to be specified, then when you select that option, it activates the option and allows you to specify the appropriate setting or value. After you select an editing option, the sub-menu remains on the screen to allow you to select other options or commands, or to move to another [Menu](#) category. To deactivate [the pull-down Menu system](#) and resume editing, press <Escape> twice. For example: - Selecting "Backups" under the Options Menu causes the editor to immediately toggle Backups [ON](#) (if initially set [OFF](#)). The option setting displayed on the sub-menu changes to [On]. If you select it again, it toggles back OFF, and [Off] displays on the sub-menu. To deactivate the pull-down Menu and resume editing, press <Escape> twice. - Selecting "[AutoIndent](#)" under the Options Menu causes the editor to display the following selection menu of available AutoIndent modes: +-----+ | Off | |*On*****| | Sticky | +-----+

To choose a setting, move the cursor bar over the desired selection and press <Enter>, or simply press the highlighted quick-key for the desired selection. The editor then removes the selection menu from the screen and changes to the indicated mode. The newly-selected setting is displayed within [] on the sub-menu. To deactivate the pull-down Menu and resume editing, press <Escape> twice. - Selecting "Right Margin" under the Options Menu causes the editor to activate the area within []. To specify a value for RightMargin, type the desired number. You can use editing commands such as Left, Right, BackSpace, DelChar, DelLine, DelToEol, CopyBlock, and Paste, within [] to input a value. Press <Enter> to have the editor accept your new setting, or press <Escape> to cancel any changes made to the option. Then press <Escape> twice to deactivate the pull-down Menu and resume editing.

Accessing the Pop-Up Ascii Chart

* AsciiChart (<Ctrl A>, or "ASCII Chart" under Util

[Menu](#)) This command pops up an ASCII chart PickList on the screen. The following information is included for each character in the ASCII chart: - the decimal representation of the character - the hexadecimal representation of the character - the display representation of the character When the ASCII chart is displayed, the cursor bar is located on the entry that corresponds to the character in the text at the cursor position. If the cursor is positioned beyond the end of the line, the cursor bar is located on the first item in the ASCII chart (the null character). You can scroll through the entries in the ASCII chart the same as you would for any PickList. (For more information on scrolling within a PickList, see "[Using PickLists](#)" in the chapter on "[Command Prompts and PickLists](#).")

To place a display character

from the ASCII chart into the text, scroll the cursor bar to the desired entry and press <Enter>. The ASCII chart is then removed from the screen, and the selected character is placed in the text at the cursor position. To remove the ASCII chart from the screen without placing a character into the text, press <Escape>.

Activating the Pull-Down Menu

While the pull-down Menu is displayed, basic cursor movements and the <Enter>

[key](#) can be used to make selections, or "quick-keys" can be pressed to provide access to [Menu](#) options. The [main Menubar](#) lists categories of commands and options, as illustrated below:

```
+-----+ |*File* Block Text Search Window Clip Macro Print Util
Options Help | +-----+ In the standard configuration of the editor,
the main Menu continually displays at the top of the screen. To activate the Menu, press <F10> or <Escape>. If
desired, you can configure the editor so the main Menu will be displayed only when you activate the Menu by pressing
<F10> or <Escape>. If the StatusLine or HelpLine is currently displayed at the top of the screen, the main Menu will
overlay it. (For more information on configuring the editor to display the main Menu only when activated, see
"Display Main Menu" under "Display/Color Options" in the chapter on "Configuring the Editor Interactively.")
```

AddAutoLoadMacro

Adds a Compiled Macro file to the list of macro files to be automatically loaded on editor startup.

Syntax: [INTEGER AddAutoLoadMacro](#)([STRING](#) macro_name) - macro_name is the name of the macro file to load. The name can either be a simple name ("mymacro") or may include the complete path ("c:\[macros](#)\mymacro.mac"). Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: The list of macro files to be loaded is maintained in the autoload file TSELOAD.DAT. This file provides a simple, optional mechanism for automatically loading macro files at the beginning of each editing session. If TSELOAD.DAT is used, it is maintained in the editor load directory. On startup, the editor loads any macros found in this autoload file, using the [LoadMacro](#)() command. See Also: [DelAutoLoadMacro](#)(), [EditAutoLoadList](#)(), [LoadMacro](#)(), [isAutoLoaded](#)() Variables: [TSEPath](#)

AddFFInfoToBuffer

Adds the file information gathered from the last

[FindFirstFile](#)() or [FindThisFile](#)() command to the indicated buffer. **Syntax:** [INTEGER AddFFInfoToBuffer](#) ([INTEGER](#) buffer_id [, [STRING](#) path]) - buffer_id is the optional buffer to which the file information will be added. If not passed, the current buffer is assumed. - path is an optional path that will be inserted before the filename. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: The file information is added after the current line in buffer_id. The added line becomes the new current line. The PB-type commands, such as [PBName](#)() and [PBDate](#)(), can be used to extract the file information added to the buffer. The editor uses this command internally for [BuildPickBufferEx](#)() processing. **Examples:** integer h, cur_id, pick_buf cur_id = [GetBufferId](#)() pick_buf = [CreateTempBuffer](#)() [GotoBufferId](#)(cur_id) h = [FindFirstFile](#)("*.*", _NORMAL_) if h <> - 1 repeat [AddFFInfoToBuffer](#)(pick_buf) until not [FindNextFile](#)(h, _NORMAL_) [FindFileClose](#)(h) endif See Also: [FindFirstFile](#) (), [FindThisFile](#) (), [FFAttribute](#) (), [FFDate](#) (), [FFName](#) (), [FFSize](#) (), [FFTime](#) (), [PBAttribute](#) (), [PBDate](#) (), [PBName](#) (), [PBSize](#) (), [PBTime](#) ()

AddFileToRing

Adds the specified file to the editor's Ring of files, without actually loading the file.

Syntax: [INTEGER](#) [AddFileToRing](#)([STRING](#) fn [, [INTEGER](#) reysize [, [INTEGER](#) display_mode]]) - fn is the name of the file to add to the Ring. fn can contain wildcards, in which case all matching files are added to the Ring. - reysize is the optional binary line length to be used for each file added to the Ring. To use normal (non-binary mode), specify a reysize of 0, which is the default if reysize is [not](#) specified. - display_mode is the optional display mode to set. If not specified, [_DISPLAY_TEXT_](#) is used. Supported [values](#) for display_mode are: - [_DISPLAY_TEXT_](#) places the screen display in standard display mode. - [_DISPLAY_HEX_](#) places the screen display in Hex display mode. - [_DISPLAY_PICKFILE_](#) places the screen display in file PickList mode. Returns: The non-zero id number of the first buffer created if successful; [otherwise](#), zero ([FALSE](#)). Notes: [AddFileToRing](#)() is used by the [EditFile](#)() and [EditThisFile](#)() command to add files to the editor's Ring of files. The last file added by this command becomes the current buffer. A file added by [AddFileToRing](#)() is not loaded into the editor until that file is switched to via [NextFile](#)() , [PrevFile](#)() , [EditFile](#)() or [EditThisFile](#)() . **Examples:** // Add all "*.c" files to the ring, using non-binary mode // The files are [not](#) loaded until actually switched to [PushPosition](#)() [AddFileToRing](#)("*.c", 0) [PopPosition](#)() See Also: [EditFile](#)() , [EditThisFile](#)() , [CreateBuffer](#)() , [BinaryMode](#)() , [DisplayMode](#)()

AddHistoryStr

Adds a string to the specified history list.

Syntax: [AddHistoryStr](#)([STRING](#) s, [INTEGER](#) history_num) - s is the string to be added to the specified history list. - history_num is an integer which identifies a specific history list. Returns: Nothing. Notes: The editor maintains a history for all built-in commands that prompt the user for a string. However, when commands are used in a non-interactive way (for instance, [EditFile](#)("some.file"), which will not prompt the user, but will immediately edit "some.file"), the history is not changed or added to. This command allows you to add to that history or to create your own. Supported [values for](#) history_num are: - A pre-defined constant value representing one of the built-in histories: History Name Associated Commands _____ [_EDIT_HISTORY_](#) [EditFile](#)() , [EditThisFile](#)() [InsertFile](#)() , [SaveBlock](#)() , [SaveAs](#)() [_NEWNAME_HISTORY_](#) [ChangeCurrFilename](#)() [_EXECMACRO_HISTORY_](#) [ExecMacro](#)() [_LOADMACRO_HISTORY_](#) [LoadMacro](#)() [_KEYMACRO_HISTORY_](#) [LoadKeyMacro](#)() , [SaveKeyMacro](#)() [_GOTOLINE_HISTORY_](#) [GotoLine](#)() [_GOTOCOLUMN_HISTORY_](#) [GotoColumn](#)() [_REPEATCMD_HISTORY_](#) [RepeatCmd](#)() [_DOS_HISTORY_](#) [Dos](#)() [_FINDOPTIONS_HISTORY_](#) [Find](#)() [_REPLACEOPTIONS_HISTORY_](#) [Replace](#)() [_FIND_HISTORY_](#) [Find](#)() , [Replace](#)() [_REPLACE_HISTORY_](#) [Replace](#)() [_FILLBLOCK_HISTORY_](#) [FillBlock](#)() - A number in the range 1..127. - A number returned by [GetFreeHistory](#)() , which will be in the range 1..127. The MaxHistorySize variable specifies the maximum number of total history items to be maintained. Note that [MaxHistorySize](#) applies to all history lists combined, [not](#) to any one single history list. If the number of history items is equal to the MaxHistorySize variable, then the oldest item will be removed to make room for the new item. The MaxHistoryPerList variable specifies the maximum number of history items to be maintained in each history list. If the number of history items in a particular history list is equal to the MaxHistoryPerList variable, then the oldest item will be removed to make room for the new item. **Examples:** /
***** Find a string, and, if successful, add the found string to the history for the find command. "s" is assumed to be a string that is declared and initialized elsewhere.
*****/ [if](#) [Find](#)(s, "") [AddHistoryStr](#)(s, [_FIND_HISTORY_](#)) [endif](#) See Also: [DelHistory](#)() , [GetFreeHistory](#)() , [DelHistoryStr](#)() , [FindHistoryStr](#)() , [GetHistoryStr](#)

(, [NumHistoryItems\(\)](#), [Ask\(\)](#), [Read\(\)](#) [Variables:](#) [MaxHistorySize](#), [MaxHistoryPerList](#), [CurrHistoryList](#)

Adding Files to the Ring: Specifying and Opening Files

Files to be placed in the Ring and opened for editing can be specified when the editor is initially started. (For more information on opening files when

[starting the editor](#), see "[Selecting Files to Edit](#)" in the chapter on "[Starting the Editor](#).") You can also open additional files while in the middle of an editing session. The EditThisFile, EditFile, and NewFile commands are available for this. * EditThisFile (<Alt E>, or "Open" under File [Menu](#)) This command prompts for a file to open by issuing the prompt, "File to edit:". (For informational purposes, the current path is displayed in the prompt.) It then attempts to locate the designated file. The file that you specify, is opened and becomes the current file. When the "File to edit:" prompt is displayed by the editor, you can enter complete filenames, including full path information, or you can input abbreviated or relative filename specifications. You can also [include command-line options](#) within this prompt. (For more information on available command-line options, see the chapter on "[Starting the Editor](#).") Also within the "File to edit:" prompt, you can include long filenames and UNC network paths; and you can specify filenames with embedded spaces. To enter multiple distinct filenames into the prompt, at least one of the filenames must be quoted, for instance: file1 file2 "file3" Will cause the loading or creation of files "file1", "file2" and "file3". However entering: file1 file2 file3 Will cause the loading or creation of the single file, "file1 file2 file3". * EditFile (<Alt E>, or "Open" under File [Menu](#)) This command prompts for one or more files to open by issuing the prompt, "File(s) to edit:". (For informational purposes, the current path is displayed in the prompt.) It then attempts to locate the designated file(s). The first (or only) file that you specify, is opened and becomes the current file. When the "File(s) to edit:" prompt is displayed by the editor, you can enter complete filenames, including full path information, or you can input abbreviated filename specifications. You can also [include](#) command-line options within this prompt. (For more information on available command-line options, see the chapter on "[Starting the Editor](#).") Also within the "File(s) to edit:" prompt, you can include long filenames and UNC network paths; and you can specify filenames with leading or embedded spaces by enclosing the filename within quotes. If you type an abbreviated entry, the editor first builds the complete path for the specification. If no drive is specified, the current drive is used. If no directory is specified, the current directory is used. The editor then determines the file(s) to select for loading, as follows: - If you enter an empty response at the "File(s) to edit:" prompt, a PickList of files in the current directory is displayed. You can then select the desired file from the list of files. A number of file-related options are available in an EditFile PickList. You can sort the entries, select an alternate drive for which to display matching files, delete listed files from disk, and select multiple files and/or directories to process. For more information, see "[Obtaining a File PickList within a Prompt Box](#)" in the chapter on "[Command Prompts and PickLists](#).") - If you enter the complete filename, that filename is used by the editor. - If you enter an ambiguous filename specification (one which includes the wildcard characters, "*" or "?"), or you include the "[Search SubDirs](#)" (-s) command-line option, the editor either presents a PickList of all matching files from which you can select the desired file, or the editor selects all matching files. (For more information on the "Search SubDirs" option, see "[Command-Line Options](#)" in the chapter on "[Starting the Editor](#).") Whether the editor presents a PickList or selects all matching files, is determined as follows: - The default behavior is set by the LoadWildFromInside variable. (For more information on configuring this variable, see "Load Wild from Inside" under "[System/File Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") - The editor can be forced to either select all matching files or to present a PickList. To do this, you precede the filename specification with the "[Load All](#)" (-a) or the "[Pick from List](#)" (-a) command-line option. (For more information on these options, see "[Command-Line Options](#)" in the chapter on "[Starting the Editor](#).") - You can also generate an ambiguous filename by using the Filename Completion option. (For more information on this option, see "[Obtaining a File PickList within a Prompt Box](#)" in the chapter on "[Command Prompts and PickLists](#).") - If you specify a partial filename (one that does not include an extension and does not end with a "."), and the partial specification does not match an existing file, then the editor attempts to locate a matching file using the list of default extensions indicated by the DefaultExt variable. (For more information on configuring the [DefaultExt](#) variable, see "[Default File Extensions](#)" under "[System/File Options](#)"

in the chapter on "[Configuring the Editor Interactively](#)." If a matching filename is located, then that file is selected. If no match is found, the editor uses the original filename as entered. - If you specify a filename that is not valid, the message, "Open error:filename Press <Escape>", is displayed on [the StatusLine](#). When you press <Escape>, the "File (s) to edit:" prompt is re-displayed. After the specific files to load have been determined, the editor updates the Ring to include any new filenames. The editor then moves to the first (or only) filename that you specified. That file becomes the current file. If the file was not previously opened, the editor opens it at this time, either by loading the file from disk if the file already exists, or by creating a new file with the specified name and displaying the message, "Editing new file...". NOTE: When you specify multiple filenames to edit, the editor only adds the names of the indicated files to the Ring (if they are not already in the Ring). The editor does not actually open a file (by loading an existing file or creating a new file) until the file becomes the current file. If you specify a filename that is already open in the current editing session, the editor DOES NOT re-open that file. Thus, the editor never loads multiple copies of the same file, nor does it re-load a file from disk that is already loaded. * NewFile ("New" under File [Menu](#)) This command creates a new temporary memory buffer with a name in the format <unnamed-#> (where "#" represents an actual number assigned by the editor). This temporary buffer becomes the current file. Because the file that is created does not have an actual name, it cannot be saved as is. If you want to save the contents of the file, you can either execute the SaveAs command (which allows you to specify a name for the file); or you can first change the name of the file using the ChangeCurrFilename command, and then save the file under the new name. (The SaveAs and ChangeCurrFilename commands are discussed later in this chapter.) This command is useful for creating a temporary file without a name, which can later be discarded, or given a name so that it can be saved.

AddLine

Adds a line after the current line. The cursor is placed on the newly-created line. The

[cursor](#) column does [not](#) change. **Syntax:** [INTEGER](#) [AddLine](#)([[STRING](#) text [, [INTEGER](#) buffer_id]]) - text is an optional string to be placed at column 1 of the newly-created line. - buffer_id is the optional id number of the file where the line is to be added. If not passed, the line is added to the current file. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command fails if there is [not](#) enough memory to successfully complete the operation. If the optional buffer_id is specified, the line is added after the current line in the specified buffer. [_ON_CHANGING_FILES_](#) hooks are [NOT](#) invoked by this command. **Examples:** // Add a blank line after the current line [AddLine\(\)](#) // Add a SAL comment after the current line [AddLine](#)("//") /
 ***** Add a line containing the contents of string s, after the current line in buffer id, where id is the id number of a buffer obtained via [GetBufferId](#)().
 *****/ [AddLine](#)(s, id) See Also: [InsertLine](#)(), [DelLine](#)(), [KillLine](#)()

Addr

Returns the 32-bit address of the specified variable.

This command is intended for ADVANCED USAGE.

Syntax: [INTEGER](#) [Addr](#)([VAR](#) [INTEGER](#) | [VAR](#) [STRING](#) variable_name) - variable_name is either a string or integer variable Returns: The 32-bit pointer to the indicated variable. Notes: The address of an INTEGER variable is a pointer to a 32-bit long. The address of a STRING variable is a pointer to the length byte of the string variable. To point to the contents of the string variable, the pointer must be incremented by 2, which requires the use of [AdjPtr](#)(). **Examples:** [integer](#) a, foo a = [Addr](#)(foo) See Also: [AdjPtr](#)(), [Ofs](#)(), [Seg](#)()

AddTrailingSlash

Adds a trailing slash to a string if the string does not already end with a slash.

Syntax: [STRING AddTrailingSlash\(STRING path\)](#) - path is a string (typically a directory path) that needs to be terminated with a trailing slash. Returns: The string specified by path with a trailing slash. Notes: If path already has a trailing slash, path is returned unchanged. This command is typically used when a filename is to be appended to a path, and it is unknown whether the path ends in a trailing slash. This function works with both [forward slashes \(/\)](#) and [backslashes \(\\)](#). **Examples:** Assume dir and fn are strings containing a directory path and a filename, respectively: [EditFile\(AddTrailingSlash\(dir\) + fn\)](#) See Also: [isTrailingSlash\(\)](#), [RemoveTrailingSlash\(\)](#)

AdjPtr

Adjusts the offset portion of a 32-bit pointer by a specified amount.

This command is intended for ADVANCED USAGE.

Syntax: [INTEGER AdjPtr\(INTEGER ptr, INTEGER adj\)](#) - ptr is a 32-bit pointer consisting of a 16-bit segment and a 16-bit offset. - adj is the amount by which to adjust the offset portion of ptr. Returns: The 32-bit pointer as adjusted by adj. Notes: All mathematical operations within SAL are performed as signed arithmetic. Since pointer segments can be equal to or greater than 8000h, ordinary signed arithmetic on pointers may yield undesirable results. Always use [AdjPtr\(\)](#) to adjust the value of a pointer so that the segment will not change when the offset is adjusted. **Examples:** To point to the contents of a STRING variable, the pointer to the string itself must be incremented by 2. [string s\[10\] integer addr_of_string, addr_of_string_contents](#) [addr_of_string = Addr\(s\)](#) [addr_of_string_contents = AdjPtr\(addr_of_string, 2\)](#) See Also: [Addr\(\)](#), [Ofs\(\)](#), [Seg\(\)](#)

Alarm

Produces a short tone on the PC's speaker.

Syntax: [Alarm\(\)](#) Returns: Nothing. Notes: This is the same sound a failed [Find\(\)](#) issues. Under WINNT4/2000/XP, the tone is sent to the PC speaker. Under WIN95/98/Me, the following action occurs: If the system is configured with a sound card, the "[default](#) sound event" is played; otherwise, the "standard system beep" is sent to the PC speaker. See Also: [Sound\(\)](#), [Delay\(\)](#)

Alternative Modes For Viewing Files

Several alternative modes for viewing and/or editing files are provided in the editor: [Hex mode](#), [Binary mode](#), and [Browse mode](#). You can use these modes separately, or in conjunction with each another. [Hex Mode](#) ----- When Hex mode is ON, the hexadecimal values for the characters in a file are displayed on the screen. When viewing a file in Hex mode, the screen (or window) containing that file is split into two columns. 16 characters of each line of the file are displayed in text mode in the left-hand column. In the right-hand column, the hexadecimal representation is displayed for each of the characters on the left. (If the screen is split into vertical windows, the hex representation column does not display if the vertical window is too small to accommodate it.) Both the text and the hex

representations of the current character are highlighted. If you select both Hex mode and Binary mode for a file, the left-hand column will contain the binary representation for the text, and the right-hand column will contain the hexadecimal representation. To toggle Hex mode ON and OFF for a file, select "Hex View" under the Util Menu. You can also set Hex mode ON at the time a file is initially loaded by specifying the "[Hex Load](#)" (-h) command-line option at the command line or at the "File(s) to edit:" prompt. (For more information on specifying the "Hex Load" command-line option, see "[Command-Line Options](#)" in the chapter on "[Starting the Editor](#).") While you are in [Hex mode](#), all normal editing functions are available. You can edit either the text format of the data (in the text area on the left), or the hexadecimal format of the data (in the hexadecimal-display area on the right). To toggle between the text area and the hexadecimal-display area, press the <Tab> key. [Binary Mode](#) ----- When a file is loaded in Binary mode, the text and data in the file are displayed exactly as stored on disk. No translation is made of line or file termination characters (that is, carriage [return](#), linefeed, and Control-Z), or of tab characters. The file is not automatically split into lines at the occurrence of line-termination characters. Instead, the file contents are split at a user-defined length, and wrapped to successive lines. If the file is saved, no line or file termination characters are added. To view a file in Binary mode, precede the filename specification with the "[Binary Load](#)" (-b) command-line option at the time the file is specified for editing, either at the command line or the "File(s) to edit:" prompt. The length at which you want the editor to split lines must be specified at the same time. (For more information on specifying the "[Binary Load](#)" option, see "[Command-Line Options](#)" in the chapter on "[Starting the Editor](#).") While viewing a file in Binary mode, all editing functions are available; however, caution should be taken. Certain operations, such as execution of the WrapPara command, could give undesirable results if you save the file. [Browse Mode](#) ----- When Browse mode is ON for a file, you can scroll through that file, and execute any commands that do not change the text; however, you cannot execute any text-changing commands in that file, and you cannot save the file. To toggle Browse mode ON and OFF for a file, select "[Browse Mode](#)" under the Options [Menu](#).

Appendix A: Technical Specifications

In the editor, you can edit multiple files whose total size is limited only by the amount of virtual memory that Windows allows. A maximum of 2,147,483,647 files may be loaded at once, given enough memory.

The total amount that is actually available for files you are editing may be somewhat reduced by the following, if you use them:

- ClipBoards used (exactly as if each was a file)
- Macro files loaded
- Overhead for each file loaded (approximately 0.5k)
- Deletion Buffer
- Per file Undo Buffer

The maximum line length accommodated by the editor is 16,000 characters. Lines longer than this are split at the maximum line length, and wrapped to the following line.

The configuration file that is installed in the editor can have a maximum size of 63k bytes, once compiled. This includes the space for all macros, key assignments,

[menus](#), helpdefs, and datadefs. A maximum of 60 external Compiled Macro files may be loaded at any one time. Each external Compiled Macro file may have up to 63k of Compiled [Macros](#).

Appendix B: Command Implementation

The table in this appendix provides a cross-reference between the editing commands as documented in

[the User's Guide](#) (first column of the table), and the actual native command or enhanced macro by which the editing command is implemented in the editor (second column of the table). Editing commands are implemented in a number of ways: 1. By invoking a native command without any parameters. Most of these commands have the same name as the editing command, and are indicated as the command name followed by "()". For example, the AddLine command described in [the User's Guide](#) is implemented as the native [AddLine\(\)](#) command itself. Certain editing commands invoke native commands with different names. For example, the Again command is implemented as the native [RepeatFind\(\)](#) command. 2. By invoking a native command that specifies a particular parameter for that command. These are indicated by a value in "()" following the command under the second column of the table. For example, the "OverlayColumnBlock" command is implemented via the native [CopyBlock\(\)](#) command using the parameter "_OVERWRITE_", as "[CopyBlock\(_OVERWRITE_\)](#)". 3. By invoking either a macro which enhances the behavior of a native command, or a macro that provides additional features not included in the native commands. Generally, these are indicated by a macro name beginning with an "m" under the second column of the table. For example, the BackSpace command is implemented via a macro named "mBackSpace()", which enhances the native BackSpace command; and the CompressView command is provided through the mCompressView() macro, which is composed of a number of native commands. A number of commands are implemented via a macro that specifies a particular parameter. For example, the FunctionList command is implemented via the "mCompressView()" macro using the parameter "1", as "mCompressView(1)". Certain editing commands are implemented via a group of macros in an external macro file. Each of these is indicated by identifying the name of the macro file. For example, the SpellCheck command is implemented via a group of macros in the external macro file "SPELLCHK.MAC". For most commands implemented as macros, the macro source can be found (under the indicated macro name) either in the standard SemWare Editor Professional user-interface macro file, TSE.UI (located in the UI\editor subdirectory), or in a macro source file in the MAC\editor subdirectory. +-----+ |

Editing Command | Native Command or Enhanced Macro | +-----+

*-----+ | AddLine | [AddLine\(\)](#) | | Again | [RepeatFind\(\)](#) | | AgainReverse | [RepeatFind\(_REVERSE_\)](#) | | AsciiChart | mAsciiChart() | | AutoLoadList | [EditAutoLoadList\(\)](#) | | BackSpace | mBackSpace() | | BegFile | mBegFile() | | BegLine | [BegLine\(\)](#) | | BegPara | mBegPara | | BegWindow | [BegWindow\(\)](#) | | Book | external macro file: BOOK.MAC | | Capitalize | external macro file: CAPITAL.MAC | | CaptureDosOutput | external macro file: CAPTURE.MAC | | cCloseCurlyBrace | mCloseBrace() | | CD | external macro file: CD.MAC | | CenterLine | external macro file: CENTER.MAC | | ChangeCurrFilename | [ChangeCurrFilename\(\)](#) | | ChangeVideoMode | external macro file: VIDEO.MAC | | CloseFileAndWindow | mCloseFileAndWindow() | | CloseWindow | [CloseWindow\(\)](#) | | CmpFiles | external macro file: CMPFILES.MAC | | Compile | external macro file: COMPILE.MAC | | CompileCurrFile | mCompileCurrFile() | | CompressView | mCompressView(0) | | Copy to a Named Clipboard | external macro file: NAMECLIP.MAC | | Copy to the System Clipboard | [Copy\(\)](#) | | CopyAppend to a Named Clipboard | external macro file: NAMECLIP.MAC | | CopyAppend to the System Clipboard | [CopyAppend\(\)](#) | | CopyBlock | [CopyBlock\(\)](#) | | CopyCharAbove | mCopyCharAbove() | | CopyCharAboveToEol | mCopyCharAboveToEol() | | CopyToWinClip | [CopyToWinClip\(\)](#) | | CReturn | mCReturn() | | CUAMark | external macro file: CUAMARK.MAC | | Cut to a Named Clipboard | external macro file: NAMECLIP.MAC | | Cut to the System Clipboard | [Cut\(\)](#) | | CutAppend to a Named Clipboard | external macro file: NAMECLIP.MAC | | CutAppend to the System Clipboard | [CutAppend\(\)](#) | | DateTimeStamp | mDateTimeStamp() | | DebugCurrFile | mDebugCurrFile()

| DebugMacro | external macro file: DEBUG.MAC | | DelBlock | [DelBlock\(\)](#) | | DelChar | mDelChar() | | DelLeftWord | [DelLeftWord\(\)](#) | | DelLine | [DelLine\(\)](#) | | DelRightWord | [DelRightWord\(\)](#) | | DelToEol | [DelToEol\(\)](#) | | DelWindow | [DelWindow\(\)](#) | | Dos | [Dos\(\)](#) | +-----+
 +-----+ | Editing Command | Native Command or Enhanced Macro | +-----*-----+ | Down | [Down\(\)](#) | | DupLine | [DupLine\(\)](#) | | EditFile | [EditFile\(\)](#) | | EndFile | mEndFile() | | EndLine | [EndLine\(\)](#) | | EndPara | mEndPara() | | EndWindow | [EndWindow\(\)](#) | | ExecMacro | [ExecMacro\(\)](#) | | ExecScrapMacro | [ExecScrapMacro\(\)](#) | | Exit | [Exit\(\)](#) | | FileManager | external macro file: F.MAC | | FillBlock | [FillBlock\(\)](#) | | Find | [Find\(\)](#) | | FindAndDo | external macro file: FIND&DO.MAC | | FindWordAtCursor | mFindWordAtCursor('+') | | FindWordAtCursorBackwards | mFindWordAtCursor('b') | | FindWordAtCursorGlobally | mFindWordAtCursor('g') | | Flip | mFlip() | | FunctionList | mCompressView(1) | | GlobalUnDelete | [GlobalUnDelete\(\)](#) | | GotoBlockBegin | [GotoBlockBegin\(\)](#) | | GotoBlockEnd | [GotoBlockEnd\(\)](#) | | GotoColumn | [GotoColumn\(\)](#) | | GotoLine | [GotoLine\(\)](#) | | GotoMark | [GotoMark\(\)](#) | | GotoWindow | [GotoWindow\(\)](#) | | GotoWindowNumber | [GotoWindow\(n\)](#) | | Help | [Help\(\)](#) | | HelpOnKeyAssignments | [Help\("Key Assignments"\)](#) | | HelpOnWordAtCursor | mHelpOnWordAtCursor() | | HWindow | [HWindow\(\)](#) | | IncrementalSearch | external macro file: ISRCH.MAC | | InsertFile | [InsertFile\(\)](#) | | InsertLine | [InsertLine\(\)](#) | | JoinLine | [JoinLine\(\)](#) | | Justify | external macro file: JUSTIFY.MAC | | KillFile | [KillFile\(\)](#) | | Left | [Left\(\)](#) | | ListOpenFiles | external macro file: LISTOPEN.MAC | | ListRecentFiles | mListRecentFiles() | | Literal | [Literal\(\)](#) | | LoadKeyMacro | [LoadKeyMacro\(\)](#) | | LoadMacro | [LoadMacro\(\)](#) | | Lower | mLower() | | MainMenu | MainMenu() | | MarkChar | [MarkChar\(\)](#) | | MarkColumn | [MarkColumn\(\)](#) | +-----+
 +-----+ | Editing Command | Native Command or Enhanced Macro | +-----*-----+ | MarkLine | [MarkLine\(\)](#) | | MarkParagraph | mMarkPara() | | MarkStream | [MarkStream\(\)](#) | | MarkToEOL | [MarkToEOL\(\)](#) | | MarkWord | [MarkWord\(\)](#) | | Match | external macro file: MATCH.MAC | | MouseMenu | MouseMenu() | | MoveBlock | [MoveBlock\(\)](#) | | NewFile | [NewFile\(\)](#) | | NextChar | [NextChar\(\)](#) | | NextFile | [NextFile\(\)](#) | | NextPara | mNextPara() | | NextWindow | [NextWindow\(\)](#) | | OneWindow | [OneWindow\(\)](#) | | OpenFileAndWindow | mOpenFileAndWindow() | | Overlay | external macro file: OVERLAY.MAC | | OverlayColumnBlock | [CopyBlock\(_OVERWRITE_\)](#) | | PageDown | [PageDown\(\)](#) | | PageUp | [PageUp\(\)](#) | | Paste from a Named Clipboard | external macro file: NAMECLIP.MAC | | Paste from the System Clipboard | [Paste\(\)](#) | | PasteFromWinClip | [PasteFromWinClip\(\)](#) | | PasteOver from a Named Clipboard | external macro file: NAMECLIP.MAC | | PasteOver from the System Clipboard | [Paste\(_OVERWRITE_\)](#) | | PasteUnDelete | [PasteUnDelete\(\)](#) | | PlaceMark | [PlaceMark\(\)](#) | | [Potpourri](#) | external macro file: POTPOURR.MAC | | PrevChar | [PrevChar\(\)](#) | | PrevFile | [PrevFile\(\)](#) | | PrevHelp | [PrevHelp\(\)](#) | | PrevPara | mPrevPara() | | PrevPosition | [PrevPosition\(\)](#) | | PrevWindow | [PrevWindow\(\)](#) | | PrintBlock | external macro file: PRINT.MAC | | PrintFile | external macro file: PRINT.MAC | | PurgeKeyMacro | [PurgeKeyMacro\(\)](#) | | PurgeMacro | [PurgeMacro\(\)](#) | | QuitFile | [QuitFile\(\)](#) | | QuoteLine | external macro file: QUOTE.MAC | | RecordKeyMacro | [RecordKeyMacro\(\)](#) | | RedisplayViewPickList | [ViewFinds\(\)](#) | | RepeatCmd | [RepeatCmd\(\)](#) | | Replace | [Replace\(\)](#) | | ReplaceFile | [ReplaceFile\(\)](#) | | ResizeWindow | [ResizeWindow\(\)](#) | | RestoreCursorLine | [RestoreCursorLine\(\)](#) | | Right | [Right\(\)](#) | | RollDown | [RollDown\(\)](#) | +-----+
 +-----+ | Editing Command | Native Command or Enhanced Macro | +-----*-----+ | RollLeft | [RollLeft\(\)](#) | | RollRight | [RollRight\(\)](#) | | RollUp | [RollUp\(\)](#) | | SaveAllAndExit | [SaveAllAndExit\(\)](#) | | SaveAllFiles | [SaveAllFiles\(\)](#) | | SaveAndQuitFile | [SaveAndQuitFile\(\)](#) | | SaveAs | [SaveAs\(\)](#) | | SaveBlock | [SaveBlock\(\)](#) | | SaveFile | [SaveFile\(\)](#) | | SaveKeyMacro | [SaveKeyMacro\(\)](#) | | ScrollDown | [ScrollDown\(\)](#) | | ScrollToCenter | [ScrollToCenter\(\)](#) | | ScrollToTop | [ScrollToTop\(\)](#) | | ScrollUp | [ScrollUp\(\)](#) | | Shell | [Shell\(\)](#) | | Shift | mShift() | | ShiftLeft | mShiftBlock(SHIFTLEFT) | | ShiftRight | mShiftBlock(SHIFTRIGHT) | | ShowEntryScreen | [ShowEntryScreen\(\)](#) | | ShowFileInfo | external macro file: FILEINFO.MAC | | Sort | external macro file: SORT.MAC | | SpellCheck | external macro file: SPELLCHK.MAC | | SplitLine | [SplitLine\(\)](#) | | SwapLines | mSwapLines() | | TabLeft | mTabLeft() | | TabRight | mTabRight() | | ToggleAutoIndent | mToggleAutoIndent() | | ToggleHexView | mToggleHexView() | | ToggleInsert | [ToggleInsert\(\)](#) |

ToggleLineDrawing | [Toggle\(LineDrawing\)](#) | | ToggleSyncScroll | external macro file: SYNCHSCR.MAC | |
 ToggleTemplateExpansion | mToggleTemplateExpand() | | ToggleWordWrap | mToggleWordWrap() | | UnDelete |
[UnDelete\(\)](#) | | UnMarkBlock | [UnMarkBlock\(\)](#) | | Up | [Up\(\)](#) | | Upper | mUpper() | | VWindow | [VWindow\(\)](#) | | Where |
 external macro file: WHERE.MAC | | WordLeft | [WordLeft\(\)](#) | | WordRight | [WordRight\(\)](#) | | WrapPara | mWrapPara()
 | | ZoomWindow | [ZoomWindow\(\)](#) | +-----+-----+

Appendix C: Commands By Category

New commands added to version 4.4 are followed by "*".
 Existing commands which are "Enhanced" for version 4.4 are followed

by "***". Adding Text Font Position ----- AddLine ChooseFont CurrCol CopyBlock
 GetCharWidthHeight CurrLine CReturn GetFont CurrPos DupLine ResizeFont CurrRow InsertData SetFont
 CurrXoffset InsertLine GetPositionInfo InsertText Help KillPosition Literal ---- PopPosition Paste PosFirstNonWhite
 PasteReplace Help PosLastNonWhite PasteFromWinClip PrevHelp PushPosition PasteReplaceFromWinClip
 PutHelpLine SwapPosition SelfInsert QuickHelp SplitLine SearchHelp GotoMark PlaceMark Bits History ----
 KillLocation PopLocation ClearBit AddHistoryStr PushLocation GetBit DelHistory SetBit DelHistoryStr Printing
 FindHistoryStr ----- Blocks GetFreeHistory ----- GetHistoryStr ClosePrint LoadHistory PrintBlock CopyBlock
 NumHistoryItems PrintChar DelAnyCharOrBlock * SaveHistory PrintFile DelBlock SetPrinterFont DelCharOrBlock
 * Hooks/Events FillBlock ----- Process GetClipboardBlockType ----- GotoBlockBegin BreakHookChain
 GotoBlockBeginCol ExecHook Disable GotoBlockEnd GetHookState Enable GotoBlockEndCol Hook EndProcess
 isBlockInCurrFile QueryEditState Process isBlockMarked SetHookState isCurrLineInBlock TerminateEvent Session
 Global [Variables](#) isCursorInBlock UnHook ----- KillBlock Mark Initialization File DelGlobalVar
 MarkBlockBegin * ----- ExistGlobalVar MarkBlockEnd * GetGlobalInt MarkChar FlushProfile
 GetGlobalStr MarkColumn GetNextProfileItem SetGlobalInt MarkFoundText GetNextProfileSectionName
 SetGlobalStr MarkLine GetProfileInt MarkStream GetProfileStr [Strings](#) MarkToEOL LoadProfileSection -----
 MarkWord LoadProfileSectionNames MoveBlock RemoveProfileItem AddTrailingSlash PopBlock
 RemoveProfileSection Asc PrintBlock WriteProfileInt Chr PushBlock WriteProfileStr CmpiStr SaveBlock CurrChar
 UnMarkBlock [Keyboard Macros](#) DelStr ----- EquiStr Copy Format CopyAppend ExecScrapMacro
 GetFileToken Cut LoadKeyMacro GetFoundText CutAppend PurgeKeyMacro GetMarkedText Paste
 RecordKeyMacro GetStrFromWinClip * PasteReplace SaveKeyMacro GetText GetToken CopyToWinClip **
 Keyboard GetWord CopyAppendToWinClip ** ----- InsStr CutToWinClip isTrailingSlash CutAppendToWinClip
 Ask isWildPath PasteFromWinClip AskFilename LeftStr * PasteReplaceFromWinClip AskNumeric Length
 ChainCmd LTrim [Bookmarks](#) Disable MatchFilename ----- Enable NumFileTokens GetKey NumTokens
 GotoMark GetKeyFlags Pos PlaceMark isKeyAssigned * QuotePath isTypeableKey RemoveTrailingSlash
 DelAllBookMarks KeyCode RightStr DelBookMark KeyName RTrim GetBookMarkInfo KeyPressed SizeOf
 isBookMarkSet lRead Str lReadNumeric StrCount * KillPosition Message StrFind * PopPosition MsgBox **
 StrReplace * PrevPosition MsgBoxBuff SubStr PushPosition MsgBoxEx Trim PressKey Val [Buffers](#) PushKey
 VersionStr * ----- PushKeyStr Read SyntaxHilite AbandonFile ReadNumeric ----- AddFFInfoToBuffer
 WaitForKeyPressed * AddFileToRing Warn GetSynFilename BinaryMode YesNo GetSynLanguageType
 BrowseMode GetSynMultiLnDlmt BufferType Low Level GetSynToEOL ChangeCurrFilename -----
 InitSynhiCurrFile CheckDefaultExt LinkSynFile CreateBuffer Addr LoadSynhiAssoc CreateTempBuffer AdjPtr
 PurgeSynhi CurrExt Ofs ReplaceSynFile CurrFilename Ptr CurrLineLen Seg Tabs DisplayMode ---- EditBuffer
 HiByte EditFile HiWord DistanceToTab EmptyBuffer LoByte EntabCurrLine FileChanged LoWord
 ExpandTabsToSpaces GetBufferId TabLeft GotoBufferId PeekByte TabRight HexEdit PeekLong TabSet InsertFile
 PeekWord LoadBuffer PokeByte Termination LongestLineInBuffer PokeLong ----- NewFile PokeWord NextFile
 AbandonEditor NumFiles [Macros](#) (Compiled) Exit NumLines ----- SaveAllAndExit PrevFile PrintFile
 AddAutoLoadMacro UnDelete QuitFile ChainCmd ----- RemoveUnloadedFiles DelAutoLoadMacro ReplaceFile
 EditAutoLoadList GlobalUnDelete SaveAllFiles ExecLoadedMacro PasteUnDelete SaveAndQuitFile ExecMacro **
 RestoreCursorLine SaveFile isAutoLoaded * UnDelete isGUI ClearBufferDaTmAttr isMacroLoaded Video

Commands GetBufferDaTmAttr LoadMacro ----- UpdateBufferDaTmAttr LoadStartupMacros
 MacroStackAvail BufferVideo DelBufferVar PurgeMacro ClrEol ExistBufferVar ClrScr GetBufferInt [Menus](#) Color
 GetBufferStr ----- DrawBox SetBufferInt FullWindow SetBufferStr MenuKey GetColorTableValue MenuOption
 GetMaxRowsCols Buffer Global [Variables](#) MenuStr GetStr ----- SetMenuBar GetStrAttr SizeOf
 GetStrAttrXY DelBufferVar GetStrXY ExistBufferVar Miscellaneous GotoXY GetBufferInt ----- ListFooter
 GetBufferStr ListHeader SetBufferInt Abs PopWinClose SetBufferStr ChrSet PopWinOpen Escape PutAttr
[ClipBoards](#) GetWindowTitle PutAttrXY ----- GetWinHandle PutChar isAlpha PutCharH Copy isAlphaNum
 PutCharHXY CopyAppend isDigit PutCharV Cut isHexDigit PutCharXY CutAppend isLower PutCtrStr Paste isUpper
 PutLine PasteReplace isWhite PutLineXY isWord PutNCharAttr CopyToWinClip ** LineDraw PutOemStrXY
 CopyAppendToWinClip ** LineTypeMenu PutStr CutToWinClip List PutStrAttr CutAppendToWinClip IList
 PutStrAttrXY PasteFromWinClip IVersion PutStrEOL PasteReplaceFromWinClip Max PutStrEOLXY Min PutStrXY
 EmptyWinClip NoOp SetColorTableValue GetStrFromWinClip * RedoCount SetCursorOff isWinClipAvailable
 RepeatCmd SetCursorOn SaveSettings SetVideoRowsCols Date/Time SetUndoOff UnBufferVideo -----
 SetUndoOn VGotoXY SetWindowTitle VGotoXYAbs GetClockTicks SignOn VHomeCursor GetDate ToggleInsert
 VWhereX GetDateStr UndoCount VWhereY GetTime UndoMode WhereX GetTimeStr UpdateDisplay WhereXAbs
 UpdateDisplayFlags WhereY Deleting Text Version WhereYAbs ----- VersionStr * Window WindowFooter
 BackSpace Mouse Write Cut ----- WriteLine DelAnyChar * DelAnyCharOrBlock * GetWheelScrollLines [Windows](#)
 DelBlock GotoMouseCursor ----- DelCharOrBlock * HideMouse DelChar MouseHotSpot CloseWindow
 DelLeftWord MouseKeyHeld DelWindow DelLine MouseMarking GotoWindow DelRightWord MouseStatus
 HWindow DelToEol MouseWindowId isZoomed JoinLine ProcessHotSpot NextWindow KillBlock ShowMouse
 NumWindows KillLine TrackMouseCursor OneWindow KillToEol WaitForMouseEvent PrevWindow ResizeWindow
 Disk File I/O Movement SetWindowHeight ----- SetWindowWidth VWindow fClose BegFile WindowId
 fCreate BegLine ZoomWindow fDup BegLineTog * fDup2 BegWindow fOpen BegWord fRead Down fReadFile
 EndFile fSeek EndLine fWrite EndLineTog * fWriteFile EndWindow isCharDevice EndWord GotoBlockBegin
 Editing GotoBlockEnd ----- GotoColumn GotoLine AddLine GotoPos BackSpace GotoRow ClearUndoRedoList
 GotoXoffset CReturn Left DelCharOrBlock * NextChar DelAnyCharOrBlock * PageDown DelChar PageUp
 DelAnyChar * PrevChar DelLeftWord PrevPosition DelLine Right DelRightWord RollDown DelToEol RollLeft
 DupLine RollRight Flip RollUp InsertData ScrollDown InsertLine ScrollLeft InsertText ScrollRight JoinLine
 ScrollToBottom * KillLine ScrollToCenter KillToEol ScrollToRow Literal ScrollToTop Lower ScrollUp Redo Up
 SelfInsert WordLeft ShiftText WordRight Sort SplitLine OS Access Undo ----- Upper WrapLine Alarm WrapPara
 ChDir CurrDir [Editor Variables](#) Delay ----- Dos DosIOResult Query ExpandPath Set FileExists Toggle
 GetDir GetDrive Files GetEnvStr ----- is32BitApp isFullScreen AbandonFile lDos AddFileToRing LoadDir
 ChangeCurrFilename LogDrive ChangedFilesExist lShowEntryScreen CopyFile MakeTempName CurrExt PickDrive
 CurrFilename RenameDiskFile CurrMacroFilename SearchPath EditFile SetFileAttr EditThisFile Shell EraseDiskFile
 ShowEntryScreen FileChanged Sound InsertFile SplitPath KillFile SqueezePath LockCurrentFile
 StartedFromDosPrompt MkDir StartPgm * MoveFile WhichOS ** NewFile NextFile BuildPickBufferEx NumFiles
 PBAttribute PickFile PBDate PrevFile PBDateStr PrintFile PBName QuitFile PBSize RemoveUnloadedFiles PBTime
 RenameAndSaveFile PBTimeStr ReplaceFile Rmdir FFAttribute SaveAllFiles FFDate SaveAndQuitFile FFDateStr
 SaveAs FFName SaveFile FFSIZE UnLockCurrentFile FFTime FFTimeStr Find/Replace FindFileClose -----
 FindFirstFile FindNextFile Find FindThisFile GetFoundText HiLiteFoundText lFind lRepeatFind lReplace
 MarkFoundText RepeatFind Replace ViewFinds RepeatFind Replace ViewFinds

Appendix C: Standard Key Assignments

This appendix identifies the key assignments in the standard editor configuration.

FUNCTION

[KEY](#) EDITOR COMMAND/MACRO ----- <F1> [Help\(\)](#) <F2> [AddLine](#)
 () <F4> [DupLine\(\)](#) <F5> [ScrollToTop\(\)](#) <F6> [DelToEol\(\)](#) <F9> [Shell\(\)](#) <F10> MainMenu() <F12> [ExecMacro](#)

("SpellChk") <Alt F1> [PrevHelp\(\)](#) <Alt F2> [InsertLine\(\)](#) <Alt F3> [ExecMacro\("match"\)](#) <Alt F4> [mLower\(\)](#) <Alt F5> [RollLeft\(\)](#) <Alt F6> [RollRight\(\)](#) <Alt F9> [Dos\(\)](#) <Alt F10> [ShowEntryScreen\(\)](#) <Ctrl F1> [mHelpOnWordAtCursor\(\)](#) <Ctrl F2> [mSwapLines\(\)](#) <Ctrl F7> [mShift\(\)](#) <Ctrl f9> [mCompileCurrFile\(\)](#) <Ctrl F10> [ExecLoadPurge\(\)](#) <Shift F1> [Help\("Key Assignments"\)](#) <Shift F3> [ExecMacro\("sort " + \[Str\\(sort_flags\\)\]\(#\)\)](#) <Shift F4> [mUpper\(\)](#) <Shift F5> [ScrollToCenter\(\)](#) <Shift F7> [mShiftBlock\(SHIFTLLEFT\)](#) <Shift F8> [mShiftBlock\(SHIFTRIGHT\)](#) <Shift F9> [ExecMacro\("Compile -m"\)](#) <Shift F10> [ExecMacro\("Potpourr"\)](#) <AltShift F9> [ExecMacro\("Capture"\)](#) SPECIAL-PURPOSE [KEY](#) EDITOR COMMAND/MACRO -----

----- <Escape> [MainMenu\(\)](#) <BackSpace> [mBackSpace\(\)](#) <Alt BackSpace> [Undo\(\)](#) <Ctrl BackSpace> [DelLeftWord\(\)](#) <AltShift BackSpace> [Redo\(\)](#) <Enter> [mCreturn\(\)](#) <Ctrl Enter> [ExecScrapMacro\(\)](#) <Tab> [mTabRight\(\)](#) <Shift Tab> [mTabLeft\(\)](#) <CtrlAlt Spacebar> IVersion [KEYPAD](#) [KEY](#) EDITOR COMMAND/MACRO -----

----- <Ins> [ToggleInsert\(\)](#) <CursorDown> [Down\(\)](#) <Ctrl CursorDown> [RollDown\(\)](#) <CursorLeft> [Left\(\)](#) <Ctrl CursorLeft> [WordLeft\(\)](#) <CursorRight> [Right\(\)](#) <Ctrl CursorRight> [WordRight\(\)](#) <CursorUp> [Up\(\)](#) <Ctrl CursorUp> [RollUp\(\)](#) [mDelChar\(\)](#) <Alt Del> [DelToEol\(\)](#) <Ctrl Del> [DelRightWord\(\)](#) <End> [EndLine\(\)](#) <Ctrl End> [EndWindow\(\)](#) <Grey*> [Paste\(\)](#) <Ctrl Grey*> [Paste\(_OVERWRITE_\)](#) <CtrlShift Grey*> [PasteFromWinClip\(\)](#) <Grey++> [Copy\(\)](#) <Ctrl Grey++> [Copy\(_APPEND_\)](#) <CtrlShift Grey++> [CopyToWinClip\(\)](#) <Grey-> [Cut\(\)](#) <Ctrl Grey-> [Cut\(_APPEND_\)](#) <CtrlShift Grey-> [CutToWinClip\(\)](#) <Home> [BegLine\(\)](#) <Ctrl Home> [BegWindow\(\)](#) <PgDn> [PageDown\(\)](#) <Alt PgDn> [mNextPara\(\)](#) <Ctrl PgDn> [mEndFile\(\)](#) <Shift PgDn> [mNextPara\(\)](#) <PgUp> [PageUp\(\)](#) <Alt PgUp> [mPrevPara\(\)](#) <Ctrl PgUp> [mBegFile\(\)](#) <Shift PgUp> [mPrevPara\(\)](#) <Ctrl PrtSc> [Paste\(_OVERWRITE_\)](#) MOUSE BUTTON EDITOR COMMAND/MACRO -----

----- <LeftBtn> [mLeftBtn\(\)](#) <Alt LeftBtn> [MouseMarking\(_LINE_\)](#) <Ctrl LeftBtn> [MouseMarking\(_COLUMN_\)](#) <Shift LeftBtn> [mTrackMouseCursor\(\)](#) <RightBtn> [MouseMenu\(\)](#) CONTROL [KEY](#) EDITOR COMMAND/MACRO -----

----- <Ctrl -> [mCopyCharAbove\(\)](#) <Ctrl 0> [mListRecentFiles\(\)](#) <Ctrl 2> [PlaceMark\(\)](#) <Ctrl 6> [GotoMark\(\)](#) <Ctrl 8> [ExecMacro\("quote"\)](#) <Ctrl => [mFindWordAtCursor\('b'\)](#) <Ctrl [> [GotoBlockBegin\(\)](#) <Ctrl \> [mFindWordAtCursor\('g'\)](#) <Ctrl]> [GotoBlockEnd\(\)](#) <Ctrl A> [mAsciiChart\(\)](#) <Ctrl B> [ExecMacro\("nameclip \[Named ClipBoards\]\(#\)"\)](#) <Ctrl C> [GotoColumn\(\)](#) <Ctrl D> [DelLine\(\)](#) <Ctrl F> [Find\(\)](#) <Ctrl G> [mCompressView\(1\)](#) <Ctrl I> [ExecMacro\("isrch"\)](#) <Ctrl J> [GotoLine\(\)](#) <Ctrl K><0> [PlaceMark\("q"\)](#) <Ctrl K><1> [PlaceMark\("r"\)](#) <Ctrl K><2> [PlaceMark\("s"\)](#) <Ctrl K><3> [PlaceMark\("t"\)](#) <Ctrl K><4> [PlaceMark\("u"\)](#) <Ctrl K><5> [PlaceMark\("v"\)](#) <Ctrl K><6> [PlaceMark\("w"\)](#) <Ctrl K><7> [PlaceMark\("x"\)](#) <Ctrl K><8> [PlaceMark\("y"\)](#) <Ctrl K><9> [PlaceMark\("z"\)](#) <Ctrl K><L> [FillBlock\(\)](#) <Ctrl K><Q> [QuitFile\(\)](#) <Ctrl K><S> [SaveFile\(\)](#) <Ctrl K><T> [MarkWord\(\)](#) <Ctrl K><X> [SaveAndQuitFile\(\)](#) <Ctrl K><Z> [KillFile\(\)](#) <Ctrl L> [RepeatFind\(\)](#) <Ctrl M> [RecordKeyMacro\(\)](#) <Ctrl N> [InsertLine\(\)](#) <Ctrl O><C> [CloseWindow\(\)](#) <Ctrl O><D> [DelWindow\(\)](#) <Ctrl O><G> [GotoWindow\(\)](#) <Ctrl O><H> [HWindow\(\)](#) <Ctrl O><N> [NextWindow\(\)](#) <Ctrl O><O> [OneWindow\(\)](#) <Ctrl O><P> [PrevWindow\(\)](#) <Ctrl O><R> [ResizeWindow\(\)](#) <Ctrl O><T> [ExecMacro\("center"\)](#) <Ctrl O><V> [VWindow\(\)](#) <Ctrl O><W> [mToggleWordWrap\(\)](#) <Ctrl O><Y> [ExecMacro\("synchscr"\)](#) <Ctrl O><Z> [ZoomWindow\(\)](#) <Ctrl P> [Literal\(\)](#) <Ctrl Q><0> [GotoMark\("q"\)](#) <Ctrl Q><1> [GotoMark\("r"\)](#) <Ctrl Q><2> [GotoMark\("s"\)](#) <Ctrl Q><3> [GotoMark\("t"\)](#) <Ctrl Q><4> [GotoMark\("u"\)](#) <Ctrl Q><5> [GotoMark\("v"\)](#) <Ctrl Q><6> [GotoMark\("w"\)](#) <Ctrl Q><7> [GotoMark\("x"\)](#) <Ctrl Q><8> [GotoMark\("y"\)](#) <Ctrl Q><9> [GotoMark\("z"\)](#) <Ctrl Q><I> [mToggleAutoIndent\(\)](#) <Ctrl Q><L> [RestoreCursorLine\(\)](#) <Ctrl Q><P> [PrevPosition\(\)](#) <Ctrl Q><Q> [RepeatCmd\(\)](#) <Ctrl R> [Replace\(\)](#) <Ctrl T> [DelRightWord\(\)](#) <Ctrl U> [GlobalUnDelete\(\)](#) <Ctrl V> [Toggle\(LineDrawing\)](#) <Ctrl W> [ScrollUp\(\)](#) <Ctrl X> [ExecMacro\(\)](#) <Ctrl Y> [DelLine\(\)](#) <Ctrl Z> [ScrollDown\(\)](#) <CtrlShift 8> [ExecMacro\("quote cmt"\)](#) <CtrlShift L> [RepeatFind\(_REVERSE_\)](#) ALT [KEY](#) EDITOR COMMAND/MACRO -----

----- <Alt -> [mCopyCharAboveToEol\(\)](#) <Alt 0> [ExecMacro\("listopen"\)](#) <Alt 1> [GotoWindow\(1\)](#) <Alt 2> [GotoWindow\(2\)](#) <Alt 3> [GotoWindow\(3\)](#) <Alt 4> [GotoWindow\(4\)](#) <Alt 5> [GotoWindow\(5\)](#) <Alt 6> [GotoWindow\(6\)](#) <Alt 7> [GotoWindow\(7\)](#) <Alt 8> [GotoWindow\(8\)](#) <Alt 9> [GotoWindow\(9\)](#) <Alt => [mFindWordAtCursor\('+'\)](#) <Alt A> [MarkStream\(\)](#) <Alt B> [ExecMacro\("wrappara"\)](#) <Alt C> [CopyBlock\(\)](#) <Alt D> [DelLine\(\)](#) <Alt E> [EditThisFile\(\)](#) <Alt F> [MainMenu\('F'\)](#) <Alt G> [DelBlock\(\)](#) <Alt I> [MarkChar\(\)](#) <Alt J> [JoinLine\(\)](#)

<Alt K> [MarkColumn\(\)](#) <Alt L> [MarkLine\(\)](#) <Alt M> [MoveBlock\(\)](#) <Alt N> [NextFile\(\)](#) <Alt O> [ChangeCurrFilename\(\)](#) <Alt P> [PrevFile\(\)](#) <Alt R> [InsertFile\(\)](#) <Alt S> [SplitLine\(\)](#) <Alt T> [MainMenu\("T"\)](#) <Alt U> [UnMarkBlock\(\)](#) <Alt V> [mCompressView\(0\)](#) <Alt W> [SaveBlock\(\)](#) <Alt X> [Exit\(\)](#) <ALT Y> [MarkToEOL\(\)](#) <Alt Z> [CopyBlock\(_OVERWRITE_\)](#) <AltShift V> [ViewFinds\(\)](#) SHIFT [KEY](#) EDITOR COMMAND/MACRO
 ----- <> [mCloseBrace\(\)](#)

Appendix D: Variables By Category

New

[variables](#) added to version 4.4 are followed by "*". Existing [variables](#) which are "Enhanced" for version 4.4 are followed by "***". Blocks History SyntaxHilite ----- [BlockBegCol](#) [CurrHistoryList](#) [Directive1Attr](#) [BlockBegLine](#) [MaxHistoryPerList](#) [Directive2Attr](#) [BlockEndCol](#) [PersistentHistory](#) [Directive3Attr](#) [BlockEndLine](#) [MaxHistorySize](#) [BlockId](#) [KeyWords1Attr](#) Keyboard [KeyWords2Attr](#) [ClipboardId](#) ----- [KeyWords3Attr](#) [InsertLineBlocksAbove](#) [KeyWords4Attr](#) [Marking](#) [EquateEnhancedKbd](#) [KeyWords5Attr](#) [UnMarkAfterPaste](#) [KeyWords6Attr](#) [UseCurrLineIfNoBlock](#) [Key](#) [KeyWords7Attr](#) [LastKey](#) [KeyWords8Attr](#) [BlockAttr](#) [KeyWords9Attr](#) [CursorInBlockAttr](#) [KbdMacroRecording](#) [KbdMacroRunning](#) [MultiLnDlmt1Attr](#) Colors [MultiLnDlmt2Attr](#) ----- Line Drawing [MultiLnDlmt3Attr](#) ----- [SingleLnDlmt1Attr](#) [BlockAttr](#) [SingleLnDlmt2Attr](#) [CurrWinBorderAttr](#) [LineDrawChar](#) [SingleLnDlmt3Attr](#) [CursorAttr](#) [LineDrawing](#) [ToEOL1Attr](#) [CursorInBlockAttr](#) [LineDrawType](#) [ToEOL2Attr](#) [EOFMarkerAttr](#) [ToEOL3Attr](#) [HelpBoldAttr](#) [Macros](#) [Quote1Attr](#) [HelpInfoAttr](#) ----- [Quote2Attr](#) [HelpLinkAttr](#) [Quote3Attr](#) [HelpSelectAttr](#) [IdleTime](#) [IncompleteQuoteAttr](#) [HelpTextAttr](#) [MacroCmdLine](#) [HiLiteAttr](#) [NumberAttr](#) [LineNumbersAttr](#) * [Menus](#) [MenuBorderAttr](#) ----- [ShowSyntaxHilite](#) [MenuGrayAttr](#) [MenuSelectAttr](#) [MenuBorderAttr](#) System Interface [MenuSelectGrayAttr](#) [MenuGrayAttr](#) ----- [MenuSelectLtrAttr](#) [MenuSelectAttr](#) [MenuTextAttr](#) [MenuSelectGrayAttr](#) [Beep](#) [MenuTextLtrAttr](#) [MenuSelectLtrAttr](#) [ChangeShellPrompt](#) [MsgAttr](#) [MenuTextAttr](#) [DosCmdLine](#) [OtherWinBorderAttr](#) [MenuTextLtrAttr](#) [EquateEnhancedKbd](#) [StatusLineAttr](#) [ShowMainMenu](#) [FileLocking](#) [TextAttr](#) [IdleTime](#) Miscellaneous [KbdPath](#) [Directive1Attr](#) ----- [KeepUndoBeyondSave](#) [Directive2Attr](#) [KeepWinOnTop](#) * [Directive3Attr](#) [Break](#) [Key](#) [IncompleteQuoteAttr](#) [GUIStartupFlags](#) [LastKey](#) [KeyWords1Attr](#) [KillMax](#) [LoadWildFromDOS](#) [KeyWords2Attr](#) [MaxHistorySize](#) [LoadWildFromInside](#) [KeyWords3Attr](#) [MaxRecentFiles](#) [MacroCmdLine](#) [KeyWords4Attr](#) [MsgLevel](#) [PickFileChangesDir](#) [KeyWords5Attr](#) [PersistentRecentFiles](#) [SingleInstance](#) * [KeyWords6Attr](#) [SaveState](#) [TSEPath](#) [KeyWords7Attr](#) [StartupFlags](#) [UseCommonDialogs](#) [KeyWords8Attr](#) [TemplateExpansion](#) [KeyWords9Attr](#) Tabs [MultiLnDlmt1Attr](#) Mouse ---- [MultiLnDlmt2Attr](#) ----- [MultiLnDlmt3Attr](#) [CursorTabWidth](#) [NumberAttr](#) [LastMouseKey](#) [DetabOnLoad](#) [Quote1Attr](#) [LastMouseX](#) [EntabOnSave](#) [Quote2Attr](#) [LastMouseY](#) [ExpandTabs](#) [Quote3Attr](#) [MouseEnabled](#) [TabType](#) [SingleLnDlmt1Attr](#) [MouseHoldTime](#) [TabShiftsBlock](#) [SingleLnDlmt2Attr](#) [MouseKey](#) [TabWidth](#) [SingleLnDlmt3Attr](#) [MouseRepeatDelay](#) [VarTabs](#) [ToEOL1Attr](#) [MouseX](#) [ZeroBasedTab](#) [ToEOL2Attr](#) [MouseY](#) [ToEOL3Attr](#) Video Display [PickFile](#) ----- [Attr](#) ----- [CurrVideoMode](#) [Date/Time](#) [PickFileFlags](#) [CurrWinBorderType](#) ----- [PickFileSortOrder](#) [Cursor](#) [PickFileChangesDir](#) [DisplayBoxed](#) [DateFormat](#) [InsertCursorSize](#) [DateSeparator](#) Printing [OtherWinBorderType](#) [TimeFormat](#) ----- [OverwriteCursorSize](#) [TimeSeparator](#) [PopWinCols](#) [PrintBotMargin](#) [PopWinRows](#) Files/Buffers [PrintCopies](#) [PopWinX1](#) ----- [PrintFirstPage](#) [PopWinY1](#) [PrinterFontFlags](#) [StartupVideoMode](#) [BackupExt](#) [PrinterFontName](#) [Transparency](#) * [BackupPath](#) [PrinterFontSize](#) [WindowX1](#) [BufferFlags](#) [PrintFooter](#) [WindowY1](#) [DefaultExt](#) [PrintHeader](#) [X1](#) [EOFTYPE](#) [PrintLastPage](#) [Y1](#) [EOLType](#) [PrintLeftMargin](#) [FileLocking](#) [PrintLineNumbers](#) [ScreenCols](#) [LoadWildFromDOS](#) [PrintLineSpacing](#) [ScreenRows](#) [LoadWildFromInside](#) [PrintLinesPerPage](#) [WindowCols](#) [MakeBackups](#) [PrintPause](#) [WindowRows](#) [ProtectedSaves](#) [PrintRightMargin](#) [QuitToPrompt](#) [PrintTopMargin](#) [ShowEOFMarker](#) [PrintUseScreenFont](#) [ShowHelpLine](#) Find/Replace [ShowLineNumbers](#) * ----- [StatusLine](#) ----- [FontFlags](#) [CenterFinds](#) [FontName](#) [FindOptions](#) [ShowStatusLine](#) [FontSize](#) [ReplaceOptions](#) [StatusLineAtTop](#) [SpecialEffects](#) ** [ViewFindsId](#) [StatusLineFillChar](#) [StatusLineRow](#) Word Processing Help [StatusLineUpdating](#) -----

[AutoIndent](#) [HelpBoldAttr](#) [Insert](#) [HelpInfoAttr](#) [LeftMargin](#) [HelpItalicsAttr](#) [ParaEndStyle](#) [HelpLinkAttr](#)
[RemoveTrailingWhite](#) [HelpSelectAttr](#) [ReturnEqNextLine](#) [HelpTextAttr](#) [RightMargin](#) [WordSet](#) [HelpLineDelay](#)
[WordWrap](#) [ShowHelpLine](#)

Appendix E: Command Implementation

The table in this appendix provides a cross-reference between the editing commands as documented in

[the User's Guide](#) (first column of the table), and the actual native command or enhanced macro by which the editing command is implemented in the editor (second column of the table). Editing commands are implemented in a number of ways: 1. By invoking a native command without any parameters. Most of these commands have the same name as the editing command, and are indicated as the command name followed by "()". For example, the AddLine command described in [the User's Guide](#) is implemented as the native [AddLine\(\)](#) command itself. Certain editing commands invoke native commands with different names. For example, the Again command is implemented as the native [RepeatFind\(\)](#) command. 2. By invoking a native command that specifies a particular parameter for that command. These are indicated by a value in "(" following the command under the second column of the table. For example, the "OverlayColumnBlock" command is implemented via the native [CopyBlock\(\)](#) command using the parameter "_OVERWRITE_", as "[CopyBlock\(_OVERWRITE_\)](#)". 3. By invoking either a macro which enhances the behavior of a native command, or a macro that provides additional features not included in the native commands. Generally, these are indicated by a macro name beginning with an "m" under the second column of the table. For example, the BackSpace command is implemented via a macro named "mBackSpace()", which enhances the native BackSpace command; and the CompressView command is provided through the mCompressView() macro, which is composed of a number of native commands. A number of commands are implemented via a macro that specifies a particular parameter. For example, the FunctionList command is implemented via the "mCompressView()" macro using the parameter "1", as "mCompressView(1)". Certain editing commands are implemented via a group of macros in an external macro file. Each of these is indicated by identifying the name of the macro file. For example, the SpellCheck command is implemented via a group of macros in the external macro file "SPELLCHK.MAC". For most commands implemented as macros, the macro source can be found (under the indicated macro name) either in the standard SemWare Editor Professional user-interface macro file, TSE.UI (located in the UI\ editor subdirectory), or in a macro source file in the MAC\ editor subdirectory. +-----+ |

Editing Command | Native Command or Enhanced Macro | +-----+

*-----+ | AddLine | [AddLine\(\)](#) | | Again | [RepeatFind\(\)](#) | | AgainReverse | [RepeatFind\(_REVERSE_\)](#) | | AsciiChart | mAsciiChart() | | AutoLoadList | [EditAutoLoadList\(\)](#) | | BackSpace | mBackSpace() | | BegFile | mBegFile() | | BegLine | [BegLine\(\)](#) | | BegPara | mBegPara | | BegWindow | [BegWindow\(\)](#) | | Book | external macro file: BOOK.MAC | | Capitalize | external macro file: CAPITAL.MAC | | CaptureDosOutput | external macro file: CAPTURE.MAC | | cCloseCurlyBrace | mCloseBrace() | | CD | external macro file: CD.MAC | | CenterLine | external macro file: CENTER.MAC | | ChangeCurrFilename | [ChangeCurrFilename\(\)](#) | | ChangeVideoMode | external macro file: VIDEO.MAC | | CloseFileAndWindow | mCloseFileAndWindow() | | CloseWindow | [CloseWindow\(\)](#) | | CmpFiles | external macro file: CMPFILES.MAC | | Compile | external macro file: COMPILE.MAC | | CompileCurrFile | mCompileCurrFile() | | CompressView | mCompressView(0) | | Copy to a Named ClipBoard | external macro file: NAMECLIP.MAC | | Copy to the System ClipBoard | [Copy\(\)](#) | | CopyAppend to a Named ClipBoard | external macro file: NAMECLIP.MAC | | CopyAppend to the System ClipBoard | [CopyAppend\(\)](#) | | CopyBlock | [CopyBlock\(\)](#) | | CopyCharAbove | mCopyCharAbove() | | CopyCharAboveToEol | mCopyCharAboveToEol() | | CopyToWinClip | [CopyToWinClip\(\)](#) | | CReturn | mCReturn() | | CUAMark | external macro file: CUAMARK.MAC | | Cut to a Named ClipBoard | external macro file: NAMECLIP.MAC | | Cut to the System ClipBoard | [Cut\(\)](#) | | CutAppend to a Named ClipBoard | external macro file: NAMECLIP.MAC | | CutAppend

to the System ClipBoard | [CutAppend\(\)](#) | | DateTimeStamp | mDateTimeStamp() | | DebugCurrFile | mDebugCurrFile() | | DebugMacro | external macro file: DEBUG.MAC | | DelBlock | [DelBlock\(\)](#) | | DelChar | mDelChar() | | DelLeftWord | [DelLeftWord\(\)](#) | | DelLine | [DelLine\(\)](#) | | DelRightWord | [DelRightWord\(\)](#) | | DelToEol | [DelToEol\(\)](#) | | DelWindow | [DelWindow\(\)](#) | | Dos | [Dos\(\)](#) | +-----+-----+
+-----+-----+ | Editing Command | Native Command or Enhanced Macro | +-----*-----+ | Down | [Down\(\)](#) | | DupLine | [DupLine\(\)](#) | | EditFile | [EditFile\(\)](#) | | EndFile | mEndFile() | | EndLine | [EndLine\(\)](#) | | EndPara | mEndPara() | | EndWindow | [EndWindow\(\)](#) | | ExecMacro | [ExecMacro\(\)](#) | | ExecScrapMacro | [ExecScrapMacro\(\)](#) | | Exit | [Exit\(\)](#) | | FileManager | external macro file: F.MAC | | FillBlock | [FillBlock\(\)](#) | | Find | [Find\(\)](#) | | FindAndDo | external macro file: FIND&DO.MAC | | FindWordAtCursor | mFindWordAtCursor('+') | | FindWordAtCursorBackwards | mFindWordAtCursor('b') | | FindWordAtCursorGlobally | mFindWordAtCursor('g') | | Flip | mFlip() | | FunctionList | mCompressView(1) | | GlobalUnDelete | [GlobalUnDelete\(\)](#) | | GotoBlockBegin | [GotoBlockBegin\(\)](#) | | GotoBlockEnd | [GotoBlockEnd\(\)](#) | | GotoColumn | [GotoColumn\(\)](#) | | GotoLine | [GotoLine\(\)](#) | | GotoMark | [GotoMark\(\)](#) | | GotoWindow | [GotoWindow\(\)](#) | | GotoWindowNumber | [GotoWindow\(n\)](#) | | Help | [Help\(\)](#) | | HelpOnKeyAssignments | [Help\("Key Assignments"\)](#) | | HelpOnWordAtCursor | mHelpOnWordAtCursor() | | HWindow | [HWindow\(\)](#) | | IncrementalSearch | external macro file: ISRCH.MAC | | InsertFile | [InsertFile\(\)](#) | | InsertLine | [InsertLine\(\)](#) | | JoinLine | [JoinLine\(\)](#) | | Justify | external macro file: JUSTIFY.MAC | | KillFile | [KillFile\(\)](#) | | Left | [Left\(\)](#) | | ListOpenFiles | external macro file: LISTOPEN.MAC | | ListRecentFiles | mListRecentFiles() | | Literal | [Literal\(\)](#) | | LoadKeyMacro | [LoadKeyMacro\(\)](#) | | LoadMacro | [LoadMacro\(\)](#) | | Lower | mLower() | | MainMenu | MainMenu() | | MarkChar | [MarkChar\(\)](#) | | MarkColumn | [MarkColumn\(\)](#) | +-----+-----+
+-----+-----+ | Editing Command | Native Command or Enhanced Macro | +-----*-----+ | MarkLine | [MarkLine\(\)](#) | | MarkParagraph | mMarkPara() | | MarkStream | [MarkStream\(\)](#) | | MarkToEOL | [MarkToEOL\(\)](#) | | MarkWord | [MarkWord\(\)](#) | | Match | external macro file: MATCH.MAC | | MouseMenu | MouseMenu() | | MoveBlock | [MoveBlock\(\)](#) | | NewFile | [NewFile\(\)](#) | | NextChar | [NextChar\(\)](#) | | NextFile | [NextFile\(\)](#) | | NextPara | mNextPara() | | NextWindow | [NextWindow\(\)](#) | | OneWindow | [OneWindow\(\)](#) | | OpenFileAndWindow | mOpenFileAndWindow() | | Overlay | external macro file: OVERLAY.MAC | | OverlayColumnBlock | [CopyBlock\(_OVERWRITE_\)](#) | | PageDown | [PageDown\(\)](#) | | PageUp | [PageUp\(\)](#) | | Paste from a Named ClipBoard | external macro file: NAMECLIP.MAC | | Paste from the System ClipBoard | [Paste\(\)](#) | | PasteFromWinClip | [PasteFromWinClip\(\)](#) | | PasteOver from a Named ClipBoard | external macro file: NAMECLIP.MAC | | PasteOver from the System ClipBoard | [Paste\(_OVERWRITE_\)](#) | | PasteUnDelete | [PasteUnDelete\(\)](#) | | PlaceMark | [PlaceMark\(\)](#) | | [Potpourri](#) | external macro file: POTPOURR.MAC | | PrevChar | [PrevChar\(\)](#) | | PrevFile | [PrevFile\(\)](#) | | PrevHelp | [PrevHelp\(\)](#) | | PrevPara | mPrevPara() | | PrevPosition | [PrevPosition\(\)](#) | | PrevWindow | [PrevWindow\(\)](#) | | PrintBlock | external macro file: PRINT.MAC | | PrintFile | external macro file: PRINT.MAC | | PurgeKeyMacro | [PurgeKeyMacro\(\)](#) | | PurgeMacro | [PurgeMacro\(\)](#) | | QuitFile | [QuitFile\(\)](#) | | QuoteLine | external macro file: QUOTE.MAC | | RecordKeyMacro | [RecordKeyMacro\(\)](#) | | RedisplayViewPickList | [ViewFinds\(\)](#) | | RepeatCmd | [RepeatCmd\(\)](#) | | Replace | [Replace\(\)](#) | | ReplaceFile | [ReplaceFile\(\)](#) | | ResizeWindow | [ResizeWindow\(\)](#) | | RestoreCursorLine | [RestoreCursorLine\(\)](#) | | Right | [Right\(\)](#) | | RollDown | [RollDown\(\)](#) | +-----+-----+
+-----+-----+ | Editing Command | Native Command or Enhanced Macro | +-----*-----+ | RollLeft | [RollLeft\(\)](#) | | RollRight | [RollRight\(\)](#) | | RollUp | [RollUp\(\)](#) | | SaveAllAndExit | [SaveAllAndExit\(\)](#) | | SaveAllFiles | [SaveAllFiles\(\)](#) | | SaveAndQuitFile | [SaveAndQuitFile\(\)](#) | | SaveAs | [SaveAs\(\)](#) | | SaveBlock | [SaveBlock\(\)](#) | | SaveFile | [SaveFile\(\)](#) | | SaveKeyMacro | [SaveKeyMacro\(\)](#) | | ScrollDown | [ScrollDown\(\)](#) | | ScrollToCenter | [ScrollToCenter\(\)](#) | | ScrollToTop | [ScrollToTop\(\)](#) | | ScrollUp | [ScrollUp\(\)](#) | | Shell | [Shell\(\)](#) | | Shift | mShift() | | ShiftLeft | mShiftBlock(SHIFLEFT) | | ShiftRight | mShiftBlock(SHIFTRIGHT) | | ShowEntryScreen | [ShowEntryScreen\(\)](#) | | ShowFileInfo | external macro file: FILEINFO.MAC | | Sort | external macro file: SORT.MAC | | SpellCheck | external macro file: SPELLCHK.MAC | | SplitLine | [SplitLine\(\)](#) | | SwapLines | mSwapLines() | | TabLeft | mTabLeft() | | TabRight | mTabRight() |

ToggleAutoIndent | mToggleAutoIndent() || ToggleHexView | mToggleHexView() || ToggleInsert | [ToggleInsert\(\)](#) ||
 ToggleLineDrawing | [Toggle\(LineDrawing\)](#) || ToggleSyncScroll | external macro file: SYNCHSCR.MAC ||
 ToggleTemplateExpansion | mToggleTemplateExpand() || ToggleWordWrap | mToggleWordWrap() || UnDelete |
[UnDelete\(\)](#) || UnMarkBlock | [UnMarkBlock\(\)](#) || Up | [Up\(\)](#) || Upper | mUpper() || VWindow | [VWindow\(\)](#) || Where |
 external macro file: WHERE.MAC || WordLeft | [WordLeft\(\)](#) || WordRight | [WordRight\(\)](#) || WrapPara | mWrapPara()
 || ZoomWindow | [ZoomWindow\(\)](#) | +-----+-----+

Appendix F: Standard Configuration Settings

This appendix provides the standard settings for the configuration options (variables) in the editor. The variable name is identified on the left, with the standard setting indicated to the right of the variable name.

+-----+

|

[System/File Options](#) | +-----+ [TSEPath](#) = "" [KbdPath](#) = ""
[StartUpFlags](#) = _STARTUP_MENU_ [GUIStartUpFlags](#) = _USE_DEFAULT_WIN_POS_|
 _USE_LAST_SAVED_FONT_|_USE_LAST_SAVED_WIN_SIZE_ [SingleInstance](#) = Off [DefaultExt](#) = "ui s si c cpp
 h java pas inc bat prg txt doc html asm" [FileLocking](#) = _NONE_ [LoadWildFromDos](#) = On [LoadWildFromInside](#) = Off
[PickFileChangesDir](#) = Off [PickFileFlags](#) = _ADD_DIRS_|_ADD_SLASH_|_DIRS_AT_TOP_ [PickFileSortOrder](#) =
 "f" [EOLType](#) = 0 // 0=As Loaded, 1=^M (CR), 2=^J (LF), 3=^M^J (CR/LF) [EOType](#) = 2 // 0=nothing, 1=^Z,
 2=[EOLType](#), 3=[EOLType](#)+^Z [MakeBackups](#) = On [ProtectedSaves](#) = On [BackupExt](#) = ".bak" [BackupPath](#) = ""
[SaveState](#) = Off [UseCommonDialogs](#) = Off +-----+ | [Keyboard/](#)
[Mouse Options](#) | +-----+ [EquateEnhancedKBD](#) = On
[MouseHoldTime](#) = 9 // 18 approx 1 sec [MouseRepeatDelay](#) = 0
+-----+ | [Command/Format Options](#) |
+-----+ [MaxHistoryPerList](#) = 50 [MaxHistorySize](#) = 1000
[PersistentHistory](#) = On [MaxRecentFiles](#) = 200 [PersistentRecentFiles](#) = On [WordSet](#) = [ChrSet](#)("0-9A-Z_a-z")
[FindOptions](#) = "I" [ReplaceOptions](#) = "" [CenterFinds](#) = Off [Beep](#) = On [ChangeShellPrompt](#) = On [QuitToPrompt](#) = Off
[KeepUndoBeyondSave](#) = Off [DateFormat](#) = 4 // 1=MM/DD/YY, 2=DD/MM/YY, 3=YY/MM/DD, 4=MM/DD/
 YYYY, 5=DD/MM/YYYY, 6=YYYY/MM/DD [DateSeparator](#) = [Asc](#)(' / ') [TimeFormat](#) = 2 // 1=24 hr (HH:MM:SS),
 2=12 hr (HH:MM am/pm) [TimeSeparator](#) = [Asc](#)(' : ') +-----+ | [Text](#)
[Entry/Block Options](#) | +-----+ [Insert](#) = On [AutoIndent](#) = On
[WordWrap](#) = Off [TemplateExpansion](#) = Off [ParaEndStyle](#) = Off [ReturnEqNextLine](#) = Off [RemoveTrailingWhite](#) = On
[LineDrawType](#) = 1 // 1=+-+, 2=***, 3=*-*, 4=***, 5=ASCII, 6=Eraser, 7=User [LineDrawChar](#) = [Asc](#)(' * ') // used when
[LineDrawType](#) == 7 [InsertLineBlocksAbove](#) = Off [UseCurrLineIfNoBlock](#) = Off [UnMarkAfterPaste](#) = On
[TabShiftsBlock](#) = Off +-----+ | [Tab/Margin Options](#) |
+-----+ [TabType](#) = _SOFT_ [TabWidth](#) = 8 [CursorTabWidth](#) = 0
[ZeroBasedTab](#) = Off [VarTabs](#) = [TabSet](#)("4 8 16 24 40") [ExpandTabs](#) = On [DetabOnLoad](#) = Off [EntabOnSave](#) = Off
[LeftMargin](#) = 0 [RightMargin](#) = 72 +-----+ | [Display/Color Options](#) |
+-----+ [StatusLineAtTop](#) = On [StatusLineFillChar](#) = [Asc](#)(' ')
[ShowMainMenu](#) = On [ShowHelpLine](#) = On [HelpLineDelay](#) = 6 [CurrWinBorderType](#) = 2 // 1=+-+, 2=***, 3=*-*,
 4=***, 5=***, 6=*** 7=ASCII 8=3D [OtherWinBorderType](#) = 2 // 1=+-+, 2=***, 3=*-*, 4=***, 5=***, 6=***
 7=ASCII 8=3D [DisplayBoxed](#) = 1 // 0=Never, 1=Mouse, 2=Always [SpecialEffects](#) = _USE_3D_CHARS_|
 _DRAW_SHADOWS_ [Transparency](#) = 0 [KeepWinOnTop](#) = Off [InsertCursorSize](#) = 4 // 0=no [cursor](#), 1..8=[cursor](#) size,

9=vertical bar [OverwriteCursorSize](#) = 2 // 0=no [cursor](#), 1..8=[cursor](#) size, 9=vertical bar [ShowEOFMarker](#) = On
[ShowLineNumbers](#) = Off [StartupVideoMode](#) = _AUTO_DETECT_ WinPosLeft = 0 WinPosTop = 0
[ShowSyntaxHilite](#) = On [FontName](#) = "Courier New" [FontSize](#) = 10 // Point Size [FontFlags](#) = _NONE_
+-----+ | Attributes |
+-----+ [CurrWinBorderAttr](#) = [Color](#)(Intense Black on White)
[OtherWinBorderAttr](#) = [Color](#)(Intense Blue on White) [MenuTextAttr](#) = [Color](#)(Black on White) [MenuTextLtrAttr](#) =
[Color](#)(Red on White) [MenuGrayAttr](#) = [Color](#)(Bright Black on White) [MenuSelectGrayAttr](#) = [Color](#)(White on Blue)
[MenuBorderAttr](#) = [Color](#)(Black on White) [MenuSelectAttr](#) = [Color](#)(Bright White on Blue) [MenuSelectLtrAttr](#) = [Color](#)
(Bright Red on Blue) [BlockAttr](#) = [Color](#)(Bright White on Black) [CursorInBlockAttr](#) = [Color](#)(Black on White)
[CursorAttr](#) = [Color](#)(Intense Black on Cyan) [HiLiteAttr](#) = [Color](#)(Red on White) [TextAttr](#) = [Color](#)(Intense Black on
White) [EOFMarkerAttr](#) = [Color](#)(Black on White) [MsgAttr](#) = [Color](#)(Black on White) [StatusLineAttr](#) = [Color](#)(Bright
White on Blue) [LineNumbersAttr](#) = [Color](#)(Black on White) +-----+ |
Help Attributes | +-----+ [HelpInfoAttr](#) = [Color](#)(Bright Green on
Blue) [HelpLinkAttr](#) = [Color](#)(Bright Yellow on Blue) [HelpSelectAttr](#) = [Color](#)(Blue on White) [HelpBoldAttr](#) = [Color](#)
(Bright White on Blue) [HelpItalicsAttr](#) = [Color](#)(Bright Magenta on Blue) [HelpTextAttr](#) = [Color](#)(White on Blue)
+-----+ | SyntaxHilite Attributes |
+-----+ [IncompleteQuoteAttr](#) = [Color](#)(Intense Black on Red)
[NumberAttr](#) = [Color](#)(Intense Magenta on White) [Directive1Attr](#) = [Color](#)(Intense Red on White) [Directive2Attr](#) = [Color](#)
(Intense Red on White) [Directive3Attr](#) = [Color](#)(Intense Red on White) [MultiLnDlmt1Attr](#) = [Color](#)(Intense Green on
White) [MultiLnDlmt2Attr](#) = [Color](#)(Intense Green on White) [MultiLnDlmt3Attr](#) = [Color](#)(Intense Green on White)
[SingleLnDlmt1Attr](#) = [Color](#)(Intense Green on White) [SingleLnDlmt2Attr](#) = [Color](#)(Intense Green on White)
[SingleLnDlmt3Attr](#) = [Color](#)(Intense Green on White) [Quote1Attr](#) = [Color](#)(Intense Magenta on White) [Quote2Attr](#) =
[Color](#)(Intense Magenta on White) [Quote3Attr](#) = [Color](#)(Intense Magenta on White) [ToEOL1Attr](#) = [Color](#)(Intense Green
on White) [ToEOL2Attr](#) = [Color](#)(Intense Green on White) [ToEOL3Attr](#) = [Color](#)(Intense Green on White)
[Keywords1Attr](#) = [Color](#)(Intense Bright Blue on White) [Keywords2Attr](#) = [Color](#)(Intense Yellow on White)
[Keywords3Attr](#) = [Color](#)(Intense Bright Magenta on White) [Keywords4Attr](#) = [Color](#)(Intense Bright Red on White)
[Keywords5Attr](#) = [Color](#)(Intense Cyan on White) [Keywords6Attr](#) = [Color](#)(Intense Cyan on White) [Keywords7Attr](#) =
[Color](#)(Intense Bright Blue on White) [Keywords8Attr](#) = [Color](#)(Intense Bright Blue on White) [Keywords9Attr](#) = [Color](#)
(Intense Bright Blue on White) +-----+ | [Print Output Options](#) |
+-----+ [PrintLeftMargin](#) = 0 [PrintRightMargin](#) = 0
[PrintLinesPerPage](#) = 0 [PrintTopMargin](#) = 0 [PrintBotMargin](#) = 0 [PrintLineSpacing](#) = 1 // 1=Single, 2=Double, 3=Triple,
etc... [PrintLineNumbers](#) = Off [PrintHeader](#) = "" // F=FileName, D=Date, T=Time, P=Page [PrintFooter](#) = "" //
F=FileName, D=Date, T=Time, P=Page [PrintFirstPage](#) = 0 [PrintLastPage](#) = 0 [PrintCopies](#) = 1 [PrintPause](#) = Off
[PrintUseScreenFont](#) = On [PrinterFontName](#) = "Courier New" [PrinterFontSize](#) = 10 // Point Size [PrinterFontFlags](#) =
NONE

Appendixes

```

+----- User's Guide -----+
|                               |
|                               |

```

[Appendix A: Technical Specifications](#) || [Appendix C: Standard Key Assignments](#) || [Appendix E: Command
Implementation](#) || [Appendix F: Standard Configuration Settings](#) || +----- Macro Reference Guide -----+ ||
| [Appendix B: Command Implementation](#) || [Appendix C: Commands by Category](#) || [Appendix D: Variables by
Category](#) || +-----+

Asc

Returns the ASCII integer value for a single character.

Syntax: [INTEGER Asc](#)([STRING](#) s) - s is the single character whose ASCII value is returned. Returns: The ASCII integer value of the character passed to it. Notes: If more than a single character is passed, the first character of that [string](#) is used. **Examples:** i = [Asc](#)("a") // i is 97 i = [Asc](#)("def") // i is 100 See Also: [Chr](#)(), [Str](#)(), [Val](#)()

Ask

Allows a macro to get string input from the user.

Syntax: [INTEGER Ask](#)([STRING](#) prompt, [VAR STRING](#) reply [, [INTEGER](#) history_number]) - prompt is a string expression by which the user is prompted. - reply is a string variable where the reply is placed. - history_number is an optional integer specifying which history list to use. Returns: Non-zero if terminated by pressing <Enter>; zero ([FALSE](#)) if terminated by pressing <Escape>. Notes: Prompts the user with prompt, and stores the user input in reply. reply must be initialized in the macro before [Ask](#)() is called. The contents of reply will be displayed at the prompt, as the default initial value. The displayed value can be edited by the user, just like prompts issued by other commands (such as the [EditFile](#)() or [Find](#)() commands). If reply is set to the empty string (""), the initial value displayed will be the last history entry, if history is specified; or blank, if history is [not](#) specified (or if there is no previous history). **Examples:** [proc](#) foo() [integer](#) ok [string](#) fn[65] = "" // Initialize reply ok = [Ask](#)("File to edit:", fn) [if](#) ok and [Length](#)(fn) > 0 // if ok and [EditFile](#)(fn) // something was entered [endif](#) [end](#) Using the Boolean nature of [Ask](#)() and [Length](#)(), the following shortcut can be used: [string](#) fn[65] = "" [if](#) [Ask](#)("File to edit:", fn) [and](#) [Length](#)(fn) [EditFile](#)(fn) // If fn was entered, and its length > 0 [endif](#) // A macro to go to a line and a column... [proc](#) GotoLineColumn() [string](#) reply[20] = " [integer](#) line, p [if](#) [Ask](#)("Goto line,col:", reply) [and](#) [Length](#)(reply) line = [Val](#)(reply) //Val stops on first non-numeric [if](#) line [GotoLine](#)(line) [endif](#) p = [Pos](#)(';', reply) [if](#) p [GotoColumn](#)([Val](#)([SubStr](#)(reply, p + 1, [Length](#)(reply) - p))) [endif](#) [endif](#) [end](#) See Also: [GetKey](#)(), [Read](#)(), [AddHistoryStr](#)(), [AskFilename](#)(), [AskNumeric](#)()

AskFilename

Allows a macro to get a filename from the user, with filename completion.

Syntax: [INTEGER AskFilename](#)([STRING](#) prompt, [VAR STRING](#) fn [, [INTEGER](#) flags [, [INTEGER](#) history_number]]) - prompt is a string expression by which the user is prompted. - fn is a string variable where the reply is placed. - flags is an optional integer that can be used to modify the behavior of this command. Valid flags are: - [_FULL_PATH_](#) instructs the editor to perform [ExpandPath](#)() on fn. The results of this are returned, rather than the actual fn as entered by the user. - [_MUST_EXIST_](#) specifies that fn must be an existing file. If fn does not exist, [AskFilename](#)() will issue a warning message, and prompt again. For example, [LoadMacro](#)() calls [AskFilename](#)() with this flag, because a macro must exist on disk in order to be loaded. - [_WRITE_ACCESS_](#) specifies that if fn is an existing file, it must [not](#) be READ ONLY. For example, [SaveAs](#)() calls [AskFilename](#)() with this flag, because [SaveAs](#)() must be able to write to the selected file if it exists. Multiple flags can be specified. To do so, use the bitwise logical OR (|) operator. For example, to specify all three flags: [_FULL_PATH_](#) | [_MUST_EXIST_](#) | [_WRITE_ACCESS_](#) - history_number is an optional integer specifying which history list to use. Returns: Non-zero if terminated by pressing <Enter>; zero ([FALSE](#)) if terminated by pressing <Escape>. Notes: Prompts the user with prompt, and stores the user input in fn. You must initialize fn before [AskFilename](#)() is called. The contents of fn will be displayed at the prompt,

as the default initial value. The displayed value can be edited by the user, just like prompts issued by other commands (such as the [EditFile\(\)](#) or [Find\(\)](#) commands). If fn is set to the empty string (""), the initial value displayed will be the last history entry, [if](#) history is specified; or blank, if history is [not](#) specified (or if there is no previous history). This command supports the "[Search SubDirs](#)" (-s) option to obtain a PickList of all files in the indicated directory and all associated subdirectories, that match the filename specification. (For more information on the "[Search SubDirs](#)" option, see "[Command-Line Options](#)" in the chapter on "[Starting the Editor](#)" in [the User's Guide](#).) This command is similar to the [Ask\(\)](#) command, and in fact, calls [Ask\(\)](#) to do the actual work of gathering user input. Additionally, [AskFilename\(\)](#) allows filename completion (if <Tab> is pressed), removes leading and trailing white space from the returned fn, and can perform additional tasks depending on the value of the flag parameter. The editor uses [AskFilename\(\)](#) to prompt for files in many commands, including [InsertFile\(\)](#), [SaveAs\(\)](#), [LoadMacro\(\)](#), [ReplaceFile\(\)](#), [ChangeCurrfilename\(\)](#), and [LoadKeyMacro\(\)](#). **Examples:** `string fn[65] = "" // Initialize reply if AskFilename("File to edit:", fn) and Length(fn) EditFile(fn) // If fn was entered, and its length > 0 endif` See Also: [GetKey\(\)](#), [Read\(\)](#), [AddHistoryStr\(\)](#), [Ask\(\)](#), [ExpandPath\(\)](#)

AskNumeric

Allows a macro to get numeric string input from the user.

Syntax: `INTEGER AskNumeric(STRING prompt, VAR STRING reply [, INTEGER history_number])` - prompt is a string expression by which the user is prompted. - reply is a string variable where the reply is placed. Only numeric (0-9, +,-) input is accepted. - history_number is an optional integer specifying which history list to use. Returns: Non-zero if terminated by pressing <Enter>; zero ([FALSE](#)) if terminated by pressing <Escape>. Notes: Prompts the user with prompt, and stores the user input in reply. You must initialize reply before [AskNumeric\(\)](#) is called. The contents of reply will be displayed at the prompt, as the default initial value. The displayed value can be edited by the user, just like prompts issued by other commands (such as the [GotoLine\(\)](#) or [GotoColumn\(\)](#) commands). If reply is set to the empty string (""), the initial value displayed will be the last history entry, if history is specified; or blank, if history is [not](#) specified (or if there is no previous history). This command is similar to the [Ask\(\)](#) command; however, [AskNumeric\(\)](#) forces user input to be numeric. **Examples:** `// A macro to go to a line. proc mGotoLine() string reply[10] = " integer line if AskNumeric("Goto line:", reply) and Length(reply) line = Val(reply) if line GotoLine(line) endif endif end` See Also: [GetKey\(\)](#), [Read\(\)](#), [AddHistoryStr\(\)](#), [Ask\(\)](#), [ReadNumeric\(\)](#), [IReadNumeric\(\)](#)

Assigning Macros and Commands to Keys

To assign macros and commands to keys in the macro language, use the following format:

<

[key](#)> command where: - key may be any of the keys supported by the editor. The key is enclosed within < > characters. - command is one of the following: - The name of a native command. - The name of a procedure declared previously in the macro file (a user-written command). To determine whether a particular single key (including single-shift and multi-shift keys) is a valid assignable key, you can use the ShowKey macro, available on the Potpourri menu. (Select "[Potpourri](#)" under the Util Menu, or press <Shift F10>; then choose "ShowKey".) In ShowKey, type the desired key. If the key is assignable, a line is displayed providing information about that key, including how to identify the key name in a key assignment (indicated under the column "[Key](#) Name"). In key assignments, the key name is not case sensitive. (For example, <Shift s> and <Shift S> are equivalent, since "S" and "s" refer to the same key on the keyboard.) Note, however, that using <Shift>, <Ctrl>, or <Alt> with another [key](#) name makes that other [key](#) name

distinct from its "unshifted" state. (For example, <Shift S> and <S> are NOT equivalent.) If a key assignment is made for a macro or command that is included in a menu definition in the same macro file, the menu will automatically display the key assignment (unless the menu definition specifies otherwise, using the NoKeys option). Following are some sample key definitions, assuming procedures foo, foo1, and foo2 have been previously defined: <f1> foo() <Shift Ins> foo1() <Ctrl BackSpace> foo2() <alt F10> [ShowEntryScreen\(\)](#) A command can also be assigned to a "two-key" sequence. The format is: <key1><key2> command For example: <Ctrl Q><Ctrl F> foo() <F1><Alt B> foo1() <Ctrl k><t> [MarkWord\(\)](#) NOTE: In two-key assignments, if you use <Ctrl> with the first key of the assignment, that key state is also assumed for the second key of the assignment if the second key is one of the alphabetic characters, a through z (or A through Z), or if the second key is <Shift> plus one of the alphabetic characters. For example, <Ctrl Q><Ctrl F> is equivalent to <Ctrl Q><F>; and <Ctrl k><t> is equivalent to <Ctrl k><Ctrl t>. Also, <Ctrl k><t> is equivalent to <Ctrl k><Shift t>. However, <Ctrl k><t> is distinct from <Ctrl k><Alt t>.

Attr

Sets the display attribute to be used in subsequent video output commands.

Values: [INTEGER](#) - [0..255] The [Color\(\)](#) command may be used to generate these values. Default: Undefined Limits: Cannot be used within a config/endconfig construct. Notes: The attribute is used by the video output commands. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." Affects: [PutChar\(\)](#), [PutCharV\(\)](#), [PutCharH\(\)](#), [PutLine\(\)](#), [PutStr\(\)](#), [Write\(\)](#), [WriteLine\(\)](#), [ClrScr\(\)](#), [ClrEol\(\)](#) See Also: Commands: [Color\(\)](#), [PutAttr\(\)](#)

Augmenting the Editor Load Directory

By default, the installation routine places the editor executable program, along with various configuration and utility files used by the editor, in the editor "

[load directory](#)." Many other supplemental editor files are placed in a variety of sub-directories off the load directory. (For example, the documentation files used by [the HelpSystem](#) are placed in a \HELP subdirectory.) However, you may find it helpful or necessary to maintain the editor executable program (g32.exe (GUI version) [or](#) e32.exe (Console version)) [in](#) a separate directory from the other files in the editor load directory. In order to do this, you will need to augment what the editor views as its load directory, by extending it to an additional directory. If you do so, the extended directory (rather than the actual directory from which the editor is loaded) will then be used as the load directory in which the editor will write any necessary configuration files. This will also extend the directory the editor searches when it is attempting to locate files in the load directory: the editor will first search the extended load directory, and if the file is still not found, the editor will then search in the directory from which the editor was actually started. You can specify an extended load directory by creating a TSELOADDIR environment variable that the editor will look for on startup. This optional extension of the editor load directory addresses two potential needs: - You want to store just the editor executable program in an alternate directory on your machine, and leave all the other editor files in the editor installation directory. For example, suppose you store all of your utility programs in C:\bin (which is included in your PATH), so you want to move the editor executable program (g32.exe or e32.exe) to C:\bin. But you do not want to move all the other editor files to this directory. To do this: 1. Install the editor in c:\TSEPro (or whatever directory name you choose). 2. Move g32.exe or e32.exe, as appropriate, to c:\bin. 3. Create an environment variable called TSELOADDIR, and assign it as follows (assuming you installed the editor in c:\TSEPro): set [TSELOADDIR](#)=c:\TSEPro You would probably want to place this in AUTOEXEC.BAT under Windows 95/98, or use the Control Panel to set it under Windows NT. Now the editor will run from c:\bin, but all configuration information will be written to the appropriate files in c:\TSEPro. - You are installing the multi-user version of the

editor in a network environment, and you want to store just one copy of the editor on a read-only network drive, but still allow each user to individually configure the editor. For example, suppose N: is a read-only drive on your network, and shared utility programs are maintained in N:\TOOLS. All users have read-only access to N:\TOOLS. Accordingly, you want to store the editor in N:\TOOLS\TSEPro. To do this: 1. Run the installation program for the editor, and select N:\TOOLS\TSEPro as the installation directory. Note that N:\TOOLS\TSEPro must be included in each user's PATH. 2. Have each user create a private directory to which the user has write access, such as c:\TSEPro. Then copy the following files to this private directory: N:\TOOLS\TSEPro*.DAT N:\TOOLS\TSEPro\SYNHI*.SYN N:\TOOLS\TSEPro\SPELL\USER.LEX 3. Have each user create an environment variable called TSELOADDIR, and assign it as follows (assuming you copied the files mentioned in the previous step to c:\TSEPro): set [TSELOADDIR](#)=c:\TSEPro You would probably want to place this in AUTOEXEC.BAT under Windows 95/98, or use the Control Panel to set it under Windows NT. Now the editor will run from N:\TOOLS\TSEPro, but all configuration information will be written to the files stored in c:\TSEPro. Instead of setting the TSELOADDIR environment variable as described above, the extended load directory can alternatively be set (or overridden) when the editor is started. To do this, specify the "[Augment Load Dir](#)" (-i) command-line option at the command line on editor startup, and indicate the appropriate extended load directory (C:\TSEPro in the examples above). (For more information on specifying the "[Augment Load Dir](#)" command-line option, see "[Command-Line Options](#)" in the chapter on "[Starting the Editor](#).")

AutoIndent

Determines placement of the cursor after issuing the

[CReturn](#)() command; also determines the formatting actions taken by the [WrapPara](#)() command. [Values](#): [INTEGER](#) - [OFF](#) turns [AutoIndent OFF](#). - [ON](#) turns [AutoIndent ON](#). - [_STICKY_](#) enables [STICKY AutoIndent](#). [Default](#): [ON](#)

Notes: When AutoIndent is ON or STICKY, the editor automatically indents each new line that is created. For more information on AutoIndent, see the chapter on "[Editing Modes](#)" in the [User's Guide](#). [Affects](#): [WrapPara](#)(), [CReturn](#)()
See Also: [WordWrap](#)

Autoindent Mode

AutoIndent mode provides for automatic indentation of text, based on the effective left margin. AutoIndent affects positioning of the cursor when you execute CReturn (that is, you press <Enter>). It also has an effect on the behavior of WordWrap mode (as described in the preceding section, "

[WordWrap](#) Mode"). In addition, it affects the formatting of text when WrapPara is executed. [AutoIndent](#) has three settings: [OFF](#), [ON](#), or [STICKY](#). To change [AutoIndent mode](#) to one of these settings, select "[AutoIndent](#)" under the Options Menu, and then choose the desired setting. When AutoIndent mode is ON or STICKY, an "A" is displayed on [the StatusLine](#). [The Effective Left Margin](#) ----- [The effective left margin](#) is the left-hand column at which the cursor is positioned when you execute CReturn and when you enter text in [WordWrap mode](#). It is also the column at which text is left-justified when you reformat text using the WrapPara command. For CReturn and [WordWrap mode](#), [the effective left margin](#) is determined according to the LeftMargin setting or to the status of AutoIndent, as follows: - If AutoIndent is OFF, [the effective left margin](#) is the column specified for the LeftMargin setting (or column 1, if LeftMargin is zero). (To set LeftMargin, select "Left Margin" under the Options Menu, and enter the desired value.) - If [AutoIndent](#) is [ON](#), [the effective left margin](#) floats, based on existing text, as follows. For purposes of this discussion, a blank line is one which is empty, or contains only white (space or tab) characters. - On a non-blank line, [the effective left margin](#) is the column position of the first non-white character on that line. - On a line that is blank, but which immediately follows a line that is not blank, [the effective left margin](#) is the column position under the first non-white character of the preceding line. - On a line that is blank, and which immediately follows a line that is also blank, [the effective left margin](#) is the column specified by the [LeftMargin](#) setting (or column 1, if

[LeftMargin](#) is zero). - If [AutoIndent](#) is STICKY, [the effective left margin](#) floats just as it does when [AutoIndent](#) is ON, but with this slight variation: - On a line that is blank, and which immediately follows a line that is also blank, the editor searches backward until it finds a non-blank line. [The effective left margin](#) is then the column position under the first non-white character of the previous non-blank line. For the WrapPara command, [the effective left margin](#) is determined as explained in "[Commands to Format Text](#)" in the chapter on "[Manipulating Text](#)."

BackSpace

Deletes the character to the left of the cursor position. Shifts the cursor to the left one position, along with the text at and to the right of the

[cursor](#). [Syntax](#): [INTEGER BackSpace\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If the cursor is at column 1, no action occurs and zero is returned. To cause <Backspace> to join the current line with the previous line when the cursor is at column 1, replace the <Backspace> key definition with the following: [proc](#) mBackSpace() [if](#) ([not](#) [BackSpace\(\)](#)) [and](#) [PrevChar\(\)](#) [JoinLine\(\)](#) [endif](#) [end](#) <Backspace> mBackSpace() To prevent <Backspace> from shifting text when in overwrite mode, replace the <Backspace> key definition with the following: [proc](#) mBackSpace() [if](#) [Query](#)([Insert](#)) [BackSpace\(\)](#) [elseif](#) [Left\(\)](#) [InsertText](#)(" ", _OVERWRITE_) [Left\(\)](#) [endif](#) [end](#) <Backspace> mBackSpace() To have <Backspace> do both of the above, use the following key definition: [proc](#) mBackSpace() [if](#) [Query](#)([Insert](#)) [if](#) ([not](#) [BackSpace\(\)](#)) [and](#) [PrevChar\(\)](#) [JoinLine\(\)](#) [endif](#) [elseif](#) [Left\(\)](#) [InsertText](#)(" ", _OVERWRITE_) [Left\(\)](#) [elseif](#) [PrevChar\(\)](#) [JoinLine\(\)](#) [endif](#) [end](#) <Backspace> mBackSpace() See Also: [DelChar\(\)](#)

BackupExt

Allows you to set the name of the extension used for backup files.

[Values](#): [STRING](#) (maximum of 30 characters) The leading "." character is [not](#) necessary. [Default](#): "BAK" Notes: When [BackupPath](#) is set, [BackupExt](#) is ignored. [Affects](#): [SaveAllAndExit\(\)](#), [SaveAllFiles\(\)](#), [SaveAndQuitFile\(\)](#), [SaveFile\(\)](#) See Also: [BackupPath](#), [MakeBackups](#), [ProtectedSaves](#)

BackupPath

Determines where backup files are written.

[Values](#): [STRING](#) [Default](#): "" [Limits](#): Notes: When [BackupPath](#) is set, [BackupExt](#) is ignored. BackupPath should be set to an existing path. If the path assigned does not exist, a backup file will not be written. It is [not](#) necessary for [BackupPath](#) to be non-ambiguous. For instance, setting [BackupPath](#) to "save-here" would cause backup files to be written to the directory "save-here", relative to the current drive and directory. Setting [BackupPath](#) to "\\save-here" would cause backup files to be written to "\\save-here", relative to the current drive. [Affects](#): [BackupExt](#) Commands: [Exit\(\)](#), [QuitFile\(\)](#), [RenameAndSaveFile\(\)](#), [SaveAllAndExit\(\)](#), [SaveAllFiles\(\)](#), [SaveAndQuitFile\(\)](#), [SaveFile\(\)](#) See Also: [BackupExt](#), [MakeBackups](#), [ProtectedSaves](#) Commands: [Exit\(\)](#), [QuitFile\(\)](#), [RenameAndSaveFile\(\)](#), [SaveAllAndExit\(\)](#), [SaveAllFiles\(\)](#), [SaveAndQuitFile\(\)](#), [SaveFile\(\)](#)

Basic Concepts

+-----+

|

[Introduction](#) | | [Overview of the Editor](#) | | [Installation of the SemWare Editor Professional](#) | | [Starting the Editor](#) | | [Command Prompts And PickLists](#) | | [Editing Modes](#) | | [The Pull-Down Menu System](#) | | [The Screen Layout](#) | | [Using Tabs](#) | +-----+

Basic Find and Replace Options

Included in the editor are the Find command and the related Replace command, along with a number of versatile options. Also included are the Again and AgainReverse features, which allow you to repeat the previous Find or Replace operation.

* Find (<Ctrl F>, or "Find" under Search

[Menu](#)) This command searches for a specified string (all on one line). Unless you indicate otherwise, the search is case sensitive, and begins searching at the cursor position and moves forward to the end of the current file. When located, the cursor is placed in the text on the first character in the string, and the string is highlighted. When you execute Find, the editor displays the prompt, "Search for:". Enter the string to be located (or press <CursorUp> or <CursorDown> to pop up and select from a History box of previous search strings). The editor then issues the following prompt to obtain any options to use for the Find operation: +-----+ |Options [BGLIW]X| (Back Global Local Ignore-[case](#) Words reg-eXp):| |I| +-----+ Note in the prompt box above that the editor's standard setting for the initial Find options is "I" (Ignore-[case](#)). (For more information on how to change the initial Find options, see "Initial Find Options" under "[Command/Format Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") Multiple options can be specified. Enter the appropriate character for each desired option, as follows: [B] Backward. Search backward from the current cursor position. The search begins one position before the current cursor position. [L] Local. Limit the search to the block marked in the current file. [G] Global. Start the search at the beginning of the current file. If the [G] and [L] options are specified and a block is marked in the current file, the search starts at the beginning of the block. If the [G] and [B] options are specified, the search starts at the end of the current file. If the [G], [L], and [B] options are specified, the search starts at the end of the block in the current file. [A] All. Continue the search through all files in the Ring, beginning at the cursor position. The search continues through all files in the Ring, and stops when it returns to the current file. Text located in the current file before the cursor position is not included in the search. To include the text before the cursor position, you must also specify the [G] option; this causes the search to start at the beginning of the current file. If the [A] and [L] options are specified and a block is marked in a file other than the current file, the editor switches to the file containing the block, then starts the search at the beginning of the block. If the [A], [G], [L], and [B] options are specified, the search starts at the end of the block, regardless of the file in which the block is located. [I] Ignore-case. Ignore the case of alphabetic characters in the search [string](#). [W] Words. Search for whole words only, as determined by the WordSet variable. (For more information on the characters that comprise the WordSet, see "[WordSet Characters](#)" under "[Command/Format Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") [X] regular eXpression. Evaluate the search string according to the Regular Expression format and notation (described in the section on "[Searching with Regular Expressions](#).") For a short notation list see "[Summary List of Regular Expression Operators](#)". [C] Current line. Limit the search to the current line. If the [C] and [G] options are specified, the search starts at the beginning of the current line. [^] beginning anchor. Anchor the search string to the beginning of the line (or block). The search operation attempts to locate a matching string that begins in column 1 of a line; or, if the [L] option is also specified, it attempts to locate a matching string that begins in the first column of the block. [\$] ending anchor. Anchor the search string to the end of the line (or block). The search operation attempts to locate a matching

string that ends on the last character of a line; or, if the [L] option is specified, it attempts to locate a matching string that ends on the last character within the block or at the end of the line, whichever comes first. [+] next character. Start the search at the next character in the text (rather than at the cursor position). [V] View. Display (view) a PickList of all lines that contain the specified search string. You can then select the desired line of text from the PickList, and the editor will place you in the file on the selected line. For more information on how to use or redisplay the view PickList, see the CompressView command under "[Listing All Occurrences of a Specified Text String](#)" later in this chapter. When the [V] option is specified, the [G] option is always assumed, and the [B] or [C] options (if specified) are ignored. * Replace (<Ctrl R>, or "Replace" under Search [Menu](#)) This command searches for a string and replaces it with another string. As an extension of the Find command, Replace searches in the same manner and recognizes the same options as Find (except the [V] option), and includes additional replacement options. Unless specified [otherwise](#), Replace prompts you, each time the indicated search string is located, to indicate whether you want the replacement to be made. When you execute Replace, the editor displays the prompt "Search for:". Enter the string to be located (or press <CursorUp> or <CursorDown> to pop up and select from a History box of previous search strings entered during the current session). The editor then issues the prompt "Replace with:". Enter the replacement string. The following prompt is then issued to obtain any options to use for the search and the subsequent Replace operation:

```
+-----+ |Options [BGLIWNX] (Back Global Local Ignore-case
Words No-promp reg-eXp):| | +-----+ (Note in the prompt
box above that the editor's standard setting does not specify an initial Replace option. For more information on how to
change the initial Replace options, see "Initial Replace Options" under "Command/Format Options" in the chapter on
"Configuring the Editor Interactively." ) Enter the appropriate character for each desired option, as described above for
the Find command, including the following additional Replace options: [N] No prompting. Do not prompt for
replacement confirmation at each occurrence of the indicated search string; that is, replace all occurrences of the string
in the designated text without prompting. [#] number of times, where "#" is an actual number. Enter the number of
occurrences of the search string to replace, without being prompted for replacement confirmation. For example, if you
enter "8", the first eight occurrences of the string are automatically replaced. When Replace locates a string in the text
that matches the search string (and the [N] or [#] options ARE NOT specified), the text string is highlighted, and the
prompt, "Replace (Yes/No/Only/Rest/Quit):", is displayed. Press the letter for the desired response, as follows: [Y]
Yes. Make the replacement, then continue to the next occurrence of the search string. [N] No. Do not make the
replacement, but continue to the next occurrence of the search string. [O] Only. Make the replacement only to this
occurrence, then terminate the Replace operation. [R] Rest. Make the replacement for this occurrence, then replace the
rest of the occurrences of the search string in the designated text WITHOUT further prompting. [Q] Quit. (Or press
<Escape>) Do not make the replacement, and quit the Replace operation. Once the Replace operation stops, the
number of replacements that were made is displayed on the StatusLine with the message, "# change(s) made.", where
"#" is the actual number of replacements. * Again (<Ctrl L>, or "Again" under Search Menu) This command repeats
the last search operation (that is, the last execution of a search command such as Find, Replace, FindWordAtCursor, or
IncrementalSearch). It begins searching one position after the current cursor position if a forward search had
previously been performed, or one position before the current cursor position if a backward search had previously been
performed. All search or Replace options other than [G] (Global) that were in effect for the previous execution of the
operation, are in effect for Again. * AgainReverse (<CtrlShift L>) This command is a variation of the Again command
that allows you to repeat the last search operation, but in the opposite direction. If the previous search was a forward-
seeking search, AgainReverse would search backwards from the current position; if the previous search was a
backward-seeking search, AgainReverse would search forward from the current position.
```

Beep

Determines if the editor will "sound" the PC speaker when

[Find\(\)](#) or [Replace\(\)](#) operations fail. [Values](#): BOOLEAN [Default](#): [ON](#) Notes: This setting does not affect the [Alarm\(\)](#) command. [Affects](#): [Find\(\)](#), [Replace\(\)](#), [RepeatFind\(\)](#) See Also: Commands: [Find\(\)](#), [Replace\(\)](#), [RepeatFind\(\)](#)

BegFile

Moves the cursor to line 1, column 1, of the current file.

Syntax: [INTEGER BegFile\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Zero is returned if the cursor position does not change. For example, if the cursor is already at the beginning of the first line of the file, zero is returned. This command is equivalent to the following: [GotoLine](#)(1) [BegLine](#)() **Examples:** In order to maintain the current cursor column when a column block is open in the current file, use the following macro instead of the [BegFile](#)() command: [proc](#) mBegFile() [integer](#) col = [CurrCol](#)() [BegFile](#)() [if](#) [Query](#)([Marking](#)) [and](#) [isBlockInCurrFile](#)() == [_COLUMN_](#) [GotoColumn](#)(col) [endif](#) [end](#) <Ctrl PgUp> mBegFile() See Also: [EndFile](#)()

BegLine

Moves the cursor to column 1 of the current line.

Syntax: [INTEGER BegLine\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If the cursor is already at the beginning of the current line, zero is returned. A common alternative for a "BegLine" key, is to have the cursor jump between the beginning of the line and the first non-white character on the line: [proc](#) mHome() [if](#) [not](#) [BegLine](#)() [GotoPos](#)([PosFirstNonWhite](#)()) [endif](#) [end](#) <Home> mHome() See Also: [BegLineTog](#)(), [EndLine](#)(), [EndLineTog](#)()

BegLineTog

Toggles the cursor position between the column 1 and the first non-blank character on the current line.

Syntax: [BegLineTog](#)() Returns: Nothing. Notes: If the cursor is already positioned at the first non-blank character on the current line, the cursor moves to the beginning of the current line, otherwise the cursor moves to the first non-blank character on the current line. See Also: [BegLine](#)(), [EndLine](#)(), [EndLineTog](#)()

BegWindow

Moves the cursor to the top line of the current window. The cursor column is

[not](#) changed. **Syntax:** [INTEGER BegWindow\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If the cursor is already at the top line of the current window, zero is returned. See Also: [EndWindow](#)()

BegWord

Positions the cursor at the beginning of the current word.

Syntax: [INTEGER BegWord\(\)](#) Returns: Zero ([FALSE](#)) if the cursor was not on a word; otherwise, non-zero. Notes: The editor uses the WordSet variable to determine what constitutes a word. See Also: [EndWord](#)(), [WordRight](#)(), [WordLeft](#)() **Variables:** [WordSet](#)

BinaryMode

Determines whether the current file is in Binary mode.

Syntax: [INTEGER BinaryMode](#)([INTEGER](#) mode_type) - mode_type is an optional new Binary mode type to use. Supported [values](#) for mode_type are: - 0 (zero) - effectively takes the current file out of Binary mode. - 1 (or any value through the maximum line length) - places the current file in Binary mode. If the file is saved in this mode, end-of-line characters will [NOT](#) be added to each line. (Use this mode with caution. Normally the editor places end-of-line character(s) at the end of each line when a file is saved. If a file is in this mode when saved, the only end-of-line characters in the file will be those manually input to the file by the user.) - -1 - places the current file in Binary mode. If the file is saved in this mode, it will be written with a length byte preceding each line. - -2 - places the current file in Binary mode. If the file is saved in this mode, it will be written with a length word (2 bytes) preceding each line. Returns: Zero (FALSE) if the current file is not in Binary mode; otherwise, the Binary mode_type value in effect when this command is invoked. Notes: In Binary mode, the tab character is not displayed as spaces; instead, the actual tab character is displayed. A file can also be placed in [Binary mode](#) when it is initially loaded, by including the -b switch ahead of the filename. A mode_type value can be specified by placing the appropriate value immediately following -b. - If a mode_type value from 1 to the maximum line length is specified, this value will be the fixed length at which each line is displayed on the screen. - If a mode_type value of -1 or -2 is specified, each line will be displayed according to the value of its length byte (if mode_type is -1) [or](#) its length word (if mode_type is -2). When a file is LOADED in Binary mode, no character translation takes place, as normally occurs with the end-of-line characters and the tab character. **Examples:** // To set Binary mode for the current file: [BinaryMode](#)(80) // To turn off Binary mode for the current file: [BinaryMode](#)(0) // To test whether the current file is in Binary mode: [if BinaryMode](#)() [Message](#)("File is in [Binary mode](#)") [endif](#) See Also: [EditFile](#)(), [EditThisFile](#)(), [DisplayMode](#)(), [AddFileToRing](#)()

Block Operations

During an editing session, you can manipulate a limited portion of a file by selecting, or "

[marking](#)," a block of text. A block can consist of entire or partial lines, or columns of text, or even a single word or character. A block can be copied, moved, deleted, printed, or written to disk. You can also limit certain operations to text within a block. In addition, you can copy or move ("cut") a block of text to a temporary area, called a ClipBoard. Text placed in a ClipBoard can later be retrieved and pasted back into the original file or a different file. This chapter describes methods available for: - [Marking and manipulating a block of text](#): - [Opening and closing a block](#) - [Extending a block](#) - [Block-marking commands](#) (MarkColumn, MarkLine, MarkStream, MarkChar, MarkWord, MarkToEOL, MarkParagraph) - [Unmarking a block](#) (UnMarkBlock) - [Manipulating text in a block](#) (CopyBlock, OverlayColumnBlock, DelBlock, FillBlock, MoveBlock, SaveBlock, Shift, ShiftLeft, ShiftRight) - [Changing the case of characters in a block](#) (Upper, Lower, Flip, Capitalize) - [Moving to the beginning or end of a block](#) (GotoBlockBegin, GotoBlockEnd) - [Manipulating blocks via ClipBoards](#) (Copy, CopyAppend, Cut, CutAppend, Paste, PasteOver, Empty, CopyToWinClip, CutToWinClip, PasteFromWinClip) Other chapters describe how to use the mouse for block operations, and how to use blocks to expand or restrict the scope of certain commands. The chapter on "[Mousing Around the Editor](#)" explains how to use the mouse to mark blocks, and how to access the Mouse menu to execute many block-related and ClipBoard-related commands. The chapter on "[Manipulating Text](#)" discusses how to limit or extend the scope of the WrapPara, CenterLine, and QuoteLine commands by [marking a block](#). The chapter on "[Search Features: Finding and Replacing Text](#)" presents the "Local" option for Find and Replace operations, which allows you to restrict a Find or Replace to the text in a block. The chapter on "[Checking Your Spelling](#)" describes how to limit SpellCheck to the words in a block. The chapter on "[Printing from the Editor](#)"

explains how to print a portion of a file limited to the contents of a block marked in that file. The chapter on "[Special Commands and Features](#)" describes how to Sort lines in a file based on a sort key defined by a block marked in the text.

BlockBegCol

The first column of a marked block.

Values: [INTEGER](#) **Default:** Undefined **Limits:** Can only be queried. Cannot be used within a config/endconfig construct. **Notes:** For line blocks, 1 is returned. For column blocks, the starting column of the block is returned. For character blocks, the starting position of the block is returned. The position is the same as the column unless hard tab characters occur before the position. See Also: [BlockEndCol](#) **Commands:** [CurrCol\(\)](#), [CurrPos\(\)](#)

BlockBegLine

The first line of a marked block.

Values: [INTEGER](#) **Default:** Undefined **Limits:** Can only be queried. Cannot be used within a config/endconfig construct. See Also: [BlockEndLine](#)

BlockEndCol

The ending column of a marked block.

Values: [INTEGER](#) **Default:** Undefined **Limits:** Can only be queried. Cannot be used within a config/endconfig construct. **Notes:** For line blocks, the maximum line length of the editor ([MAXLINELEN](#)) is returned. For column blocks, the ending column of the block is returned. For character blocks, the ending position of the block is returned. The position is the same as the column unless hard tab characters occur before the position. See Also: [BlockBegCol](#) **Commands:** [CurrCol\(\)](#), [CurrPos\(\)](#)

BlockEndLine

The ending line of a marked block.

Values: [INTEGER](#) **Default:** Undefined **Limits:** Can only be queried. Cannot be used within a config/endconfig construct. See Also: [BlockBegLine](#)

BlockId

The id of the file which contains the marked block.

Values: [INTEGER](#) **Default:** Undefined **Limits:** Can only be queried. Cannot be used within a config/endconfig construct. **Notes:** If no blocks exist, [BlockId](#) is zero. See Also: **Commands:** [GetBufferId\(\)](#), [GotoBufferId\(\)](#)

Bookmark Commands

The editor provides a command to place (or "set") a Bookmark, and a command to return to a specific Bookmark location.

* PlaceMark (<Ctrl 2>, or "Place Bookmark" under Search

[Menu](#)) This command allows you to place a Bookmark at a location in a file. To place a Bookmark, you position the cursor at the desired location in the file, then execute this command. The prompt, "Set Bookmark [A..Z]:", is displayed on [the StatusLine](#). Press a letter from A through Z. (The case of the letter does not matter.) * GotoMark (<Ctrl 6>, or "Go to Bookmark" under Search [Menu](#)) This command allows you to return to a particular Bookmark. When you execute this command, the prompt, "Go to Bookmark [A..Z]:", is displayed on [the StatusLine](#). Press the appropriate letter from A through Z that identifies that Bookmark. (The case of the letter does not matter.) The file containing that Bookmark becomes the current file, and you are positioned at the location where that Bookmark was placed.

Bookmark Shortcut

For added convenience, a shortcut method for placing or returning to Bookmarks is provided. The editor allows a Bookmark command (PlaceMark or GotoMark), along with an identifying letter, to be assigned to a specific key (or key combination). Then you can simply press the appropriate key assignment to place or return to a particular Bookmark, without being prompted to enter the identifying Bookmark letter.

The editor is configured with predefined

[key](#) assignments for placing and returning to ten different Bookmarks. For these predefined Bookmarks, the editor uses the last ten alphabetic characters, Q through Z, and numbers them from 0 through 9, respectively. (Keep this in mind if you choose to use the two different Bookmark methods in the same editing session. To eliminate conflicts between the letters used in the two methods, do not use letters Q through Z when placing Bookmarks directly with the PlaceMark command.) To place one of the predefined Bookmarks, press <Ctrl K><#>, where "#" is a number from 0 through 9 (located above the alphabetic keys, NOT on the numeric keypad). To return to one of these Bookmarks, press <Ctrl Q><#>, where "#" is the same number as the one selected to place the Bookmark. Of course, you can create your own predefined Bookmark [key](#) assignments. (For more information on configuring commands and their [key](#) assignments, see "[Customization Using SAL Compiled Macros](#)" in the chapter on "[Fully Customizing the Editor](#).")

Bookmarks

While reading this User's Guide, occasionally you may have referred from one chapter to another, and then returned to that previous chapter. In doing so, you may simply have remembered the page number to which to return, or you may have placed a bookmark at that page.

Similarly, during an editing session, you may wish to move from your current position in one file, to another position in that file or to a different file, and then return to your first file position. You can do so by remembering the file and the line number to which you wish to return. Or, as with a book, you can place a bookmark to mark that position.

This chapter provides the following information about the editor's Bookmark feature:

-

[Using Bookmarks](#) - [Bookmark commands](#) (PlaceMark, GotoMark) - [Bookmark shortcut](#) The Book macro, an additional Bookmark feature available on the Potpourri menu, provides a convenient way to set, go to, or list the currently-set [Bookmarks](#). Information [about](#) this macro is provided under "[Executing Supplemental Macros from the Potpourri](#)" in the chapter on "[Special Commands and Features](#)."

Break

Allows pressing <Ctrl Break> to terminate the execution of a running macro.

Values: BOOLEAN **Default:** [OFF](#) **Limits:** Cannot be used within a config/endconfig construct. **Notes:** USE WITH CAUTION: Use this only when you are writing/debugging your macros. "Breaking" from a running macro can leave the editor in an unstable state since the normal "housecleaning" is bypassed. Once a macro runs to completion, break is automatically set OFF. Break only has effect within a running macro. It has no effect in normal editing mode.

BreakHookChain

Ends the hook chain for the current event handler.

Syntax: [BreakHookChain\(\)](#) **Returns:** Nothing. **Notes:** [BreakHookChain\(\)](#) only has an effect when called from within a hooked event. Use it when you do [not](#) want any following hooks to be processed. The actual editor event is [not](#) affected, and proceeds normally, except for the calling of additional hooks. **Examples:** /
 ***** Prevent any _ON_CHANGING_FILES_ hooks from being run [while](#) we are busy. *****/ [proc](#)
 StopOnChangingFilesHook() [BreakHookChain\(\)](#) [end proc](#) foo() [Hook](#)(_ON_CHANGING_FILES_,
 StopOnChangingFilesHook) // Code goes here... [UnHook](#)(StopOnChangingFilesHook) [end](#) See Also: [Hook\(\)](#), [UnHook](#)
 (), [ExecHook\(\)](#), [SetHookState\(\)](#), [TerminateEvent\(\)](#)

Breaking Out of Run-Away Macros

As you begin testing

[macros](#) that you are developing, you may find errors in your logic that result in unexpected or undesirable results. In some cases, the macro may even run "out of control." To give you tighter control over a macro during the testing and debugging phases, the editor provides the Break variable. Its intended use is to allow you to regain control of an errant macro during development. To enable the Break option within a macro, place the following statement at the beginning of your macro: [Set\(Break, ON\)](#) This allows you to interrupt the macro, at any time while it is executing, by pressing the <Ctrl Break> key. When you press <Ctrl Break> on a macro with Break enabled, the following selection menu pops up: + Macro interrupted... + |*Terminate*Macro*****| Continue | +-----+ Select "Terminate Macro" to immediately terminate the macro; select "Continue" if you want the macro to resume processing. When a macro is terminated this way, normal "clean-up" routines are bypassed. As a result, editor variables and editing modes may not have been properly restored, and you may notice erratic or unexpected behavior by the editor. To restore

normal operation, you should exit and reload the editor. Because of the unpredictable effects of interrupting a macro while it is running, the Break feature should only be enabled while you are developing [macros](#). Once you have completed testing and debugging your macro, you should remove the Break statement from the macro.

BrowseMode

Sets or queries the browse status of the current buffer.

Syntax: [INTEGER BrowseMode](#)([INTEGER](#) status) - status is the optional browse status (TRUE or FALSE) to which to set the buffer. Returns: The current setting of the browse status of the current buffer (before being set by status, if specified). Notes: [Buffers](#) that are marked as browse cannot be saved. **Examples:** // determine the browse status of the current buffer [integer](#) browse_only browse_only = [BrowseMode](#)() // Set the current buffer to browse-only [BrowseMode](#) ([TRUE](#)) // Set browse-only OFF for the current buffer [BrowseMode](#)([FALSE](#)) // Toggle the current buffer's browse-only status [BrowseMode](#)([not BrowseMode](#)())

BufferFlags

Contains information about the current buffer, as a bit-mapped flag.

Values: [INTEGER](#) - [_READONLY_LOCKING_](#) indicates the file is Read-Only locked. - [_HANDLE_LOCKING_](#) indicates the file is handle locked. - [_CANT_LOCK_](#) indicates the file could not be locked. - [_NEWFILE_](#) indicates the file is new (that is, it does not exist on disk). - [_LOADED_](#) indicates the file is loaded into the editor. - [_PARTIAL_FILE_](#) indicates the entire file could not be loaded into the editor. **Default:** Undefined Limits: Can only be queried. Cannot be used within a config/endconfig construct. See Also: Commands: [AddFileToRing](#)(), [CreateBuffer](#)(), [EditBuffer](#)(), [EditFile](#)(), [EditThisFile](#)(), [LoadBuffer](#)()

Buffers

Buffers are the memory areas in which the editor does its work. Buffers are where files are stored for user editing. When a file is in memory, it is also called a buffer. Not surprisingly, the editor provides a rich assortment of commands for manipulating buffers through the macro language.

[Manipulating Text in Buffers](#) ----- Many commands can be used in macros to manipulate text. These include the block and ClipBoard commands, such as [CopyBlock](#)(), [MoveBlock](#)(), [DelBlock](#)(), [Cut](#)(), [Copy](#)(), and [Paste](#)(); the deletion commands, such as [DelChar](#)(), [DelLine](#)(), and [DelToEol](#)(); and other text-changing commands, such as [DupLine](#)(), [InsertFile](#)(), [JoinLine](#)(), [SplitLine](#)(), [Upper](#)(), [Lower](#)(), [Flip](#)(), and [WrapPara](#)(). [Retrieving Text from Buffers](#) Many times, it is useful to get text from the current buffer into a SAL string in order to further manipulate it. The editor provides several commands for doing this, as described below. In the examples that follow, assume the existence of the following SAL [variables](#): [string](#) c[1], s[80] * [CurrChar](#)() Because [CurrChar](#)() retrieves the character at the cursor position, it is very common to use [CurrChar](#)() in a macro to examine the character the cursor is on. For example, to see if the cursor character is "!", use the following: [if CurrChar](#)() == [Asc](#)('!') Note the use of [Asc](#)() with [CurrChar](#)(). This is necessary because [CurrChar](#)() returns an integer (as illustrated in the next example). Another common usage of [CurrChar](#)() is to see if the cursor is past the text on the current line. For example:

[if CurrChar\(\)](#) < 0 // if CurrChar < 0, past text on current line As mentioned previously, [CurrChar\(\)](#) returns the current character represented as an integer. If, at the time of the call to [CurrChar\(\)](#), the current position is beyond the last character on the current line, [CurrChar\(\)](#) returns a negative number to indicate this condition. To distinguish between the position that is one column beyond the last character on the line (which is considered the "[end-of-line](#)" position), and a position more than one column beyond the last character on the line, the following [constants](#) are provided: [_AT_EOL_](#), [_BEYOND_EOL_](#) For example, to determine if the cursor is one column past the last character of the line (that is, at the end-of-line position), use the following: [if CurrChar\(\) == _AT_EOL_](#) Another use of [CurrChar\(\)](#) is to copy the current character into a string, such as the following: `c = Chr\(CurrChar\(\)\) * GetText\(\)` The [GetText\(\)](#) command can be used to copy one or more characters from the current line into a SAL string. For example: `s = GetText(1, sizeof\(s\))` This example copies text, starting at position 1 on the current line, into the string s. The text is copied until either the end of line is reached, or the string s has no more room to accept new characters. A note about the first parameter of [GetText\(\)](#): This is the value of the character position in the current line where text retrieval begins. This value represents the number of actual characters from the beginning of the line to the current character. The value of this "position" is not always the same as the number of the cursor column at which the character is displayed on the screen; specifically, the two are different when hard tab characters exist in the current line, tab expansion is ON, and the TabWidth is greater than 1. To get the current "position" within the line (that is, the number of the character position from the beginning of the line), use the [CurrPos\(\)](#) command. This is in contrast to [CurrCol\(\)](#), which returns the current [cursor](#) column. To start copying text at the current position, you can use the following: `s = GetText\(CurrPos\)(SizeOf\(s\)) * GetMarkedText\(\)` The [GetMarkedText\(\)](#) command is handy for copying a marked block into a SAL string. [GetMarkedText\(\)](#) is commonly used in a sequence as follows: // Position to the text you want to work with [PushBlock\(\)](#) // do [not](#) disturb pre-existing blocks [MarkWord\(\)](#) // mark the word at the cursor `s = GetMarkedText\(\)` // retrieve it into a string [PopBlock\(\)](#) // restore block state If the length of s is smaller than the length of the marked text, the stored text will be truncated to the length of s. [Storing Text in Buffers](#) Just as the editor provides commands to get text from buffers, there are also commands to put text back in [buffers](#), as described below. * [AddLine\(\)](#) and [InsertLine\(\)](#) The basic formats of these commands are (respectively): [AddLine](#)([[STRING](#) st [, [INTEGER](#) buffer_id]]) [InsertLine](#)([[STRING](#) st [, [INTEGER](#) buffer_id]]) These commands place a new line in the file, containing the text passed in st. [AddLine\(\)](#) places the new line after the current line, while [InsertLine\(\)](#) places the new line before the current line. In either case, the newly-created line becomes the new current line. The string st is optional. If st is omitted, an empty line is placed in the file. If the optional buffer_id is passed, the text is placed in the file with that buffer id. * [InsertText\(\)](#) The basic format of this command is: [InsertText](#)([STRING](#) st [,flag]) [InsertText\(\)](#) is used to place text into the buffer, at the current position. If flag is not specified, the current value of the Insert variable is used to determine whether the text is inserted, or whether it overwrites text at the current cursor position. To control how text is input, the constants [_INSERT_](#) and [_OVERWRITE_](#) can be used. For example, to insert the string "Beam me up, Scotty!" at the current position: [InsertText](#)("Beam me up, Scotty!", [_INSERT_](#)) The text is inserted at the current position regardless of the setting of [Insert](#). * [InsertData\(\)](#) The basic format of this command is: [InsertData](#)(data_name) This command inserts a set of data (defined by the datadef statement) into the current buffer. (For more information, see "[Datadef](#)" later in this chapter.) The following commands are available especially for use in [macros](#). * [EmptyBuffer\(\)](#) The basic format of this command is: [EmptyBuffer](#)([[INTEGER](#) buffer_id]) This command removes all the text associated with the current or specified buffer. It is especially useful when re-using a work buffer, and the work buffer needs to be empty. * [ShiftText\(\)](#) The basic format of this command is: [ShiftText](#)([INTEGER](#) shift_amount) This command shifts the text on the current line by the number of positions indicated as shift_amount. The text shifts either right or left, depending on the sign of shift_amount: if positive, text is shifted right; if negative, text is shifted left. At times it may be useful to place a SAL string into the editor's Clipboard. The way to do so may [not](#) be obvious at first; but once understood, it can be applied in many ways. The following illustrates this method. /

```
***** Macro to place a string in the Clipboard. The
current contents of the Clipboard are lost. Returns TRUE if successful, FALSE if not.
***** / integer proc String2Clipboard(string s) if
CreateTempBuffer\(\) MarkChar\(\) InsertText(s) //Insert the string, it will be marked Copy\(\) //Copy it to the Clipboard
```


[AbandonFile\(\)](#) //Remove the temp buffer [return\(TRUE\)](#) [endif](#) [return\(FALSE\)](#) [end](#) This procedure could be called as follows: [String2Clipboard\("Make it so..."\)](#) Now, the next time [Paste\(\)](#) is executed, "Make it so..." will be pasted into the text at the current cursor position. Some general notes regarding the above procedure: - When putting text in [ClipBoards](#) or [Named ClipBoards](#), always use the [Cut\(\)](#) or [Copy\(\)](#) commands. These commands set internal information, making it possible for the [Paste\(\)](#) command to work properly. - In [String2Clipboard\(\)](#), a character block is used. Other block types (line or column blocks) can also be used, if desired. - [String2Clipboard\(\)](#) takes advantage of the ring-like structure in which buffers are kept. Since new buffers are added immediately after the current buffer, and abandoning the current buffer makes the previous buffer the new current buffer, there is no need to save the current buffer Id and [return](#) back to it, as it is handled automatically in this [case](#). [Creating Buffers](#) ----- There are several types of [buffers](#), which can be created in several ways. The buffer types [include](#): - [_NORMAL_](#), which is used for an interactively-edited file. For example, buffers (files) created with the [EditFile\(\)](#) command are [_NORMAL_ buffers](#). - [_HIDDEN_](#), which is very similar to [_NORMAL_](#), except this type of buffer will not show up in buffer lists, and is skipped over by the [NextFile\(\)](#) and [PrevFile\(\)](#) commands. This type of buffer is used as a work buffer. For example, the [PickList](#) of loaded macros (optionally displayed by the [ExecMacro\(\)](#) command) is stored in a [_HIDDEN_](#) buffer. - [_SYSTEM_](#), which is very different from a [_NORMAL_](#) or [_HIDDEN_](#) buffer. This should only be used if a system buffer is truly needed, and should never be edited interactively. A [_SYSTEM_](#) buffer, like a [_HIDDEN_](#) buffer, is not easily accessible and is used as a work buffer. However, [_SYSTEM_](#) buffers handle text in a raw fashion, so that tabs are not expanded and trailing spaces are [not](#) removed. The [buffers](#) that hold [ClipBoards](#) and histories are created as [_SYSTEM_ buffers](#). There are a number of ways to create buffers, as follows: - The easiest, and probably most familiar way to create a buffer, is with the [EditFile\(\)](#) command. [EditFile\(\)](#) creates a [_NORMAL_](#) buffer. The buffer is associated with a file on disk, and can be readily saved and loaded. - [CreateBuffer\(\)](#) can be used to create a buffer of any type. Buffers created by [CreateBuffer\(\)](#) are not associated with a file on disk. - [CreateTempBuffer\(\)](#) can be used to create a temporary [_SYSTEM_](#) buffer. As with [CreateBuffer\(\)](#), these buffers are not associated with a file on disk. In fact, these buffers are not given a name when they are created. Since these are [_SYSTEM_ buffers](#), these [buffers](#) should never be used for interactive editing. The current buffer type can be queried and changed, by using the [BufferType\(\)](#) command. For example, to query the buffer type, use: if [BufferType\(\)](#) == [_SYSTEM_](#) // This is a system buffer ... [endif](#) and to set the buffer type, use: [BufferType\(_HIDDEN_\)](#) // Make this buffer hidden [Referencing and Switching to Buffers](#) ----- [_NORMAL_](#) buffers can be switched to via the [NextFile\(\)](#), [PrevFile\(\)](#), and [EditFile\(\)](#) commands. The method for switching [to](#) [_HIDDEN_](#) or [_SYSTEM_ buffers](#) is quite different. Each buffer has associated with it a buffer Id. These Ids are used to switch to [_HIDDEN_](#) or [_SYSTEM_ buffers](#). [CreateBuffer\(\)](#) and [CreateTempBuffer\(\)](#) [return](#) a buffer Id, and [GetBufferId\(\)](#) can be used to get the Id of the current or a specified buffer. These Ids can be used in conjunction with the [GotoBufferId\(\)](#) command to change to the specified buffer. Note that unlike the [NextFile\(\)](#), [PrevFile\(\)](#) and [EditFile\(\)](#) commands, [GotoBufferId\(\)](#) does not invoke any [event macros](#). (For more information on [event macros](#), see "[Event Macros](#)" later in this chapter.)

BufferType

Determines the current editing buffer "type."

Syntax: [INTEGER BufferType](#)([[INTEGER](#) type]) - type is an optional integer that represents the new buffer type that is set. Supported [values](#) for type are: - [_NORMAL_](#) causes the buffer to have no special attributes associated with it. [Buffers](#) created with [EditFile\(\)](#) have this buffer type. - [_HIDDEN_](#) simply causes the buffer to not show up in the list of files, and makes the buffer inaccessible via the [NextFile\(\)](#) and [PrevFile\(\)](#) commands. The [PickList](#) of loaded [macros](#) (optionally displayed by the [ExecMacro\(\)](#) command) is stored in a [_HIDDEN_](#) buffer. - [_SYSTEM_](#) causes the buffer to be hidden, and places the buffer in a raw mode, so that trailing spaces are never removed, and tabs are treated as a normal character. Buffers created with [CreateTempBuffer\(\)](#) have this buffer type. It is strongly recommended that

system [buffers not](#) be used for interactive editing. Undo/Redo information is [NOT](#) recorded for `_SYSTEM_ buffers`. Returns: The buffer type of the current buffer. Notes: The return value of [BufferType\(\)](#) will be the current editing buffer "type" which will be `_NORMAL_`, `_SYSTEM_`, or `_HIDDEN_`. (For more information on buffer types, see the description of [CreateBuffer\(\)](#).) Examples: // Determine the current buffer type `integer type = BufferType()` // Set the current buffer type to `_HIDDEN_` `BufferType(_HIDDEN_)` // Save the current buffer type, and set it to `_HIDDEN_` `integer save_type save_type = BufferType(_HIDDEN_)` See Also: [CreateBuffer\(\)](#), [CreateTempBuffer\(\)](#), [GotoBufferId\(\)](#), [GetBufferId\(\)](#), [EditFile\(\)](#), [EditThisFile\(\)](#), [NextFile\(\)](#), [PrevFile\(\)](#)

BufferVideo

Routes all video output to the editor's virtual screen.

This command is intended for ADVANCED USAGE.

Syntax: [BufferVideo\(\)](#) Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the editing buffer. It is used to temporarily "buffer" informational text that is to be displayed on the screen (in order to improve screen update performance). When the [BufferVideo\(\)](#) command is executed, all subsequent video output commands operate on the editor's virtual screen (rather than the visible screen). This continues until the [UnBufferVideo\(\)](#) command is executed. To determine when video output is to be buffered (rather than displayed), an internal "buffer mode" counter is maintained. Normally, this counter is set to zero, causing video output to be visibly displayed on the screen. Each time the [BufferVideo\(\)](#) command executes, the buffer mode counter is incremented by 1. Each time the [UnBufferVideo\(\)](#) command executes, the buffer mode counter is decremented by 1. While the buffer mode counter is greater than zero, video output is buffered. To restore normal "displayed" video output, the buffer mode counter must be returned to zero. Thus, each call to [BufferVideo\(\)](#) should be followed, at the appropriate time, by a corresponding call to [UnBufferVideo\(\)](#). If [BufferVideo\(\)](#) is called multiple times without intervening corresponding calls to [UnBufferVideo\(\)](#), pending screen output will not be displayed until the same number of calls are made to [UnBufferVideo\(\)](#). This behavior makes it possible for [macros to](#) call other macros without disturbing the current overriding state of video output buffering. For example: - Macro A makes a call to [BufferVideo\(\)](#) to buffer video output. - Macro A then calls Macro B to perform some standard routine. - Macro B makes a call to [BufferVideo\(\)](#) to buffer video output. Before terminating (and returning to the calling macro, Macro A), Macro B makes a call to [UnBufferVideo\(\)](#). - Macro A resumes execution, with video output still buffered, even though Macro B called [UnBufferVideo\(\)](#). - When ready to update the display screen, Macro A calls [UnBufferVideo\(\)](#). All text in the editor's virtual screen (as modified by both Macro A and Macro B) is written to the visible screen, and normal display is restored. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." Examples: `integer i BufferVideo()` // turn off screen updating `for i = 1 to 10 VGotoxy(1, i) PutStr("This is a test") endfor UnBufferVideo()` // turn on screen updating, causing // buffered text to appear See Also: [UnBufferVideo\(\)](#)

BuildPickBufferEx

Generates a "PickBuffer" list of the files in a specified directory, using selected attributes. The PickBuffer list is created by adding file entries to the current buffer, at the current position.

This command is intended for ADVANCED USAGE.

Syntax: [INTEGER BuildPickBufferEx](#)([STRING](#) filename, [INTEGER](#) attributes [, [INTEGER](#) flags]) - filename is the filename specification used to build the list of files. filename should contain wildcards. For example: "*.*" "d:\work*. *" "*.c" "b*.doc" - attributes are the attributes to use when searching. Supported [values](#) for attributes are: - [_NORMAL_](#) specifies files that can be read from or written to. - [_READONLY_](#) specifies files that can be read from but [not](#) written to. - [_HIDDEN_](#) specifies files that are hidden and do [not](#) appear in file directory listings. - [_SYSTEM_](#) specifies files that are system files. - [_VOLUME_](#) specifies the volume label entry. - [_DIRECTORY_](#) specifies directory entries. - [_ARCHIVE_](#) specifies files that are marked for archiving. - flags is optional. Supported [values](#) for flags are: - [_INSERT_PATH_](#) causes [BuildPickBufferEx](#)() to insert the entire path, rather than just the name of the file. - [_SEARCH_SUBDIRS_](#) causes [BuildPickBufferEx](#)() to also search any subdirectories that are under the current search path. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: When this command is executed, the current buffer itself must be a [_SYSTEM_](#) buffer (as created or changed by the [CreateBuffer](#)(), [CreateTempBuffer](#)()), or [BufferType](#)() commands). If you specify any combination of [_SYSTEM_](#), [_HIDDEN_](#), or [_DIRECTORY_](#), this command returns normal files in addition to files with the specified attribute(s). If you want to work only with files with the specified attribute(s), you must examine the specific attribute for each file, via the [FFAttribute](#)() command. The PickFileFlags variable can affect the output of this command: - If [_ADD_SLASH_](#) is specified, slashes are inserted at the beginning of each directory name. - If [_ADD_DIRS_](#) is specified, and the [_DIRECTORY_](#) attribute is specified for [BuildPickBufferEx](#)(), all directories in the specified path will be included. The first position of each entry in the generated PickBuffer list of files is reserved for user use, and can be set as desired. Typically, it is used as a tag indicator, as in the standard PickList used by the built-in file-editing commands (such as [EditFile](#)() and [InsertFile](#)()). The PB-type commands, such as [PBName](#)() and [PBDate](#)(), can be used to extract information from the file entries in a PickBuffer created via this command (for manipulation in a macro). To display a PickBuffer on the screen, use the [DisplayMode](#)() command. **Examples:** // build a list of all the files in the current directory [integer](#) rc rc = [BuildPickBufferEx](#)("*.*", [_READONLY_](#) | [_HIDDEN_](#) | [_SYSTEM_](#) | [_DIRECTORY_](#)) // build list of all INI files in C:\WINDOWS subdirectory: rc = [BuildPickBufferEx](#)("C:\WINDOWS*.INI", [_NORMAL_](#)) // build a list of all BAT files on the C: drive: rc = [BuildPickBufferEx](#)("c:*.bat", [_NORMAL_](#), [_SEARCH_SUBDIRS_](#) | [_INSERT_PATH_](#)) See Also: [DisplayMode](#)(), [PickFile](#)(), [PBAttribute](#)(), [PBName](#)(), [PBSize](#)(), [PBDate](#)(), [PBDateStr](#)(), [PBTime](#)(), [PBTimeStr](#)(), [AddFFInfoToBuffer](#)() **Variables:** [PickFileFlags](#)

CenterFinds

Centers the "found" text (vertically) within the current editing window.

Values: BOOLEAN **Default:** [OFF](#) Notes: When CenterFinds is ON, the current cursor line that contains any "found" text due to the use of [Find](#)(), [Replace](#)(), or [RepeatFind](#)(), will be made the center row of the current window. If set OFF, the current cursor line position within the display does not change due to text being "found". **Affects:** [Find](#)(), [Replace](#)(), [RepeatFind](#)() See Also: Commands: [HiLiteFoundText](#)(), [MarkFoundText](#)()

ChainCmd

Invokes the command (if any) previously assigned to the keystroke currently being processed.

Syntax: [ChainCmd](#)() Returns: Nothing. Notes: [ChainCmd](#)() is used when you want to pass control to a command that may have been previously assigned to the overridden key assignment. **Examples:** The following macro illustrates a common use for this command. It is assumed that mWordCompletion() is part of a larger external macro that is loaded by the user. (The macros mLookupWord and mInsertReplacement are [not](#) specified here, but are used for example

purposes.) If mWordCompletion decides that it should [not](#) do anything (for whatever reason), it passes control to a possible previous assignment of the <Tab> key by using [ChainCmd\(\)](#). [proc](#) mWordCompletion() [if](#) mLookupWord() mInsertReplacement() [else](#) [ChainCmd\(\)](#) [endif](#) [end](#) <Tab> mWordCompletion() See Also: [GetKey\(\)](#), [PressKey\(\)](#)
[Variables:](#) [Key](#)

ChangeCurrFilename

Changes the name of the current file to a new name.

Syntax: [INTEGER](#) [ChangeCurrFilename](#)([[STRING](#) new_name [, [INTEGER](#) flags]]) - new_name is an optional string expression that becomes the current file's new name. - flags is an optional integer that affects the operation of this command. Supported [values](#) for flags are: - [_DONT_PROMPT_](#) instructs the editor to suppress warning or informational messages that may be issued for any problems that occur when this command is executed. - [_DONT_EXPAND_](#) prevents the editor from performing [ExpandPath\(\)](#) on new_name; but rather accepts the name as is. - [_OVERWRITE_](#) instructs the editor to change the name of the file even if a file by that name exists on disk. Multiple flags can be specified using the bitwise logical OR (|) operator. For example: [_DONT_PROMPT_](#) | [_OVERWRITE_](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If new_name is not passed, [ChangeCurrFilename\(\)](#) prompts for the name to use with "New filename:". If the drive and/or path are not specified, the current drive and/or path are used. If [ChangeCurrFilename\(\)](#) successfully changes the file's name, the file is marked as modified. If the filename supplied by the user already exists in the editor's Ring of files, the following error message will be displayed: "Filename in use Press <Escape>". The user will then be allowed to specify a new name. If the filename supplied already exists on disk (and [_OVERWRITE_](#) is not specified), the user will receive the prompt: + Overlay existing file? + |*Yes*****| | No | | Cancel | +-----+ If "Yes" is selected, the file will be overlaid the next time a save command is issued on the current file. If "No" or "Cancel" is selected, the user will be allowed to specify a new filename. If <Escape> is pressed in response to the "New filename:" prompt, no action occurs and zero is returned. If flags was specified as [_DONT_PROMPT_](#), no prompt is displayed for the aforementioned situations, and the command immediately returns [FALSE](#). This command includes a history list of previous entries passed to it. See [AddHistoryStr\(\)](#) for additional information. See Also: [SaveAs\(\)](#), [RenameDiskFile\(\)](#), [ReplaceFile\(\)](#), [NewFile\(\)](#), [AddHistoryStr\(\)](#) [Variables:](#) [UseCommonDialogs](#)

ChangedFilesExist

Determines if there are any changed buffers in the editor's Ring of files.

Syntax: [INTEGER](#) [ChangedFilesExist](#)() Returns: Non-zero if there are any changed [buffers](#) loaded; [otherwise](#), zero ([FALSE](#)). Notes: Only buffers with a buffer type of [_NORMAL_](#) (typically, buffers used to hold files) are checked. Buffers with a buffer type of [_HIDDEN_](#) or [_SYSTEM_](#) are excluded from this command. (For more information on buffer types, see the [BufferType\(\)](#) command.) See Also: [FileChanged\(\)](#), [BufferType\(\)](#)

ChangeShellPrompt

Causes the editor to add "TSE-Shell" to your command-line prompt whenever you shell out of the editor, serving as a reminder that you are in an editor shell.

Values: BOOLEAN **Default:** [ON](#) **Limits:** Has effect at startup only. Notes: If you shell out of the editor, you should set ChangeShellPrompt ON. This will serve as a reminder that you are shelled out of the editor. When set ON, the editor will add "TSE-Shell" to your prompt.

Changing Position in a File

Moving through a file can be accomplished as simply as one character or line at a time, or as broadly as going from one end of the file to another - with many other choices in between. The commands in this section move the cursor to a new position in the text. The text is scrolled only if the text at the new position is not already displayed in the current window.

[Changing Position by One Character or Line](#) ----- These commands allow you to move the cursor by one character or line at a time. * CReturn (<Enter>) This command moves the cursor down one line, and places the cursor on the next line at [the effective left margin](#). If Insert mode is ON, the current line is split at the cursor position. (For more information on the effects of Insert mode on this command and for a discussion of the effective left margin, see "[Insert Mode](#)" and "[AutoIndent Mode](#)," respectively, in the chapter on "[Editing Modes](#).") When the cursor reaches the bottom edge of the window, the text scrolls up. If you execute CReturn on the last line of the file, a new line is added at the end of the file, and the cursor moves to that line. +----- Tip -----+ | When the configuration option, "CReturn Equals Next Line" is ON, | | Insert mode has no effect on the CReturn command. That is, CReturn | | always behaves as if Insert mode is OFF, and causes the cursor to | | go to the next line without splitting the current line. (For more | | information on this option, see "[Text Entry/Block Options](#)" in the | | chapter on "[Configuring the Editor Interactively](#).") | +-----+ * Down (<CursorDown>) This command moves the cursor down in the file by one line. The column position remains unchanged. When the cursor reaches the bottom edge of the window, the text scrolls up. The cursor stops once it reaches the last line in the file. * Up (<CursorUp>) This command moves the cursor up in the file by one line. The column position remains unchanged. When the cursor reaches the top edge of the window, the text scrolls down. The cursor stops once it reaches the first line in the file. * Left (<CursorLeft>) This command moves the cursor one column to the left on the current line. If the text was scrolled to the left, the Left command causes the text to scroll right when the cursor reaches the left edge of the window. The cursor stops when it reaches column 1. (For a similar command, see PrevChar below.) * Right (<CursorRight>) This command moves the cursor one column to the right on the current line. When the cursor reaches the right edge of the window, the text scrolls left. If Right is executed beyond the last character on the line, the cursor continues moving right on the same line. The cursor stops once it reaches the maximum line length. (For a similar command, see NextChar below.) * NextChar Like the Right command, this command moves the cursor one column to the right on the current line. If the text on the line extends beyond the right edge of the window, NextChar causes the text to scroll left. However, if NextChar is executed after the last character on the current line, the cursor advances (or wraps) to the beginning of the line below. * PrevChar Like the Left command, this command moves the cursor one column to the left on the current line. If the text was scrolled left, PrevChar causes the text to scroll right when the cursor reaches the left edge of the window. However, if PrevChar is executed at column 1, the cursor moves (or wraps) up to the end of the text on the preceding line. +----- Tip -----+ | The Left and Right commands do not wrap the cursor at the beginning | | and end of the line. If you would like the <CursorLeft> and | | <CursorRight> keys to wrap at the beginning and end of a line, | | reassign those keys to the PrevChar and NextChar commands, | | respectively. (For more information on reconfiguring keys, see | | "[Adding or Changing Key Assignments](#)" in the chapter on | | "[Fully Customizing the Editor](#).") | +-----+ [Changing Position by Multiple Characters or Lines](#) ----- These commands allow you to move the cursor by multiple characters or lines. * BegLine (<Home>) This command moves the cursor to the beginning of the current line, scrolling the text as

needed. The cursor is positioned at column 1. * EndLine (<End>) This command moves the cursor to the end of the current line, scrolling the text as needed. The cursor is positioned one column past the last character on the line.

+----- Tip -----+ | The last character on a line may or may not be the last white | | character, depending on the setting of the variable | | RemoveTrailingWhite. If set ON (which is the standard setting), | | the last character is the last non-white character on a line. If | | set OFF, the last character is the last non-white or white (space | | or tab) character at the end of a line. (For more information, see | | "Remove Trailing Whitespace" under "[Text Entry/Block Options](#)" | | in the chapter on "[Configuring the Editor Interactively](#).") |

+-----+ * BegFile (<Ctrl PgUp>) This command moves the cursor to the beginning of the current file. The cursor is positioned on the first line, at column 1; or at the same column position if a column block has been opened and not yet closed. * EndFile (<Ctrl PgDn>) This command moves the cursor to the end of the current file. The cursor is positioned on the last line, one column past the last character; or at the same column position if a column block has been opened and not yet closed. * BegWindow (<Ctrl Home>) This command moves the cursor to the first line of text in the current window. The column position remains unchanged. * EndWindow (<Ctrl End>) This command moves the cursor to the last line of text in the current window. The column position remains unchanged. * GotoColumn (<Ctrl C>, or "Go to Column" under Search [Menu](#)) This command moves the cursor to the specified column on the current line. When you execute GotoColumn, the editor displays the prompt, "Go to column:". Type the appropriate column number and press <Enter>. The cursor then moves to the column number specified. You can designate a column position relative to the current column by preceding the column number with "+" to move to the right of the current column, or with "-" to move to the left of the current column. For example, enter "+20" at the "Go to column:" prompt to move the cursor 20 columns to the right of its current position. * GotoLine (<Ctrl J>, or "Go to Line" under Search [Menu](#)) This command scrolls the text of the current file so that the specified line becomes the current line. The cursor does not change its position relative to the screen. When you execute GotoLine, the editor displays the prompt, "Go to line:". Type the appropriate line number and press <Enter>. The text is scrolled to make the specified line the current line. You can designate a line relative to the current line by preceding the line number with "+" to specify a line after the current line, or with "-" to specify a line before the current line. For example, enter "-50" at the "Go to Line:" prompt to scroll to the line that is 50 lines before the current line. * PrevPosition (<Ctrl Q P>) This command moves the cursor to the line and column at which it was previously positioned, immediately prior to moving to the current line. For example, suppose you are in the middle of a file and you issue the BegFile command to go to the beginning of the file. After viewing the text at the beginning, you then wish to return to your previous position in the middle. Provided you have not moved the cursor off the beginning line of the file (where it was initially placed by BegFile), you can then execute PrevPosition and the cursor will immediately move to the previous line. (Changing the column position on the new line does not affect the behavior of PrevPosition.) * WordLeft (<Ctrl CursorLeft>) This command moves the cursor to the first character of the word that begins to the left of the cursor on the current line. If you execute this command from the first character of the first word on the current line, the cursor moves to the end of the previous line. (For more information on how the editor defines a word, see "[WordSet Characters](#)" under "[Command/Format Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") * WordRight (<Ctrl CursorRight>) This command moves the cursor to the first character of the word that begins to the right of the cursor on the current line. If you execute this command on the last word of the current line, the cursor moves to the end of the line. If you execute WordRight beyond the last word, the cursor moves to the first character of the first word on the line below. (For more information on how the editor defines a word, see "[WordSet Characters](#)" under "[Command/Format Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") * BegPara This command moves the cursor to the first line of the current paragraph (or current function/procedure), scrolling the text as needed. The column position remains unchanged. If the beginning of the paragraph cannot be determined, the cursor does not move. A "paragraph," as defined by this command, is based on the file type (extension) of the current file: - For certain programming-language file types, a paragraph is a function or procedure. Supported file types [include](#) SAL (.s, .ui), C (.c, .h), C++ (.cpp, .hpp), Pascal (.pas), and xBase (.prg, .spr, .mpr, .qpr, .fmt, .frg, .lbg, .ch). - For non-programming-language file types (or other non-supported programming-language file types), a paragraph is any contiguous group of non-blank lines. * EndPara This command moves the cursor to the last line of the current paragraph (or current function/procedure), scrolling the text as needed. The column position remains unchanged. If the end of the paragraph cannot be determined, the cursor does not move. A "paragraph", as defined by this command, is based on the file type (extension) of the current file: - For certain programming-language file types, a

paragraph is a function or procedure. Supported file types [include](#) SAL (.s, .ui), C (.c, .h), C++ (.cpp, .hpp), Pascal (.pas), and xBase (.prg, .spr, .mpr, .qpr, .fmt, .frg, .lbg, .ch). - For non-programming-language file types (or other non-supported programming-language file types), a paragraph is any contiguous group of non-blank lines. * NextPara (<Shift PgDn>) This command moves the cursor to the first line of the next paragraph (or next function/procedure), scrolling the text as needed. The column position remains unchanged. If the next paragraph cannot be determined, the cursor does [not](#) move. A "paragraph", as defined by this command, is based on the file type (extension) of the current file: - For certain programming-language file types, a paragraph is a function or procedure. Supported file types [include](#) SAL (.s, .ui), C (.c, .h), C++ (.cpp, .hpp), Pascal (.pas), and xBase (.prg, .spr, .mpr, .qpr, .fmt, .frg, .lbg, .ch). - For non-programming-language file types (or other non-supported programming-language file types), a paragraph is any contiguous group of non-blank lines. * PrevPara (<Shift PgUp>) This command moves the cursor to the first line of the current paragraph (or the current function/procedure); or to the first line of the previous paragraph (or previous function/procedure), if the cursor is already on the first line of the paragraph. The text is scrolled as needed. The column position remains unchanged. If the previous paragraph cannot be determined, the cursor does not move. A "paragraph", as defined by this command, is based on the file type (extension) of the current file: - For certain programming-language file types, a paragraph is a function or procedure. Supported file types [include](#) SAL (.s, .ui), C (.c, .h), C++ (.cpp, .hpp), Pascal (.pas), and xBase (.prg, .spr, .mpr, .qpr, .fmt, .frg, .lbg, .ch). - For non-programming-language file types (or other non-supported programming-language file types), a paragraph is any contiguous group of non-blank lines.

Changing Window Size and Orientation

When two or more windows are opened, you can adjust the size of any of the windows. The minimum size for a window is four lines by fifteen columns. When you change the size of one window, compensating changes are made to the

[window](#)(s) adjacent to the side you adjust. The following commands allow you to change the size of the current window. * ResizeWindow (<Ctrl O R>, or "Resize" under Window [Menu](#)) This command allows you to change the size of the current window by moving an interior window edge. When you execute this command, the editor displays the following StatusLine prompt to request the window edge you want to move: Select window edge to resize (press [cursor key](#)): Press the cursor key (left, right, up, or down) that points toward the appropriate edge. The edge you select must be an interior edge; that is, it must be an edge that is adjacent to another window. Once you select the edge to resize, a resizing bar appears and the editor displays the following [StatusLine prompt](#): Press [cursor](#) keys to resize, <Escape> to cancel, <Enter> when done: Press the cursor keys to move the resizing bar either up and down (if you selected the top or bottom window edge), or left and right (if you selected the left or right window edge). Note that the minimum window size indicated earlier may affect the edge you wish to adjust. Press <Enter> to complete the resizing operation, or press <Escape> to cancel the operation and keep the window size as it was before you executed this command. Note that you can resize a particular window edge only if its length is either the same as the length of the adjacent window edge, or is the same as the total length of all edges adjacent to the window edge being resized. For example, in the screen illustrations presented earlier in this chapter, you can resize the lower edge of window 1; however, you cannot resize the upper edge of window 2 or the upper edge of window 3. You can also resize the shared inside vertical edge of windows 2 and 3. (There are no other edges in those illustrations that can be resized.) * ZoomWindow (<Ctrl O Z>, or "Zoom" under Window [Menu](#)) This command expands the current window so that it covers the entire screen and temporarily hides all other windows. A full border is placed around the editing window. ZoomWindow leaves the other windows intact, though hidden from view. To restore all previous windows to the screen, execute the ZoomWindow command again. While a window is zoomed on the screen, the window number that is displayed on the title line is temporarily replaced with a "Z". Once the previous windows are restored to the screen, the appropriate window number is also restored. (If you are [using a mouse](#), the "Z" is not displayed.) When exactly two windows are opened, you can use the ToggleHorizVertical command to change the window orientation from horizontal to vertical, or from vertical to horizontal. * ToggleHorizVertical ("Toggle Horiz/Vertical" under Window [Menu](#)) This command toggles the screen between horizontal and vertical windows, if exactly two windows are open. If

the screen is split into two vertical windows, the windows will be changed to horizontal windows; if the screen is split into two horizontal windows, the [windows](#) will be changed to vertical [windows](#). This command does not do anything if there is only one window on the screen, nor if there are more than two [windows](#).

ChDir

Changes to the indicated directory.

Syntax: [INTEGER ChDir](#)([STRING](#) dir) - dir is the name of the directory to change to. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If the directory specification contains a drive, and it is different than the current drive, the current drive and directory are not changed. To change the current drive (and directory), use the [LogDrive\(\)](#) command. The trailing slash is accepted but [not](#) required (unless you are changing to the root directory). **Examples:** // Make \foo the current directory, on the current drive [ChDir](#)("\\foo") // Make \plans the current directory on the c: drive. // The current drive does [not](#) change. [ChDir](#)("c:\\plans") // Make c:\\plans the current drive and directory [LogDrive](#)("c") [ChDir](#)("\\plans") See Also: [GetDir\(\)](#), [CurrDir\(\)](#), [LogDrive\(\)](#)

CheckDefaultExt

Checks an extension against the default file extension list (as specified by the DefaultExt variable).

Syntax: [INTEGER CheckDefaultExt](#)([STRING](#) ext) - ext is the file extension to check. The leading period is optional. Returns: Non-zero if the passed extension is in the default file extension list; [otherwise](#), zero ([FALSE](#)). **Examples:** [if CheckDefaultExt](#)("bat") [Message](#)("bat is a default extension") [else Message](#)("bat is not a default extension") [endif](#) See Also: [Variables: DefaultExt](#)

Checking Your Spelling

To assist in verifying and correcting the spelling in your files, the editor includes a built-in spelling checker, SpellCheck. When invoked, SpellCheck examines the words in the current file (or in the indicated portion of the file) and attempts to match each word against the entries in the available SpellCheck word

[list](#)(s). If no match for a particular word is found, SpellCheck gives you several options for dealing with that word, including options to modify the word, ignore the word, or add the word to your own supplementary user-defined word list. This chapter provides the following information on SpellCheck: - [Running SpellCheck](#) - [The user-defined word list](#) - [Files needed by SpellCheck](#)

ChooseFont

Returns information about a font selected from the standard ChooseFont dialog box.

This command has no effect in the Console version of the editor.

Syntax: [INTEGER ChooseFont](#)([VAR STRING fontname](#), [VAR INTEGER](#) pointsize, [VAR INTEGER](#) flags) - fontname is the name of the selected font. Examples are "Terminal" [and](#) "Courier". - pointsize identifies the font size. - flags can be any combination of [_FONT_BOLD_](#) and/or [_FONT_OEM_](#) or just 0. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command displays a dialog box, allowing the user to select a particular font. Once selected, the information about the font is returned in the appropriate variables. The selected font is not used by the editor. Information about the font is simply returned. In order for the editor to use this font, the [SetFont\(\)](#) command would need to be called, passing the appropriate information. Contrast this command with the [GetFont\(\)](#) command, which returns information about the font that is being used for the editing display. **Examples:** [string fontname](#)[255] [integer](#) pointsize, flags [if ChooseFont](#)(fontname, pointsize, flags) ... // at this point [fontname](#), pointsize, and flags are set to // describe the selected font. See Also: [GetFont\(\)](#), [SetFont\(\)](#), [GetCharWidthHeight\(\)](#), [ResizeFont\(\)](#)

Chr

Returns the single-character string equivalent of the passed integer.

Syntax: [STRING Chr](#)([INTEGER](#) i) - i is the integer for which the string equivalent is to be returned. Valid [values](#) for i are 0..255 inclusive. Since [Chr\(\)](#) is evaluated at compile time, if i is a constant value, and outside of the valid range, a compiler syntax error will be issued when this statement is compiled. Returns: The single character string value of the passed integer. **Examples:** Assume the existence of a string variable, s: s = [Chr](#)(97) // s is "a" In the next example, if the current character in the file (the character at the cursor position) is B, then s is "B": s = [Chr](#)([CurrChar](#)()) See Also: [Asc\(\)](#), [Val\(\)](#), [Str\(\)](#)

ChrSet

Used to create the string to be assigned to the WordSet variable.

Syntax: [STRING ChrSet](#)([STRING](#) s) - s is a string that contains the characters to be considered as part of the set. The user may specify a range of characters by placing a dash ("-") between the first and last characters of the range. Returns: A bit-encoded 32-character string of the set of characters supplied in s. Notes: This command is used solely in conjunction with the WordSet variable. The slash ("/") character is used as an escape character. In order to include a slash in the set, use two slashes ("\\/"). The \nnnn notation or \nnn notation, where nnn is a decimal number, and nn is a hexadecimal number, is allowed in specifications. To include the dash in the set, place it as either the first or last character. The "~" character is used as "[not](#) in this set," if it occurs as the first character. To set the WordSet variable within a config/endconfig construct, you must use the following format: [WordSet](#) = [ChrSet](#)("A-Za-z") **Examples:** In the following example procedure, the WordSet is saved to a local variable at the beginning, then restored to the original value at the end: [proc](#) foo() [string](#) saved_wordset[32] // must be 32 chars // save current WordSet and set the new one saved_wordset = [Set](#)([WordSet](#), [ChrSet](#)("a-z")) // user code to do something goes here // then restore the original WordSet [Set](#)([WordSet](#), saved_wordset) [end](#) // make [WordSet](#) everything but a-z [Set](#)([WordSet](#), [ChrSet](#)("~a-z")) // make [WordSet](#) all characters [Set](#)([WordSet](#), [ChrSet](#)("\\d000-\\d255")) See Also: [WordRight\(\)](#), [WordLeft\(\)](#), [MarkWord\(\)](#), [DelLeftWord\(\)](#), [DelRightWord\(\)](#) **Variables:** [WordSet](#)

ClearBit

Sets the specified bit to 0.

This command is intended for ADVANCED USAGE.

Syntax: [ClearBit](#)([STRING](#) bitset, [INTEGER](#) bit) - bitset is the bit-mapped string in which to clear a particular bit. - bit is the specific bit number to be cleared. Returns: Nothing. Notes: The editor uses bit-mapped strings for the WordSet variable and the [TabSet](#)() command. [SetBit](#)(), [ClearBit](#)(), and [GetBit](#)() can be used to modify these [values](#). Bit numbering starts at 0. Warning: No checks are done to verify that bit is valid. For example, given a string that is 10 characters long means that there are 80 bits available (8 bits per character/byte * 10 = 80, bits 0 - 79). Specifying a value greater than 79 would be out of bounds for this [string](#), but this boundary condition is [not](#) checked! **Examples:** Suppose you want to remove the "_" character from the WordSet without changing the other WordSet characters. Since the WordSet variable is stored as a bit string (256 bits, stored in 32 bytes), the [ClearBit](#)() command is just right for the job: [string](#) word_set[32] word_set = [Query](#)(WordSet) // get the WordSet [ClearBit](#)(word_set, [ASC](#)('_')) // clear appropriate bit [Set](#)(WordSet, word_set) // and reset it See the WordSet variable for additional information. See Also: [SetBit](#)(), [GetBit](#)(), [TabSet](#)() **Variables:** [WordSet](#)

ClearBufferDaTmAttr

Clears the date, time, size and attribute fields for the current buffer.

Syntax: [ClearBufferDaTmAttr](#)() Returns: Nothing. Notes: For each file loaded, the editor maintains a record of the file's date, time, size, and file attributes, as of the time the file was loaded into the editor, or the last time it was saved. [ClearBufferDaTmAttr](#)() will set these fields to zero. When a file is edited that does not have an associated file on disk, this command is invoked. When the [SaveFile](#)() command is executed, the editor uses this file information to determine if the file on disk has been changed by another process. See Also: [UpdateBufferDaTmAttr](#)(), [GetBufferDaTmAttr](#)()

ClearUndoRedoList

Clears all Undo/Redo information associated with the current file.

Syntax: [ClearUndoRedoList](#)() Returns: Nothing. See Also: [Undo](#)(), [Redo](#)(), [SetUndoOn](#)(), [SetUndoOff](#)(), [UndoCount](#)(), [RedoCount](#)(), [UndoMode](#)() **Variables:** [KeepUndoBeyondSave](#)

ClipboardId

Determines the id of the Clipboard used by

[Cut](#)(), [Copy](#)() and [Paste](#)() commands. **Values:** [INTEGER](#) **Default:** Undefined Limits: Cannot be used within a config/endconfig construct. Notes: ClipboardId can be used to set up and control a user-defined Clipboard, without affecting the system Clipboard. To use ClipboardId in a macro to set up the user's own Clipboard (so that macros do not change the contents of the default Clipboard), first retrieve and store the default Clipboard id so that it can later be restored at the end of the macro. **Examples:** // To retrieve the current ClipboardId: id = [Query](#)(ClipboardId) // To set the current ClipboardId to foo_id: [Set](#)(ClipboardId, foo_id) // To retrieve the current ClipboardId, and set it to foo_id: id = [Set](#)(ClipboardId, foo_id) The following procedure will save the id of the current Clipboard, change the Clipboard to another buffer, then restore the Clipboard at the end of the procedure. [integer proc](#) DoWork() [integer](#) original = [Set](#)(ClipboardId, [CreateTempBuffer](#)()) [if](#) original == [Query](#)(ClipboardId) [return](#) (FALSE) [endif](#) DoSomething() //Do

something [Set\(ClipBoardId, original\) //restore ClipBoard return\(TRUE\) end Affects: Cut\(\), Copy\(\), Paste\(\)](#) See Also: Commands: [CreateBuffer\(\)](#), [CreateTempBuffer\(\)](#)

ClosePrint

Closes the print device, if opened as a file.

Syntax: [ClosePrint\(\)](#) Returns: Nothing. Notes: The PrintDevice can be set to a filename, allowing print operations to be routed to a file. The file will not be opened until the first [PrintChar\(\)](#) command is invoked, but will remain open so that subsequent [PrintChar\(\)](#) commands will print to the same file. In order to edit, view, or otherwise manipulate the file, it must first be closed. Use the [ClosePrint\(\)](#) command to close the file. Once the file has been closed, a subsequent call to the [PrintChar\(\)](#) command (assuming that the PrintDevice has not been changed) will reopen the file and all data previously in the file will be lost. The commands [PrintFile\(\)](#) and [PrintBlock\(\)](#) call [ClosePrint\(\)](#) upon completion. See Also: [PrintChar\(\)](#) **Variables:** PrintDevice

CloseWindow

Closes the current editing window.

Syntax: [INTEGER CloseWindow\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If only one window is open or the current window is zoomed, [CloseWindow\(\)](#) returns zero. Adjacent editing windows grow to fill the space of the closed window. The cursor is placed in the window that occupies the space the closed window was in. See Also: [HWindow\(\)](#), [VWindow\(\)](#), [OneWindow\(\)](#), [DelWindow\(\)](#)

Closing Editing Windows

Any

[windows](#) that you no longer need can be closed, and thereby removed from the screen. The following commands allow you to close windows. With the first group of commands, the file that was displayed in the window remains open after the window is removed. * [CloseWindow](#) (<Ctrl O C>, or "Close" under Window [Menu](#)) This command closes the current window. The current window disappears, and an adjacent [window\(s\)](#) expands to fill the vacated space. The cursor is placed in an adjacent window, making it the new current window. * [DelWindow](#) (<Ctrl O D>) This command closes (or deletes) a window adjacent to the current window. When you execute this command with more than two windows on the screen, the editor displays the following StatusLine prompt to request the window to close: Delete window (press [cursor key](#)): Press the cursor key (left, right, up, or down) that points toward the appropriate window. A [window\(s\)](#) adjacent to the deleted window expands to fill the vacated space. When you execute this command with only two windows on the screen, the editor automatically closes the non-current window, and the current window expands to fill the entire screen. In this case, [DelWindow](#) behaves the same as [OneWindow](#), which is described below. * [OneWindow](#) (<Ctrl O O>, or "One" under Window [Menu](#)) This command closes all windows except the current window. The current window then fills the entire screen. Do not confuse the [OneWindow](#) command with [ZoomWindow](#). While both commands cause the current window to fill the screen, [OneWindow](#) CLOSES all other [windows](#). [ZoomWindow](#) only temporarily HIDES all other [windows](#). The following command can be used to close the current window, and also close the file in that window. * [CloseFileAndWindow](#) ("Close File & Window" under Window [Menu](#)) This command prompts you to save the file in the current window (if modified), and then closes (or "quits") the file. (For more information on closing a file, see the QuitFile command in the chapter on "[Loading](#)."

[Processing, and Saving Files.](#)") The current window is then closed (if there are multiple windows on the screen). That window disappears, and an adjacent [window](#)(s) expands to fill the vacated space. The cursor is placed in an adjacent window, making it the new current window.

ClrEol

Blanks the display from the current video output position to the end of the current line of the video output window.

Syntax: [ClrEol\(\)](#) Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. Make sure the video output position has been positioned on the screen (via [GotoXY\(\)](#) or [VGotoXY\(\)](#)) before calling this command. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [ClrScr\(\)](#), [GotoXY\(\)](#), [VGotoXY\(\)](#), [VGotoXYAbs\(\)](#), [VHomeCursor\(\)](#)

ClrScr

Blanks the current video output display window.

Syntax: [ClrScr\(\)](#) Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [ClrEol\(\)](#), [VGotoXY\(\)](#), [VGotoXYAbs\(\)](#), [GotoXY\(\)](#), [VHomeCursor\(\)](#)

CmpiStr

Compares two strings in a case-insensitive manner.

Syntax: [INTEGER CmpiStr](#)([STRING](#) s1, [STRING](#) s2) - s1 is the first string to be compared. - s2 is the second string to be compared Returns: 0 if s1 is equal s2, [otherwise](#), less than 0 if s1 is lexically less than s2, or greater than zero if s1 is lexically greater than s2. Notes: This command is provided as an adjunct to the normal string comparison operator, "==" . The "==" compares strings in a [case](#) sensitive manner. [CmpiStr\(\)](#) makes it easy to compare two [strings](#) for equality, regardless of their [case](#). When to use [CmpiStr\(\)](#) vs. [EquiStr\(\)](#) If you want to know if two [strings](#) are equal, ignoring differences in [case](#), use [EquiStr\(\)](#). If you want to know the lexical ordering of two strings (if one is less than or greater than the other), use [CmpiStr\(\)](#). **Examples:** [if CmpiStr](#)(s1, s2) == 0 [Message](#)("s2 and s2 are the same") [elseif CmpiStr](#)(s1, s2) <> 0 [Message](#)("s1 is different than s2") [if CmpiStr](#)(s1, s2) > 0 [Message](#)("s1 is > s2") [elseif CmpiStr](#)(s1, s2) < 0 [Message](#)("s1 is < s2") [endif endif](#) See Also: [EquiStr\(\)](#)

Color

Returns the integer value of the requested color combination.

Syntax: [INTEGER Color](#)([INTENSE] [BRIGHT] foreground [[ON](#) background]) - Valid foreground and background

colors are Black, Blue, Green, Cyan, Red, Magenta, Yellow, White. - The optional keywords, INTENSE and BRIGHT, allow you to select an intense color (INTENSE) or a bright color (BRIGHT). Returns: The integer value of the requested color combination. Notes: Because of PC hardware design, pre-defined colors may not always be what the user expects. White is actually grey. To get white, use "BRIGHT White". [Examples](#): The following will set the current video attribute to a white foreground on a blue background. [Set\(Attr, Color\(BRIGHT White ON Blue\)\)](#) See Also: [PutAttr\(\)](#) [Variables](#): [Attr](#)

COLOR Set

The standard set of

[color variables](#). [Values](#): [INTEGER](#) - [0..255] [Limits](#): None. See Also: Commands: [Color\(\)](#) Following are the color variables, including a description and the default value of each. [Text Color Variables](#) -----
[CursorAttr](#) Text on the cursor line. [Default](#): 176 [or Color](#)(Intense Black on Cyan) [TextAttr](#) Text that is not on the cursor line nor in a marked block. [Default](#): 240 [or Color](#)(Intense Black on White) [HiLiteAttr](#) Highlighted text located by a search command. [Default](#): 116 [or Color](#)(Red on White) [BlockAttr](#) Text within a marked block. [Default](#): 15 [or Color](#)(Bright White on Black) [CursorInBlockAttr](#) Text on the cursor line within a block. [Default](#): 112 [or Color](#)(Black on White) [Window Border Variables](#) ----- [CurrWinBorderAttr](#) Border for the current window. [Default](#): 240 [or Color](#)(Intense Black on White) [OtherWinBorderAttr](#) Border for non-current [windows](#). [Default](#): 241 [or Color](#)(Intense Blue on White) [Menu/HelpLine Color Variables](#) ----- [MenuTextAttr](#) Enabled selection items in a menu, and command/option names in a HelpLine. [Default](#): 112 [or Color](#)(Black on White) [MenuTextLtrAttr](#) Quick-key letters in a menu, and HelpLine key assignments. [Default](#): 116 [or Color](#)(Red on White) [MenuBorderAttr](#) [Menu](#) borders. [Default](#): 112 [or Color](#)(Black on White) [MenuSelectAttr](#) [Menu](#) selection [cursor](#) bar for enabled [menu](#) items. [Default](#): 31 [or Color](#)(Bright White on Blue) [MenuSelectLtrAttr](#) [Menu](#) selection quick-key letter. [Default](#): 28 [or Color](#)(Bright Red on Blue) [MenuGrayAttr](#) Disabled "grayed" selection items in a menu. [Default](#): 120 [or Color](#)(Bright Black on White) [MenuSelectGrayAttr](#) [Menu](#) selection [cursor](#) bar for disabled "grayed" [menu](#) items. [Default](#): 23 [or Color](#)(White on Blue) [HelpSystem Color Variables](#) ----- [HelpTextAttr](#) Normal text in [the HelpSystem](#). [Default](#): 23 [or Color](#)(White on Blue) [HelpLinkAttr](#) Links to the next topic in [the HelpSystem](#). [Default](#): 30 [or Color](#)(Bright Yellow on Blue) [HelpInfoAttr](#) InfoBox items in [the HelpSystem](#). [Default](#): 26 [or Color](#)(Bright Green on Blue) [HelpBoldAttr](#) Highlighted (bold) text in [the HelpSystem](#). [Default](#): 31 [or Color](#)(Bright White on Blue) [HelpItalicsAttr](#) "Italicized" text in [the HelpSystem](#). [Default](#): 29 [or Color](#)(Bright Magenta on Blue) [HelpSelectAttr](#) Selected field in [the HelpSystem](#). [Default](#): 113 [or Color](#)(Blue on White) [SyntaxHilite Color Variables](#) ----- [Directive1Attr](#) SyntaxHilite directive text, entry 1. [Default](#): 244 [or Color](#)(Intense Red on White) [Directive2Attr](#) SyntaxHilite directive text, entry 2. [Default](#): 244 [or Color](#)(Intense Red on White) [Directive3Attr](#) SyntaxHilite directive text, entry 3. [Default](#): 244 [or Color](#)(Intense Red on White) [Keywords1Attr](#) SyntaxHilite keyword text, group 1. [Default](#): 249 [or Color](#)(Intense Bright Blue on White) [Keywords2Attr](#) SyntaxHilite keyword text, group 2. [Default](#): 246 [or Color](#)(Intense Yellow on White) [Keywords3Attr](#) SyntaxHilite keyword text, group 3. [Default](#): 253 [or Color](#)(Intense Bright Magenta on White) [Keywords4Attr](#) SyntaxHilite keyword text, group 4. [Default](#): 252 [or Color](#)(Intense Bright Red on White) [Keywords5Attr](#) SyntaxHilite keyword text, group 5. [Default](#): 243 [or Color](#)(Intense Cyan on White) [Keywords6Attr](#) SyntaxHilite keyword text, group 6. [Default](#): 243 [or Color](#)(Intense Cyan on White) [Keywords7Attr](#) SyntaxHilite keyword text, group 7. [Default](#): 249 [or Color](#)(Intense Bright Blue on White) [Keywords8Attr](#) SyntaxHilite keyword text, group 8. [Default](#): 249 [or Color](#)(Intense Bright Blue on White) [Keywords9Attr](#) SyntaxHilite keyword text, group 9. [Default](#): 249 [or Color](#)(Intense Bright Blue on White) [NumberAttr](#) SyntaxHilite number text. [Default](#): 245 [or Color](#)(Intense Magenta on White) [MultiLnDlmt1Attr](#) SyntaxHilite multi-line delimited text, entry 1. [Default](#): 242 [or Color](#)(Intense Green on White) [MultiLnDlmt2Attr](#) SyntaxHilite multi-line delimited text, entry 2. [Default](#): 242 [or Color](#)(Intense Green on White) [MultiLnDlmt3Attr](#) SyntaxHilite multi-line delimited text, entry 3. [Default](#): 242 [or Color](#)(Intense Green on White) [SingleLnDlmt1Attr](#) SyntaxHilite single-line

delimited text, entry 1. [Default](#): 242 [or Color](#)(Intense Green on White) [SingleLnDlmt2Attr](#) SyntaxHilite single-line delimited text, entry 2. [Default](#): 242 [or Color](#)(Intense Green on White) [SingleLnDlmt3Attr](#) SyntaxHilite single-line delimited text, entry 3. [Default](#): 242 [or Color](#)(Intense Green on White) [ToEOL1Attr](#) SyntaxHilite to-EOL delimited text, entry 1. [Default](#): 242 [or Color](#)(Intense Green on White) [ToEOL2Attr](#) SyntaxHilite to-EOL delimited text, entry 2. [Default](#): 242 [or Color](#)(Intense Green on White) [ToEOL3Attr](#) SyntaxHilite to-EOL delimited text, entry 3. [Default](#): 242 [or Color](#)(Intense Green on White) [Quote1Attr](#) SyntaxHilite quote delimited text, entry 1. [Default](#): 245 [or Color](#)(Intense Magenta on White) [Quote2Attr](#) SyntaxHilite quote delimited text, entry 2. [Default](#): 245 [or Color](#)(Intense Magenta on White) [Quote3Attr](#) SyntaxHilite quote delimited text, entry 3. [Default](#): 245 [or Color](#)(Intense Magenta on White) [IncompleteQuoteAttr](#) SyntaxHilite incomplete quote text. [Default](#): 192 [or Color](#)(Intense Black on Red) [Other Color Variables](#) ----- [EOFMarkerAttr](#) [End](#)-of-file marker. [Default](#): 112 [or Color](#)(Black on White) [MsgAttr](#) Messages and warnings. [Default](#): 112 [or Color](#)(Black on White) [StatusLineAttr](#) StatusLine. [Default](#): 31 [or Color](#)(Bright White on Blue) [LineNumbersAttr](#) LineNumbers. [Default](#): 112 [or Color](#)(Black on White)

COLOR VARIABLES

The editor allows you to set different colors for many items on the editor's screen display, using the variables described in this section.

Colors are set by assigning to a color variable the number associated with the desired foreground/background color. You can use the

[Color\(\)](#) command to return the number of the color attribute you want to assign to the color variable, or you can enter a color directly if you know the decimal value of the color combination.

Command Prompts

As listed above, three types of command prompts are issued by the editor: prompt boxes, StatusLine prompts, and selection [menus](#). Prompt boxes request additional user-defined string input; StatusLine prompts generally request a single character or keystroke response; selection menus display a set of pre-defined selections from which to choose. The editor's [Menu](#) system uses the following notations to indicate that a command issues a prompt when executed: - A series of three dots ("...") follows a command entry that issues a [prompt box](#) or [StatusLine prompt](#). This notation is also used to identify a command that issues a PickList. - The symbol ">" follows a command entry that issues a selection menu. [Prompt Box](#) ----- For those commands which require additional [string](#) input in order to continue, a prompt box is displayed on the screen once the command is executed. Within the prompt box, a number of editing commands and special prompt features are available to simplify entry of the desired string response. The following table identifies commands that are operational in prompt boxes: COMMANDS OPERATIONAL IN PROMPT BOXES

Key /Mouse Command	Assignment* Operation within Prompt Box
CReturn	<Enter>* Accepts user entry. or <LeftBtn>* Escape <Escape>* Cancels command or operation. or <RightBtn>* Exit <Alt X> Cancels command or operation. BackSpace <Backspace>* Deletes character to left of cursor or <Ctrl H>* in prompt box ; any text to right of cursor is shifted left. DelChar * Deletes character at cursor position in prompt box . DelLeftWord <Ctrl Backspace> Deletes all/part of word to left of cursor in prompt box . DelRightWord <Ctrl Del> Deletes all/part of word from and or <Ctrl T> to right of cursor in prompt box . DelLine <Alt D> Deletes contents of prompt box . DelToEol <F6> Deletes text from and to right of or <Alt Del> cursor position in prompt box . BegLine <Home>* Moves cursor to beginning of prompt box. EndLine <End>* Moves cursor to end of prompt string . Left <CursorLeft>* Moves cursor left in prompt box . Right <CursorRight>* Moves cursor right in prompt box . Down <CursorDown>* Pops up History box. Up <CursorUp>* Pops up History box. WordLeft <Ctrl CursorLeft> Moves cursor to first character of word left of cursor in prompt box .

WordRight <Ctrl CursorRight> Moves [cursor](#) to first character of word right of [cursor](#) in [prompt box](#). TabRight <Tab>* Filename completion. AsciiChart <Ctrl A>* Pops up ASCII chart. Help <F1>* Relevant Help (if available). Dos <Alt F9> Displays "OS command:" prompt [for](#) execution of external command. Shell <F9> Temporarily suspends editor and opens an OS prompt. Copy <Grey+> Copies text in [prompt box](#) to System ClipBoard. Cut <Grey-> Cuts text in [prompt box](#) to System ClipBoard. CopyBlock <Alt C> Copies marked block into prompt box. If no block marked, copies word at [cursor](#) position in text. CopyToWinClip <CtrlShift Grey+> Copies text in [prompt box](#) to or <Ctrl Ins> [Windows](#) Clipboard. CutToWinClip <CtrlShift Grey-> Cuts text in [prompt box](#) to or <Shift Del> [Windows](#) Clipboard. ListRecentFiles <Ctrl 0> Displays Recent Files list; places selected filename into [prompt box](#). Literal <Ctrl P> Places literal representation of a character into prompt box at [cursor](#) position. ("Literal:" [StatusLine prompt](#) is [not](#) displayed; however, character entered following execution of Literal command is processed.) Paste <Grey*> Pastes contents of [System ClipBoard](#) into [prompt box](#). PasteFromWinClip <CtrlShift Grey*> Pastes contents of [Windows](#) or <Shift Ins> Clipboard into [prompt box](#). ToggleInsert <Ins>* Toggles [Insert mode ON](#) and [OFF](#). Keyboard Macro You can run a keyboard macro by pressing it's assigned [key](#) while in a prompt box. Use with caution. If the keyboard macro contains commands which are not valid with a prompt box, undesirable results may occur. * An "*" following a key/mouse assignment indicates the editor has forced that key or mouse button to invoke the indicated command in a [prompt box](#), regardless of how that [key](#) or mouse button is configured by the user. For all other key/mouse assignments listed, if the user has reconfigured the command, the command is invoked by pressing the user-defined [key](#) or mouse button. When a prompt box is displayed, it is placed on the screen immediately above the cursor line. If there is not enough room to place the entire prompt box above the cursor line, it is placed immediately below the cursor line. Within a prompt box, string input is entered by typing characters and/or executing any of the commands listed above. The numeric keypad can also be used to input the decimal value of a character that is not directly available from the keyboard. (For more information on using the numeric keypad to input characters, see the Tip under "[Commands to Add Text](#)" in the chapter on "[Manipulating Text](#).") After your response is completed, press <Enter> for the operation to continue. To cancel an operation from a prompt box, press <Escape>. [History Buffer for Prompt Boxes](#) The editor maintains a History Buffer that contains response strings previously entered within prompt boxes. This allows you to view and select entries from a pop-up History box containing previous responses for a particular command or group of related commands. Certain groups of commands share the same set of History entries. For example, the commands EditFile, InsertFile, SaveAs, and SaveBlock share the same set of History entries; the Search prompts for the Find and Replace commands also share the same set. As each response to a prompt box is entered, it is added to the top of the History Buffer. If that entry already exists in the History Buffer for the same command or command group, the prior entry is deleted. For each subsequent execution of a particular command (or command group), the prompt box initially displays the last response string that was processed. To enter a response, you have the following options: - To enter the same response as the one displayed, simply press <Enter>. - To enter a modified response from the one that is displayed, first execute one of the cursor-movement commands (BegLine, EndLine, Left, Right, Down, Up), one of the deletion commands (BackSpace, DelChar, DelLine, DelToEol), the AsciiChart command, or the ToggleInsert command. This makes the displayed response active, and allows it to be modified. Now you can make desired changes to the response string. Press <Enter> when the string is complete. - To automatically discard the response that is displayed and enter a new response, immediately begin typing the new string, and/or use the CopyBlock, Literal, or Paste commands to input a new string within the prompt box. Press <Enter> when the string is complete. - To select a previous response, press <CursorDown> or <CursorUp> to pop up a History box containing a PickList of History Buffer entries for the command. Position the cursor bar on the desired response and press <Enter> to select that entry, or press <Escape> to remove the History entries without selecting a previous response. (For more information on how to select entries from a PickList, see "[Using PickLists](#)" later in this chapter.) The selected entry is then placed in the prompt box, and you can make any desired changes to the response string. Press <Enter> when the string is complete. The editor gives you the option to maintain History entries across editing sessions, via the PersistentHistory variable. If PersistentHistory is ON, the entries in the History Buffer are saved at the end of each editing session, and then reloaded into the History Buffer at the start of the next editing session. (For more information, see "[Saving/Restoring Entries in the History Buffer](#)" in the chapter on "[Loading, Processing, and Saving Files](#).") If PersistentHistory is OFF, the History Buffer is cleared for each new editing session, and only those responses entered during the current editing session will be included in the History Buffer. Within each new session under this condition, the first time one of the

applicable commands is executed, the prompt box displayed by the command will be empty. An exception is the Options prompt box for the Find and Replace commands; when first executed, these commands will optionally display initial [values](#). (For more information on setting these initial Options [values](#), see "Initial Find Options" [and](#) "Initial Replace Options" under "[Command/Format Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") The editor maintains up to 1000 total History entries (for ALL commands). The number of total entries maintained can be decreased or eliminated, if desired. The editor also allows you to specify the maximum number of entries (up to 100) that you want to maintain for each particular command or command group (that is, for each History "list"). (For more information, see "Max History Entries" [and](#) "Max History Per List" under "[Command/Format Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") If desired, you can remove individual entries from the History Buffer. This is accomplished while the appropriate History box is displayed on the screen. Simply position the cursor on the entry to be deleted, and press . [Obtaining a File PickList within a Prompt Box](#) -----

For commands that display a prompt box requesting the filename to process (such as the EditFile and SaveAs commands), you can obtain a PickList of files from which to choose the desired filename. A number of options are available which allow you to: configure the behavior of certain features of the file-related PickList, limit the entries in the PickList, and quickly scroll through the displayed entries. From a file-related prompt box, you can also sort the PickList entries, select an alternate drive for which to display the matching list of files, and delete listed files from disk. Within a file-related prompt box, you can enter an abbreviated filename to obtain a file PickList. Such a PickList includes the names of files that match the abbreviated filename specification, as well as associated directory entries. By default, the entries in a file PickList consist of: - All matching files, in ascending order by filename. - All directories in the indicated path. The directory entries are grouped together at the top of the PickList, and are designated with the "\" symbol preceding the name, and the notation < DIR > following the name. - The parent directory (if applicable), indicated as "\.", as the first entry in the PickList. PickList file entries also include the size of the file in bytes, and the date and time each file was last updated or each directory was created. You can configure certain features of file-related PickLists, using the "[File PickList Settings](#)" configuration options. These options control the treatment of directories and the order of the entries in a file PickList, as follows: - You can configure whether or not the editor will change the current directory when a new directory is selected from a file PickList. (By default, the current directory is changed.) - You can configure the editor to include ALL directories that are in the path containing the files displayed in the current PickList (the default behavior); or you can limit directory entries to those that match the filename specification. (This option has no effect on the Filename Completion option discussed later in this section.) - You can configure the editor to display any included directory entries at the top of the PickList (the default behavior); or you can have the directory entries located according to the sort order of the PickList. - You can configure whether or not the editor adds a slash to the beginning of any included directory entries. (By default, the slash is included.) - You can select the default sort order (ascending or descending) of the entries in the file PickList, and you can select the default key on which the entries will be sorted. You can choose to sort the entries according to: full filename, including extension; filename extension only; file size; file date; or file attributes. (The standard [default](#) sort order is ascending, by full filename.) (For more information on configuring the file PickList options, see "[File PickList Settings](#)" under "[System/File Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") Several methods are available to allow you to specify an abbreviated filename within a file-related prompt box, as follows. (For more information on specifying abbreviated filenames, see "[Adding Files to the Ring: Specifying and Opening Files](#)" in the chapter on "[Loading, Processing, and Saving Files](#).") - Enter a blank (empty) response for a PickList of all files and directories in the current directory. - Enter an ambiguous filename specification (by including the wildcard characters, "*" or "?") for a PickList of all directories in the current directory, and all matching files. Or you can specify the "[Search SubDirs](#)" (-s) option to obtain a PickList of all files in the indicated directory and all associated subdirectories, that match the filename specification. This option is available for the EditFile and InsertFile commands. (For more information on the "[Search SubDirs](#)" option, see "[Command-Line Options](#)" in the chapter on "[Starting the Editor](#).") For this method to display a PickList when you execute the EditFile command, the variable LoadWildFromInside must be OFF, or you must precede the filename specification with the "-a-" ("[Pick from List](#)") command-line option. (For more information on the LoadWildFromInside variable, see "Load Wild from Inside" under "[System/File Options](#)" in the chapter on "[Configuring the Editor Interactively](#)." For more information on the "[Pick from List](#)" option, see "[Command-Line Options](#)" in the chapter on "[Starting the Editor](#).") - Invoke the Filename Completion option to

generate an ambiguous filename specification. To do this, begin typing the filename. When you have entered enough of the characters in the filename to identify the file(s) from which to choose, press the <Tab> key. The editor will then complete your entry by appending "*.*", ".*", or "**", as appropriate, to create an ambiguous specification, and present a PickList of matching files and only matching directories. Note that the Filename Completion option is not affected by the setting of the LoadWildFromInside variable, nor is it affected by any command-line switches that are entered within the prompt box. Once the file PickList is displayed, the following options are available: - You can select a different drive for which to display a PickList of files that match the initially entered filename specification. Press <Alt F10> to display a PickList of drives. Move the cursor bar over the desired drive and press <Enter>. A new PickList of matching files/directories on the indicated drive is displayed. - You can change the sort order of the files displayed in the PickList. Press <Alt S> to display a selection menu of sort options. Select the sort order (ascending or descending). Then select the key (name, extension, size, date, or attribute) on which to sort the entries. Once you select the key, the entries are immediately sorted based on your sort selections and the newly-sorted PickList is displayed. - You can select files from the entries in the PickList to delete from disk. - To delete a single file, move the cursor bar over the name of the file to be deleted, and press . The indicated file is then deleted and its entry removed from the PickList. - To delete multiple files, move the cursor bar over the name of each file to be deleted, and press either <Shift Spacebar> or <Ctrl Spacebar> to tag each indicated filename. Once your selections are complete, press <Enter>. The indicated files are then deleted and their entries are removed from the PickList. - You can select a file or directory from the PickList for processing. In a PickList displayed by the EditFile command, you can select multiple files and/or directories. - To select a single file or directory, move the cursor bar over the desired entry and press <Enter>. - To select multiple files and/or directories in a PickList displayed by the EditFile command, move the cursor bar over each desired entry, and press either <Shift Spacebar> or <Ctrl Spacebar> to tag the indicated files. Once your selections are complete, press <Enter>. The editor then begins processing your selection(s): - If your selection is a directory entry, a PickList of files and directories under the selected directory is displayed. (Additionally, in the standard configuration of the editor, the selected directory becomes the current directory.) You can configure the editor so that the current directory is not changed when you select a directory from a PickList. (For more information, see "[File PickList Settings](#)" under "[System/File Options](#)" in the chapter on "[Configuring the Editor Interactively](#)".) - In an EditFile PickList, if your selection included multiple directories, a PickList of files and directories under the first directory is displayed. Then, once your file selections from this newly-displayed PickList are complete, the remaining directories are successively processed. - The editor adds the selected file(s) to the Ring of files. - Finally, the first, or only, selected file is made the current file, and the command being executed resumes, using that file. (For more information on how to select entries from a PickList, see "[Using PickLists](#)" later in this chapter.)

StatusLine Prompt -----

Certain commands request additional information by displaying a prompt on the StatusLine. Generally, the prompt requests a single key or character response, and the command immediately resumes once the appropriate key has been pressed (without <Enter> being pressed). PlaceMark, Literal, and GotoWindow are [examples](#) of commands that issue such prompts. To remove a StatusLine prompt from the screen and cancel the command that was executed, press <Escape>. The Literal command, however, is [not](#) canceled by pressing <Escape>. A few commands issue StatusLine prompts that behave in a slightly different manner than the general behavior. For example: - The ResizeWindow command first displays a StatusLine prompt requesting the window border to be resized, and then displays a second prompt to allow you to move the resizing bar. At the second StatusLine prompt, you must press <Enter> to complete the command once the resizing bar has been positioned at the desired location. - The IncrementalSearch command displays a StatusLine prompt to request the successive character(s) of the string to be located. The prompt remains active on [the StatusLine](#) until you terminate the command by pressing <Enter> or <Escape>. [Selection Menu](#)

----- For commands that accept a response from predefined choices, a selection menu of available choices is displayed. To choose a response, place the cursor bar over the desired selection and press <Enter>. The command resumes, using the selected response. You can also choose the desired option within a selection menu by simply pressing the highlighted quick-key for the option. The command immediately resumes execution once the quick-key is pressed (without <Enter> being pressed). (For more information on using quick-keys in [menus](#), see "[Accessing Sub-Menu Commands and Options](#)" in the chapter on "[The Pull-Down Menu System](#)".) To cancel a command or operation from within a selection menu, press <Escape>.

Command Prompts and Keydefs

Many commands in the editor, such as

[EditFile\(\)](#) and [Find\(\)](#), issue a prompt box when you execute the command. The prompt box is displayed on the screen, and the user is allowed to enter a response for the command to process. Within a prompt box, a limited set of native commands are available for entering a response. Following are the native commands that you can execute once a prompt box is displayed on the screen: - Editing commands: [BackSpace\(\)](#), [Copy\(\)](#), [CopyBlock\(\)](#), [CopyToWinClip\(\)](#), [Cut\(\)](#), [CutToWinClip\(\)](#), [DelChar\(\)](#), [DelLine\(\)](#), [DelLeftWord\(\)](#), [DelRightWord\(\)](#), [DelToEol\(\)](#), [Literal\(\)](#), [Paste\(\)](#), [PasteReplace\(\)](#), [PasteFromWinClip\(\)](#), [PasteReplaceFromWinClip\(\)](#), [SelfInsert\(\)](#) - Movement commands: [BegLine\(\)](#), [Down\(\)](#), [EndLine\(\)](#), [Left\(\)](#), [NextChar\(\)](#), [PrevChar\(\)](#), [Right\(\)](#), [Up\(\)](#), [WordRight\(\)](#), [WordLeft\(\)](#) - Other commands: [CReturn\(\)](#), [Escape\(\)](#), [Exit\(\)](#), [Help\(\)](#), [TabRight\(\)](#), [ToggleInsert\(\)](#) The editor also forces certain of these pre-defined commands to be executed when a particular key (or mouse button) is pressed within a prompt box, as follows: [Key/](#)
Mouse Assignment Command Invoked in [Prompt Box](#)

	<Enter> or <LeftBtn>	CReturn()	<Escape> or <RightBtn>	Escape()	<Backspace>
--	----------------------	---------------------------	------------------------	--------------------------	-------------

or <Ctrl H> [BackSpace\(\)](#) [DelChar\(\)](#) <Home> [BegLine\(\)](#) <End> [EndLine\(\)](#) <CursorLeft> [Left\(\)](#) <CursorRight> [Right\(\)](#) <CursorDown> [Down\(\)](#) <CursorUp> [Up\(\)](#) <Ins> [ToggleInsert\(\)](#) <TabRight> [TabRight\(\)](#) <F1> [Help\(\)](#) For those keys (or mouse buttons) that the editor forces to invoke a pre-defined command, it does so regardless of the command or macro that is assigned to the key (or mouse button). For example, in a prompt box, the <Enter> key (or <LeftBtn>) invokes the CReturn command, even if you have assigned a different command or macro to this key (or mouse button). The editor does not force key assignments for the following prompt-box commands: [Copy\(\)](#), [CopyBlock\(\)](#), [CopyToWinClip\(\)](#), [Cut\(\)](#), [CutToWinClip\(\)](#), [DelLine\(\)](#), [DelLeftWord\(\)](#), [DelRightWord\(\)](#), [DelToEol\(\)](#), [Exit\(\)](#), [Literal\(\)](#), [NextChar\(\)](#), [Paste\(\)](#), [PasteFromWinClip\(\)](#), [PasteReplace\(\)](#), [PasteReplaceFromWinClip\(\)](#), [PrevChar\(\)](#), [SelfInsert\(\)](#), [WordLeft\(\)](#), and [WordRight\(\)](#). Thus, any of these commands can be executed within a prompt box only if you have assigned the command to a key. If you have replaced one of these native editor commands with a macro of your own, and you do not have a key assignment for the replaced native command in your copy of the editor, you will not be able to execute the command in a prompt box. To resolve this potential problem, the editor lets you identify your own group of additional keys that are forced to execute a particular command within a [prompt box](#). A special "PromptKeys" [keydef](#) allows you to have your own macro assigned to a key for use while editing, and also have one of the prompt-box commands assigned to that same key for use only when the key is pressed in a [prompt box](#). The syntax to declare the PromptKeys keydef is: [keydef PromptKeys](#) <key_list> command ... <key_list> command [end](#) [[PromptKeys](#)] where: - [keydef](#) specifies this as a key definition. - [PromptKeys](#) identifies this as a PromptKeys definition. - [key_list](#) is a key (or two-key) name. - [command](#) is one of the commands that are functional within a prompt box. - [end](#) specifies the end of the key definition. - The [PromptKeys](#) identifier may optionally be repeated after the end statement on the same line. It is not necessary to enable and disable [PromptKeys](#) keydefs with the [Enable\(\)](#) or [Disable\(\)](#) commands. Instead, key assignments made in a [PromptKeys](#) keydef (and installed in the editor) are automatically enabled when a prompt box is displayed. This effectively overrides the existing assignments for the keys for the duration of the prompt. Once the prompt is complete, the [PromptKeys](#) assignments are automatically disabled and the previous key assignments are restored. To illustrate use of this feature, suppose you want a customized version of the [Paste\(\)](#) command to be assigned to the <Grey*> key. In your custom version, "paste" acts differently depending on the state of Insert mode, and is defined as follows: [proc](#) myPaste() [Paste\(iif\(Query\(Insert\), _DEFAULT_, _OVERWRITE_\)\)](#) [end](#) <Grey*> myPaste() With this macro, you have a paste command that works the way you like. However, you can no longer use the <Grey*> key to Paste inside of prompts, because myPaste() is a macro, rather than the native [Paste\(\)](#) command that is operational in a prompt box. To remedy this, locate the [PromptKeys](#) keydef in the TSE.UI user-interface file in the UI\editor subdirectory (assuming you are using the standard user interface), and add the following key assignment: [keydef PromptKeys](#) <Grey*> [Paste\(\)](#) [end](#) After you compile and install your new TSE.UI interface, the native [Paste\(\)](#) command will again be available in prompt boxes when you press <Grey*>. But when you press <Grey*> while editing (outside a prompt box), the myPaste() macro is executed.

Command Prompts and Picklists

Many commands in the editor require that additional information be provided or that selections be made before the command can continue or complete the operation. For this, the editor issues command prompts and/or PickLists. Within PickLists (and certain other types of displayed text), the editor offers SpeedSearch capabilities.

This chapter describes the following features in the editor:

-

[Command Prompts](#): - Prompt box (including use of the History Buffer and file PickLists) - [StatusLine prompt](#) - [Selection menu](#) - [Using PickLists](#) - [SpeedSearch Feature](#)

Command-Line Options

You can specify a number of editing options when you start the editor (or when you later open additional files during an editing session using the EditFile command). These

[command-line options](#) can be entered either by including them on the command line itself; or (unless otherwise noted in this section) by passing them to the editor at the "File(s) to edit:" prompt, if the editor is started without specifying any filenames. Command-line options are identified by either the "-" or "/" character, immediately followed by the letter that designates the desired option. You can enter the designating letter in either upper or lower case. You can also select multiple options (except when selecting the [Restore State](#) option). This section describes the command-line options recognized by the editor. Note that if you include the [Load Macro](#) (-l) or [Execute Macro](#) (-e) command-line option, but do not specify any filenames to edit, the editor does not display the prompt, "File(s) to edit:" (as described in the previous section, [Selecting Files to Edit](#)). Instead, the editor creates its own "dummy" ("unnamed") file. This allows the editor to load/execute the indicated macro file without forcing you to specify a file to be loaded. This feature can be useful for certain types of [macros](#).

* [Load Keyboard Macro Syntax](#): - kKeyMacroFilename This option instructs the editor to invoke the LoadKeyMacro command using the indicated KeyMacroFilename. This causes the editor to load the [Keyboard Macros](#) contained in KeyMacroFilename and make these [macros](#) available in the current editing session. KeyMacroFilename must immediately follow "-k", with no intervening space. Only one [Load Keyboard Macro](#) option is recognized. If you enter more than one, only the final entry is processed. If KeyMacroFilename does not include an extension, ".KBD" (the editor convention for Keyboard Macro filename extensions) is assumed. If you do not specify the path for KeyMacroFilename, the editor searches for the file in several pre-determined directories. For more information on where the editor searches for the indicated filename, see [Loading Keyboard Macros Saved in a Previous Session](#) in the chapter on [Creating and Using Macros](#).

* [Examples](#): Console version: Replace g32 with e32 g32 foo.doc abc.txt -kKeyMacro.kbd g32 -kKeyMacro foo.doc abc.txt These examples instruct the editor to select the files foo.doc and abc.txt, and then to load the Keyboard Macros in the Keyboard Macro file KeyMacro.kbd. (For more information on creating and loading Keyboard Macro files, see [Keyboard Macros](#) in the chapter on [Creating and Using Macros](#).)

* [Load Macro Syntax](#): - lMacroFilename This option instructs the editor to invoke [the LoadMacro command](#) using the indicated MacroFilename. This causes the editor to load the Compiled Macros contained in the external MacroFilename and make these macros available for execution. If MacroFilename contains a WhenLoaded macro, it is executed. MacroFilename must immediately follow "-l", with no intervening space. The [Load Macro](#) option can be specified for multiple MacroFilename entries. If MacroFilename does not include an extension, ".MAC" (the editor convention for external Compiled Macro filename extensions) is assumed. If you do not specify the path for MacroFilename, the

editor searches for the file in several pre-determined directories, according to the "[macro file search order](#)" described under "[Creating and Using Compiled Macros](#)" in the chapter on "[Creating and Using Macros](#)." **Examples:** Console version: Replace g32 with e32 g32 foo.doc abc.txt -lMyMacro.mac g32 -lMyMacro foo.doc abc.txt These examples instruct the editor to select the files foo.doc and abc.txt, and then to load the Compiled Macros in the Compiled Macro file MyMacro.mac. If MyMacro.mac contains a WhenLoaded macro, that WhenLoaded macro is then executed. (For more information on creating and loading Compiled Macro files, see "[Introduction to External Compiled Macros](#)" in the chapter on "[Creating and Using Macros](#)," and also see "[LoadMacro\(\)](#)" in the chapter on "[Editor Commands](#)" in the [Macro Reference Guide](#). For more information on processing WhenLoaded macros, see "[Event Macros](#)" in the chapter on "[Using Macro Language Features](#)" in the [Macro Reference Guide](#).)

*** Execute Macro Syntax:** -eMacroFilename This option instructs the editor to invoke [the ExecMacro command](#) using the indicated MacroFilename. This causes the editor to load the Compiled Macros contained in the external MacroFilename and make these macros available for execution. If MacroFilename contains a WhenLoaded macro, it is executed. Additionally, if MacroFilename contains a Main macro, it is also executed. MacroFilename must immediately follow "-e", with no intervening space. The "[Execute Macro](#)" option can be specified for multiple MacroFilename entries. If MacroFilename does not include an extension, ".MAC" (the editor convention for external Compiled Macro filename extensions) is assumed. If you do not specify the path for MacroFilename, the editor searches for the file in several pre-determined directories, according to the "[macro file search order](#)" described under "[Creating and Using Compiled Macros](#)" in the chapter on "[Creating and Using Macros](#)." **Examples:** Console version: Replace g32 with e32 g32 foo.doc abc.txt -eMyMacro.mac g32 -eMyMacro foo.doc abc.txt These examples instruct the editor to select the files foo.doc and abc.txt, and then to load the Compiled Macros in the Compiled Macro file MyMacro.mac. If MyMacro.mac contains a WhenLoaded macro, that WhenLoaded macro is then executed. Finally, if MyMacro.mac contains a Main macro, that Main macro is executed. (For more information on creating and executing Compiled Macro files, see "[Introduction to External Compiled Macros](#)" in the chapter on "[Creating and Using Macros](#)," and also see "[LoadMacro\(\)](#)" and "[ExecMacro\(\)](#)" in the chapter on "[Editor Commands](#)" in the [Macro Reference Guide](#). For more information on processing WhenLoaded and Main macros, see "[Event Macros](#)" in the chapter on "[Using Macro Language Features](#)" in the [Macro Reference Guide](#).)

*** Go To Line/Column Number Syntax:** -nLineNumber[,ColumnNumber] This option instructs the editor to position the cursor at the line number indicated as LineNumber, in the FIRST file that is loaded. It also allows you to specify an optional column number, indicated as ColumnNumber, on which to position the cursor on the specified line. LineNumber must immediately follow "-n", with no intervening space. To specify a column number, place a comma and the appropriate ColumnNumber immediately after LineNumber. To specify ColumnNumber without specifying LineNumber, place a comma and the appropriate ColumnNumber immediately after "-n"; the editor assumes line number 1. Only one "[Go To Line/Column Number](#)" option is recognized. If you enter more than one, only the final entry is processed. This option is always processed on the FIRST file that is loaded, regardless of the placement of the option within the list of any filenames or other command-line options. **Examples:** Console version: Replace g32 with e32 g32 foo.doc abc.txt -n79 g32 -n79,5 foo.doc abc.txt Both examples instruct the editor to select the files foo.doc and abc.txt, and position the cursor at line 79 of foo.doc. The cursor will be positioned in column 1 in the first example, and in column 5 in the second example.

*** Binary Load Syntax:** -bLineLength This option instructs the editor to load the first filename following "-bLineLength", in [Binary mode](#). The length of each line displayed is determined by the value specified as LineLength. That is, the file contents are split at the requested length and wrapped to successive lines. LineLength must immediately follow "-b", with no intervening space. Each filename preceded by a "[Binary Load](#)" indicator will be loaded in Binary mode, and the lines will be split at the line length specified for that file. If you follow this option with a file specification containing wildcard characters ("*" or "?"), all matching files that are loaded will be displayed in [Binary mode](#). If you do not specify LineLength for this option, a line length of 64 is assumed. If you specify a LineLength value of zero, the editor ignores that "[Binary Load](#)" option. If you specify a value for LineLength that exceeds the editor's maximum line length, the editor's maximum line length is assumed. You can load a file in Binary mode in combination with Hex mode by preceding the filename with "-bLineLength -h" (or "-h -bLineLength"). (Hex mode is described below.) (For more information on editing files in [Binary mode](#), see "[Alternative Modes for Viewing Files](#)" in the chapter on "[Loading, Processing, and Saving Files](#).")

Examples: Console version: Replace g32 with e32 g32 foo.doc -b40 abc.txt g32 -b *.doc abc.txt The first example instructs the editor to

select the files foo.doc and abc.txt. The file abc.txt will be loaded in [Binary mode](#), with lines split at column 40. The second example instructs the editor to select all files (in the current directory) with an extension of ".doc", and to also select the file abc.txt. Any ".doc" files that are loaded will be displayed in Binary mode, and the lines split at column 64. * [Hex Load Syntax](#): -h This option instructs the editor to load the first filename following "-h", in Hex (hexadecimal) mode. If you follow this option with a file specification containing wildcard characters ("*" or "?"), all matching files that are loaded will be displayed in [Hex mode](#). You can load a file in Hex mode in combination with Binary mode by preceding the filename with "-h -bLineLength" (or "-bLineLength -h"). (Binary mode is described above.) (For more information on viewing files in [Hex mode](#), see "[Alternative Modes for Viewing Files](#)" in the chapter on "[Loading, Processing, and Saving Files](#).") [Examples](#): Console version: Replace g32 with e32 g32 foo.doc -h abc.txt g32 -h *.doc abc.txt g32 -h -b abc.txt The first example instructs the editor to select the files foo.doc and abc.txt. The file abc.txt will be loaded in [Hex mode](#). The second example instructs the editor to select all files (in the current directory) with an extension of ".doc", and to also select the file abc.txt. Any ".doc" files that are loaded will be displayed in [Hex mode](#). The third example instructs the editor to load abc.txt in Hex and Binary modes. * [Browse Load](#) This option instructs the editor to load the first filename following "-y", in [Browse Mode](#). If you follow this option with a file specification containing wildcard characters ("*" or "?"), all matching files that are loaded will be displayed in [Browse mode](#). (For more information on editing files in [Browse mode](#), see "[Alternative Modes for Viewing Files](#)" in the chapter on "[Loading, Processing, and Saving Files](#).") [Examples](#): Console version: Replace g32 with e32 g32 foo1.doc -y foo2.txt g32 -y *.doc abc.txt The first example instructs the editor to select the files foo1.doc and foo2.txt. The file foo1.txt will be loaded in normal mode, and foo2.txt will be loaded in [Browse mode](#). The second example instructs the editor to select all files (in the current directory) with an extension of ".doc", and to also select the file abc.txt. Any ".doc" files that are loaded will be loaded in [Browse mode](#), with abc.txt being loaded in normal mode. * [Load All Syntax](#): -a This option instructs the editor to select all matching files for the FIRST filename specification following "-a", and make those files available for editing. This option is in effect only for ambiguous filenames (those that contain wildcard characters, "*" or "?"), or if the "[Search SubDirs](#)" (-s) option is also in effect. Normally, files are selected from the command line according to the setting of the LoadWildFromDOS variable. Similarly, files are selected within the editor from the "File(s) to edit:" prompt according to the setting of the [LoadWildFromInside](#) variable. The "[Load All](#)" option allows you to override the setting of LoadWildFromDOS or LoadWildFromInside, as appropriate. (For more information on the LoadWildFromDOS and LoadWildFromInside variables, see "Load Wild from Command Line" [and](#) "Load Wild from Inside," respectively, under "[System/File Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") The "[Load All](#)" option has no effect on filename specifications that do [not](#) contain wildcard characters, unless the "[Search SubDirs](#)" (-s) option is also in effect. Example: Console version: Replace g32 with e32 g32 *.doc -a *.txt This example instructs the editor to handle "*.doc" according to the setting of the LoadWildFromDOS variable. However, the editor will automatically select all files matching the "*.txt" specification (regardless of the [LoadWildFromDOS](#) setting), and make them available for editing. * [Pick from List Syntax](#): -a- This option instructs the editor to provide a PickList of all matching files for the FIRST filename specification following "-a-", and allows you to select the file to edit. This option is in effect only for ambiguous filenames (those that contain wildcard characters, "*" or "?"), or if the "[Search SubDirs](#)" (-s) option is also in effect. Normally, files are selected from the command line according to the setting of the LoadWildFromDOS variable. Similarly, files are selected within the editor from the "File(s) to edit:" prompt according to the setting of the LoadWildFromInside variable. The "[Pick from List](#)" option allows you to override the setting of LoadWildFromDOS or LoadWildFromInside. (For more information on the LoadWildFromDOS and LoadWildFromInside variables, see "Load Wild from Command Line" [and](#) "Load Wild from Inside," respectively, under "[System/File Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") The "[Pick from List](#)" option has no effect on filename specifications that do not contain wildcard characters, unless the "[Search SubDirs](#)" option is also in effect. Example: Console version: Replace g32 with e32 g32 *.doc -a- *.txt This example instructs the editor to handle "*.doc" according to the setting of the LoadWildFromDOS variable. However, the editor will provide a PickList of all files matching the "*.txt" specification (regardless of the [LoadWildFromDOS](#) setting). (For more information on PickLists, see "[Using PickLists](#)" in the chapter on "[Command Prompts and PickLists](#).") * [Search SubDirs Syntax](#): -s This option

instructs the editor to search not only the indicated directory [for](#) any filename specifications, but also all associated subdirectories. It [affects](#) all filename specifications which follow "-s". If "-s" is specified, then any matching files that the editor locates will be handled according to the setting of the LoadWildFromDOS variable (if entered on the command line) or the LoadWildFromInside variable (if entered at the "File(s) to edit:" prompt), or according to the "[Load All](#)" (-a) or "[Pick from List](#)" (-a-) option, if specified. Note that the "[Search SubDirs](#)" option affects all filename specifications which follow, even those that do [not](#) contain wildcard characters. [Examples](#): Console version: Replace g32 with e32 g32 -s abc.doc xyz.txt g32 *.txt -s abc.doc g32 -s -a \files\abc.doc g32 -s -a- \files*.txt g32 -a- -s c:\ The first example instructs the editor to locate all occurrences of the filename abc.doc, and all occurrences of the filename xyz.txt, in the current directory and all subdirectories of the current directory. The located files will be handled according to the setting of the [LoadWildFromDOS](#) variable. The second example instructs the editor to locate all files, in the current directory only, with an extension of ".txt"; and to handle such files according to the setting of the LoadWildFromDOS variable. The editor will also locate all occurrences of the filename abc.doc, in the current directory and all subdirectories of the current directory; and will handle the located files according to the setting of the LoadWildFromDOS variable. The third example instructs the editor to locate all occurrences of the filename abc.doc, in the \files directory and all subdirectories of the \files directory. All the located files will automatically be selected and made available for editing. The fourth example instructs the editor to locate all occurrences of all files with an extension of ".txt", that are in the \files directory and all subdirectories of the \files directory. The editor will provide a PickList of all the files that are located. The fifth example instructs the editor to find all files on the c: drive, beginning at the root directory, and provide a PickList of the files. * [Restore State Syntax](#): -r This option instructs the editor to restore the editing environment from a previous session. This includes the file(s) that were loaded and the [window\(s\)](#) and file(s) that were displayed on the screen; the position of the cursor at the time the session was ended; the block (if any) that was marked; the bookmarks that were placed; and the settings of selected editing options (such as those on the "Options" Menu). If this option is entered at the "File(s) to edit:" prompt, any files that are currently loaded remain loaded; any additional files included in the restored environment are added to the current Ring of files. If the "-r" option is specified, the editor ignores any filenames or other [command-line options](#) that may be specified. This option is only valid if the editing environment was saved when a previous editing session was terminated. Environment information is stored in the "state file" (TSESTATE.DAT). To locate the state file, the editor looks first in the current directory; then, if not located, it looks in the editor load directory. (For more information on saving and restoring the editing environment, see "[Saving/Restoring the Editing Environment](#)" in the chapter on "[Loading, Processing, and Saving Files](#).") Example: Console version: Replace g32 with e32 g32 -r This example instructs the editor to restore the editing environment from the last session for which the environment was saved. * [Augment Load Dir Syntax](#): -iDirectoryPath This option instructs the editor to extend its view of the "[load directory](#)," to a specified additional directory. This extended directory (rather than the actual directory from which the editor is loaded) will then be used as the load directory in which the editor will write any necessary configuration files. This will also extend the directories the editor searches when it is attempting to locate certain files in the load directory: the editor will first search the extended load directory, and if the file is still not found, the editor will then search in the directory from which the editor was actually loaded on startup. This option is only valid when included on the command line at editor startup. It is not recognized at the "File(s) to edit:" prompt. You can also extend the editor load directory by setting the TSELOADDIR environment variable. Once this is done, the "-i" command-line option can then be used, as needed, to override the value of TSELOADDIR at startup. (For more information on using the TSELOADDIR environment variable, see "[Augmenting the Editor Load Directory](#)" in the chapter on "[Installation of the SemWare Editor Professional](#).") Example: Console version: Replace g32 with e32 g32 -ic:\TSEPro abc.txt This example instructs the editor to extend its load directory to include c:\TSEPro, in addition to the actual directory from which the editor is loaded. File abc.txt will be selected for editing. * [Single Instance of the Editor On Syntax](#): -1 (The option is "minus one") This option instructs the editor to start/use a single instance of the editor for editing purposes. See [SingleInstance](#) for more information. * [Single Instance of the Editor Off Syntax](#): -1- (The option is "minus one minus") This option instructs the editor start as a separate, non-shared instance [for](#) editing purposes. See [SingleInstance](#) for more information.

Commands and Features That Allow Recovery of Text

The editor offers three features for restoring text: Undo (with a corresponding Redo), which allows you to step back through changes made to the text; RestoreCursorLine, which allows you to restore the current line with one keystroke; and the Deletion Buffer, which allows you to recover deleted text and restore it to the same or different file(s).

Using Undo/Redo ----- * Undo (<Alt Backspace>, or "Undo" under Text [Menu](#)) This command cancels the last change made to the text in the current file. You can use this command to successively cancel each preceding change to the text. For characters that are typed in the text, Undo cancels those changes in groups of characters. For example, words that are typed in succession may be deleted by word, rather than character by character. Or, a series of successive backspace commands may be removed by a single execution of Undo. But if continuous typing is "broken," such as by [cursor](#)-movement commands, then a single execution of Undo only deletes all changes since the last "[break](#)". For example, if you type the following without "breaking" your typing: This is a test. Then the first execution of Undo will remove " test." A second execution of Undo will remove " a". A third execution of Undo will remove " is". A fourth execution of Undo will remove "This". However, if you type the following: This is a tt Then immediately execute Backspace <Backspace> to remove the last "t", and then proceed to complete the line as: This is a test. Then the first execution of Undo will remove "est.". A second execution of Undo will restore the duplicate "t" that was originally entered and deleted (so that the text reads "This is a tt"). A third execution of Undo will remove " tt". A fourth execution of Undo will remove " a". And so on. For broad changes made to the text, Undo cancels the entire scope of the changes made by the last previous keystroke. For example, if you delete a block of text, then a single execution of Undo will restore the entire block. Or, for example, if you issue the Replace command to replace all occurrences of "this" in the text with "that", and you choose the "No-prompt" option, then a single execution of Undo will restore all changed occurrences of "that" back to "this". But if you do not specify the "No-prompt" option, and instead let the editor prompt you for each replacement, then you will have to continue to execute Undo to successively restore each changed occurrence of "that" back to "this" (from the last back to the first). Note that if you Undo more than you intended, you can use Redo (if you do so immediately) to restore the change (that is, to cancel that Undo action). Whenever you use the SaveFile command to save a file, the Undo/Redo information for that file is purged (unless the editor is configured otherwise). Thus, you cannot Undo any changes made prior to the time the file was last saved. (You can configure the editor to keep Undo/Redo information even after the file is saved. For more information on configuring this option, see "Keep Undo Beyond Save" under "[Command/Format Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") * Redo (<AltShift Backspace>, or "Redo" under Text [Menu](#)) This command cancels the last Undo action. Successive executions of Redo will cancel each previous Undo action, in reverse order. If you Redo more than you intended, you can use Undo (if you do so immediately) to cancel the change (that is, to cancel the Redo action). Once you make a change to the text using any command other than Redo or Undo, the Redo information is purged. For example, if you type the following: This is aa test. And then you execute Undo one time, the text will be left as: This is aa If you then execute Backspace to remove the final "a", you cannot execute Redo to restore "test" (which was removed by the previous Undo). Restoring the Current Line ----- * RestoreCursorLine (<Ctrl Q L>, or "Restore [Cursor](#) Line" under Text [Menu](#)) This command generally restores the current line to what it was when the cursor initially moved to the line, provided the cursor has not moved off the line. This command is useful if part of a line is accidentally deleted or altered, or if you wish to undo changes made to the line. Issue this command before the cursor moves from the current line, and before executing File and Window commands. The Deletion Buffer: A Safety Net ----- During an editing session, you can recover text deleted by certain deletion commands from a temporary storage area of deleted text, called the Deletion Buffer. When you execute the following commands, deleted text is automatically placed in the Deletion Buffer: DelLine (<Alt D>, or "Delete Line" under Text [Menu](#)) DelRightWord (<Ctrl Del>, or "Delete Right Word" under Text [Menu](#)) DelToEol (<Alt Del>, or "Delete to End of Line" under Text [Menu](#)) DelBlock (<Alt G>, or "Delete" under Block [Menu](#)) You can remove deleted text from the Deletion Buffer and restore it to the same or different file(s). The

Deletion Buffer operates as a "last in, first out" stack of entries. That is, the last deletion placed into the Deletion Buffer is the first deletion removed. Following is an illustration of the Deletion Buffer:

Entered	Buffer Contents	ID
4 File2 DelToEol	portion of deleted line 3 File2 DelRightWord	deleted word 2 File1 DelLine
contents of entire deleted line 1 File1 DelLine	contents of entire deleted line	Each time one of the above deletion commands is executed, a new entry is stored in the Deletion Buffer. In this example, four deletion commands were executed in two different files. The text deleted by each of these commands was stored in the Buffer from top to bottom. When the DelToEol command was executed (Entry #4), the first three deletions were pushed toward the bottom of the Buffer, placing the newly deleted text on top. The number of deletion entries that can be stored in the Deletion Buffer is limited only by available virtual memory. The following commands allow you to recover text from the Deletion Buffer.

* UnDelete ("Local UnDelete" under Text [Menu](#)) This command recovers the last deletion placed into the Deletion Buffer from the current file. The associated text is removed from the Buffer, and each remaining Buffer entry below that one moves up one entry, from the bottom toward the top. In the illustration above, executing UnDelete from File1 (the current file) retrieves Entry #2 and restores the entire deleted line to the current file. Entry #2 is removed from the Deletion Buffer, and Entry #1 moves up one entry in the Buffer.

* GlobalUnDelete (<Ctrl U>, or "Global UnDelete" under Text [Menu](#)) This command recovers the last deletion placed into the Deletion Buffer regardless of the file from which it was deleted, and restores it to the current file. The associated text is removed from the Buffer, and each remaining Buffer entry below that one moves up one entry, from the bottom toward the top. In the illustration above, executing GlobalUnDelete from File1 (the current file) retrieves Entry #4, and restores the deleted portion of the line to the current file. Entry #4 is removed from the Deletion Buffer, and Entries #1, #2, and #3 each move up one entry in the Buffer.

* PasteUnDelete ("Paste UnDelete" under Text [Menu](#)) Similar to the GlobalUnDelete command, this command recovers the last deletion placed into the Deletion Buffer regardless of the file from which it was deleted, and restores it to the current file. However, unlike GlobalUnDelete, the associated text is NOT removed from the Buffer; thus, PasteUnDelete can be used to copy deleted text to multiple locations in one or more files. In the illustration above, executing PasteUnDelete from File1 (the current file) retrieves Entry #4, and restores the deleted portion of the line to the current file. Entry #4 remains in the Deletion Buffer so that the Buffer is unchanged. Text recovered from the Deletion Buffer is inserted in the current file based on the cursor position. Execution of an UnDelete command does not alter the position of the cursor relative to the screen. Text is restored just as it was stored, according to the command you used to delete the text, as follows:

- If DelLine was used, the deleted line is inserted above the current line. The current line, and any lines below it, shift down. The recovered line becomes the current line.
- If DelRightWord was used, the deleted word is inserted on the current line immediately before the cursor position. Any text from and to the right of the cursor shift right. The cursor is positioned on the first character of the recovered word.
- If DelToEol was used, the deleted portion of the line is inserted on the current line beginning at the cursor position. Any text from and to the right of the cursor shift right. The cursor is positioned on the first character of the recovered text.
- If DelBlock was used, the entire deleted block of text is restored. (You cannot recover just a portion of the block of text. For example, you could not recover just one line out of a deleted block of lines.) The recovered text is placed in the current file according to the type of block you deleted, as follows:
 - If a line block was deleted, the block of deleted lines is inserted above the current line. The current line, and any lines below it, shift down. The first line of the recovered lines becomes the current line.
 - If a character block was deleted, the block of deleted characters is inserted beginning at the cursor position. Any text from and to the right of the cursor is moved to the end of the recovered text. If the deleted characters spanned multiple lines, line breaks occur at the same places in the recovered text as they were when the text was deleted. The cursor is positioned on the first character of the recovered text.
 - If a column block was deleted, the deleted column of text is inserted beginning at the cursor position. If the column of deleted text spanned lines, it is restored across lines. Any text from and to the right of the cursor position shifts right on each line, as necessary to accommodate the recovered text. Any text to the left of the cursor on the affected lines is unchanged. The cursor is positioned on the first character of the recovered text. (For more information on the Block operations discussed in this section, see the chapter on "[Block Operations](#).")

+----- Tip -----+ | The Deletion Buffer provides a quick way to move a word or line of | | text. Position the cursor on the word or line, and execute | | DelRightWord or DelLine, as appropriate. Then move the cursor to | | the new location and execute one of the UnDelete commands. Use | | PasteUnDelete to insert the text multiple places. | +-----+

Commands to Add Text

* AddLine (<F2>, or "Add Line (below)" under Text

[Menu](#)) This command inserts a blank line immediately BELOW the current line. The cursor moves to the new line without changing its column position. * InsertLine (<Alt F2>, or "[Insert](#) Line (above)" under Text [Menu](#)) This

command inserts a blank line immediately ABOVE the current line. The cursor moves to the new line without changing its column position. * Literal (<Ctrl P>) This command places the literal representation of a character into the text, at the cursor position. When you execute this command, the editor displays the prompt "Literal:" on [the StatusLine](#). You then press the key(s) for the desired character. Depending on the status of Insert mode, the editor either overwrites the current character, or inserts the new character and shifts existing text to the right. This command is useful for entering control characters (ASCII 1 through ASCII 31), such as formfeed. At the "Literal:" prompt, enter <Ctrl> plus the appropriate keyboard character (as identified in a standard ASCII chart). For example, to place a formfeed character (ASCII 12) into the text, press <Ctrl P> to display the "Literal:" prompt, and then press <Ctrl L>. To enter a null character, press <Ctrl P> and then <Ctrl @>. (Note that <Ctrl @> is the same as <Ctrl 2>.) Unlike other commands, the Literal command cannot be canceled by pressing the <Escape> key. Rather, the escape character (ASCII 27) is accepted as the response to the command, and an escape character is placed in the text. Note that if you enter a linefeed (ASCII 10) or carriage return (ASCII 13) character into the text, the line will be split at that position the NEXT time the file is loaded into the editor. +----- Tip -----+ | An alternate method for entering control characters or any ASCII | | character (1 - 255) into the text, is to hold down the <Alt> key | | while typing the decimal equivalent of the ASCII character using | | the numeric keypad. For example, to include a formfeed character | | (ASCII 12) in the text, enter <Alt 12>, using the numeric keypad. | | | | You can also use the AsciiChart feature of the editor to place any | | ASCII character into the text. (For more information, see | | "[Accessing the Pop-up ASCII Chart](#)" in the chapter on | | "[Special Commands and Features](#).") |

+-----+

Commands to Copy Text

* DupLine (<F4>, or "Dup Line" under Text

[Menu](#)) This command makes a copy of the current line and inserts it below the current line. The cursor moves to the new line without changing its column position. * CopyCharAbove (<Ctrl ->)* This command copies the character directly above the cursor onto the current line. Depending on the status of Insert mode, the editor either overwrites the character at the cursor position, or inserts the character and shifts existing text to the right. (For a similar command, see CopyCharAboveToEol below.) To use this command, place the cursor beneath a line of text and press <Ctrl ->. The character above the cursor is copied onto the current line, and the cursor moves right one position. To copy multiple characters, hold down <Ctrl -> until all characters are copied. * <Ctrl -> indicates the dash character above the alphabetic keys, NOT on the numeric keypad. * CopyCharAboveToEol (<Alt ->)* Like CopyCharAbove, this command copies characters from the line above the cursor onto the current line. Copying begins with the character immediately above the cursor, and continues through the last character of the line above. Depending on the status of Insert mode, the editor either overwrites the text on the current line beginning at the cursor position, or inserts the new text at the cursor position and shifts existing text to the right. * <Alt -> indicates the dash character above the alphabetic keys, NOT on the numeric keypad.

Commands to Delete Text

* BackSpace (<Backspace>)

This command deletes the character to the left of the cursor and moves the cursor to that position. If Insert mode is ON, the text from and to the

right of the cursor moves with the cursor.

If you execute BackSpace at column 1, the cursor moves to the end of the line above and the current line is appended to the preceding line.

* DelChar ()

This command deletes the character at the cursor position. The cursor does not move. Any text to the right of the cursor is shifted to the left.

If you execute this command beyond the last character on the current line, the line immediately below is appended to the current line.

* DelLine (<Alt D>, or "Delete Line" under Text

[Menu](#)) This command deletes the entire current line. Text below the current line shifts up one line. The cursor does not change its position relative to the screen. * DelLeftWord (<Ctrl Backspace>) This command deletes the word (or portion of the word) to the left of the cursor, and moves the cursor to the left. White characters to the immediate left of the word are deleted, and any text from and to the right of the cursor is shifted to the left. The character at the cursor position is not deleted; thus, for this command to work effectively, the cursor should be positioned immediately after the last character of the word. If you execute this command at column 1, the cursor, and any text on the current line, moves to the end of the line above. This command also deletes non-word characters, one character at a time. Deletion of a string of word characters stops when a non-word character is encountered. But if you execute DelLeftWord from one position to the right of a non-word character, that non-word character is then deleted. (For more information on how the editor defines a word, see "[WordSet Characters](#)" under "[Command/Format Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") * DelRightWord (<Ctrl Del>, or "Delete Right Word" under Text [Menu](#)) This command deletes the word (or portion of the word) to the right of the cursor, including the character at the cursor position. White characters to the immediate right of the word are deleted, and any remaining text to the right is shifted to the left. The cursor does not move. If you execute this command beyond the last character on the current line, the line immediately below is appended to the current line. This command also deletes non-word characters, one character at a time. Deletion of a string of word characters stops when a non-word character is encountered. But if you execute DelRightWord with the cursor positioned on a non-word character, that non-word character is then deleted. (For more information on how the editor defines a word, see "[WordSet Characters](#)" under "[Command/Format Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") * DelToEol (<Alt Del>, or "Delete to End of Line" under Text [Menu](#)) This command deletes the text on the current line from and to the right of the cursor. The cursor position does not change.

Commands to Format Text

* CenterLine (<Ctrl O T>, or "Center Line" under Text

[Menu](#)) This command centers the text on the current line between left and right margins, as specified by the settings of LeftMargin and RightMargin. If LeftMargin is zero, column 1 is used for the left margin. If RightMargin is zero, no action occurs when you execute CenterLine. (To set [LeftMargin](#) or RightMargin, select "Left Margin" or "Right Margin," respectively, under the Options [Menu](#).) Note that if the left and right margins are not wide enough to accommodate the text being centered, the editor attempts to center the text around the mid-point between the left and right margins. If this mid-point is too close to column 1 to accommodate the text being centered, the text is placed beginning at column 1. If you execute CenterLine with the cursor positioned in a marked block, text on each line in the block is centered. The left and right margins used for centering depend on the type of block that is marked, as follows:

- For a line or character block, the text on each line of the block is centered between the left and right margins, as

described in the preceding paragraphs. (For a character block, the entire text is centered on each line spanned by the block, even if the block does not include all text on the first or last lines of the block.) - For a column block, only text within the block is centered. The text is centered on each line between the left- and right-most columns of the block. * WrapPara (<Alt B>, or "Wrap Paragraph" under Text [Menu](#)) This command formats text in paragraph style between left and right margins, by aligning text at [the effective left margin](#) and wrapping text at the right margin, as necessary. The following paragraphs describe the behavior of WrapPara when you execute it outside a block. (For a description of the behavior of WrapPara when it is executed within a block, see "[Behavior of WrapPara within a Block](#)," below.)

When you execute WrapPara, text is formatted through the end of the paragraph following the cursor position. If the current line is not blank, formatting begins with the current line; otherwise, formatting begins with the first non-blank line following the current line. Formatting continues until the end of the paragraph is reached, as determined by the setting of the Paragraph-End Style option. (For more information on designating the style used to separate paragraphs, see "Paragraph-End Style" under "[Text Entry/Block Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") In the portion of text being formatted, the beginning column position is not changed on the first non-blank line. However, as text is wrapped on all subsequent lines in the paragraph, text is aligned at the effective left margin. [The effective left margin](#) is determined as follows: - If AutoIndent is OFF, [the effective left margin](#) is the column specified for the LeftMargin setting (or column 1, if LeftMargin is zero). - If AutoIndent is ON or STICKY, [the effective left margin](#) is the column position of the first non-white character on the second line of the text being formatted. (For more information on [AutoIndent mode](#), see "[AutoIndent Mode](#)" in the chapter on "[Editing Modes](#).") As the text is formatted, it is wrapped on each line at the right margin so that no words (continuous strings of non-white characters) extend beyond the right margin, except as noted in the next paragraph. The right margin is based on the value of RightMargin. If RightMargin is zero, or if the effective left margin is greater than the right margin, no action occurs [when](#) you execute WrapPara. (To set [RightMargin](#), select "Right Margin" under the Options Menu.) Note that the editor does not break a line in the middle of a word. If the first word on a line extends beyond the right margin, that word is not wrapped. In this case, the word will extend beyond the right margin. [Behavior of WrapPara within a Block](#)

----- If you execute WrapPara with the cursor in a block, its behavior is affected. Within a block, formatting begins with the first line of the block and continues through each paragraph in the entire block. However, the beginning column position for the first line of each paragraph in the block is not changed. Formatting stops once the end of the block is reached. The left and right margins used for formatting depend on the type of block that is marked, as follows: - For a line or character block, the left and right margins are determined as described in the preceding paragraphs for this command. For a character block, the full text is formatted on each line spanned by the block, even if the block does not include all text on the first or last lines of the block. If AutoIndent is ON or STICKY, the effective left margin is determined separately for each paragraph, based on the second line of each paragraph. - For a column block, only text within the block is formatted. The left and right margins are set to the left- and right-most columns of the block, respectively. * QuoteLine (<Ctrl 8>, or "Quote/Comment Line" under Text [Menu](#)) This command notates the text on the current line as a quote or comment. Depending on the file type (extension) of the current file, quote/comment notation(s) are inserted at the beginning, or at the beginning and end, of the line. For example: - For SAL (.s, .ui) [and](#) C++ (.cpp, .hpp) file types, "/" is inserted at the beginning of the line. - For Assembly language (.asm) file types, ";" is inserted at the beginning of the line. - For C (.c, .h) file types, "/*" is inserted at the beginning of the line, and "*/" is inserted at the end of the line. - For Pascal (.pas) file types, "{" is inserted at the beginning of the line, and "}" is inserted at the end of the line. - For other file types (such as ".txt" files, or any non-supported file type), ">" is inserted at the beginning of the line. If you execute QuoteLine with the cursor positioned in a marked block, the text on each line in the block is notated as a quote/comment. You can also use this command to "unquote" [or](#) "uncomment" text. If you execute QuoteLine on text that has already been notated as a quote or comment, the notations will be removed (in accordance with the file type, as indicated above).

Commands to Move Text

* CReturn (<Enter>)

This command moves the cursor down one line and positions it at the

effective left margin of the line. If Insert mode is ON, and the variable ReturnEqNextLine is OFF, the current line is split at the cursor position, and the text from and to the right of the cursor is placed on the new line beginning at

[the effective left margin](#). (For more information on the effective left margin, see "[AutoIndent Mode](#)" in the chapter on "[Editing Modes](#)." For more information on the ReturnEqNextLine variable, see "CReturn Equals Next Line" under "[Text Entry/Block Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") * JoinLine (<Alt J>, or "Join Line" under Text [Menu](#)) This command joins the current line with the line immediately below it. The text on the line below is moved to the end of the current line. The remaining text below the current line is shifted up one line. If the cursor is positioned on or before the last character of the line when you execute this command, the line below is appended immediately following the last character. Otherwise, if the cursor is positioned beyond the last character of the line, the line below is appended at the cursor position. * SplitLine (<Alt S>, or "Split Line" under Text [Menu](#)) This command splits the current line at the cursor position. The text from and to the right of the cursor is moved to a new line immediately below the current line, beginning at column 1. The [cursor](#) does [not](#) move. * SwapLines (<Ctrl F2>, or "Swap Lines" under Text [Menu](#)) This command swaps the current line with the line immediately following it. The cursor does not change its position relative to the screen.

Comments

SAL allows both single-line and multi-line

[comments](#). This makes it possible [for](#) you [to](#) be as descriptive as you like in commenting your macro code. To place a single-line comment at the end of a line, precede the comment with //. All text on the line which follows // is ignored. For example: // This entire line is ignored a = 10 // a is set to 10, but this comment is ignored To place multiple lines of comments within the source, enclose the comments within /* and */. All text, and all lines, within these tokens are ignored. Multi-line [comments](#) may [not](#) be nested. For example: /* This is a multi-line comment. All text is ignored until the closing indicator which follows */ The multi-line comment tokens, /* and */, can also be used to nest single-line [comments](#). It is recommended that single-line [comments](#) be used within procedures, and multi-line [comments](#) be used outside procedures. This makes it easy to comment out entire procedures by simply enclosing the procedure within /* and */.

Compiling Macros

Compiling a macro source file into an object file is a necessary step in the process of

[creating macros](#). The editor will [not](#) directly interpret macro source files, but can efficiently and quickly interpret macro object files. Macro source files can be compiled from within the editor, or at the command-line prompt. Note that [macros](#) only have to be compiled when created or changed. After being compiled, the Compiled Macro files can be executed again and again, without the need for re-compiling. To compile a macro file within the editor, load the desired macro source file into the editor, making it the current file. Now execute the Compile() command (select "Compile" under the Macro Menu, or press <Ctrl F9>). The Compile() command executes the SAL Compiler program (SC32.EXE). (For more information on the Compile() command, see "[Introduction to External Compiled Macros](#)" in the chapter on "[Creating and Using Macros](#)" in the User's Guide.) If there are errors, the screen will be split horizontally and the lower window will contain any warning or error messages produced by the compiler. Correct the indicated problems, then execute the Compile() command again. If there are no errors, the editor pops up a selection menu that allows you to "Load" or "Execute" the newly-created Compiled Macro file (or "Cancel" the operation at this point). If you wish to load and execute the macro file, select "[Execute Macro](#)" from the menu. If you only wish to load

the macro file, select "[Load Macro](#)". If you do not wish to load or execute the macro file, select "Cancel" [or](#) press <Escape>. Compiling a macro source file from within the editor is the recommended method for compiling. However, if necessary, you can also compile a macro source file from the command-line prompt, using the SAL compiler program directly. You can type just "SC32" at the prompt, and the program will prompt you for the proper parameters. If you would prefer to pass the necessary parameters on the command line, the format is: sc32 [[options] source_filename [object_filename]] where: - [options] are option flags that may be passed to the compiler. Available options [include](#): -m instructs the compiler to create a map file. This file will show the sizes (code, data and stack) for each procedure, menu, keydef, and datadef included in the macro file. The map file will have the same name as the macro source, but with an extension of ".MAP". This is useful if you are trying to write the smallest possible macros, or if you are just interested in seeing additional information about the [macros](#) you have created. -b instructs the compiler to install a new interface into the editor by attempting to burn in that interface. This is used to burn a new or changed interface macro file (".ui" or ".cfg") into the editor. If this burn-in operation is successful, the editor executable program itself (e32.exe, if the Console version is present in the current directory, or g32.exe, if the GUI version is the only version present in the current directory) is updated with the new interface. Thus, only a single file will be required to run the editor. If BOTH the console version AND GUI version are present in the current directory, the burn-in process will only update the console version (e32.exe). In this case, for the burn-in process to update the GUI version (g32.exe), re-run the compiler with the same options as before, but modify the -b parameter portion to include the name of the editor executable immediately following -b (no spaces between -b and the editor executable name). For example, modify the -b portion to be -bg32.exe If you have renamed the editor executable, you will need to specify the name of the editor executable immediately following -b (no spaces between -b and the editor executable name). For example, if you have renamed the editor to MyRenamedEditor.exe, you will need to specify -bMyRenamedEditor.exe Note, however, that if the editor is running when you execute sc32, the compiler may not be able to update the editor program. If this is the case, the compiler will display the following: Unable to burn in interface - continuing... Create separate automatically-loaded interface file(s) instead? [Y/N] If you type "n", the compiler will terminate, and the interface file will [not](#) be installed. If you type "y", then the compiler will create separate interface file(s) in the current directory. (For these files to be effectively installed, they must be created in the same directory in which the editor program resides.) If you are compiling a ".ui" user-interface file, then a "tseui.dat" interface file will be created; and if the ".ui" file contains a configuration (Config/EndConfig) section, then a "tsecfg.dat" interface file will also be created containing the configuration settings from that section. If you are compiling a ".cfg" settings file, then a "tsecfg.dat" interface file will be created. These interface files are effectively installed in the editor, in that they will be automatically activated by the editor each time the editor is started; no further action is necessary. Note: If, when the editor is started, the date/time stamp on the editor program (e32.exe or g32.exe, as appropriate) is later than the date/time stamp any interface files ("tseui.dat" [or](#) "tsecfg.dat") located in the editor load directory, the editor will ignore those older files. Otherwise, if the date/time stamp on any interface files is later than the date/time stamp on the editor program, the interface files will automatically be loaded. Thus, the most recent configuration of the editor will be effective (regardless of whether it is burned in, or included in separate interface files.) -s[path] instructs the compiler to install a new ".ui" or ".cfg" interface file in the editor, by forcing creation of separate automatically-loaded interface file(s). The compiler does [not](#) even attempt to burn in the interface. The separate interface file(s) are placed in path, if specified, or in the current directory if path is [not](#) specified. (For these files to be effectively installed, they must be created in the same directory in which the editor program resides.) If you are compiling a ".ui" user-interface file, then a "tseui.dat" interface file will be created; and if the ".ui" file contains a configuration (Config/EndConfig) section, then a "tsecfg.dat" interface file will also be created containing the configuration settings from that section. If you are compiling a ".cfg" settings file, then a "tsecfg.dat" interface file will be created. These interface files are effectively installed in the editor, in that they will be automatically activated by the editor each time the editor is started; no further action is necessary. -? (or -h) instructs the compiler to display the compiler syntax. -v instructs the compiler to generate debug information in the resulting object (".mac") file. (For more information, see "[Debugging Macros](#)" later in this chapter.) -dsymbolic_constant[=integer_constant] instructs the compiler to define symbolic_constant and optionally assign it the value integer_constant. If integer_constant is not specified, a value of 1 is given to symbolic_constant. The -d switch is useful when you want to dynamically assign a value for a conditional compilation directive. (For more information, see "[Conditional Compilation Directives](#)" in the chapter on "[Macro Language Syntax](#)".) - source_filename is the name of the macro source file to compile. If

source_filename contains wildcards, all matching files are compiled. List files are supported using the "@" character. For example, to compile the three macros, MOE.S, LARRY.S, and CURLY.S, you could create a file called STOOGES and place the following lines in it: MOE.S LARRY.S CURLY.S Then run sc32: sc32 @stooges All files listed in STOOGES are then compiled. - object_filename is an optional parameter that specifies the desired name of the resulting object file, if you are compiling an external ".s" macro file. If not specified, the object (Compiled Macro) filename will be the same as the source filename, but with an extension of ".mac". The object file will be placed in the same directory as the source file, unless you specify a path for the object file.

Compiling Macros (Overview)

Once you have written the source code for a macro, it must be translated from the original English-like format, to a binary format (called an "object file") that is executable by the editor. This process is called "compiling." When you execute a macro file, it is the compiled version that the editor actually processes. Thus, it is necessary to compile the source for a macro before it can be executed. If you later make changes to the macro source code, you must compile it again before your changes will take effect.

[Compiling Macros from Inside the Editor](#) ----- To compile a macro file from within the editor, use the Compile command. To execute Compile, press <Ctrl F9>, or select "Compile" from the Macro Menu. If the compile completes successfully, the editor will pop up a selection menu prompting you to "Load" or "Execute" the macro file, or "Cancel" the command. Select "[Execute Macro](#)". You will then see the message "Hello, World!" appear on [the StatusLine](#). Congratulations! You have just written and compiled your first macro file! [Compiling Macros from Outside the Editor](#) ----- If, for some reason, you need to compile a macro file from outside the editor, you will need to run the SAL macro compiler program. The format for the previous example would be: sc32 hello This will create a Compiled Macro file called "HELLO.MAC".

Compiling Programs from Within the Editor

The Compile command provides a convenient tool for compiling source for a number of programming languages. The error-handling features of the Compile command assist you in locating source code errors for correction.

The section "

[Compiling Macro Source](#)," in the chapter on "[Creating and Using Macros](#)," describes the operation of the Compile command, specifically when used to compile external macros written in the editor's macro programming language (SAL). When used with other programming languages, Compile operates similarly. To execute Compile, press <Ctrl F9>. Or, you can press <Shift F9> to pop up the Compile menu, and then select "Compile Current File". When you execute Compile, the editor saves the current file (if modified) and attempts to execute the appropriate compiler, based on the filename extension of the current file. (If a compiler has not been pre-defined for the extension of the current file, the editor pops up a PickList of defined compilers from which you can choose an appropriate compiler.) For Compile to operate successfully, the compiler program you wish to execute must exist in the current directory or in a directory included in your PATH environment variable setting. Also, the compiler program must be one that has been defined in the editor (as described later in this section). The editor then invokes the indicated compiler program, using the current file as the source input. The output (executable) file is created according to the operation of the compiler program. If the compiler program encounters any statements for which it issues warning messages, the following occurs: - A temporary Errors buffer is created containing the error and/or warning messages issued by the compiler

program. - The editing screen is split horizontally, and the temporary Errors buffer is loaded into the new window. The cursor is placed on the first Error message in the new window. As you move through the Error messages, the source file scrolls so that the corresponding source line, as indicated by the Error message, is displayed in the source window.

- Special key assignments are activated to allow you to move through the Error messages, and to readily switch windows to go to any line in the source file that is identified in an Error message. In the Errors window, you can position the cursor on an Error message, then press <Enter> or <Escape> to switch to the source window and automatically position the cursor at the corresponding source line. Or, you can press <Alt F7> or <Alt F8> to move the cursor to the next or previous error (respectively) in either the Errors buffer or the source file, depending on the window in which you are located. To display a list of the special Errors [key](#) assignments and their associated functions, press <F1> from the Errors window, or press <Shift F9> and select "Help on Keys" from the Compile menu. In the source file window, you can make necessary corrections to the source. To switch back to the Errors window, use the window-changing commands. Once you have corrected your source, you can then re-execute the Compile command. If the compilation is successful, the Errors buffer is cleared. The pop-up Compile menu gives you a convenient way to perform the Compile options and Errors functions. To pop up this [menu](#), press <Shift F9>. From this menu, you can execute the Compile command, select the next or previous error, switch between the source and Errors windows (which re-opens the Errors window if it is currently closed), close the Errors window, and display Help on the special Errors key assignments. Two additional entries on the Compile menu allow you to access and maintain the compiler setup entries that are defined in the editor: "Compile from List" and "Debug Compiler Setup". The compiler setup entries identify the compiler programs that are available for execution by the Compile command, and dictate how the editor is to handle error and warning messages issued by the compiler programs. The selection "Compile from List" allows you to display a PickList of the compilers defined for the current filename extension. If no compilers are defined for the current extension, then all defined compiler setup entries are listed. You can select a compiler entry from this list for execution. The "Compile from List" option also allows you to change or delete existing compiler setup entries, or even add your own compiler entries. The compiler setup entries are maintained by filename extension. For each indicated filename extension, the compiler setup information specifies: - the command-line information (including command-line switches) necessary to execute the appropriate compiler; - a short description of the compiler setup entry; - the method for handling output messages from the compiler; and - the rules (or parameters) used by the editor to find and display the compiler output messages. For Help on adding or changing compiler setup entries, press <F1> from the list of setup entries. Once you have added or changed a compiler setup entry, you can use the debugging mode to work out any errors you may have in your compiler setup. First load (or create) a source file with the appropriate filename extension for the language compiler you are using. Then select "Debug Compiler Setup" on the Compile menu. A PickList of available compiler setup entries is displayed. Select the compiler entry that you want to debug. The editor invokes the indicated compiler, but as it encounters a compiler-generated output message, based on the parameters identified in your setup entry, the editor pauses. The error identification type (such as "error," "warning," [or](#) "note") issued by the compiler is displayed, along with the line number, column number, and filename of the erroneous source line. The compiler's error or warning message is also displayed. At any time during the debugging process, you can cancel the compiling operation or instruct the editor to complete compilation without further pausing. In addition to giving you access to your external language compiler from within the editor, a number of other features are included to help you when entering program source code. The chapter on "[Editing Modes](#)" describes [SyntaxHilite mode](#), [SmartIndent mode](#), [C mode](#), and Template mode, which are helpful for entering code for certain programming languages. The chapter on "[Search Features: Finding and Replacing Text](#)" describes the Match command (which allows you to search for the matching bracketing character for a group of opening/closing character pairs), and the FunctionList command (which lists all function or procedure declaration lines in a file). The chapter on "[Moving Through Text in a File](#)" describes the BegPara, EndPara, NextPara, and PrevPara commands, which move the cursor to the beginning or end of the current function or procedure, or to the next or previous function or procedure.

Conditional Compilation Directives

A conditional compilation directive is an advanced-usage feature that allows you to selectively compile portions of a macro based on condition(s) supplied

at compile time.

To use this feature, you define special

[constant](#) directives that are used to determine whether certain portions should be compiled. The value for each of the constant directives can be set statically (by setting the appropriate values in the code itself before compilation), or dynamically (by passing command-line switches to the SAL compiler program). To statically set a conditional compilation directive in the code itself, use the `#define` statement. The syntax for declaring a `#define` statement is: `#define identifier constant_numeric_expression` where: - `#define` declares the following identifier as a compilation directive. - identifier is any valid user-definable identifier. (For more information, see "[Identifiers](#)" earlier in this chapter.) - constant_numeric_expression is any numeric [constant](#) expression. (For more information, see "[Constants](#)" earlier in this chapter.) For example: `#define D40_VERSION 1 #define DEBUG TRUE` To dynamically assign a value to a conditional compilation directive, use the "-d" command-line switch for the SAL compiler program. (For more information on passing a command line to the SAL compiler, see "[Compiling Macros](#)" in the chapter on "[Using Macros](#)".) When the program is compiled, the value assigned to the directive (either by a `#define` statement in the macro source code, or by a value passed on the SAL compiler command line) is used to control conditional compilation of the code. The following statements are available for referencing the directives, as appropriate, within the macro source: `#ifdef identifier #ifndef identifier #ifexist filename #ifnexist filename #if numeric_expression #else #endif` The "`#ifdef`" and "`#ifndef`" statements are used to see if a conditional compilation directive has been defined. For example: `#define JOHNS_VERSION 1 #ifdef JOHNS_VERSION // normal SAL statements go here #endif #ifndef PAMS_VERSION // normal SAL statements go here #endif` In the above example, the statements under `JOHNS_VERSION` would be compiled, while the statements under `PAMS_VERSION` would not be compiled, since `PAMS_VERSION` is [not](#) defined. The "`#if`" statement is used to see if a conditional compilation directive evaluates to [TRUE](#) (non-zero) or [FALSE](#) (zero). The directive referenced via the "`#if`" statement must have been defined, or a compile-time error will be generated. For example: `#define MIKES_VERSION 0 #if MIKES_VERSION // normal SAL statements go here #endif #if RAYS_VERSION // normal SAL statements go here #endif` In the above examples, the statements under `MIKES_VERSION` would not be compiled, since the value of `MIKES_VERSION` is 0. The second "`#if`" test would generate a compile-time error, since `RAY_VERSION` has not been defined. As with the normal "`if`" statement, "`#else`" and "`#endif`" statements are available for use with the "`#if`" and "`#ifdef`" statements.

Configuration Options Available Interactively

The various configuration options are grouped into the following categories, which correspond to the pop-up menu selections available in IConfig: - [System/File Options](#) This category includes system and file-related options. - [Keyboard/Mouse Options](#) This category includes keyboard and mouse options. - [Command/Format Options](#) This category includes options that affect the behavior of certain commands, and that indicate the format of the date and time stamps. - [Text Entry/Block Options](#) This category includes options that affect the behavior of certain text-entry and [block operations](#). - [Tab/Margin Options](#) This category includes tab options, along with settings for the left and right editing margins. - [Print Output Options](#) This category includes printing options. - [Display/Color Options](#) This category includes options that affect the appearance of the screen display, and that indicate settings for the screen colors. The following sections describe the options that are available under each of the above categories. The editor's internal option name (that is, the variable name used by the editor's macro language) is indicated in parentheses to the immediate right of each option. The value of the standard setting for each option is also indicated to the far right of each option, preceded by "Std:" ("Standard"). NOTE: Certain options take effect only when the editor is restarted. These options are indicated by the character "*" preceding the option. Changes made to any of these options do [not](#) take effect until you save the settings, and then exit and restart the editor. [System/File Options](#) ----- * Path (s) for Supplemental Files ([TSEPath](#)) Std: empty Indicates additional drive/directory paths in which the editor is to search [for](#) supplemental files that are needed for certain commands. This includes: external Compiled Macro files (that

you specify when you execute the LoadMacro or ExecMacro commands, or that are needed by other operations that invoke [macros](#)), along with data files required by certain Compiled Macro files; the Help files required by [the HelpSystem](#); and the word list files used by SpellCheck. When this option is selected, a prompt box is displayed containing the message, "Supplemental file path(s):", along with the current designation (if any) for this option. The existing path(s) can be modified or removed. Multiple paths can be specified; however, no individual path can exceed 80 characters. To specify multiple paths, include a semicolon at the end of each path (except the final path). If multiple paths are specified, the editor searches in the order in which the paths are listed. The path(s) specified by TSEPath are used by the editor as part of its search procedure when attempting to locate supplemental files. The editor successively searches the following directories to locate a needed supplemental file: 1. The current directory 2. The drive/directory path(s) indicated by the TSEPath variable, as well as the following subdirectories (if they exist) of each directory indicated by [TSEPath](#): MAC\ for external Compiled Macro files and any required data files, SPELL\ for SpellCheck word-list files, and HELP\ [for](#) Help files 3. The editor load directory, as well as the subdirectories identified in 2. above (if they exist in the editor load directory) * Path for Keyboard Macro Files ([KbdPath](#)) Std: empty Indicates the drive and directory path in which Keyboard Macro files are stored. The indicated drive/directory is the directory to which Keyboard Macro files are written when no path is specified when you execute the SaveKeyMacro command. It is also the directory that the editor searches when no path is specified when you execute the LoadKeyMacro command, and the Keyboard Macro file could not be located in the current directory. When this option is selected, a prompt box is displayed containing the message, "Keyboard Macro path:", along with the current designation (if any) [for](#) this option. The existing path can be modified or removed. Only one drive/directory path can be specified. If this option is empty (which is the standard setting), the editor uses only the current directory. * *Empty Command-Line Action ([StartupFlags](#)) Std: [MENU](#) Indicates the file-selection behavior of the editor that will be in effect when the editor is started with no command-line parameters specified (that is, with no file specifications or editing options included on the command line). The available options are: Selection File-selection Behavior _____

_____ PROMPT Instructs the editor to issue the "File(s) to edit:" prompt so that a filename specification can be entered FILE MANAGER Instructs the editor to execute the FileManager command, passing it a file specification of "*.*)" RECENT Instructs the editor to display the Recent Files list This option valid only if "Persistent Recent Files", described under "[Command/Format Options](#)", is [ON](#) UNNAMED Instructs the editor to open an "unnamed" file, by executing the NewFile command PICKLIST Instructs the editor to display a PickList of files in the current directory RESTORE STATE Instructs the editor to restore a previous editing environment (provided environment information was saved for a previous session) MENU Instructs the editor to display a [selection menu](#) from which you can choose one of the above options or execute the Where command Also instructs the editor to re-display selection menu when you exit the editor Causes "EditFile Prompt on Quit" to have no effect (For more information on [starting the editor](#) without specifying any options or filenames, see "[Starting the Editor with No Command-Line Parameters](#)" in the chapter on "[Starting the Editor](#).") * Single Instance of the Editor (SingleInstance) Std: OFF If SingleInstance is Off, the editor will always start as a separate, non-shared instance for editing purposes. If SingleInstance is On, a single shared instance of the editor is started, if a shared instance of the editor is not running. If a shared instance of the editor is running, then the existing instance of the editor is used for editing purposes. * [Default](#) File Extensions ([DefaultExt](#)) Std: UI S SI C CPP H JAVA PAS INC BAT PRG TXT DOC HTML ASM Identifies the extensions used by the EditFile command to locate a file when a filename is specified without an extension (and does not end with a "."). The editor searches for a matching filename in the order in which the extensions are specified. When this option is selected, the editor displays the prompt, "[Default](#) file extensions:", and lists the current set of extensions. Make any changes or additions by typing extension value (without the preceding ".") separated by a space, and then press <Enter>. * File Locking ([FileLocking](#)) Std: NONE Indicates whether the editor will lock files that are loaded. Two types of file-locking are available: Read-Only and Handle. Valid selections for this option are: Selection Description _____ NONE Disables the editor's file-locking routine. READONLY Instructs the editor to lock each file by setting its read-only attribute. HANDLE Instructs the editor to lock each file by keeping its associated file handle open until the file is closed (that is, removed from the editor). (For more information on file-locking, see "[Locking Files](#)" in the chapter on "[Loading, Processing, and Saving Files](#).") * *Load Wild from Command Line ([LoadWildFromDOS](#)) Std: [ON](#) Toggles whether the editor selects all matching files when a filename containing wildcard characters ("*" or "?") is entered at

the command line. Set this option ON to have the editor select all matching files; set this option OFF to have the editor display a PickList of matching files from which you can choose. * Load Wild from Inside ([LoadWildFromInside](#)) Std: [OFF](#) Toggles whether the editor selects all matching files when a filename containing wildcard characters ("*" or "?") is entered at a "File(s) to edit:" prompt within the editor. Set this option ON to have the editor select all matching files; set this option OFF to have the editor display a PickList of matching files from which you can choose. * [File PickList Settings](#) Allows you to select a number of options for file-related PickLists. These options control the treatment of directories and the order of the entries in a PickList. Following are the configurable PickList options that are available on the [File PickList Settings](#) pop-up [menu](#):

- * Changes Directory ([PickFileChangesDir](#)) Std: [ON](#) Toggles whether the editor changes the current directory when a new directory is selected from a file PickList. Set this option ON to have the editor change the current directory to the directory selected from a PickList; set this option OFF to prevent the editor from changing the current directory when a different directory is selected from a PickList.
- * Force Showing Directories ([PickFileFlags](#)) Std: [ON](#) Toggles whether the editor includes all directories in the path that contains the files displayed in the current PickList. Set this option ON to have all directories included, regardless of the filename specification; set this option OFF to omit directories from the PickList, except those that match the filename specification. (This option has no effect on the operation of the Filename Completion option. For more information, see "[Obtaining a File PickList within a Prompt Box](#)" in the chapter on "[Command Prompts and PickLists](#).")
- * Show Directories at Top ([PickFileFlags](#)) Std: [ON](#) Toggles whether the editor displays any directory entries in the current PickList at the top of the PickList. Set this option ON to have all directories placed at the top of the PickList; set this option OFF to have any included directory entries located according to the sort order of the PickList.
- * Precede Directory Entries with Slash ([PickFileFlags](#)) Std: [ON](#) Toggles whether the editor places a slash at the beginning of any directory entries in the current PickList. Set this option ON to have a slash placed at the beginning of each directory entry; set this option OFF to prevent the editor from placing a slash at the beginning of each directory entry.
- * Sort Order ([PickFileSortOrder](#)) Std: ASCENDING Indicates whether file-related PickList entries will be displayed in ascending or descending order, according to the Sort Key that is selected. (See "Sort [Key](#)" below.) Toggle this option to ASCENDING or DESCENDING, as needed.
- * Sort [Key](#) ([PickFileSortOrder](#)) Std: NAME Indicates the sort key that will be used to display file-related PickList entries. (The sort order is determined by the "Sort Order" option above.) Sort [key](#) choices are: Selection Description _____ NAME
- Sort by full filename, including extension EXTENSION
- Sort by filename extension only SIZE
- Sort by file size DATE
- Sort by file date ATTRIBUTE
- Sort by file attributes
- * Line-Termination [String](#) ([EOLType](#)) Std: AS LOADED

Identifies the character string to be placed at the end of each line (except the last line) when the file is saved to disk. (See "File-Termination [String](#)" below.) Line-termination choices are: Selection Character(s) to Add to End of Each Line _____ AS LOADED Use the same termination character as is in the file when it is initially loaded

CR ONLY Add carriage [return](#) only

LF ONLY Add linefeed only

BOTH CR/LF Add carriage [return](#)/linefeed pair

If you select AS LOADED, the line termination string is determined by what is in the file itself when it is loaded into the editor. If ALL lines in the file are terminated by a linefeed character (LF), then the editor will terminate each line with a LF when the file is saved. If ALL lines in the file are terminated by a carriage return character (CR), then the editor will terminate each line with a CR when the file is saved. For all other files, the editor will terminate each line with a CR/LF when the file is saved.

- * File-Termination [String](#) ([EOFTYPE](#)) Std: EOL

[STRING](#) Identifies the optional character string to be placed at the end of the file [when](#) saved to disk. (See "Line-Termination [String](#)" above.) File-termination choices are: Selection Character(s) to Add to End of File _____

_____ NOTHING Do [not](#) add any termination characters

^Z ONLY Add Control-Z only

EOL [STRING](#) Add Line-Termination [String](#) only

EOL [STRING](#)/^Z Add Line-Termination [String](#) and Control-Z

- * Create Backups ([MakeBackups](#)) Std: [ON](#) Toggles whether the editor creates a backup before a file is saved. The backup is an image of the file as it is on disk before the copy in memory is written to disk. The name of the backup copy is the same as that of the file in memory, but with a file extension as specified by "Backup Extension [String](#)" (described later in this section). The backup copy is created in the same directory from which the file in memory was loaded. Set this option ON to create a backup copy of a file when the file is saved; set this option [OFF](#) if you do [not](#) want backup copies made. Note that this option takes precedence over the "Protected Saves" option (below). If "Create Backups" is set [ON](#), "Protected Saves" is ignored.
- * Protected Saves ([ProtectedSaves](#)) Std: [ON](#)

Toggles whether the editor saves files using its normal protected, safe method. Set this option ON for the editor to first write the file to disk under a temporary name, then rename the file to the original name once the temporary file is successfully created on disk (and optionally create a backup copy of the original file). Set this option OFF for files to be saved by directly overwriting the original file on disk (with no backup copy created). Note that the "Create Backups" option (above) takes precedence over this option. If "Create Backups" is set [ON](#), "Protected Saves" is ignored. * Backup Extension [String \(BackupExt\)](#) Std: BAK Indicates the extension to be used for backup files. The usual extension is "BAK". To specify a different backup extension, enter a 1- to 3-character extension value (without the preceding "."). * Backup Files Path ([BackupPath](#)) Std: empty Determines where backup files are written. BackupPath should be set to an existing path. If the path assigned does not exist, a backup file will not be written. * Save State on Exit ([SaveState](#)) Std: [OFF](#) Indicates whether the editor saves certain information about the current editing environment ("state") when you exit from the editor. The editing environment that is saved includes the file(s) that are loaded and the [window\(s\)](#) and file(s) that are displayed on the screen; the position of the cursor at the time the session is ended; the block (if any) that is marked; the bookmarks that are placed; and the settings of selected editing options (such as those on the "Options" Menu). This option is valid only when you terminate the current editing session using either the Exit (<Alt X>) or the SaveAllAndExit command. Set this option to ON or LOCAL for the editor to save environment information when you exit using one of these two commands. Editing environment information will be saved in a file TSESTATE.DAT. If set to ON, the TSESTATE.DAT file is saved in the editor load directory. If set to LOCAL, the TSESTATE.DAT file is saved in the current directory. Set this option [OFF](#) if you do [not](#) want environment information to be saved when you exit the editor. If environment information is saved, it can be restored in a subsequent editing session. (For more information on restoring the editing environment, see "[Saving/Restoring the Editing Environment](#)" in the chapter on "[Loading, Processing, and Saving Files](#).") * Use the Windows Common File Dialogs (UseCommonDialogs) Std: OFF By default, the editor uses its own style of prompts to request filenames. If this variable is set ON, the familiar Windows common file dialogs are used instead. (For more information on related topics, see the chapters on "[Starting the Editor](#)," "[Loading, Processing, and Saving Files](#)," and "[Creating and Using Macros](#).") [Keyboard/Mouse Options](#) ----- * Equate Enhanced Keyboard ([EquateEnhancedKbd](#)) Std: [ON](#) Toggles whether the editor is to equate the enhanced grey keys (grey Cursor Up/Down/Left/Right, Page Up/Down, Home, End, Insert, and Delete) to the equivalent "normal" keys (on the numeric keypad). For this option to be effective, you must have an enhanced keyboard, such as a 101-key keyboard, (and the option "Use Enhanced Keyboard" must be ON in the DOS version). Set "Equate Enhanced Keyboard" ON if you have an enhanced keyboard and want the enhanced keys to operate identically to the normal keys; set this option OFF if you have an enhanced keyboard and want the editor to distinguish between the enhanced keys and the normal keys (to allow assignment of different commands to the two sets of keys). * Mouse Hold Time ([MouseHoldTime](#)) Std: 9 Indicates the amount of time a mouse button must be held down to distinguish between the mouse button being clicked (pressed and immediately released) versus being pressed and held. When a mouse button is clicked, a command executed from a mouse scrollbar is executed one time only. When a mouse button is pressed and held, the command is executed repeatedly until the mouse button is released. (See "Mouse Repeat Delay" below.) To select the amount of time that you must hold down the mouse button before the editor begins repeating a mouse-executed command, set "Mouse Hold Time" to the desired value, from 1 through 36. The time is measured in terms of machine timer ticks, in 1/18-second increments. For example: A value of 9 means that the mouse button is considered clicked if pressed and released within 1/2 second; otherwise, if the mouse button is pressed and held for longer than 1/2 second, the editor begins repeating the selected command. * Mouse Repeat Delay ([MouseRepeatDelay](#)) Std: 0 Indicates the relative amount of time the editor is to wait between repetitions of a command when a mouse button is pressed and held. (See "Mouse Hold Time" above.) When a mouse button is pressed and held, a command executed from a mouse scrollbar is repeatedly executed as long as the mouse button is held down. To control the amount of time that the editor delays between each repetition of a mouse-executed command, set Mouse Repeat Delay to an appropriate value, from 0 through 65535. This option is hardware dependent, and is relative to the speed of your particular machine. To slow down repetition, increase the value of Mouse Repeat Delay; to speed up repetition, decrease the value. (For more information on [using a mouse](#) with the editor, see the chapter on "[Mousing Around the Editor](#).") [Command/Format Options](#) ----- * Max History Per List ([MaxHistoryPerList](#)) Std: 50 Indicates the maximum number of History entries to be stored in the History Buffer for each History list, ranging from 1 to 1000. This number is the

maximum number of History items that are maintained for each particular command, or command group, prompt. Note that the total number of History entries, for all command/command groups, is limited to the maximum number of History entries as specified by "Max History Entries" (described below). If the "Persistent History" option is ON (described later in this section), the items stored in the History Buffer will be maintained across editing sessions. The History Buffer then has the potential to grow quite large. "Max History Per List," along with "Max History Entries" (described below), can be used to limit the History Buffer to a manageable size. * Max History Entries ([MaxHistorySize](#)) Std: 1000 Indicates the maximum number of History entries to be stored in the History Buffer, ranging from 1 to 10,000. This number is the total number of History items for all command prompts. The number of History entries that are stored for each particular command prompt may also have an effect on the total number of History entries that are maintained. See "Max History Per List" (described above). Note that setting "Max History Entries" to a smaller number during an editing session does not immediately remove entries already in the History Buffer. Thus, the effect of decreasing "Max History Entries" may not be realized until the editor is restarted. If the "Persistent History" option is ON (described later in this section), the items stored in the History Buffer will be maintained across editing sessions. The History Buffer then has the potential to grow quite large. "Max History Entries," along with "Max History Per List" (described above), can be used to limit the History Buffer to a manageable size. * Persistent History ([PersistentHistory](#)) Std: [ON](#) Toggles whether the entries stored in the History Buffer are maintained across editing sessions. Set this option ON for entries in the History Buffer to be saved at the end of each editing session, and then reloaded into the History Buffer at the start of the next editing session; set this option OFF for the History Buffer to be cleared at the end of each editing session. If "Persistent History" is ON, History Buffer items are maintained in the file TSEHIST.DAT in the editor load directory. The maximum number of items that are kept in this file is governed by the settings of "Max History Entries" [and](#) "Max History Per List" (described earlier in this section). * Max Recent Files ([MaxRecentFiles](#)) Std: 200 Indicates the maximum number of files to be stored in the Recent Files list, ranging from 1 to 5000. This number is the total number of filenames that will be maintained and displayed by the editor when you execute the ListRecentFiles command. If the "Persistent Recent Files" option is ON (described later in this section), the items stored in the Recent Files list will be maintained across editing sessions. "Max Recent Files" can be used to limit the size of the Recent Files list. * Persistent Recent Files ([PersistentRecentFiles](#)) Std: [ON](#) Toggles whether the entries stored in the Recent Files list are maintained across editing sessions. Set this option ON for entries in the Recent Files list to be saved at the end of each editing session, and then reloaded into the Recent Files list at the start of the next editing session; set this option OFF for the Recent Files list to be cleared at the end of each editing session. If "Persistent Recent Files" is ON, entries in the Recent Files list are maintained in the file TSEFILES.DAT in the editor load directory. The maximum number of items that are kept in this file is governed by the setting of "Max Recent Files" (described earlier in this section). * [WordSet](#) Characters ([WordSet](#)) Std: 0-9A-Z_a-z Identifies the characters by which the editor defines a word for word-sensitive commands, such as WordRight, WordLeft, DelLeftWord, DelRightWord, FindWordAtCursor, MarkWord, and the "Words" option of Find and Replace. The [WordSet](#) specification is [case](#) sensitive. When this option is selected, a prompt box is displayed containing the message, "[WordSet](#) characters:", along with the current specification. You can modify the existing set, or enter a whole new set, according to the following guidelines: - Any of the 256 ASCII characters can be designated as part of the [WordSet](#). - Do not enter a space between characters, unless you want to include the space character as a WordSet character. - To indicate a range of sequential characters, enter the first character in the range, immediately followed by "-", and then enter the last character in the range. (For example, to indicate the lower-case letters of the alphabet, enter: a-z) - To include the dash character ("-") as a WordSet character, it must be listed as the first or last character within the set. - The back-slash ("\") character is used as an escape character. To include the back-slash character in the set, use two back slashes ("\\"). - The regular-expression notations for decimal and hexadecimal [numbers](#) can be used in a WordSet specification, as "\dnnn" (where "nnn" is a decimal number) or "\xnn" (where "nn" is a hexadecimal number). - The tilde ("~") character, if listed as the first character in the WordSet specification, is used to indicate that the remainder of the specification contains all the characters that are NOT part of the WordSet. That is, the WordSet consists of all characters that are NOT listed in the specification. * Initial Find Options ([FindOptions](#)) Std: I Identifies the initial option(s) for the Find command. A maximum of six options can be specified. These options are placed in the Find options prompt box the first time the Find command is executed in each editing session, unless "Persistent History" (described earlier in this section) is [ON](#). These options are effective only when

there are no History entries for the Find options prompt. The "Initial Find Options" are also used as the effective options for the FindWordAtCursor command. (For more information on the valid Find options, see the chapter on "[Search Features: Finding and Replacing Text](#).") * Initial Replace Options ([ReplaceOptions](#)) Std: empty Identifies the initial option(s) for the Replace command. A maximum of six options can be specified. These options are placed in the Replace options prompt box the first time the Replace command is executed in an editing session, unless "Persistent History" (described earlier in this section) is [ON](#). These options are effective only when there are no History entries for the Replace options prompt. (For more information on the valid Replace options, see the chapter on "[Search Features: Finding and Replacing Text](#).") * Center Found Text ([CenterFinds](#)) Std: [OFF](#) Toggles whether the line containing the text located by the Find or Replace commands is centered vertically in the current window. Set this option ON to center the line containing the "found" text. Set this option OFF to scroll the line containing the "found" text to the current cursor line (leaving the cursor line position unchanged relative to the screen). * [Beep](#) on Failed Find ([Beep](#)) Std: [ON](#) Toggles whether the editor sounds a beep when the specified search string is not located by the Find or Replace commands. Set this option ON for a beep to sound when a string is not located; set this option OFF to suppress the [beep](#). * Change Prompt in Shell ([ChangeShellPrompt](#)) Std: [ON](#) Toggles whether the editor changes the command-line prompt string when the Shell command is executed. Set this option ON to have the notation "TSE-Shell" added to the current prompt string when Shell is executed. Set this option OFF to leave the prompt string unchanged. * EditFile Prompt on Quit ([QuitToPrompt](#)) Std: [OFF](#) Toggles whether the editor issues the "File(s) to edit:" prompt after closing the last file following execution of the QuitFile or SaveAndQuitFile commands. Set this option ON to have the "File(s) to edit:" prompt displayed after the last file is closed via the QuitFile or SaveAndQuitFile commands. Set this option OFF to have the editor exit to the operating system (or the calling program) after the last file is closed. NOTE: If "Empty Command-Line Action" is set to [MENU](#), then this option has no effect. * Keep Undo Beyond Save ([KeepUndoBeyondSave](#)) Std: [OFF](#) Toggles whether the editor purges Undo/Redo information for a file after the SaveFile command is executed for that file. Set this option OFF to have the Undo/Redo information purged when a file is saved; thus, you cannot restore any changes made before that file was saved. Set this option ON to have the Undo/Redo information maintained even after a file is saved; thus, you can restore changes made before that file was saved. * Date Format ([DateFormat](#)) Std: MM-DD-YYYY Indicates the format of the current date stamp. Six formats are available: MM-DD-YYYY, DD-MM-YYYY, YYYY-MM-DD, MM-DD-YY, DD-MM-YY, or YY-MM-DD (where "MM" represents the two-character current month, "DD" represents the two-digit current day of the month, "YYYY" represents the four-digit current year, and "YY" represents the last two digits of the current year). This option affects the format of the date from the DateTimeStamp command, as well as the format of the date in the PrintHeader and PrintFooter print options. * Date Separator ([DateSeparator](#)) Std: / Indicates the character to use as the separator between the three parts of the current date. (For example, in the date specification "01/23/57", the separator is the "/" character.) Any character can be selected as the date separator. This option affects the format of the date from the DateTimeStamp command, as well as the format of the date in the PrintHeader and PrintFooter print options. * Time Format ([TimeFormat](#)) Std: 12-HOUR Indicates the format of the current time stamp. Two formats are available: 12-HOUR or 24-HOUR. 12-HOUR time is in the format "HH:MM Xm" (where "HH" represents the hour from 01 through 12, "MM" represents the minutes from 00 through 59, and "Xm" represents the morning or evening as "am" or "pm", respectively). 24-HOUR time is in the format "HH:MM:SS" (where "HH" represents the hour from 00 through 23, "MM" represents the minutes from 00 through 59, and "SS" represents the seconds from 00 through 59). This option affects the format of the time from the DateTimeStamp command, as well as the format of the time in the PrintHeader and PrintFooter print options. * Time Separator ([TimeSeparator](#)) Std: : Indicates the character to use as the separator between the two parts of the current time. (For example, in the time specification "02:15 pm", the separator is the ":" character.) Any character can be selected as the time separator. This option affects the format of the time from the DateTimeStamp command, as well as the format of the time in the PrintHeader and PrintFooter print options. (For more information on related topics, see the chapters on "[Command Prompts and PickLists](#)," "[Loading, Processing, and Saving Files](#)," "[Moving Through Text in a File](#)," "[Manipulating Text](#)," "[Block Operations](#)," "[Search Features: Finding and Replacing Text](#)," "[Printing from the Editor](#)," and "[Special Commands and Features](#).") [Text Entry/Block Options](#) ----- * [Insert Mode](#) ([Insert](#)) Std: [ON](#) Toggles [Insert mode ON](#) or [OFF](#). (For more information on [Insert mode](#), see the chapter on "[Editing Modes](#).") * [AutoIndent Mode](#) ([AutoIndent](#)) Std: [ON](#) Indicates the state of AutoIndent mode, which affects placement of the [cursor](#) when

CReturn (<Enter>) is executed. Three options are available: OFF, ON, or STICKY. (For more information on AutoIndent mode, see the chapter on "[Editing Modes](#).") * [WordWrap Mode \(WordWrap\)](#) Std: [OFF](#) Indicates the state of WordWrap mode, which affects wrapping of text as characters are inserted or deleted. Three options are available: [OFF](#), [ON](#), or AUTO. (For more information on WordWrap mode, see the chapter on "[Editing Modes](#).") * [Template Expansion \(TemplateExpansion\)](#) Std: [OFF](#) Indicates the state of Template mode, which affects expansion of pre-defined templates. When set ON, the <Tab> and <Ctrl Tab> keys are configured to operate in Template mode. <Tab> expands a template-defined keyword into its pre-assigned text. (If the cursor is not positioned at the end of a Template-defined keyword when you press <Tab>, the <Tab> key behaves as it is normally configured.) Pressing <Ctrl Tab> pops up the Template [menu](#). When set OFF, the <Tab> and <Ctrl Tab> keys behave as they are normally configured. (For more information on Template mode, see the chapter on "[Editing Modes](#).") * [Paragraph-End Style \(ParaEndStyle\)](#) Std: BLANK LINE Identifies the style used in the text to indicate the end of a paragraph. This determines when the WrapPara command stops formatting text. The two recognized paragraph-end styles are: Selection Description _____ BLANK LINE Paragraphs are separated by blank line(s). WrapPara stops formatting text when a blank line (or the end of the file) is encountered following the text being formatted. EQUAL IN/OUTDENT Paragraphs are separated by indenting or outdenting the first line of each paragraph at a particular column, and/or by blank line(s). WrapPara stops formatting text when a subsequent line is encountered on which text begins in the same column as the first line of text being formatted, or until a blank line is encountered. If the text on the first line is not indented or outdented with respect to the line immediately following it (that is, it begins in the same column as the text on the following line), then the end of the paragraph is not assumed until a blank line is encountered. This behavior is the same as that for the BLANK LINE selection. * CReturn Equals Next Line ([ReturnEqNextLine](#)) Std: [OFF](#) Toggles whether the CReturn command moves the cursor to the next line without splitting the line when Insert mode is ON. Set this option ON for CReturn to simply move the cursor to the next existing line of text even [when Insert mode](#) is [ON](#). Set this option [OFF](#) for CReturn [to](#) behave according to its native behavior when Insert mode is ON; that is, by splitting the line at the cursor position and moving the cursor, along with the text at and to the right of the cursor, to a new line immediately following the current line. This option has no effect on CReturn when [Insert mode](#) is [OFF](#). When [Insert](#) mode is OFF, CReturn always moves the cursor to the next line without splitting the line, regardless of the setting of this option. * [Remove Trailing Whitespace \(RemoveTrailingWhite\)](#) Std: [ON](#) Toggles whether the editor removes all white (space and tab) characters located beyond the last non-white character on the cursor line. Set this option ON to have the editor remove trailing white characters on any line to which the cursor moves; set this option OFF to retain any trailing white characters on the cursor line. * [Line Type \(LineDrawType\)](#) Std: SINGLE Identifies the line-drawing style to be used when LineDrawing mode is ON. The available styles are: Selection Line-Drawing Characters _____ SINGLE +-+ Single-line top and side DOUBLE TOP *** Double-line top, single-line side DOUBLE SIDE *-* Double-line side, single-line top DOUBLE BOTH *** Double-line both top and side ASCII +-| ASCII top and side ERASER Blank USER-DEFINED [*] User-defined top and side If you select USER-DEFINED, you then input the character you wish to use, within []. (The numeric keypad or the pop-up AsciiChart can be used to input a character that is not directly available from the keyboard.) * [Insert Line Blocks Above \(InsertLineBlocksAbove\)](#) Std: [OFF](#) Toggles whether the editor inserts line blocks above the current line. Set this option ON for line blocks to be inserted ABOVE the current line; set this option OFF for line blocks to be inserted BELOW the current line. (Note that this option [affects](#) only line blocks, and [not](#) character or column blocks.) * [Use Current Line If No Block \(UseCurrLineIfNoBlock\)](#) Std: [OFF](#) Toggles whether the editor treats the current line as a line block when the Clipboard Cut/Copy, the CopyToWinClip, or the CutToWinClip commands are executed AND no block is marked in any file. Set this option ON for the current line to be treated as a block for the Clipboard Cut/Copy, CopyToWinClip, and CutToWinClip commands if no block is marked; set this option [OFF](#) for Clipboard Cut/Copy, CopyToWinClip, and CutToWinClip [to](#) operate only on a block marked in the current file. The Clipboard Cut/Copy, CopyToWinClip, and CutToWinClip commands fail regardless of this option's setting if a block is marked in a file other than the current file. * [UnMark After Paste \(UnMarkAfterPaste\)](#) Std: [ON](#) Toggles whether the editor unmarks text when placed into a file from a Clipboard via the Paste command. Set this option ON for the text to be unmarked once it is placed into a file with the Paste command; set this option OFF for the text to remain marked following Paste. (For more information on related topics, see the chapters on "[Moving Through Text in a File](#)," "[Manipulating Text](#),"

"[Editing Modes](#)," "[Block Operations](#)," and "[Special Commands and Features](#).") [Tab/Margin Options](#) ----- *

Tab Type ([TabType](#)) Std: SOFT Indicates the type of tab to be used in the editor. Four options are available: HARD, SOFT, SMART, or VARIABLE. * Tab Width ([TabWidth](#)) Std: 8 Identifies the fixed interval (TabWidth) for placement of tab stops with [Soft and Hard Tabs](#). Valid [TabWidth values](#) range from 1 through 16. * [Cursor](#) Tab Width ([CursorTabWidth](#)) Std: 0 If [CursorTabWidth](#) is non-zero, it is used by tabbing commands in lieu of [TabWidth](#). On-screen display of spaces for tabs is still controlled by [TabWidth](#). If [CursorTabWidth](#) is 0, [TabWidth](#) is used by the tabbing commands, and CursorTabWidth has no effect. * Zero-Based Tab Stops ([ZeroBasedTab](#)) Std: [OFF](#) Toggles ZeroBasedTab ON or OFF. This determines the offset used to compute Soft and Hard Tab stops. Set this option ON to have the tab stops based on column 0; set this option OFF to have the tab stops based on column 1. (For more information, see "[ZeroBasedTab](#)" under "[Soft and Hard Tabs](#)" in the chapter on "[Using Tabs](#).") * Variable Tab Stops ([VarTabs](#)) Std: 4 8 16 24 40 Identifies the tab stop locations used with Variable Tabs. Tab stops are designated by specifying the actual columns at which to place tab stops, and may occur at variable intervals. The maximum variable tab stop location is column 255. When this option is selected, a prompt box is displayed containing the message, "Variable Tab stops:", along with a listing of the current set of Variable Tab stops. You can modify the existing set, or enter a whole new set, by typing the appropriate column number(s), separated by spaces. * Expand Tabs (Display Only) ([ExpandTabs](#)) Std: [ON](#) Toggles ExpandTabs ON or OFF. This dictates whether the editor displays tabs as the appropriate number of spaces, as determined by the value of TabWidth. Set this option ON to have the editor interpret tab characters so that the text is positioned on the screen according to the current TabWidth. Set this option OFF to have the editor display each tab character on the screen as the physical tab character itself (ASCII 9). Note, however, that [ExpandTabs](#) has no effect on any line longer than 255 characters. ExpandTabs affects only the screen display of tab characters; it does not affect how tab characters are stored in the file itself. If ExpandTabs is ON and the file is subsequently saved, the actual tab characters are stored in the file, and NOT the spaces that displayed on the screen. * Detab files when loading ([DetabOnLoad](#)) Std: [OFF](#) If ON, tabs are converted to the appropriate number of spaces when a file is loaded into the editor. Tabs are expanded based on the value of the TabWidth editor variable. * Entab files when saving ([EntabOnSave](#)) Std: [OFF](#) If ON, spaces are converted to tabs (as appropriate) when a file is saved. Spaces are converted to tabs based on the TabWidth editor variable. * Left Margin ([LeftMargin](#)) Std: 0 Indicates the LeftMargin setting for editing. The value (if any) specified may affect the operation of the CReturn and WrapPara commands, as well as WordWrap mode. Valid values for LeftMargin range from 0 through the maximum allowable line length. The setting of AutoIndent may affect whether the editor recognizes the [LeftMargin](#) setting. * Right Margin ([RightMargin](#)) Std: 72 Indicates the RightMargin setting for editing. The value (if any) specified affects the operation of the WrapPara command and of WordWrap mode. Valid values for RightMargin range from 0 through the maximum allowable line length. A value of 0 for this option is treated the same as a value of 1. (For more information on related topics, see the chapters on "[Using Tabs](#)," "[Manipulating Text](#)," and "[Editing Modes](#).") [Display/Color Options](#) -----

* StatusLine At Top ([StatusLineAtTop](#)) Std: [ON](#) Toggles whether the editor displays [the StatusLine](#) at the top of the screen. Set this option ON for [the StatusLine](#) to display at the top of the screen; set this option OFF for [the StatusLine](#) to display at the bottom of the screen. [The HelpLine](#), if active, is placed on the screen on the side opposite the StatusLine. (See "Display HelpLine" later in this section.) * StatusLine Fill Character ([StatusLineFillChar](#)) Std: space Identifies the optional character used to fill the unused space on the StatusLine. *

Display [Main Menu](#) ([ShowMainMenu](#)) Std: [ON](#) Toggles whether the editor continually displays the main pull-down Menu. Set this option ON to have the main Menu remain on the screen throughout the editing session. The main Menu will be displayed at the top of the screen. Set this option OFF if you do not want the main Menu to remain on the screen throughout the editing session. * Display HelpLine ([ShowHelpLine](#)) Std: [ON](#) Toggles whether the editor displays [the HelpLine](#) bars. [The HelpLine](#) is displayed at either the top or bottom of the screen, opposite the StatusLine. (See "StatusLine At Top" earlier in this section.) * HelpLine Delay ([HelpLineDelay](#)) Std: 6 Indicates the amount of time a "shift" (<Shift>, <Alt>, or <Ctrl>) key must be held down before the editor updates [the HelpLine](#) with the associated HelpLine bar. This option is only valid if "Display HelpLine" (described above) is [ON](#). To select the amount of time that you must hold down the "shift" key before [the HelpLine](#) is updated, set "HelpLine Delay" to the desired value, from 0 through 18. The time is measured in 1/18-second increments. For example: A value of 0

means that [the HelpLine](#) will be updated immediately (with no delay) once a "shift" key is pressed; a value of 18 means there will be approximately a 1-second delay. * Current Window Divider ([CurrWinBorderType](#)) Std: DOUBLE TOP Identifies the type of divider line that is used to separate the current window from any other windows on the screen. Divider choices for the current window are: Selection Window Dividers _____

_____ SINGLE +-+ Single-line top and side DOUBLE TOP *** Double-line top, single-line side DOUBLE SIDE *-* Double-line side, single-line top DOUBLE BOTH *** Double-line both top and side THIN SOLID *** Thin solid-line top and side THICK SOLID *** Thick solid-line top and side ASCII +-+ ASCII characters (+, -, |) 3D 3D characters are used If the screen is split into multiple windows, the "top" divider separates a horizontal current window, and the "side" divider separates a vertical current window. The selected top AND side dividers are used to display a FULL border around the current window, if either: - you have the editor configured to display a full border around each window on the screen, or - the current window is zoomed. (For information about displaying a full border around each window, see "Display Border Around [Windows](#)" later in this section.) * Non-Current Window Divider ([OtherWinBorderType](#)) Std: DOUBLE TOP Identifies the type of divider line that is used to separate any non-current windows from other windows on the screen. Divider choices for non-current windows are the same as those for "Current Window Divider" (above). If the screen is split into multiple windows, the "top" divider separates each horizontal non-current window, and the "side" divider separates each vertical non-current window. The selected top AND side dividers are used to display a FULL border around each non-current window, provided you have the editor configured to display a full border around each window on the screen. (For information about displaying a full border around each window, see "Display Border Around [Windows](#)" below.)

* Display Border Around [Windows](#) ([DisplayBoxed](#)) Std: MOUSE Indicates whether the editor displays a full border around the entire screen, so that each window (including a single window) is fully "boxed." The setting of this option has no effect on the window display when a window is zoomed. If you change the setting for this option while the screen is split into multiple windows, the editor will close all but one window and make the new setting effective. If you are [using a mouse](#), note that mouse scrollbars in the editor are displayed within the window border. Therefore, to use the mouse scrollbars, it is necessary that a full border be displayed around each window, by setting this option to either MOUSE or ALWAYS (as described below). Valid selections for this option are: Selection Description

_____ NEVER Instructs the editor to never display the outside screen border. MOUSE Instructs the editor to display the outside screen border only if a mouse is present on your system. If a mouse is not present, the editor does not display the outside border. ALWAYS Instructs the editor to always display the outside screen border. * [Cursor](#) Size: [Insert](#) ([InsertCursorSize](#)) Std: 4 Indicates the size of the cursor when the editor is in Insert mode. The size is specified by selecting a value from 1 through 8, 1 being the smallest and 8 the largest. * [Cursor](#) Size: Overwrite ([OverwriteCursorSize](#)) Std: 2 Indicates the size of the cursor when the editor is in overwrite mode. The size is specified by selecting a value from 1 through 8, 1 being the smallest and 8 the largest. * Show End-of-File Marker ([ShowEOFMarker](#)) Std: [ON](#) Toggles whether the editor displays a marker indicating that the end of the file has been reached. Set this option ON to have the following line appear after the last line of the file to signal the file's end: <*** End of File ***> Set this option OFF to prevent the marker from displaying at the end of the file. * Show Line [Numbers](#) ([ShowLineNumbers](#)) Std: [OFF](#) Toggles whether line numbers are displayed on the left side of the editing window. * *Startup Video Mode ([StartupVideoMode](#)) Std: AUTODETECT Indicates the number of lines and columns to be displayed on the screen (that is, the video mode in effect) when the editor is invoked. Depending on your screen resolution and current font, you can select from the following choices: Selection Video Mode _____

_____ AutoDetect Instructs the editor to use the same mode in effect at editor startup 25 Lines by 80 Cols 25 lines by 80 columns 28 Lines by 80 Cols 28 lines by 80 columns 30 Lines by 80 Cols 30 lines by 80 columns 30 Lines by 90 Cols 30 lines by 90 columns 30 Lines by 94 Cols 30 lines by 94 columns 33 Lines by 80 Cols 33 lines by 80 columns 34 Lines by 90 Cols 34 lines by 90 columns 34 Lines by 94 Cols 34 lines by 94 columns 36 Lines by 80 Cols 36 lines by 80 columns 40 Lines by 80 Cols 40 lines by 80 columns 40 Lines by 90 Cols 40 lines by 90 columns 40 Lines by 94 Cols 40 lines by 94 columns 43 Lines by 80 Cols 43 lines by 80 columns 44 Lines by 80 Cols 44 lines by 80 columns 50 Lines by 80 Cols 50 lines by 80 columns Max Lines/Cols Maximum lines/columns supported by current screen resolution and font Custom Lines/Cols User-specified lines and columns To change the current video mode in effect during an editing session, change CurrVideoMode. (For more information, see "[Selecting the Video Mode](#)" in the chapter on "[Special](#)

[Commands and Features.](#)") * [Display SyntaxHilite Colors \(ShowSyntaxHilite\)](#) Std: [ON](#) Toggles whether SyntaxHilite colors are displayed for any files for which [SyntaxHilite mode](#) is active. Set this option [ON](#) to display SyntaxHilite colors for any files for which [SyntaxHilite mode](#) is active; set this option OFF to suppress the display of SyntaxHilite colors for all files in the editing session. (For more information on [SyntaxHilite mode](#), see "[SyntaxHilite Mode](#)" in the chapter on "[Editing Modes](#).") * [Configure SyntaxHilite Mapping Sets](#) Allows you to configure the mapping sets to be used by SyntaxHilite mode. When you select this item, the following options are available: Selection Description

[CURRENT](#) Selects the SyntaxHilite mapping set for the current file, and allows you configure that set. [OTHER](#) Displays a PickList of SyntaxHilite mapping files, from which you can select a mapping set to configure. [CREATE NEW](#) Creates a new SyntaxHilite mapping file, so that you can configure a new mapping set. [CONFIGURE](#) Displays a modifiable list of available SyntaxHilite ASSOCIATIONS mapping (.SYN) files, from which you can maintain the file extension associations. (For more information on configuring SyntaxHilite mapping sets, see "[Mapping SyntaxHilite Tokens to Colors & Setting Other Options](#)" and "[Associating a SyntaxHilite Mapping Set with a Group of Files](#)" under "[SyntaxHilite Mode](#)" in the chapter on "[Editing Modes](#).") * [Set Colors](#) Allows you to change the configurable screen display colors. Three groups of colors can be customized: editing colors, HelpSystem colors, and [SyntaxHilite mode](#) colors. A color-customization screen displays to allow you to select the desired colors. Your color selections are shown on a sample portion of a screen as you move through the available color patterns. From "[Set Colors](#)," you select the color group to configure, and the appropriate color-customization Item Selection screen is displayed. On the left side of this screen is a PickList of all the configurable items. On the right side of the screen is a sample screen that illustrates your current color settings. Following are the configurable items on the Item Selection screen for "editing" colors, accessed via the "Select Editing Colors" option: * [Text - Normal \(TextAttr\)](#) Std: Intense Black on White Indicates the color of normal text (that is, text that is not on the cursor line, and is not highlighted or in a marked block). * [Text - Blocked \(BlockAttr\)](#) Std: Bright White on Black Indicates the color of the text within a marked block. * [Text - Cursor Line in Block \(CursorInBlockAttr\)](#) Std: Black on White Indicates the color of the text on the cursor line when located within a marked block. * [Text - Cursor Line \(CursorAttr\)](#) Std: Intense Black on Cyan Indicates the color of the text on the cursor line (when not located within a marked block). * [Text - Highlighted \(HiLiteAttr\)](#) Std: Red on White Indicates the color of the text that is highlighted when located by a search command. * [Menu/Help - Item Names \(MenuTextAttr\)](#) Std: Black on White Indicates the color of the selection item names listed on the pull-down Menu, including the item names under the sub-menu categories; also determines the color of the command/option names included on [the HelpLine](#). * [Menu/Help - Quick Letter \(MenuTextLtrAttr\)](#) Std: Red on White Indicates the color of the highlighted quick-key letters for the items listed on the pull-down Menu, including the quick-keys for the sub-menu items; also determines the color of the key assignment values included on [the HelpLine](#). * [Menu - Grayed \(MenuGrayAttr\)](#) Std: Bright Black on White Indicates the color of disabled "grayed" selection items on the pull-down Menu and on the sub-menu categories. * [Menu - Selected Gray \(MenuSelectGrayAttr\)](#) Std: White on Blue Indicates the color of the Menu cursor bar when it is located on a disabled "grayed" item on the pull-down Menu or on a sub-menu category. * [Menu - Border \(MenuBorderAttr\)](#) Std: Black on White Indicates the color of the border around the pull-down Menu categories. * [Menu - Selected Item \(MenuSelectAttr\)](#) Std: Bright White on Blue Indicates the color of the Menu cursor bar that identifies the selected item on the pull-down Menu and on the sub-menu categories. * [Menu - Selected Quick Letter \(MenuSelectLtrAttr\)](#) Std: Bright Red on Blue Indicates the color of the quick-key letter (within the Menu cursor bar) for the selected item on the pull-down Menu and on the sub-menu categories. * [Border - Current Window \(CurrWinBorderAttr\)](#) Std: Intense Black on White Indicates the color of the border around the current window. * [Border - Non-Current Window \(OtherWinBorderAttr\)](#) Std: Intense Blue on White Indicates the color of the border around the non-current [window\(s\)](#). * [End-of-File Marker \(EOFMarkerAttr\)](#) Std: Black on White Indicates the color of the optional line that identifies the end of the file. * [Messages \(MsgAttr\)](#) Std: Black on White Indicates the color of various messages and warnings displayed on the StatusLine. * [StatusLine \(StatusLineAttr\)](#) Std: Bright White on Blue * [LineNumbers \(LineNumbersAttr\)](#) Std: Black on White Indicates the color of displayed Line Numbers (see [ShowLineNumbers](#)). Following are the configurable items on the Item Selection screen for HelpSystem colors, accessed via the "Select Help Colors" option: * [Help Text - Normal \(HelpTextAttr\)](#) Std: White on Blue Indicates the

color of normal text within [the HelpSystem](#). * Help Text - Highlighted ([HelpBoldAttr](#)) Std: Bright White on Blue Indicates the color of highlighted text within [the HelpSystem](#). * Help Text - Italicized ([HelpItalicsAttr](#)) Std: Bright Magenta on Blue Indicates the color of "italicized" text within [the HelpSystem](#). * Help Field - InfoBox ([HelpInfoAttr](#)) Std: Bright Green on Blue Indicates the color of HelpSystem InfoBox fields. * Help Field - Link ([HelpLinkAttr](#)) Std: Bright Yellow on Blue Indicates the color of the fields that serve as Links to other HelpSystem topics. * Help Field - Selected ([HelpSelectAttr](#)) Std: Blue on White Indicates the color of the field that is selected in [the HelpSystem](#).

Following are the configurable items on the Item Selection screen for SyntaxHilite mode colors, accessed via the "Select SyntaxHilite Colors" option. (For more information on colors used by [SyntaxHilite mode](#), see "[Selecting Colors for SyntaxHilite Classes](#)" under "[SyntaxHilite Mode](#)" in the chapter on "[Editing Modes](#).")

- * Directives - 1 ([Directive1Attr](#)) Std: Intense Red on White Indicates the color of SyntaxHilite directive text, entry 1.
- * Directives - 2 ([Directive2Attr](#)) Std: Intense Red on White Indicates the color of SyntaxHilite directive text, entry 2.
- * Directives - 3 ([Directive3Attr](#)) Std: Intense Red on White Indicates the color of SyntaxHilite directive text, entry 3.
- * Multi-Line - 1 ([MultiLnDlmt1Attr](#)) Std: Intense Green on White Indicates the color of SyntaxHilite multi-line delimited text, entry 1.
- * Multi-Line - 2 ([MultiLnDlmt2Attr](#)) Std: Intense Green on White Indicates the color of SyntaxHilite multi-line delimited text, entry 2.
- * Multi-Line - 3 ([MultiLnDlmt3Attr](#)) Std: Intense Green on White Indicates the color of SyntaxHilite multi-line delimited text, entry 3.
- * Single-Line - 1 ([SingleLnDlmt1Attr](#)) Std: Intense Green on White Indicates the color of SyntaxHilite single-line delimited text, entry 1.
- * Single-Line - 2 ([SingleLnDlmt2Attr](#)) Std: Intense Green on White Indicates the color of SyntaxHilite single-line delimited text, entry 2.
- * Single-Line - 3 ([SingleLnDlmt3Attr](#)) Std: Intense Green on White Indicates the color of SyntaxHilite single-line delimited text, entry 3.
- * To-EOL - 1 ([ToEOL1Attr](#)) Std: Intense Green on White Indicates the color of SyntaxHilite to-EOL delimited text, entry 1.
- * To-EOL - 2 ([ToEOL2Attr](#)) Std: Intense Green on White Indicates the color of SyntaxHilite to-EOL delimited text, entry 2.
- * To-EOL - 3 ([ToEOL3Attr](#)) Std: Intense Green on White Indicates the color of SyntaxHilite to-EOL delimited text, entry 3.
- * Quote - 1 ([Quote1Attr](#)) Std: Intense Magenta on White Indicates the color of SyntaxHilite quote delimited text, entry 1.
- * Quote - 2 ([Quote2Attr](#)) Std: Intense Magenta on White Indicates the color of SyntaxHilite quote delimited text, entry 2.
- * Quote - 3 ([Quote3Attr](#)) Std: Intense Magenta on White Indicates the color of SyntaxHilite quote delimited text, entry 3.
- * [Numbers](#) ([NumberAttr](#)) Std: Intense Magenta on White Indicates the color of SyntaxHilite number text.
- * Incomplete Quote ([IncompleteQuoteAttr](#)) Std: Intense Black on Red Indicates the color of SyntaxHilite incomplete quote text.
- * KeyWords - 1 ([KeyWords1Attr](#)) Std: Intense Bright Blue on White Indicates the color of SyntaxHilite keyword text, group 1.
- * KeyWords - 2 ([KeyWords2Attr](#)) Std: Intense Yellow on White Indicates the color of SyntaxHilite keyword text, group 2.
- * KeyWords - 3 ([KeyWords3Attr](#)) Std: Intense Bright Magenta on White Indicates the color of SyntaxHilite keyword text, group 3.
- * KeyWords - 4 ([KeyWords4Attr](#)) Std: Intense Bright Red on White Indicates the color of SyntaxHilite keyword text, group 4.
- * KeyWords - 5 ([KeyWords5Attr](#)) Std: Intense Cyan on White Indicates the color of SyntaxHilite keyword text, group 5.
- * KeyWords - 6 ([KeyWords6Attr](#)) Std: Intense Cyan on White Indicates the color of SyntaxHilite keyword text, group 6.
- * KeyWords - 7 ([KeyWords7Attr](#)) Std: Intense Bright Blue on White Indicates the color of SyntaxHilite keyword text, group 7.
- * KeyWords - 8 ([KeyWords8Attr](#)) Std: Intense Bright Blue on White Indicates the color of SyntaxHilite keyword text, group 8.
- * KeyWords - 9 ([KeyWords9Attr](#)) Std: Intense Bright Blue on White Indicates the color of SyntaxHilite keyword text, group 9.

To select a screen item for which to configure the color, move the cursor bar over the desired item and press <Enter>. A color chart replaces the PickList of configurable items on the left of the screen. The name of the current item selection is indicated above the sample screen. As you move through the selections in the color chart, the color of the affected item is changed on the sample screen to match. The PC attribute number that represents the foreground/background color combination is displayed in the upper left corner of the color chart. Select the desired settings from the color chart, as follows:

- Use the cursor keys (up, down, left, right) to select the desired foreground/background color combination.
- Press <Ins> to toggle bright colors ON or OFF for the selected item.
- Press to restore the item to its initial setting.
- Press <Escape> to cancel any changes made to the item and return to the Item Selection screen.
- Press <Enter> to accept the color change and return to the Item Selection screen.

Mouse users: You can use the mouse to select a color combination by positioning the mouse pointer on the desired color combination, and clicking the <LeftBtn>. You can perform the functions assigned to <Ins>, , <Escape>.

and <Enter> by positioning the mouse pointer on the desired key assignment or function, and clicking the <LeftBtn>. (These key assignments are displayed on a "helpline" below the color chart.) To cancel the color changes that you have made, select "Cancel Color Changes" from the Item Selection screen. (For more information on related topics, see the chapters on "[The Screen Layout](#)," "[The Pull-Down Menu System](#)," "[Getting Help in the Editor](#)," "[Splitting the Screen into Windows](#)," "[Editing Modes](#)," and "[Special Commands and Features](#).") [Print Output Options](#) -----

Following is a list of the Print Output Options, including the editor's internal option name and the value of the standard setting for each option. For a description of each option, see the chapter on "[Printing From the Editor](#)." * Left Margin ([PrintLeftMargin](#)) Std: 0 * Right Margin ([PrintRightMargin](#)) Std: 0 * Lines Per Page ([PrintLinesPerPage](#)) Std: 0 * Top Margin ([PrintTopMargin](#)) Std: 0 * Bottom Margin ([PrintBotMargin](#)) Std: 0 * Line Spacing ([PrintLineSpacing](#)) Std: 1 * Use Current Screen Font ([PrintUseScreenFont](#)) Std: On * Set Printer Font ([SetPrinterFont\(\)](#)) Std: empty * Print Line Numbers ([PrintLineNumbers](#)) Std: [OFF](#) * Header ([PrintHeader](#)) Std: empty * Footer ([PrintFooter](#)) Std: empty * First Page ([PrintFirstPage](#)) Std: 0 * Last Page ([PrintLastPage](#)) Std: 0 * Number of Copies ([PrintCopies](#)) Std: 1 * Pause Between Pages ([PrintPause](#)) Std: [OFF](#)

Configuring the Editor Interactively

Customizing editing options to suit your particular editing style is accomplished from within the editor using IConfig, the Interactive Configuration facility. A limited set of options can also be configured directly from the pull-down Menu. Configuration changes can be applied to the current session only, or they can be installed in the editor so they remain in effect for future sessions.

This chapter provides the following information on configuring editing options from within the editor:

-

[Methods for interactively configuring the editor](#) - [Configuration options available interactively](#) - [Saving configuration settings](#) - [Writing configuration settings to a Settings File](#) - [Files needed by IConfig](#) This chapter does not attempt to fully explain the effects of or uses for the various configuration options (variables) available within the editor. To make effective use of this chapter, you may wish to review the other chapters in this manual to familiarize yourself with the operation of the editor. (For more information on specific editor variables, see the chapter on "[Editor Variables](#)" in the [Macro Reference Guide](#).) This chapter also does not discuss how to customize the editor using the editor's macro programming language, the SemWare Applications Language (SAL). (For more information on how to do this, see the chapter on "[Fully Customizing the Editor](#).")

Constants

To give [constant numbers](#) more descriptive names, [constant](#) declarations may be used. [Constants](#) may be used anywhere [constant numbers](#) are required. [Constant identifiers](#) must be initialized when they are declared. The syntax for declaring a constant is: [constant](#) identifier = constant_expression [, identifier = constant_expression] ... where: - constant declares the following identifier as constant. - identifier is any valid user-definable identifier. (For more information, see "[Identifiers](#)" earlier in this chapter.) - constant_expression is any [constant](#) expression. (For more information, see "[Constant Expressions](#)" later in this chapter.) To declare multiple constants, separate the identifiers with commas as shown above. [Examples](#): [constant](#) YES = 1, NO = 0 [constant](#) TEN_IN_HEX = 0ah [constant](#) TEN_IN_BINARY = 1010b [Constants](#) may also be used to create other [constants](#): [constant](#) ONE = 1, TWO = 2, FOUR = 4, NINE = (TWO * FOUR) + ONE

Copy

Copies the marked block to the ClipBoard. The current contents of the ClipBoard are discarded before the operation starts, unless the `_APPEND_` constant is passed to the command.

Syntax: [INTEGER Copy](#)([\[INTEGER flag\]](#)) - flag is an optional integer specifying whether [Copy\(\)](#) should keep or discard the current ClipBoard contents. If not specified, `_DEFAULT_` is used. Supported [values](#) for flag are: - `_DEFAULT_` causes [Copy\(\)](#) to replace the contents of the ClipBoard with the block. - `_APPEND_` causes [Copy\(\)](#) to append the block to the ClipBoard. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command fails if there is [not](#) enough memory to successfully complete the operation. [Copy\(\)](#) will not copy a block that exists outside the current file. The block is unmarked after the copy is performed. If there is no marked block, but the `UseCurrLineIfNoBlock` variable is set ON, the current line is copied to the ClipBoard. To retrieve text from the ClipBoard, use the [Paste\(\)](#) or [PasteReplace\(\)](#) command, as appropriate. **Examples:** // Causes <Grey+> to Copy a block to the ClipBoard, // deleting the current contents <Grey+> [Copy\(\)](#) // Causes <Ctrl Grey+> to Copy a block to the ClipBoard, // appending it to the current contents <Ctrl Grey+> [Copy\(_APPEND_\)](#) See Also: [CopyAppend\(\)](#), [Cut\(\)](#), [CutAppend\(\)](#), [Paste\(\)](#), [PasteReplace\(\)](#) [CopyToWinClip\(\)](#), [CopyAppendToWinClip\(\)](#), [CutToWinClip\(\)](#), [CutAppendToWinClip\(\)](#), [PasteFromWinClip\(\)](#), [PasteReplaceFromWinClip\(\)](#) [CopyBlock\(\)](#), [DelBlock\(\)](#), [MoveBlock\(\)](#) [isBlockMarked\(\)](#), [isBlockInCurrFile\(\)](#) **Variables:** [BlockId](#), [ClipboardId](#), [UseCurrLineIfNoBlock](#)

CopyAppend

Copies the marked block to the ClipBoard, appending it to the current contents.

Syntax: [INTEGER CopyAppend](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command fails if there is [not](#) enough memory to successfully complete the operation. [CopyAppend\(\)](#) will not copy a block that exists outside the current file. The block is unmarked after the copy is performed. If there is no marked block, but the `UseCurrLineIfNoBlock` variable is set ON, the current line is copied to the ClipBoard. To retrieve text from the ClipBoard, use the [Paste\(\)](#) command. **Examples:** // Causes <Ctrl Grey+> to Copy a block to the ClipBoard, // appending it to the current contents <Ctrl Grey+> [CopyAppend\(\)](#) See Also: [Copy\(\)](#), [Cut\(\)](#), [CutAppend\(\)](#), [Paste\(\)](#), [PasteReplace\(\)](#) [CopyAppendToWinClip\(\)](#), [CopyToWinClip\(\)](#), [CutToWinClip\(\)](#), [CutAppendToWinClip\(\)](#), [PasteFromWinClip\(\)](#), [PasteReplaceFromWinClip\(\)](#) [CopyBlock\(\)](#), [DelBlock\(\)](#), [MoveBlock\(\)](#) [isBlockMarked\(\)](#), [isBlockInCurrFile\(\)](#) **Variables:** [BlockId](#), [ClipboardId](#), [UseCurrLineIfNoBlock](#)

CopyAppendToWinClip

Copies the marked block to the Windows Clipboard, appending it to the current contents.

Syntax: [INTEGER CopyAppendToWinClip](#)([\[STRING text\]](#)) - Optional text that is appended to the Windows Clipboard in lieu of the marked block. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command fails if there is [not](#) enough memory to successfully complete the operation. [CopyAppendToWinClip\(\)](#) will not copy a block that exists outside the current file. The block is unmarked after the copy is performed. If there is no marked block, but the `UseCurrLineIfNoBlock` variable is set ON, the current line is copied to the Clipboard. To

retrieve text from the Windows Clipboard, use the [PasteFromWinClip\(\)](#) command. If the optional string is passed, any marked block is ignored and only the passed string is appended to the Clipboard. See Also: [CutToWinClip\(\)](#), [PasteFromWinClip\(\)](#), [Cut\(\)](#), [Paste\(\)](#), [isBlockMarked\(\)](#), [isBlockInCurrFile\(\)](#), [CutAppendToWinClip\(\)](#) [Variables:](#) [BlockId](#), [UseCurrLineIfNoBlock](#)

CopyBlock

The marked block is copied to the current cursor position.

Syntax: [INTEGER CopyBlock](#)([[INTEGER](#) overwrite]) - overwrite is an optional integer that specifies whether [CopyBlock\(\)](#) should overwrite or [insert](#) column blocks. If [not](#) specified, [_DEFAULT_](#) is used. overwrite has no effect if the block being copied is not a column block. Supported values for overwrite are: - [_DEFAULT_](#) causes column blocks to be inserted. - [_OVERWRITE_](#) causes column blocks to overwrite text at the current [cursor](#) position. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If no block is marked, no action occurs and zero is returned. This command fails if there is [not](#) enough memory to successfully complete the operation. Character blocks are always inserted at the current cursor position. Column blocks are inserted or overlaid based on the value of the overwrite parameter. Where line blocks are placed depends on the value of the [InsertLineBlocksAbove](#) variable. If set [ON](#), line blocks are inserted above the current cursor line. If set OFF, line blocks are added below the current cursor line. This command can copy blocks between files. This command also works within dialog boxes, if it is assigned to a key (and no parameter is specified). See Also: [DelBlock\(\)](#), [MoveBlock\(\)](#) [Copy\(\)](#), [CopyAppend\(\)](#), [Cut\(\)](#), [CutAppend\(\)](#), [Paste\(\)](#), [PasteReplace\(\)](#) [CopyToWinClip\(\)](#), [CopyAppendToWinClip\(\)](#), [CutToWinClip\(\)](#), [CutAppendToWinClip\(\)](#), [PasteFromWinClip\(\)](#), [PasteReplaceFromWinClip\(\)](#) [isBlockMarked\(\)](#), [isBlockInCurrFile\(\)](#) [Variables:](#) [InsertLineBlocksAbove](#), [BlockId](#)

CopyFile

Copies a disk file from one location to another.

Syntax: [INTEGER CopyFile](#)([STRING](#) src, [STRING](#) dst [, [INTEGER](#) overwrite]) - src is the name of the file to copy. - dst is the name of the copied file. - overwrite defaults to [FALSE](#). In this [case](#), [CopyFile\(\)](#) fails if dst exists. If overwrite is set to [TRUE](#), dst will be overwritten if it exists. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Wildcards cannot be specified with this command. Only single files may be copied. If path information is not specified, the current directory is assumed. The original file is left unchanged. [Examples:](#) [CopyFile](#)("foo", "bar") [CopyFile](#)("c:\dir\foo", "d:\path\bar") // assume the existence of two strings, s1 and s2 [CopyFile](#)(s1, s2) // fails if s2 exists [CopyFile](#)(s1, s2, [TRUE](#)) // s2 will be overwritten See Also: [MoveFile\(\)](#), [EraseDiskFile\(\)](#), [RenameDiskFile\(\)](#)

CopyToWinClip

Copies the marked block to the Windows Clipboard. The current contents of the Windows Clipboard are discarded before the operation starts.

Syntax: [INTEGER CopyToWinClip](#)([[STRING](#) text]) - Optional text that is copied to the Windows Clipboard in lieu of the marked block. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command fails if there is [not](#) enough memory to successfully complete the operation. [CopyToWinClip\(\)](#) will not copy a block that exists outside the current file. The block is unmarked after the copy is performed. If there is no marked block, but the

UseCurrLineIfNoBlock variable is set ON, the current line is copied to the Clipboard. To retrieve text from the Windows Clipboard, use the [PasteFromWinClip\(\)](#) or [PasteReplaceFromWinClip\(\)](#) command, as appropriate. If the optional string is passed, any marked block is ignored and only the passed string is copied to the Clipboard. See Also: [CopyAppendToWinClip\(\)](#), [CutToWinClip\(\)](#), [CutAppendToWinClip\(\)](#), [PasteFromWinClip\(\)](#), [PasteReplaceFromWinClip\(\)](#) [Copy\(\)](#), [CopyAppend\(\)](#), [Cut\(\)](#), [CutAppend\(\)](#), [Paste\(\)](#), [PasteReplace\(\)](#) [CopyBlock\(\)](#), [DelBlock\(\)](#), [MoveBlock\(\)](#) [isWinClipAvailable\(\)](#), [EmptyWinClip\(\)](#), [isBlockMarked\(\)](#), [isBlockInCurrFile\(\)](#) [GetStrFromWinClip\(\)](#) [Variables: BlockId, UseCurrLineIfNoBlock](#)

CreateBuffer

Creates a new buffer in memory.

Syntax: [INTEGER CreateBuffer](#)([STRING](#) fn [, [INTEGER](#) type]) - fn is the name of the buffer to be created. fn must be a valid filename only if it is desired to save the file to disk. - type is an optional integer parameter which tells what type of buffer is to be created. If [not](#) specified, [_NORMAL_](#) is used. Supported [values](#) for type are: - [_NORMAL_](#) causes the buffer to have no special attributes associated with it. [Buffers](#) created with [EditFile\(\)](#) have this buffer type. - [_HIDDEN_](#) simply causes the buffer to not show up in the list of files, and makes the buffer inaccessible via the [NextFile\(\)](#) and [PrevFile\(\)](#) commands. The PickList of loaded [macros](#) (optionally displayed by the [ExecMacro\(\)](#) command) is stored in a [_HIDDEN_](#) buffer. - [_SYSTEM_](#) causes the buffer to be hidden, and places the buffer in a raw mode, so that trailing spaces are never removed, and tabs are treated as a normal character. Buffers created with [CreateTempBuffer\(\)](#) have this buffer type. It is strongly recommended that system [buffers not](#) be used for interactive editing. Undo/Redo information is [NOT](#) recorded for [_SYSTEM_ buffers](#). Returns: The non-zero id number of the newly-created buffer if successful; [otherwise](#), zero ([FALSE](#)). Notes: This command fails if there is [not](#) enough memory to successfully complete the operation. A buffer is created only if a buffer of the same name does not already exist in the editor's internal list of files/buffers. If created, the new buffer becomes the current buffer. If buffer fn already exists, no action occurs and zero is returned. [CreateBuffer\(\)](#) is primarily intended for creating workspace areas [for macros](#). It is [not](#) intended for interactive editing of [buffers](#). Unlike [EditFile\(\)](#), [CreateBuffer\(\)](#) does [NOT](#) check to see if fn exists on disk. Since [CreateBuffer\(\)](#) is intended for creating macro workspace areas, checking file existence would defeat its purpose, and cause it to execute much more slowly. Likewise, [buffers](#) created via this command are [not](#) intended for saving to disk. Since the disk is not checked before the buffer is created, a file on disk could be accidentally overwritten if a buffer is saved. The [FileExists\(\)](#) command can be used to prevent this. This command does not invoke the [_ON_CHANGING_FILES_](#) and [_ON_FIRST_EDIT_](#) events. (For more information on events, see the [Hook\(\)](#) command description.) [_NORMAL_](#) buffers show up in the editor's list of files, and can be accessed via the following commands: [EditFile\(\)](#), [NextFile\(\)](#), [PrevFile\(\)](#), [GotoBufferId\(\)](#). [_SYSTEM_](#) and [_HIDDEN_ buffers](#) are hidden [buffers](#) and can only be accessed via the [GotoBufferId\(\)](#) command. [_SYSTEM_](#) buffers are not intended for interactive editing by the user, as many editor functions are [not](#) in effect (such as [RemoveTrailingWhite](#) and [ExpandTabs](#), among others). [_HIDDEN_](#) files are treated as [_NORMAL_](#) buffers with the exception that they can only be accessed via [GotoBufferId\(\)](#). **Examples:** rc = [CreateBuffer](#)("foo") // Creates a buffer named foo /
***** Creates a file named "foo2". "foo2" can only be viewed by switching to it directly. *****/ rc =
[CreateBuffer](#)("foo2", [_HIDDEN_](#)) /***** Create a new buffer, and then switch back to the original buffer.
*****/ [integer](#) currid, newid currid = [GetBufferId\(\)](#) // retrieve the current file's // id newid = [CreateBuffer](#)("foo") // create the new buffer [if](#) newid = [FALSE](#) // error, could [not](#) create ... [else](#) ... // do whatever with new buffer [GotoBufferId](#)(currid) // switch back to original // buffer [endif](#) See Also: [CreateTempBuffer\(\)](#), [GotoBufferId\(\)](#), [GetBufferId\(\)](#), [EditFile\(\)](#), [EditThisFile\(\)](#), [NextFile\(\)](#), [PrevFile\(\)](#), [Hook\(\)](#), [UnHook\(\)](#), [FileExists\(\)](#), [BufferType\(\)](#), [AddFileToRing\(\)](#)

CreateTempBuffer

Creates a temporary buffer of type `_SYSTEM_`.

Syntax: [INTEGER CreateTempBuffer\(\)](#) Returns: The non-zero id number of the newly-created system buffer; [otherwise](#), zero ([FALSE](#)). Notes: This command fails if there is [not](#) enough memory to successfully complete the operation. The buffer is always created unless the editor runs out of memory. The newly-created buffer becomes the current buffer. The buffer created by `CreateTempBuffer` is a `_SYSTEM_` buffer and can only be accessed via the [GotoBufferId\(\)](#) command. It will not show up in the file lists shown by [PickFile\(\)](#), and cannot be accessed via the [NextFile\(\)](#) and [PrevFile\(\)](#) commands. Since [buffers](#) created by this command are `_SYSTEM_` [buffers](#), they are [NOT](#) intended for interactive editing. Undo/Redo information is [NOT](#) recorded for `_SYSTEM_` [buffers](#). [Buffers](#) created by [CreateTempBuffer\(\)](#) last until they are explicitly removed by the user. For more information on `_SYSTEM_` buffers, see the description of [CreateBuffer\(\)](#). See Also: [CreateBuffer\(\)](#), [GotoBufferId\(\)](#), [GetBufferId\(\)](#), [BufferType\(\)](#)

Creating and Using Macros

You can create

[macros](#) to simplify and automate editing tasks and to further customize and extend the features and capabilities of the editor. Two types of [macros](#) are available: [Keyboard Macros](#) and Compiled [Macros](#). Keyboard Macros are designed to make it easy to repeatedly execute a series of commands and keystrokes entered from the keyboard. Compiled Macros give you access to the editor's macro programming language, allowing you to write simple or complex [macros](#) to customize or create your own commands. This chapter describes the following macro features: - [Keyboard Macros](#): - [Recording Keyboard Macros](#) (`RecordKeyMacro`) - [Re-playing Keyboard Macros](#) ([macros](#) assigned to keys, `ExecScrapMacro`) - [Saving Keyboard Macros for use in a later session](#) (`SaveKeyMacro`) - [Loading Keyboard Macros saved in a previous session](#) (`LoadKeyMacro`) - [Purging Keyboard Macros from the current session](#) (`PurgeKeyMacro`) - [Introduction to external Compiled Macros](#): - [Creating and using Compiled Macros](#) - [Compiling macro source](#) (`Compile`) - [Loading and executing Compiled Macros](#) (`LoadMacro`, `ExecMacro`, `AutoLoadList`) - [Purging Compiled Macros](#) (`PurgeMacro`) - [Debugging Compiled Macros](#) (`Debug`) There are two types of Compiled Macros in the editor: external and user-interface. This chapter deals with external Compiled Macros (in the section, "[Introduction to External Compiled Macros](#)"). The chapter on "[Fully Customizing the Editor](#)" explains how to create both external and user-interface [macros](#), and provides additional information [about](#) both types of Compiled [Macros](#).

Creating Macros

The first step in creating macros, is to create a macro source file using the editor. Macro source files are standard ASCII text files that contain a sequence of SAL macro statements that perform a specific task. In order for the SAL compiler to recognize the source file, the extension must be ".S" for an external macro file, or ".UI" if the macro is a user-interface macro file. The editor has some knowledge of SAL, and recognizes `SmartIndent` when you press `<Enter>` on a line containing certain SAL statements. The `Compile()` and `FunctionList()` commands also know

[about](#) SAL, allowing them to act appropriately. Once you have created a SAL source file, you are ready to compile the source file into an object file that can be directly interpreted by the editor.

Creating Macros (Overview)

The first step in creating a macro is to type a series of macro commands and statements (called "source code") that are recognized by the SemWare Applications Language (SAL). For example, to write a very simple macro, edit a new file called "HELLO.S", and place the following text in it:

```
proc Main() Message("Hello, World!") end
```

CReturn

Performs a carriage return at the current cursor location.

Syntax: [INTEGER CReturn](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command fails if there is not enough memory to successfully complete the operation. The action of the [CReturn](#)() command will depend on several editor [variables](#) and [editing modes](#). The following assumes that [Insert](#) is currently [ON](#): - If [ReturnEqNextLine](#) is OFF, the current line will be split at the current cursor location, and the cursor will be placed at the beginning of the next line. The cursor position on the new line may be different, depending on the following conditions: - If [AutoIndent](#) is OFF, the cursor will be placed at the position specified by the [LeftMargin](#) variable. - If [AutoIndent](#) is ON, the cursor position on the new line will be equal to the position of the first non-white character of the previous line. - If [AutoIndent](#) is set to STICKY, the cursor position on the new line will be equal to the position of the first non-white character of the previous non-blank line. - If [ReturnEqNextLine](#) is ON, the cursor will be moved to the following line (without splitting the current line). The cursor position on the new line is determined by the following conditions: - If [AutoIndent](#) is OFF, the cursor will be placed at the position specified by [LeftMargin](#). - If [AutoIndent](#) is ON or STICKY, and the line is non-blank, the cursor will be placed at the first non-white character on the line. If the line is blank, the cursor will be placed under the first non-white character of the previous line if [AutoIndent](#) is ON, or under the first non-white character of the most recent non-blank line if [AutoIndent](#) is STICKY. - If the cursor is on the last line, a new line will be created below the current line, and the cursor will be placed on the new line according to the previous conditions dealing with blank lines. If [Insert](#) is [OFF](#), [ReturnEqNextLine](#) has no effect. The action of the [CReturn](#)() command is the same as described above for [Insert](#) set [ON](#) and [ReturnEqNextLine](#) set [ON](#). In prompts, menus, and list boxes, [CReturn](#)() is assigned to the <Enter> key (sometimes called the <Return> key) regardless of the current key assignment. See Also: [SplitLine](#)(), [JoinLine](#)(), [AddLine](#)(), [InsertLine](#)() **Variables:** [AutoIndent](#), [ReturnEqNextLine](#), [Insert](#)

CurrChar

Returns the integer value of the character at the current (or specified) position.

Syntax: [INTEGER CurrChar](#)([INTEGER](#) pos) - pos is the optional position within the current line from which to retrieve the character. If not passed, the current position ([CurrPos](#)()) is used. Returns: The ASCII integer value of the current character if within the text on the line, or: - [_AT_EOL_](#) if at the end-of-line position (that is, at the position immediately after the last character on the line). - [_BEYOND_EOL_](#) if past the end-of-line position. Notes: To pass the results from [CurrChar](#)() to commands requiring a character string, use the [Chr](#)() command. This command can be

used in a macro to easily determine whether the cursor position is within the text of the current line or is at or beyond the end-of-line position. If the value returned is equal to or greater than zero, then the current cursor position is within the text of the current line. If the value returned is less than zero, then the current cursor position is at or beyond the end-of-line position, and the value can be referenced as `_AT_EOL_` or `_BEYOND_EOL_`, respectively. Idioms: /

***** TRUE if the current cursor position is at the [end-of-line](#) position. *****/ `if CurrChar() == _AT_EOL_ /`

***** TRUE if the current cursor position is beyond the [end-of-line](#) position. *****/ `if CurrChar() == _BEYOND_EOL_ /`

***** TRUE if the current cursor position is at or beyond the [end-of-line](#) position. *****/ `if CurrChar() < 0 /`

***** TRUE if the current cursor position is within the text of the current line. *****/ `if CurrChar() >= 0 /`

***** The [Asc\(\)](#) command is used with [CurrChar\(\)](#) to test for a literal ASCII character. To test for an "s" at the current position, use the following: *****/ `if CurrChar() == Asc("s")` [Examples:](#) /

***** Example 1 Return TRUE if the current character is alphabetic; [otherwise](#), [FALSE](#). *****/ `integer proc isAlpha() case Chr(CurrChar()) when "A".. "Z", "a".. "z" return (TRUE) endcase return (FALSE) end /`

***** Example 2 Return the first word on the current line. Use the current WordSet definition to determine what comprises a word. *****/ `string proc GetFirstWord() string word[12] = "" PushPosition() // Save current position GotoPos(PosFirstNonWhite()) // Start at first // character on line while not isWord() // Scan for start of word if CurrChar() < 0 // Bail out if past return ("") // end-of-line endif endwhile // Accumulate the word while isWord() and Length(word) < SizeOf(word) word = word + Chr(CurrChar()) if not Right () // break on max line length break endif endwhile PopPosition() // Restore original position return (word) // and return the word end` See Also: [GetText\(\)](#), [InsertText\(\)](#), [GetWord\(\)](#), [GetFoundText\(\)](#)

CurrCol

Returns the number of the current column position on the current line.

[Syntax:](#) `INTEGER CurrCol()` Returns: The number of the current column position on the current line. Notes: [CurrCol\(\)](#) does [not](#) equal [CurrPos\(\)](#) if: Hard (real) tab characters exist in the current line before the current position; AND [ExpandTabs](#) is [ON](#); AND [TabWidth](#) is greater than one. ([CurrCol\(\)](#) counts expanded hard tab characters as equal to the number of spaces to which the hard tab is expanded. [CurrPos\(\)](#) always counts a hard tab as one character.) Idioms: /

***** TRUE if the current cursor position is at the first character of the current line. *****/ `if CurrCol() == 1`

See Also: [CurrPos\(\)](#), [CurrXOffset\(\)](#), [CurrLine\(\)](#) [Variables:](#) [ExpandTabs](#), [TabType](#), [TabWidth](#)

CurrDir

Returns the current drive and directory.

[Syntax:](#) `STRING CurrDir()` Returns: The current drive and directory. Notes: The trailing slash is always included in the returned string. [Examples:](#) Given: The current drive is c: and the current directory is \foo\bar. `string s[80]` Then: `s =`

[Currdir\(\)](#) // s is "c:\foo\bar\" See Also: [GetDir\(\)](#), [ChDir\(\)](#)

CurrExt

Returns the extension portion of the filename for the current file.

Syntax: [STRING CurrExt\(\)](#) Returns: The extension of the current file. Notes: If the current file does not have an extension, an empty string is returned; otherwise, the extension, including the period, is returned. The returned extension is always lower-cased. To return the actual case of the extension, use the following code: [SplitPath](#) ([CurrFilename\(\)](#), _EXT_) **Examples:** // assign the extension of the current file to s [string](#) s[_MAXPATH_] s = [CurrExt](#) () See Also: [CurrFilename\(\)](#), [ChangeCurrFilename\(\)](#), [SplitPath\(\)](#)

CurrFilename

Returns the full name of the current buffer, including drive and path for disk files.

Syntax: [STRING CurrFilename\(\)](#) Returns: The name of the current buffer. Notes: [SplitPath\(\)](#) can be used to separate out the different parts of the name. The case of the filename is returned as is. **Examples:** // A simple proc to return the extension of the current // file as a string [string proc](#) [CurrExt\(\)](#) [return](#) ([SplitPath](#)([CurrFilename\(\)](#), _EXT_)) [end](#) See Also: [ChangeCurrFilename\(\)](#), [CurrExt\(\)](#), [SplitPath\(\)](#)

CurrHistoryList

The id number of the most recently used history list.

Values: [INTEGER](#) - [1..255] **Default:** Undefined Limits: Cannot be used within a config/endconfig construct. Notes: Can be used in the _PROMPT_STARTUP_ event to determine the last or current prompt/history used. (See the [Hook\(\)](#) command for more information on this event.) See Also: Commands: [AddHistoryStr\(\)](#)

CurrLine

Returns the line number of the current line.

Syntax: [INTEGER CurrLine\(\)](#) Returns: The number of the current line. Idioms: /
 ***** TRUE if the current cursor position is at the first line of the file. *****/ [if](#) [CurrLine\(\)](#) == 1 /
 ***** TRUE if the current cursor position is at the last line of the file. [NumLines\(\)](#) can be 0 in an empty file, and can also be less than [CurrLine\(\)](#) if the last line has just been deleted, and the cursor has not moved off that line.
 *****/ [if](#) [CurrLine\(\)](#) >= [NumLines\(\)](#) See Also: [NumLines\(\)](#), [CurrCol\(\)](#), [CurrPos\(\)](#), [CurrRow\(\)](#)

CurrLineLen

Returns the length of the current line.

Syntax: [INTEGER CurrLineLen\(\)](#) Returns: The length of the current line. Notes: If tab characters are contained within the current line, they will be counted as single characters. Idioms: /

***** TRUE if the cursor is on an empty line

*****/ [if CurrLineLen\(\) == 0 /](#)

***** TRUE if the current cursor position is within the text of the current line. *****/ [if CurrPos\(\) <=](#)

[CurrLineLen\(\)](#) /***** TRUE if the current cursor

position is beyond the text of the current line. *****/ [if](#)

[CurrPos\(\) > CurrLineLen\(\)](#) See Also: [Variables:](#) [ExpandTabs](#), [TabType](#), [TabWidth](#)

CurrMacroFilename

Returns the full path of the Compiled Macro file containing the currently executing macro.

Syntax: [STRING CurrMacroFilename\(\)](#) Returns: The full path of the Compiled Macro file containing the currently executing macro. Notes: Use [SplitPath\(\)](#) in order to isolate the macro filename from the complete path. [Examples:](#)

[string](#) fn[8] fn = [SplitPath\(CurrMacroFilename\(\), _NAME_\)](#) See Also: [ExecMacro\(\)](#), [LoadMacro\(\)](#), [PurgeMacro\(\)](#), [SplitPath\(\)](#)

CurrPos

Returns the current position on the current line.

Syntax: [INTEGER CurrPos\(\)](#) Returns: The position on the current line. Notes: Expanded tab characters do not affect the value returned, as they [do](#) with [CurrCol\(\)](#). [CurrPos\(\)](#) will [not](#) equal [CurrCol\(\)](#) if Hard (real) tabs exist in the current line before the current position, and [ExpandTabs](#) is [ON](#), and [TabWidth](#) is greater than 1. Idioms: /

***** [CurrPos\(\)](#) is commonly used with [GetText\(\)](#) to copy text from the current cursor position for some length "len", into a macro language string.

*****/ s = [GetText\(CurrPos\(\), len\)](#) See Also: [CurrCol](#)

(), [CurrXOffset\(\)](#), [CurrLineLen\(\)](#) [Variables:](#) [ExpandTabs](#), [TabType](#), [TabWidth](#)

CurrRow

Returns the current cursor row relative to the top line of the current window, in the range 1 through

[Query\(WindowRows\).](#) **Syntax:** [INTEGER CurrRow\(\)](#) Returns: The current [cursor](#) row. See Also: [CurrLine\(\)](#), [CurrXOffset\(\)](#), [CurrPos\(\)](#) [Variables:](#) [WindowRows](#)

CurrVideoMode

The current editor video mode.

Values: [INTEGER](#) - [_25_LINES_](#) - [_28_LINES_](#) - [_30_LINES_](#) - [_33_LINES_](#) - [_36_LINES_](#) - [_40_LINES_](#) - [_43_LINES_](#) - [_44_LINES_](#) - [_50_LINES_](#) - [_MAX_LINES_](#) - [_MAX_COLS_](#) - [_MAX_LINES_COLS_](#) **Default:** Undefined Limits: Cannot be used within a config/endconfig construct. Notes: When [Set](#)(CurrVideoMode, nn) is issued, the video system will be re-initialized. This means that if you have split the screen into horizontal or vertical windows, or are in a zoomed window, the windows will be closed and you will be placed in a full window in the current file. In a macro, [Set](#)(CurrVideoMode, nn) should not be used if menus or pop-up [windows](#) are open. If this happens, unpredictable results may occur. If [Set](#)(CurrVideoMode, nn) is assigned to a menu item, then [_MF_CLOSE_ALL_BEFORE_](#) should be specified for that [menu](#) item. **Values** for [CurrVideoMode](#) > 255 are considered as cols x rows. For example: // to get the rows and cols, if the value is >= 256: cols = [HiByte](#)([CurrVideoMode](#) value) rows = [LoByte](#)([CurrVideoMode](#) value) // to set 100 columns by 36 rows: [CurrVideoMode](#) value = cols * 256 + rows The value for [CurrVideoMode](#), for 100 x 36 (cols 100, rows 36), would be expressed as: [CurrVideoMode](#) value = (100 * 256) + 36 = 25636 [CurrVideoMode](#) can be set and queried. - [_25_LINES_](#) forces 25-line mode - [_28_LINES_](#) forces 28-line mode - [_30_LINES_](#) forces 30-line mode - [_33_LINES_](#) forces 33-line mode - [_36_LINES_](#) forces 36-line mode - [_40_LINES_](#) forces 40-line mode - [_43_LINES_](#) forces 43-line mode - [_44_LINES_](#) forces 44-line mode - [_50_LINES_](#) forces 50-line mode - [_MAX_LINES_](#) forces maximum number of lines for current font - [_MAX_COLS_](#) forces maximum number of columns for current font - [_MAX_LINES_COLS_](#) forces maximum number of lines and columns [for](#) current font See Also: [StartupVideoMode](#)

CurrWinBorderType

Determines the type of "line" used for outlining the current editing window.

Values: [INTEGER](#) - 1 - single-line characters - 2 - single-line side, double-line top characters - 3 - double-line side, single-line top characters - 4 - double-line characters - 5 - small block characters - 6 - large block characters - 7 - ASCII characters (+, -, |) - 8 - 3D characters are used **Default:** 2 Notes: CurrWinBorderType allows you to set the current editing window border style. Setting it to a value that is different from the value assigned to [OtherWinBorderType](#) makes it easier to tell which window is the current editing window. See Also: [CurrWinBorderAttr](#), [OtherWinBorderType](#), [OtherWinBorderAttr](#)

CurrXoffset

Returns the number of columns that the current window is scrolled horizontally.

Syntax: [INTEGER](#) [CurrXoffset](#)() Returns: The number of columns the current window is scrolled horizontally, or zero if the current window is not scrolled. Notes: The following relationship holds for [CurrXoffset](#)(), [CurrCol](#)(), and the logical x position (in 1 through [Query](#)(WindowCols)): $x = \text{CurrCol}() - \text{CurrXoffset}()$ **Examples:** The following macros provide commands that position the cursor at the left and right edges of the current window, without causing additional horizontal scrolling, even if the window is already horizontally scrolled. The commands are activated by pressing the <Home> and <End> keys. [integer proc](#) mCurrX() [return](#) ([CurrCol](#)() - [CurrXoffset](#)()) [end proc](#) mLeftEdge() [GotoColumn](#)([CurrCol](#)() - mCurrX() + 1) [end proc](#) mRightEdge() [GotoColumn](#)([CurrCol](#)() + [Query](#)(WindowCols) -

mCurrX()) [end](#) <Home> mLeftEdge() <End> mRightEdge() See Also: [GotoXoffset\(\)](#), [CurrCol\(\)](#), [GotoColumn\(\)](#)
[Variables](#): [WindowCols](#)

Cursor

Determines if the hardware cursor is displayed.

[Values](#): BOOLEAN [Default](#): [ON](#) Limits: Cannot be used within a config/endconfig construct. Notes: Cursor is used to turn the hardware cursor ON and OFF. See Also: [InsertCursorSize](#), [OverwriteCursorSize](#)

CursorTabWidth

If non-zero, is used by tabbing commands in lieu of

[TabWidth](#). [Values](#): [INTEGER](#) [Default](#): 0 [Limits](#): 1..16 Notes: If [CursorTabWidth](#) is non-zero, it is used by tabbing commands in lieu of [TabWidth](#). On-screen display of spaces for tabs is still controlled by [TabWidth](#). If [CursorTabWidth](#) is 0, [TabWidth](#) is used by the tabbing commands, and [CursorTabWidth](#) has no effect. Why a [CursorTabWidth](#) when there is already a [TabWidth](#) variable? So that you can distinguish between the width to which so-called hard tabs are expanded on display, and the measure used when the [TabRight\(\)](#) and [TabLeft\(\)](#) commands are run. For instance, you might have a [TabWidth](#) of 8, preferring hard tabs to be displayed this way, but have a [CursorTabWidth](#) of 4, as you want the [TabRight\(\)](#) command to compute tab stops based on 4. [Affects](#): [TabWidth](#)
 Commands: [TabRight\(\)](#), [TabLeft\(\)](#) See Also: [TabWidth](#), [TabType](#) Commands: [TabRight\(\)](#), [TabLeft\(\)](#)

Customization Using Sal Compiled Macros

The ability to write and compile macros can be a useful feature of the editor. Using SAL (the SemWare Applications Language), you can develop macros to extend or modify the features in the editor. Macros are created by writing source code using SAL instructions. The source code is compiled using the SAL compiler. Compiled Macros can be external to the editor, or can be installed in the editor as part of the user interface.

External Compiled

[Macros](#) are maintained in separate, independent ".MAC" files outside the editor. They must be explicitly loaded into the editor before they can be executed, and they can be subsequently purged (if desired) from an editing session when they are no longer needed. For access to external macros, the editor offers a variety of commands and features (including special classes of [macros](#)). The LoadMacro and ExecMacro commands allow you to load external Compiled Macro files, as needed. [Macros](#) can be classified as WhenLoaded, [Main](#), [Public](#), or WhenPurged, which allows you to determine when and how the external macros are executed. You can assign external macros to keys, allowing you to execute them by pressing the designated key; or you can assign them to menus, allowing you to execute them by choosing the appropriate [menu](#) selection. The [Potpourri](#) facility allows you to [include](#) external macros on the Potpourri PickList. The macro autoload file allows you to automatically load Compiled Macro files each time you start the editor. [The TSEStart macro](#) allows you to [include](#) Compiled [Macros](#) in your own startup routine for the editor. User-interface Compiled [Macros](#) are maintained in unique macro files that are installed in the editor, so they are implicitly invoked when the editor is started. Such macros effectively become part of the editor, and thus, cannot be purged. A user-interface file is also used to assign commands or [macros](#) to keys; define the HelpLines, Help screen, and pull-

down Menu; and optionally set editing options. (For more information on writing macros, see the chapters on "[Overview of the Macro Programming Language](#)," "[Macro Language Syntax](#)," "[Using Macros](#)," and "[Using Macro Language Features](#)" in the Macro Reference Guide.) [Creating an External Compiled Macro](#)

----- External Compiled Macros are created in separate macro source files with a filename extension of ".S". They are compiled as external macro files with a filename extension of ".MAC", and should be maintained in directories that are accessible by the editor. External Compiled Macros that are supplied with the editor are placed in the MAC\ editor subdirectory. It is recommended that you maintain your own external macro files in a separate subdirectory. You may wish to create a \TSE\USERMAC\ subdirectory in which to keep your .S and .MAC macro files. You should then include this subdirectory in the TSEPath variable so that the editor can find your Compiled [Macros](#) whenever you execute them. By keeping your custom macro files in a separate directory from the MAC\ editor subdirectory, it will be much easier for you [to](#) manage your [macros](#) and [to](#) later upgrade to newer versions of the editor. (For more information on where the editor searches for external Compiled Macro files, see the "[macro file search order](#)" described under "[Creating and Using Compiled Macros](#)" in the chapter on "[Creating and Using Macros](#)." For more information on the TSEPath variable, see "Path(s) for Supplemental Files" under "[System/ File Options](#)" in the chapter on "[Configuring the Editor Interactively](#)." To be executed, the external Compiled Macro (.MAC) file must first be loaded into the editor (via either the LoadMacro or ExecMacro command). Loading external macros into the editor is a simple process; in fact, many of the features included in the standard version of the editor are actually implemented as external [macros](#). These features [include](#) SpellCheck, Template mode, the Potpourri facility, and the multi-language Compile command. For a simple example of how to create an external macro, suppose you need a command similar to the editor's Cut command. In the editor's version of the command, surrounding text shifts to "close up the hole" that results from the deletion of the text. But assume you want the surrounding text to remain in place, and to fill the area, from which the text was deleted, with spaces. To accomplish this, you could create a simple macro, as follows: - Edit a new "macro source" file called (for example) CUTFILL.S, in your \TSE\USERMAC\ subdirectory. (The name you choose for this file should have the extension ".S".) - Type the following text into the file: // Cut a block to the ClipBoard, back-filling with spaces [proc Main\(\)](#) [if isBlockinCurrFile\(\)](#) [PushBlock\(\)](#) [Copy\(\)](#) [PopBlock\(\)](#) [FillBlock\(' '\)](#) [endif](#) [end](#) <Alt Grey-> [Main\(\)](#) - Press <Ctrl F9> to compile the macro source file. When the Compile operation is complete, a Compiled Macro file, named CUTFILL.MAC, is placed in the same directory as CUTFILL.S. In this example, the macro is assigned to <Alt Grey->. To execute this macro, you can use [the ExecMacro command](#) to invoke the macro file by name, or you can simply press the key assignment for the macro (once the macro file is loaded). Alternatively, you could add the macro file to the Potpourri facility or to the pull-down Menu (as described later in this chapter). (For more information on compiling and executing external [macros](#), see "[Introduction to External Compiled Macros](#)" in the chapter on "[Creating and Using Macros](#)." Also see "[Compiling Macros](#)" in the chapter on "[Using Macros](#)" [in the Macro Reference Guide](#).) [Adding an External Macro to the Potpourri](#)

----- Do you ever have difficulty remembering the key assignment for that useful, but seldom-used macro? Do you have some really useful [macros](#) that you do [not](#) use often enough to put on the regular menus or assign to a key? Do you have macros you would like to be available via a menu, but you do not want to go to the trouble to put them there? If so, the editor's Potpourri may be helpful for you. The Potpourri facility allows you to maintain a pop-up PickList of external Compiled Macro files, including a short description of each. From this PickList, you can select a particular macro file to execute. You can also add a new entry or delete an existing entry from the PickList. To invoke Potpourri, select "[Potpourri](#)" from the Util Menu (or press <Shift F10>). Once the PickList of available macro files is displayed, you can use the cursor keys and/or SpeedSearch to select and execute a macro file. Simply position the cursor on the desired entry, and press <Enter>. To add a new macro file to the Potpourri, press <Ins>. To delete an entry that is no longer needed, position the cursor on the entry in question, and press . For more information on the Potpourri facility, see "[Executing Supplemental Macros from the Potpourri](#)" in the chapter on "[Special Commands and Features](#)." [Maintaining a Macro Autoload File](#)

----- The editor provides for the use of an optional macro autoload file that allows you to maintain a list of Compiled Macros to automatically load each time you start the editor. This autoload file, TSELOAD.DAT, is a convenient facility [for](#) loading user-written [external macro files](#) that you want [to](#) have available every time you start the editor. This eliminates the need to manually load the macros, and is an alternative to using the "[Load Macro](#)" (-l) command-line

option when [starting the editor](#). The [AutoLoadList command](#) ("AutoLoad List" under the Macro Menu) allows you to view and modify the list of entries in the autoloading file. (For more information on the macro autoloading file, see "[Editor Startup Processing](#)" in the chapter on "[Starting the Editor](#)," and "[The AutoLoadList Command](#)" in the chapter on "[Creating and Using Macros](#).")

[Creating a TSEStart Macro File](#) ----- To further facilitate user customization, the editor provides the TSEStart macro file. This is a special user-defined external macro file that, if used, is automatically invoked every time you start the editor. The [TSEStart macro](#) file is an excellent facility for specifying startup procedures, such as initially setting [variables](#) and [editing modes](#), and loading custom macros. It can serve as an alternative to modifying the WhenLoaded macro procedure included in the standard TSE.UI user-interface macro file. The [TSEStart macro](#) feature also provides an alternate method for creating versions of the editor that behave differently depending on the directory from which the editor is started. You can include a unique TSEStart macro file in the desired directories. The editor will execute the [TSEStart macro](#) file located in the directory where you start the editor, and perform the functions specified. Following are steps to create a very simple TSEStart macro file. In this example, TSEStart merely loads the CutFill macro file, created earlier in "[Creating an External Compiled Macro](#)," and makes it automatically available during each editing session.

- In your \TSE\USERMAC\ subdirectory (or other suitable directory included in your [TSEPath](#) variable, as discussed earlier in "[Creating an External Compiled Macro](#)"), edit a macro source file called TSESTART.S. This is the required name for the file.
- Type the following text into the file (assuming that the CUTFILL.MAC macro file is in the same directory as TSESTART.MAC will be): [proc Main\(\)](#) [LoadMacro\("cutfill"\)](#) [end](#)
- Press <Ctrl F9> to Compile the macro source file. When Compile is complete, the Compiled Macro file, TSESTART.MAC, is placed in the same directory as TSESTART.S. Now each time you run the editor, the [TSEStart macro](#) file will automatically be invoked. And in turn, TSEStart will load your new CutFill macro. (For more information on the [TSEStart macro](#) file, see "[Editor Startup Processing](#)" in the chapter on "[Starting the Editor](#).")

[Understanding User-Interface \(UI\) Macro Files](#) ----- As mentioned previously, a useful feature of the editor is its customizability. Control over much of the behavior of the editor has been taken out of the editor itself, and placed into the hands of the user. This is accomplished through a user-interface (UI) macro source file, which you can modify according to your preferences. A user-interface macro source file has a filename extension of ".UI", and includes the user's preferred key assignments, menus, and help information, as well as source for additional macros. (Optionally, the user's preferred configuration options ([variables](#)) can be included in this file, though this is not the recommended method. The recommended method is to use IConfig.) For the modifications and macros in the user-interface source file to become effective, the user-interface source file must be compiled and then installed in the editor. These installed features, including any macros that are part of the user-interface macro, are automatically activated and available during an editing session. Several user-interface macro source files are included with the editor, and are located in the UI\editor subdirectory. The standard user-interface macro file for the editor is TSE.UI. User-interface files that emulate several other popular editors and word processors are also included. These user-interface files [include](#) editor variable (configuration) settings, various commands implemented as macros, as well as menus, help, and all the key assignments for commands. Any or all of these can be changed by the user. The overall layout of the user-interface macro file is:

- The configuration section, which is optional. The configuration section, contained within the "[Config](#)" and "[EndConfig](#)" keywords, is where the editor variables can be set to the user's preferences. In the standard version of the editor, this section is not used. An alternate, easier method for setting [editor variables](#) is provided through IConfig, the editor's menu-driven Interactive Configuration facility (as described previously under "[Changing Settings for Editing Options Using IConfig](#)"). It is recommended that you use IConfig to set [editor variables](#), and then install the settings into the editor by selecting "Save Current Settings" under the Options Menu. The two methods for setting [editor variables](#) are [not](#) compatible with each other, so you should [not](#) use both. If you choose [not](#) to use IConfig, you will have to add a configuration (Config/EndConfig) section to the user-interface file.
- The "Help screen" of [key](#) assignments that are displayed when "[Key](#) Assignments" is selected from the Help Menu. In the standard version, the only key assignments that are listed here are those commands that are not available on the pull-down Menu, or not included on the [HelpLines](#).
- All the macros for commands that are implemented in the editor as enhancements or additions to the native editor commands.
- The entire pull-down [Menu](#) system.
- The actual [key](#) assignments for all commands and [macros](#) assigned [to](#) keys. Key assignments are generally placed at the end of the user-interface macro file. For any keys that are assigned to a macro, rather than to a native command, the key

assignment must follow the macro definition. By placing the key assignments at the end of the file, they will be located after any associated macro procedures defined within the user-interface file. Many of the editor's commands are built-in native commands, and are used by simply assigning them to keys in the user-interface file. Examples of these native commands [include](#): - [Left\(\)](#), [Right\(\)](#), [Up\(\)](#), [Down\(\)](#) Other native editor commands are augmented by macros included in the user-interface file. These macros enhance the behavior of the native commands. These enhanced commands are assigned, in turn, to keys in the standard version of the editor (in TSE.UI), rather than the built-in native commands. [Examples](#) of these [macros include](#): - [mDelChar\(\)](#), [mBackSpace\(\)](#), [mCReturn\(\)](#), [mBegFile\(\)](#), [mEndFile\(\)](#) Additional macros are included in the standard version of the editor, and provide enhanced features not available with the native editor commands. [Examples](#) of these [include](#): - [mSwapLines\(\)](#), [mAsciiChart\(\)](#), [mListRecentFiles\(\)](#), [mCompressView\(\)](#) Note the "m" preceding these last two types of "macro enhanced" commands. This is used to distinguish native commands from commands implemented as [macros](#). (For more information on user-interface macro files, see the chapters on "[Using Macros](#)" and "[Using Macro Language Features](#)" in the Macro Reference Guide. For more information on the native editor commands, see the chapter on "[Editor Commands](#)" in [the Macro Reference Guide](#).)

[Installing a User-Interface \(UI\) Macro File](#) ----- Installing a new user-interface (UI) macro file causes the changes you have made to take effect. This is done by compiling the ".UI" macro source file. Typically, these changes include new key assignments, changes to the menus, changes to the HelpLines, or changes and additions to the macros comprising the user-interface file. Keep in mind that any changes you make to your user-interface macro file will [not](#) automatically be maintained when you upgrade to a newer version of the editor. You may want to document the changes you make so that you can review those changes and re-apply them, as necessary, to future versions. As a precaution, it is recommended that you make a backup copy of the supplied user-interface files before making changes to them. Then, if your changes do not give the results you expected, you can restore the original user-interface file(s). There are two ways to compile and install a new user-interface (".UI") macro file. The following outlines the simplest (and the recommended) way to do this. It assumes you are installing the standard editor user interface, and that the current directory is \TSEPro. - In the editor, load the standard editor user-interface file, TSE.UI, located in the UI\ editor subdirectory. - Execute Compile (press <Ctrl F9>) to compile and install the new user interface. If the compile operation is successful, the new interface is automatically installed. However, in order for the changes to take effect, you must first quit and then restart the editor. NOTE: When installing your new user interface, the compile operation will create a separate automatically-loaded interface file, "TSEUI.DAT", in the editor load directory. Also, if the .UI file contains a configuration (Config/EndConfig) section, then a "TSECFG.DAT" interface file will also be created containing the configuration settings from that section. Even though these files are separate files, they are effectively installed in the editor, in that they will be automatically activated by the editor each time the editor is started; no further action is necessary. As an alternative, you can compile and install the new user interface (".UI") macro file from the command line using the SAL compiler program directly, as follows (assuming you are installing TSE.UI, and that the current directory is \TSEPro): - Type "sc32". - At the first prompt, type "b" (to "burn in" a user interface) and press <Enter>. - At the "Filename of interface file:" prompt, type "ui\tse" and press <Enter> - At the "Filename of The SemWare Editor:" prompt, press <Enter> to accept the default choice, or modify as necessary before pressing <Enter>. - The compiler will compile the user-interface file, and then attempt to burn that user interface directly into the editor executable program. If, however, the editor is currently running, it will not be able to burn in the user interface, and it will display the following: Unable to burn in interface - continuing... Create separate automatically-loaded interface file(s) instead? [Y/N] Type "y". A separate "TSEUI.DAT" interface file will be created in the editor load directory. If the ".UI" file contains a configuration (Config/EndConfig) section, then a "TSECFG.DAT" interface file will also be created containing the configuration settings from that section. These files are effectively installed in the editor, in that they will be automatically activated by the editor each time the editor is started; no further action is necessary. If you type "n" at the above prompt, the user interface will not be installed. Note that if you want to burn the user interface into the editor program itself, so that only a single file is required to run the editor, then type "n". Then terminate the editor in any sessions where it may be running, and re-run the SAL compiler program (SC32.EXE). - The TSE.UI file is compiled and installed. The next time you load the editor, your changes will be in effect. As noted in the previous section, using the optional configuration (Config/EndConfig) section of a user-interface macro file is not compatible with using IConfig to set variable options (or with setting any other options available under the Options Menu). Once you select "Save Current Settings" under the Options Menu, the variable settings currently in effect are installed in the editor. If you later compile and install a user-interface macro file that

contains a Config/EndConfig section, the settings installed by the "Save Current Settings" option are overwritten by those specified in the Config/EndConfig section. Likewise, if you install a user-interface macro file with a Config/EndConfig section, and then change settings interactively (using IConfig or other Options [Menu](#) options) [and](#) select "Save Current Settings," you overwrite the Config/EndConfig settings. [Adding or Changing Key Assignments](#)

----- As discussed earlier in this chapter, the key assignments for commands and macros are maintained in a user-interface macro file. Adding or changing key assignments is quite simple, as described in the following steps. The first step is to decide the key you would like to assign. You can assign commands or [macros](#) to single keystrokes, either unshifted (such as <K> or <F1>), or shifted (such as <Ctrl K> or <Alt F1>). You can also assign commands or [macros](#) to "two-keys", such as <F4 A> (which is assigned as <F4><A>), or <Ctrl K S> (which is assigned as <Ctrl K><S>). Additionally, you can assign commands or [macros](#) to multi-shift keys using <Ctrl>, <Alt>, and <Shift> (such as <CtrlShift K> or <CtrlAlt F1>). Most keys are available for assignment; and generally, any keys that are available as single keys are also available as two-keys and multi-shift keys. To verify whether a particular single key (including single-shift and multi-shift keys) is a valid assignable key, you can use the ShowKey macro, available on the Potpourri menu. (Select "[Potpourri](#)" under the Util Menu, or press <Shift F10>; then choose "ShowKey".) In ShowKey, type the desired key. If the key is assignable, a line is displayed providing information about that key, including how to identify the key in a key assignment (indicated under the column "[Key](#) Name"). If the key is not assignable, nothing is displayed when you press the key. The next step in adding or changing a key assignment, is to decide the command or macro you would like to assign to the key you selected. This can be any of the 400+ commands that are native to the editor, or it can be a macro written by you or someone else. To assign a macro to a key, the macro itself must also be included in the user-interface file. Numerous macros included with the editor can be found in the TSE.UI user-interface file. (For more information on the native editor commands, see the chapter on "[Editor Commands](#)" in the [Macro Reference Guide](#).) Now you are ready to edit the TSE.UI file, assuming you are using the standard editor user interface, located in the UI\ editor subdirectory. The key assignments for the TSE.UI user interface are maintained in this file (following the comment, "Keyboard bindings..."), and grouped by functions. The format for [key](#) assignments is: <[key](#)> command where key is the name of the key you wish to assign, enclosed within < >, and command is the command or macro you wish to assign to <key>. If a command or macro is already assigned to the key that you have chosen, locate the line containing the existing <key> assignment. On that line, delete the existing command, and replace it with your new command or macro. If the key assignment does not already exist in the file, you can place the new assignment anywhere in the file (as long as it is located after the associated macro, if you are [assigning a macro to a key](#)). However, you may wish to add new key assignments at the end of the current key assignments. To add a key assignment, add a line in the format indicated above, with the <[key](#)> and command of your choice. Once you have changed and/or added key assignments, save the file. For the new key assignments to take effect, you must install your changes in the editor. To do this, follow the steps outlined previously in this chapter under "[Installing a User-Interface \(UI\) Macro File](#)." When you install your new user interface, the pull-down Menu is automatically updated to reflect the current key assignments for any commands included in Menu. If the same command is assigned to multiple keys in the user interface, then the first key assignment indicated in the user interface for that command is the one that is identified in the Menu, where applicable. Note that any given [key](#) can be assigned to only one command or macro. If you inadvertently assign the same key to more than one command or macro in a user-interface file, and then install that user interface, only the first occurrence of that key assignment in the file will become effective. (For more information on [key](#) assignments, see "[Assigning Macros and Commands to Keys](#)" in the chapter on "[Using Macro Language Features](#)" in the [Macro Reference Guide](#).) [Changing the HelpLines](#)

----- HelpLines are used in the editor to display up to four different user-defined lines of text associated with the following key states: - The normal "unshifted" [key](#) state (that is, no shift keys pressed) - The "control" shifted key state (that is, the <Ctrl> key pressed) - The "shift" shifted key state (that is, the <Shift> key pressed) - The "alternate" shifted key state (that is, the <Alt> key pressed) HelpLines are optional, and if displayed, appear at the top or bottom of the screen, opposite [the StatusLine](#). [HelpLines](#) are declared similarly to [key](#) assignments, as follows:
 <HelpLine> constant_string <Alt HelpLine> constant_string <Ctrl HelpLine> constant_string <Shift HelpLine> constant_string where constant_string is the text you want to display on [the HelpLine](#), in a form such as: "{F1}-Help {F2}-AddLn {F4}-DupLn {F5}-ScrollToTop {F6}-DelToEol {F9}-Shell {F10}-[Menu](#)" The constant_string HelpLine text must be enclosed in single or double quotes. Text enclosed within braces, { }, is highlighted on the screen. You

can display whatever single line of text you wish in the HelpLines, up to the width of the screen. The unshifted HelpLine (<HelpLine>) is normally displayed. To display the shifted HelpLines, press and hold down the appropriate shift [key](#). It is a simple process to add or change a HelpLine. Begin by editing the file TSE.UI, assuming you are using the standard editor user interface, located in the UI\ editor subdirectory. [The HelpLine](#) assignments for the TSE.UI user interface are maintained in this file (following the comment, "Keyboard bindings..."), grouped under the comment, "HelpLine assignments". If you want to change an existing HelpLine, locate the line containing the HelpLine assignment. On that line, delete the existing constant_string text, and replace it with your new text (enclosed in quotes). If [the HelpLine](#) assignment does not already exist in the file, you can place the new assignment anywhere in the file. However, you may wish to add the new HelpLine assignment near the other HelpLine assignments. To add a HelpLine assignment, add a line in the format indicated above, with [the HelpLine](#) selection and text of your choice. Once you have changed and/or added HelpLine assignments, save the file. For the new HelpLines to take effect, you must install your changes in the editor. To do this, follow the steps outlined previously in this chapter under "[Installing a User-Interface \(UI\) Macro File](#)." (For more information on HelpLines, see the chapter on "[Getting Help in the Editor](#)." Also see "[HelpLines](#)" in the chapter on "[Using Macro Language Features](#)" in the Macro Reference Guide.)

[Modifying the Key-Assignment Help Screen](#) ----- The standard configuration of the editor includes a multi-page Help screen that lists the key assignments for commands which are not included on the [menus](#) or standard [HelpLines](#). This standard [key](#)-assignment Help screen is displayed when "[Key](#) Assignments" is selected from either the Help Menu or from [the HelpSystem](#) "[Table of Contents](#)". If you change key assignments in the editor, you may wish to update the Help screen to reflect your new assignments. Or, you may want to [include](#) additional key assignments on the Help screen. To change or create a new Help screen, begin by editing the file TSE.UI, assuming you are using the standard editor user interface, located in the UI\ editor subdirectory. The Help screen key assignments for the TSE.UI user interface are maintained in this file (following the comment, "[Key](#)-assignment Help screen"). You will notice that the Help screen key-assignment section consists of special macro (SAL) keywords in the first few lines at the beginning of the section, followed by the SAL "[end](#)" statement at the end of this section. The other lines in between contain text enclosed within quotes, and is the text that is displayed on the Help screen. To change the contents of the Help screen, modify the text within quotes, as desired. Each line of display text must be enclosed within a pair of single (') or double (") quote characters. You may add as many lines to the Help text as you wish. If the additional lines force the Help text to exceed the number of lines on your monitor, scroll bars will appear when the Help screen is displayed, and you will be able to use the cursor keys or the mouse to scroll through the Help text. Once you have changed the Help screen text, save the file. For the new Help screen to take effect, you must install your changes in the editor. To do this, follow the steps outlined previously in this chapter under "[Installing a User-Interface \(UI\) Macro File](#)." Though you can modify the text on this Help screen to display whatever you like, it is recommended that you use it as [key](#) assignment Help. This Help screen is displayed when you select "[Key](#) Assignments" from the Help Menu or [the HelpSystem](#) "[Table of Contents](#)." The Help facility, used by the editor to create the key-assignment Help screen, is also available for creating your own additional help screens. The special macro (SAL) "[helpdef](#)" construct is used for this. (For more information on [the HelpSystem](#) that uses the key-assignment Help screen, see the chapter on "[Getting Help in the Editor](#)." For more information on creating your own help screens, see "[Helpdef](#)" in the chapter on "[Using Macro Language Features](#)" in the Macro Reference Guide.)

[Making Changes to the Pull-Down Menu](#) ----- The editor's pull-down Menu system is defined in the main user-interface macro file (TSE.UI), immediately following the macro procedures, after the comment, "TSE Pull-down [Menus](#)". You can modify [the pull-down Menu system](#) by deleting or changing an existing entry, or by adding a new entry to a Menu. To make a change to one of the pull-down Menus, first edit the file TSE.UI, assuming you are using the standard editor user interface, located in the UI\ editor subdirectory. In TSE.UI, locate the pull-down Menu definition that you want to change. Each pull-down Menu definition has the same name as shown on the main pull-down Menubar, with the word "[Menu](#)" appended to it. Each Menu definition is also preceded by the "[menu](#)" keyword. To delete a Menu entry, locate that entry in the appropriate Menu definition, and delete the line containing that entry. For example, suppose that you never use the "Local UnDelete" option on the Text Menu, and would like to remove it from the Menu to shorten the Text Menu list. To do so: - Locate the Text Menu definition by searching for "TextMenu". - Locate the "Local UnDelete" line within the Text Menu definition, and simply delete this line. Note that the actual text in the Menu

definition is "&Local UnDelete". The "&" preceding "L" indicates to the editor that the quick-key for this Menu entry is the letter "l". To change a Menu entry, locate that entry in the appropriate Menu definition, and make the desired change. For example, suppose you want the "Mark Character" option on the Block Menu to invoke the MarkChar command rather than the MarkStream command. To make this change: - Locate the Block Menu definition by searching for "BlockMenu". - Locate the "Mark Character" line within the Block Menu definition. Note that the actual text in the Menu definition is "Mark Ch&aracter". The "&" preceding the first "a" indicates to the editor that the quick-key for this Menu entry is the letter "a". - Change the command assignment, located on the right, from "[MarkStream\(\)](#)" to "[MarkChar\(\)](#)" (without the quotes). The key assignment for the MarkChar command (<Alt I>) will automatically be displayed beside this option when you pull down the Block Menu. To add a Menu entry, add an entry line in the appropriate Menu definition. For example, suppose you frequently use the MarkWord command, but have difficulty remembering the key assignment. (MarkWord is not included on the Menu in the standard configuration of the editor.) You might want to add it to the pull-down Menu, so it is readily available. Because it is a block-marking command, you may want to put it under the Block Menu, and place it immediately after the "Mark Column" option on the Menu list. To add this entry: - Locate the Block Menu definition by searching for "BlockMenu". - Locate the "Mark Column" line within the Block Menu definition. Note that the actual text in the Menu definition is "Mar&k Column". The "&" preceding "k" indicates to the editor that the quick-key for this Menu entry is the letter "k". - Add a line immediately after the "Mar&k Column" line, similar to: "Mark W&ord", [MarkWord\(\)](#) The text in quotes will display on the screen when you pull down the Block [Menu](#). You can use whatever text you prefer. The "&" in front of "o" indicates that the quick-key for this entry is the letter "o". You can choose any letter in the text you prefer, but be sure that the letter is not already used by another entry in the Block [Menu](#) definition. [MarkWord\(\)](#) is the proper name of the native editor command to be invoked when this Menu option is selected. (For more information on the names of commands and [macros](#) by which editing commands are implemented, see "[Appendix E: Command Implementation](#).") Note that the key assignment for the MarkWord command (<Ctrl K><T>) will also be displayed to the right of this option (as <Ctrl K T>) when you pull down the Block Menu. Once you have completed your pull-down Menu changes in TSE.UI, save the file. For the new Menu definitions to take effect, you must install your changes in the editor. To do this, follow the steps outlined previously in this chapter under "[Installing a User-Interface \(UI\) Macro File](#)." (For more information on [the pull-down Menu system](#), see the chapter on "[The Pull-Down Menu System](#)." For more information on the syntax for menu definitions, see "[Menus](#)" in the chapter on "[Using Macro Language Features](#)" in [the Macro Reference Guide](#).) [Adding Macros to the User-Interface \(UI\) Macro File](#) ----- Suppose that you would like to have the CutFill macro (created previously in "[Creating an External Compiled Macro](#)") added to the standard user-interface macro file, and also have it appear on the Clip Menu. By adding it to the user-interface file and installing it in the editor, it can be executed, as needed, without first having to load the CutFill macro file. To add a macro to your user-interface file, edit the file TSE.UI, assuming you are using the standard editor user interface, located in the UI\ editor subdirectory. In TSE.UI, you need to find an appropriate place to add the CutFill macro. It must be placed in TSE.UI at a location that precedes any other lines in TSE.UI that will reference it. Since it is referenced by a key (<Alt Grey->), and you will be adding it to the Menu, it must be placed before the pull-down Menu definitions, and before the key assignments. A suitable place to add it would be just before the Menu definitions. - Locate the string "TSE Pull-down Menus". This identifies the beginning of the Menu definitions for [the pull-down Menu system](#) in the standard version of the editor. - Add the CutFill macro procedure just before the comment lines containing the string you just located. - Certain changes must be made to the CutFill macro, as originally created earlier in this chapter. Because this macro has now been included in a larger macro file, you need to change the macro name from "[Main](#)" to something more appropriate, such as mCutFill. (The "m" preceding "CutFill" indicates this is a macro, rather than a native editor command.) You may also want to move the key assignment to the key-assignment area of the user-interface file. The entire macro definition should now look like: // Cut a block to the Clipboard, back-filling with spaces [proc](#) mCutFill() [if isBlockinCurrFile\(\)](#) [PushBlock\(\)](#) [Copy\(\)](#) [PopBlock\(\)](#) [FillBlock\(''\)](#) [endif](#) [end](#) Next, add the CutFill macro to the Clip pull-down Menu. A good place to add it is immediately after the "Cut Append" option on the Menu list. (For more information on modifying the pull-down Menu definitions, see "[Making Changes to the Pull-Down Menu](#)" earlier in this chapter.) To add this entry: - Locate the Clip Menu definition by searching for "ClipMenu". (Each pull-down Menu definition has the same name as shown on the main pull-down Menubar, with the word "[Menu](#)" appended to it.) - Locate the "Cut Append" line within the Clip Menu definition. Note that the actual text

in the Menu definition is "C&ut Append". The "&" preceding "u" indicates to the editor that the quick-key for this Menu entry is the letter "u". - Add a line immediately after the "C&ut Append" line, similar to: "Cut &Fill", mCutFill() The text in quotes will display on the screen when you pull down the Clip [Menu](#). You can use whatever text you prefer. The "&" in front of "F" indicates that the quick-key for this entry is the letter "f". You can choose any letter in the text you prefer, provided the letter is not already used by another entry in the Clip [Menu](#) definition. mCutFill() is the name of the command (which YOU just created as a macro in TSE.UI) to be invoked when this [Menu](#) option is selected. Note that the key assignment for mCutFill (<Alt Grey->), to be added in the following steps, will also be displayed to the right of this option when you pull down the Clip Menu. Now, add the key assignment for mCutFill to the user-interface file. (For more information on adding a key assignment, see "[Adding or Changing Key Assignments](#)" earlier in this chapter.) - Locate the line containing the key definition for <Ctrl Grey->. - After the <Ctrl Grey-> definition line, add a line with the following text: <Alt Grey-> mCutFill() - Save the TSE.UI file. Finally, you are ready to install your changes in the editor. To do this, follow the steps outlined previously in this chapter under "[Installing a User-Interface \(UI\) Macro File](#)." Congratulations! You have just added a new command to the editor, added that command to the Menu, and created a new key assignment. Your new mCutFill command will behave as if it had been originally built in to the editor. [Creating a Custom User-Interface \(UI\) Macro File](#)

----- Instead of using or modifying the standard user-interface macro file (TSE.UI), or one of the other user-interface files included with the editor, you may wish to create your own. This section walks you through development of a user-interface macro file, beginning with a very simple user interface, and then adding more functionality. (You should be familiar with the topics covered earlier in this chapter before proceeding.) [Starting Simple](#) To begin creating your user-interface macro file, edit a new macro source file called (for example) MY.UI. The name you choose for this file should have the extension ".UI", which identifies this file as a user-interface macro file. This extension instructs the Compile command to automatically install this macro file, once compilation of the file is successfully completed. At the beginning of your user-interface file, type the following key assignment: <Alt X> [Exit](#) () Now you have a very simple user-interface file for the editor. To install it, execute the Compile command (by pressing <Ctrl F9>). For the changes to actually take effect, quit the editor and then restart it. Of course, this is not a very useful user interface, and it is not recommended that you install this interface in the editor! In this simple user interface, you have assigned only a single editor command to a key; that is, the Exit command. There are no other [key](#) assignments and no [menus](#) available. Thus, in the version of the editor that would result from installing this user interface, you could not execute any commands other than the Exit command, and it could be executed only by pressing <Alt X>. All you could do with this editor would be to edit a file, view the first page, and exit (by pressing <Alt X>). Note that in a macro file, the case and the spacing of the characters are not significant. You could enter your code in a style that suits you, as illustrated by the following key assignments: <Alt x> [Exit](#)() <ALT X> [EXIT](#)() <alt x> [exit](#)() <aLt X> [eXiT](#)() [Adding More Functionality](#) To add more functionality to your custom editor, edit your user-interface file MY.UI, and include key assignments for a few cursor movement and editing commands, so that MY.UI looks like the following: <F2> [AddLine](#)() <F4> [DupLine](#)() <F6> [DelToEol](#)() <F9> [Shell](#)() <BackSpace> [BackSpace](#)() <Enter> [CReturn](#)() <Tab> [TabRight](#)() <CursorDown> [Down](#)() <CursorLeft> [Left](#)() <CursorRight> [Right](#)() <CursorUp> [Up](#)() [DelChar](#)() <End> [EndLine](#)() <Home> [BegLine](#)() <Ins> [ToggleInsert](#)() <PgDn> [PageDown](#)() <PgUp> [PageUp](#)() <Alt E> [EditFile](#)() <Alt N> [NextFile](#)() <Alt X> [Exit](#)() You can organize the key assignments in any order you prefer. To install this new user interface, execute the Compile command (by pressing <Ctrl F9>). For the changes to take effect, quit the editor and then restart it. This user interface is a little more useful than the first, though it is not recommended that you actually install this particular interface in the editor. With this version of the editor, you could move around in a file using the [cursor](#) keys, perform simple editing tasks, select additional files for editing (by pressing <Alt E>), switch between files (by pressing <Alt N>), and even Shell out of the editor to the operating system (by pressing <F9>). However, it is still [not](#) complete enough to be truly useful. It does [not](#) even give you access to any commands that save files! To complete your user interface, you will want to add more [key](#) assignments, and probably menus and help, to allow you to access the many commands that you need for editing. If you are [not](#) using IConfig [to](#) set [editor variables](#), you will need to add a variable configuration (Config/EndConfig) section. (For more information on how to do this, see the chapters on "[Using Macros](#)" and "[Using Macro Language Features](#)" [in the Macro Reference Guide](#).) [Adding Your Own Commands](#) To make your user interface more useful for your particular needs, you may

want to add your own commands or modify the behavior of some of the editor's built-in commands. Perhaps you have a preferred way of performing an editing task, that is different from the way the editor performs that task. Or maybe there is an editing feature that you really need, but that feature is not included in the editor. Or you may possibly want to combine several editor commands into one command that you can execute with a single keystroke. For this type of extended customizability, user-written [macros](#) are beneficial. For example, suppose you want to change the behavior of the native (built-in) [DelChar\(\)](#) command, which deletes the current character. But if the cursor is beyond the end of the text on the current line when you execute [DelChar\(\)](#), nothing happens. Like many other users, you would like [DelChar\(\)](#) to join the next line to the current line, when it is executed beyond the end of the line. You can write a macro to implement your own DelChar command to behave in the manner you prefer. Begin by once again editing your user-interface file, MY.UI. Near the beginning of the file, before your key assignments, and before any menu or help definitions, add the following: `proc mDelChar() if CurrChar() >= 0 // if not past the end-of-line DelChar() // just delete the current char else // otherwise, JoinLine() // join the next line to this one endif end` Next, locate the key assignment line containing " [DelChar\(\)](#)", and change it to: mDelChar() You have now replaced the key assignment for (currently assigned to the native [DelChar\(\)](#) command) with a user-written macro named mDelChar(). (The "m" at the beginning of the macro name is used as a convention to identify this "command" as a macro.) Once you install your new user interface file (and restart the editor), the key will behave like the native command when the cursor is within the text on a line; however, when the cursor is past the end-of-line, it will join the next line to the end of the current line. Note that in the standard editor user interface (TSE.UI), the key is assigned to a macro that is very similar to the macro presented here. When adding macros to a user-interface file, keep in mind that new identifiers (such as macro names) must be declared BEFORE they are referenced. Thus, in the example given, the key assignment used for mDelChar() has to occur AFTER the mDelChar() procedure definition. To keep things properly organized in the user-interface files supplied with the editor, macro procedures (for commands implemented as macros) are placed at the beginning of the file. These are followed by menus. Finally, key assignments are placed at the very end of the user-interface file. To make your user-interface file easier to manage, you can take advantage of "[include files](#)." This allows you to logically break the user-interface components into several smaller files. (For more information on the #include statement, see "[Include Files](#)" in the chapter on "[Using Macro Language Features](#)" [in the Macro Reference Guide](#).)

Customizing the Editor

Because no two users are alike, the editor offers extensive customization options.

From within the editor itself, you can configure the behavior of many commands and features using the Interactive Configuration facility (IConfig). Your configuration selections are in effect for the current session only, unless you update the editor program so that they are in effect each time the editor is started. To access IConfig, select "

[Full Configuration](#)" under Options on the pull-down Menu. To update the editor program with your configuration settings, select "Save Current Settings" under the Options Menu. Further customizing is provided through the editor's macro programming language, the SemWare Applications Language (SAL), which is complete with its own debugger. SAL allows you to modify the pull-down Menu, [the HelpLine](#) bars, and the key-assignment Help screen. It also allows you to assign commands to different or additional keys. You can use SAL to enhance or extend the editor's commands by writing and compiling your own Compiled Macro commands: if you do not like the way the editor performs an operation, you can change it; or if you need an additional feature, you can add it. (For more information on configuring the editor using IConfig, see the chapter on "[Configuring the Editor Interactively](#)." For more information on customizing the Menu, HelpLine, Help screen, key assignments, and commands in the editor, see the chapter on "[Fully Customizing the Editor](#)." For more information on the details of SAL, see [the Macro Reference Guide](#).)

Cut

Copies the marked block to the ClipBoard. The block is then removed from the file. The current contents of the ClipBoard are discarded before the operation starts unless the `_APPEND_` constant is passed to the command.

Syntax: [INTEGER Cut\(\[INTEGER flag\]\)](#) - flag is an optional integer that specifies whether [Cut\(\)](#) should keep or discard the current ClipBoard contents. If not specified, `_DEFAULT_` is used. Supported [values](#) for flag are: - `_DEFAULT_` specifies that the current ClipBoard contents should be discarded. - `_APPEND_` causes [Cut\(\)](#) to append the block to the current ClipBoard contents. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command fails if there is [not](#) enough memory to successfully complete the operation. [Cut\(\)](#) will not cut a block that exists outside the current file. If no block is marked and the `UseCurrLineIfNoBlock` variable is set ON, the current line is cut to the ClipBoard. If no block is marked and [UseCurrLineIfNoBlock](#) is set [OFF](#), no action occurs and zero is returned. See Also: [CutAppend\(\)](#), [Copy\(\)](#), [CopyAppend\(\)](#), [Paste\(\)](#), [PasteReplace\(\)](#) [CutToWinClip\(\)](#), [CutAppendToWinClip\(\)](#), [CopyToWinClip\(\)](#), [CopyAppendToWinClip\(\)](#), [PasteFromWinClip\(\)](#), [PasteReplaceFromWinClip\(\)](#) [DelBlock\(\)](#), [CopyBlock\(\)](#), [MoveBlock\(\)](#) [isBlockMarked\(\)](#), [isBlockInCurrFile\(\)](#)
Variables: [BlockId](#), [ClipboardId](#), [UseCurrLineIfNoBlock](#)

CutAppend

Copies the marked block to the ClipBoard, appending it to the current contents. The block is then removed from the file.

Syntax: [INTEGER CutAppend\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command fails if there is [not](#) enough memory to successfully complete the operation. [CutAppend\(\)](#) will not cut a block that exists outside the current file. If no block is marked and the `UseCurrLineIfNoBlock` variable is set ON, the current line is cut to the ClipBoard. If no block is marked and [UseCurrLineIfNoBlock](#) is set [OFF](#), no action occurs and zero is returned. To retrieve text from the ClipBoard, use the [Paste\(\)](#) command. **Examples:** // Causes <Ctrl Grey-> to Cut a block to the ClipBoard, // appending it to the current contents <Ctrl Grey-> [CutAppend\(\)](#) See Also: [Cut\(\)](#), [Copy\(\)](#), [CopyAppend\(\)](#), [Paste\(\)](#), [PasteReplace\(\)](#) [CutAppendToWinClip\(\)](#), [CutToWinClip\(\)](#), [CopyToWinClip\(\)](#), [CopyAppendToWinClip\(\)](#), [PasteFromWinClip\(\)](#), [PasteReplaceFromWinClip\(\)](#) [DelBlock\(\)](#), [CopyBlock\(\)](#), [MoveBlock\(\)](#) [isBlockMarked\(\)](#), [isBlockInCurrFile\(\)](#) **Variables:** [BlockId](#), [ClipboardId](#), [UseCurrLineIfNoBlock](#)

CutAppendToWinClip

Copies the marked block to the Windows Clipboard, appending it to the current contents. The block is then removed from the file.

Syntax: [INTEGER CutAppendToWinClip\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command fails if there is [not](#) enough memory to successfully complete the operation. [CutAppendToWinClip\(\)](#) will not cut a block that exists outside the current file. The block is removed after the copy is performed. If there is no marked block, but the `UseCurrLineIfNoBlock` variable is set ON, the current line is cut to the Clipboard. To retrieve text from the Windows Clipboard, use the [PasteFromWinClip\(\)](#) command. See Also: [CopyToWinClip\(\)](#), [PasteFromWinClip\(\)](#), [Cut\(\)](#), [Paste\(\)](#), [isBlockMarked\(\)](#), [isBlockInCurrFile\(\)](#), [CopyAppendToWinClip\(\)](#) **Variables:** [BlockId](#),

[UseCurrLineIfNoBlock](#)

CutToWinClip

Copies the marked block to the Windows Clipboard. The block is then removed from the file. The current contents of the Windows Clipboard are discarded before the operation starts.

Syntax: [INTEGER CutToWinClip\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command fails if there is [not](#) enough memory to successfully complete the operation. [CutToWinClip\(\)](#) will not cut a block that exists outside the current file. The block is removed after the copy is performed. If there is no marked block, but the [UseCurrLineIfNoBlock](#) variable is set ON, the current line is cut to the Clipboard. To retrieve text from the Windows Clipboard, use the [PasteFromWinClip\(\)](#) or [PasteReplaceFromWinClip\(\)](#) command, as appropriate. See Also: [CutAppendToWinClip\(\)](#), [CopyToWinClip\(\)](#), [CopyAppendToWinClip\(\)](#), [PasteFromWinClip\(\)](#), [PasteReplaceFromWinClip\(\)](#) [Cut\(\)](#), [CutAppend\(\)](#), [Copy\(\)](#), [CopyAppend\(\)](#), [Paste\(\)](#), [PasteReplace\(\)](#) [DelBlock\(\)](#), [CopyBlock\(\)](#), [MoveBlock\(\)](#) [isWinClipAvailable\(\)](#), [EmptyWinClip\(\)](#), [isBlockMarked\(\)](#), [isBlockInCurrFile\(\)](#) [GetStrFromWinClip\(\)](#) Variables: [BlockId](#), [UseCurrLineIfNoBlock](#)

Datadef

The datadef construct is used to embed data within a macro without having to maintain an additional file. This data is then inserted into the current buffer using the

[InsertData\(\)](#) command. The data in the buffer can be accessed and manipulated by the macro itself, or can be viewed and edited by the user. (For more information on manipulating data in [buffers](#), see "[Manipulating Text in Buffers](#)" under "[Buffers](#)" earlier in this chapter.) The format for data definition is: [datadef](#) data_name data_string ... data_string [end](#) [data_name] where: - [datadef](#) specifies this as a data definition. - data_name is a unique name associated with this datadef. - data_string is a line of data. It must be specified as a constant [string](#) enclosed within single or double quotes. Each separate data_string will be inserted as a separate line in the current buffer [when InsertData\(\)](#) is executed using this [datadef](#). - end signifies the end of the data definition. - The data_name may optionally be repeated after the end statement on the same line. The [InsertData\(\)](#) command references a datadef to insert the contents of the indicated data definition into the current file, just as if a file had been read using [InsertFile\(\)](#). The following example shows how a datadef can be used to define a Main macro template that is inserted into the current file when you press <F12>.

[datadef](#) MainTemplate 'proc [Main\(\)](#)' ' [integer](#) i' ' 'end [main](#)' [end](#) MainTemplate <F12> [InsertData](#)(MainTemplate)

DateFormat

Determines the format of the string returned by

[GetDateStr\(\)](#). Values: [INTEGER](#) - 1 - mm/dd/yy - 2 - dd/mm/yy - 3 - yy/mm/dd - 4 - mm/dd/yyyy - 5 - dd/mm/yyyy - 6 - yyyy/mm/dd - 7 - MMM dd yy - 8 - dd MMM yy - 9 - yy MMM dd - 10 - MMM dd yyyy - 11 - dd MMM yyyy - 12 - yyyy MMM dd **Default:** 4 Notes: mm = 2 digit month dd = 2 digit day yy = 2 digit year yyyy = 4 digit year MMM = 3 character abbreviation for Month If necessary, digits are padded with zero. Allows you to set the format of the date string that is returned by the [GetDateStr\(\)](#) command. [GetDateStr\(\)](#) is used in the [PrintFile\(\)](#) and [PrintBlock\(\)](#) commands [when](#) printing header and footer lines. Therefore, [DateFormat](#) will affect how the date is printed in a header or footer. **Affects:** [GetDateStr\(\)](#) See Also: [DateSeparator](#), [TimeFormat](#), [TimeSeparator](#), [PrintHeader](#), [PrintFooter](#)

DateSeparator

Determines the date separator used by

[GetDateStr\(\)](#). Values: INTEGER - ASCII value of a character. [Default](#): [ASC\("/"\)](#) Notes: Allows you to change the character that separates the elements of the date string returned from [GetDateStr\(\)](#). [GetDateStr\(\)](#) is used in the [PrintFile\(\)](#) and [PrintBlock\(\)](#) commands [when](#) printing header and footer lines. Therefore, [DateSeparator](#) will affect how the date is printed in a header or footer. [Affects](#): [GetDateStr\(\)](#) See Also: [DateFormat](#), [TimeFormat](#), [TimeSeparator](#), [PrintHeader](#), [PrintFooter](#)

Debugging Macros

At some point in the development or maintenance of a Compiled Macro, it may be helpful to debug the macro using the SAL macro debugger. The debugger allows you to examine the execution of a macro to verify proper execution or to help determine the cause of problems while executing. When you invoke the debugger and execute a macro, control of the macro is passed to you. You can then step through execution of the macro, display the contents of affected files and variables, and even modify the contents of variables as the macro executes.

To debug a macro file, execute the `Debug()` command (select "Debug" under the Macro Menu). If the file you are currently editing has an extension of ".S", `Debug()` operates on that file; otherwise, `Debug()` issues a prompt to request the name of the source file to debug.

Once `Debug()` determines the file on which to operate, the following occurs:

- If the selected source file is already loaded, it is first saved, if necessary. (This occurs if the source file had been modified since it was last saved.)

Otherwise, if the source file is not already loaded, it is loaded into the editor at this time.

- The source file is then compiled, if necessary, with "debug information" included. (This occurs if `Debug()` determines that the current source file has

[not](#) already been compiled with debug information.) If the compiler detects any errors or warnings in the macro source, you will be placed in the Compile window. You can then correct any errors or unacceptable warning conditions, and re-execute the `Compile()` command. Upon successful compilation, select "Debug" from the prompt that is displayed to continue the debugging process. If the compiler detects only warning conditions (with no fatal errors), you can optionally continue the debugging process (without re-compiling) by pressing <Ctrl Enter>. (For more information on the `Compile()` command, see "[Compiling Macros](#)" earlier in this chapter.) - The Compiled Macro (.MAC) file, for the selected source file, is then loaded into the editor, if a current Compiled Macro file for the source file is [not](#) already loaded. If a current Compiled Macro file is already loaded, `Debug()` gives you the option to either re-load the Compiled Macro file and begin the Debug process once again, or to resume the Debug process without reloading the macro. - The Debug process is now ready to begin, or possibly resume. The Debug window is displayed, and the

source file for the macro you are debugging is placed in this window. The Debug [menubar](#) is placed at the top of this window, and the Debug helpline is placed at the bottom. When you begin a Debug process, breakpoints are automatically set on the WhenLoaded() and Main() macros, if these are included in the macro file. The cursor is placed in the source at the first breakpoint, if any are set; otherwise, the cursor is placed at the top of the source file. No macros are executed until you select one of the available tracing options. Once you select the first tracing option, initial execution of the macro file occurs. If you resume (rather than begin) a Debug process, you are returned to the previous execution state of the macro. Initial execution of the macro file is bypassed. At this point, you control the continuing execution of the macro by selecting the appropriate Debug options. Debug options can be accessed from the Debug menu. (To activate the Debug menu, press <Escape>, or press <Alt> plus the quick-key of the desired menu category). Alternatively, you can execute a Debug option by pressing the key assignment for that option. (All available options are included in the Debug menu. The menu also shows the key assignments for the Debug options.) Debug options are available that allow you to set additional breakpoints, trace or step through code, display the contents of affected files and variables, and even modify the contents of variables as the macro executes. The following describes the operation of the most commonly used Debug options.

- Setting breakpoints: Breakpoints are used to temporarily halt the macro when a certain point in the code is reached. Options are available to set, toggle, and delete breakpoints. Just position the cursor on the line on which you want to set or delete a breakpoint, and execute the desired option. Select the "BreakPoint" menu category to display the available options. Breakpoints remain set until you specifically delete them, or until you unload the current macro from memory or debug another macro.
- Tracing through code: The basic modes for tracing through code are "Run," "Step," [and](#) "Trace." Select the "Run" menu category to display these options. "Run" returns control to the current macro and allows it to run until a breakpoint is encountered. The debugger then halts the execution of the macro and re-displays the Debug window. "Step" causes the macro to execute a single line of code and then automatically suspend execution and re-display the Debug window. Each time you select the step option, the next line of code is executed. This feature does [not](#) require breakpoints to be manually set. And unlike trace, this option bypasses procedure calls within the macro. "Trace" is similar to step, except that it also executes user-written procedure calls. (Native commands are bypassed.)
- Animation: Animation automates the tracing mode of the debugger. By using animation, you can run, step, or trace through lines of code at regular time intervals without having to continually select a run, step, or trace option. This is especially useful when you want to watch or view variables as they change within a looping construct, without having to manually continue through each line in the loop. "Animate Step" [and](#) "Animate Trace" cause execution to suspend temporarily as each new line of the macro is executed. "Animate Run" temporarily suspends execution only when breakpoints are encountered. When you invoke animation, you will be prompted to provide the "animation delay." This is specified in 1/18 second intervals, and indicates the amount of time to display the Debug window (along with any associated "watch" [windows](#)) before continuing execution. To halt animation, press <LeftShift><RightShift> at a point when the Debug window is displayed.
- Exiting from the Debug process: Once the Debug process begins (that is, once the Debug window is initially displayed), you should [not](#) terminate it until you completely trace through the code (via the run, step, or trace options). Terminating the Debug process before the macro finishes execution would leave the editor in an unstable state. However, if a situation occurs where you must terminate the Debug process prematurely, select "[Halt](#)" from the "Run" menu. This action is [not](#) recommended, and should only be taken in extreme cases where you have no other choice.
- Watching and modifying memory [variables](#): To watch a variable (that is, to display its contents as the macro executes), select "Add Watch" from the "Data" menu. You will be prompted for the name of the variable you wish to display. As the macro executes, the value of this variable will be displayed in the lower "watch window." (You can also select a variable to watch by placing the cursor on the desired variable name in the source file and pressing <Ctrl F7>.) If you specify a global variable name, the value of the variable will always be available. If you specify a local variable, the contents of the variable will only be available when a function is traced in which a variable by that name exists. For local variables which are not in the currently active procedure, the message "[not in scope](#)" will be displayed. To display or modify the contents of a variable, select "Modify/Evaluate" from the "Data" menu. You will be prompted for the name of the desired variable. Once you have specified a variable name, you can display the current value by pressing <Enter>, or change the value by pressing <Tab>. (You can also specify a variable to display or modify by placing the cursor on the desired variable name in the source file and pressing <Ctrl F4>.) If you specify a string variable, you can display or modify either all or only part of the string. For example, given string "s", the entire [string](#) can be displayed or modified by specifying s as the variable name. To view or modify only a part of the string, you can use one of the following

formats: s[n] s[n:len] s[n..p] where n is a constant number indicating a starting position, len is a constant number indicating a length, and p is a constant number indicating an ending position. Note: Local variables are only initialized when done so by the macro itself. Thus, if you select a local variable to watch, or to display or modify, the value of the variable may be "garbage" until the variable is initialized by the macro.

DefaultExt

List of

[default](#) file extensions. **Values:** [STRING](#) (maximum of 128 characters) A quoted [string](#) containing one or more extensions separated by spaces. **Default:** "ui s si c cpp h java pas inc bat prg txt doc html asm" Notes: When a filename, without an extension, is specified on the command line or in [EditFile\(\)](#), the editor attempts to load the file from disk. If it does not exist, the editor attempts to match the filename with each of the default extensions. Once a match is found, the process stops and the matched file is loaded. If a match is not found, the editor treats the filename you originally entered as a new file. To keep the editor from checking the disk for files from the default extension list, enter the filename with a trailing "." (period). **Affects:** [EditFile\(\)](#), [EditThisFile\(\)](#)

DelAllBookMarks

Removes all

[bookmarks](#). **Syntax:** [DelAllBookMarks\(\)](#) Returns: Nothing. Notes: Removes all bookmarks that have been placed with the [PlaceMark\(\)](#) command. This is a simple and efficient way to remove all [bookmarks](#). See Also: [PlaceMark\(\)](#), [GotoMark\(\)](#), [DelBookMark\(\)](#), [GetBookMarkInfo\(\)](#)

DelAnyChar

Deletes the character at the cursor position.

Syntax: [INTEGER DelAnyChar\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Unlike [DelChar\(\)](#), [DelAnyChar\(\)](#) will join the next line with the current line when executed at or beyond the end of the current line. See Also: [DelChar\(\)](#), [DelCharOrBlock\(\)](#), [DelAnyCharOrBlock\(\)](#), [BackSpace\(\)](#), [JoinLine\(\)](#)

DelAnyCharOrBlock

If the cursor is in a marked block, the block is deleted. Otherwise, deletes the character at the cursor position.

Syntax: [INTEGER DelAnyCharOrBlock\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Unlike [DelCharOrBlock\(\)](#), [DelAnyCharOrBlock\(\)](#) will join the next line with the current line when executed at or beyond the end of the current line. See Also: [DelAnyChar\(\)](#), [DelBlock\(\)](#), [DelChar\(\)](#), [DelCharOrBlock\(\)](#), [BackSpace\(\)](#), [JoinLine\(\)](#)

DelAutoLoadMacro

Removes a Compiled Macro file from the list of macro files to be automatically loaded on editor startup.

Syntax: [INTEGER DelAutoLoadMacro](#)([STRING](#) macro_name) - macro_name is the name of the macro file to be removed. The name must be an exact match (not including case) for it to be removed from the list. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: The list of macro files to be loaded is maintained in the autoloading file TSELOAD.DAT. This file provides a simple, optional mechanism for automatically loading macro files at the beginning of each editing session. If TSELOAD.DAT is used, it is maintained in the editor load directory. On startup, the editor loads any macro files found in this autoloading file, using the [LoadMacro](#)() command. See Also: [AddAutoLoadMacro](#)(), [EditAutoLoadList](#)(), [isAutoLoaded](#)()

Delay

Causes the editor to pause for the amount of time specified.

Syntax: [Delay](#)([INTEGER](#) duration) - duration is the number of 0.055 parts of a second that the editor should be idle. (0.055 is about 1/18 of a second.) Returns: Nothing. Notes: The delay duration is accurate to within [about](#) 0.055 second. **Examples:** [Delay](#)(36) // delay for approximately two seconds See Also: [Alarm](#)(), [Sound](#)()

DelBlock

Deletes the marked block if it exists in the current file.

Syntax: [INTEGER DelBlock](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: [DelBlock](#)() will not delete a block outside the current file. If no block is marked in the current file or there was not enough memory to complete the operation, no action occurs and zero is returned. Deleted blocks are stored in the Deletion Buffer and can be retrieved with the [UnDelete](#)(), [GlobalUnDelete](#)(), and [PasteUnDelete](#)() commands. To delete a block without it being stored in the Deletion Buffer, use the [KillBlock](#)() command. Alternatively, the KillMax variable may be set to 0, which is essentially what the [KillBlock](#)() command does. Idioms: /

***** To delete a block WITHOUT it being stored in the Deletion Buffer, temporarily set the KillMax variable to 0, delete the block, and then restore KillMax to its previous value. *****/ [integer](#) km = [Set](#)([KillMax](#), 0)

[DelBlock](#)() [Set](#)([KillMax](#), km) See Also: [UnDelete](#)(), [GlobalUnDelete](#)(), [isBlockInCurrFile](#)(), [isBlockMarked](#)(), [PasteUnDelete](#)(), [KillBlock](#)(), [DelAnyCharOrBlock](#)(), [DelCharOrBlock](#)(), **Variables:** [KillMax](#), [BlockId](#)

DelBookMark

Removes a specific bookmark.

Syntax: [DelBookMark](#)([STRING](#) bookmark_name) - bookmark_name is the single-character name (A through Z, case-insensitive) of the bookmark to remove. Returns: Nothing. Notes: This command is used to remove a specific bookmark placed with the [PlaceMark](#)() command. **Examples:** // remove bookmark a [DelBookMark](#)("a") See Also: [PlaceMark](#)(), [GotoMark](#)(), [DelAllBookMarks](#)(), [GetBookMarkInfo](#)()

DelBufferVar

Deletes the specified buffer variable.

Syntax: [INTEGER DelBufferVar](#)([STRING](#) buffer_var_name [, [INTEGER](#) buffer_id]) - buffer_var_name is the name of the buffer variable to delete. - buffer_id is an optional buffer id, as returned by [GetBufferId](#)(). Returns: Non-zero if buffer_var_name can be deleted (it must exist); [otherwise](#), zero ([FALSE](#)). Notes: If buffer_id is not specified, the current buffer is assumed. See Also: [ExistBufferVar](#)(), [GetBufferInt](#)(), [GetBufferStr](#)(), [SetBufferInt](#)(), [SetBufferStr](#)()

DelChar

Deletes the character at the cursor position. Will not delete newline characters.

Syntax: [INTEGER DelChar](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: [DelChar](#)() does not join lines. Thus, it will not join the next line with the current line when executed at or beyond the end of the current line, and returns zero under these conditions. For a key definition that joins the next line with the current line, replace the key definition in TSE.UI (in the UI\ editor subdirectory) with: /
 ***** [CurrChar](#)() returns < 0 if the cursor is at or beyond the end of the current line. This is used to determine whether to join the next line, or issue the normal [DelChar](#)() command. *****/ [proc](#) mDelChar() [if](#) [CurrChar](#)() >= 0 [DelChar](#)() [else](#) [JoinLine](#)() [endif](#) [end](#) mDelChar() See Also: [BackSpace](#)(), [DelCharOrBlock](#)(), [DelAnyCharOrBlock](#)(), [DelAnyChar](#)(), [JoinLine](#)()

DelCharOrBlock

If the cursor is in a marked block, the block is deleted. Otherwise, deletes the character at the cursor position. Will not delete newline characters.

Syntax: [INTEGER DelCharOrBlock](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: [DelCharOrBlock](#)() does [not](#) join lines. Thus, it will [not](#) join the next line with the current line when executed at or beyond the end of the current line, and returns zero under these conditions. For a key definition that joins the next line with the current line, see the [DelAnyCharOrBlock](#)() command. See Also: [DelAnyCharOrBlock](#)(), [DelBlock](#)(), [DelChar](#)(), [DelAnyChar](#)(), [BackSpace](#)(), [JoinLine](#)()

DelGlobalVar

Deletes the specified session global variable.

Syntax: [INTEGER DelGlobalVar](#)([STRING](#) var_name) - var_name is the name of the session global variable to delete. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: [FALSE](#) is returned if var_name does [not](#) exist.
Examples: [DelGlobalVar](#)("myvar") See Also: [GetGlobalInt](#)(), [GetGlobalStr](#)(), [SetGlobalInt](#)(), [SetGlobalStr](#)(), [ExistGlobalVar](#)()

DelHistory

Deletes all history entries associated with a history number, and places the history number back on the available history list.

Syntax: [DelHistory](#)([INTEGER](#) history_num) - history_num specifies the history to be removed. Returns: Nothing. Notes: history_num should be in the range 1..127, which constitutes the user history [numbers](#). The pre-defined [constant values](#) (see [AddHistoryStr\(\)](#)) that represent the built-in histories cannot be deleted. The [AddHistoryStr\(\)](#) command contains detailed information [about](#) the use of history in the macro language. See Also: [AddHistoryStr\(\)](#), [GetFreeHistory\(\)](#), [DelHistoryStr\(\)](#), [FindHistoryStr\(\)](#), [GetHistoryStr\(\)](#), [NumHistoryItems\(\)](#)

DelHistoryStr

Deletes an item (string) from the History Buffer.

Syntax: [INTEGER DelHistoryStr](#)([INTEGER](#) history, [INTEGER](#) n) - history is an integer specifying the history list from which the item will be deleted. history can be: - A pre-defined constant value representing one of the built-in histories. See the [AddHistoryStr\(\)](#) command for a listing of these [constants](#). - A number in the range 1..127. - A number returned by [GetFreeHistory\(\)](#), which will be in the range 1..127. - n is the number, within the specified history list, of the history item to delete. n should be in the range 1 through the value returned by [NumHistoryItems](#)(history), where 1 represents the item added most recently, and higher numbers represent older items. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If the specified history list does not exist, or n is greater than the number of items that exist for that particular history list, then this command has no effect, and [FALSE](#) is returned. **Examples:** // To delete an item such as "foo.bar" from history: [DelHistoryStr](#)(histno, [FindHistoryStr](#)("foo.bar", histno)) // Prune history to specific number of items [proc](#) PruneHist([integer](#) histno, [integer](#) num2keep) [repeat until NOT DelHistoryStr](#)(histno, num2keep + 1) [end](#) See Also: [GetHistoryStr\(\)](#), [NumHistoryItems\(\)](#), [GetFreeHistory\(\)](#), [AddHistoryStr\(\)](#), [FindHistoryStr\(\)](#) **Variables:** [MaxHistoryPerList](#), [PersistentHistory](#)

DelLeftWord

Deletes the word to the left of the cursor.

Syntax: [INTEGER DelLeftWord](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: The current value of the WordSet variable determines what the editor considers a word. [DelLeftWord](#)() deletes characters beginning with the character to the left of the cursor, and continues until a character that is not part of the current WordSet is encountered. If the character to the left of the cursor is not part of the WordSet, only that character is deleted. This command joins the current line with the previous line when issued at column 1. When issued at column 1 of the first line of the file, no action occurs and zero is returned. See Also: [DelRightWord](#)() **Variables:** [WordSet](#)

DelLine

Deletes the current line.

Syntax: [INTEGER DelLine](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command fails if there is [not](#) enough memory to successfully complete the operation. Upon deletion of the current line, the cursor is placed on the following line with the cursor column remaining unchanged. The deleted line is stored in the Deletion

Buffer and can be retrieved via the [UnDelete\(\)](#) or [GlobalUnDelete\(\)](#) commands. To delete a line without saving the deleted line in the Deletion Buffer, use the [KillLine\(\)](#) command, or use the following idiom. Idioms: /

***** To delete a line WITHOUT it being stored in the Deletion Buffer, temporarily set the KillMax variable to 0, delete the line, and then restore KillMax to its previous value. *****/ [integer](#) km = [Set](#)(KillMax, 0) [DelLine](#)() [Set](#)(KillMax, km) See Also: [KillLine\(\)](#), [AddLine\(\)](#), [InsertLine\(\)](#), [UnDelete\(\)](#), [GlobalUnDelete\(\)](#) Variables: [KillMax](#)

DelRightWord

Deletes the word to the right of the cursor.

Syntax: [INTEGER DelRightWord\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: The current value of the WordSet variable determines what the editor considers a word. [DelRightWord\(\)](#) deletes characters beginning with the character at the cursor position, and continues until a character that is not part of the current WordSet is encountered. If the character at the cursor position is not in the WordSet, only that character is deleted. This command joins lines when issued at or beyond the end of the current line. When issued at the end of the last line of the file, no action occurs and zero is returned. The deleted text is stored in the Deletion Buffer, and can be retrieved via the [UnDelete\(\)](#) or [GlobalUnDelete\(\)](#) commands. See Also: [DelLeftWord\(\)](#), [UnDelete\(\)](#), [GlobalUnDelete\(\)](#) Variables: [WordSet](#)

DelStr

Deletes specified portions of a string variable.

Syntax: [STRING DelStr](#)([STRING](#) s, [INTEGER](#) p, [INTEGER](#) len) - s is the string from which to delete. - p is the position from which to start deleting. - len is the number of characters to delete. Returns: The new string with the specified portion deleted. Notes: If p is out of range, or len is less than or equal to 0, nothing is deleted from s.

Examples: [string](#) s[80] = "A rose by any other name" s = [DelStr](#)(s, 15, 6) // s is now "A rose by any name" s = [DelStr](#)(s, 7, 12) // s is now "A rose" s = [DelStr](#)(s, 1, 2) // s is now "rose" See Also: [InsStr\(\)](#), [SubStr\(\)](#), [Pos\(\)](#), [LeftStr\(\)](#), [RightStr\(\)](#), [StrCount\(\)](#), [StrFind\(\)](#), [StrReplace\(\)](#)

DelToEol

Deletes the text from the cursor position to the end of the current line.

Syntax: [INTEGER DelToEol\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command fails if there is [not](#) enough memory to successfully complete the operation. Zero is returned if this command is executed when the cursor is at or past the end of the current line. The character at the cursor position is included with the text that is deleted. This command does [not](#) cause lines to be joined. The deleted text is stored in the Deletion Buffer, and can be retrieved via the [UnDelete\(\)](#) or [GlobalUnDelete\(\)](#) commands. To delete text without placing the deleted text in the Deletion Buffer, use the [KillToEol\(\)](#) command. See Also: [KillToEol\(\)](#), [DelChar\(\)](#), [DelLine\(\)](#), [UnDelete\(\)](#), [GlobalUnDelete\(\)](#) Variables: [KillMax](#)

DelWindow

Deletes an adjacent editing window.

Syntax: [INTEGER DelWindow\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If more than two editing [windows](#) are open and non-zoomed, this command prompts for the window to delete, which must be adjacent to the current window. The window to be deleted is indicated by pressing the cursor key that "points" to that window. If exactly two [windows](#) are open and non-zoomed, this command does [not](#) prompt; it deletes the non-current window. When a window is deleted, windows adjacent to the deleted window grow to fill in the empty space. This command [affects](#) only [windows](#) created by [HWindow\(\)](#) and [VWindow\(\)](#), and not those created by the video output commands. Zero is returned if <Escape> is pressed at the "Window to delete:" prompt, if there is no window in the direction indicated, there is only one window, or the window is zoomed. See Also: [CloseWindow\(\)](#), [HWindow\(\)](#), [VWindow\(\)](#), [OneWindow\(\)](#), [ZoomWindow\(\)](#)

DetabOnLoad

If on, tabs are converted to the appropriate number of spaces when a file is loaded into the editor.

Values: BOOLEAN **Default:** [OFF](#) Notes: Tabs are expanded based on the value of the TabWidth editor variable. See Also: [EntabOnSave](#), [TabWidth](#)

Disable

Disables a previously enabled set of key definitions.

Syntax: [Disable\(KEYDEF keydef_name\)](#) - keydef_name is the name of the keydef to disable. Returns: Nothing. Notes: Once a keydef has been enabled via [Enable\(keydef_name\)](#), the [keydef](#) will remain in effect until [Disable\(keydef_name\)](#) is called. For more information on the keydef identifier, see "[Keydef](#)" and "[Process\(\) and Keydefs](#)" in the chapter on "[Using Macro Language Features](#)." **Examples:** See the [Process\(\)](#) command for an example. See Also: [Enable\(\)](#), [Process\(\)](#)

DisplayBoxed

Determines whether the editor should display a full border around the entire screen, so that each window on the screen (including a single window) is fully "boxed."

Values: [INTEGER](#) - 0 - do not "box" the entire display - 1 - only "box" the entire display if the mouse is enabled and present - 2 - always "box" the entire display **Default:** 1 See Also: [MouseEnabled](#)

DisplayMode

Sets or queries the current display mode for the current buffer.

Syntax: [INTEGER DisplayMode](#)([INTEGER mode](#)) - mode is the optional display mode to set. Supported values for mode are: - [_DISPLAY_TEXT_](#) places the screen display in standard display mode. - [_DISPLAY_HEX_](#) places the screen display in Hex display mode. - [_DISPLAY_PICKFILE_](#) places the screen display in the PickFile display mode. Returns: The display mode in effect when this command is invoked. Notes: [Displaymode\(\)](#) only affects the current buffer. When [Hex mode](#) is enabled, each line is displayed in both text and hexadecimal format. In Hex mode, the hexadecimal values are displayed on the right, while the text is displayed on the left. (See the [HexEdit\(\)](#) command for information on how to set ON hex-editing mode, to allow the user to enter or modify the hexadecimal format of the data on the right.) Activating Hex mode does not place the file in Binary mode, but rather simply changes the view from text to hexadecimal. To place a file in Binary mode, it must be loaded in Binary mode using the -b switch, or by using the [BinaryMode\(\)](#) command. PickFile display mode is only valid for files in which the data was generated with the [BuildPickBufferEx\(\)](#) command. The [PickFile\(\)](#) command uses this display mode. **Examples:** // Toggle between Hex and text display modes: [DisplayMode](#)([iif](#)([DisplayMode](#)() == [_DISPLAY_TEXT_](#), [_DISPLAY_HEX_](#), [_DISPLAY_TEXT_](#))) See Also: [BinaryMode\(\)](#), [EditFile\(\)](#), [EditThisFile\(\)](#), [BuildPickBufferEx\(\)](#), [PickFile\(\)](#), [HexEdit\(\)](#)

DistanceToTab

Returns the distance between the current cursor position and the next or previous tab stop.

Syntax: [INTEGER DistanceToTab](#)([INTEGER right](#)) - right is an optional integer that defaults to zero. If right has a non-zero value, the distance to the next tab stop is computed. If right has a value of zero, the distance to the previous tab stop is computed. Returns: The distance to the next or previous tab stop. Notes: The distance from the current cursor location to the next or previous tab stop is based on the current setting of the TabType and [TabWidth variables](#). **Examples:** One usage for this command is to create tabbing commands that are only [cursor](#) movement commands, even in [insert mode](#). `proc mGotoNextTabStop() GotoColumn(CurrCol() + DistanceToTab(1)) end proc`
`mGotoPreviousTabStop() GotoColumn(CurrCol() - DistanceToTab()) end` <Tab> mGotoNextTabStop() <Shift Tab> mGotoPreviousTabStop() See Also: [Variables: TabType, TabWidth, VarTabs](#)

Dos

Passes a command to the operating system for execution.

Syntax: [INTEGER Dos](#)([STRING cmdline](#) [, [INTEGER flag](#)]) - cmdline is the optional command line passed to the operating system, in order to execute an external command from within the editor. If omitted, the editor will prompt for the command to execute. - flag is an optional integer that determines how the [Dos\(\)](#) command operates. If [not](#) specified, [_DEFAULT_](#) is used. Supported [values](#) for flag are: - [_DEFAULT_](#) causes the screen to be cleared and the command issued. After completion, the editor prompts with "Press a key to continue...". Once a key is pressed, the editor's screen is restored. - [_DONT_PROMPT_](#) causes the screen to be cleared and the command issued. After completion, control returns immediately to the editor, and the editor's screen is restored. This option differs from [_DEFAULT_](#) (above) in that the editor does not issue the "Press a key to continue..." prompt. - [_DONT_CLEAR_](#) causes the command to be issued without the screen being cleared. After completion, control returns immediately to the editor. The editor does not issue the "Press a key to continue..." prompt, and the editor's screen is not restored. This setting is useful for piping the command's output, or when the command itself restores the application screen after completion. - [_TEE_OUTPUT_](#) is useful when the external command's output is being redirected to another file. Using the [_TEE_OUTPUT_](#) flag will also place that redirected output on the screen. This is primarily useful for long-running commands, in order to give the user feedback as to the progression of the command. Good examples would be a long MAKE, or compiling with a very slow compiler. The return code of the command passed to the operating

system is available via the [DosIOResult\(\)](#) command. - `_RETURN_CODE_` causes the [Dos\(\)](#) command to process in such a way as to make the return code from the command passed to the operating system available via the [DosIOResult\(\)](#) command. - `_RUN_DETACHED_` causes cmdline to be run as a detached process. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command fails if there is [not](#) enough memory to successfully complete the operation. This command includes a history list of previous entries passed to it. See [AddHistoryStr\(\)](#) for additional information. [Examples:](#) `***** Example 1 The user is prompted for a command. *****/ Dos() / ***** Example 2 Issue the DIR command. The screen will be cleared, the command executed, and after completion, the editor will issue a "continue" prompt. *****/ Dos("dir *.bat") / ***** Example 3 Issue the dir command, and pipe the output to the file "save.dat". The editor's screen will not be cleared, and control will return to the editor without a "continue" prompt being issued. *****/ Dos("dir *.bak >save.dat", _DONT_CLEAR_) / ***** Example 4 Invoke the CC program, passing it the name of the currently edited file (which is saved first, so that the disk copy and what is in memory are the same), and pipe the output to the file "results". *****/ SaveFile() Dos("CC " + CurrFilename() + ">results", _DONT_CLEAR_) / ***** Example 5 Invoke the CC program, passing it the name of the currently edited file (which is saved first, so that the disk copy and what is in memory are the same), and pipe the output to the file "results". At the same time, any piped output is also displayed on the screen. *****/ SaveFile() Dos("CC " + CurrFilename() + ">results", _DONT_PROMPT_ | _TEE_OUTPUT_) See Also: IDos\(\), Shell\(\), AddHistoryStr\(\), DosIOResult\(\), StartPgm\(\)`

DosCmdLine

The command line passed to the editor. For use within macros.

[Values:](#) [STRING](#) (maximum of 128 characters) [Default:](#) The command-line [string](#). Limits: Setting this has an effect only in the installed user-interface [WhenLoaded macro](#). Notes: This variable can be used to add custom command-line arguments to the editor, or to peek at the command line passed to the editor and take user-determined appropriate action. In the installed user-interface file, the `WhenLoaded()` macro can access the command line (before the editor processes it). This allows the `WhenLoaded()` macro to change the command line that the editor actually processes. For example, additional command-line options could be added, a default filename could be loaded, etc. Note, however, that if the "Augment LoadDir" (-i) command-line option is specified, the editor processes this parameter before calling the `WhenLoaded` macro. See Also: [MacroCmdLine](#)

DosIOResult

Returns result information about the last-executed

[Dos\(\)](#) command. This command is intended for ADVANCED USAGE. [Syntax:](#) [INTEGER](#) [DosIOResult\(\)](#) Returns: The return code of the external command run by the last-executed [Dos\(\)](#) command, along with the return code of the last-executed [Dos\(\)](#) command itself. Notes: For this command to work, either `_RETURN_CODE_` or `_TEE_OUTPUT_` must have been specified as a flag value in the last-executed [Dos\(\)](#) command. When this command is executed, the two return codes are placed in the lower 16 bits of the returned (32-bit) integer. The return code of the relevant external command is placed in the upper byte (of the lower 16 bits); the return code of the [Dos\(\)](#) command is

placed in the lower byte (of the lower 16 bits). [Examples](#): `proc CheckDosIOResult() integer rc Dos("pkzip", _RETURN_CODE_) rc = DosIOResult() Warn("Return code of pkzip: ", HiByte(rc), " Return from Dos() command: ", LoByte(rc)) end` See Also: [Dos\(\)](#)

Down

Moves the cursor down one line in the current file. The cursor column does

[not](#) change. [Syntax](#): `INTEGER Down()` Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command returns zero if the cursor is on the last line of the file. If the cursor is on the last line of the current window, the text will scroll up one line. Idioms: `/****** To perform a given operation on every line of the current file, code similar to the following can be used.`

`*****/ BegFile() repeat // some operation until not Down()` See Also: [Up\(\)](#), [RollDown\(\)](#), [ScrollDown\(\)](#)

DrawBox

Draws a box around the current video output window.

[Syntax](#): `DrawBox(INTEGER boxtype, INTEGER attribute)` - boxtype is the box-drawing character that will be used. - attribute is the color to use for the box. Supported [values](#) for boxtype are: - 1 - single-line characters - 2 - double-line top, single-line side characters - 3 - double-line side, single-line top characters - 4 - double-line characters - 5 - small block characters - 6 - large block characters Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [Color\(\)](#), [Window\(\)](#)

Drawing Lines in the Text

The editor offers a customizable means for drawing lines and boxes, which can be useful for creating diagrams, charts, and tables. For this,

[LineDrawing](#) mode is provided. When LineDrawing mode is ON, the cursor movement keys (<CursorUp>, <CursorDown>, <CursorLeft>, and <CursorRight>) draw lines on the screen as the cursor moves. You can select the style used to draw lines from a predefined set, or you can choose your own line-drawing character. To toggle LineDrawing mode ON and OFF, choose "Line Drawing" under the Util Menu. When this mode is ON, an "L" displays on [the StatusLine](#). To specify the LineDrawing style, select "Line Type" under the Util Menu, and choose one of the following: Selection Line-Drawing Characters _____
Single +-+ Single-line top and side Double top *** Double-line top, single-line side Double side *- * Double-line side, single-line top Double both *** Double-line both top and side ASCII +-| ASCII top and side Eraser Blank User-defined [*] User-defined top and side To select a style, move the cursor bar to the desired style and press <Enter>, or simply press the quick-key associated with the desired style. If you select the user-defined style, you then input the character you wish to use and press <Enter> again. (The numeric keypad can be used to input the decimal value of a character that is not directly available from the keyboard. For more information on using the numeric keypad to input characters, see the Tip under "[Commands to Add Text](#)" in the chapter on "[Manipulating Text](#).")

+----- Tip -----+ | The first four line-drawing styles listed above, along with the || standard user-defined character, use graphics characters. To print || these characters, your printer must be set to a graphics mode. | +-----+

DupLine

Makes a copy of the current line and adds it immediately following the current line.

Syntax: [INTEGER DupLine\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command fails if there is not enough memory to successfully complete the operation. The newly-added line becomes the current line. The cursor column does not change. See Also: [AddLine\(\)](#), [InsertLine\(\)](#)

EditAutoLoadList

Allows interactive editing (adding and deleting) of the list of macro files to be automatically loaded on editor startup.

Syntax: [EditAutoLoadList\(\)](#) Returns: Nothing. Notes: The list of macro files to be loaded is maintained in the autoload file TSELOAD.DAT. This file provides a simple, optional mechanism for automatically loading macro files at the beginning of each editing session. If TSELOAD.DAT is used, it is maintained in the editor load directory. On startup, the editor loads any macro files found in this autoload file, using the [LoadMacro\(\)](#) command. See Also: [AddAutoLoadMacro\(\)](#), [DelAutoLoadMacro\(\)](#), [isAutoLoaded\(\)](#)

EditBuffer

A replacement for the common sequence "

[CreateBuffer\(\)](#) ... [InsertFile\(\)](#)", typically to create a system buffer and load a file into it without causing any hooks to be called. **Syntax:** [INTEGER EditBuffer](#)([STRING](#) fn [, [INTEGER](#) type [, [INTEGER](#) reclen]]) - fn is the name of the file to load into the newly-created temporary buffer. - type is an optional integer parameter that specifies the type of buffer to be created. If not specified, `_SYSTEM_` is used. Supported [values](#) for type are: - `_NORMAL_` causes the buffer to have no special attributes associated with it. Buffers created by the [EditFile\(\)](#) command have this buffer type. - `_HIDDEN_` simply causes the buffer to not show up in the list of files, and makes the buffer inaccessible via the [NextFile\(\)](#) and [PrevFile\(\)](#) commands. The PickList of loaded macros (optionally displayed by the [ExecMacro\(\)](#) command) is stored in a `_HIDDEN_` buffer. - `_SYSTEM_` causes the buffer to be hidden, and places the buffer in a raw mode, so that trailing spaces are never removed, and tabs are treated as a normal character. Buffers created with [CreateTempBuffer\(\)](#) have this buffer type. It is strongly recommended that system [buffers not](#) be used for interactive editing. - reclen is an optional integer parameter. If passed and non-zero, the binary record length of the buffer is set to reclen. Returns: The non-zero id number of the newly-created buffer, if successful; [otherwise](#), zero ([FALSE](#)). Notes: This command creates a buffer of the specified type, with optional reclen, and loads fn into it. No hooks are called. The buffer is unnamed. The newly created buffer becomes the current buffer. This a common operation performed by macros, that save and restore configuration or other types of information. It performs the following sequence of instructions: [if](#) [CreateBuffer](#)("", type) [BinaryMode](#)(reclen) [PushBlock](#)() [InsertFile](#)(fn, _DONT_PROMPT_) [PopBlock](#)() [endif](#) See Also: [CreateBuffer\(\)](#), [CreateTempBuffer\(\)](#), [EditFile\(\)](#), [EditThisFile\(\)](#), [InsertFile\(\)](#), [LoadBuffer\(\)](#)

EditFile

Loads a file into the editor and makes it the current file.

Syntax: [INTEGER EditFile](#)([[STRING](#) fn [[STRING](#) fn]... [, [INTEGER](#) flag]]) - fn is an optional string expression specifying the file(s) to be loaded (and may [include command-line options](#)). fn can include long filenames and UNC network paths. If a filename includes leading or embedded spaces, that filename must be enclosed in quotes. - flag is an optional integer that affects the operation of [EditFile](#)(). Supported [values](#) for flag are: - [_DONT_PROMPT_](#) specifies that [EditFile](#)() should [not](#) display any warning or informational messages if a problem occurs. - [_DONT_LOAD_](#) specifies that [EditFile](#)() should [not](#) actually load any files, but simply place them in the editor's Ring of files. The file can later be loaded by switching to it; or by calling [EditFile](#)() again, with the appropriate filename, and without the [_DONT_LOAD_](#) flag. - [_FROM_CMDLINE_](#) specifies that [EditFile](#)() should perform as if it were called from the command line. The editor uses this to determine whether to load or to prompt for ambiguous filenames, as determined by the LoadWildFromDos and LoadWildFromInside variables. (See the WhenLoaded() macro in the TSE.UI user-interface file (located in the UI\ editor subdirectory) for examples of using this value with the [EditFile](#)() command.) Returns: The non-zero id number of the file just edited if successful; [otherwise](#), zero ([FALSE](#)). Notes: fn can also [include command-line options](#), if desired. (For more information on available [command-line options](#), see "[Command-Line Options](#)" in the chapter on "[Starting the Editor](#)" in [the User's Guide](#).) If no filename is specified, the editor will prompt for a filename. If an ambiguous filename is specified, the editor either pops up a list of all matching files from which the user can select a file, or automatically selects all matching files, depending on the setting of the LoadWildFromInside variable. An ambiguous filename is indicated by either the presence of the wildcard characters (? and *), by a blank or empty filename string, by a string ending in the "\" character, or by using the "[Search SubDirs](#)" (-s) command-line option. (For more information on the "Search SubDirs" command-line option, see "[Command-Line Options](#)" in the chapter on "[Starting the Editor](#)" in [the User's Guide](#).) This command also supports filename completion. This feature is invoked by typing a partial filename at the "File(s) to edit:" prompt, and then pressing the <Tab> key. The editor will add *.* , *. , or * as appropriate to complete the filename specification. A list of all matching files will be presented from which the user can select the appropriate file. To build the filename or list of files to process, the editor first determines the complete path. If a drive is not specified, the current drive is used; if a path is not specified, the current path is used. For example: - If the current drive/path is C:\WORK\STUFF and the user issues [EditFile](#)("new.doc"), then the editor will expand this to c:\work\stuff\new.doc. Once a particular file to be edited has been selected, the editor attempts to match the complete filename with a file already in the Ring, as follows: - If a match is found, that file becomes the current file and the [_ON_CHANGING_FILES_](#) event is called, if it exists. - If no match is found, the editor creates a new entry in the Ring. If the file exists on disk, it is read into memory. That file becomes the current file. If the [_ON_FIRST_EDIT_](#) event exists, it is called. If the [_ON_CHANGING_FILES_](#) event exists, it is also called. (Note that if [_DONT_LOAD_](#) is specified, the file is not read into memory and neither of the indicated events are called.) When multiple filenames are selected, the editor repeats the above process. However, it only loads into memory the file derived from the first fn entry, and makes that file the current file. Each of the other selected files is added to the Ring, but is not loaded into memory until the user switches to it and makes it the current file. If a filename without an extension is passed to [EditFile](#)(), and that file does not exist, the editor will try to find an existing file by appending any default extensions that the user has specified via IConfig. This command includes a history list of previous entries passed to it. See [AddHistoryStr](#)() for additional information. The description of the [Hook](#)() command contains information concerning the [_ON_CHANGING_FILES_](#) and [_ON_FIRST_EDIT_](#) events. [EditFile](#)() and [EditThisFile](#)() work similarly when filenames are quoted. **Examples:** // Edit a file named "my.file" [EditFile](#)("my.file") // Edit a file, whose name is stored in the string s [EditFile](#)(s) // Edit 3 files, named "file1", "file2", and "file3" [EditFile](#)("file1 file2 file3") See Also: [EditThisFile](#)(), [CreateBuffer](#)(), [CreateTempBuffer](#)(), [PickFile](#)(), [AddHistoryStr](#)(), [Hook](#)(), [UnHook](#)(), [BufferType](#)(), [AddFileToRing](#)(), [ReplaceFile](#)(), [NewFile](#)() **Variables:** [LoadWildFromDOS](#), [LoadWildFromInside](#), [UseCommonDialogs](#)

Editing Modes

A number of flexible editing modes are provided to control the way certain

commands or keys operate, or to affect the manner in which text is displayed on the screen. These modes not only allow you to further tailor the editor to your particular editing style, but also help you to increase your productivity at the keyboard.

The

[editing modes](#) discussed in this chapter are: - [Insert mode](#) - [WordWrap mode](#) - [AutoIndent mode](#) - [Template mode](#) - [SyntaxHilite mode](#) - [SmartIndent mode](#) - [C mode](#) To use Insert, WordWrap, AutoIndent, and Template modes, you must set the mode to an active state (by setting it ON, or setting it to another applicable state). If you set the mode to a non-active state (by setting it OFF), it will not affect editing. Options are provided for each of these modes [to](#) interactively toggle them between active and non-active states. Multiple modes can be in effect at the same time. Be sure to note the effects that these modes have on certain editing commands. SyntaxHilite mode is a special editing display mode that causes certain portions of the text to be distinctly colorized, according to the filename extension. [SmartIndent mode](#) and [C mode](#) are [special programming-language modes](#), and are automatically activated for files with certain filename extensions, if [AutoIndent mode](#) is set appropriately. The following chapters present additional [editing modes](#). The chapter on "[Loading, Processing, and Saving Files](#)" discusses Hex and Binary modes, which provide alternate modes for editing files. The chapter on "[Special Commands and Features](#)" explains how to use [LineDrawing](#) mode to draw lines and create boxes. The chapter on "[Creating and Using Macros](#)" describes RecordKeyMacro mode, which allows you to create [Keyboard Macros](#).

Editor Commands

To effectively use this chapter of

[the Macro Reference Guide](#), you need a working knowledge of the SemWare Applications Language (SAL). Detailed information about SAL is provided in the first five chapters of the Macro Reference Guide. Descriptions of the built-in editor variables available in SAL are provided in the chapter on "[Editor Variables](#)." This chapter provides information about the commands that comprise the native command set for SAL. These native commands can be used [to](#) write [macros](#) [to](#) customize existing commands or create additional editing commands. The commands presented in this chapter are distinct from the "editing commands" described in [the User's Guide](#). The editing commands are implemented using native commands; however, not every editing command is an actual native command. The editing commands available in the editor are implemented using the native commands, in the following ways: Many editing commands are, indeed, implemented as actual native commands. For example, AddLine is implemented as the unmodified native command [AddLine\(\)](#), and EditFile is implemented as the unmodified native command [EditFile\(\)](#). A few editing commands are implemented by simply passing a specific parameter to a native command. An example of this is the GotoWindowNumber commands. These are implemented as a series of commands using the native command [GotoWindow](#)(n), where "n" is each of the numbers 1 through 9. The result is nine GotoWindowNumber commands to allow you to go to any one of nine windows, with a single keystroke. Several other editing commands are implemented as [macros](#) that merely enhance the behavior of a native command. The BackSpace editing command is implemented as the macro mBackSpace(), which enhances the native [BackSpace\(\)](#) command to accommodate various [editing modes](#). The CReturn editing command is implemented as the macro mCReturn(), which enhances the native [CReturn\(\)](#) command to recognize "smart" indenting for several programming languages. A number of editing commands are implemented as simple or complex [macros](#) that provide additional features not included in the native commands themselves. The CompressView command is provided through the mCompressView() macro, which takes advantage of a number of native commands to provide the ability to list all lines that match a user-defined string. The CenterLine command is implemented via the mCenterLine() macro, which allows you to center a line or block of text. To distinguish editing commands from the native or macro commands by which editing commands are implemented, slightly different naming conventions are used in the editor documentation. The editing command names (used

predominantly in [the User's Guide](#)) are notated merely as a descriptive name, such as "Find" [or](#) "CompressView". The native and macro command names (used predominantly in [the Macro Reference Guide](#)) are notated as a descriptive name followed by "()", such as "[Find\(\)](#)" [or](#) "mCompressView()". Further, to distinguish a macro command implementation from a native command, the macro command name generally includes a prefix of "m" (for example, "mCompressView()"). And for those macro commands that are just enhanced versions of a native command, "m" is prefixed to the actual native command name that has been enhanced (such as, "mBackSpace()" [or](#) "mCReturn()"). In this chapter, descriptions are given for each of the native commands in the editor. A command description indicates the command name, a summary definition, the command syntax, the return type, and the return value(s). Additional information provided for many native commands includes usage notes, [examples](#), and related commands. NOTE: This chapter is NOT intended to provide descriptions of the editing commands as implemented in the editor, or of any enhanced or additional macro commands. An asterisk (*) following a command name in this chapter indicates that an editing command by the same name, has been implemented via an enhanced macro in the standard user-interface file, TSE.UI (located in the UI\ editor subdirectory). Those editing commands do not behave exactly as the native commands documented in this chapter. For a cross-reference list of the editing commands, and the native or macro commands by which the editing commands are implemented, see "[Appendix B: Command Implementation.](#)" [Syntax](#)

----- The format for the syntax of each command in this chapter is: RETURN_TYPE CommandName(PARM_TYPE identifier [, PARM_TYPE identifier]) where: - RETURN_TYPE is the type of Return Value (INTEGER or STRING) associated with the command. Some commands do not return a value; therefore, the syntax for those commands does not indicate a RETURN_TYPE. - CommandName is the name of the command. - PARM_TYPE is the parameter type of the identifier which follows. Valid parameter types are: - [INTEGER](#) (including [VAR INTEGER](#)) INTEGER is a whole number (... -2, -1, 0, 1, 2, 3, ...), in the range -2,147,483,648 (the MININT value in the editor) through +2,147,483,647 (the MAXINT value in the editor). A parameter denoted specifically as INTEGER can be coded as an integer constant (that is, an actual numeric value, such as 123), as an integer expression (such as, i * 7 + b), or as an integer variable name (such as line_number). [Integer constants](#) may be specified as binary, decimal, or hexadecimal values. (For more information, see "[Numbers](#)" in the chapter on "[Macro Language Syntax](#)".) A parameter denoted specifically as VAR INTEGER must be coded as an [integer](#) variable name. Such parameters may possibly be altered by the command. - [STRING](#) (including [CONSTANT STRING](#) and [VAR STRING](#)) STRING is a sequence of up to 255 characters, enclosed within single (") [or](#) double (") quotes. It can [include](#) any alphabetic, numeric, or other characters. A parameter denoted specifically as STRING can be coded as a constant string (that is, an actual string value, such as "ABC123"), as a string expression (such as, s + "abc" + s2, where s and s2 are string variables, and "abc" is a constant string), or as a string variable name (such as saved_wordset). A parameter denoted specifically as [CONSTANT STRING](#) must be coded as an actual literal string value. A parameter denoted specifically as VAR STRING must be coded as a [string](#) variable name. Such parameters may possibly be altered by the command. - PROCEDURE PROCEDURE is the name of another macro that is to be called by the command. - [MENU](#) MENU is the name of a pull-down or pop-up menu, or a menubar, that is to be displayed by the command. - [KEYDEF](#) KEYDEF is the name of a key definition (or set of key definitions) that may be enabled or disabled by the command. - identifier is the descriptive name of the parameter. - Parameters within square brackets [] are optional. For example: [INTEGER InsertText](#)([STRING](#) s [, [INTEGER](#) flag]) This command returns an integer, requires one parameter (a string), and accepts an optional parameter (an integer). [INTEGER Ask](#)([STRING](#) prompt, [VAR STRING](#) reply [, [INTEGER](#) history_number]) This command returns an integer and requires two parameters, the first being a string and the second being a string variable. It also has an optional third parameter, an integer. [Return Values](#) ----- Generally, [INTEGER](#) commands [return](#) non-zero for success, and zero ([FALSE](#)) for failure. To test a command's return value for success or failure, use the following style code: [if](#) command() <> 0 // if command is successful (non-zero) [[statements](#)] [endif](#) To loop while a command is successful, use the following style code: [while](#) command() <> 0 // while command result is non-zero [[statements](#)] [endwhile](#) Note that [not](#) all commands [return](#) non-zero for success and zero for failure. [String Length](#) ----- The editor's maximum [string length](#) is 255 characters. This applies to [strings](#) that are input to a command, as well as strings that are returned by a command. Note that the maximum string length and the editor's maximum line length are [not](#) related. [Optional Parameters](#) ----- All parameters are required unless they are indicated as optional by enclosing them in brackets []. Some commands with an optional parameter prompt for input

when that parameter is omitted. Other commands with [optional parameters](#) use a default value when the parameter is omitted. These are identified in the command description. [Examples](#) ----- In the examples provided in this chapter, unless declared otherwise, "i", "j", and "len" are integer variables; "fn" is a string variable used for filenames; and "s" is a string variable.

Editor Files and Directories

The files on The SemWare Editor Professional distribution diskette(s) are in compressed format. To install the editor, you must run the menu-driven installation routine. This routine decompresses and installs all or selected files onto your system.

By default, the installation routine creates the editor "

[load directory](#)" with the name \TSEPro, off the root directory of the drive you are logged onto during installation. The following files are placed in the editor load directory: g32.exe The SemWare Editor Professional (TSE Pro) program. (if installing the GUI version) or e32.exe The SemWare Editor Professional/32 (TSE Pro/32) program. (if installing the Console version) Once configured, the above (as appropriate) is the only file needed to run the basic editor. sc32.exe SAL Compiler program, used to compile [macros](#) that you write using the SemWare Applications Language (SAL). read.me Supplemental information about the editor, including changes or additions not covered in the printed documentation. *.dat Various files in which the editor stores configuration information. *.exe, *.com Various utility programs called by the editor. *.dll Various utility subroutines called by the editor. Additional files provide further functionality in the editor. If installed, the following subdirectories are created, and the indicated files are copied into them: \TSEPro\mac\ Compiled Macro (.mac) files supplied with the editor (including iconfig.mac, the Interactive Configuration macro file) are placed in this subdirectory. Also included are source (.s) files for many of the macro files; DLL (.dll) files required to compile certain macro files; and data (.dat) files required by certain macro files. \TSEPro\ui\ Source files for the user-interface macro files supplied with the editor are placed in this subdirectory. Several user-interface emulations are provided, including the standard editor emulation (tse.ui), a TSE Jr. emulation (tsejr.ui), a Brief(R) emulation (brief.ui), a WordStar(R) emulation (ws.ui), and a Windows compliant User Interface (win.ui). \TSEPro\doc\ Supplemental documentation files are placed in this subdirectory. Review these files for information [about](#) the editor. \TSEPro\spell\ SpellCheck word-list (.lex) files are placed in this subdirectory. \TSEPro\help\ HelpSystem (.hlp) files are placed in this subdirectory. \TSEPro\synhi\ [Syntax](#) highlighting files (.syn) are placed in this subdirectory. If you wish to install the editor in a different directory than that indicated above, you can specify an alternate drive and/or directory during installation. You can also omit certain of the additional files from the installation process. The installation routine lets you select which sets of user-interface emulation files are copied onto your machine. It also lets you select the particular user interface with which you want to configure the editor when it is initially installed. You can select the standard SemWare Editor Professional user interface, or you can choose one of the alternate user-interface emulations. Once the editor is installed, you can use it to view the READ.ME file, located in the editor load directory. This will help acquaint you with the editor and provide you with useful information. To run the editor from any drive and directory, modify the PATH statement in your AUTOEXEC.BAT file to include the editor drive and directory. After you update AUTOEXEC.BAT, reboot your machine so the modified PATH will be effective. (See your system manual for information on modifying the PATH statement.) NOTE: The installation routine creates ONLY the editor load directory, and subdirectories under it. No other directories, and no hidden files, are created or modified by the installation routine.

Editor Startup Processing

When the editor is started, the following events occur in the order shown:

1. The editor checks to see if a WhenLoaded macro is defined in the

installed user-interface file. If present, the WhenLoaded macro is executed, and events 2 - 5 (below) are performed by the WhenLoaded macro.

2. The editor searches for a Compiled Macro file named TSESTART.MAC. It searches directories according to the "

[macro file search order](#)" described under "[Creating and Using Compiled Macros](#)" in the chapter on "[Creating and Using Macros](#)." If a TSEStart macro file is found, it is then processed. [The TSEStart macro](#) file is simply a user-defined external Compiled Macro file that the editor attempts to run each time the editor is started. When the editor processes [the TSEStart macro](#) file, it loads the macro file, and executes the WhenLoaded and Main macros in the file (if present). This feature can be useful for specifying such startup actions as loading custom [macros](#), and initially setting [variables](#) and [editing modes](#). [The TSEStart macro](#) feature also provides an alternate method for creating versions of the editor that behave differently depending on the directory from which the editor is started. You can include a unique TSEStart macro file in the desired directories. The editor will process [the TSEStart macro](#) file located in the directory where you start the editor, and perform the functions specified.

3. The editor searches for a macro autoload file by the name of TSELOAD.DAT. It searches for this file only in the editor load directory. If this file is found, the editor loads each macro file specified in the autoload file, and executes the WhenLoaded macro (if any) located in each macro file. The purpose of the autoload file is to provide you with a simple means for automatically loading your own set of external Compiled Macro files, each time the editor is started. To add to or delete from the list of macro files in the autoload file, use the AutoLoadList command.
4. The editor sets the current video mode, using the startup video mode.
5. The editor processes the filenames and options included on the command line, and loads the initial file (if one is included). If the command line does not contain any options or filenames, the editor checks the setting of the StartupFlags variable to determine the appropriate startup action to take (as described under "[Starting the Editor with No Command-Line Parameters](#)" earlier in this chapter.)
6. Once the WhenLoaded macro (if present) is complete, the editor then checks to see if a Main macro is defined in the installed user-interface file. If present, that [Main macro](#) is executed.
7. The _ON_EDITOR_STARTUP_ event is called.
8. The editor loads and/or executes any [external macro files](#) specified by the "[Load Macro](#)" (-l) or "[Execute Macro](#)" (-e) command-line options. (For more information on creating, loading, and executing external Compiled Macro files, including use of the macro autoload file, see "[Introduction to External Compiled Macros](#)" in the chapter on "[Creating and Using Macros](#)." Also see "[LoadMacro\(\)](#)" and "[ExecMacro\(\)](#)" in the chapter on "[Editor Commands](#)" in [the Macro Reference Guide](#). For more information on processing WhenLoaded and [Main macros](#), see "[Event Macros](#)" in the chapter on "[Using Macro Language Features](#)" in the Macro Reference Guide. For more information on creating a TSEStart macro file, see "[Creating a TSEStart Macro File](#)" in the chapter on "[Fully Customizing the Editor](#)." For more information on setting the video mode that the editor uses each time it starts, see "Startup Video Mode" under "[Display/Color Options](#)" in the chapter on "[Configuring the Editor Interactively](#).")

Editor Variables

To effectively use this chapter of

[the Macro Reference Guide](#), you need a working knowledge of the SemWare Applications Language (SAL). Detailed information about SAL is provided in the first five chapters of the Macro Reference Guide. Descriptions of the native editor commands available in SAL are provided in the chapter on "[Editor Commands](#)." This chapter provides information on setting and querying the editor variables (options) available within SAL that allow you to customize the editing environment. The description for each variable in this chapter indicates the variable name, a summary definition, variable type, valid values or range of values, and [default](#) value (if any). Additional information provided for many [variables](#) includes limitations, usage notes, related variables, and a list of commands affected by the value assigned to the variable. (The commands indicated in this chapter are native commands, as described in the "[Editor](#)

[Commands](#)" chapter. Command names are distinguished from variable names by "(" following each command name, such as [Find\(\)](#).) [Values](#) ----- For each variable, the variable type is identified as either INTEGER, STRING, or BOOLEAN. The valid [values](#) or range of [values](#) for each variable are also indicated. Note that BOOLEAN is a subset of INTEGER, with only two allowable values, ON and [OFF](#) (or their equivalent [values](#), [TRUE](#) and [FALSE](#), or 1 and 0, respectively). [Default](#) ----- The initial value of each variable at editor startup is indicated, if appropriate. (If you have installed the editor using an emulation other than the standard editor user interface, the initial values for the variables may differ.) [Limits](#) ----- Any limitations relating to the use and operation of a variable are indicated. Unless otherwise noted in the limits, all variables can be manipulated by the [Query\(\)](#) and [Set\(\)](#) commands, and all BOOLEAN type [variables](#) can also be manipulated by the [Toggle\(\)](#) command. Unless otherwise indicated, all variables can be used within the config/endconfig construct of an editor configuration file. [Affects](#) ----- Commands that may be affected by the setting of a variable are indicated for that variable.

EditThisFile

Loads a file into the editor and makes it the current file.

Syntax: [INTEGER](#) [EditThisFile](#)([[STRING](#) fn [, [INTEGER](#) flag]]) - fn is an optional string expression specifying the file to be loaded. - flag is an optional integer that affects the operation of [EditThisFile\(\)](#). See the [EditFile\(\)](#) command for supported [values](#) of flag. Returns: The non-zero id of the file just edited if successful; otherwise, zero ([FALSE](#)). Notes: If no filename is specified, the editor will prompt for a filename. This command is very similar to the [EditFile\(\)](#) command, except that [EditThisFile\(\)](#) expects a single filename, whereas [EditFile\(\)](#) can accept multiple filenames. [EditFile\(\)](#) and [EditThisFile\(\)](#) work similarly when filenames are quoted. For instance: If at the [EditThisFile\(\)](#) prompt, one types: some file name Either "some file name" is loaded from disk (if it exists), or a new editing buffer is created, and given the specified name. In contrast, if the above was typed at an [EditFile\(\)](#) prompt, three separate files would be loaded/created, "some", "file", and "name". See the [EditFile\(\)](#) command for additional notes. **Examples:** // Edit a file named "my.file" [EditThisFile](#)("my.file") // Edit a file, whose name is stored in the string s [EditThisFile](#)(s) See Also: [EditFile\(\)](#), [CreateBuffer\(\)](#), [CreateTempBuffer\(\)](#), [PickFile\(\)](#), [AddHistoryStr\(\)](#), [Hook\(\)](#), [UnHook\(\)](#), [BufferType\(\)](#), [AddFileToRing\(\)](#), [ReplaceFile\(\)](#), [NewFile\(\)](#) **Variables:** [LoadWildFromDOS](#), [LoadWildFromInside](#), [UseCommonDialogs](#)

Effects of Expandtabs

This section explains how tabs are visibly handled on the screen, and describes some of the effects that may be noticed, when a file containing physical tab characters is edited and

[ExpandTabs](#) is [ON](#). The [ExpandTabs](#) option can cause differences in the physical versus the visible effects of tab characters in a file. When the ExpandTabs option is ON, tab characters in a file affect the columns at which text is displayed. The tab characters themselves are [not](#) displayed. Note, however, that [ExpandTabs](#) has no effect on any line longer than 255 characters. (For more information on how to set the [ExpandTabs](#) option, see "Expand Tabs (Display Only)" under "[Tab/Margin Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") When text is displayed on the screen, the presence of a tab character causes the text following the tab character to be shifted out to the next tab stop. The tab stops are determined based on the current setting of TabWidth. This also applies to the text display when an existing file containing tab characters is loaded. If the TabWidth value is changed during an editing session, the display of the text visibly changes to accommodate the new [TabWidth](#). When text is shifted to a tab stop, it will appear that space characters have been inserted into the text to fill the void area. However, this is not the [case](#). That area is

empty and does [not](#) contain any actual characters. The behavior of CursorRight and CursorLeft may be affected as the cursor moves past the tab characters and over the void area. The cursor jumps from a non-displayed tab character to the next or previous tab stop. If you execute CursorDown, and the current cursor position on the new line is within a void area, the cursor shifts left to an actual text character (which may be a space character), or to the beginning of the line if no text characters precede the cursor. Inserting text within a line containing tab characters may also cause unusual visible effects. Non-displayed tab characters may be overwritten or moved as new characters are typed or shifted into the void area, causing text to the right of the tab to possibly shift. Deleting text may also cause text to shift, if a non-displayed tab character is deleted. Caution should be exercised when executing TabLeft with [Insert ON](#).

Because the tab characters are not visible, this action could result in inadvertent deletion of tab characters.

+----- Tip -----+ | To print a file containing tab characters, use one of the editor's | | Print commands (under the Print Menu). The editor then interprets | | tab characters according to your current [TabWidth](#). This should | | help achieve the desired placement of the text on the printed page. |

+-----+

EmptyBuffer

Deletes all the text in the current or specified buffer.

Syntax: [INTEGER EmptyBuffer](#)([[INTEGER](#) buffer_id]) - buffer_id is the optional id number of the file in which to delete all lines. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: The cursor is positioned at line 1, column 1, after the completion of this command. Since this command deletes all the text in the specified buffer without confirmation, it should be limited to use within a macro procedure, and not assigned to a key by itself.

_ON_CHANGING_FILES_ hooks are [NOT](#) invoked by this command. See Also: [AbandonFile](#)()

EmptyWinClip

Empties the contents of the Windows Clipboard.

Syntax: [INTEGER EmptyWinClip](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. See Also: [CopyToWinClip](#)(), [CutToWinClip](#)(), [CopyAppendToWinClip](#)(), [CutAppendToWinClip](#)(), [PasteFromWinClip](#)(), [isWinClipAvailable](#)()

Enable

Activates the specified set of key definitions.

Syntax: [INTEGER Enable](#)([KEYDEF](#) keydef_name [,[INTEGER](#) exclusive]) - keydef_name is the name of the keydef to enable. - exclusive is an optional integer that determines whether the keys in keydef_name will be the only keys active, or merged in with the keys already active. If not specified, _DEFAULT_ is used. Supported [values](#) for exclusive are: - _DEFAULT_ specifies that only the keys listed in the keydef will be changed to the new definition. All other keys will continue to be active and work as before. - _EXCLUSIVE_ specifies that only the keys listed in this [keydef](#) will be active. All other keys will have no effect. - _TYPEABLES_ enables all typeable keys (alpha, numeric, and punctuation keys) that do [not](#) have any [macros](#) already assigned to them. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Any keydefs that are enabled will remain in effect until they are specifically disabled with the [Disable](#)() command, or when the editor is exited. This command returns zero if there is not room in the active

[keydef](#) table for another entry. For more information on the keydef identifier, see "[Keydef](#)" and "[Process\(\) and Keydefs](#)" in the chapter on "[Using Macro Language Features](#)." [Examples](#): /

***** Make my_keydef active, make it exclusive (only keys in this [keydef](#) are active), and allow all typeable keys.

*****/ [Enable](#)(my_keydef, _EXCLUSIVE_|_TYPEABLES_) /***** Make my_keydef active. Other defined keys are still active, but keys in my_keydef take precedence.

*****/ [Enable](#)(my_keydef, _DEFAULT_) See the [Process\(\)](#) command for additional examples. See Also: [Disable\(\)](#), [Process\(\)](#)

EndFile

Moves the cursor to the end of the last line of the current file.

Syntax: [INTEGER EndFile\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Zero is returned if the cursor position does not change. For example, if the cursor is already at the end of the last line of the file, zero is returned. This command is equivalent to the following: [GotoLine](#)([NumLines](#)()) [EndLine](#)() [Examples](#): In order to maintain the current cursor column when a column block is open in the current file, use the following macro instead of the [EndFile\(\)](#) command: [proc](#) mEndFile() [integer](#) col = [CurrCol](#)() [EndFile](#)() [if](#) [Query](#)([Marking](#)) [and](#) [isBlockInCurrFile](#)() == _COLUMN_ [GotoColumn](#)(col) [endif](#) [end](#) <Ctrl PgDn> mEndFile() See Also: [BegFile](#)()

EndLine

Moves the cursor to the end of the current line (that is, one position past the last character on the line).

Syntax: [INTEGER EndLine\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If the cursor is already at the end of the current line, zero is returned. If the [RemoveTrailingWhite](#) variable is set ON, this command will remove any trailing white space found on the current line before going to the end-of-line position. See Also: [BegLine](#)(), [BegLineTog](#)(), [EndLineTog](#)() **Variables:** [RemoveTrailingWhite](#)

EndLineTog

Toggles the cursor position between end end of line and the right edge of the editing window.

Syntax: [EndLineTog](#)() Returns: Nothing. If the cursor is already positioned at the end-of-line position on the current line, the cursor moves to the right edge of the window on the current line, otherwise the cursor moves to the end-of-line position on the current line. See Also: [EndLine](#)(), [BegLine](#)(), [BegLineTog](#)()

EndProcess

Terminates the current process and returns control to the previous process.

Syntax: [EndProcess](#)([[INTEGER](#) status]) - status is an optional result that can be passed to the calling process, which could be [Process](#)() itself, or [List](#)() or [IList](#)() if the appropriate event is hooked. status defaults to 0 if [not](#) passed. Returns: Nothing. Notes: [EndProcess](#)() will terminate the current process level and return control back to the previous process at the instruction following the [Process](#)() command that invoked the process level. If the current process level is the only one, control will be returned to the editor. Use this command to terminate a process that was started with the [Process](#)() command. Typical usage is to assign [EndProcess](#)() to a key via a keydef. **Examples:** See the [Process](#)() command for an example. See Also: [Process](#)(), [Enable](#)(), [Disable](#)()

EndWindow

Moves the cursor to the bottom line of the current window, without changing the cursor column.

Syntax: [INTEGER](#) [EndWindow](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If the cursor is already at the bottom line of the current window, zero is returned. If there are not enough lines in the current file to extend to the bottom line of the window, the cursor is moved to the last line in the file. See Also: [BegWindow](#)()

EndWord

Moves the cursor to the end of the current word.

Syntax: [INTEGER](#) [EndWord](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Zero is returned if the cursor character and the character to the left of the cursor are not in the currently defined WordSet. Otherwise, the cursor is moved to the end of the word, if the [cursor](#) was [not](#) already there. See Also: [BegWord](#)(), [WordLeft](#)(), [WordRight](#)() **Variables:** [WordSet](#)

EntabCurrLine

Converts consecutive space characters on the current line to the appropriate number of tabs.

Syntax: [INTEGER](#) [EntabCurrLine](#)([[INTEGER](#) respect_column_block]) - respect_column_block, if non-zero, optionally [limits](#) entabbing to the text on the line that is within a column block. If no text on the line is within a column block, this parameter is ignored. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: The editor uses the TabWidth variable to determine the number of consecutive space characters to replace with a tab character. If [TabWidth](#) is 0, [EntabCurrLine](#)() does nothing and returns [FALSE](#). A single space character is not replaced with a tab unless [TabWidth](#) is set to 1. **Examples:** [EntabCurrLine](#)() // entab the current line See Also: [ExpandTabsToSpaces](#)() **Variables:** [TabWidth](#)

EntabOnSave

If on, spaces are converted to tabs (as appropriate) when a file is saved.

Values: BOOLEAN **Default:** [OFF](#) Notes: Spaces are converted to tabs based on the TabWidth editor variable. See

Also: [DetabOnLoad](#), [TabWidth](#)

EOFFType

Determines what character(s) the editor will place at the end of the last line of the file when the file is saved to disk.

Values: [INTEGER](#) - 0 - do [not](#) append any characters at last line in file - 1 - append a Control-Z (^Z) at last line in file - 2 - force [EOLType](#) at last line in file - 3 - force [EOLType](#) at last line in file and append a Control-Z **Default:** 2 **Notes:** This setting has no effect on how files are loaded. See Also: [EOLType](#)

EOLType

Determines whether the editor adds CR/LF, CR, or LF at the end of each line when the file is saved to disk.

Values: [INTEGER](#) - 0 - AS LOADED. If the file, when read into the editor, contains only lines terminated by a linefeed character (LF), then the editor will terminate each line with a LF when the file is saved. If the file contains only lines terminated by a carriage return character (CR), then the editor will terminate each line with a CR when the file is saved. For all other files, the editor will terminate each line with a CR/LF when the file is saved. - 1 - ^M (CR) - 2 - ^J (LF) - 3 - ^M^J (CR/LF) **Default:** 0 **Notes:** For most applications, the default setting will be sufficient. **Values** 1 [and](#) 2 are provided to support file formats from other platforms such as UNIX and Macintosh. This variable has no effect on how the editor interprets lines [when](#) loading files. **Affects:** [SaveAllAndExit\(\)](#), [SaveAllFiles\(\)](#), [SaveAndQuitFile\(\)](#), [SaveFile\(\)](#), [SaveAs\(\)](#) See Also: [EOFFType](#)

EquateEnhancedKbd

Causes the editor to equate enhanced keys to their standard counterpart.

Values: BOOLEAN **Default:** [ON](#) **Notes:** If [EquateEnhancedKbd](#) is [ON](#), then enhanced keyboard keys, such as <GreyCursorRight>, are translated to their corresponding non-enhanced keys, such as <CursorRight>. For example: If you assign a command to <GreyCursorRight> and have [EquateEnhancedKbd](#) turned [ON](#), then whenever you press <GreyCursorRight>, you will get the command assigned to <CursorRight>, instead of the one assigned to <GreyCursorRight>. See Also: [Commands: GetKey\(\)](#)

EquiStr

Compares two strings in a case-insensitive manner.

Syntax: [INTEGER](#) [EquiStr](#)([STRING](#) s1 , [STRING](#) s2) - s1 is the first string to be compared. - s2 is the second string to be compared **Returns:** Non-zero ([TRUE](#)) if s1 is equal to s2; [otherwise](#), zero ([FALSE](#)). **Notes:** This command is provided as an adjunct to the normal string comparison operator, "=". The "=" operator compares [strings](#) in a case sensitive manner. [EquiStr](#)() makes it easy to compare two [strings](#) for equality, regardless of their [case](#). When to use [CmpiStr](#)() vs. [EquiStr](#)() If you want to know if two [strings](#) are equal, ignoring differences in [case](#), use [EquiStr](#)(). If you

want to know the lexical ordering of two strings (if one is less than or greater than the other), use [CmpiStr\(\)](#).

Examples: // compare two [strings](#) to see if they are equal, regardless of // [case if EquiStr](#)(s1, s2) [Message](#)("s1 and s2 are the same") [else Message](#)("si is different than s2") [endif](#) See Also: [CmpiStr\(\)](#)

EraseDiskFile

Erases the specified file from disk.

Syntax: [INTEGER EraseDiskFile](#)([STRING](#) file_to_erase) - file_to_erase is the name of the file to erase. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Zero is returned if the file cannot be erased. Possible reasons why the file cannot be erased are that the file does not exist, or the file is read-only. Wildcard characters (* and ?) are [not](#) supported by this command. See Also: [FileExists\(\)](#)

Escape

Terminates editor prompts; closes pop-up and pull-down

[menus](#); ends [List\(\)](#). **Syntax:** [INTEGER Escape](#)() Returns: Zero ([FALSE](#)). Notes: When the editor prompts for information, or a menu or selection box is displayed on the screen, this command is always assigned to the <Escape> key. This command is made available so that other keys can be used as additional keys for an escape function.

Event Macros

An event macro is one that is automatically executed (or "triggered") any time certain pre-defined editor events occur. There are two types of

[event macros](#): - [Macros](#) with certain pre-defined macro names that are triggered by certain pre-defined events. - Macros specifically defined by the user to be triggered by certain pre-defined events. [Event Macros with Predefined Names](#) ----- The [event macros](#) that are triggered in accordance with pre-defined names are those that are named [Main\(\)](#), [WhenLoaded\(\)](#), or [WhenPurged\(\)](#). These event [macros](#) behave as follows: - A [Main macro](#) can be used in external or user-interface [macros](#). In user-interface [macros](#), it is executed (triggered) on editor startup. In external macros, it is associated with the name of the external Compiled Macro file in which it resides. The [Main macro](#) is the one that is executed (triggered) when a macro filename is passed to [ExecMacro\(\)](#). - A [WhenLoaded macro](#) can be used in external or user-interface [macros](#). In user-interface [macros](#), it is executed (triggered) on editor startup. In external macros, it is executed (triggered) when the external Compiled Macro file containing it is loaded. - A [WhenPurged macro](#) can be used in external or user-interface [macros](#). In external macros, it is executed (triggered) when the Compiled Macro file containing it is purged. In user-interface [macros](#), it is executed (triggered) when the editor is terminated. To use these types of event macros, simply declare a macro to have the name of [Main\(\)](#), [WhenLoaded\(\)](#), or [WhenPurged\(\)](#), as appropriate. Note that only one occurrence of each can occur in a macro file. (For more information on these types of event macros, see the chapter on "[Using Macros](#).") [Event Macros Defined by the User](#) ----- The [event macros](#) that are triggered in accordance with specific definition by the user, are those for which the user explicitly establishes an association between the desired macro and the particular triggering event, by "hooking" the macro to the event. Once you hook a macro to an event, that triggering event is called a "hooked" event. When the hooked event occurs, the editor automatically executes the associated macro. Following are the editor events that can be hooked, along with a short explanation of when each event occurs. - [_ON_EDITOR_STARTUP_](#) is called just before the editor starts the main editing loop. It is useful when you want to gain control after the command line has been processed, the file to edit has been loaded, but the editor has not yet

drawn the window or gotten keystrokes from the user. - `_ON_CHANGING_FILES_` is called each time a different file is switched to, with the following exceptions: - If the file was switched to via the [GotoBufferId\(\)](#) command, this event is **NOT** called. - If the file was switched to using the `_DONT_LOAD_` option of the [NextFile\(\)](#) or [PrevFile\(\)](#) commands, this event is **NOT** called. See the [OnChangingFiles\(\)](#) macro in the TSE.UI user-interface macro file for [examples](#) using this event. - `_ON_FIRST_EDIT_` is called the first time a file is loaded into the editor. Therefore, it is only called once for each file edited. This event is NOT called by the [CreateBuffer\(\)](#) or [CreateTempBuffer\(\)](#) commands. - `_ON_FILE_LOAD_` is called immediately before a file is loaded into the editor. It is called before `_ON_FIRST_EDIT_`. - `_ON_FILE_SAVE_` is called just before the editor saves a file. It is NOT called by the [SaveAs\(\)](#) and [SaveBlock\(\)](#) commands. - `_AFTER_FILE_SAVE_` is called just after the editor saves a file. It is NOT called by the [SaveAs\(\)](#) and [SaveBlock\(\)](#) commands. - `_ON_FILE_QUIT_` is called just before the editor quits a file via the single-file quitting commands, namely [QuitFile\(\)](#), [SaveAndQuitFile\(\)](#), and [AbandonFile\(\)](#). If only a single file is loaded, this event is not called unless the editor variable `QuitToPrompt` is set ON. - `_ON_EXIT_CALLED_` is called at the beginning of the [Exit\(\)](#) and [SaveAllAndExit\(\)](#) commands. - `_ON_ABANDON_EDITOR_` is called just before the editor terminates (that is, just before the editor exits to the operating system or to the calling program). - `_IDLE_` is called from the editor's keyboard processing routine, when no keys are available to be processed, AND the editor is in a "text-editing" state (that is, the focus is a text-editing window, as opposed to a prompt, menu, list, or other similar input focus). - `_NONEDIT_IDLE_` is called from the editor's keyboard processing loop, when no keys are available to be processed, AND the editor is in a "non-text-editing" state (that is, the focus is a prompt, menu, list, or other similar non-text-editing input focus). - `_BEFORE_UPDATE_DISPLAY_` is called before each window is updated. - `_AFTER_UPDATE_DISPLAY_` is called after each window is updated. - `_AFTER_UPDATE_STATUSLINE_` is called after [the StatusLine](#) is updated. - `_BEFORE_COMMAND_` is called before each text-editing command is processed. (This event is called only by those commands that are executed by pressing a key, rather than being selected from the pull-down [Menu](#).) - `_AFTER_COMMAND_` is called after each text-editing command has completed processing. (This event is called only by those commands that are executed by pressing a key, rather than being selected from the pull-down [Menu](#).) - `_BEFORE_NONEDIT_COMMAND_` is called before each non-text-editing command is processed. (This event is called only by those commands that are executed when the editor is in a "non-text-editing" state; that is, the focus is a prompt, menu, list, or other similar non-text-editing input focus.) - `_AFTER_NONEDIT_COMMAND_` is called after each non-text-editing command has completed processing. (This event is called only by those commands that are executed when the editor is in a "non-text-editing" state; that is, the focus is a prompt, menu, list, or other similar non-text-editing input focus.) - `_LIST_STARTUP_` is called when the [List\(\)](#) and [IList\(\)](#) commands start processing. - `_LIST_CLEANUP_` is called when the [List\(\)](#) and [IList\(\)](#) commands terminate. - `_PICKFILE_STARTUP_` is called when the directory `PickList` routine is invoked. (The `PickList` routine is invoked by the [PickFile\(\)](#) command. Any commands that display a `PickList` of files, such as the [EditFile\(\)](#) command, also call the [PickFile\(\)](#) command.) - `_PICKFILE_CLEANUP_` is called when the [PickFile\(\)](#) command terminates. - `_PROMPT_STARTUP_` is called when the [Ask\(\)](#) and [Read\(\)](#) family of commands start processing. - `_PROMPT_CLEANUP_` is called when the [Ask\(\)](#) and [Read\(\)](#) family of commands terminate. - `_ON_SELFINSERT_` is called after a character is inserted into a text-editing window. - `_ON_DELCHAR_` is called after any of the following commands are executed in a text-editing window: [DelChar\(\)](#), [DelRightWord\(\)](#), [DelLeftWord\(\)](#), [BackSpace\(\)](#) - `_PRE_UPDATE_ALL_WINDOWS_` is called once at the beginning of the [UpdateDisplay\(\)](#) command processing. - `_POST_UPDATE_ALL_WINDOWS_` is called once at the end of the [UpdateDisplay\(\)](#) command processing. - `_ON_UNASSIGNED_KEY_` is called whenever an unassigned key is pressed, AND the editor is in a "text-editing" state (that is, the focus is a text-editing window, as opposed to a prompt, menu, list, or other similar input focus). - `_ON_NONEDIT_UNASSIGNED_KEY_` is called whenever an unassigned key is pressed, AND the editor is in a "non-text-editing" state (that is, the focus is a prompt, menu, list, or other similar non-text-editing input focus). - `_BEFORE_GETKEY_` is called just before the editor gets another key or mouse button, regardless of whether the editor is in a "text-editing" or "non-text-editing" state. (To determine the current state of the editor, use the [QueryEditState\(\)](#) command.) - `_AFTER_GETKEY_` is called just after the editor has gotten another key or mouse button, regardless of whether the editor is in a "text-editing" or "non-text-editing" state. (To determine the current state of the editor, use the [QueryEditState\(\)](#) command.) A macro is hooked to a triggering event by using the [Hook\(\)](#)

command. * [Hook\(\)](#) The basic format for this command is: [Hook](#)([INTEGER](#) event_name, PROCEDURE macro_name) This command is used to associate, or hook, the macro indicated by macro_name, with the triggering event indicated by event_name. event_name must be one of the events listed previously. After this command executes successfully, macro_name is then called each time the hooked event occurs. The same macro can be hooked to multiple events, and multiple [macros](#) can be hooked to the same event. However, the same macro cannot be hooked more than once to the same event. If multiple macros are hooked to the same event, then when the triggering event occurs, the hooked macros are scheduled for execution in chronological hooking order; that is, the first event that is hooked is the first one that is executed. For example, given: [proc](#) foo() ... [end proc](#) foobar() ... [end proc](#) main() [Hook](#) (_ON_FILE_SAVE_, foobar) [Hook](#)(_ON_FILE_SAVE_, foo) [end](#) then, when the _ON_FILE_SAVE_ event occurs, foobar() is executed, and then foo() is executed. A macro can subsequently be unhooked from an event by using the [UnHook\(\)](#) command. * [UnHook\(\)](#) The basic format for this command is: [UnHook](#)(PROCEDURE macro_name) This command breaks the association between the macro indicated by macro_name, and any triggering events; that is, it unhooks macro_name from any events. After this command executes successfully, macro_name is no longer called by any events. When you purge a Compiled Macro file, any macros in that file that are hooked to events, are automatically unhooked. You can also temporarily unhook macros from an event by using the [SetHookState\(\)](#) command. This command allows you to specify whether the hooks for a particular editor event are currently enabled or disabled. If the hooks are enabled, the editor event will trigger any event macros that are hooked to the event. If disabled, the event will not trigger event macros that are hooked to it. * [SetHookState\(\)](#) The basic format for this command is: [SetHookState](#)([INTEGER](#) event_state, [INTEGER](#) event_name) This command sets the current hook state of the editor event indicated by event_name. If event_state is zero (FALSE), the hook state is disabled; if non-zero (TRUE), the hook state of the specified event is re-enabled. This command can be used within macros to disable the hooks for an event, do some required processing, and then re-enable the previous hooks. A command that complements the [SetHookState\(\)](#) command is the [GetHookState\(\)](#) command. The [GetHookState\(\)](#) command allows you to check the current state of an editor event to see if the macros that are hooked to it are currently enabled. * [GetHookState\(\)](#) The basic format for this command is: [GetHookState](#)([INTEGER](#) event_name) This command allows you to determine if the hooks for the editor event, indicated by event_name, are currently enabled or disabled. If the hooks are enabled, the editor event will trigger any event macros that are hooked to the event. If disabled, the event will not trigger event macros hooked to it. This command, used in conjunction with the [SetHookState\(\)](#) command, is useful for saving, setting, and then restoring the hook state of an event. Typically, the event hooks are disabled, a task is performed, and then the event hooks are re-enabled. When you have multiple macros hooked to an event, and that event has just occurred, it is possible to break the chain of macros triggered by the event (for this occurrence of the event only). To do this, use the [BreakHookChain\(\)](#) command within an appropriate event macro in the chain. The remaining event macros, currently scheduled for execution by the event, are immediately canceled. * [BreakHookChain\(\)](#) The basic format for this command is: [BreakHookChain\(\)](#) This command terminates the remaining chain of event macros that are currently scheduled for execution by the occurrence of a multi-hooked event. If [BreakHookChain\(\)](#) is called by the currently-executing macro in a chain of hooked [macros](#), no additional [macros](#) hooked to that event are called. [BreakHookChain\(\)](#) breaks the chain of event macros only for the current occurrence of the event. The next time the hooked event occurs, the entire chain of [event macros](#) will be scheduled for execution. For example: /

```
***** Prevent any _ON_CHANGING_FILES_ hooks
from being run while we are busy. *****/
proc
StopOnChangingFilesHook() BreakHookChain\(\) end proc foo() Hook(_ON_CHANGING_FILES_,
StopOnChangingFilesHook) // Code goes here... UnHook(StopOnChangingFilesHook) end
It is possible to explicitly call all of the macros hooked to an event, using the ExecHook\(\) command. * ExecHook\(\) The basic format for this command is: ExecHook(INTEGER event_name) This command provides a means of calling all macros hooked to a specified event, indicated by event_name. Its primary purpose is to simulate the actual event for testing purposes; however, it is also useful in certain other situations. event_name must be one of the events listed above.
```

ExecHook

Executes all hooks associated with the specified event.

Syntax: [INTEGER ExecHook](#)([INTEGER](#) event) - event is the name of the event to execute. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: All hooked procedures for event are executed in the same way as when the specified event actually occurs. See the [Hook\(\)](#) command for a list of the supported events. **Examples:** // Execute any hooks associated with _ON_FIRST_EDIT_ [ExecHook](#)(_ON_FIRST_EDIT_) See Also: [Hook\(\)](#), [UnHook\(\)](#), [BreakHookChain\(\)](#), [GetHookState\(\)](#), [SetHookState\(\)](#), [TerminateEvent\(\)](#)

ExecLoadedMacro

Displays a PickList of the macros currently loaded, then executes the macro selected from the PickList.

Syntax: [INTEGER ExecLoadedMacro](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If there are no [macros](#) in memory (that is, no [macros](#) are loaded), or the PickList is abandoned without executing a macro, zero ([FALSE](#)) is returned. See Also: [ExecMacro\(\)](#), [LoadMacro\(\)](#), [isMacroLoaded\(\)](#)

ExecMacro

Executes the requested Compiled Macro.

Syntax: [INTEGER ExecMacro](#)([[STRING](#) macro_str [, [STRING](#) public_name]]) - macro_str consists of macro_name and an optional cmd_line_str, in the format "macro_name cmd_line_str", where: - macro_name is the name of a Compiled Macro file or an already-loaded [Public](#) macro to execute. macro_name may contain wildcards or other characters used in filenames ("\" , ":" , or "."). - cmd_line_str is an optional command-line string to pass to macro_name. cmd_line_str must be separated from macro_name by one or more spaces. Spaces may be included within the text for cmd_line_str. - public_name is an optional name of a specific public procedure within macro_name. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: When macro_name is specified, the editor attempts to execute the specified macro as follows: - If macro_name includes path information: - The editor checks to see if a macro with the specified name is already loaded, and was loaded from the indicated drive and/or directory. If one is located, that macro is executed. - If one is not found, the editor calls the [LoadMacro](#)() command to attempt to locate the indicated Compiled Macro file from the specified drive and/or directory. If located, the Compiled Macro file is loaded: the Main() macro, if present, and any Public macros are added to the Public macro table; and the WhenLoaded() macro, if present, is executed. The editor then executes the Main() macro (if one is present). - If the specified Compiled Macro file still cannot be located, the command fails. - If macro_name does [not include](#) path information: - The editor checks to see if a macro with the specified name is already loaded. If one is located, that macro is executed. - If one is not found, the editor calls the [LoadMacro](#)() command to attempt to locate the indicated Compiled Macro file on disk. If located, the Compiled Macro file is loaded: the [Main](#)() macro, if present, and any [Public macros](#) are added to the Public macro table; and the WhenLoaded() macro, if present, is executed. The editor then executes the Main() macro (if one is present). - If the specified macro still cannot be located, the command fails. If macro_name is not specified, the editor pops up a list of currently-loaded macros. It executes the selected macro and returns non-zero, or returns zero if the user presses <Escape>. cmd_line_str is valid only if macro_name is also specified. If cmd_line_str is specified, the contents of this command-line string are placed in the MacroCmdLine variable. The invoked macro(s) can then access the command-line string by querying the MacroCmdLine variable. If the invoked macro was called by another macro, information can also be passed back to the calling macro by setting the value of the MacroCmdLine variable. If [ExecMacro](#)() is executed and cmd_line_str is [not](#) specified, the MacroCmdLine variable is set to the empty

string. To run a specific public procedure in a macro, the `public_name` variable should be specified. To run public procedures in a .ui file, specify a `macro_str` of ".ui". Any Compiled [Macros](#) intended for repeated execution from memory (that is, they are [not](#) always reloaded before execution) should be fully reentrant. Simply put, this means that the user should be aware that any global [variables](#) will start with whatever [values](#) they ended with on the last invocation of the macro. If the user needs a global variable to have a certain value each time the macro is run, assign it that value via an assignment statement in the macro rather than in a global variable definition. For a Compiled Macro that should be loaded from disk each time it is to be executed, you can do either of the following: 1) Purging the macro from memory, by calling: [PurgeMacro](#)([SplitPath](#)([CurrMacroFilename](#)()), [_NAME_](#)) 2) Explicitly reloading the macro by first calling [LoadMacro](#)() and then [ExecMacro](#)() [LoadMacro](#)(macro_name) [ExecMacro](#)(macro_name) This command includes a history list of previous entries passed to it. See [AddHistoryStr](#)() for additional information.

[Examples](#): /***** A macro to ALWAYS load and execute [macros](#) from disk. *****/ [proc](#) [ExecMacroFromDisk](#)() [string](#) fn[65] = "" [if](#) [Ask](#)("Macro to Exec:", fn) [and](#) [Length](#)(fn) [if](#) [LoadMacro](#)(fn) [ExecMacro](#)(fn) [PurgeMacro](#)(fn) [endif](#) [endif](#) [end](#) Following is an example of how to pass a command line to a macro. Everything following the space(s) after "state" is assigned to the MacroCmdLine editor variable. [ExecMacro](#)("state -s -l -ftemp. sta") To run a public macro procedure named DelBlankLines() found in a macro file named delete-blank: [ExecMacro](#)("delete-blank", "DelBlankLines") To run a public macro procedure named mMarkPara() found in the .ui file: [ExecMacro](#)".ui", "mMarkPara") See Also: [isMacroLoaded](#)(), [LoadMacro](#)(), [PurgeMacro](#)(), [AddHistoryStr](#)(), [ExecLoadedMacro](#)() [Variables](#): [MaxHistorySize](#), [TSEPath](#), [MacroCmdLine](#)

ExecScrapMacro

Executes the scrap Keyboard Macro.

[Syntax](#): [INTEGER](#) [ExecScrapMacro](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Executes the last scrap Keyboard Macro defined in the current editing session. Scrap [macros](#) are created by pressing <Enter> in response to the "Assign to key" prompt of [RecordKeyMacro](#)(). If no scrap macro is defined, no action occurs and zero is returned. See Also: [RecordKeyMacro](#)()

Executing External Commands from Within the Editor

The Dos, CaptureDosOutput, and Shell commands allow you to execute external commands from within the editor.

* Dos (<Alt F9>)

This command allows you to execute a single external command from the editor. When you execute this command, the editor displays the prompt "OS command:". Enter any valid external command and press <Enter>. The specified command is executed. Upon its completion, the prompt "Press a key to continue..." is displayed. Press any key on the keyboard to return to the editor and re-display the screen as it was prior to execution of this command.

* CaptureDosOutput (<AltShift F9>, or "Capture OS Output" under Util

[Menu](#)) Like the Dos command, this command allows you to execute a single external command from the editor. However, CaptureDosOutput captures the screen output from the command and loads it into a temporary file upon

completion. When you execute CaptureDosOutput, the editor displays the prompt "OS command capture:". Enter any valid external command and press <Enter>. The specified command is executed. Upon its completion, any screen output that can be redirected is placed in a temporary file. (Screen output that cannot be redirected is not included in the temporary file.) You are then placed in the temporary file. To return to the file you were editing at the time you executed CaptureDosOutput, you must switch back to that file. * Shell (<F9>, or "Shell" under Util [Menu](#)) This command temporarily suspends the editor and places you at an operating system prompt, allowing you to execute any number of external commands. Type "exit" at a system prompt and press <Enter> to return to the editor and re-display the screen as it was prior to execution of the Shell command. To remind you that you are in the editor's Shell facility, the notation "TSE-Shell" is added to the operating system prompt string when Shell is executed. If you do not want the system prompt string to be changed when you execute Shell, you can configure the editor to leave the prompt string unchanged. (For more information, see "Change Prompt in Shell" under "[Command/Format Options](#)" in the chapter on "[Configuring the Editor Interactively](#)".)

Executing Macros (Overview)

To invoke (or "execute") your macro file at any time after you have compiled it, press <Ctrl X> (or select "Execute" from the Macro Menu). The editor displays the prompt, "

[Execute macro](#):". To execute the "hello" macro file created previously, simply type "hello" at this prompt. Note that [macros](#) can also be: - Assigned to a key, so that they execute when you press that key. - Assigned to a user-defined menu, so that they execute when you select that [menu](#) item. - Assigned to certain editor events (such as when you start or terminate the editor, or switch files). This is discussed in more detail later.

Executing Supplemental Macros from the Potpourri

A miscellaneous collection, or "

[potpourri](#)," of supplemental [macros](#) is available by selecting "[Potpourri](#)" under the Util Menu (or by pressing <Shift F10>). This option pops up a PickList of supplemental macros that can be executed. An optional short description is included with each macro in the Potpourri PickList. When the Potpourri PickList pops up, the following options are available: - You can select a macro for execution. - You can display and edit Help information for macros in the list. - You can add your own macros (and Help information) to the list for ready access from the pull-down Menu. - You can remove existing entries from the list. To execute a macro from the Potpourri PickList, move the cursor bar to the desired macro and press <Enter>. The editor's [SpeedSearch feature](#) and other PickList scrolling options are available within this pop-up list. (For more information, see "[Using PickLists](#)" in the chapter on "[Command Prompts and PickLists](#)".) To display Help information about a macro, move the cursor bar to the macro in the Potpourri list and press <F1>. (For more information on obtaining Help on a topic, see "[Context-Sensitive Help](#)" in the chapter on "[Getting Help in the Editor](#)".) To add a new macro to the Potpourri list, press <Ins>. The editor prompts you for the name of the macro file you want to add. This must be a Compiled Macro file. However, do not include the ".MAC" extension; the editor forces a ".MAC" extension to whatever macro filename you enter. The editor then displays a one-page window in which you can enter your own description. The text on the top line of this "help description" window will be displayed beside the macro name in the Potpourri PickList. When complete, the editor updates the editor program to include the indicated macro file and description in the Potpourri list. To edit Help information for an existing macro in the Potpourri list, move the cursor bar to the desired macro and press <Alt E>. To remove a macro from the Potpourri list, move the cursor bar to the desired macro in the PickList and press . The editor prompts you for confirmation to delete the macro. Upon confirmation, the editor removes the macro from the Potpourri list. In order to execute a macro from the Potpourri list, or to add a macro to the list, the Compiled Macro file containing the macro must be available to the editor in one of several pre-determined macro file directories. If the macro requires a

data file, the data file should be placed in the same directory as the Compiled Macro file. (For more information on the directories in which the editor searches for Compiled Macro files, see the "[macro file search order](#)" described under "[Creating and Using Compiled Macros](#)" in the chapter on "[Creating and Using Macros](#).") When you select a macro for execution from the Potpourri PickList, the editor issues [the ExecMacro command](#) for the indicated Compiled Macro file. (For more information on executing Compiled [Macros](#), see "[Loading and Executing Compiled Macros](#)" in the chapter on "[Creating and Using Macros](#).") Following are some of the macros included in the standard Potpourri macro list. (For a complete list, pop up the Potpourri PickList!) Additional information and prompts are provided by the macros themselves.

- * Adjust ("Adjust" on [Potpourri](#) PickList) This macro allows you to adjust text within a block by aligning it either on the left or right, or by centering it. If a column block is marked, the macro operates on the text in the block using the leftmost and/or rightmost column boundary (as appropriate). If a line or character block is marked, it operates on the text on the lines spanned by the block, using the left and/or right margin (as appropriate). A selection menu issued by the macro allows you to choose whether you want to left adjust, right adjust, or center the text.
- * Book ("Book" on [Potpourri](#) PickList) This macro provides a convenient way to set Bookmarks, or to go to or list the Bookmarks that are currently set. The macro maintains the current set of Bookmarks in a circular ring of 26 Bookmarks. The following key assignments are activated to allow you to perform the indicated function: <Ctrl /> Set a Bookmark (uses the next available Bookmark in the Bookmark ring) <Ctrl <> Go to the previous Bookmark in the Bookmark ring <Ctrl >> Go to the next Bookmark in the Bookmark ring <CtrlShift /> Displays a Bookmark list, consisting of the filename, line number, and text in which each of the Bookmarks are set. This option allows you to delete or go to a selected Bookmark.
- * CD ("CD" on [Potpourri](#) PickList) This macro provides a facility for changing the current drive and/or directory. You can indicate a specific drive/directory to which to change, or optionally select a new directory from a database containing the drives and directories on your system. When you execute the CD macro, the current directory is displayed on the StatusLine; and a prompt box is displayed that contains the directory of the current file, along with the message "Change to directory:". To change to the directory of the current file (as displayed in the prompt), just press <Enter>. To change to another directory, you can type the entire path name of the desired directory and press <Enter>. Unless a drive is included in the path you specify, the macro looks for the indicated directory on the current drive. You can also change to another directory using the optional "database directory" feature included in the macro. This allows you to select a directory based on a partial or ambiguous directory name specification. Use this macro to build and maintain a database of directories on your machine. Once this database has been created, you can specify a partial directory name to change to; or you can enter an ambiguous directory entry, and then select the desired directory from a PickList displayed by the macro. If the database directory exists, the CD macro processes the entries you make at the "Change to directory:" prompt as follows:
 - The macro first attempts to match the specified directory path exactly. If a matching directory exists, that directory becomes the current directory and the macro terminates.
 - If an ambiguous entry is made (that is, one which contains the wildcard character "*"), the macro searches the database for matching directory entries, and displays a PickList of those entries. You can select the desired directory by moving the cursor bar to that directory entry and pressing <Enter>. The selected directory becomes the current directory and the macro terminates.
 - Otherwise, the macro treats your entry as a partial entry, and searches the database for a directory path(s) that begins with the string entered at the prompt.
 - If only one directory is located that matches the partial entry, that directory becomes the current directory and the macro terminates.
 - If more than one matching directory is located, the macro displays a PickList of matching entries from which you can select the desired directory. The selected directory becomes the current directory and the macro terminates.
 - If a match still has not been found, the macro then searches the database for any directory that contains the indicated string, anywhere in the directory name. If only one match is found, that directory becomes the current directory and the macro terminates. If multiple matching entries are found, a PickList of matching entries is displayed and the macro terminates once you select an entry.
 - Finally, if no match is found, the macro issues a message that it could [not](#) find any matches, and then terminates.
 To initially create or to update the directory database, press <Alt U> once the "Change to directory:" prompt is displayed. The macro then builds, or rebuilds, a database of all the directory entries that are currently on the selected drives. The directory entries are stored in the CD "database file" (TSECD.DAT), which is placed in the same directory from which the CD macro was loaded. To configure several options of the macro, press <Alt C> once the "Change to directory:" prompt is displayed. You can then select the drives that are to be searched when the directory database is being built (or rebuilt); you can choose to add the CD macro to your AutoLoad List; or you can choose to remove the CD macro from your AutoLoad List.
- * CmpFiles ("CmpFiles" on [Potpourri](#) PickList)

This macro allows you to compare the current file with another file, by simultaneously displaying and scrolling the two files, and highlighting the differences. When you execute this macro, it prompts you for the name of the file to compare with the current file. You can even compare the current file with a file by the same name located in an archive file (that is, in a compressed ".ARC", ".ARJ", ".LZH", or ".ZIP" file). To select a file in an archive, specify the name of the archive file at the prompt; the macro will attempt to locate a file within the archive that has the SAME NAME as the current file, and uncompress that particular file. Once you specify the comparison file, the macro then loads the comparison file into the editor, and splits the screen into two vertical windows, with the current file displayed in one, and the comparison file in the other. To toggle the display between vertical and horizontal windows, press the <Spacebar>. To synchronize the text display of the two files and begin the comparison, press <Enter>. Once the macro encounters differences in the files, it displays the relevant portion of each file (in the appropriate window). The text that differs between the two files is highlighted in each window, for easy comparison. You can continue this process throughout the files. While this macro is in operation, a number of features are in effect: - [Cursor](#) movement commands that you execute in either window will cause simultaneous scrolling in the other window. - To move to the beginning of the highlighted "difference" text, press <Grey ->; to move to the end of the difference text, press <Grey +>. - To continue the comparison process, press <Enter> again. - To display the HelpScreen for this macro, press <F1>; to access the macro's pull-down [menu](#), press <F10>. - To minimize false comparisons, white (space or tab) characters, by default, are automatically converted to a single space character. To toggle this option [OFF/ON](#), press <W>. - You can change the video mode. To terminate the macro, press <Escape> (or select "Exit" from the "File" entry of the macro pull-down menu). The macro gives you the option to "flush" the comparison file (that is, remove it from memory). * CUAMark ("CUAMark" on [Potpourri](#) PickList) This macro enables CUA-style (Common User Access) key assignments in the editor for [marking](#) and manipulating character and column blocks. When you execute this command, the shifted cursor-movement keys can be used for [marking](#) blocks, and additional [block operations](#) can be accomplished using select shifted keys. CUA-style blocks are also called non-persistent blocks. This means that in general, if you press a non Block Mark/extend key, the block is unmarked. When using CUA-style blocks, typing and pasting (from the Windows Clipboard) will replace the block with the typed character or the contents of the [Windows](#) Clipboard. The following shifted keys are assigned to the indicated block operation or command: CUA-style [Key](#) Block-[marking](#) Operation or Command _____

_____ <Shift CursorLeft> Mark/extend Block, move left one column
 <Shift CursorRight> Mark/extend Block, move right one column <Shift CursorUp> Mark/extend Block, move up one line
 <Shift CursorDown> Mark/extend Block, move down one line <Shift Home> Mark/extend Block, go to beginning of line
 <Shift End> Mark/extend Block, go to end of line <Shift PgUp> Mark/extend Block, move up one page <Shift PgDn> Mark/extend Block, move down one page
 <CtrlShift CursorRight> Mark/extend Block, move right one word <CtrlShift CursorLeft> Mark/extend Block, move left one word
 <CtrlShift Home> Mark/extend Block, go to top of window <CtrlShift End> Mark/extend Block, go to bottom of window <CtrlShift PgUp> Mark/extend Block, go to beginning of file
 <CtrlShift PgDn> Mark/extend Block, go to end of file Adding the <alt> key to any of the above combinations will instead Mark/extend a Column Block. <Shift Ins> PasteReplace (from the Windows ClipBoard)
 <Shift Del> Cut (to the Windows ClipBoard) <Ctrl Ins> Copy (to the Windows ClipBoard) <Ctrl Del> DelBlock (delete marked Block)
 delete marked Block or delete character <GreyDel> delete marked Block or delete character <BackSpace> delete marked Block or backspace Optionally, (by selecting CUAMark options) the following keys will also be assigned: <LeftBtn> Mark/extend Block via the mouse by dragging <Shift LeftBtn> Mark/extend Block via the mouse clicking
 <Ctrl v> PasteReplace (from the Windows ClipBoard) <Ctrl c> Copy (to the Windows ClipBoard) <Ctrl x> Cut (to the Windows ClipBoard) <Ctrl F> Find. If text is found, it is marked as a CUA-style block. <Ctrl L> Repeat Find commands. If text is found, <F3> it is marked as a CUA-style block. <CtrlShift L> Repeat Find Reverse commands. If text is <Shift F3> found, it is marked as a CUA-style block. If you want to automatically enable this feature every time you start the editor, add the CUAMark macro to your autoload file. (For more information on adding a macro to your autoload file, see "[The AutoLoadList Command](#)" in the chapter on "[Creating and Using Macros](#).") The first time the CUAMark macro is loaded, it will self-configure by asking you several questions. To reconfigure CUA [Marking](#) at another time, assuming that you viewing the CUAMark help from the Potpourri list, press <escape>, <enter> to reconfigure CUAMark. If you are [not](#) in [Potpourri](#), and assuming that CUAMark is already loaded, select Macro, Execute, clear the prompt by pressing your delete line key, press <enter>.

and select setup under CUAMark. * DelBlank ("DelBlank" on [Potpourri](#) PickList) This macro removes blank lines from a file or block. It gives you the option to either remove all blank lines, or replace multiple consecutive blank lines with a single blank line. If the cursor is located within a marked block in the current file when you execute this macro, the editor prompts to confirm whether to limit operation to the block; otherwise, it will operate on the entire file. If the cursor is not located at the beginning of the block or file, as appropriate, the editor prompts for you to select whether to begin operation from the beginning of the block or file, or to begin with the current line. Finally, the editor prompts for you to select whether you want to replace multiple consecutive blank lines with one blank line, or to remove all blank lines. * Expand ("Expand" on [Potpourri](#) PickList) This macro file provides a macro that operates like a limited, dynamic Template mode. It expands, or completes, the "partial" word to the left of the cursor based on other words in the file. When you execute Expand from the Potpourri PickList, the Expand macro is assigned to <Ctrl H>. With this macro activated, just press <Ctrl H> each time you want to complete the partial word immediately preceding the cursor. The macro searches backward and forward in the file for words that begin with the string that you typed as the partial word. If only one match is found, the partial word is replaced with the expanded word. If multiple matches are found, the macro first displays a PickList of expanded entries from which you can choose. To select an expanded word from the PickList, move the cursor bar over the desired entry and press <Enter>. Your partial word is then replaced in the text with the expanded word you selected. * Expr ("Expr" on [Potpourri](#) PickList) This macro operates as a simple calculator, and handles decimal or hexadecimal [numbers](#). For decimal [numbers](#), only whole [numbers](#) are accepted. The following operations are available: Operator Operation _____ >
[SHR](#) (bitwise shift right) < [SHL](#) (bitwise shift left) ~ COMP (2's complement) % [MOD](#) (modulo division) & AND (bitwise AND) | OR (bitwise OR) ^ XOR (bitwise eXclusive-OR) + Addition - Subtraction * Multiplication / Division
The order of precedence is the same as that in SAL, unless appropriately overridden using parentheses. Hexadecimal numbers can be notated in C-language format, or by placing an "h" or "\$" at the end of the number. However, the first character of the number MUST be numeric. * FillNum ("FillNum" under [Potpourri](#) PickList) This macro fills a column block with a series of numbers that can be incremented or decremented. You specify the beginning and (optionally) ending number in the series, as well as the positive or negative amount by which to increment or decrement the series. * FindEOL ("FindEOL" on [Potpourri](#) PickList) This macro file provides a group of macros that allow you to find and replace text that includes the end of line, if you specify the end of line (as "\$") in the search string. These macros are useful for locating a text [string](#) that begins on one line and ends on another. They are also useful [for](#) such tasks as: - Deleting extra blank lines in a file For example, to change all occurrences of 2 blank lines to a single blank line, use this macro to search for "\$\$\$" [and](#) replace with "\$\$". - Removing trailing text from certain lines For example, to remove a trailing comma from any lines immediately followed by a blank line, use this macro to search for ",\$\$" and replace with "\$\$". When you execute this macro file, special find, replace, and repeat find/replace [macros](#) are assigned to keys. A message is displayed to identify the key to which each macro is assigned. * Grep ("Grep" on [Potpourri](#) PickList) This macro searches for a specified string in multiple files on disk, and displays a PickList of all lines, within each relevant file, that contain the specified string (in a listing similar to the CompressView command). When you execute this macro, it prompts you for the search string; the file, directory, and/or drive specifications of the files in which to search; and the desired search options. The macro supports all the search options of the Find command (including Regular Expressions), as well as the "[Search SubDirs](#)" (-s) option (to allow searching within subdirectories of any specified directories). * InsMatch ("InsMatch" on [Potpourri](#) PickList) This macro causes the editor to insert the matching character for a specific group of bracketing character pairs, including the following: [], (), {}, <>, ", ""'. Execute this macro to toggle the "[insert](#) matching" mode ON or OFF. When ON, the matching character of a pair is automatically inserted when you type the first character. For example, if you enter the character {, this macro inserts the matching character, }. * Justify ("Justify" on [Potpourri](#) PickList) This macro is an extension of the WrapPara command. Like WrapPara, this macro formats text in paragraph style between left and right margins: the text is aligned on the left at [the effective left margin](#), and wrapped on the right at the right margin. In addition, though, this macro also justifies each line of text at the right margin. * LongstLn ("LongstLn" on [Potpourri](#) PickList) This macro locates the longest line in the current file. When you execute this macro, you are positioned on the longest line, and its length is displayed on [the StatusLine](#). * Overlay ("Overlay" on [Potpourri](#) PickList) This macro is a form of synchronized scrolling that allows you to check minor differences between two files. This macro will "overlay" the current file with the next file in the ring, by switching back and forth between the two files. This will allow you to

quickly spot minor differences between the files. While the macro is switching between files, it will allow you to move the cursor around in the files, and keep the two files synchronized. Before executing the macro, set the screen display for the two files at the area of text that you want to compare. Once you activate the macro, it repeatedly overlays the current file with the next file in the file Ring. It accomplishes this by continuously switching back and forth between the two files. Differences in similar files become readily apparent, as the screen display rapidly alternates where the text is different. While the macro is switching between files, only the cursor keys are active (<CursorUp>, <CursorDown>, <CursorLeft>, <CursorRight>, <PgUp>, <PgDn>). [Cursor](#) movement commands are executed simultaneously in both files, allowing you to scroll through the files to see any differences. To terminate the macro, press <Escape>, or any non-cursor key. You are then placed back in the file from which you initiated the macro. *

RePage ("RePage" on [Potpourri](#) PickList) This macro places formfeed characters (ASCII 12) throughout a file, based on the page size. (The page size is determined by the value of the PrintLinesPerPage variable, which can be set from the Print Menu, by selecting "Lines Per Page" under "Set Options"). When you execute this macro, it pauses on each line where a formfeed should go, according to the page size, and places a cursor bar on that line. You can then select that line on which to place the formfeed, or move the cursor bar to a previous line in the file to select a different line on which to place the formfeed. After you select the line for the formfeed, the macro resumes, relative to the location of the selected line. *

ShellExe ("ShellExe" on [Potpourri](#) PickList) If a command line is passed, it is simply passed along to the Windows ShellExecute function. If a block is marked, the first line of the block is passed to the Windows ShellExecute function. You can use this to go to a URL in the default browser, or to send an email via the default mailer. If you are editing an html file and would like to preview it in a Browser window, simply run ShellExe, and as long as the file extension is .htm or .html, the file will open in the default browser. To go to a web page, mark the following URL and run ShellExe: www.semware.com To send an email, mark any of the following addresses and run ShellExe: james.t.kirk@starfleet.command.org James T. Kirk <james.t.kirk@starfleet.command.org> [mailto:James T. Kirk](mailto:James.T.Kirk@starfleet.command.org) <james.t.kirk@starfleet.command.org> *

ShowKey ("ShowKey" on [Potpourri](#) PickList) This macro displays information about the assignable keys on the keyboard. When you execute this macro, an information chart is displayed on the screen. You then press a key or key-combination (using the <Ctrl>, <Shift>, and <Alt> keys). If the key is one that the editor recognizes as an assignable key, a line is displayed in the chart that identifies the following information for that [key](#): - screen display character (if applicable) - ASCII code - scan code - hexadecimal representation - decimal representation - key name (that is, the "name" to use for that key in an editor key assignment) If the key is not assignable, nothing is displayed when you press that key. *

State ("State" on [Potpourri](#) PickList) This macro allows you to interactively save the editing environment for the current session, or restore the previously-saved environment. The State macro can come in handy when you need to interrupt your current editing session to view another set of files. Without exiting from the editor, you can save the environment for the current session; load additional files and/or change certain editing options; then automatically restore the previous environment so you can return to the set of files and options as they were previously. When you execute this macro, the editor displays a prompt box from which you can select "Save State" to save the current editing environment, or select "[Restore State](#)" to restore the previously-saved environment. (For more information on saving and restoring the editing environment, see "[Saving/Restoring the Editing Environment](#)" in the chapter on "[Loading, Processing, and Saving Files](#).") *

Sum ("Sum" on [Potpourri](#) PickList) This macro sums up a column of numbers that have been marked in the text as a column block. The result is optionally inserted in the text. Sum handles decimal and hexadecimal [numbers](#). For decimal [numbers](#), it accepts positive and negative (identified with a preceding "-" sign) numbers, as well as decimal-point fractions. *

SynHiUtl ("SynHiUtl" on [Potpourri](#) PickList) This macro allows you to dynamically load an arbitrary SyntaxHilite mapping (.SYN) file and associate it with the current file. It also allows you to activate or terminate (as appropriate) SyntaxHilite mode for the current file. When you execute this macro, it displays the name of the mapping (.SYN) file associated with the current file (if applicable). You can then select a new mapping file to load and associate with the current file, or you can turn SyntaxHilite mode ON or OFF for the current file. *

TabUtil ("TabUtil" on [Potpourri](#) PickList) This macro converts tab characters in the current file to spaces, or converts spaces to tab characters. When you execute this macro, select "DeTab" to convert each tab character to the number of spaces that is equal to the current TabWidth. Alternatively, select "EnTab" to convert each occurrence of the number of spaces equal to the TabWidth, to a tab character. To limit the scope of this utility to a block, execute the macro from within a marked block. *

Uniq ("Uniq" on [Potpourri](#) PickList) This macro removes duplicate lines from a file or block. Only

the first occurrence of a given line of text is retained; all others are deleted. The duplicated lines do not have to be adjacent to each other, and the order of the remaining lines in the file is not changed. When you execute this command, if the cursor is located on a line that includes a column block that is less than 128 characters in width, the editor will allow you to limit the comparison for duplicates to text within the block. For any duplicate text that is found, the entire line containing that text will be deleted. If the cursor is located within a marked block in the current file, the editor prompts to confirm whether to limit operation to the block; otherwise, it will operate on the entire file. If the cursor is not located at the beginning of the block or file, as appropriate, the editor prompts for you to select whether to begin operation from the beginning of the block or file, or to begin with the current line. * vMatch ("vMatch" on [Potpourri PickList](#)) Show the matching opening/closing character when the cursor is placed on the corresponding opening/closing character. Additionally, when an opening/closing character is typed, the matching opening/closing character is temporarily highlighted. Characters matched are:)}] * Where ("Where" on [Potpourri PickList](#) or "Where Is" under [Util Menu](#)) This macro searches one or more drives for files that match an indicated filename specification. A PickList of all matching filenames is displayed, allowing you to select file(s) to edit or to delete (or on which to perform other available operations). When you execute the macro, it prompts you for the filename for which to search, with "Search for file:". Following is the accepted format of the filename specification: - A complete name or an ambiguous name (with wildcard characters) can be specified, and can optionally [include](#) drive and/or path information. - Multiple filename specifications can be entered. - One or more drives can be specified. If particular drive(s) are specified, the macro will limit the search to the indicated drive(s); if no drive is specified, all drives (for which the macro is configured to search) will be searched. The drive(s) can be specified only as the beginning character(s) of the file specification entry; the indicated drive(s) will apply to all filename specifications that are entered. - If a particular directory is included in a filename specification, the macro will limit the search for that filename to the indicated directory and all of its subdirectories; if no directory is specified, the macro will search all directories and subdirectories of each drive that is searched. Once you enter a filename specification, the macro then searches the applicable drives and directories. All matching filenames are displayed in a PickList. You can select a particular file to edit or delete by positioning the cursor bar on that filename in the PickList, and pressing <Enter>. Or you can tag multiple files (by pressing the <Spacebar> with the cursor bar positioned on a file) on which to operate. The PickList is automatically removed once the indicated operation is performed. Otherwise, to remove the PickList (and terminate the command), press <Escape>. Or, to terminate the macro and release any memory used to store the PickList (if one exists), press <Alt X>. While a search is in progress, you can immediately stop the search by pressing <Escape>. A PickList will be presented of those filenames that were located before the search was terminated. While the "Search for file:" prompt is displayed, a number of options are available: - To display an initial HelpScreen for the macro, press <F1>. - To configure several options of the macro, press <Alt C>. You can then select the drives that are to be searched (when no drives are included in the specification entry); you can choose to add the Where macro to your AutoLoad List; or you can choose to remove the Where macro from your AutoLoad List. - To re-display the PickList from a previous search, press <Alt L>. While a PickList is displayed, other options are available: - To display additional HelpScreen information [about](#) other file operations available from the PickList, press <F1>. - To sort the list of entries in the PickList, press <Ctrl S>. A pop-up menu allows you to select the sorting sequence. - To tag the file on which the cursor bar is positioned, press the <Spacebar>. - To display the entire directory containing the file on which the cursor bar is positioned, press <Alt L>. * Wipe ("Wipe" on [Potpourri PickList](#)) This macro file provides two macros: MoveWipe, which is a variation of the MoveBlock command; and CutWipe, which is a variation of the Cut (to the System ClipBoard) command. When you execute Wipe from the Potpourri PickList, MoveWipe is assigned to <AltShift M>, and CutWipe is assigned to <Shift Grey*>. These macros behave similarly to the commands on which they are based, except that the area of text from which the block is moved or cut, is filled with spaces. Also, with MoveWipe, if a column block is moved, the editor overlays existing text with the contents of the block, rather than inserting the contents of the block into its new location.

ExistBufferVar

Determines if the specified buffer variable exists.

Syntax: [INTEGER ExistBufferVar](#)([STRING](#) buffer_var_name [, [INTEGER](#) buffer_id]) - buffer_var_name is the name of the buffer variable to find. - buffer_id is an optional buffer id, as returned by [GetBufferId\(\)](#). Returns: Non-zero if buffer_var_name exists; [otherwise](#), zero ([FALSE](#)). Notes: If buffer_id is not specified, the current buffer is assumed. See Also: [DelBufferVar\(\)](#), [GetBufferInt\(\)](#), [GetBufferStr\(\)](#), [SetBufferInt\(\)](#), [SetBufferStr\(\)](#)

ExistGlobalVar

Verifies that the specified session global variable exists.

Syntax: [INTEGER ExistGlobalVar](#)([STRING](#) var_name) - var_name is the name of the session global variable for which to check. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. **Examples:** [if ExistGlobalVar](#)("myvar") [Message](#)("myvar exists!") [else Message](#)("myvar does [not](#) exist!") [endif](#) See Also: [GetGlobalInt\(\)](#), [GetGlobalStr\(\)](#), [SetGlobalInt\(\)](#), [SetGlobalStr\(\)](#), [DelGlobalVar\(\)](#)

Exit

Exits the editor, allowing the user to optionally save modified files, if there are any.

Syntax: [INTEGER Exit](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If no modified files exist, this command causes the editor to exit without further prompting. For every modified file loaded into the editor, this command issues the protected quit command, [QuitFile\(\)](#). If the user selects cancel as a response to the [QuitFile\(\)](#) menu, the exit operation is halted. The `_ON_EXIT_CALLED_` event is called at the beginning of this command. (See the [Hook\(\)](#) command for a description of this event.) If the SaveState variable is ON, the editor's state is saved [when](#) this command begins execution. See Also: [SaveAllAndExit\(\)](#), [QuitFile\(\)](#), [AbandonEditor\(\)](#), [Hook\(\)](#) [Variables:](#) [SaveState](#)

Exiting from the Menu System

If you select a sub-menu item that executes a command, the editor automatically deactivates the Menu and returns you to the editing session, once the command is completed or cancelled. To otherwise exit from the pull-down Menu system, press <Escape> until the main Menu is deactivated. You can then resume editing.

If the editor is configured to continually display the main Menu, the Menu system is deactivated once there is no longer a main Menu option that is highlighted. If configured otherwise, the Menu system is deactivated once the Main menu is removed from the screen.

ExpandPath

Expands the specified ambiguous path to a complete path specification.

Syntax: [STRING ExpandPath](#)([STRING](#) partial_path [, [INTEGER](#) respect_case]) - partial_path is the ambiguous or incomplete path to expand. - respect_case is an optional integer specifying whether [ExpandPath\(\)](#) should respect the case of an existing filename. If respect_case is zero (or [not](#) specified) [ExpandPath\(\)](#) performs the [Lower\(\)](#) command on the returned string. If respect_case is non-zero, [ExpandPath\(\)](#) will retain the case of the filename as it exists on disk. **Returns:** The expanded path, or the empty string if the path could not be expanded. **Notes:** If the name passed to [ExpandPath\(\)](#) is a directory name, "*.*)" is appended to the returned string. The long filename value is returned, if applicable. The [Lower\(\)](#) command is issued on the return string just before returning it to the user, if respect_case is not specified or [FALSE](#) (zero). **Examples:** Given: string st[_MAXPATH_], and the current path is "c:\tse\doc\ref" A file on disk named "Bar" exists. then: st = [ExpandPath](#)("foo") // st is "c:\tse\doc\ref\foo" st = [ExpandPath](#)("bar") // st is "c:\tse\doc\ref\bar" st = [ExpandPath](#)("bar", [TRUE](#)), // st is "c:\tse\doc\ref\Bar" See Also: [SearchPath\(\)](#), [SplitPath\(\)](#)

ExpandTabs

Expands TAB characters (ASCII 9) to spaces on the display (but not in the file).

Values: [BOOLEAN](#) **Default:** [ON](#) **Notes:** This expands the tab character to spaces only on the display. The TAB character remains unchanged in the file. The number of spaces shown will depend on the position of the tab within the line and the setting of the TabType and TabWidth variables. **Affects:** Many commands See Also: [TabType](#), [TabWidth](#)

ExpandTabsToSpaces

Expand hard tab characters on the current line to spaces.

Syntax: [ExpandTabsToSpaces\(\)](#) **Returns:** Nothing. **Notes:** The current [TabWidth](#) is used when expanding hard tabs to spaces. **Examples:** //Expand all tabs to spaces in the current file: [proc](#) [ExpandTabsCurrentFile\(\)](#) [PushPosition\(\)](#) [BegFile\(\)](#) [repeat](#) [ExpandTabsToSpaces\(\)](#) [until](#) [not](#) [Down\(\)](#) [PopPosition\(\)](#) [end](#) See Also: [EntabCurrLine\(\)](#) **Variables:** [TabWidth](#)

External Macros Vs. the User-Interface Macro

There are two fundamental types of Compiled Macro files: external and user-interface. External macro source files have extensions of ".S", whereas user-interface macro source files have extensions of ".UI".

An external macro file contains procedures,

[key](#) assignments, help text, and menus. The source file is compiled as an object file with an extension of ".MAC", and the elements in the Compiled Macro file become active when the macro (.MAC) file is loaded or executed. A user-interface macro file can contain the same elements as an external macro file. It can also contain editor variable (configuration) settings, though this is [not](#) generally recommended. A user-interface file is installed in the editor, and the elements in the user-interface automatically become active when the editor is started. (For more information on the elements included in macro files, see the chapters on "[Macro Language Syntax](#)" and "[Using Macro Language Features](#)." Also see the chapter on "[Fully Customizing the Editor](#)" in the User's Guide.) [External Macro Files](#)

----- External macro files are compiled from ".S" source files by the SAL compiler to create executable files with an extension of ".MAC". They must be explicitly loaded into the editor before they can be executed. The [LoadMacro\(\)](#) and [ExecMacro\(\)](#) commands are used to load and execute external macro files. They can subsequently be

purged during an editing session if they are no longer needed. External macro files may contain the following: *

- * [Public/Private macros](#) Public macros are global in scope and are declared with the "public" keyword. Public macros are added to the Public macros list when the macro file containing them is loaded or executed. You can execute [Public macros](#) by name with the [ExecMacro\(\)](#) command. Public macros also display in the [Public macros](#) PickList, allowing you to interactively select them, without having to type their name. Private macros are local in scope. Generally, any macro without the "public" keyword is a private macro. Certain exceptions to this rule are noted below. Private macros can only be called by other macros within the same macro file. [Private macros](#) can not be directly executed interactively.
- * [Main macro](#) The Main macro is a special macro that is always public. If present in a macro file, it is automatically executed when a macro filename is passed to [ExecMacro\(\)](#), either interactively or via a macro. External macro files are not required to have a Main macro. If the specified macro file is not already loaded when [ExecMacro\(\)](#) is called, and the macro file to execute contains a Main macro, that macro is invoked just after the macro file is loaded and the WhenLoaded macro executed (if one exists in the macro file). If the macro file does not have a Main macro, the macro file is loaded and only the WhenLoaded macro is executed (if one exists). In contrast to [ExecMacro\(\)](#), the [LoadMacro\(\)](#) command does not execute the Main macro even if one exists in the indicated macro file.
- * [WhenLoaded macro](#) The WhenLoaded macro is another special macro type. As with the Main macro, the WhenLoaded macro is not required. The WhenLoaded macro, if one exists in the macro file, is automatically executed when a Compiled Macro file is loaded into the editor from disk (that is, the first time the macro file is loaded during an editing session, or re-loaded after the macro file has been purged). When [LoadMacro\(\)](#) is called and the macro file to load has a WhenLoaded macro, that macro is invoked just after the macro file is loaded. (If the specified macro file is already loaded into the editor when [LoadMacro\(\)](#) is called, the macro file is reloaded and the WhenLoaded macro is executed.) When [ExecMacro\(\)](#) is called and the specified macro file is not already loaded, the WhenLoaded macro (if present in the macro file) is invoked first, followed by the Main macro (if present). If the specified macro file is already loaded into the editor when [ExecMacro\(\)](#) is called, the WhenLoaded macro is not executed, but the Main macro is.
- * [WhenPurged macro](#) The WhenPurged macro is a special macro that is called whenever an external macro file is purged. It is useful for removing buffers created by the macro file that contains it, or for handling any cleanup that might be necessary.
- * [Key assignments](#) Key assignments can also be included in external macro files. When the macro file is loaded, any key assignments in the included macros override the current assignments. When the macro file is purged, the previous key assignments are restored.
- * [Help](#) Help definitions (helpdefs) may be placed throughout an external macro file. Help definitions are private and can only be referenced by other [macros](#) in the same macro file.
- * [Menus](#) Menu definitions may also be placed throughout an external macro file. Menus are private and can only be referenced by other macros in the same macro file.

[The User-Interface Macro File \(TSE.UI\)](#) ----- User-interface macro files are created in the same manner as external macro files; however, you place the source in a file with an extension of ".UI" (rather than ".S"). A [key](#) difference between [external macro files](#) and user-interface files is that a user-interface file is installed in the editor, so there is no need to load or execute the macro file to gain access to any commands, menus, or help that may be defined by the macro. Until replaced (by installing a new user interface file), these are essentially a part of the editor. This is the primary advantage of installing a user-interface macro file into the editor. However, because user-interface macros are installed in the editor, they cannot be purged. Another difference between the two types of Compiled Macro files is that the user-interface macro file may contain an editor variable configuration section (though this is not generally recommended). This configuration section is denoted by the keywords "[Config](#)" and "[EndConfig](#)". User-interface macro files may contain the following: *

- * [Macros](#) The macros in a user-interface file may be assigned to keys, menus, or called by other macros (that also reside in the user-interface macro file). The major difference between these [macros](#) and [macros](#) in external macro files, is that these [macros](#) are available immediately upon editor startup.
- * [Main macro](#) The Main macro in a user-interface file behaves somewhat differently than a corresponding Main macro in an external macro file. If a Main macro exists in a user-interface macro file that is installed in the editor, the Main macro is invoked immediately after the first file has been loaded into the editor.
- * [WhenLoaded macro](#) The WhenLoaded macro in a user-interface file behaves somewhat differently than a corresponding WhenLoaded macro in an external macro file. If this macro exists in a user-interface macro that is installed in the editor, the WhenLoaded macro is invoked just after the editor has been loaded, but before the command-line specifications and any files are processed. Thus, the WhenLoaded macro in a user-interface file can

actually process the command line, and add to or remove files and/or options from the command line. The WhenLoaded macro in a user-interface macro file is often used to perform certain setup tasks before you begin your editing session. An exception to the above is the -i (Augment Load Dir) command-line option. If present on the command line when the editor is started, the -i option will be processed before the WhenLoaded macro is invoked. *

[WhenPurged macro](#) In a user-interface file, the WhenPurged macro (if it exists) is called when the editor is terminating, just before the _ON_ABANDON_EDITOR_ event is called. *

[Key](#) assignments [Key](#) assignments can also be included in user-interface macro files. [Key](#) assignments in the user-interface file may be placed anywhere throughout the file except in the editor variable configuration section. These key assignments are installed in the editor and are always available unless overridden by another key assignment enabled by the loading of an external macro file. *

[Help](#) definitions (helpdefs) may be placed throughout the user-interface macro file. Help definitions are private and can only be referenced by other macros in the user-interface macro file. *

[Menus](#) Menu definitions may also be placed throughout the user-interface macro file. They are also private and can only be referenced by macros in the user-interface macro file. Although it is [not](#) required, most user-interface macro files contain a menu definition called "MainMenu". In the standard editor user interface, the MainMenu is bound to both the <F10> and the <Escape> keys, and serves as the primary pull-down Menu system in the editor. In most situations, you will only want to modify the contents of this MainMenu to [include](#) or exclude certain commands or [macros](#) that you wish to have available through the pull-down Menu. *

[Editor Variable Configuration Section](#) The editor variable configuration section is defined by the "[Config](#)" and "[EndConfig](#)" keywords. This is an optional section that can be used to make editor variable assignments. Most [editor variables](#) can be set in this section. There are a few editor variables which can ONLY be set in this section (as they only take effect when the editor is initially started), and there are other [editor variables](#) which cannot be set in this section. (For more information, see the descriptions of the variables (which note restrictions for the variable) in the chapter on "[Editor Variables](#).") Use of a configuration section in the user-interface file to set or modify editor variable assignments is not recommended. In the standard version of the editor, this section is not used. An alternate, easier method for setting editor variables is provided through IConfig, the editor's [menu](#)-driven Interactive Configuration facility. It is recommended that you use IConfig to set editor variables, and then install the settings into the editor by selecting "Save Current Settings" under the Options Menu. Note that these two methods for setting [editor variables](#) are [not](#) compatible with each other, so you should [not](#) use both. (For more information on using IConfig, see the chapter on "[Configuring the Editor Interactively](#)" in the User's Guide.) (For more information on customizing the user-interface macro file, see the chapter on "[Fully Customizing the Editor](#)" in [the User's Guide](#).)

fClose

Low-level file-access command: closes a specified file using its handle.

This command is intended for ADVANCED USAGE.

[Syntax](#): [INTEGER](#) fClose([INTEGER](#) handle) - handle is the handle, as assigned by the operating system, of the file to be closed. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Use this command to close files opened by the low-level file-access commands [fOpen\(\)](#) or [fCreate\(\)](#). [Examples](#): // Write the current date and time to a file // Assume success if > 0 bytes written // Returns [TRUE](#) or [FALSE](#) [integer proc](#) SaveDateAndTime([string](#) fn) [integer](#) h, bytes h = [fCreate](#)(fn) [if](#) h == -1 [return](#) ([FALSE](#)) [endif](#) bytes = [fWrite](#)(h, [GetDateStr](#)() + ' ' + [GetTimeStr](#)()) [fClose](#)(h) [return](#) (bytes > 0) [end](#) See Also: [fCreate\(\)](#), [fOpen\(\)](#), [fRead\(\)](#), [fReadFile\(\)](#), [fSeek\(\)](#), [fWrite\(\)](#), [fWriteFile\(\)](#)

fCreate

Low-level file-access command: creates a new file on disk and returns the associated handle.

This command is intended for ADVANCED USAGE.

Syntax: [INTEGER fCreate](#)([STRING](#) filename [, [INTEGER](#) attributes]) - filename is name of the file to create. - attributes are the optional desired attributes for the newly-created file. attributes may be a combination of the following: - [_NORMAL_](#) specifies the file can be read from or written to. - [_READONLY_](#) specifies the file can be read from but not written to. - [_HIDDEN_](#) specifies the file is hidden and does not appear in a file directory listing. - [_SYSTEM_](#) specifies the file is a system file. - [_ARCHIVE_](#) specifies the file is marked for archiving. If [not](#) specified, attributes defaults to [_NORMAL_](#). Returns: The handle, as assigned by the operating system, of the newly-created file; or -1, if the file could not be created. Notes: If the file specified by filename already exists, it is first erased (truncated). Until it is closed, you have read/write access to a file created with this command. The specified attributes are assigned to the file when it is closed. **Examples:** // Write the current date and time to a file // Assume success if > 0 bytes written // Returns [TRUE](#) or [FALSE](#) [integer proc](#) SaveDateAndTime([string](#) fn) [integer](#) h, bytes h = [fCreate](#)(fn) [if](#) h == -1 [return](#) ([FALSE](#)) [endif](#) bytes = [fWrite](#)(h, [GetDateStr](#)() + ' ' + [GetTimeStr](#)()) [fClose](#)(h) [return](#) (bytes > 0) [end](#) See Also: [fClose](#)(), [fDup](#)(), [fDup2](#)(), [fOpen](#)(), [fRead](#)(), [fReadFile](#)(), [fSeek](#)(), [fWrite](#)(), [fWriteFile](#)(), [FileExists](#)()

fDup

Low-level file-access command: creates a new file handle by duplicating an existing handle for a currently-open file.

This command is intended for ADVANCED USAGE.

Syntax: [INTEGER fDup](#)([INTEGER](#) handle) - handle is a file handle previously created by [fOpen](#)(), [fCreate](#)(), [fDup](#)(), or [fDup2](#)(). Returns: The new handle, as assigned by the operating system, for the specified file handle; or -1, if the handle could not be duplicated. Notes: The new file handle created by [fDup](#)() shares the same file read/write pointer with the original handle. That is, if the file read/write pointer is changed using one handle (for example, by [fSeek](#)()), the change is reflected when using the alternate handle. **Examples:** // redirect stdout and stderr to file fn [integer proc](#) Redirect([string](#) fn, [var integer](#) _fd1, [var integer](#) _fd2) [integer](#) fd1, fd2, fd_out, fd_err fd_out = [fCreate](#)(fn) fd1 = [fDup](#)([_STDOUT_](#)) [fDup2](#)(fd_out, [_STDOUT_](#)) fd_err = fd_out fd2 = [fDup](#)([_STDERR_](#)) [fDup2](#)(fd_err, [_STDERR_](#)) [fClose](#)(fd_out) _fd1 = fd1 _fd2 = fd2 [return](#) ([TRUE](#)) [end](#) // restore normal stdout and stderr [integer proc](#) CloseRedirect([integer](#) fd1, [integer](#) fd2) [fDup2](#)(fd1, [_STDOUT_](#)) [fClose](#)(fd1) [fDup2](#)(fd2, [_STDERR_](#)) [fClose](#)(fd2) [return](#) ([TRUE](#)) [end proc main](#)() [integer](#) h1, h2 Redirect("foo.bar", h1, h2) //redirect stdout to foo.bar [lDos](#)("grep", "[if](#) *.*") //output of this command goes //to foo.bar, same as if we //ran [Dos](#)("grep if *.* >foo.bar") CloseRedirect(h1, h2) //restore stdout [end](#) See Also: [fOpen](#)(), [fCreate](#)(), [fDup2](#)()

fDup2

Low-level file-access command: forces a file handle to be a duplicate handle

for another currently-open file. This command is intended for ADVANCED USAGE. **Syntax:** [INTEGER fDup2](#)([INTEGER](#) first_handle, [INTEGER](#) second_handle) - first_handle is the file handle for which to create a duplicate. - second_handle is the file handle to force to be a duplicate of first_handle. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: second_handle shares the same file read/write pointer as first_handle. That is, if the file read/write pointer is changed using one handle (for example, by [fSeek](#)()), the change is reflected when using the alternate handle. [fDup2](#)() creates a new handle with the value of second_handle. If the file associated with second_handle is open when [fDup2](#)() is called, the file is closed. **Examples:** See the [fDup](#)() command for an example.

See Also: [fOpen\(\)](#), [fCreate\(\)](#), [fDup\(\)](#)

FFAttribute

Returns the attribute of the last file found by a find-file command (such as by the

[FindFirstFile\(\)](#) command). This command is intended for ADVANCED USAGE. **Syntax:** [INTEGER FFAttribute\(\)](#)

Returns: The attribute of the last file found via any of the find-file commands. Notes: Supported attribute [values](#) returned by this command are: - [_NORMAL_](#) indicates files that can be read from or written to. - [_READONLY_](#) indicates files that can be read from but [not](#) written to. - [_HIDDEN_](#) indicates files that are hidden and do [not](#) appear in file directory listings. - [_SYSTEM_](#) indicates files that are system files. - [_VOLUME_](#) indicates the volume label entry. - [_DIRECTORY_](#) indicates directory entries. - [_ARCHIVE_](#) indicates files that are marked for archiving.

Examples: [integer](#) attribute [if](#) [FindThisFile](#)("foo.bar", [_HIDDEN_](#)[_SYSTEM_](#)[_READONLY_](#)) attribute = [FFAttribute](#)() [endif](#) See Also: [FindFirstFile\(\)](#), [FindNextFile\(\)](#), [FindFileClose\(\)](#), [FindThisFile\(\)](#), [FFName\(\)](#), [FFSize\(\)](#), [FFDate\(\)](#), [FFDateStr\(\)](#), [FFTime\(\)](#), [FFTimeStr\(\)](#)

FFDate

Returns the date, in encoded format, of the last file found by a find-file command (such as by the

[FindFirstFile\(\)](#) command). This command is intended for ADVANCED USAGE. **Syntax:** [INTEGER FFDate\(\)](#)

Returns: The date of the last file found via any of the find-file commands. Notes: The date is returned in an encoded format that is suitable for comparison of dates returned by this command and the [PBDate\(\)](#) command. **Examples:**

[integer](#) date [if](#) [FindThisFile](#)("foo.bar", [_HIDDEN_](#)[_SYSTEM_](#)[_READONLY_](#)) date = [FFDate](#)() [endif](#) See Also: [FindFirstFile\(\)](#), [FindNextFile\(\)](#), [FindFileClose\(\)](#), [FindThisFile\(\)](#), [FFName\(\)](#), [FFSize\(\)](#), [FFAttribute\(\)](#), [FFDateStr\(\)](#), [FFTime\(\)](#), [FFTimeStr\(\)](#)

FFDateStr

Returns the date, in displayable format, of the last file found by a find-file command (such as by the

[FindFirstFile\(\)](#) command). This command is intended for ADVANCED USAGE. **Syntax:** [STRING FFDateStr\(\)](#)

Returns: The date of the last file found via any of the find-file commands. Notes: The date is returned in a string format that is suitable for displaying. **Examples:** [string](#) date[8] [if](#) [FindThisFile](#)("foo.bar", [_HIDDEN_](#)[_SYSTEM_](#)[_READONLY_](#)) date = [FFDateStr](#)() [endif](#) See Also: [FindFirstFile\(\)](#), [FindNextFile\(\)](#), [FindFileClose\(\)](#), [FindThisFile\(\)](#), [FFName\(\)](#), [FFSize\(\)](#), [FFAttribute\(\)](#), [FFDate\(\)](#), [FFTime\(\)](#), [FFTimeStr\(\)](#)

FFName

Returns the filename of the last file found by a find-file command (such as by the

[FindFirstFile\(\)](#) command). This command is intended for ADVANCED USAGE. **Syntax:** [STRING FFName\(\)](#)

Returns: The filename of the last file found via any of the find-file commands. Notes: The filename that is returned includes the name and the extension, if any. The drive and/or directory of the file is not included. **Examples:** [string](#) fn

[[_MAXPATH_](#)] if [FindThisFile](#)("*.*", [_HIDDEN_](#)[_SYSTEM_](#)[_READONLY_](#)) fn = [FFName](#)() endif See Also: [FindFirstFile](#)(), [FindNextFile](#)(), [FindFileClose](#)(), [FindThisFile](#)(), [FFSize](#)(), [FFAttribute](#)(), [FFDate](#)(), [FFDateStr](#)(), [FFTime](#)(), [FFTimeStr](#)()

FFSize

Returns the size of the last file found by a find-file command (such as by the [FindFirstFile](#)() command). This command is intended for ADVANCED USAGE. **Syntax:** [INTEGER](#) [FFSize](#)() Returns: The size of the last file found via any of the find-file commands. Notes: This command is valid only for files less than 2 gigabytes in size. **Examples:** [integer](#) size if [FindThisFile](#)("foo.bar", [_HIDDEN_](#)[_SYSTEM_](#)[_READONLY_](#)) size = [FFSize](#)() endif See Also: [FindFirstFile](#)(), [FindNextFile](#)(), [FindFileClose](#)(), [FindThisFile](#)(), [FFName](#)(), [FFAttribute](#)(), [FFDate](#)(), [FFDateStr](#)(), [FFTime](#)(), [FFTimeStr](#)()

FFTime

Returns the time, in encoded format, of the last file found by a find-file command (such as by the [FindFirstFile](#)() command). This command is intended for ADVANCED USAGE. **Syntax:** [INTEGER](#) [FFTime](#)() Returns: The time of the last file found via any of the find-file commands. Notes: The time is returned in an encoded format that is suitable for comparison of dates returned by this command and the [PBTime](#)() command. **Examples:** [integer](#) time if [FindThisFile](#)("foo.bar", [_HIDDEN_](#)[_SYSTEM_](#)[_READONLY_](#)) time = [FFTime](#)() endif See Also: [FindFirstFile](#)(), [FindNextFile](#)(), [FindFileClose](#)(), [FindThisFile](#)(), [FFName](#)(), [FFSize](#)(), [FFAttribute](#)(), [FFDate](#)(), [FFDateStr](#)(), [FFTimeStr](#)()

FFTimeStr

Returns the time, in displayable format, of the last file found by a find-file command (such as by the [FindFirstFile](#)() command). This command is intended for ADVANCED USAGE. **Syntax:** [STRING](#) [FFTimeStr](#)() Returns: The time of the last file found via any of the find-file commands. Notes: The time is returned in a string format that is suitable for displaying. **Examples:** [string](#) time[8] if [FindThisFile](#)("foo.bar", [_HIDDEN_](#)[_SYSTEM_](#)[_READONLY_](#)) time = [FFTimeStr](#)() endif See Also: [FindFirstFile](#)(), [FindNextFile](#)(), [FindFileClose](#)(), [FindThisFile](#)(), [FFName](#)(), [FFSize](#)(), [FFAttribute](#)(), [FFDate](#)(), [FFDateStr](#)(), [FFTime](#)()

FileChanged

Determines the current file's or buffer's modified (changed) status.

Syntax: [INTEGER](#) [FileChanged](#)([[INTEGER](#) change_flag]) - change_flag, if specified, is either [TRUE](#) or [FALSE](#). Returns: The file's current modified status, as of the time this command is invoked. If the file is not modified, zero (FALSE) is returned. Otherwise, a non-zero number that represents the current change state. On each change, this number is incremented. Notes: If TRUE is passed to [FileChanged](#)(), the file is marked as modified. If FALSE is passed, the file is marked as unmodified. Use with caution. The modified status of a file is the method the editor uses

to detect that a file has been changed. If a modified file is marked as unmodified with [FileChanged\(\)](#), normal file exiting routines will not detect changes to the file. [Examples:](#) // A LoseChanges command. [proc](#) LoseChanges() [if](#) [FileChanged\(\)](#) [if](#) [YesNo](#)("Lose Changes?") == 1 // Yes selected [FileChanged\(FALSE\)](#) [else return](#) () [endif endif](#) [QuitFile\(\)](#) [end](#) See Also: [QuitFile\(\)](#), [SaveFile\(\)](#), [ChangedFilesExist\(\)](#)

FileExists

Verifies that the specified file exists on disk, and returns the file's attributes.

[Syntax:](#) [INTEGER FileExists\(STRING fn\)](#) - fn is the name of the file for which to check on disk. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Zero is returned if the specified fn does not exist on disk. The filename can contain wildcard (? and *) characters. If this is the case, the attributes of the first matching file found will be returned. If the drive and/or path are omitted, the current drive and/or path are assumed. The following pre-defined editor [constants](#) can be used to determine what attributes are associated with a particular file: Associated Bit Editor [Constant](#) _____ 0 - Read-only Attribute [_READONLY_](#) 1 - Hidden Attribute [_HIDDEN_](#) 2 - System Attribute [_SYSTEM_](#) 3 - Volume Attribute [_VOLUME_](#) 4 - Directory Attribute [_DIRECTORY_](#) 5 - Archive Attribute [_ARCHIVE_](#) [Examples:](#) The following code shows how to determine if particular attributes exist for a specified file: [integer](#) attribute attribute = [FileExists](#)("some.file") // if the file exists at all... [if](#) attribute <> 0 // is it readonly? [if](#) attribute & [_READONLY_](#) ... [endif](#) // is it readonly or hidden or system? [if](#) attribute & ([_READONLY_](#) | [_HIDDEN_](#) | [_SYSTEM_](#)) ... [endif endif](#) // If the current file being edited exists on disk, erase // it, and force the file's modified (changed) status to // be [TRUE](#). [if FileExists\(CurrFilename\(\)\)](#) [EraseDiskFile](#) ([CurrFilename\(\)](#)) [FileChanged\(TRUE\)](#) [endif](#) See Also: [EraseDiskFile\(\)](#), [SetFileAttr\(\)](#)

FileLocking

Determines whether the editor locks files that are loaded.

[Values:](#) [INTEGER](#) - [_NONE_](#) disables file-locking. - [_READONLY_LOCKING_](#) enables Read-Only locking. - [_HANDLE_LOCKING_](#) enables Handle locking. [Default:](#) [_NONE_](#) Notes: - When FileLocking is set to [_READONLY_LOCKING_](#), the editor locks each file that is loaded by setting its read-only attribute. If the editor terminates abnormally (such as it would if you turn off or reboot your computer while in the middle of an editing session), the read-only attribute would not be removed from any files that were loaded in that session. In such [case](#), the read-only attribute would have to be removed manually. (The editor FileManager can be used to accomplish this.) - When FileLocking is set to [_HANDLE_LOCKING_](#), the editor locks each file by keeping its associated handle open until the file is closed (that is, removed from the editor). (Normally, the editor releases the handle associated with a file once the file is loaded into the editor.) Files locked in this manner can still be read by other programs, but write access will be denied to those programs. For more information on file-locking, see the chapter on "[Loading, Processing, and Saving Files](#)" in the User's Guide. [Affects:](#) [EditFile\(\)](#), [EditThisFile\(\)](#), and all commands that close files. See Also: Commands: [LockCurrentFile\(\)](#), [UnlockCurrentFile\(\)](#)

Files Needed By Iconfig

Iconfig is written in the editor's own macro programming language, SAL. To keep the size of the editor executable program itself as small as possible,

IConfig is provided as an external macro. For IConfig to be accessible from within the editor, the following file must be available:

ICONFIG.MAC	The IConfig Compiled Macro file
-------------	---------------------------------

When you invoke IConfig, the editor successively searches the following directories to locate this file:

1. The current directory
2. The directories indicated by the TSEPath variable, as well as the subdirectory MAC\ (if one exists) of each directory indicated by

[TSEPath](#) (For more information on the TSEPath variable, see "Path(s) for Supplemental Files" under "[System/File Options](#)" earlier in this chapter.) 3. The editor load directory, as well as the subdirectory MAC\ (if one exists) of the editor load directory (For more information on where the IConfig macro (ICONFIG.MAC) is placed when you install the editor, see the chapter on "[Installation of The SemWare Editor Professional](#).")

Files Needed By Spellcheck

To use SpellCheck, the following files must be available to the editor:

SPELLCHK.MAC	The SpellCheck Compiled Macro file
--------------	------------------------------------

SEMWARE.LEX	The standard SpellCheck word list
-------------	-----------------------------------

and, optionally:

USER.LEX	
----------	--

[The user-defined word list](#) When you invoke SpellCheck, the editor successively searches the following directories to locate these files: 1. The current directory 2. The directories indicated by the TSEPath variable; as well as the subdirectory MAC\ (if one exists) of each [TSEPath](#) directory to locate SPELLCHK.MAC, and the subdirectory SPELL\ (if one exists) of each TSEPath directory to locate the word-list (.LEX) files (For more information on the TSEPath variable, see "Path(s) for Supplemental Files" under "[System/File Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") 3. The editor load directory; as well as the subdirectory MAC\ (if one exists) of the editor load directory to locate SPELLCHK.MAC, and the subdirectory SPELL\ (if one exists) of the editor load directory to locate the word-list (.LEX) files (For more information on where the SpellCheck files are placed when you install the editor, see the chapter on "[Installation of The SemWare Editor Professional](#).")

FillBlock

Fills a marked block with the specified string.

Syntax: [INTEGER FillBlock](#)([\[STRING s\]](#)) - s is the optional string with which to fill the block. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If a marked block does not exist in the current file, no action occurs and zero is returned. If a fill string is not passed, [FillBlock](#)() will prompt for a fill [string](#). If <Escape> is pressed, or no

[string](#) is given before pressing [<Return>](#), zero is returned. [FillBlock\(\)](#) is useful for filling an area of existing text with spaces; thus accomplishing a deletion of text without surrounding text collapsing to "fill the gap", as will normally happen with block deletions. This command includes a history list of previous entries passed to it. See [AddHistoryStr\(\)](#) for additional information. [Examples](#): Fill a block with the string "Live long and prosper" [FillBlock](#)("Live long and prosper") See Also: [MarkColumn\(\)](#), [AddHistoryStr\(\)](#), [isBlockInCurrFile\(\)](#), [isBlockMarked\(\)](#) [Variables](#): [Marking](#)

Find

Searches for the specified text in the current file.

Syntax: [INTEGER Find](#)([\[STRING](#) search, [STRING](#) options]) - search is the optional string for which to search. - options is the optional string containing the Find options; multiple options may be specified in any combination. Supported [values](#) for options are: - b - search backwards from the current position. (The search begins one character before the current cursor position.) - l - limit the search to the marked block. - i - ignore differences between upper-[case](#) and lower-[case](#) characters. - w - limit the search to whole words. (See the WordSet variable for what constitutes a word.) - x - recognize regular expression characters in the search [string](#). - g - start the search at the beginning of the file. - c - limit the search to the current line. - a - search all files currently loaded in the editor's Ring of files. The current file is searched from the current position, and all other loaded files are searched from the beginning of each file. - v - display a PickList of all lines containing the search [string](#). - ^ - limit the search to the beginning of the line. - \$ - limit the search to the end of the line. - + - start the search at the next character, rather than starting at the current position. (This option is valid only with a forward search.) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Either both parameters must be passed, or no parameters can be passed. If no parameters are passed, [Find\(\)](#) prompts for the search string and [Find\(\)](#) options. In the prompt for options, [Find\(\)](#) initially uses the default options as specified by the FindOptions variable. Subsequent finds use the previously entered (or passed via a macro) options as the default for the prompt. Certain combinations of options affect each other, as follows: - If both b and g are specified, the search starts at the end of the file. - If both l and g are specified, the search starts at the beginning of the marked block. - If b, l, and g are specified together, the search starts at the end of the marked block. - If both c and g are specified, the search starts at the beginning of the current line. - If b, c, and g are specified together, the search starts at the end of the current line. - If both a and g are specified, all files (including the current file) are searched from the beginning of each file. Zero is returned if [<Escape>](#) is pressed at either prompt, or if the search string is not found. Macros can use the [lFind\(\)](#) command to search for text in a macro without the side effects of placing the search string and options string in the [Find\(\)](#) histories, and setting the [RepeatFind\(\)](#) command's execution strategy. This command includes a history list of previous entries passed to it. See [AddHistoryStr\(\)](#) for additional information. For more information on regular expressions, see the chapter on "[Search Features: Finding and Replacing Text](#)" in the User's Guide. [Examples](#): A [Find\(\)](#) that ALWAYS uses the current FindOptions variable setting: [integer proc](#) mFind([string](#) needle[40] [return](#) ([Ask](#)("Search for:", needle, _FIND_HISTORY_) and [Find](#)(needle, [Query](#)(FindOptions))) [end](#) The next example is a macro to find the next occurrence of the word at the current cursor position. It uses the GetWordAtCursor() macro presented in the [GetMarkedText\(\)](#) command example. Since [Find\(\)](#) normally starts at the current position, use the "Next" option (+) to force [Find\(\)](#) to begin its search starting at the next character position. (Otherwise, the [Find\(\)](#) would never move from its current position.) If there is not a word at the current position, the normal [Find\(\)](#) command is called, and the user will be prompted for a search [string](#). [integer proc](#) mFindWordAtCursor([string](#) word[80] word = GetWordAtCursor() [return](#) (iif([Length](#)(word), [Find](#)(word, [Query](#)(FindOptions) + "+"), [Find](#)())) [end](#) See Also: [Replace\(\)](#), [RepeatFind\(\)](#), [lFind\(\)](#), [lReplace\(\)](#), [lRepeatFind\(\)](#), [AddHistoryStr\(\)](#), [GetFoundText\(\)](#) [Variables](#): [Beep](#), [FindOptions](#), [MaxHistorySize](#), [WordSet](#)

FindFileClose

Closes a specified search handle that was previously opened using the

[FindFirstFile\(\)](#) command. This command is intended for ADVANCED USAGE. **Syntax:** [INTEGER FindFileClose](#) ([INTEGER](#) search_handle) - search_handle is a search handle returned by the [FindFirstFile\(\)](#) command. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: The [FindFileClose\(\)](#) command should always be called when search processing is complete following the [FindFirstFile\(\)](#) and [FindNextFile\(\)](#) commands. This will release that search handle to the operating system. **Examples:** See the [FindFirstFile\(\)](#) command for an example. See Also: [FindFirstFile\(\)](#), [FindNextFile\(\)](#), [FindThisFile\(\)](#), [FFName\(\)](#), [FFSize\(\)](#), [FFDate\(\)](#), [FFDateStr\(\)](#), [FFTime\(\)](#), [FFTimeStr\(\)](#), [FFAttribute\(\)](#)

FindFirstFile

Searches a directory for the first file or directory whose name and attributes match the specified name pattern and attributes; and opens a search handle (that can be used for subsequent searches).

This command is intended for ADVANCED USAGE.

Syntax: [INTEGER FindFirstFile](#)([STRING](#) filename, [INTEGER](#) attributes) - filename is the filename pattern for which to search. - attributes are the optional attributes to use when searching. attributes may be a combination of the following: - [_NORMAL_](#) specifies a file that can be read from or written to. - [_READONLY_](#) specifies a file that can be read from but not written to. - [_HIDDEN_](#) specifies a file that is hidden and does not appear in a file directory listing. - [_SYSTEM_](#) specifies a file that is a system file. - [_VOLUME_](#) specifies a volume label entry. - [_DIRECTORY_](#) specifies a directory entry. - [_ARCHIVE_](#) specifies a file that is marked for archiving. Returns: The search handle of the first file or directory that matches filename, if successful; -1 on failure. Notes: The search handle that is returned by the [FindFirstFile\(\)](#) command is used by the commands [FindNextFile\(\)](#) and [FindFileClose\(\)](#). The [FindFileClose\(\)](#) command should always be called when search processing is complete following the [FindFirstFile\(\)](#) and [FindNextFile\(\)](#) commands. To retrieve information about the found file, use the FF-type commands, such as [FFName\(\)](#) and [FFDate\(\)](#). This command is intended for use with an ambiguous file specification. For a similar command intended for finding a unique file, and that does not require you to close the search handle, see the [FindThisFile\(\)](#) command. If you specify any combination of [_SYSTEM_](#), [_HIDDEN_](#), or [_DIRECTORY_](#), this command returns normal files in addition to files with the specified attribute(s). If you want to work only with files with the specified attribute(s), you must examine the specific attribute for each file, via the [FFAttribute\(\)](#) command. To search for all files, an attribute of -1 can be used. **Examples:** // Display all files that have an extension of '.c' [integer](#) handle = [FindFirstFile](#)("*.c", -1) //allocate search handle //search for ALL files [if](#) handle <> -1 [repeat](#) [WriteLine](#) ([FFName\(\)](#)) [until not](#) [FindNextFile](#)(handle, -1) //get next file [FindFileClose](#)(handle) //release search handle [endif](#) See Also: [FindNextFile\(\)](#), [FindFileClose\(\)](#), [FindThisFile\(\)](#), [FFName\(\)](#), [FFSize\(\)](#), [FFDate\(\)](#), [FFDateStr\(\)](#), [FFTime\(\)](#), [FFTimeStr\(\)](#), [FFAttribute\(\)](#), [AddFFInfoToBuffer\(\)](#)

FindHistoryStr

Locates the relative position of a specified item (string) in the History Buffer.

Syntax: [INTEGER FindHistoryStr](#)([STRING](#) history_str, [INTEGER](#) history_no) - history_str is the string for which to search in the History Buffer. - history_no is an integer specifying the history list in which to search. Returns: The

relative position of the located string; otherwise, zero ([FALSE](#)) if [not](#) located. Notes: The return value is the number, within the specified history list, of the located history item. It will be in the range 1 through the value returned by [NumHistoryItems](#)(history_no), where 1 represents the item added most recently, and higher [numbers](#) represent older items. The return value of this command can be used as input to the [DelHistoryStr](#)() and [GetHistoryStr](#)() commands. A [case](#)-sensitive search is performed. Contrast this command with the [GetHistoryStr](#)() command, which returns the history string itself. [Examples](#): // locate "myfile" history item in the EditFile history `integer i i = FindHistoryStr("myfile", _EDIT_HISTORY_)` See Also: [GetHistoryStr](#)(), [NumHistoryItems](#)(), [GetFreeHistory](#)(), [AddHistoryStr](#)(), [DelHistoryStr](#)() [Variables](#): [MaxHistoryPerList](#), [PersistentHistory](#)

FindNextFile

Continues the search for the next file or directory associated with the specified search handle, and with the specified attributes.

This command is intended for ADVANCED USAGE.

[Syntax](#): [INTEGER FindNextFile](#)([INTEGER](#) search_handle, [INTEGER](#) attributes) - search_handle is a search handle returned by the [FindFirstFile](#)() command. - attributes are the optional attributes to use when searching. attributes may be a combination of the following: - [_NORMAL_](#) specifies a file that can be read from or written to. - [_READONLY_](#) specifies a file that can be read from but not written to. - [_HIDDEN_](#) specifies a file that is hidden and does not appear in a file directory listing. - [_SYSTEM_](#) specifies a file that is a system file. - [_VOLUME_](#) specifies a volume label entry. - [_DIRECTORY_](#) specifies a directory entry. - [_ARCHIVE_](#) specifies a file that is marked for archiving. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command is used following the [FindFirstFile](#)() command. To retrieve information about the found file, use the FF-type commands, such as [FFName](#)() and [FFDate](#)(). When [FindNextFile](#)() processing is complete for a particular search handle, the [FindFileClose](#)() command should be called to close that search handle. If you specify any combination of [_SYSTEM_](#), [_HIDDEN_](#), or [_DIRECTORY_](#), this command returns normal files in addition to files with the specified attribute(s). If you want to work only with files with the specified attribute(s), you must examine the specific attribute for each file, via the [FFAttribute](#)() command. The same attribute used in the [FindFirstFile](#)() command should be used with this command. To search for all files, an attribute of -1 can be used. [Examples](#): See the [FindFirstFile](#)() command for an example. See Also: [FindFirstFile](#)(), [FindFileClose](#)(), [FindThisFile](#)(), [FFName](#)(), [FFSize](#)(), [FFDate](#)(), [FFDateStr](#)(), [FFTime](#)(), [FFTimeStr](#)(), [FFAttribute](#)()

FindOptions

Defines the options for the initial execution of

[Find](#)(). [Values](#): [STRING](#) (maximum of 11 characters) - B - search backwards - G - global search - C - search current line - L - local search - I - ignore [case](#) - W - whole words only - X - regular [expressions](#) - ^ - tie [string](#) to beginning of line - \$ - tie [string](#) to end of line - + - start search at the next character [Default](#): "I" [Limits](#): Has effect at startup only. Notes: FindOptions is only effective the first time the [Find](#)() command is executed. On subsequent executions, the option string from the previous execution is used. See the [Find](#)() command for detailed explanations of each option. [Affects](#): [Find](#)() See Also: [ReplaceOptions](#)

FindThisFile

Searches a directory for the first file or directory whose name and attributes match the specified name and attributes.

This command is intended for ADVANCED USAGE.

Syntax: [INTEGER FindThisFile](#)([STRING](#) filename [, [INTEGER](#) attributes]) - filename is the file for which to search. - attributes are the optional attributes to use when searching. attributes may be a combination of the following: - [_NORMAL_](#) specifies a file that can be read from or written to. - [_READONLY_](#) specifies a file that can be read from but not written to. - [_HIDDEN_](#) specifies a file that is hidden and does not appear in a file directory listing. - [_SYSTEM_](#) specifies a file that is a system file. - [_VOLUME_](#) specifies a volume label entry. - [_DIRECTORY_](#) specifies a directory entry. - [_ARCHIVE_](#) specifies a file that is marked for archiving. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command is a specialized version of the [FindFirstFile\(\)](#) command. It is intended for use when you are searching for a single file. It saves you the overhead of opening and then closing a search handle. **Examples:** [if FindThisFile](#)("foo.bar") [Message](#)("file found") [else Message](#)("file [not](#) found") [endif](#) See Also: [FindFirstFile\(\)](#), [FindFileClose\(\)](#), [FindNextFile\(\)](#), [FFName\(\)](#), [FFSize\(\)](#), [FFDate\(\)](#), [FFDateStr\(\)](#), [FFTime\(\)](#), [FFTimeStr\(\)](#), [FFAttribute\(\)](#), [AddFFInfoToBuffer\(\)](#)

Flip

Toggles alphabetic characters between upper and lower

case. **Syntax:** [STRING Flip](#)([STRING](#) s) - s is an optional string to convert. or: [Flip\(\)](#) Returns: If the first Syntax format is used, the resulting flipped string is returned; [otherwise](#), nothing is returned. Notes: [Flip\(\)](#) "flips" the target characters: all alphabetic characters (a to z and A to Z) in the string are "flipped" (that is, all lower-[case](#) characters become upper-[case](#), and vice versa). Other characters are [not](#) changed. If the first Syntax format is used, s is flipped. If the second Syntax format is used: if the cursor is in a marked block, the characters in the block are flipped; otherwise, the character at the cursor position is flipped. **Examples:** Given: [string](#) s[3], = "aBc", s2[3] then: s2 = [Flip](#)(s) // s2 is "AbC" s2 = [Flip](#)("Def") // s2 is "dEF" s2 = [Flip](#)(s2) // s2 is "Def" See Also: [Upper\(\)](#), [Lower\(\)](#)

FlushProfile

The OS may keep a cached version of an initialization file. This command will signal the OS to flush the cache for that initialization file.

Syntax: [FlushProfile](#)([[STRING](#) profile_name]) - profile_name is an optional initialization file's name. If not specified, the tse.ini file, located in the editor's load directory, is assumed. If profile_name is specified, and does not include a path, the profile_name that resides in the WINDOWS directory (specified by the windir environment variable) is assumed. Returns: Nothing. Notes: The OS does [not return](#) any information as to whether this operation succeeds or fails. **Examples:** // The following displays all the items in a section of the // tse.ini file if that section exists, [otherwise](#) that section, // and two items for that section, are added to the tse.ini file. // These added items are displayed, then one item is removed. // The remaining item is displayed, then the section is removed. [proc main\(\)](#) [string](#) item[255] = "" [string](#) value[255] = "" [integer](#) counter = 0 [FlushProfile](#)() [if LoadProfileSection](#)("\$\$test\$\$") [while GetNextProfileItem](#)(item, value) counter = counter + 1 [Warn](#)("Item ", [Str](#)(counter), " is ", item, " which is equal to ", value, " ...") [endwhile](#) [Warn](#)("There were ", counter, " items. There are no more original items ...") [else WriteProfileInt](#)("\$\$test\$\$", "First Item", 200) [WriteProfileStr](#)("\$\$test\$\$", "Second Item", "Two Hundred") [if LoadProfileSection](#)("\$\$test\$\$") [while GetNextProfileItem](#)(item, value) counter = counter + 1 [Warn](#)("Item ", [Str](#)(counter), " is ", item, " which is equal to ", value, " ...") [endwhile](#) [Warn](#)("There were ", counter, " items. There are no more items ...") [RemoveProfileItem](#)("\$\$test\$\$", "Second Item") [if LoadProfileSection](#)("\$\$test\$\$") counter = 0 [while GetNextProfileItem](#)(item, value) counter =

```

counter + 1 Warn("Item ", Str(counter), " is ", item, " which is equal to ", value, " ...") endwhile Warn("There were ",
counter, " items. There are no more items ...") endif RemoveProfileSection("$$test$$") if LoadProfileSection("$$test$
$") counter = 0 while GetNextProfileItem(item, value) counter = counter + 1 Warn("Item ", Str(counter), " is ", item, "
which is equal to ", value, " ...") endwhile Warn("There were ", counter, " items. There are no more items ...") else
Warn("$$test$$ section not found") endif endif endif end See Also: GetProfileInt(), GetProfileStr(), WriteProfileInt(),
WriteProfileStr(), LoadProfileSection(), GetNextProfileItem(), LoadProfileSectionNames(),
GetNextProfileSectionName(), RemoveProfileItem(), RemoveProfileSection()

```

FontFlags

Determines the initial font attributes.

This variable has no effect in the Console version of the editor.

Values: [INTEGER](#) [_FONT_BOLD](#) specifies a bold font. [_FONT_OEM](#) specifies a font that uses the OEM character set. **Default:** 0 - normal weight, ANSI character set. Limits: Must be a combination of the above specified values.

Notes: It is [not](#) necessary to explicitly set this variable. **Affects:** The editor's display. See Also: [FontName](#), [FontSize](#)

Commands: [SetFont](#)(), [ChooseFont](#)(), [GetFont](#)(), [ResizeFont](#)() [GetCharWidthHeight](#)()

FontName

Determines the initial font.

This variable has no effect in the Console version of the editor.

Values: [STRING](#) (maximum of 32 characters) **Default:** "Courier New" Limits: Must be an available Windows font.

Notes: It is [not](#) necessary to explicitly set this variable. **Affects:** The editor's display. See Also: [FontFlags](#), [FontSize](#)

Commands: [SetFont](#)(), [ChooseFont](#)(), [GetFont](#)(), [ResizeFont](#)() [GetCharWidthHeight](#)()

FontSize

Determines the initial font size.

This variable has no effect in the Console version of the editor.

Values: [INTEGER](#) **Default:** 10 Limits: Must be a valid font size. Notes: If FontSize is <= 255, it is assumed to be a point size, and is applied appropriately. If [FontSize](#) is > 255, it is assumed to be a specific character width and height.

For instance: FontSize 10 specifies a point size of 10. FontSize 0x080c specifies a width of 8, and a height of 0x0c, or 12. It is [not](#) necessary to explicitly set this variable. **Affects:** The editor's display. See Also: [FontName](#), [FontFlags](#)

Commands: [SetFont](#)(), [ChooseFont](#)(), [GetFont](#)(), [ResizeFont](#)() [GetCharWidthHeight](#)()

fOpen

Low-level file-access command: opens a specified file and returns the associated handle.

This command is intended for ADVANCED USAGE.

Syntax: [INTEGER](#) [fOpen](#)([STRING](#) filename [, [INTEGER](#) modes]) - filename is the name of the file to open. - modes are optional modes to use when opening the file. modes may be a combination of the following: - [_OPEN_READONLY_](#) opens the file for read-only access. - [_OPEN_WRITEONLY_](#) opens the file for write-only access. - [_OPEN_READWRITE_](#) opens the file for read-and-write access. - [_OPEN_COMPATIBILITY_](#) permits other programs any access to the file. - [_OPEN_DENYALL_](#) prevents any other program from opening the file. - [_OPEN_DENYWRITE_](#) prevents any other program from opening the file for write access. - [_OPEN_DENYREAD_](#) prevents any other program from opening the file for read access. - [_OPEN_DENYNONE_](#) permits other programs to open the file for read or write access. - [_OPEN_NOINHERIT_](#) prevents any child programs from inheriting the file handle. If [not](#) specified, modes is set to: [_OPEN_READONLY_](#) | [_OPEN_COMPATIBILITY_](#) Returns: The handle, as assigned by the operating system, of the newly-opened file; or -1, if the file could not be opened. Notes: This command opens an existing file. If the file does not exist, or the file could not be accessed, the command fails, and -1 is returned. **Examples:** See the [fReadFile](#)() command. See Also: [fClose](#)(), [fCreate](#)(), [fDup](#)(), [fDup2](#)(), [fRead](#)(), [fReadFile](#)(), [fSeek](#)(), [fWrite](#)(), [fWriteFile](#)()

Format

Formats the specified expression(s) into a single string.

Syntax: [STRING](#) [Format](#)([[STRING](#)] | [[INTEGER](#)] expr [: [INTEGER](#) width [: [STRING](#) pad_char [: [INTEGER](#) base]]] [, [[STRING](#)] | [[INTEGER](#)] expr [: [INTEGER](#) width [: [STRING](#) pad_char [: [INTEGER](#) base]]] ...) - expr represents each expression. Multiple [expressions](#) must be separated by commas. [Expressions](#) can be of type [integer](#) or [string](#). - width is the optional format width. A negative value will left justify the expression, whereas a positive value will right justify the expression. - pad_char is the optional character that is used to pad the expression to fit the specified width. The space character is used if this character is [not](#) specified. - base is the optional base to use when converting numeric [expressions](#). Base 10 is used if this value is [not](#) specified. Returns: A string consisting of the concatenation of the passed expressions, formatted to the optional width, justification, and number base. Notes: Specifying any [optional parameters](#) requires specifying all [optional parameters](#) which precede it. Thus, to specify base requires that width and pad_char be specified. If a numeric expression yields a negative result and the expression is to be right justified (width is a positive value), then the pad_char character is placed between the negative sign "-" (which is left justified) and the actual number. base can only be specified for [numeric expressions](#). If an expression will not fit within the specified width, the expression is truncated to width less 1, and the ">" character is placed in the last position. [Warn](#)(), [Message](#)(), [Write](#)(), and [WriteLine](#)() use the [Format](#)() [syntax](#) as well. **Examples:** [Format](#)("abc":5) // returns " abc" [Format](#)("abc":5:"*") // returns "***abc" [Format](#)("abc":-5:"*") // returns "abc***" [Format](#)(7:5) // returns " 7" [Format](#)(7:-5) // returns "7 " [Format](#)(7:5:"0") // returns "00007" [Format](#)(-7:5:"0") // returns "-0007" // returns "7 base 2 is 00111" [Format](#)(7," base 2 is ",7:5:"0":2) [Format](#)(32:-5) // returns "32 " // Assume i = 73 [Format](#)(i:4) // returns " 73" [Format](#)(i) // returns "73" [Format](#)(i:-4) // returns "73 " // Assume [string](#) name[] = "Bobbi", [integer](#) age = 39 [Format](#)("My name is ", name, " [and](#) I am ", age, " years old") // Returns: "My name is Bobbi and I am 39 years old" [Format](#)("This [string](#) is too wide":5) returns "This>" // initialize "s" to 32 spaces [string](#) s[32] = [Format](#)(" ":32) See Also: [Warn](#)(), [Message](#)(), [Write](#)(), [WriteLine](#)()

Formatting Print Output

A number of options are available for specifying the page format of printed

text and controlling the print output, including the ability to:

- specify left and right print margins (

[PrintLeftMargin](#), [PrintRightMargin](#)) - indicate the number of lines available on a page, and specify top and bottom print margins ([PrintLinesPerPage](#), [PrintTopMargin](#), [PrintBotMargin](#)) - select the line-spacing so that you can print a document single-spaced, double-spaced, triple-spaced, etc. ([PrintLineSpacing](#)) - include line numbers in the printed output ([PrintLineNumbers](#)) - designate certain information to be printed as a header and/or footer on each page ([PrintHeader](#), [PrintFooter](#)) - select a beginning and/or ending page within a file or block to print ([PrintFirstPage](#), [PrintLastPage](#)) - print multiple copies of a file or block ([PrintCopies](#)) - suspend printing between pages to allow manual feeding of paper ([PrintPause](#))

fRead

Low-level file-access command: reads selected data from a disk file (specified by its handle) into a string variable.

This command is intended for ADVANCED USAGE.

Syntax: [INTEGER](#) [fRead](#)([INTEGER](#) handle, [VAR STRING](#) data, [INTEGER](#) n) - handle is the handle, as assigned by the operating system, of the file from which to read. - data is the string in which to store the data read from the specified file. - n is the number of bytes to read from the specified file. n must be less than or equal to the maximum length of data. Returns: The number of bytes read; or -1, on error. Notes: [fRead](#)() increments the current position in the file by the number of bytes read. The current position can be queried and set by [fSeek](#)(). Before calling this command, the file must have been previously opened by [fOpen](#)() or [fCreate](#)(). Note that these two "open" commands return the handle of the file. **Examples:** // Read the first 10 bytes from the file "foo.bar" [integer](#) h, n [string](#) s[10] = " h = [fOpen](#) ("foo.bar") [if](#) h <> - 1 n = [fRead](#)(h, s, 10) [endif](#) [fClose](#)(h) See Also: [fClose](#)(), [fCreate](#)(), [fOpen](#)(), [fReadFile](#)(), [fSeek](#)(), [fWrite](#)(), [fWriteFile](#)()

fReadFile

Low-level file-access command: loads a file, specified by its handle, into the current editing buffer.

This command is intended for ADVANCED USAGE.

Syntax: [INTEGER](#) [fReadFile](#)([INTEGER](#) handle) - handle is the handle, as assigned by the operating system, of the file to read. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Before calling this command, the file must have been previously opened by [fOpen](#)() or [fCreate](#)(). Note that these two "open" commands return the handle of the file. Loading of the file contents begins at the current position in the file. The [fSeek](#)() command can be used to change the current position before executing this command. Thus, the file can be loaded starting at any position. **Examples:** The following macro will load standard input into a buffer, if standard input has been piped to a file. By adding this to your ".UI" WhenLoaded() procedure, you can start the editor as follows: dir | e32 // Load stdin as a file [string](#) CONSOLE[] = "CONIN\$" [proc](#) LoadStdin() [integer](#) new_handle [if](#) [isCharDevice](#)(_STDIN_) == [FALSE](#) and [CreateBuffer](#)("") [fReadFile](#)(_STDIN_) [BegFile](#)() [ChangeCurrFilename](#)("[<stdin>]", _DONT_EXPAND_) new_handle = [fOpen](#)(CONSOLE, _OPEN_READWRITE_) [fDup2](#)(new_handle, _STDIN_) [fClose](#)(new_handle) [endif](#) [end](#)

Emulate the built-in [EditFile\(\)](#) command. We assume the filename that is passed does exist, and no errors are encountered. [proc](#) mLoadFile([string](#) fn) [integer](#) h, ok ok = [FALSE](#) [if](#) [CreateBuffer](#)(fn) // create buffer h = [fOpen](#)(fn, [_OPEN_DENYNONE](#)) // open the file [if](#) h <> -1 ok = [fReadFile](#)(handle) // load it in [fClose](#)(handle) [endif](#) [BegFile](#)() [endif](#) [return](#) (ok) [end](#) See Also: [fClose](#)(), [fCreate](#)(), [fOpen](#)(), [fRead](#)(), [fSeek](#)(), [fWrite](#)(), [fWriteFile](#)()

fSeek

Low-level file-access command: moves the read/write pointer for a disk file (specified by its handle) to a selected position.

This command is intended for ADVANCED USAGE.

Syntax: [INTEGER](#) [fSeek](#)([INTEGER](#) handle, [INTEGER](#) offset, [INTEGER](#) method) - handle is the handle, as assigned by the operating system, of the file for which to position the read/write pointer. - offset is the relative position at which to place the file pointer. - method indicates the position from which to move the pointer, by the offset value. Supported values for method are: - [_SEEK_BEGIN](#) specifies the beginning of the file. - [_SEEK_CURRENT](#) specifies the current file position. - [_SEEK_END](#) specifies the end of the file. Returns: The new position in the file; or -1, on failure. Notes: Before calling this command, the file must have been previously opened by [fOpen](#)() or [fCreate](#)(). Note that these two "open" commands return the handle of the file. **Examples:** // seek to the end of the file associated with handle h [fSeek](#)(h, 0, [_SEEK_END](#)) See Also: [fClose](#)(), [fCreate](#)(), [fOpen](#)(), [fRead](#)(), [fReadFile](#)(), [fWrite](#)(), [fWriteFile](#)()

FullWindow

A video output command that resets the video window system to full screen.

Syntax: [FullWindow](#)() Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. Before terminating, [FullWindow](#)() should be called by any [macros](#) that change the video output window using the [Window](#)() command. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [Window](#)()

Fully Customizing the Editor

A great deal of effort has been spent in the development of the editor to make it customizable. You can make customizations from within the editor by selecting editing options from the interactive pull-down Menu system, and by creating Keyboard Macros and editing templates. Using the editor's macro language, SAL (the SemWare Applications Language), you can further customize the editor by changing key assignments for commands or macros, writing entirely new macros to add your own commands, or re-designing the editor's user interface.

This chapter presents an overview of many of the customization capabilities, from

[simple customization from within the editor](#), to extensive customization [using macros](#). The topics covered are: - [Simple Customization from Within the Editor](#): - [Changing settings for editing options using IConfig](#) - [Creating](#)

[Keyboard Macros](#) - [Creating templates/abbreviations](#) - [Customization using SAL Compiled Macros](#): - [Creating an external Compiled Macro](#) - [Adding an external macro to the Potpourri](#) - [Maintaining a macro autoload file](#) - [Creating a TSEStart macro file](#) - [Understanding user-interface \(UI\) macro files](#) - [Installing a user-interface \(UI\) macro file](#) - [Adding or changing key assignments](#) - [Changing the HelpLines](#) - [Modifying the key-assignment Help screen](#) - [Making changes to the pull-down Menu](#) - [Adding macros to the user-interface \(UI\) macro file](#) - [Creating a custom user-interface \(UI\) macro file](#) These topics are covered in more detail throughout [the User's Guide](#) and the Macro Reference Guide.

fWrite

Low-level file-access command: writes data from a string to a disk file (specified by its handle).

This command is intended for ADVANCED USAGE.

Syntax: [INTEGER](#) [fWrite](#)([INTEGER](#) handle, [STRING](#) data) - handle is the handle, as assigned by the operating system, of the file to which to write. - data is the string to be written to the specified file. Returns: The number of bytes written; or -1, on error. Notes: Before calling this command, the file must have been previously opened, with write access, by [fOpen\(\)](#) or [fCreate\(\)](#). Note that these two "open" commands return the handle of the file. **Examples:** // Write "Hello, World!" to a file called "hello" // Assume no errors [integer](#) h h = [fCreate](#)("hello") [fWrite](#)(h, "Hello, World!") [fClose](#)(h) See Also: [fClose\(\)](#), [fCreate\(\)](#), [fOpen\(\)](#), [fRead\(\)](#), [fReadFile\(\)](#), [fSeek\(\)](#), [fWriteFile\(\)](#)

fWriteFile

Low-level file-access command: writes the contents of the current buffer to a disk file (specified by its handle).

This command is intended for ADVANCED USAGE.

Syntax: [INTEGER](#) [fWriteFile](#)([INTEGER](#) handle [, [INTEGER](#) flags]) - handle is the handle, as assigned by the operating system, of the file to which to write. - flags is an optional value that affects the operation of [fWriteFile\(\)](#). Supported [values](#) for flags are: - [_DONT_PROMPT_](#) disables displaying of informational and error messages. - [_APPEND_](#) causes the file to be appended to an existing file. - [_OVERWRITE_](#) causes the file to overwrite any existing file. - [_USEBLOCK_](#) indicates that only the marked block is to be written. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Before calling this command, the file must have been previously opened, with write access, by [fOpen\(\)](#) or [fCreate\(\)](#). Note that these two "open" commands return the handle of the file. **Examples:** // emulate the built-in [SaveFile\(\)](#) command [proc](#) mSaveFile() [integer](#) h, ok ok = [FALSE](#) h = [fCreate](#)([CurrFilename\(\)](#)) [if](#) h <> -1 ok = [fWriteFile](#)(h) [fClose](#)(h) [endif](#) [return](#) (ok) [end](#) See Also: [fClose\(\)](#), [fCreate\(\)](#), [fOpen\(\)](#), [fRead\(\)](#), [fReadFile\(\)](#), [fSeek\(\)](#), [fWrite\(\)](#)

General Use of the Mouse

To initiate an action with the mouse, position the mouse pointer at the appropriate place on the screen, and then press one of the buttons on the mouse (either the <LeftBtn> or <RightBtn>), as indicated by the mouse command assignments. Depending on the desired action, you either "click," "press and

hold," or "drag" the mouse, as follows:

- To click a mouse button, press and immediately release the button.

For certain commands, you execute the command one time only by clicking the mouse button. For some commands, you must click the indicated mouse button two or three times, as instructed for that command. For multiple clicks, the mouse pointer must remain at the same location while the button is repeatedly clicked the specified number of times.

- To press and hold a mouse button, press and hold the button for a period of time before releasing it. The amount of time that the button must be held is determined by the MouseHoldTime variable.

For commands that are invoked by clicking a mouse button, a command can be executed multiple times by pressing and holding the button. The command is executed repeatedly until the mouse button is released. The value assigned to the MouseRepeatDelay variable determines the number of times that a command is repeated while the button is held.

(For more information on the variables MouseHoldTime and

[MouseRepeatDelay](#), see "Mouse Hold Time" [and](#) "Mouse Repeat Delay" under "[Keyboard/Mouse Options](#)" in the chapter on "[Configuring the Editor Interactively](#)." - To drag the mouse, press the indicated mouse button, and continue to hold down the button as you drag the mouse pointer to another location on the screen. Do not release the button until the mouse pointer is positioned at the desired location. The action caused by the mouse is determined by the location of the mouse pointer when a mouse button is pressed: - On [the StatusLine](#) or HelpLine, clicking the <LeftBtn> activates the pull-down [Menu](#). - Within the main Menu, pop-up selection menus, StatusLine prompts (including "warning" messages), prompt boxes, PickLists, IConfig, and [the HelpSystem](#), clicking the <LeftBtn> is the same as pressing <Enter>, and clicking the <RightBtn> is the same as pressing <Escape>. For entries in the main Menu, pop-up selection menus, PickLists, IConfig, and [the HelpSystem](#), the position of the mouse pointer determines the entry that is selected when you click the <LeftBtn>. - Within the text portion of a window, various operations with the <LeftBtn> are used to position the editing cursor or to mark a block. (A few of the block-marking commands are executed by pressing the <LeftBtn> in combination with the <Ctrl> or <Alt> keys on the keyboard.) - Anywhere in a window while in editing mode (when no menus, prompts, or PickLists are active), clicking the <RightBtn> pops up the [Mouse selection menu](#). - In a window border, the <LeftBtn> is used to scroll the text, or to create, resize, or change [windows](#). The special mouse symbols and scrollbars ("hot spots") in the window borders are used to execute specific scrolling or window commands. The remainder of this chapter describes how to perform specific tasks using the mouse.

GetBit

Returns the setting of the specified bit.

Syntax: [INTEGER GetBit](#)([STRING](#) bitset, [INTEGER](#) bit) - bitset is the bit-mapped string for which to return the setting of a particular bit. - bit is the specific bit number for which to return the setting. Returns: The setting of the specified bit (either 0 or non-zero) within the specified bitset. Notes: Bit numbering starts at 0. The editor uses bit-mapped strings for the WordSet variable and the [TabSet](#)() command. [SetBit](#)(), [ClearBit](#)(), and [GetBit](#)() can be used to

modify these [values](#). [Examples](#): Suppose you want to know if the "_" character is within the current [WordSet](#): `string word_set[32] integer is_word_set_char word_set = Query(WordSet) // if underscore is in the WordSet, is_word_set_char is 1 // (or TRUE). If not, is_word_set_char is 0 (or FALSE). is_word_set_char = GetBit(word_set, ASC('_')) See the WordSet variable for additional information. See Also: ClearBit(), SetBit()`

GetBookMarkInfo

Returns information about the location of a specified bookmark.

Syntax: [INTEGER](#) [GetBookMarkInfo](#)([STRING](#) bookmark, [VAR INTEGER](#) id, [VAR INTEGER](#) line, [VAR INTEGER](#) pos, [VAR INTEGER](#) xoffset, [VAR INTEGER](#) row) - bookmark is the single-character name (A through Z, case-insensitive) that identifies the bookmark. - id is the id number of the buffer where the bookmark is located. - line is the number of the line in the file on which the bookmark is located. - pos identifies the position of the bookmark on the line. - xoffset indicates the number of columns by which the screen was scrolled when the bookmark was placed. - row indicates the screen row on which the bookmark was placed. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command allows you to save the location of a bookmark so that it can be restored at a later time. If this command fails, the values of the bookmark parameters are undefined. [isBookMarkSet](#)() should always be used prior to [GetBookMarkInfo](#)() to verify that a particular bookmark exists. **Examples:** // Save bookmark info to a buffer `integer proc mSaveBookMarkInfo() integer i, id, line, cpos, xoff, row, curr_id = GetBufferId(), state_buffer = CreateTempBuffer() for i = Asc('a') to Asc('z') if isBookMarkSet(Chr(i)) GetBookMarkInfo(Chr(i), id, line, cpos, xoff, row) if GotoBufferId(id) AddLine(Format(Chr(i); line; row; cpos; xoff; CurrFilename()), state_buffer) endif endif endfor GotoBufferId(curr_id) return (state_buffer) end // Restore saved bookmarks proc mRestoreBookMarkInfo (integer state_buffer) integer curr_id = GotoBufferId(state_buffer) string s[128] BegFile() if CurrLineLen() repeat s = GetText(1, CurrLineLen()) EditFile(GetToken(s, ' ', 6), _DONT_PROMPT_) PushPosition() GotoLine(Val(GetToken(s, ' ', 2))) ScrollToRow(Val(GetToken(s, ' ', 3))) GotoPos(Val(GetToken(s, ' ', 4))) GotoXOffset(Val(GetToken(s, ' ', 5))) PlaceMark(GetToken(s, ' ', 1)) PopPosition() GotoBufferId(state_buffer) until not Down() endif GotoBufferId(curr_id) end See Also: PlaceMark(), GotoMark(), DelBookMark()`

GetBufferDaTmAttr

Retrieves the date, time, size and attribute fields for the current buffer, as set via the last

[UpdateBufferDaTmAttr](#)() command. **Syntax:** [GetBufferDaTmAttr](#)([VAR INTEGER](#) f_date, [VAR INTEGER](#) f_time, [VAR INTEGER](#) f_attributes, [VAR INTEGER](#) f_size) - f_date is the file's date in encoded format. - f_time is the file's time in encoded format. - f_attributes is the file's attributes. See [FFAttribute](#)() for a detailed listing of available attributes. - f_size is the size of the file on disk. Returns: Nothing. Notes: For each file loaded into the editor, the editor maintains a record of the file's date, time, size, and file attributes, as of the time the file was loaded into the editor, or the last time it was saved. This command can be used to retrieve some of those [values](#). See Also: [ClearBufferDaTmAttr](#)(), [UpdateBufferDaTmAttr](#)()

GetBufferId

Returns the unique id number of the requested or current buffer.

Syntax: [INTEGER GetBufferId](#)([[STRING](#) name]) - name is an optional buffer name. Returns: If name is passed, and is that of a buffer currently in the editor's Ring of files, the non-zero id number of that buffer is returned. If name is [not](#) found, zero ([FALSE](#)) is returned. If name is not passed, the non-zero id number of the current buffer is returned.

Examples: `i = GetBufferId()` // returns the id of the current // buffer `i = GetBufferId("foo")` // returns the id of buffer "foo" // if it exists; [FALSE](#) if [not](#). `i = GetBufferId("c:\tmp\foo")` // returns the id of // buffer "c:\tmp\foo", // which is not the same // as the previous // example. See the [AbandonEditor](#)(), [CreateBuffer](#)(), and [NumFiles](#)() commands [for](#) additional [examples](#). See Also: [GotoBufferId](#)(), [BufferType](#)()

GetBufferInt

Retrieves the value of the specified buffer variable, as set via

[SetBufferInt](#)(). **Syntax:** [INTEGER GetBufferInt](#)([STRING](#) buffer_var_name [, [INTEGER](#) buffer_id]) - buffer_var_name is the name of the buffer variable to find. - buffer_id is an optional buffer id, as returned by [GetBufferId](#)(). Returns: The value of buffer_var_name if buffer_var_name exists; [otherwise](#), zero ([FALSE](#)). Notes: If buffer_id is not specified, the current buffer is assumed. See Also: [GetBufferStr](#)(), [SetBufferInt](#)(), [SetBufferStr](#)(), [ExistBufferVar](#)(), [DelBufferVar](#)()

GetBufferStr

Retrieves the value of the specified buffer variable, as set via

[SetBufferStr](#)(). **Syntax:** [STRING GetBufferStr](#)([STRING](#) buffer_var_name [, [INTEGER](#) buffer_id]) - buffer_var_name is the name of the buffer variable to find. - buffer_id is an optional buffer id, as returned by [GetBufferId](#)(). Returns: The value of buffer_var_name if buffer_var_name exists; otherwise, the empty ("") string. Notes: If buffer_id is not specified, the current buffer is assumed. See Also: [GetBufferInt](#)(), [SetBufferStr](#)(), [SetBufferInt](#)(), [ExistBufferVar](#)(), [DelBufferVar](#)()

GetCharWidthHeight

Retrieves the width and height of the font that is being used for the editing display.

This command has no effect in the Console version of the editor.

Syntax: [GetCharWidthHeight](#)([VAR INTEGER](#) width, [VAR INTEGER](#) height) - width is the character width in pixels. - height is the character height in pixels. Returns: This command does not have a return value, but rather, returns information through the passed VAR parameters. Notes: The character width and height in pixels are passed back through the VAR variables. This command can be used to find out the pixel width and height of the currently selected font. Contrast this with the [GetFont](#)() command, which returns the point size of the current font. **Examples:** `integer width, height` [GetCharWidthHeight](#)(width, height) // width and height are now set appropriately See Also: [GetFont](#)(), [SetFont](#)(), [ChooseFont](#)(), [ResizeFont](#)()

GetClipboardBlockType

Returns the block type of the text in a Clipboard.

Syntax: [INTEGER GetClipboardBlockType](#)([INTEGER](#) id) - id is the optional id of a Clipboard in which to check the block type. If not specified, the current Clipboard will be used. Returns: Zero (FALSE) if the Clipboard does not contain a block; otherwise, the return value is one of the following: - [_INCLUSIVE_](#) for an inclusive character block - [_NONINCLUSIVE_](#) for a non-inclusive character block - [_LINE_](#) for a line block - [_COLUMN_](#) for a column block
See Also: [isBlockMarked\(\)](#), [isBlockInCurrFile\(\)](#) **Variables:** [BlockId](#)

GetClockTicks

Returns the number of system clock ticks since the editing session was started.

Syntax: [INTEGER GetClockTicks](#)() Returns: The current value of the system clock. Notes: One second is approximately 18 system clock ticks. **Examples:** /

***** [Return TRUE](#) if one or more seconds has elapsed since last call. For DOS only, account for midnight roll over. However, [if](#) called just after midnight, may possibly [return TRUE](#) even though 1 second has not passed since the last call.

***** [integer](#) last_tick [integer proc](#)

OneSecondElapsed() [integer](#) ticks ticks = [GetClockTicks](#)() [if](#) ticks < last_tick or ticks - last_tick > 18 last_tick = ticks
[return](#) (TRUE) [endif](#) [return](#) (FALSE) [end](#) See Also: [GetTime](#)(), [GetDate](#)()

GetColorTableValue

Gets the specified color value.

This command is intended for ADVANCED USAGE.

Syntax: [INTEGER GetColorTableValue](#)([INTEGER](#) tab, [INTEGER](#) index) - tab must be either [_FOREGROUND_](#) or [_BACKGROUND_](#) - index must be in [_BLACK_](#) .. [_WHITE_](#), where index represents the following editor color:
Index Color-name [Default](#) Editor Color Value [_BLACK_](#) 0x000000 [_BLUE_](#) 0x000080 [_GREEN_](#) 0x008000
[_CYAN_](#) 0x008080 [_RED_](#) 0x800000 [_MAGENTA_](#) 0x800080 [_BROWN_](#) 0x808000 [_LIGHT_GRAY_](#) 0xc0c0c0
[_DARK_GRAY_](#) 0x808080 [_LIGHT_BLUE_](#) 0x0000ff [_LIGHT_GREEN_](#) 0x00ff00 [_LIGHT_CYAN_](#) 0x00ffff
[_LIGHT_RED_](#) 0xff0000 [_LIGHT_MAG_](#) 0xff00ff [_YELLOW_](#) 0xffff00 [_WHITE_](#) 0xffffffff Returns: RGB value in tab at index. Notes: See [SetColorTableValue](#)() for additional information. **Examples:** [integer](#) rgb // get the current white foreground color value rgb = [GetColorTableValue](#)([_FOREGROUND_](#), [_WHITE_](#)) See Also: [SetColorTableValue](#)()

GetDate

Returns the system date.

Syntax: [GetDate](#)([VAR INTEGER](#) mon, [VAR INTEGER](#) day, [VAR INTEGER](#) year, [VAR INTEGER](#) dow) - mon is the month (1 to 12) - day is the day (1 to 31) - year is the year (1980 to 2099) - dow is the day of the week (1 to 7) (Sunday is day 1) Returns: Nothing. **Examples:** // Insert a line with the date before the current line // in the format mm-dd-yyyy [proc](#) InsertDateLine() [integer](#) month, day, year, dow [GetDate](#)(month, day, year, dow) [InsertLine](#)([Format](#)

(month:2, '-', day:2:'0', '-', year)) [end](#) See Also: [GetTime\(\)](#), [GetDateStr\(\)](#), [GetTimeStr\(\)](#)

GetDateStr

Returns the current system date as a string.

Syntax: [STRING GetDateStr\(\)](#) Returns: The current system date. Notes: The format of the date string returned is dependent upon the value of the DateFormat variable. Supported formats are: - 1 - mm-dd-yy - 2 - dd-mm-yy - 3 - yy-mm-dd - 4 - mm-dd-yyyy - 5 - dd-mm-yyyy - 6 - yyyy-mm-dd For example, the date, April 27th, 1992, would be expressed as: [DateFormat Return Value Value of GetDateStr\(\)](#) _____ 1 04-27-92 2 27-04-92 3 92-04-27 4 04-27-1992 5 27-04-1992 6 1992-04-27 The character used as the date separator is controlled by the [DateSeparator](#) variable. The default for DateFormat is 1, and the default for DateSeparator is "-". **Examples:** // Insert the system date at the cursor position [InsertText\(GetDateStr\(\), _INSERT_\)](#) See Also: [GetTimeStr\(\)](#), [GetDate\(\)](#), [GetTime\(\)](#) **Variables:** [DateFormat](#), [DateSeparator](#)

GetDir

Returns the current directory on the specified drive.

Syntax: [STRING GetDir\(STRING drive\)](#) - drive specifies the drive for which to identify the current directory. Returns: The current directory on the specified drive. Notes: The drive may be specified with or without the ":" appended. **Examples:** To display the current directory on drive c: [Message\(GetDir\("c"\)\)](#) To get the current directory on drive d: into a string, s: s = [GetDir\("d"\)](#) See Also: [CurrDir\(\)](#), [ChDir\(\)](#), [GetDrive\(\)](#), [LogDrive\(\)](#)

GetDrive

Determines the current drive.

Syntax: [STRING GetDrive\(\)](#) Returns: A single lower-case character denoting the current drive letter. Notes: The colon is not appended to the drive letter. See Also: [LogDrive\(\)](#), [ChDir\(\)](#), [CurrDir\(\)](#), [GetDir\(\)](#)

GetEnvStr

Searches for and returns a specified environment string.

Syntax: [STRING GetEnvStr\(STRING s\)](#) - s is the environment string for which to search. It must be specified as the exact case of the environment string; for example, "PATH", "COMSPEC" [or](#) "windir". Returns: The string associated with the environment string s. If environment string s does not exist, an empty string is returned. Notes: Use [Length\(\)](#) to test whether the environment string was found. A [Length\(\)](#) return value of 0 indicates that the environment string was [not](#) found. **Examples:** // Get the PATH environment string [string s\[128\]](#) s = [GetEnvStr\("PATH"\)](#)

GetFileToken

Returns a specific filename token (element) from a specified string.

Syntax: [STRING GetFileToken](#)([STRING](#) s, [INTEGER](#) token_no) - s is the string from which to return the filename token. - token_no is the number of the filename token to return. Returns: The specified filename token from the string. Notes: This command supports spaces and double quotes as delimiters. **Examples:** [string](#) s[80] = 'filename1 filename2 "name w/spaces" filename3.ext' [integer](#) i [for](#) i = 1 [to](#) [NumFileTokens](#)(s) // go through a series of warns: // 1 -> filename1 // 2 -> filename2 // 3 -> name w/spaces // 4 -> filename3.ext [warn](#)([GetFileToken](#)(s, i)) [endfor](#) See Also: [NumFileTokens](#)(), [GetToken](#)(), [NumTokens](#)(), [SubStr](#)()

GetFont

Retrieves the name, size, and style of the font that is being used for the editing display.

This command has no effect in the Console version of the editor.

Syntax: [GetFont](#)([VAR STRING](#) fontname, [VAR INTEGER](#) pointsize, [VAR INTEGER](#) flags) - fontname is the name of the font. Examples are "Terminal", "Courier", and "Lucida Console". - pointsize identifies the font size. - flags can be any combination of [_FONT_BOLD_](#) and/or [_FONT_OEM_](#) or just 0. Returns: This command does not have a return value, but rather, returns information through the passed VAR parameters. Notes: The point size is relative, and the actual font size can vary depending on [Windows](#) system settings such as Large Fonts vs. Small Fonts. For instance, with Small Fonts selected, "Terminal" point size 14 resolves to an 10x18 font. With Large Fonts selected, the same point size resolves into 10x20 font. To get the actual size of the font in pixels, use [GetCharWidthHeight](#)(). Contrast this command with the [ChooseFont](#)() command, which returns information about a selected font. **Examples:** [string](#) fontname[255] [integer](#) pointsize, flags [GetFont](#)(fontname, pointsize, flags) ... // at this point fontname, pointsize, and flags are set to // describe the selected font. See Also: [ChooseFont](#)(), [GetCharWidthHeight](#)(), [ResizeFont](#)() [SetFont](#)()

GetFoundText

Returns the results of the immediately preceding find.

Syntax: [STRING GetFoundText](#)([[INTEGER](#) tag_number]) - tag_number is the optional tagged pattern number to return from a regular expression search string. If not passed, the entire found [string](#) is returned. Returns: The found text. Notes: For this command to work properly, it should immediately follow a [Find](#)() or [IFind](#)() command, before any other commands that might change the current position are invoked. The optional tag_number parameter only applies if a regular expression find was performed. For more information on regular expressions, see the chapter on "[Search Features: Finding and Replacing Text](#)" in the User's Guide. **Examples:** [string](#) s[80] [if](#) [IFind](#)("^a.*z\$", "x") s = [GetFoundText](#)() // get the found text in s [endif](#) // Second example, this time retrieving the 2nd tagged // expression found. [string](#) s[80] [if](#) [IFind](#)("{^a}{.*}{z\$}", "x") s = [GetFoundText](#)(2) // get tag #2 [endif](#) See Also: [IFind](#)(), [Find](#)(), [GetMarkedText](#)(), [MarkFoundText](#)()

GetFreeHistory

Returns an unused history number.

Syntax: [INTEGER GetFreeHistory](#)([STRING](#) history_name) - history_name is a name to associate with the history number. Returns: An available history number in the range 1..127, or zero (FALSE) [if](#) there are no more history [numbers](#) available. Notes: This command can be used to get an unused history number for later use by user-written commands that need a history capability. The convention for history names is as follows: - For user-interface (.UI) files - UI:descriptive_name - For external macro source (.S) files - MACRO_NAME:descriptive_name **Examples:** Given a macro called foo, that implements the widget commands, widget_x1, and widget_y1: [integer](#) x1_hist, y1_hist x1_hist = [GetFreeHistory](#)("FOO:widget_x1") y1_hist = [GetFreeHistory](#)("FOO:widget_y1") See Also: [AddHistoryStr](#)(), [DelHistory](#)(), [DelHistoryStr](#)(), [FindHistoryStr](#)(), [GetHistoryStr](#)(), [NumHistoryItems](#)()

GetGlobalInt

Returns the value of an integer variable previously defined with the

[SetGlobalInt](#)() command. **Syntax:** [INTEGER GetGlobalInt](#)([STRING](#) integer_name) - integer_name is the name of the session global integer whose value is returned. Returns: The value of session global [integer](#) integer_name. Notes: This command can be used for communication between [macros](#) in separate macro files. **Examples:** [integer](#) foo foo = [GetGlobalInt](#)("myvar") See Also: [GetGlobalStr](#)(), [SetGlobalInt](#)(), [SetGlobalStr](#)(), [DelGlobalVar](#)(), [ExistGlobalVar](#)()

GetGlobalStr

Returns the value of a string variable previously defined with the

[SetGlobalStr](#)() command. **Syntax:** [STRING GetGlobalStr](#)([STRING](#) string_name) - string_name is the name of the session global string whose value is returned. Returns: The value of session global [string](#) string_name. Notes: This command can be used for communication between [macros](#) in separate macro files. **Examples:** [string](#) s[128] s = [GetGlobalStr](#)("password") See Also: [GetGlobalInt](#)(), [SetGlobalInt](#)(), [SetGlobalStr](#)(), [DelGlobalVar](#)(), [ExistGlobalVar](#)()

GetHistoryStr

Returns a specified item (string) from the History Buffer.

Syntax: [STRING GetHistoryStr](#)([INTEGER](#) history, [INTEGER](#) n) - history is an integer specifying the history list from which to return an item. history can be: - A pre-defined constant value representing one of the built-in histories. See the [AddHistoryStr](#)() command for a listing of these [constants](#). - A number in the range 1..127. - A number returned by [GetFreeHistory](#)(), which will be in the range 1..127. - n is the number, within the specified history list, of the history item to return. n should be in the range 1 through the value returned by [NumHistoryItems](#)(history), where 1 represents the item added most recently, and higher numbers represent older items. Returns: The specified history [string](#). Notes: This command is typically used in conjunction with the [FindHistoryStr](#)() or [NumHistoryItems](#)() command. **Examples:** // Display the [EditFile](#)() history [proc](#) DisplayHistory() [integer](#) i [PopWinOpen](#)(1, 1, 80, [NumHistoryItems](#)(_EDIT_HISTORY_) + 4, 1, "Edit History", 15) [ClrScr](#)() [for](#) i = 1 [to](#) [NumHistoryItems](#)(_EDIT_HISTORY_) [WriteLine](#)([GetHistoryStr](#)(_EDIT_HISTORY_, i)) [endfor](#) [Write](#)("Press any [key](#) to continue") [GetKey](#)() [PopWinClose](#)() [end](#) See Also: [DelHistoryStr](#)(), [NumHistoryItems](#)(), [GetFreeHistory](#)(), [AddHistoryStr](#)(), [FindHistoryStr](#)() **Variables:** [MaxHistoryPerList](#), [PersistentHistory](#)

GetHookState

Determines the current hook state of a specified editor event.

Syntax: [INTEGER GetHookState](#)([INTEGER](#) event) - event is the optional editor event for which to get the state. If event is not specified, the "global" editor event state is returned. See the [Hook\(\)](#) command for a list of editor events. Returns: Non-zero (TRUE) if the specified event is enabled; otherwise, zero ([FALSE](#)). Notes: This command allows you to determine if the hooks for a specific editor event are currently enabled or disabled. If the hooks are enabled, the editor event will trigger any event macros that are hooked to the event. If disabled, the event will not trigger [event macros](#) hooked to it. This command, used in conjunction with the [SetHookState\(\)](#) command, is useful for saving, setting, and then restoring the hook state of an event. Typically, the event hooks are disabled, a task is performed, and then the event hooks are re-enabled. The editor maintains a "global" event state. When the editor is started, the global event state is enabled, which means that the hooks for all editor events are enabled. Specific events can be individually disabled/enabled using the [SetHookState\(\)](#) command. The global event state can also be disabled using the [SetHookState\(\)](#) command, forcing all editor events to be disabled. When the global event state is re-enabled (again using the [SetHookState\(\)](#) command), each editor event is then restored to its previous hook state (just prior to the disabling of the global state). **Examples:** [if GetHookState\(_ON_CHANGING_FILES_\) Warn](#) (" _ON_CHANGING_FILES_ hook is enabled") [else Warn](#) (" _ON_CHANGING_FILES_ hook is disabled") [endif if GetHookState\(\)](#) [Warn](#) ("Hooks are enabled") [else Warn](#) ("Hooks are disabled") [endif](#) See Also: [BreakHookChain\(\)](#), [ExecHook\(\)](#), [Hook\(\)](#), [SetHookState\(\)](#), [UnHook\(\)](#)

GetKey

Waits for the user to press a key and returns the keycode of the pressed key.

Syntax: [INTEGER GetKey](#)([INTEGER](#) flag) - flag is an optional integer that affects certain keycodes returned to the macro. flag can be a combination of the following (OR'ed together to obtain the desired result): -
[_EQUATEENHANCED_KBD_](#) translates enhanced keyboard keys to their non-enhanced [key](#) counterpart. Use of this flag overrides the current EquateEnhancedKbd editor variable setting. For example, <GreyCursorRight> (an enhanced keyboard [key](#)) will be translated to <CursorRight> (the corresponding non-enhanced keyboard [key](#)). -
[_UNEQUATEENHANCED_KBD_](#) distinguishes between the enhanced keyboard keys and their non-enhanced keyboard counterpart. Use of this flag overrides the current EquateEnhancedKbd editor variable setting. -
[_LOWERCASE_KBD_](#) translates upper [case](#) keys to their lower [case](#) counterpart. For example, <A> will be translated to <a>. - [_UPPERCASE_KBD_](#) translates lower [case](#) keys to their upper [case](#) counterpart. For example, <z> will be translated to <Z>. Returns: The character and scan code for the key pressed. Notes: See the output from the ShowKey macro (located in the MAC\ editor subdirectory) for a list of keys and their respective codes. Key names in the format <editor key-name> (as shown by ShowKey) can be compared with the results returned from [GetKey\(\)](#). Keycodes returned from [GetKey\(\)](#) are made up of a scan code and the ASCII character of the key. (The character is 0 (zero) if the key was a function key.) Idioms: To isolate the separate portions of the keycode, use the following code: [integer key](#), char_code, scan_code [key](#) = [GetKey\(\)](#) char_code = [LoByte\(key\)](#) scan_code = [HiByte\(key\)](#) **Examples:** In the first example, if the editor's EquateEnhancedKbd variable is FALSE, pressing <GreyCursorUp> will cause the "Non arrow key pressed" message to be displayed. However, if the editor's EquateEnhancedKbd is TRUE, pressing <GreyCursorUp> causes the "CursorUp pressed" message to be displayed. [Case GetKey\(\)](#) [when](#) <CursorLeft> [Message](#) ("CursorLeft pressed") [when](#) <CursorRight> [Message](#) ("CursorRight pressed") [when](#) <CursorUp> [Message](#) ("CursorUp pressed") [when](#) <CursorDown> [Message](#) ("CursorDown pressed") [Otherwise Message](#) ("Non arrow [key](#) pressed") [endcase](#) In the second example, pressing <GreyCursorUp> will cause the "CursorUp pressed" message to be

displayed regardless of the editor's [EquateEnhancedKbd](#) variable setting. [Case GetKey](#) (`_EQUATEENHANCED_KBD_`) [when](#) <CursorLeft> [Message](#)("CursorLeft pressed") [when](#) <CursorRight> [Message](#)("CursorRight pressed") [when](#) <CursorUp> [Message](#)("CursorUp pressed") [when](#) <CursorDown> [Message](#)("CursorDown pressed") [Otherwise Message](#)("Non arrow key pressed") [endcase](#) See Also: [GetKeyFlags\(\)](#), [KeyPressed\(\)](#), [KeyCode\(\)](#), [KeyName\(\)](#), [PushKey\(\)](#), [PressKey\(\)](#), [PushKeyStr\(\)](#), [WaitForKeyPressed\(\)](#), [isKeyAssigned\(\)](#) [Variables: EquateEnhancedKbd](#)

GetKeyFlags

Returns the current state of the keyboard shift flags.

Syntax: [INTEGER GetKeyFlags\(\)](#) Returns: The keyboard shift flags. Notes: Pre-defined editor [constants](#) may be used when testing for various key flags. The following table shows the actual bit settings returned by [GetKeyFlags\(\)](#), along with the associated pre-defined editor [constant](#): Associated Bit Editor [Constant](#)

_____ 0 - Right Shift	<code>_RIGHT_SHIFT_KEY_</code>	1 - Left Shift	<code>_LEFT_SHIFT_KEY_</code>	2 - Ctrl	<code>_CTRL_KEY_</code>	3 - Alt	<code>_ALT_KEY_</code>	4 - Scroll Lock	<code>_SCROLL_LOCK_DEPRESSED_</code>	5 - Num Lock	<code>_NUM_LOCK_DEPRESSED_</code>	6 - Caps Lock	<code>_CAPS_LOCK_DEPRESSED_</code>	7 - Insert	<code>_INSERT_DEPRESSED_</code>
-----------------------	--------------------------------	----------------	-------------------------------	----------	-------------------------	---------	------------------------	-----------------	--------------------------------------	--------------	-----------------------------------	---------------	------------------------------------	----------------------------	---------------------------------

Examples: The following code shows how to determine if particular shift states are active: [integer](#) flags = [GetKeyFlags](#)() // see if left shift key pressed [if](#) flags & `_LEFT_SHIFT_KEY_` ... [endif](#) // see if either shift key and the alt key is pressed [if](#) (flags & (`_LEFT_SHIFT_KEY_` | `_RIGHT_SHIFT_KEY_`)) [and](#) flags & `_ALT_KEY_` ... [endif](#) See Also: [GetKey\(\)](#), [KeyCode\(\)](#), [KeyName\(\)](#), [KeyPressed\(\)](#)

GetMarkedText

Copies a marked block into a string, for use in macros.

Syntax: [STRING GetMarkedText\(\)](#) Returns: The first line of the marked block. Notes: If no block exists, an empty string is returned. The first line of the block is returned, even if the block spans multiple lines. **Examples:** /

```
***** Return the word at (or to the left of) the cursor
as a string. *****/ string proc GetWordAtCursor()
string word[80] = " PushPosition\(\) PushBlock\(\) // Save current block status if MarkWord\(\) or (Left\(\) and MarkWord\(\))
// Mark it word = GetMarkedText\(\) // Get it endif PopBlock\(\) // Restore block status PopPosition\(\) return (word) //
That's all, folks! end See Also: GetText\(\), isBlockInCurrFile\(\), isBlockMarked\(\), MarkFoundText\(\), GetWord\(\),
GetFoundText\(\)
```

GetMaxRowsCols

Returns the maximum number of rows and columns that are currently supported.

Syntax: [GetMaxRowsCols](#)([VAR INTEGER](#) rows, [VAR INTEGER](#) cols) - rows is the maximum number of rows available. - cols is the maximum number of columns available. Returns: Nothing. **Examples:** [integer](#) rows, cols [GetMaxRowsCols](#)(rows, cols) See Also: [SetVideoRowsCols\(\)](#)

GetNextProfileItem

Retrieves the current item (and its value) from the current section of the initialization file that was accessed in the last call to

[LoadProfileSection\(\)](#). **Syntax:** [INTEGER GetNextProfileItem](#)([VAR STRING](#) item, [VAR STRING](#) item_value) - item is the current item, based on the item pointer's position due to the last call to this command. - item_value is the value of item. Returns: Non-zero if an item exists; zero (FALSE) when no items are left. Notes: Each call to this command causes the item pointer to be set to the next item in the list. For this command to [return](#) any useful information, [LoadProfileSection\(\)](#) must have been called prior to the first calling of this command. Each call to [LoadProfileSection\(\)](#) resets the item pointer to the top of the item list. **Examples:** // The following loads the "CUAMARK" section from the tse.ini file // (located in the editor's load directory) into memory and then // displays the name of each item and its value for each item // found in the CUAMARK section. [proc main\(\)](#) [string](#) item[255] = "" [string](#) value[255] = "" [integer](#) counter = 0 [if](#) [LoadProfileSection](#)("CUAMARK") [while](#) [GetNextProfileItem](#)(item, value) counter = counter + 1 [warn](#)("Item ", [str](#)(counter), " is ", item, " which is equal to ", value, " ...") [endwhile](#) [warn](#)("There were ", counter, " items. There are no more items ...") [else](#) [warn](#)("CUAMARK section [not](#) found") [endif](#) [end](#) See Also: [LoadProfileSection\(\)](#), [LoadProfileSectionNames\(\)](#), [GetNextProfileSectionName\(\)](#), [GetProfileInt\(\)](#), [GetProfileStr\(\)](#), [WriteProfileInt\(\)](#), [WriteProfileStr\(\)](#), [RemoveProfileItem\(\)](#), [RemoveProfileSection\(\)](#), [FlushProfile\(\)](#)

GetNextProfileSectionName

Retrieves the current section name from an initialization file, based on the current section name pointer as was set by the last call to this command.

Syntax: [INTEGER GetNextProfileSectionName](#)([VAR STRING](#) section) - The current section name is returned in section. Returns: Non-zero if a section name exists (see notes); zero (FALSE) when no more section names exist. Notes: Each call to this command causes the section name pointer to be set to the next section name in the list. For this command to [return](#) any useful information, [LoadProfileSectionNames\(\)](#) must have been called for an initialization file prior to the first calling of this command. Note that each call to [LoadProfileSectionNames\(\)](#) resets the section name pointer to the top of the section name list for the initialization file. This command returns non-zero the first time it is called (after calling [LoadProfileSectionNames\(\)](#)), even if no section names exist (and in this case, section will contain an empty string). **Examples:** See the [LoadProfileSectionNames\(\)](#) command example. See Also: [LoadProfileSectionNames\(\)](#), [LoadProfileSection\(\)](#), [GetNextProfileItem\(\)](#), [GetProfileInt\(\)](#), [GetProfileStr\(\)](#), [WriteProfileInt\(\)](#), [WriteProfileStr\(\)](#), [RemoveProfileItem\(\)](#), [RemoveProfileSection\(\)](#), [FlushProfile\(\)](#)

GetPositionInfo

Returns information about the location of a specified position, as set by the

[PushPosition\(\)](#) and [PopPosition\(\)](#) commands. **Syntax:** [INTEGER GetPositionInfo](#)([INTEGER](#) position, [VAR INTEGER](#) id, [VAR INTEGER](#) line, [VAR INTEGER](#) pos, [VAR INTEGER](#) xoffset, [VAR INTEGER](#) row) - position is a number (1 through the number of positions currently set) that specifies the position. - id is the id number of the buffer where the position is located. - line is the number of the line in the file on which the position is located. - pos identifies the position on the line of the specified position. - xoffset indicates the number of columns by which the screen was scrolled when the position was set. - row indicates the screen row on which the position was located. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command allows you to examine information about a position. This command fails if the specified position number does not exist. If this command fails, the values of the position parameters are undefined. See Also: [PushPosition\(\)](#), [PopPosition\(\)](#), [SwapPosition\(\)](#)

GetProfileInt

Retrieves the integer value of an item that resides in a section of an initialization file.

Syntax: [INTEGER GetProfileInt](#)([STRING](#) section, [STRING](#) item, [INTEGER](#) default_value [, [STRING](#) profile_name])
 - section is the section name within an initialization file. - item is the item name within the section of an initialization file. - default_value is the default integer value to return if item is [not](#) found. - profile_name is an optional initialization file to search. If not specified, the tse.ini file located in the editor's load directory is searched. If profile_name is specified, and does not include a path, the profile_name that resides in the WINDOWS directory (specified by the windir environment variable) is searched. Returns: The integer value of the requested item; otherwise, default_value [if](#) item does [not](#) exist. Notes: The spelling of section and item are [not case](#) sensitive. **Examples:** // The following searches the tse.ini file (located in the // editor's load directory) and assigns "i" the integer // value of "MyItem" [or](#) -1000, if "MyItem" is [not](#) found. i = [GetProfileInt](#)("MySection", "MyItem", -1000) // The following searches the mymac.ini file (located in the // WINDOWS directory) and assigns "i" the integer // value of "MyOtherItem" [or](#) 999 if "MyOtherItem" is [not](#) found. i = [GetProfileInt](#)("MyOtherSection", "MyOtherItem", 999, "mymac.ini") // The following searches the newmac.ini file (located in the // directory that was current at the time the editor was started) // and assigns "i" the integer value of "AnItem" or 2000 if // "AnItem" is [not](#) found. i = [GetProfileInt](#)("ASection", "AnItem", 2000, [Query](#)(StartUpPath) + "newmac.ini") See Also: [GetProfileStr](#)(), [WriteProfileInt](#)(), [WriteProfileStr](#)(), [LoadProfileSection](#)(), [GetNextProfileItem](#)(), [LoadProfileSectionNames](#)(), [GetNextProfileSectionName](#)(), [RemoveProfileItem](#)(), [RemoveProfileSection](#)(), [FlushProfile](#)()

GetProfileStr

Retrieves the string value of an item that resides in a section of an initialization file.

Syntax: [STRING GetProfileStr](#)([STRING](#) section, [STRING](#) item, [STRING](#) default_value [, [STRING](#) profile_name]) - section is the section name within an initialization file. - item is the item name within the section of an initialization file. - default_value is the default string value to return if item is [not](#) found. - profile_name is an optional initialization file to search. If not specified, the tse.ini file located in the editor's load directory is searched. If profile_name is specified, and does not include a path, the profile_name that resides in the WINDOWS directory (specified by the windir environment variable) is searched. Returns: The string value of the requested item; otherwise, default_value [if](#) item does [not](#) exist. Notes: The spelling of section and item are [not case](#) sensitive. **Examples:** // The following searches the tse.ini file (located in the // editor's load directory) and assigns "i" the string // data of "MyItem" [or](#) "My [String](#)", if "MyItem" is [not](#) found. i = [GetProfileStr](#)("MySection", "MyItem", "My [String](#)") // The following searches the mymac.ini file (located in the // WINDOWS directory) and assigns "i" the string // data of "MyOtherItem" [or](#) "nine" if "MyOtherItem" is [not](#) found. i = [GetProfileStr](#)("MyOtherSection", "MyOtherItem", "nine", "mymac.ini") // The following searches the newmac.ini file (located in the // directory that was current at the time the editor was started) // and assigns "i" the string data of "AnItem" or "TWO" if // "AnItem" is [not](#) found. i = [GetProfileStr](#)("ASection", "AnItem", "TWO", [Query](#)(StartUpPath) + "newmac.ini") See Also: [GetProfileInt](#)(), [WriteProfileStr](#)(), [WriteProfileInt](#)(), [LoadProfileSection](#)(), [GetNextProfileItem](#)(), [LoadProfileSectionNames](#)(), [GetNextProfileSectionName](#)(), [RemoveProfileItem](#)(), [RemoveProfileSection](#)(), [FlushProfile](#)()

GetStr

Reads characters from the screen, at the current video output position.

Syntax: [INTEGER GetStr](#)([VAR STRING](#) s, [INTEGER](#) max_len) - s is the string in which the characters that are read are placed. - max_len is the maximum number of characters to read. Returns: The number of characters actually read. Notes: This is a video output command. It does not read the actual text in the buffer. It only reads what is displayed on the screen. Reading starts at the current video output position. To set the video output position use the [vGotoXY\(\)](#) command. Reading is limited to one screen row at a time. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." **Examples:** [string](#) s[80] = "" [vGotoXY](#)(1, 1) [GetStr](#)(s, [sizeof](#)(s)) See Also: [GetStrXY\(\)](#), [GetStrAttr\(\)](#), [GetStrAttrXY\(\)](#)

GetStrAttr

Reads character/attribute pairs from the screen, at the current video output position.

Syntax: [INTEGER GetStrAttr](#)([VAR STRING](#) s, [VAR STRING](#) a, [INTEGER](#) max_len) - s is the string in which the characters that are read are placed. - a is the string in which the attributes that are read are placed. - max_len is the maximum number of character/attribute pairs to read. Returns: The number of character/attribute pairs actually read. Notes: This is a video output command. It does not read the actual text in the buffer. It only reads what is displayed on the screen. Reading starts at the current video output position. To set the video output position use the [vGotoXY\(\)](#) command. Reading is limited to one screen row at a time. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." **Examples:** // read 80 chars/attributes into s and a, respectively [string](#) s[80] = "", a[80] = " [vGotoXY](#)(1, 1) [GetStrAttr](#)(s, a, [sizeof](#)(s)) See Also: [GetStr\(\)](#), [GetStrXY\(\)](#), [GetStrAttrXY\(\)](#)

GetStrAttrXY

Reads character/attribute pairs from the screen, at the specified video output coordinates.

Syntax: [INTEGER GetStrAttrXY](#)([INTEGER](#) x, [INTEGER](#) y, [VAR STRING](#) s, [VAR STRING](#) a, [INTEGER](#) max_len) - x is the column from which to start reading character/attribute pairs. - y is the row from which to start reading character/attribute pairs. - s is the string in which the characters that are read are placed. - a is the string in which the attributes that are read are placed. - max_len is the maximum number of character/attribute pairs to read. Returns: The number of character/attribute pairs actually read. Notes: This is a video output command. It does not read the actual text in the buffer. It only reads what is displayed on the screen. Reading starts at the specified video output position, and is limited to one screen row at a time. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." **Examples:** // read 80 char/attributes into s, a, respectively [string](#) s[80] = "", [string](#) a[80] = " [GetStrAttrXY](#)(1, 1, s, a, [sizeof](#)(s)) See Also: [GetStr\(\)](#), [GetStrXY\(\)](#), [GetStrAttr\(\)](#)

GetStrFromWinClip

Retrieves up to 255 characters of the text that is currently in the Windows clipboard.

Syntax: [STRING GetStrFromWinClip\(\)](#) Returns: The contents of the Windows clipboard as a string, truncated at 255 characters if necessary. If the Windows clipboard is empty, an empty string is returned. See Also: [PasteFromWinClip\(\)](#), [CopyToWinClip\(\)](#), [CutToWinClip\(\)](#),

GetStrXY

Reads characters from the screen, at the specified video output coordinates.

Syntax: [INTEGER GetStrXY\(INTEGER x, INTEGER y, VAR STRING s, INTEGER max_len\)](#) - x is the column from which to start reading characters. - y is the row from which to start reading characters. - s is the string in which the characters that are read are placed. - max_len is the maximum number of characters to read. Returns: The number of characters actually read. Notes: This is a video output command. It does not read the actual text in the buffer. It only reads what is displayed on the screen. Reading starts at the specified video output position, and is limited to one screen row at a time. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." **Examples:** `string s[80] = " GetStrXY(1, 1, s, sizeof(s))` See Also: [GetStr\(\)](#), [GetStrAttr\(\)](#), [GetStrAttrXY\(\)](#)

GetSynFilename

Returns the full name of the ".syn" file associated with the current buffer, including drive and path.

This command is intended for ADVANCED USAGE.

Syntax: [STRING GetSynFilename\(\)](#) Returns: The name of the ".syn" file linked with the current buffer, if one is linked; otherwise, an empty string. **Examples:** `string s[_MAXPATH_] s = GetSynFilename()` See Also: [GetSynLanguageType\(\)](#), [InitSynhiCurrFile\(\)](#), [PurgeSynhi\(\)](#), [LinkSynFile\(\)](#), [LoadSynhiAssoc\(\)](#), [ReplaceSynFile\(\)](#) **Variables:** [ShowSyntaxHilite](#)

GetSynLanguageType

Returns the filename of the ".syn" file associated with the current buffer.

This command is intended for ADVANCED USAGE.

Syntax: [STRING GetSynLanguageType\(\)](#) Returns: The name of the ".syn" file linked with the current buffer, if one is linked; otherwise, an empty string. **Examples:** `string s[_MAXPATH_] s = GetSynLanguageType()` See Also: [GetSynFilename\(\)](#), [InitSynhiCurrFile\(\)](#), [PurgeSynhi\(\)](#), [LinkSynFile\(\)](#), [LoadSynhiAssoc\(\)](#), [ReplaceSynFile\(\)](#) **Variables:** [ShowSyntaxHilite](#)

GetSynMultiLnDlmt

Returns the specified multi-line delimiter token from the currently linked

[syntax](#) highlighting file. This command is intended for ADVANCED USAGE. **Syntax:** [INTEGER GetSynMultiLnDlmt](#) ([VAR STRING](#) s1, [VAR STRING](#) s2 [, [INTEGER](#) which]) - s1 is where the beginning delimiter is returned. - s2 is where the ending delimiter is returned. - which is which delimiter pair to [return](#), 1, 2 or 3. If [not](#) specified, which defaults to 1. If specified, must be in the range 1..3. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If which is greater than the number of defined multi-line delimiter tokens, [GetSynMultiLnDlmt\(\)](#) returns [FALSE](#). **Examples:** [string](#) d1[30]="", d2[30]=""" // get the 2nd pair of multi-line-delimiter tokens [if](#) [GetSynMultiLnDlmt](#)(d1, d2, 2) See Also: [GetSynToEOL\(\)](#)

GetSynToEOL

Returns the specified To-EOL delimiter token from the currently linked

[syntax](#) highlighting file. This command is intended for ADVANCED USAGE. **Syntax:** [INTEGER GetSynToEOL](#) ([VAR STRING](#) s [, [INTEGER](#) which]) - s is where the To-EOL delimiter is returned. - which is the delimiter to return, 1, 2 or 3. If not specified, which defaults to 1. If specified, must be in the range 1..3. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If which is greater than the number of defined To-EOL delimiter tokens, [GetSynToEOL\(\)](#) returns [FALSE](#). **Examples:** [string](#) d[30]=""" // get the 2nd To-EOL delimiter token [if](#) [GetSynToEOL](#)(d, 2) See Also: [GetSynMultiLnDlmt\(\)](#)

GetText

Retrieves text from the current line of the current file.

Syntax: [STRING GetText](#)([INTEGER](#) pos, [INTEGER](#) len) - pos is the position on the line to start retrieving text. To start at the current position, use [CurrPos\(\)](#). - len is the number of characters to retrieve. Returns: The string containing the retrieved text. Notes: [CurrPos\(\)](#) will [not](#) equal [CurrCol\(\)](#) if real (Hard) tabs exist in the current line before the current position and ExpandTabs is ON. Note that tabs are never expanded in the returned string. [GetText\(\)](#) stops if it reaches the end of the line, so the length of the returned string may be less than len. [GetText\(\)](#) returns an empty string, if len is less than or equal to zero; or if pos is less than or equal to zero or greater than the length of the current line. **Examples:** /***** A macro to return the word at the cursor position. The word returned starts at the cursor position and is based on the current WordSet definition. Any portion of the word to the left of the cursor IS included. The word is limited to 32 characters or the end of line. To return non-white composed words at the cursor position, change the tests "isWord()" to "(not isWhite())" *****/ [string proc](#) [GetWord](#)() [integer](#) count = 0, start_pos [if](#) [not](#) [isWord](#)() [return](#) ("") [endif](#) [PushPosition](#)() // Save current position [loop](#) // Scan left for word start [if](#) [not](#) [Left](#)() [break](#) [endif](#) [if](#) [not](#) [isWord](#)() [Right](#)() [break](#) [endif](#) [endloop](#) start_pos = [CurrPos](#)() // Remember start [while](#) [isWord](#)() [and](#) count < 32 // Scan right for end count = count + 1 // of word. [if](#) [not](#) [Right](#)() // Do [not](#) get caught! [break](#) [endif](#) [endwhile](#) [PopPosition](#)() // Restore position [return](#) ([GetText](#)(start_pos, count)) // [Return](#) it [end](#) For a much simpler solution, see the example in [GetMarkedText](#)(). See Also: [InsertText](#)(), [CurrChar](#)(), [GetMarkedText](#)(), [GetWord](#)(), [GetFoundText](#)() **Variables:** [ExpandTabs](#)

GetTime

Returns the system time.

Syntax: [GetTime](#)([VAR INTEGER](#) hrs, [VAR INTEGER](#) min, [VAR INTEGER](#) sec, [VAR INTEGER](#) hun) - hrs is the hours (0 to 23) - min is the minutes (0 to 59) - sec is the seconds (0 to 59) - hun is hundredths of seconds (0 to 99) or: [INTEGER GetTime](#)() Returns: If the first Syntax format is used, nothing is returned; otherwise, the current time is returned, expressed in hundredths of a second. See Also: [GetTimeStr](#)(), [GetDate](#)(), [GetDateStr](#)()

GetTimeStr

Returns the current system time as a string.

Syntax: [STRING GetTimeStr](#)() Returns: The current system time. Notes: The format of the time string returned is dependent upon the value of the [TimeFormat](#) variable. Supported formats are: - 1 - hh:mm, where hh is the hours in military format. - 2 - hh:mm am/pm, where hh is the hours in 12-hour format. For example, the time, 1:30 pm, would be expressed as: [TimeFormat Return](#) Value Value of [GetTimeStr](#)() _____ 1 13:30 2 01:30 pm The character used as the time separator is controlled by the [TimeSeparator](#) variable. The default for [TimeFormat](#) is 2, and the default for [TimeSeparator](#) is ":". **Examples:** // Insert the system time at the cursor position [InsertText](#) ([GetTimeStr](#)(), [_INSERT_](#)) See Also: [GetDateStr](#)(), [GetTime](#)(), [GetDate](#)() Variables: [TimeFormat](#), [TimeSeparator](#)

Getting Help in the Editor

Two methods are available to obtain information

[about](#) commands and options in the editor: - [The customizable HelpLine](#) - [The HelpSystem](#) [The HelpLine](#) serves as a simple, ready reminder of key assignments for a select group of commands. [The HelpSystem](#) provides detailed information on the editor's commands, options, and macro programming language.

Getting User Input

The editor offers a rich assortment of commands to get input from the user. Following are the most commonly used commands of this type:

*

[Ask](#)() The [Ask](#)() command pops up a prompt box and allows the user to type a string in response. Pressing <Escape> or <Enter> terminates the prompt and returns control back to the caller. History is also supported, if specified. * [Read](#)() [Read](#)() is a lower-level user-input command. There is no prompt. User input is typed and read at the current video output position. (See "[Manipulating the Video Output Position and the Visible Cursor](#)" under "[Video Output Commands](#)" later in this chapter.) As with [Ask](#)(), pressing <Escape> or <Enter> terminates this command. You would use this command if you were building a data entry screen or a custom dialog box, probably in combination with [PopWinOpen](#)() and other [video output commands](#). [Read](#)() can also be used within menus. See the standard user-interface macro file, TSE.UI (in the UI\ editor subdirectory), for examples. * [GetKey](#)() [GetKey](#)() is the lowest-level user-input command. It simply returns the next keystroke typed by the user. This is useful for branching to other routines, depending on the key that is pressed.

GetToken

Returns a specific token (element) from a specified string.

Syntax: [STRING GetToken](#)([STRING](#) s, [STRING](#) delimiter, [INTEGER](#) token_no) - s is the string from which to return the token. - delimiter is the delimiting character (or set of characters) that separate each token. - token_no is the number of the token to return. Returns: The specified token from the string. Notes: In the string "A rose by any other name", if delimiter is specified as a space (" "), then token number 1 is "A", token number 2 is "rose", token number 3 is "by", etc. **Examples:** string t[70] = "hello world, this is a test" [integer](#) i [for](#) i = 1 [to](#) [NumTokens](#)(t, " ") // go through a series of warns: // 1 -> hello // 2 -> world, // 3 -> this // 4 -> is // 5 -> a // 6 -> test [warn](#)([GetToken](#)(t, " ", i)) [endfor](#) See Also: [NumTokens](#)(), [SubStr](#)()

GetWheelScrollLines

Gets the user specified scrolling range for the mouse wheel.

This command has no effect in the Console version of the editor.

Syntax: [INTEGER GetWheelScrollLines](#)() Returns: The number of lines the window should scroll when the mouse is rotated. **Examples:** <WheelUp> [RollUp](#)([GetWheelScrollLines](#)()) <WheelDown> [RollDown](#)([GetWheelScrollLines](#)())

GetWindowTitle

Returns the title bar string of the console window.

Syntax: [STRING GetWindowTitle](#)() Returns: The title bar string of the console window. Notes: The console window is the window in which the editor itself is running. **Examples:** [string](#) title[255] title = [GetWindowTitle](#)() See Also: [SetWindowTitle](#)()

GetWinHandle

Returns the handle of the console window.

This command is intended for ADVANCED USAGE.

Syntax: [INTEGER GetWinHandle](#)() Returns: A non-zero window handle if successful; [otherwise](#), zero ([FALSE](#)). Notes: The console window is the window in which the editor itself is running. **Examples:** [integer](#) handle handle = [GetWinHandle](#)()

GetWord

Returns the word at or to the immediate left of the cursor.

Syntax: [STRING GetWord](#)([[INTEGER](#) go_left [, [STRING](#) bit_encoded_32_char]]) - go_left optionally instructs [GetWord](#)() to check one character to the left for a word to return (if one does not exist at current position). If specified, go_left must be non-zero (TRUE) or zero (FALSE). If not specified, the default is zero ([FALSE](#)). - bit_encoded_32_char is an optional bit-encoded 32-character string denoting a user-defined WordSet. Use the [ChrSet](#)()

command to generate this [string](#). Returns: Returns the word at or to the immediate left of the cursor. Notes: [GetWord\(\)](#) is the same as: [if MarkWord\(\) return GetMarkedText\(\) endif](#) [GetWord\(1\)](#) is the same as: [if MarkWord\(\) or \(Left\(\) and MarkWord\(\)\) return GetMarkedText\(\) endif](#) Examples: /

```
***** Return the current word. Use built-in function.
Remove all leading and trailing quotes. *****/ string
proc GetCurrWord() string word[32] word = GetWord\(\) while Length(word) and word[1] == "" word = substr(word,
2, sizeof(word)) endwhile while Length(word) and word[Length(word)] == "" word = substr(word, 1, Length(word) -
1) endwhile return (word) end /***** Return the
current word. The word must be composed of numbers only. Note how 0 is passed as the go_left parameter.
*****/ string word[48] word = GetWord(0, ChrSet("0-
9")) See Also: MarkWord\(\), GetFoundText\(\), isWord\(\) Variables: WordSet
```

GlobalUnDelete

Undeletes (restores) deletions made by

[DelBlock\(\)](#), [DelRightWord\(\)](#), [DelLine\(\)](#), and [DelToEol\(\)](#), inserting the text at the cursor position. Syntax: [INTEGER GlobalUnDelete\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command fails if there is [not](#) enough memory to successfully complete the operation. If the Deletion Buffer is empty, no action occurs and zero is returned. Deletions from all buffers are stored in the Deletion Buffer. [GlobalUnDelete\(\)](#) undeletes without regard to the buffer from which the text was originally deleted, and hence may be used to move deleted text to a file other than its original file. ([UnDelete\(\)](#) restores deleted text to the original file only.) [GlobalUnDelete\(\)](#) restores text, deletion by deletion, until the Deletion Buffer is empty. See Also: [UnDelete\(\)](#), [DelBlock\(\)](#), [DelRightWord\(\)](#), [DelLine\(\)](#), [DelToEol\(\)](#)

GotoBlockBegin

Moves the cursor to the beginning of the marked block.

Syntax: [INTEGER GotoBlockBegin\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Zero is returned if there is not a marked block, or if the cursor does not move from the current position (the cursor is already at the beginning of the block). The cursor is moved only if there is a marked block. If the block is not in the current file, the cursor moves to the file containing the marked block, and that file becomes current. If a different file does become the current file, it does so in the current window, which might be different from the original window the file was located in. If the user is currently [marking a block](#), this command forces the block END (closes the block at the cursor position) and moves to the beginning of the block (if it is not already at the beginning). If a different file than the current file is switched to, the `_ON_CHANGING_FILES_` event is called, if it exists. See Also: [isBlockMarked\(\)](#), [isBlockInCurrFile\(\)](#), [GotoBlockEnd\(\)](#), [Hook\(\)](#), [UnHook\(\)](#), [GotoBlockBeginCol\(\)](#), [GotoBlockEndCol\(\)](#) Variables: [BlockId](#), [Marking](#)

GotoBlockBeginCol

Moves the cursor to the left-most column of the block, if a block is on the current line.

Syntax: [INTEGER GotoBlockBeginCol\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If there is

not a block marked on the current line, the cursor position is [not](#) changed and zero is returned. The left-most column of a line block is always column one. This command does not change the current line. See Also: [GotoBlockEndCol\(\)](#), [GotoBlockBegin\(\)](#), [GotoBlockEnd\(\)](#)

GotoBlockEnd

Moves the cursor to the end of the marked block.

Syntax: [INTEGER GotoBlockEnd\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Zero is returned if there is not a marked block, or if the cursor does not move from the current position (the cursor is already at the end of the block). The cursor is moved only if there is a marked block. If the block is not in the current file, the cursor moves to the file containing the marked block, and that file becomes current. If a different file does become the current file, it does so in the current window, which might be different from the original window the file was located in. If the user is currently [marking a block](#), this command forces the block END (closes the block at the cursor position) and returns zero (since block END has been forced at the current position). If a different file than the current file is switched to, the `_ON_CHANGING_FILES_` event is called, if it exists. See Also: [isBlockMarked\(\)](#), [isBlockInCurrFile\(\)](#), [GotoBlockBegin\(\)](#), [Hook\(\)](#), [UnHook\(\)](#), [GotoBlockBeginCol\(\)](#), [GotoBlockEndCol\(\)](#) Variables: [BlockId](#), [Marking](#)

GotoBlockEndCol

Moves the cursor to the right-most column of the block, if a block is on the current line.

Syntax: [INTEGER GotoBlockEndCol\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If there is not a block marked on the current line, the cursor position is [not](#) changed and zero is returned. The right-most column of a line block is the same as the [end](#)-of-line position. This command does not change the current line. See Also: [GotoBlockBeginCol\(\)](#), [GotoBlockBegin\(\)](#), [GotoBlockEnd\(\)](#)

GotoBufferId

Makes the specified buffer the new current file.

Syntax: [INTEGER GotoBufferId\(INTEGER id\)](#) - id is the id number of the buffer to which to go. It must be a value returned via [CreateBuffer\(\)](#), [CreateTempBuffer\(\)](#) or [GetBufferId\(\)](#). Returns: Zero ([FALSE](#)) if buffer id does not exist; otherwise, the id number of the current buffer. Notes: [GotoBufferId\(\)](#) is intended for use within [macros](#), to switch back and forth between [buffers](#) without causing side effects. It is [not](#) intended for use to switch to files that will be edited by the user; [EditFile\(\)](#) is supplied for this purpose. [GotoBufferId\(\)](#) assumes that the id passed to it is the id number of an already-loaded buffer or the id number of a buffer created via [CreateBuffer\(\)](#) or [CreateTempBuffer\(\)](#). This command (unlike [EditFile\(\)](#), [NextFile\(\)](#) and [PrevFile\(\)](#)) does not invoke the `_ON_CHANGING_FILES_` and `_ON_FIRST_EDIT_` events. **Examples:** See the [CreateBuffer\(\)](#) and [NumFiles\(\)](#) commands for examples. See Also: [GetBufferId\(\)](#), [CreateBuffer\(\)](#), [CreateTempBuffer\(\)](#), [BufferType\(\)](#), [NumFiles\(\)](#)

GotoColumn

Moves the cursor to the column you specify.

Syntax: [INTEGER GotoColumn](#)([[INTEGER](#) column]) - column is the column to which to move the cursor. It is calculated such that any expanded tab characters are counted as the number of spaces each occupies. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If column is not passed to this command, it prompts the user for the column. If there are real (Hard) tabs on the current line and ExpandTabs is ON, the column you specified may not be accessible and the cursor may not end up in the column expected. If used interactively, preceding the column number with + or - moves the specified number of columns relative to the current column position. This command returns zero if the cursor column does not change. This command includes a history list of previous entries passed to it. See [AddHistoryStr\(\)](#) for additional information. **Examples:** // Goto the tenth column in the current line [GotoColumn](#)(10) // Goto the tenth column past the current column [GotoColumn](#)([CurrCol](#)() + 10) // Prompt the user, and goto the requested column [GotoColumn](#)() See Also: [CurrCol](#)(), [GotoLine](#)(), [AddHistoryStr](#)() **Variables:** [ExpandTabs](#)

GotoLine

Moves the cursor to the specified line.

Syntax: [INTEGER GotoLine](#)([[INTEGER](#) line_number]) - line_number is the line to which to move the cursor. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If line_number is not specified, [GotoLine](#)() prompts for the line number. If used interactively, preceding the line number with + or - moves the specified number of lines relative to the current line. This command returns zero if the cursor line does not change. This command includes a history list of previous entries passed to it. See [AddHistoryStr](#)() for additional information. **Examples:** // Goto the tenth line in the current file [GotoLine](#)(10) // Goto the tenth line past the current line [GotoLine](#)([CurrLine](#)() + 10) // Prompt the user, and goto the requested line [GotoLine](#)() See Also: [CurrLine](#)(), [GotoColumn](#)(), [AddHistoryStr](#)()

GotoMark

Moves to the specified bookmark.

Syntax: [INTEGER GotoMark](#)([[STRING](#) bookmark]) - bookmark is the bookmark to which the user wants to move. bookmark is optional. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If the user does not pass bookmark to this command, it prompts for the bookmark to which to go. The editor allows up to 26 [bookmarks](#), A through Z. The bookmark [identifiers](#) are [case](#) insensitive. Bookmarks work across files, so [GotoMark](#)() can cause the editor to switch to another file. If another file is switched to and an _ON_CHANGING_FILES_ event exists, it will be called. If a different file does become the current file, it does so in the current window, which might be different from the original window the file was located in. See Also: [PlaceMark](#)(), [DelAllBookMarks](#)(), [DelBookMark](#)(), [GetBookMarkInfo](#)(), [isBookMarkSet](#)(), [Hook](#)(), [UnHook](#)()

GotoMouseCursor

Moves the text cursor to the mouse cursor, switching windows if necessary.

Syntax: [INTEGER GotoMouseCursor](#)() Returns: A non-zero value if the mouse cursor (as indicated by (MouseX, MouseY)) is within an editing window; otherwise, zero ([FALSE](#)) if the mouse position is on a border or outside all editing [windows](#). Notes: [GotoMouseCursor](#)() will automatically change [windows](#), if necessary, before moving the text

cursor within the new window. To avoid moving the text cursor to the mouse cursor when changing [windows](#), use [GotoWindow\(MouseWindowId\(\)\)](#) to change [windows](#). See Also: [GotoWindow\(\)](#), [MouseWindowId\(\)](#)

GotoPos

Moves the cursor to the specified position on the current line.

Syntax: [INTEGER GotoPos\(INTEGER pos\)](#) - pos is the position in the current line to which to go. It is calculated such that any tab characters encountered are counted as one character. It is the same as the column number unless hard tab characters exist in the line and ExpandTabs is ON. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Similar to [GotoColumn\(\)](#), except that this command uses a physical position within the current line. Tabs are counted as a single character regardless of tab expansion. Unlike [GotoColumn\(\)](#), pos is required; this command will [not](#) prompt for a parameter. Zero is returned if the position does not change. **Examples:** [GotoPos\(CurrLineLen\(\) + 1\)](#) // Simulates [EndLine\(\)](#) See Also: [CurrPos\(\)](#), [GotoColumn\(\)](#) **Variables:** [ExpandTabs](#)

GotoRow

Moves the cursor to the specified row of the current window.

Syntax: [INTEGER GotoRow\(INTEGER y\)](#) - y is the requested row. The valid range is from 1 to the [return](#) value of [Query\(WindowRows\)](#). Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If the requested row is such that it would position the cursor past the last line in the file, the row of the last line of the file is used. Zero is returned if the cursor is not moved. **Examples:** The [BegWindow\(\)](#) command is simply: [GotoRow\(1\)](#) Likewise, the [EndWindow\(\)](#) command is: [GotoRow\(Query\(WindowRows\)\)](#) To create a key that moves the cursor to the center line of the editing window without scrolling text, define the key as: [GotoRow\(Query\(WindowRows\)/2\)](#) See Also: [ScrollToRow\(\)](#), [ScrollToBottom\(\)](#), [ScrollToCenter\(\)](#), [ScrollToTop\(\)](#), [CurrRow\(\)](#) **Variables:** [WindowRows](#)

GotoWindow

Switches to the requested window.

Syntax: [INTEGER GotoWindow\(\[INTEGER window_id\]\)](#) - window_id is the number of the window to move to. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: window_id must be the number of a currently open window. When more than one window is open, window numbers are shown in the window border. Window numbers are returned from the [WindowId\(\)](#) command. If window_id is not specified, [GotoWindow\(\)](#) prompts for a direction, and moves to the adjacent window in the direction specified by the arrow key that is pressed. Zero is returned if the current window does not change. This command is inoperative when [windows](#) are zoomed. See Also: [WindowId\(\)](#), [NextWindow\(\)](#), [PrevWindow\(\)](#)

GotoXoffset

Sets a new horizontal scroll amount.

Syntax: [GotoXoffset](#)([INTEGER](#) new_Xoffset) - new_Xoffset is the new horizontal scroll amount. Returns: Nothing.
Notes: The following relationship holds for CurrXoffset, CurrCol, and the logical x position (in 1 through [Query](#) ([WindowCols](#))): $x = \text{CurrCol}() - \text{CurrXoffset}()$ See Also: [CurrXoffset\(\)](#), [CurrCol\(\)](#), [GotoColumn\(\)](#) [Variables:](#) [WindowCols](#)

GotoXY

Sets the current video output position to the coordinates specified, relative to the current video output window (as defined by

[Window\(\)](#); [PopWinOpen\(\)](#); or the editor's default [FullWindow\(\)](#), which is the entire screen). The visible [cursor](#) is also set to this position. **Syntax:** [GotoXY](#)([INTEGER](#) x, [INTEGER](#) y) - x is the column coordinate. - y is the row coordinate. Returns: Nothing. **Notes:** This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. The position specified determines where writing performed by the next video output command will begin. The hardware (visible) [cursor](#) is also set to this position. Contrast this with [VGotoXY\(\)](#), which also sets the video output position, but does not change the visible cursor position. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [VGotoXY\(\)](#), [VGotoXYAbs\(\)](#), [VHomeCursor\(\)](#), [WhereX\(\)](#), [WhereXAbs\(\)](#), [WhereY\(\)](#), [WhereYAbs\(\)](#), [VWhereX\(\)](#), [VWhereY\(\)](#)

GUIStartupFlags

Determines how the editor positions and sizes the window at startup, and if and how that position, size, and font are remembered and restored.

This variable has no effect in the Console version of the editor.

Values: [INTEGER](#) Three groups of FLAGS: Group 1 - Startup Window Position: [_USE_DEFAULT_WIN_POS_](#) [_USE_LAST_SAVED_WIN_POS_](#) [_USE_LAST_SESSION_WIN_POS_](#) Group 2 - Startup Window Size: [_USE_DEFAULT_WIN_SIZE_](#) [_USE_LAST_SAVED_WIN_SIZE_](#) [_USE_LAST_SESSION_WIN_SIZE_](#) Group 3 - Startup Font: [_USE_LAST_SAVED_FONT_](#) [_USE_LAST_SESSION_FONT_](#) [Default:](#) [_USE_DEFAULT_WIN_POS_](#) [_USE_LAST_SAVED_FONT_](#) [_USE_LAST_SAVED_WIN_SIZE_](#) [Limits:](#) Has effect at startup only. **Notes:** The above [values](#) can be OR'ed together, however, only one value from each group can be used as part of the OR'ed set. Group 1 - Startup Window Position flags: [_USE_DEFAULT_WIN_POS_](#) [Windows default:](#) Let [Windows](#) determine the window position. [_USE_LAST_SAVED_WIN_POS_](#) Last Saved Settings: When you select "Save Current Settings", the current window position is saved, and that position is used in future sessions, until updated by reselecting "Save Current Settings" again. [_USE_LAST_SESSION_WIN_POS_](#) Last Session: Save the window position at the end of each session, and restore it when the editor starts. Group 2 - Startup Window Size flags: [_USE_DEFAULT_WIN_SIZE_](#) [Windows default:](#) Let [Windows](#) determine the window size. [_USE_LAST_SAVED_WIN_SIZE_](#) Last Saved Settings: When you select "Save Current Settings", the current window size is saved, and that size is used in future sessions, until updated by reselecting "Save Current Settings" again. [_USE_LAST_SESSION_WIN_SIZE_](#) Last Session: Save the window size at the end of each session, and restore it when the editor starts. Group 3 - Startup Font flags: [_USE_LAST_SAVED_FONT_](#) Last Saved Settings: When you select "Save Current Settings", the currently selected font is saved, and that font is used in future sessions, until updated by reselecting "Save Current Settings" again. [_USE_LAST_SESSION_FONT_](#) Last Session: Save the currently selected font at the end of each session, and restore it when the editor starts. It is [not](#) necessary to explicitly set this variable. See Also: [StartupVideoMode](#) Commands: [SetFont\(\)](#), [ChooseFont\(\)](#), [GetFont\(\)](#), [ResizeFont\(\)](#),

[GetCharWidthHeight\(\)](#)

Help

Invokes the context-sensitive HelpSystem for a specified topic.

Syntax: [INTEGER Help](#)([[STRING](#) topic [, [INTEGER](#) index]]) - topic is the optional name of a keyword or topic on which to provide Help. topic defaults to "[Table of Contents](#)" if [not](#) specified. - index has special meaning if the topic does not exist within the Help file, as follows: - If index is 0, the index is not automatically invoked. - If index is 1, the index is automatically invoked. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: From within [the HelpSystem](#), the Table of Contents, an Index, and [Key](#) Assignments may be viewed. In the standard editor setup, Help is assigned to <F1> as follows: <F1> [Help](#)() When <F1> is pressed from editing mode, [the HelpSystem](#) Table of Contents is displayed. **Examples:** To invoke Help on a particular command such as [AddLine](#)(): [Help](#)("AddLine") To invoke Help on the word at the cursor: [proc](#) HelpOnWordAtCursor() [PushBlock](#)() // save the current block [MarkWord](#)() // mark the current word [Help](#)([GetMarkedText](#)(), 1) // Invoke Help on marked // word. If no word is // marked, the index is // displayed. [PopBlock](#)() // restore previous block [end](#) An enhanced version of this macro is supplied in the standard TSE.UI user-interface file (located in the UI\ editor subdirectory). See Also: [PrevHelp](#)(), [SearchHelp](#)()

Helpdef

The helpdef construct is used to define a help screen. With a helpdef, you embed help information directly within a macro so that you do not have to maintain an additional file. This help is displayed on the screen using the

[QuickHelp](#)() command. The format for help definition is: [helpdef](#) help_name [title = constant_string] [width = constant_integer] [height = constant_integer] [x = constant_integer] [y = constant_integer] help_string ... help_string [end](#) [help_name] where: - helpdef specifies this as a help definition. - help_name is a unique name associated with this helpdef. - A number of setup options for the helpdef can optionally be set. These options define the help screen display area. You can include any or all of the following options. The order of these options is not important, but they MUST be declared BEFORE any help_strings are declared. - title indicates the title of the help screen, and is specified as a constant string. It displays within the top border of the help screen, and is centered over the help area. - width indicates the width of the help screen, and is specified as a [constant integer](#). - height indicates the height of the help screen, and is specified as a constant integer. - x indicates the left column boundary of the help screen, and is specified as a constant integer. - y indicates the top boundary of the help screen, and is specified as a constant integer. - help_string is a line of help text. It must be specified as a [constant string](#) enclosed within single or double quotes. Each separate help_string is displayed as a separate line of help in the help screen. Multi-page help screens can be created. - end signifies the end of the help definition. - The help_name may optionally be repeated after the end statement on the same line. The [QuickHelp](#)() command references a helpdef to display multi-page help at the specified coordinates. The user can browse through the defined help text using the cursor movement keys.

HelpLineDelay

Length of time that a key must be held down before a new HelpLine will display.

Values: [INTEGER](#) - [0..18] **Default:** 6 Notes: HelpLineDelay is set in 1/18-second increments. Setting the value to 0

causes [HelpLines](#) to be updated immediately. See Also: [ShowHelpLine](#)

Helplines

[HelpLines](#) display user-defined help associated with Ctrl, Shift, Alt, or no shift [key](#) being pressed. HelpLines appear opposite [the StatusLine](#) on the screen. If [the StatusLine](#) is at the top of the screen, [the HelpLine](#) will appear at the bottom of the screen, and vice versa. Whether or not the defined HelpLines are displayed, is determined by the value of the ShowHelpLine editor variable. HelpLines are assigned in the same fashion as key assignments, and may be placed within keydefs if desired. To declare HelpLines, use the following format: <HelpLine> constant_string <Alt HelpLine> constant_string <Ctrl HelpLine> constant_string <Shift HelpLine> constant_string where constant_string is of the form: "{F1}-Help {F2}-AddLn {F4}-DupLn {F5}-ScrollToTop {F6}-DelToEol {F9}-Shell {F10}-[Menu](#)" Be sure to use single or double quotes as delimiters. Text enclosed within "{}" is highlighted on the screen. [HelpLines](#) may also be placed within any [keydef](#), and automatically take effect [when Enable\(\)](#) is called, and released when [Disable\(\)](#) is called. The standard configuration supplied with the editor includes definitions for all four [HelpLines](#). The unshifted HelpLine (<HelpLine>) displays all of the time (if ShowHelpLine is ON). To display the shifted HelpLines, hold down the appropriate shift key. For example, to display the Ctrl HelpLine, you must hold the <Ctrl> key down for a short period before the help text is displayed. You can use the unshifted HelpLine to have any specified string displayed on the screen at all times. You can use this string to list your key assignments, or any other information you would like to have continually displayed.

HexEdit

Returns, and optionally changes, the current hex-editing status.

Syntax: [INTEGER HexEdit](#)([BOOLEAN new_state]) - new_state is the optional new state for hex-editing (TRUE or FALSE). Returns: The current status of hex-editing (before changing it to new_state, if specified). Notes: When hex-editing is [ON](#) (TRUE is passed to [HexEdit\(\)](#)), and Hex display mode is [ON](#) (_DISPLAY_HEX_ is passed to [DisplayMode\(\)](#)), then the editor allows the user to enter or modify the hexadecimal format of the data in the hexadecimal-display portion of the screen. (See the [DisplayMode\(\)](#) command for information [about](#) Hex display mode.) **Examples:** The following macro will toggle hex-editing: [proc](#) ToggleHexEdit() [if](#) [HexEdit](#)(ON) [HexEdit](#)(OFF) [endif](#) [end](#) A common use would be to extend the use of <Tab> [proc](#) FancyTab() [if](#) [DisplayMode](#)() == _DISPLAY_HEX_ [ToggleHexEdit](#)() [else](#) [TabRight](#)() [endif](#) [end](#) <Tab> FancyTab() See Also: [DisplayMode\(\)](#)

HiByte

Returns the upper byte from the lower 16 bits of a specified integer value.

Syntax: [INTEGER HiByte](#)([INTEGER](#) i) - i is the integer value from which to return the upper byte of the lower 16 bits. Returns: The upper byte of the lower 16 bits of the specified integer value. Notes: SAL utilizes 32-bit integers. [HiByte\(\)](#) returns the upper byte of the lower 16 bits. The value is always interpreted as positive, and will always be in the range 0 to 255. **Examples:** Keycodes returned from [GetKey\(\)](#) are made up of a scan code and the ASCII character of the key. (The ASCII character is 0 (zero) if the key was a function key.) One way to extract the scan code and character code is as follows: [integer](#) key, char_code, scan_code [key](#) = [GetKey](#)() char_code = [key](#) & 0xff scan_code = [key](#) shr 8 A more readable solution would be: [key](#) = [GetKey](#)() char_code = [LoByte](#)(key) scan_code = [HiByte](#)(key) See Also: [LoByte\(\)](#), [HiByte\(\)](#), [LoWord\(\)](#)

HideMouse

Hides the mouse cursor.

Syntax: [HideMouse\(\)](#) Returns: Nothing. Notes: [HideMouse\(\)](#) removes the mouse from the screen and decrements an internal mouse cursor flag. If the mouse is already hidden, the internal flag is simply decremented. To restore the mouse cursor, use [ShowMouse\(\)](#). The position of the mouse and mouse keys pressed are still active when the mouse is hidden. Each call to [HideMouse\(\)](#) should have a corresponding [ShowMouse\(\)](#) at the appropriate location. For example, if [HideMouse\(\)](#) is called 3 times, without any calls to [ShowMouse\(\)](#), then 3 [ShowMouse\(\)](#) calls will be necessary to cause the mouse to re-appear. See Also: [ShowMouse\(\)](#)

HiLiteFoundText

Highlights the text that was found by the most recent find command.

Syntax: [HiLiteFoundText\(\)](#) Returns: Nothing. Notes: For this command to work properly, the cursor must remain at the beginning of the found text. If the cursor is moved to another location, or the found text is scrolled out of the window, then the text that is highlighted will not be the actual text that was found. **Examples:** In the following example, the mFind() procedure will center the found text in the current window. Note that the editor variable, CenterFinds accomplishes the same thing. [integer proc mFind\(STRING find_string, STRING find_opts\) if Find \(find_string, find_opts\) ScrollToRow\(Query\(WindowRows\) / 2\) HiLiteFoundText\(\) return \(TRUE\) endif return \(FALSE\) end](#) See Also: [Find\(\)](#), [lFind\(\)](#), [MarkFoundText\(\)](#), [GetFoundText\(\)](#) Variables: [CenterFinds](#)

HiWord

Returns the upper 16-bit word from a specified integer value.

Syntax: [INTEGER HiWord\(INTEGER i\)](#) - i is the integer value from which to return the upper 16-bit word. Returns: The upper 16 bits of the specified integer value. Notes: SAL utilizes 32-bit integers. [HiWord\(\)](#) returns the upper 16-bit word. The value is always interpreted as positive, and will always be in the range 0 to 65535. See Also: [LoByte\(\)](#), [HiByte\(\)](#), [LoWord\(\)](#)

Hook

Associates a procedure with an event.

Syntax: [Hook\(INTEGER event, PROCEDURE proc_name\)](#) - event is one of the editor's pre-defined events. - proc_name is the name of a procedure to be associated with event. Returns: Non-zero if successful. Zero ([FALSE](#)) if proc_name is already hooked to the same event, or if the maximum number of hooks is reached. Notes: This command gives the user a way to intercept and act on certain editor events. Events that may be hooked **include:** - [_ON_EDITOR_STARTUP_](#) This event is called just before the editor starts the main editing loop. It is useful when you want to gain control after the command line has been processed, the file to edit has been loaded, but the editor has not yet drawn the window or gotten keystrokes from the user. - [_ON_CHANGING_FILES_](#) This event is called each time a different file is switched to, with the following exceptions: - If the file was switched to via the [GotoBufferId\(\)](#)

command, this event is [NOT](#) called. - If the file was switched to using the `_DONT_LOAD_` option of [NextFile\(\)](#) or [PrevFile\(\)](#) commands, this event is [NOT](#) called. See the `OnChangingFiles()` macro in the TSE.UI user-interface file (located in the UI editor subdirectory) for examples of using this event. - `_ON_FIRST_EDIT_` This event is called the first time a file is loaded into the editor. Therefore, it is only called once for each file edited. This event is not called by the [CreateBuffer\(\)](#) or [CreateTempBuffer\(\)](#) commands. When this event is called, the file has already been loaded into the editor. - `_ON_FILE_LOAD_` This event is called immediately before a file is loaded into the editor. It is called before `_ON_FIRST_EDIT_`. - `_ON_FILE_SAVE_` This event is called just before the editor saves a file. It is not called by the [SaveAs\(\)](#) and [SaveBlock\(\)](#) commands. - `_AFTER_FILE_SAVE_` This event is called just after the editor saves a file. It is not called by the [SaveAs\(\)](#) and [SaveBlock\(\)](#) commands. - `_ON_FILE_QUIT_` This event is called just before the editor quits a file, via the single-file quitting commands ([QuitFile\(\)](#), [AbandonFile\(\)](#)). If only a single file is loaded, this hook will not be called unless the `QuitToPrompt` variable is ON. - `_ON_EXIT_CALLED_` This event is called at the beginning of the [Exit\(\)](#) and [SaveAllAndExit\(\)](#) commands. - `_ON_ABANDON_EDITOR_` This event is called just before the editor is terminated. - `_IDLE_` This event is called from the editor's keyboard processing [loop](#), when no keys are available to be processed, AND the editor is in a "text-editing" state (that is, the focus is a text-editing window, as opposed to a prompt, menu, list, or other similar input focus). - `_NONEDIT_IDLE_` This event is called from the editor's keyboard processing [loop](#), when no keys are available to be processed, AND the editor is in a "non-text-editing" state (that is, the focus is a prompt, menu, list, or other similar non-text-editing input focus). - `_BEFORE_UPDATE_DISPLAY_` This event is called before each window is updated. The [UpdateDisplay\(\)](#) command has no effect [if](#) called from this event. Note that if there is more than one window displayed on the screen when [UpdateDisplay\(\)](#) is executed, this event will be called for each window that has been modified. - `_AFTER_UPDATE_DISPLAY_` This event is called after each window is updated. The [UpdateDisplay\(\)](#) command has no effect [if](#) called from this event. Note that if there is more than one window displayed on the screen when [UpdateDisplay\(\)](#) is executed, this event will be called for each window that has been modified. - `_AFTER_UPDATE_STATUSLINE_` This event is called after the `StatusLine` is updated. - `_BEFORE_COMMAND_` This event is called before each text-editing command is processed. - `_AFTER_COMMAND_` This event is called after each text-editing command is processed. - `_BEFORE_NONEDIT_COMMAND_` This event is called before each non-text-editing command is processed. - `_AFTER_NONEDIT_COMMAND_` This event is called after each non-text-editing command is processed. - `_LIST_STARTUP_` This event is called when the [List\(\)](#) and [IList\(\)](#) commands start processing. - `_LIST_CLEANUP_` This event is called when the [List\(\)](#) and [IList\(\)](#) commands terminate. - `_PICKFILE_STARTUP_` This event is called when the [PickFile\(\)](#) command (called explicitly or implicitly by [EditFile\(\)](#) or others) starts processing. - `_PICKFILE_CLEANUP_` This event is called when the [PickFile\(\)](#) command terminates. - `_PROMPT_STARTUP_` This event is called when the [Ask\(\)](#) and [Read\(\)](#) family of commands start processing. - `_PROMPT_CLEANUP_` This event is called when the [Ask\(\)](#) and [Read\(\)](#) family of commands terminate. - `_ON_SELFINSERT_` This event is called after a character is inserted into a text-editing window. - `_ON_DELCHAR_` This event is called after any of the following commands are executed in a text-editing window: [DelChar\(\)](#), [DelRightWord\(\)](#), [DelLeftWord\(\)](#), [BackSpace\(\)](#) - `_PRE_UPDATE_ALL_WINDOWS_` This event is called once at the beginning of the [UpdateDisplay\(\)](#) command, before any other processing takes place, regardless of the number of windows displayed. This event is only called when the editor is in the [main](#) editing [loop](#). - `_POST_UPDATE_ALL_WINDOWS_` This event is called once at the end of the [UpdateDisplay\(\)](#) command processing, regardless of the number of windows displayed. This event is only called when the editor is in the main editing loop. - `_ON_UNASSIGNED_KEY_` This event is called whenever an unassigned key is pressed, AND the editor is in a "text-editing" state (that is, the focus is a text-editing window, as opposed to a prompt, menu, list, or other similar input focus). - `_ON_NONEDIT_UNASSIGNED_KEY_` This event is called whenever an unassigned key is pressed, AND the editor is in a "non-text-editing" state (that is, the focus is a prompt, menu, list, or other similar non-text-editing input focus). - `_BEFORE_GETKEY_` This event is called just before the editor gets another key or mouse button, regardless of whether the editor is in a "text-editing" or "non-text-editing" state. (To determine the current state of the editor, use the [QueryEditState\(\)](#) command.) - `_AFTER_GETKEY_` This event is called just after the editor has gotten another key or mouse button, regardless of whether the editor is in a "text-editing" or "non-text-editing" state. (To determine the current state of the editor, use the [QueryEditState\(\)](#) command.) The following events are available in

the GUI version of the editor, and ignored in the Console version of the editor: - `_GAINING_FOCUS_` Called whenever the editor gains the focus, and the editor is in the main editing loop. - `_LOSING_FOCUS_` Called whenever the editor loses the focus, and the editor is in the main editing loop. - `_NONEDIT_GAINING_FOCUS_` Called whenever the editor gains the focus, and the editor is not in the main editing loop. - `_NONEDIT_LOSING_FOCUS_` Called whenever the editor loses the focus, and the editor is not in the main editing loop. Multiple procedures can be hooked to the same event. If this is the case, the hooked procedures are called in a last-hooked, first-called order. The same procedure can also be hooked by multiple events. The same procedure can not be hooked more than once to the same event, however. The editor prevents hooked events from being recursively called. A hooked event may be removed by calling the [UnHook\(\)](#) command or purging (or re-loading, since loading a macro file will cause an already loaded macro file of the same name to be purged) the macro file containing the hooked procedure. Hooks may be temporarily disabled by using the [BreakHookChain\(\)](#) or [SetHookState\(\)](#) commands. Most editor events are only called when the editor's focus is a text-editing window. Several events (those with NONEDIT in the name, as well as the `_AFTER_GET_KEY_` and `_BEFORE_GET_KEY_` events in the Windows version) are called when the editor's focus is a non-text-editing window, such as a menu, a prompt, or a list. During these events, care must be taken not to disturb the editor state. In particular, the screen should not be updated, and other events must not be called. Typically, the best thing to do in a non-text-editing event is to set a flag that a later text-editing event or normal macro procedure can act upon. If the same procedure is hooked to multiple events, some of which are text-editing events, and some are non-text-editing events, the need may arise to know which state (text-editing versus non-text-editing) the editor is in. This can be determined by the [QueryEditState\(\)](#) command. **Examples:** `proc foo() Warn("I am in the ON_FIRST_EDIT hook!") end proc main() Hook(_ON_FIRST_EDIT_, foo) end` If this macro file is executed, every time a new file (that is, one that is not currently in the editor's Ring of files) is loaded, the `foo()` macro will be executed. See Also: [UnHook\(\)](#), [ExecHook\(\)](#), [BreakHookChain\(\)](#), [GetHookState\(\)](#), [SetHookState\(\)](#), [AbandonFile\(\)](#), [Ask\(\)](#), [CreateBuffer\(\)](#), [CreateTempBuffer\(\)](#), [EditFile\(\)](#), [EditThisFile\(\)](#), [Exit\(\)](#), [List\(\)](#), [NextFile\(\)](#), [PickFile\(\)](#), [PrevFile\(\)](#), [QuitFile\(\)](#), [Read\(\)](#), [SaveAllAndExit\(\)](#), [SaveAs\(\)](#), [SaveBlock\(\)](#), [IList\(\)](#), [QueryEditState\(\)](#), [TerminateEvent\(\)](#) **Variables:** [QuitToPrompt](#)

HWindow

Creates a new editing window, by splitting the current window in half horizontally.

Syntax: [INTEGER HWindow\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If there is not enough room to create the window, or if the maximum number of [windows](#) already exists, no action occurs and zero is returned. The new window becomes the current editing window. If only a single file is loaded, it is shown in the new window at the same cursor position. Otherwise, the next file in the Ring is shown in the new window. To force the [HWindow\(\)](#) command to always load the current file into the new window, use the following key assignment: `<key> if HWindow() PrevFile() endif` See Also: [CloseWindow\(\)](#), [VWindow\(\)](#), [ZoomWindow\(\)](#) **Variables:** [CurrWinBorderAttr](#), [OtherWinBorderAttr](#)

Identifiers

[Identifiers](#) (names) are used to denote [variables](#), procedures, [constants](#), labels and other user-defined language elements. Names consist of letters, digits, and underscores. Names must begin with either a letter or an underscore ("`_`"), and there must be at least one letter or digit in each name. Because pre-defined editor [constants](#) are enclosed within underscore ("`_`") characters, the user is prohibited from creating user-defined names that begin AND end with underscore characters. Names are not case sensitive; for instance, "myvar" is considered the same as "MyVar". Only the first 32 characters of a name are significant. The following are valid names: myvar SomeVar foo apple Orange _digit a43 The following are [not](#) valid names: my var (spaces not allowed as part of a name) 3foo (names must start with a letter or underscore) _ _ _ (names must contain at least one letter or digit) foo\$abc (\$ is not a valid name

character)

IdleTime

Amount of time the editor has been idle, in hundredths of a second.

Values: [INTEGER](#) **Default:** Undefined **Limits:** Can only be queried. Cannot be used within a config/endconfig construct. **Notes:** This variable is intended to be used in a macro procedure that is hooked to the `_IDLE_` or `_NONEDIT_IDLE_` events. It is only valid to query [IdleTime](#) during one of these idle events. **See Also:** [Commands: Hook\(\)](#)

Include Files

Sometimes you may have common information that you would like to

[include](#) in the source of many different macros. These could be constant definitions or even common utility procedures. Or, you might have a large complex macro file that would be easier to grasp if it were divided into logical parts. For these purposes, the include file facility is ideal. The format is: `#include include_filename` or, alternatively: `#include [include_filename]` where: - `include_filename` is the name of the file to include in the source macro. `include_filename` must be specified as a constant string, enclosed within single or double quotes. If the alternate format is used, you **MUST** enclose `include_filename` within the `[ ]` characters. The `[ ]` characters instruct the macro compiler to locate `include_filename` in the same directory as the macro source file (that is, the file in which this include statement occurs). The macro is compiled as if `include_filename` were a part of the macro source file. If `include_filename` is not enclosed within `[ ]`, then the macro compiler looks for the file only in the current directory, if NO path is specified; or in the indicated directory, if a path IS specified. If `include_filename` is enclosed within `[ ]`, then the macro compiler looks for the file only in the directory in which the macro source file is located. For example, assume that `ROUTINES.S` (located in the directory `\TOOLS`) contains some common procedures that are to be used in `MYMACRO.S`. Assuming `MYMACRO.S` is in a different directory than `\TOOLS`, you would use the following format within `MYMACRO.S` to include the source code from `ROUTINES.S` at an appropriate location: ... `#include "c:\tools\routines.s"` ... Alternatively, if `MYMACRO.S` is also located in the directory `\TOOLS`, you could use the following format: ... `#include ["routines.s"]` ...

Incrementalsearch Feature

* `IncrementalSearch` (<Ctrl I>, or "Incremental Search" under Search

[Menu](#)) This command is a special search feature that causes the editor to begin searching for a string while you type. As you enter each new character, the editor attempts to locate a string in the text that matches the incremented search string. If you type a character that would expand the search string to a string that cannot be matched in the text, that character is not added to the search string. When you execute `IncrementalSearch`, the editor displays the following [StatusLine prompt](#): `L # I-Search []`: where "L #" indicates the current line number, and the current search options are identified within `[]`. Enter the string to be located. The cursor remains in the `IncrementalSearch` prompt until you terminate the operation by pressing <Enter> or <Escape>. As you type characters, a forward search is performed, beginning at the text position where the cursor was located when you invoked `IncrementalSearch`. As each matching string is found, the string is highlighted in the text. When a search character is entered that does not match the next character of the currently-located text, the editor searches forward and moves to the first occurrence that does match the newly-expanded search string. If a match including that character cannot be located, the character is rejected and the currently-located text remains highlighted. Within the `IncrementalSearch` prompt, you can use the following keys to perform the operations indicated: - <Backspace> Deletes the last character typed within the `IncrementalSearch`

prompt. The editor re-locates the previously-highlighted string. - <Ctrl N> or <Ctrl L> Searches forward for the next occurrence of the current IncrementalSearch [string](#). - <Ctrl P> or <Ctrl R> or <CtrlShift L> Searches backward for the previous occurrence of the current IncrementalSearch [string](#). - <Ctrl A> Toggles the "all-files" [a] search option. - <Ctrl I> Toggles the "ignore-case" [i] search option. - <Ctrl W> Toggles the "words-only" [w] search option. - <Ctrl 4> Toggles the "anchor to end of line" [\$] search option. - <Ctrl 6> Toggles the "anchor to beginning of line" [^] search option. - <Ctrl B> or <Ctrl G> If there is no search text, the "global" [g] search option (to start at the beginning of the file) is set. Otherwise, searches forward from the beginning of the file for the first occurrence of the current IncrementalSearch string.

InitSynhiCurrFile

Determines if the current file extension has been associated with a SyntaxHilite mapping (.syn) file, and if so, initiates

[SyntaxHilite mode](#) for the current file. This command is intended for ADVANCED USAGE. [Syntax: INTEGER InitSynhiCurrFile\(\)](#) Returns: Non-zero if SyntaxHilite mode was initiated for the current file; [otherwise](#), zero ([FALSE](#)). Notes: This function is called internally when a new file is loaded, or the name of the current file is changed. If the extension of the current file is found in the SyntaxHilite extension-association list, [LinkSynFile\(\)](#) is called with the ".syn" filename associated with this extension. To see the visual effects of SyntaxHilite mode, the [ShowSyntaxHilite](#) variable must be [ON](#). To terminate SyntaxHilite mode for the current file, call [PurgeSynhi\(\)](#). See Also: [PurgeSynhi\(\)](#), [GetSynFilename\(\)](#), [GetSynLanguageType\(\)](#), [LinkSynFile\(\)](#), [LoadSynhiAssoc\(\)](#), [ReplaceSynFile\(\)](#)
[Variables: ShowSyntaxHilite](#)

Insert

Characters will be inserted into the text or overwrite existing text.

[Values:](#) BOOLEAN [Default:](#) [ON](#) Notes: When set [ON](#), existing text will be shifted right to accommodate the new text. When set OFF, any existing text will be overwritten with the new text. For more information on Insert, see the chapter on "[Editing Modes](#)" in [the User's Guide](#). [Affects:](#) [CReturn\(\)](#), [TabRight\(\)](#), [TabLeft\(\)](#), [InsertText\(\)](#) See Also: Commands: [Toggle\(\)](#)

Insert Mode

[Insert mode affects](#) basic text entry, causing new text to be added by shifting existing text, rather than overwriting it. Insert mode also alters the operation of the CopyCharAbove, Literal, BackSpace, CReturn, TabRight, and TabLeft commands. To toggle Insert mode ON and OFF, press the <Ins> key. When Insert mode is ON, an "I" is displayed on [the StatusLine](#). Note that when Insert mode is OFF, the editor is considered to be in overwrite mode. The following table describes differences in the operation of commands affected by Insert mode, distinguishing the effect on the behavior of the commands when [Insert mode](#) is [ON](#) and when it is [OFF](#). COMMANDS AFFECTED BY [INSERT MODE](#)

Command/	Effect on Behavior	Effect on Behavior	Keystroke with Insert ON	with Insert OFF
Any Character,	Inserts text at cursor			
Overwrites existing CopyCharAbove position, shifting text at cursor				
<Ctrl ->, existing text to right. position. Literal				
<Ctrl P> BackSpace As cursor moves left, As cursor moves left, <Backspace> text to right of cursor text to right of				
also shifts left. cursor does NOT shift. CReturn Splits line at cursor Does not split line. <Enter> position, adding new				
Cursor is placed on line. Cursor , along next line at effective with text from and to left margin (see right of cursor ,				
moves " AutoIndent Mode " to new line beginning below). at effective left margin (see " AutoIndent Mode " below).				

TabRight Moves [cursor](#) to next or Moves [cursor](#) to next <Tab>, previous tab stop and or previous tab stop TabLeft adds or removes spaces without altering <Shift Tab> or tab characters, as existing text. appropriate. (For more information on tab handling, see chapter on "[Using Tabs.](#)") +----- Tip -----

+ | If the configuration option, "CReturn Equals Next Line" is ON, then | | Insert mode has no effect on the CReturn command. That is, CReturn | | always behaves as if Insert mode is OFF, and simply causes the | | cursor to go to the next line without splitting the current line. | | (For more information on this option, see | | "CReturn Equals Next Line" under "[Text Entry/Block Options](#)" in | | the chapter on "[Configuring the Editor Interactively.](#)") |

+-----+

InsertCursorSize

Sets the size (height) to make the cursor when in Insert mode. Size is measured in units equal to 1/8th of the maximum cursor height.

Values: [INTEGER](#) - 0 - no visible [cursor](#) - 1..8 - size of [cursor](#) - 9 - A vertical 'bar' (line) [cursor](#). - 255 - [cursor](#) size is [not](#) changed **Default:** 4 **Notes:** 8 gives the maximum cursor size. 1 gives the minimum cursor size. 2..7 give increasing [cursor](#) sizes. Because of differences in video card resolution, the cursor size on different machines may differ slightly. **Affects:** [ToggleInsert\(\)](#) See Also: [Cursor](#), [Insert](#), [OverwriteCursorSize](#)

InsertData

Inserts a previously defined set of data into the current buffer.

Syntax: [INTEGER InsertData](#)(data_name) - data_name is the name of a defined set of data. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. **Notes:** This command fails if there is [not](#) enough memory to successfully complete the operation. For more information on the data identifier, see "[Datadef](#)" in the chapter on "[Using Macro Language Features.](#)" **Examples:** The following example shows how datadef can be used to define a template that gets inserted into the current file when <F12> is pressed. [datadef](#) MainTemplate "[proc Main\(\)](#)" " [integer i](#)" "" "[end main](#)" [end](#) <F12> [InsertData](#)(MainTemplate) See Also: [QuickHelp\(\)](#)

InsertFile

Inserts a file from disk at the current cursor position.

Syntax: [INTEGER InsertFile](#)([STRING](#) filename [, [INTEGER](#) flag]) - filename is the name of the file to insert. - flag is an optional integer that allows you to prevent [InsertFile\(\)](#) from displaying any warning or informational messages. Specify a value of [_DONT_PROMPT_](#) to suppress such messages for any problems that occur during execution; in such case, the command will simply return FALSE. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. **Notes:** This command fails if there is [not](#) enough memory to successfully complete the operation. filename can also include the "[Search SubDirs](#)" (-s) option to obtain a PickList of all files in the indicated directory and all associated subdirectories, that match the filename specification. (For more information on the "[Search SubDirs](#)" option, see "[Command-Line Options](#)" in the chapter on "[Starting the Editor](#)" in [the User's Guide.](#)) If filename is not passed, the editor prompts for the name. Filename completion and the file PickList are available with this command. The inserted file is marked as a character block. This command includes a history list of previous entries passed to it. See [AddHistoryStr\(\)](#) for additional information. See Also: [SaveBlock\(\)](#), [ReplaceFile\(\)](#), [NewFile\(\)](#), [AddHistoryStr\(\)](#)

Inserting the Date and Time Into the Text

* DateTimeStamp ("Date/Time Stamp" under Util

[Menu](#)) This command inserts the current date and time into the current file at the cursor position. The format of the date and time stamp is determined by several configuration options. (For more information on selecting the format for the date and time, see "Date Format," "Date Separator," "Time Format," [and](#) "Time Separator" under "[Command/Format Options](#)" [in](#) the chapter on "[Configuring the Editor Interactively](#).") To place a date/time stamp into a file, position the cursor at the desired location. Then select "Date/Time Stamp" under the Util Menu. The current date and time, in the currently configured format, is inserted into the text at the cursor position, shifting any text to the right of the cursor. Special print options allow you to include the date and/or time in a header or footer when you print a file. (For more information, see "[PrintHeader](#)" and "[PrintFooter](#)" under "[Setting Print Options](#)" in the chapter on "[Printing from the Editor](#).")

InsertLine

Inserts a line before the current line. The cursor is placed on the newly-created line. The

[cursor](#) column does [not](#) change. **Syntax:** [INTEGER](#) InsertLine([[STRING](#) text [, [INTEGER](#) buffer_id]]) - text is an optional string placed on the newly-inserted line. - buffer_id is the optional id number of the file where the line is to be added. If not passed, the line is added to the current file. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command fails if there is [not](#) enough memory to successfully complete the operation. If the optional buffer_id is specified, the line is inserted before the current line in the specified buffer. [_ON_CHANGING_FILES_](#) hooks are [NOT](#) invoked by this command. See Also: [AddLine\(\)](#), [DelLine\(\)](#)

InsertLineBlocksAbove

Place line blocks above or below the current cursor line.

Values: BOOLEAN **Default:** [OFF](#) Notes: [InsertLineBlocksAbove](#) only [affects](#) [_LINE_](#) blocks. When set ON, line blocks are inserted above the current cursor line. The cursor is placed on the first line of the inserted block. Macros that manipulate line blocks should save the value of [InsertLineBlocksAbove](#), set it as desired, and then restore it when the macro is finished. **Affects:** [Paste\(\)](#), [CopyBlock\(\)](#), [MoveBlock\(\)](#)

InsertText

Inserts a string into the current buffer at the current cursor position.

Syntax: [INTEGER](#) InsertText([STRING](#) s [, [INTEGER](#) flag]) - s is the text to be inserted. - flag is optional, and determines whether text will be overlaid or inserted. If [not](#) specified, [_DEFAULT_](#) is used. Supported [values](#) for flag are: - [_DEFAULT_](#) uses the current Insert setting to determine how s is placed in the buffer. - [_INSERT_](#) forces s to be inserted into the buffer. - [_OVERWRITE_](#) forces s to overwrite text at the current position. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: The cursor is placed at the end of the string. **Examples:** /

***** Insert "abc" at the current position if in insert

mode, or overwrite three characters starting at the current [cursor](#) position with "abc".

*****/ [InsertText](#)("abc") // "fox" is inserted at the current position [InsertText](#)("fox",_INSERT_) // "*" overwrite the character at the cursor [InsertText](#)("*",_OVERWRITE_) See Also: [GetText](#)()

InsStr

Inserts a string into another string.

Syntax: [STRING](#) [InsStr](#)([STRING](#) ins_st, [STRING](#) orig_s, [INTEGER](#) p) - ins_st is the string to insert. - orig_s is the string in which to insert ins_st. - p is the position within the orig_s to insert ins_st. Returns: The new string with the specified characters inserted. Notes: If p is out of range (that is, it is less than 0 [or](#) greater than the length of orig_s), nothing is inserted in orig_s. **Examples:** [string](#) s[80] s = "this a test" s = [InsStr](#)(" is", s, 5) // s = "this is a test: See Also: [DelStr](#)(), [SubStr](#)(), [Pos](#)(), [LeftStr](#)(), [RightStr](#)(), [StrCount](#)(), [StrFind](#)(), [StrReplace](#)()

Installation of the Semware Editor Professional

This chapter provides the following information:

-

[Installation steps](#) - [Editor files and directories](#) - [Augmenting the editor load directory](#)

Installation Steps

To install The SemWare Editor Professional on your machine, follow these steps:

- Place the distribution diskette ("DISK 1" if you have multiple diskettes) in Drive A: (or the appropriate floppy drive on your machine).
- Type "A:SETUP"

[and](#) press <Enter>. (If appropriate, replace "A:" with the drive name in which the diskette was placed.) OR, if installing the Console version (TSE Pro/32), Type "A:SETUP32" [and](#) press <Enter>. (If appropriate, replace "A:" with the drive name in which the diskette was placed.) - Follow the instructions displayed on the screen. (To cancel installation at any time, press <Escape>.) - Once installation is successfully completed, remove the distribution diskette from the floppy drive, and store the distribution diskette(s) in a safe place. - Modify your AUTOEXEC.BAT to [include](#) The SemWare Editor Professional directory in your PATH, and reboot your machine. To begin using The SemWare Editor Professional, just type "g32" at the operating system prompt. OR, if the Console version has been installed: To begin using The SemWare Editor Professional/32, just type "e32" at the operating system prompt.

Introduction

This product is The SemWare Editor Professional v4.4 for

[Windows](#) (95/98/Me/NT/2000/XP). This chapter provides the following information: - [The editor design](#) - [The editor features](#) - [The editor manuals](#)

Introduction to External Compiled Macros

In addition to simple Keyboard Macro capabilities, the editor offers a full programming language, the SemWare Applications Language (SAL), for creating more complex Compiled Macros to execute within the editor. These macros can be used to simplify routine editing tasks, to customize commands to behave according to your needs or preferences, and to extend the editor by creating your own editing commands. Because this macro language is fully integrated into the editor, macros you write can be executed just like the editor's own built-in (native) commands.

The editor accommodates two types of Compiled

[Macros](#): external and user-interface. External [macros](#) are created and maintained in separate, independent macro (.MAC) files outside the editor, and must be explicitly loaded into the editor before they can be executed. Further, an external macro, once loaded, can subsequently be purged from an editing session if you no longer need that macro. User-interface macros are placed in a unique user-interface (.UI) macro file and installed in the editor, so that they are implicitly invoked when the editor is started. Such macros effectively become part of the editor; thus, user-interface [macros](#) cannot be purged. The remainder of this chapter describes how to write and use external Compiled Macros. It also includes information on how to maintain a set of external Compiled Macros that are automatically loaded each time the editor is started. (For more information on external and user-interface [macros](#), see "[Customization Using SAL Compiled Macros](#)" in the chapter on "[Fully Customizing the Editor](#)." For detailed information about writing and using Compiled [Macros](#), see [the Macro Reference Guide](#).) [Creating and Using Compiled Macros](#)

----- To create a Compiled Macro, first type a series of macro commands and statements (source code) that are recognized by SAL. Enter the source code in an external macro source file with a filename extension of ".S". You can include one or more macros in a macro source file, distinguishing the macros by assigning each macro a unique name. Once the source code for the macros in the source file is complete, you then compile the macro source code using the SAL Compiler, SC32.EXE. The compiler translates the source code into a format that is executable within the editor. It writes the compiled (executable) version of the macros to an external Compiled Macro file (also called an "object file"). The compiler gives this Compiled Macro file the same filename as the source file, but with an extension of ".MAC", and places the Compiled Macro (.MAC) file in the same directory as the macro source (.S) file. External Compiled Macro files should be maintained in an appropriate directory for subsequent loading into the editor. When you attempt to load an external Compiled Macro file, the editor successively searches directories, according to the following "[macro file search order](#)", to locate the file to load: 1. The current directory 2. The directories indicated by the TSEPath variable, as well as the subdirectory MAC\ (if one exists) of each directory indicated by [TSEPath](#) (For more information on the TSEPath variable, see "Path(s) for Supplemental Files" under "[System/File Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") 3. The editor load directory, as well as the subdirectory MAC\ (if one exists) of the editor load directory It is recommended that you maintain your own external macro files in a dedicated subdirectory. You may wish to create a \TSEPro\USERMAC\ subdirectory in which to keep your .S and .MAC macro files. You should then [include](#) this subdirectory in the TSEPath variable so that the editor can find your Compiled [Macros](#) whenever you execute them. By keeping your custom macro files in their own subdirectory, it will be much easier for you [to](#) manage your [macros](#) and [to](#) later upgrade to newer versions of the editor. Before a Compiled Macro can be executed, the external Compiled Macro (.MAC) file containing the macro must be loaded into the editor. When you load a Compiled Macro file, ALL Compiled [Macros](#) within that macro file are loaded. The individual macros included within a macro file can be macros that you execute directly or [macros](#) that are called by other [macros](#). Special macro functions are available that even allow you to create [macros](#) to serve as your

own menus and help screens. The editor makes the Compiled Macros available for execution according to the assignment you gave to the macro when you created the macro source, as follows:

- If you assigned the name Main to a particular macro within a macro file (one per macro file), that macro is executed each time you "execute" the Compiled Macro file by name. You do this by entering the name of the macro file as the Compiled Macro that you want to execute. This causes the editor to execute the Main macro contained in that Compiled Macro file (if a Main macro is present).
- If you assigned the name WhenLoaded to a particular macro within a macro file (one per macro file), that macro is executed each time you load that Compiled Macro file.
- If you assigned the name WhenPurged to a particular macro within a macro file (one per macro file), that macro is executed whenever you purge that Compiled Macro file.
- If you declared a particular macro as Public within a macro file, you can execute that macro by specifying its own assigned name. Declaring a macro as Public allows you to make that macro available for execution by name. You can declare multiple macros as Public within a macro file.
- If you assigned a particular macro to a key, you can execute that macro by pressing the assigned key.
- If you assigned a particular macro to a menu, you can execute that macro by selecting the associated menu item.

Once the Compiled Macro file is initially loaded, the editor automatically executes the WhenLoaded macro in that file, if one is present. The WhenLoaded macro is executed only one time; specifically, when the macro file containing the WhenLoaded macro is loaded. To make any other Compiled Macros available for execution, the editor does the following:

- For each individual Compiled Macro in the macro file that is explicitly designated as Public, an entry is created in the Public Macros PickList, using each [Public](#) macro's assigned name. This allows that macro to be individually selected for execution, [by](#) its own name. These macro names are listed immediately below the entry for the macro file itself, and are indented.
- An entry is created for the macro file itself in the Public Macros PickList, using the Compiled Macro filename (without the ".MAC" extension). If the macro file contains a Main macro, the Main macro is executed whenever that macro name is selected for execution.
- For each Compiled Macro that is assigned to a key, the editor binds the macro to the indicated key. If that key is already assigned to another command or macro, the key assignment is temporarily replaced. If the new macro is later purged during the editing session, the key resumes its previous function.

Once the macros in an external macro file have been loaded, they can be selected for execution as needed, provided they are a Main or Public macro, or have been assigned to a key or to a menu. When the macros in a macro file are no longer needed within an editing session, the macro file can be purged from memory (which causes the WhenPurged macro, if one is present, to be executed). The following sections provide an overview of commands and options available to compile, load, execute, purge, debug external Compiled Macro files.

[Compiling Macro Source](#) ----- Once the source code for a macro (or group of macros) has been written, the macro source must first be compiled before the macros can be executed within the editor. Use the Compile command to compile the macros within a macro source file. * Compile (<Ctrl F9>, or "Compile" under Macro [Menu](#))* This command compiles a macro file containing one or more macros written in the editor's programming language (SAL). It also allows you to optionally load or execute the resulting Compiled Macro file once the macros are compiled. Compile operates on the current file, which must be a macro source file with the extension ".S" (if you are compiling external macro files). Before compilation begins, the editor saves the current file to disk (if it has been modified). The Compile command is also used to compile user-interface macro files, which have an extension of ".UI". Upon successful compilation of a user-interface file, the user-interface is installed in the editor. (For more information, see "[Installing a User-Interface \(UI\) Macro File](#)" in the chapter on "[Fully Customizing the Editor](#)".) Additionally, the Compile command can be used to compile source files for other programming languages. (For more information, see "[Compiling Programs from Within the Editor](#)" in the chapter on "[Special Commands and Features](#)".) When you execute Compile, the editor saves the current file (if modified), then executes the SAL compiler. For Compile to operate successfully, the SAL compiler (SC32.EXE) must exist either in the current directory, in a directory included in your PATH environment variable setting, or in the editor [load directory](#). When the SAL compiler executes, it reads the macro source code in the current file and attempts to compile it by translating the source code into an executable format. The compiler writes the compiled version of the macros to a Compiled Macro file. The resulting Compiled Macro file is given the same name as the current source (.S) file, but with an extension of ".MAC". If compilation of the macros in the macro source file is successful, the editor creates the .MAC file in the same directory as the current .S file, and displays the following selection menu:

```
+ Compilation Successful... No Errors. + |
*Execute*Macro*****| | Load Macro | | Cancel | +-----+ To load
the newly-compiled macro file, select "Load Macro". If the macro file contains a WhenLoaded macro, the editor
automatically executes it. If you select "Execute Macro", the editor loads the macro file, and automatically executes
```


the WhenLoaded macro (if one is included) and then the [Main macro](#) (if one is included). If you do not wish to load or execute the newly-compiled macro file, select "Cancel" (or press <Escape>). If compilation of the macros is not successful, or the compiler encounters any macro statements for which it issues user warning messages, the following occurs: - A temporary Errors buffer of error and/or warning messages is created. For each line in the macro source file that is in error (or that generated a warning message), a message is placed in the Errors buffer. Each error/warning message identifies the editor's internal message number for the error or warning condition, the line and column in which the error or warning condition occurs (within parentheses), and a short message description. - The editing screen is split horizontally, and the temporary Errors buffer is loaded into the new window. The cursor is placed on the first Error message in the new window. As you move through the Error messages, the source file scrolls so that the corresponding source line is displayed in the source window. - Special key assignments are activated to allow you to move through the Error messages, and to readily switch windows to go to any line in the source file that is identified in an Error message. In the Errors window, you can position the cursor on an Error message, then press <Enter> or <Escape> to switch to the source window and automatically position the cursor at the corresponding source line. Or, you can press <Alt F7> or <Alt F8> to move the cursor to the next or previous error (respectively) in either the Errors buffer or the source file, depending on the window in which you are located. (Press <F1> from the Errors window to display a list of the special Errors key assignments and their associated functions.) In the source file window, you can make necessary corrections to the source. To switch back to the Errors window, use the window-changing commands. Once you have corrected your macro source, you can then re-execute the Compile command. If the compilation is successful, the Errors buffer is cleared. The Compile options and Errors functions are also available from the [Compile menu](#). To pop up this [menu](#), press <Shift F9>. From this [menu](#), you can execute the Compile command, select the next or previous error, switch between the source and Errors windows (which re-opens the Errors window if it is currently closed), close the Errors window, re-open the Errors window, and display Help on the special Errors key assignments. (For more information on the Compile command and using the Compile menu, see "[Compiling Programs from within the Editor](#)" in the chapter on "[Special Commands and Features](#)." For more information on compiling a macro file outside the editor, see "[Compiling Macros](#)" in the chapter on "[Using Macros](#)" in the [Macro Reference Guide](#).)

[Loading and Executing Compiled Macros](#) ----- Before an external Compiled Macro can be executed, it must be loaded into the editor. You load a Compiled Macro by loading the Compiled Macro (.MAC) file in which the macro is included. To load an external macro file once you are in the editor, you can use the LoadMacro or ExecMacro commands. Using the AutoLoadList command, you can optionally specify a set of external macro files that are to be automatically loaded each time you start the editor. When you load a macro file, the WhenLoaded macro in that file (if one is included) is immediately executed. For other Compiled Macros within the macro file, you can use [the ExecMacro command](#) to execute a Public or Main macro by name; or you can press the designated key to execute a macro assigned to a key; or you can choose the appropriate menu selection to execute a macro assigned to a menu. [The LoadMacro Command](#) * LoadMacro ("Load" under Macro [Menu](#))* This command loads a user-specified external Compiled Macro file into the current editing session. The WhenLoaded macro (if one is present in the macro file) is automatically executed. All [macros](#) within that macro file are made available for execution; any Main or Public macros in the file are added to the Public Macros PickList; and any key assignments specified in the macro file are activated. When you execute LoadMacro, the editor displays the prompt "[Load macro](#):". Enter the name of the external macro file to be loaded. If no extension is specified, ".MAC" is assumed. To locate the macro file that you specify, the editor searches for the file, as follows: - If you specify a path for the macro file at the "[Load macro](#):" prompt, the editor searches only in the indicated directory. - However, if you do not specify any path information at the "[Load macro](#):" prompt, the editor successively searches directories according to the "[macro file search order](#)" described in the previous section, "[Creating and Using Compiled Macros](#)." If the indicated macro file cannot be located, the command fails and issues [the StatusLine](#) message, "File [not](#) found:macro filename Press <Escape>". When you press <Escape>, the editor re-displays the "[Load macro](#):" prompt. You can enter a different macro filename, or press <Escape> again to cancel the LoadMacro command. If desired, you can obtain a PickList of external Compiled Macro files at the "[Load macro](#):" prompt by either of the following methods: - Press <Enter> on an empty "[Load macro](#):" prompt. This causes the editor to search for the first directory it can find that contains files with an extension of ".MAC". The editor searches directories according to its "[macro file search order](#)" (described in "[Creating and Using Compiled Macros](#)").

In the first directory in which any such files are located, the editor displays a PickList of the .MAC files in that directory. - Or, enter an ambiguous filename specification at the "[Load macro:](#)" prompt. The editor searches directories according to its "[macro file search order](#)" and displays a PickList of matching files from the first directory in which it finds any matching files. With either of these two methods, if no matching files are found, the editor displays the [the StatusLine](#) message, "File [not](#) found:macro filename Press <Escape>". When you press <Escape>, the editor re-displays the "[Load macro:](#)" prompt. [The ExecMacro Command](#) * ExecMacro (<Ctrl X>, or "Execute" under Macro [Menu](#)) * This command executes a user-specified Compiled Macro, if a Main or Public macro with the indicated name is already loaded. Otherwise, if a macro by that name is not already loaded, this command causes the editor to first issue [the LoadMacro command](#), using the indicated name as a macro file name. This instructs the editor to load the specified external macro file, and execute the WhenLoaded macro in the file (if one is present). The editor then executes the Main macro in the file (if one is included). This command also allows you to pass a command-line string to the macro that is executed. A command-line string, if included, must follow the name of the macro to be executed (separated by a space). When you execute ExecMacro, the editor displays the prompt "[Execute macro:](#)". Enter the name of the macro to be executed. To enter an optional command-line string, follow the macro name by a space and then the desired command-line text. The editor then processes your macro entry as follows: - If the macro name that you enter matches that of a Main or Public macro already loaded and included in the Public Macros PickList, the editor executes the indicated macro. - If a macro by that name is not found in the Public Macros PickList, the editor assumes the indicated macro name is an external Compiled Macro file (with an extension of ".MAC" if no extension is specified). It then attempts to load that macro file by issuing [the LoadMacro command](#) (see "[The LoadMacro Command](#)" above). If the macro file is successfully loaded, the WhenLoaded macro (if present in the file) is executed, and the editor then executes the Main macro in the file (if included). If a macro file with the indicated name is not located, the editor issues the prompt, "File [not](#) found:macro filename Press <Escape>". When you press <Escape>, the editor re-displays the "[Execute macro:](#)" prompt. You can enter a different macro filename, or press <Escape> again to cancel [the ExecMacro command](#). - If a command-line string is included at the "[Execute macro:](#)" prompt, the editor sets the MacroCmdLine variable to the specified string. The macro that is executed then has access to this variable. (For more information on passing a command line to a macro, see "[Using Command Lines in Macros](#)" in the chapter on "[Using Macro Language Features](#)" in the Macro Reference Guide.) If desired, you can obtain a PickList of Main and Public macros that are currently loaded and available for execution. To do so, press <Enter> on an empty "[Execute macro:](#)" prompt. The entries (if any) in the Public Macros PickList is displayed. To execute a macro from this list, place the cursor bar over the desired macro and press <Enter>. Press <Escape> to cancel the ExecMacro command. If you press <Enter> on an empty "[Execute macro:](#)" prompt and there are no external Compiled Macro files currently loaded, the editor issues the LoadMacro command, and attempts to display a PickList of external macro files on disk. To do this, the editor searches for the first directory it can find that contains files with an extension of ".MAC". The editor searches directories according to its "[macro file search order](#)" (described in "[Creating and Using Compiled Macros](#)"). In the first directory in which any such files are located, the editor displays a PickList of the .MAC files in that directory. If no matching files are found in any of the directories that the editor searches, the editor displays the [the StatusLine](#) message, "File not found:macro filename Press <Escape>". When you press <Escape>, the editor re-displays the "[Execute macro:](#)" prompt. Enter a different macro filename specification, or press <Escape> again to cancel [the ExecMacro command](#).

+-----+ Tip -----+ | External Compiled Macro files can also be loaded, or loaded and || executed, when the editor is started. (For more information, see || "[Load Macro](#)" and "[Execute Macro](#)" under "[Command-Line Options](#)" || in the chapter on "[Starting the Editor](#).") |

+-----+ [The AutoLoadList Command](#) You may wish to have certain external Compiled Macro files loaded automatically each time you start the editor. To accomplish this, you can maintain a list of those macro files in a special autoload file, named [TSELOAD.DAT](#), using [the AutoLoadList command](#). * AutoLoadList ("AutoLoad List" under Macro [Menu](#)) This command allows you to maintain a list of external Compiled Macro files to be automatically loaded on editor startup. When you execute this command, the editor displays the current list of macro files in the autoload file (TSELOAD.DAT). Press <Ins> to add a macro file to the list; press on an entry to delete it from this list. To add a new macro file to the autoload list, execute AutoLoadList. Once the current list of macro files is displayed, position the cursor bar on the existing macro filename

after which you want the new macro file to be loaded. Then press <Ins>. The editor issues the prompt, "Macro filename:", for the name of the macro file you want to add. Enter the name of the Compiled Macro file, including any necessary path information. The editor assumes an extension of ".MAC", so it is not necessary to enter the extension. When your response is complete, press <Enter>. The editor then updates the autoloading file to include the indicated macro file. To remove a macro file from the autoloading list, move the cursor bar to the macro file entry in the list, and press . The editor prompts you to confirm that you want to delete that macro file. Upon your confirmation, the editor removes that macro file from the autoloading list. The autoloading file, TSELOAD.DAT, is maintained in the editor load directory. The macro files included in this file are automatically loaded each time you start the editor. The editor loads the macro files, in the order in which they are listed, by issuing [the LoadMacro command](#) for each macro file. If path information is included as part of a macro file entry, the editor searches for that macro file in the indicated directory when it attempts to load it. If path information is not included in the entry, the editor successively searches directories according to the "[macro file search order](#)" described in an earlier section, "[Creating and Using Compiled Macros](#)." For any macro file that the editor cannot locate, the editor skips any further processing of that file, and attempts to load the next macro file in the list. [Purging Compiled Macros](#) ----- External Compiled Macro files that are no longer needed during an editing session can be purged. Once a macro file is purged, each of the individual macros within that file is purged and the memory allocated to the macros is released. To purge an external macro file, use the PurgeMacro command. * PurgeMacro ("Purge" under Macro [Menu](#))* This command purges a user-specified external Compiled Macro file from the current editing session. If a WhenPurged macro is present in the macro file, it is executed. Memory allocated to the macro file is then released, and all Compiled Macros included in the file are removed. Any key assignments defined by the macros are also removed, and the affected key assignments are returned to their previous functions. When you execute PurgeMacro, the editor displays a "Purge Macro" PickList of the external macro files that are currently loaded. (The macro filenames are listed without an extension.) To purge one of the files, move the cursor bar to the desired macro file, and press <Enter>. The selected file, and all macros contained in the file, are purged from memory. [Debugging Compiled Macros](#) ----- While developing a Compiled Macro, you may find it helpful to debug the macro using the macro debugger. The debugger allows you to examine the execution of a macro to verify proper execution or to help determine the cause of problems while executing. To invoke the macro debugger, use the Debug command. * Debug ("Debug" under Macro [Menu](#))* This command activates the macro debugger, allowing you to execute a macro with control of its execution passed to you. When you execute Debug, the editor first determines the macro source file to debug. If the file you are currently editing has an extension of ".S", Debug operates on that file; otherwise, Debug issues the prompt, "Macro source to debug:", to request the name of the macro source file to debug. The selected macro source is compiled, if necessary, and then execution of that macro begins; however, control of the executing macro is passed to you. You can then step through execution of the macro, display the contents of affected files and variables, and even modify the contents of variables as the macro executes. (For more information on using the macro debugger, see "[Debugging Macros](#)" in the chapter on "[Using Macros](#)" in [the Macro Reference Guide](#).) * As an alternative to using the pull-down Menu to execute certain Compiled Macro commands, press <Ctrl F10> to pop up the following selection menu: +---- Macro [Menu](#) -----+ |*Execute...*<Ctrl*X>| | Load... | | Purge... | | Compile <Ctrl F9> | | Debug... | +-----+

is32BitApp

Determines whether a program is a 32-bit executable program, and if so, whether it is a GUI or console program.

Syntax: [INTEGER is32BitApp](#)([STRING](#) pgm_name) - pgm_name is the name of the program to check. If an extension is not specified, .exe is assumed. If a path is not specified, a standard Windows search is used to find the file. Returns: Zero (FALSE) if pgm_name could not be found, or is not a 32-bit executable program; [otherwise](#), either _GUI_32BIT_ or _CONSOLE_32BIT_, as appropriate. Notes: Certain [macros](#), especially those that are running external programs, may need to know if the program in question is a 32-bit executable or a non-32-bit executable. For instance, the Compile() command uses this to determine the optimal way to run (and tee the output) of the compiler in

question. [Examples](#): [is32BitApp\("e32"\)](#) // returns _CONSOLE_32BIT_, if e32 is in // the path [is32BitApp\("g32"\)](#) // returns _GUI_32BIT_, if g32 is in // the path [is32BitApp\("notepad"\)](#) // returns _GUI_32BIT_, if notepad is // in the path See Also: [Dos\(\)](#), [IDos\(\)](#), [isFullScreen\(\)](#), [WhichOS\(\)](#)

isAlpha

Determines whether a string, or the current character, is alphabetic (a through z, or A through Z).

Syntax: BOOLEAN [isAlpha](#)([[STRING](#) s]) - s is an optional string. If s is not specified, the character at the cursor position is examined. Returns: Non-zero if the current or specified character or string is alphabetic; [otherwise](#), zero ([FALSE](#)). Notes: If s is specified, then if any character in s is [not](#) alphabetic, the return value is FALSE. [Examples](#): [isAlpha\(\)](#) // returns [TRUE](#) if current character alphabetic [isAlpha\("abcd"\)](#) // returns [TRUE](#) [isAlpha\("12"\)](#) // returns [FALSE](#) [isAlpha\("a2c"\)](#) // returns [FALSE](#) [isAlpha\("a "\)](#) // returns [FALSE](#) [isAlpha\(""\)](#) // returns [FALSE](#) See Also: [isAlphaNum\(\)](#), [isDigit\(\)](#), [isHexDigit\(\)](#), [isLower\(\)](#), [isUpper\(\)](#), [isWord\(\)](#), [isWhite\(\)](#)

isAlphaNum

Determines whether a string, or the current character, is alphanumeric.

Syntax: BOOLEAN [isAlphaNum](#)([[STRING](#) s]) - s is an optional string. If s is not specified, the character at the cursor position is examined. Returns: Non-zero if the current or specified character or string is alphanumeric; [otherwise](#), zero ([FALSE](#)). Notes: If s is specified, then if any character in s is [not](#) alphanumeric, the return value is FALSE. This command is implemented via [isAlpha\(\)](#) and [isDigit\(\)](#). [Examples](#): [isAlphaNum\(\)](#) // returns [TRUE](#) if current character alphanumeric [isAlphaNum\("abcd"\)](#) // returns [TRUE](#) [isAlphaNum\("12"\)](#) // returns [TRUE](#) [isAlphaNum\("abc123"\)](#) // returns [TRUE](#) [isAlphaNum\("a!c123"\)](#) // returns [FALSE](#) [isAlphaNum\("a!c1.3"\)](#) // returns [FALSE](#) [isAlpha\("a "\)](#) // returns [FALSE](#) [isAlpha\(""\)](#) // returns [FALSE](#) See Also: [isAlpha\(\)](#), [isDigit\(\)](#), [isHexDigit\(\)](#), [isLower\(\)](#), [isUpper\(\)](#), [isWord\(\)](#), [isWhite\(\)](#)

isAutoLoaded

Determines if a macro is in the editors AutoLoad list.

Syntax: BOOLEAN [isAutoLoaded](#)([[STRING](#) macro_fn]) - macro_fn is the optional name of the macro. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If macro_fn is supplied, [isAutoLoaded\(\)](#) checks to see if that macro is in the editors AutoLoad list. Otherwise, [isAutoLoaded\(\)](#) checks to see if the currently running macro is in the editors AutoLoad list. [Examples](#): // determines if the currently executing macro is AutoLoaded [if isAutoLoaded\(\)](#) ... // determines if a specific macro is AutoLoaded [if isAutoLoaded\("cuamark"\)](#) ... See Also: [AddAutoLoadMacro\(\)](#), [DelAutoLoadMacro\(\)](#), [EditAutoLoadList\(\)](#)

isBlockInCurrFile

Determines whether a block is marked in the current file or buffer and, if so, its type.

Syntax: [INTEGER isBlockInCurrFile\(\)](#) Returns: Zero (FALSE) if there is not a marked block in the current file or buffer. If there is one, the return value is one of the following: - `_INCLUSIVE_` for an inclusive character block - `_NONINCLUSIVE_` for a non-inclusive character block - `_LINE_` for a line block - `_COLUMN_` for a column block
See Also: [isBlockMarked\(\)](#), [isCurrLineInBlock\(\)](#), [isCursorInBlock\(\)](#) **Variables:** [BlockId](#), [Marking](#)

isBlockMarked

Determines whether a block is currently marked in any file or buffer.

Syntax: [INTEGER isBlockMarked\(\)](#) Returns: Zero (FALSE) if there is not a marked block in any file or buffer. If there is a marked block, the return value is one of the following: - `_INCLUSIVE_` for an inclusive character block - `_NONINCLUSIVE_` for a non-inclusive character block - `_LINE_` for a line block - `_COLUMN_` for a column block
See Also: [isBlockInCurrFile\(\)](#), [isCurrLineInBlock\(\)](#), [isCursorInBlock\(\)](#) **Variables:** [BlockId](#), [Marking](#)

isBookMarkSet

Determines whether the specified bookmark is set.

Syntax: [INTEGER isBookMarkSet\(\[STRING\]\(#\) bookmark\)](#) - bookmark is the single-character name (A through Z, case-insensitive) of the bookmark being verified. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command is typically used just prior to the [GetBookMarkInfo\(\)](#) command to verify that a particular bookmark exists.
Examples: See the [GetBookMarkInfo\(\)](#) command. See Also: [PlaceMark\(\)](#), [GotoMark\(\)](#), [DelBookMark\(\)](#), [GetBookMarkInfo\(\)](#)

isCharDevice

Determines whether or not a file handle refers to a character device.

Syntax: [INTEGER isCharDevice\(\[INTEGER\]\(#\) handle\)](#) - handle is the device for which to determine the type. Returns: Non-zero if handle represents a character device; otherwise, zero ([FALSE](#)). Notes: [isCharDevice\(\)](#) determines whether handle is associated with a character device, such as a terminal, console, printer, or serial port. If [isCharDevice\(\)](#) returns FALSE, and handle is a valid handle, you can assume that handle is associated with a block device (for example, a disk device). **Examples:** // is stdin redirected? [if isCharDevice\(_STDIN_\)](#) // no [else](#) // yes [endif](#) See Also: [fDup\(\)](#), [fDup2\(\)](#)

isCurrLineInBlock

Determines whether the current line is in a marked block and, if applicable, the type of block.

Syntax: [INTEGER isCurrLineInBlock\(\)](#) Returns: Zero (FALSE) if the current line is not in a marked block. If it is in a marked block, the return value is one of the following: - `_INCLUSIVE_` for an inclusive character block -

`_NONINCLUSIVE_` for a non-inclusive character block - `_LINE_` for a line block - `_COLUMN_` for a column block
 Notes: This command is similar to the [isCursorInBlock\(\)](#) command. See Also: [isBlockMarked\(\)](#), [isBlockInCurrFile\(\)](#), [isCursorInBlock\(\)](#) Variables: [BlockId](#), [Marking](#)

isCursorInBlock

Determines whether the cursor is inside a marked block and, if so, the type of block.

Syntax: [INTEGER isCursorInBlock\(\)](#) Returns: Zero (FALSE) if the cursor is not in a marked block. If it is in a marked block, the return value is one of the following: - `_INCLUSIVE_` for an inclusive character block - `_NONINCLUSIVE_` for a non-inclusive character block - `_LINE_` for a line block - `_COLUMN_` for a column block
 See Also: [isBlockMarked\(\)](#), [isBlockInCurrFile\(\)](#), [isCurrLineInBlock\(\)](#) Variables: [BlockId](#), [Marking](#)

isDigit

Determines whether a string, or the current character, is numeric (0 through 9).

Syntax: [BOOLEAN isDigit\(\[STRING s\]\)](#) - s is an optional string. If s is not specified, the character at the cursor position is examined. Returns: Non-zero if the current or specified character or string is composed of numeric digits; [otherwise](#), zero ([FALSE](#)). Notes: If s is specified, then if any character in s is [not](#) numeric, the return value is FALSE.
Examples: [isDigit\(\)](#) // returns TRUE if current character is a digit [isDigit\("abcd"\)](#) // returns [FALSE](#) [isDigit\("12"\)](#) // returns [TRUE](#) [isDigit\("abc123"\)](#) // returns [FALSE](#) [isDigit\("1.1234"\)](#) // returns [FALSE](#) [isDigit\("1 "\)](#) // returns [FALSE](#) [isDigit\(""\)](#) // returns [FALSE](#) See Also: [isAlpha\(\)](#), [isAlphaNum\(\)](#), [isHexDigit\(\)](#), [isLower\(\)](#), [isUpper\(\)](#), [isWord\(\)](#), [isWhite\(\)](#)

isFullScreen

Determines if the editor is running in a window, or in Windows full-screen console mode.

Syntax: [INTEGER isFullScreen\(\)](#) Returns: Non-zero ([TRUE](#)) if in full-screen console mode; [otherwise](#), zero ([FALSE](#)).
 Notes: Certain Windows API's do not work correctly, or can cause the editing session to lose the focus, if called in full-screen mode. This command can be used to help prevent that from happening. See Also: [is32BitApp\(\)](#), [WhichOS\(\)](#)

isGUI

Determines if the GUI or Console version of the editor is running.

Syntax: [INTEGER isGUI\(\)](#) Returns: Zero (FALSE) if the Console version of the editor is running; [otherwise](#), [TRUE](#) is returned. **Examples:** // Example autoload macro to load a macro that allows use // of the mouse wheel in TSE Pro/32 console version // (Support is already present in the GUI version of TSE Pro) [proc](#) whenloaded() [if not isGUI\(\)](#) // [if not](#) GUI version, load mswheel macro [LoadMacro\("mswheel"\)](#) [endif](#) [PurgeMacro\(CurrMacroFilename\(\)\)](#) [end](#) See Also:

[is32BitApp\(\)](#), [WhichOS\(\)](#), [isFullScreen\(\)](#)

isHexDigit

Determines whether a string, or the current character, is composed of hexadecimal characters (0 through 9, a through f, or A through F).

Syntax: BOOLEAN [isHexDigit](#)([[STRING](#) s]) - s is an optional string. If s is not specified, the character at the cursor position is examined. Returns: Non-zero if the current or specified character or string is composed of hexadecimal characters; [otherwise](#), zero ([FALSE](#)). Notes: If s is specified, then if any character in s is not a hexadecimal character, the return value is FALSE. The hexadecimal characters are 0 through 9, a through f, and A through F. **Examples:** [isHexDigit](#)() // returns [TRUE](#) if current character is hex [isHexDigit](#)("abcd") // returns [TRUE](#) [isHexDigit](#)("12") // returns [TRUE](#) [isHexDigit](#)("abc123") // returns [TRUE](#) [isHexDigit](#)("1.1234") // returns [FALSE](#) [isHexDigit](#)("1 ") // returns [FALSE](#) [isHexDigit](#)("") // returns [FALSE](#) See Also: [isAlpha\(\)](#), [isAlphaNum\(\)](#), [isDigit\(\)](#), [isLower\(\)](#), [isUpper\(\)](#), [isWord\(\)](#), [isWhite\(\)](#)

isKeyAssigned

Determines if the specified key has an assignment.

Syntax: BOOLEAN [isKeyAssigned](#)([INTEGER](#) key_code) - key_code is the integer value for a key. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: key_code is in the range 0 to 65535. To obtain information about a specific key, you can use the <key-name> format, as in: <Ctrl F1>. **Examples:** // determine if CtrlAltShift SpaceBar has an assignment [if isKeyAssigned](#)(<CtrlAltShift SpaceBar>) ... // a macro to create a list of all keys that are // assigned/unassigned [proc main](#)() [integer](#) i [string](#) s[255] [CreateBuffer](#)("key buffer") [for](#) i = 0 [to](#) 65535 s = [KeyName](#)(i) [if](#) s <> "" [and](#) s[1:2] <> "0x" [and](#) i == [KeyCode](#)(s) [if isKeyAssigned](#)(i) [AddLine](#)([Format](#)("assigned ", i:10; s)) [else](#) [AddLine](#)([Format](#)("unassigned ", i:10; s)) [endif](#) [endif](#) [endfor](#) [end](#) See Also: [GetKey\(\)](#), [KeyCode\(\)](#), [KeyName\(\)](#)

isLower

Determines whether a string, or the current character, is composed of lower-

[case](#) alphabetic characters (a through z). **Syntax:** BOOLEAN [isLower](#)([[STRING](#) s]) - s is an optional string. If s is not specified, the character at the cursor position is examined. Returns: Non-zero if the current or specified character or string is a lower-[case](#) alphabetic character; [otherwise](#), zero ([FALSE](#)). Notes: If s is specified, then if any character in s is not a lower-case alphabetic character, the return value is FALSE. **Examples:** [isLower](#)() // returns [TRUE](#) if current character is lower-[case](#) [isLower](#)("abcd") // returns [TRUE](#) [isLower](#)("aBcd") // returns [FALSE](#) [isLower](#)("12") // returns [FALSE](#) [isLower](#)("abc123") // returns [FALSE](#) [isLower](#)("1.1234") // returns [FALSE](#) [isLower](#)("a ") // returns [FALSE](#) [isLower](#)("") // returns [FALSE](#) See Also: [isAlpha\(\)](#), [isAlphaNum\(\)](#), [isDigit\(\)](#), [isHexDigit\(\)](#), [isUpper\(\)](#), [isWord\(\)](#), [isWhite\(\)](#)

isMacroLoaded

Used to determine if a particular Compiled Macro is loaded.

Syntax: [INTEGER isMacroLoaded](#)([STRING](#) macro_file) - macro_file is the name of the Compiled Macro file being verified as loaded. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Used to determine if a particular Compiled Macro is loaded into the editor. macro_file can be an explicitly declared Public name, or an implicitly declared Public name (such as the actual macro_file filename). If a path is given, then that particular macro must be loaded for this command to [return TRUE](#). For instance, if "c:\tse\other\mymac.mac" is loaded (via [LoadMacro\(\)](#)), then [isMacroLoaded](#)("c:\tse\mac\mymac.mac") returns [FALSE](#). However, [isMacroLoaded](#)("mymac") returns [TRUE](#). See Also: [ExecMacro\(\)](#), [ExecLoadedMacro\(\)](#), [LoadMacro\(\)](#), [PurgeMacro\(\)](#)

isTrailingSlash

Determines if a string contains a trailing slash character.

Syntax: [INTEGER isTrailingSlash](#)([STRING](#) path) - path is the string (typically a directory path) to check for the presence of a trailing slash. Returns: Non-zero if path is terminated with a slash; otherwise, zero ([FALSE](#)). Notes: This function works with both [forward](#) slashes (/) [and](#) backslashes (\). **Examples:** Assume a string variable, fn. [if isTrailingSlash](#)(fn) [Message](#)("Filename ends in a slash") [endif](#) See Also: [AddTrailingSlash\(\)](#), [RemoveTrailingSlash\(\)](#)

isTypeableKey

Determines if a key is a typeable key.

Syntax: [INTEGER isTypeableKey](#)([INTEGER](#) key) - key is the key to check. Returns: Non-zero if key is a typeable key; otherwise, zero ([FALSE](#)). Notes: key should be a value returned from the [GetKey\(\)](#) command. Characters with an ASCII value less than 32 decimal are not considered typeable keys. **Examples:** [INTEGER](#) key key = [GetKey\(\)](#) [if isTypeableKey](#)(key) [Message](#)("Key is typeable") [endif](#) See Also: [GetKey\(\)](#)

isUpper

Determines whether a string, or the current character, is composed of upper -

[case](#) alphabetic characters (A through Z). **Syntax:** [BOOLEAN isUpper](#)([[STRING](#) s]) - s is an optional string. If s is not specified, the character at the cursor position is examined. Returns: Non-zero if the current or specified character or string is an upper-[case](#) alphabetic character; [otherwise](#), zero ([FALSE](#)). Notes: If s is specified, then if any character in s is not an upper-case alphabetic character, the return value is [FALSE](#). **Examples:** [isUpper](#)() // returns [TRUE](#) if current character is upper [case isUpper](#)("abcd") // returns [FALSE](#) [isUpper](#)("aBCd") // returns [FALSE](#) [isUpper](#)("12") // returns [FALSE](#) [isUpper](#)("ABC123") // returns [FALSE](#) [isUpper](#)("1.1234") // returns [FALSE](#) [isUpper](#)("ABC") // returns [TRUE](#) [isUpper](#)("A ") // returns [FALSE](#) [isUpper](#)("") // returns [FALSE](#) See Also: [isAlpha\(\)](#), [isAlphaNum\(\)](#), [isDigit\(\)](#), [isHexDigit\(\)](#), [isLower\(\)](#), [isWord\(\)](#), [isWhite\(\)](#)

isWhite

Determines whether a string, or the character at the current position, is

composed of

[white characters](#) (space or tab). **Syntax:** [INTEGER isWhite](#)([\[STRING s\]](#)) - s is an optional string to check for the presence of white characters. If s is not specified, the character at the cursor position is examined. Returns: Non-zero if the current character or specified string consists only of spaces and/or tab characters; [otherwise](#), zero ([FALSE](#)). Notes: If s is specified, and any character in s is not a white character (space or tab), then zero is returned. If s is not specified, zero is returned at or past the [end](#)-of-line position. **Examples:** [isWhite\(\)](#) // returns [TRUE](#) if current character is white [isWhite\(" "\)](#) // returns [TRUE](#) [isWhite\("a b"\)](#) // returns [FALSE](#) [isWhite\(""\)](#) // returns [FALSE](#) See Also: [isAlpha\(\)](#), [isAlphaNum\(\)](#), [isDigit\(\)](#), [isHexDigit\(\)](#), [isLower\(\)](#), [isUpper\(\)](#), [isWord\(\)](#)

isWildPath

Determines if a string contains "?" or "*" characters.

Syntax: [INTEGER isWildPath](#)([STRING path](#)) - path is the string to check. Returns: Non-zero if path contains wildcard characters; [otherwise](#), zero ([FALSE](#)). **Examples:** Assume a string variable, fn. [if isWildPath](#)(fn) [Message](#)("File contains either '*' or '?' or both") [endif](#) See Also: [isTrailingSlash\(\)](#)

isWinClipAvailable

Determines if there is any data in the Windows Clipboard that can be pasted into the editor.

Syntax: [INTEGER isWinClipAvailable](#)() Returns: Zero ([FALSE](#)) if there is not suitable data in the Windows Clipboard; If there is a block, the return value is one of the following: - [_INCLUSIVE_](#) for an inclusive character block - [_NONINCLUSIVE_](#) for a non-inclusive character block - [_LINE_](#) for a line block - [_COLUMN_](#) for a column block Notes: The primary use for this command is [to](#) disable, and "gray out," appropriate menu selections related to pasting from the Windows Clipboard, if there is no suitable data in the Clipboard. See Also: [PasteFromWinClip\(\)](#), [PasteReplaceFromWinClip\(\)](#), [CopyAppendToWinClip\(\)](#), [CutToWinClip\(\)](#), [CutAppendToWinClip\(\)](#) [Paste\(\)](#), [PasteReplace\(\)](#), [Copy\(\)](#), [CopyAppend\(\)](#), [Cut\(\)](#), [CutAppend\(\)](#) [CopyBlock\(\)](#), [DelBlock\(\)](#), [MoveBlock\(\)](#) [EmptyWinClip\(\)](#), [isBlockMarked\(\)](#), [isBlockInCurrFile\(\)](#)

isWord

Determines whether the characters in a string, or the character at the cursor position, are in the currently-defined or specified WordSet.

Syntax: [INTEGER isWord](#)([\[STRING s \[, STRING bit_encoded_32_char\]\]](#)) - s is an optional string of characters to check against the WordSet characters. If s is not specified, the character at the cursor position is examined. - [bit_encoded_32_char](#) is an optional bit-encoded 32-character string denoting a user-defined WordSet. Use the [ChrSet\(\)](#) command to generate this [string](#). Returns: Non-zero if the current character or specified string consists only of characters in the WordSet; otherwise, zero ([FALSE](#)). Notes: If s is specified, and any character in s is not in the [WordSet](#), then zero is returned. If s is not specified, zero is returned at or past the [end](#)-of-line position. **Examples:** [isWord\(\)](#) // returns TRUE if current character is in the WordSet [isWord\(" "\)](#) // returns TRUE if a space is in the WordSet [isWord\("a b"\)](#) // returns TRUE if "a b" are all in the WordSet [isWord\("a b", ChrSet\("0-9"\)\)](#) // returns [FALSE](#)

since "a b" are // not in the WordSet comprised // of the characters 0 through 9. [isWord\(""\)](#) // returns [FALSE](#) See Also: [isWhite\(\)](#), [WordRight\(\)](#), [WordLeft\(\)](#), [BegWord\(\)](#), [EndWord\(\)](#), [isAlphaNum\(\)](#), [isDigit\(\)](#), [isHexDigit\(\)](#), [isLower\(\)](#), [isUpper\(\)](#), [isAlpha\(\)](#), [MarkWord\(\)](#), [GetWord\(\)](#) Variables: [WordSet](#)

isZoomed

Determines if the current window is Zoomed.

Syntax: [INTEGER isZoomed\(\)](#) Returns: Non-zero if the current window is Zoomed; otherwise, zero ([FALSE](#)). Notes: Use this command to determine if the current window is zoomed. Some operations require the user to be in a non-zoomed window in order to use them. See Also: [ZoomWindow\(\)](#)

JoinLine

Joins the next line to the end of the current line.

Syntax: [INTEGER JoinLine\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If the combined length of the current line and the next line is greater than the editor's maximum line length, only that portion of the next line that fits is joined to the current line. If the cursor is beyond the end of line position, the line is padded with spaces to the cursor position before the join occurs. If the cursor is within the line or at the end of line position, no additional space is inserted at the point where the lines are joined. If the current line is already equal to the maximum line length or is the last line in the file, no action occurs and zero is returned. See Also: [SplitLine\(\)](#)

KbdMacroRecording

Determines whether a Keyboard Macro is currently being recorded.

Values: BOOLEAN **Default:** Undefined **Limits:** Can be queried, or set [OFF](#); cannot be set [ON](#). Cannot be used within a config/endconfig construct. See Also: [KbdMacroRunning](#) Commands: [RecordKeyMacro\(\)](#), [PressKey\(\)](#)

KbdMacroRunning

Determines whether a Keyboard Macro is running.

Values: BOOLEAN **Default:** Undefined **Limits:** Can be queried, or set [OFF](#); cannot be set [ON](#). Cannot be used within a config/endconfig construct. See Also: [KbdMacroRecording](#) Commands: [RecordKeyMacro\(\)](#), [PressKey\(\)](#)

KbdPath

Used to specify where the editor loads and stores Keyboard Macros when using the

[LoadKeyMacro\(\)](#) and [SaveKeyMacro\(\)](#) commands. **Values:** [STRING](#) (maximum of 65 characters) **Default:** "" Notes: Unlike TSEPath, KbdPath can contain only a single path. **Affects:** [LoadKeyMacro\(\)](#), [SaveKeyMacro\(\)](#)

KeepUndoBeyondSave

Determines whether Undo information is kept or purged when a file is saved.

[Values](#): BOOLEAN [Default](#): [OFF](#) Notes: Normally, when a file is saved, previously-accumulated Undo information for that file is purged. Thus, you cannot [Undo\(\)](#) any changes made to a file that occurred prior to the file being saved. If you want to be able to [Undo\(\)](#) changes made prior to the file being saved, then set this variable [ON](#). [Affects](#): [Undo\(\)](#), [Redo\(\)](#)

KeepWinOnTop

Determines whether the editor window behaves normally, or stays on top of all other

[windows](#). [Values](#): BOOLEAN [Default](#): [OFF](#)

Key

The key or mouse key that was pressed at the time the last

[GetKey\(\)](#) was called, either internally by the editor or by a user-written macro. [Values](#): [INTEGER](#) [Default](#): Undefined Notes: Whenever [GetKey\(\)](#) is called and a mouse key is pressed, [MouseKey](#), [MouseX](#), and [MouseY](#) are also updated with the current information. To separate a key into its character code and scan code, use the following: [integer key](#), character_code, scan_code [key](#) = [GetKey\(\)](#) character_code = [key](#) & 0xff scan_code = [key shr](#) 8 or alternatively: [integer](#) character_code, scan_code [GetKey\(\)](#) // sets [Key](#) character_code = [Query\(Key\)](#) & 0xff scan_code = [Query\(Key\) shr](#) 8 [GetKey\(\)](#) is used internally by the editor to process keyboard and mouse input from the user. Thus, [Query\(Key\)](#) can be used to determine the last key that was pressed. See Also: [LastKey](#), [MouseKey](#), [MouseX](#), [MouseY](#) Commands: [GetKey\(\)](#)

Key Assignments Displayed on the Menu

Most commands that are accessible from the pull-down Menu are also pre-assigned to keys. These commands can be executed by pressing the assigned key without using the Menu. Once you become familiar with the key assignment for a command, you can press that key to quickly and easily execute the command.

For those commands in the Menu that have key assignments, the key assignment is included on the sub-menu. It is enclosed within < > and displayed to the right of the command. For those items in the Menu that allow you to change an editing option, the key assignment (if there is one) is not shown. Instead, the Menu displays the current setting of the option within [].

Some commands have multiple key assignments; however, only one of the key assignments is included on the Menu. For example, [DelRightWord](#) is assigned to <Ctrl Del> and to <Ctrl T>, but the Text sub-menu displays only <Ctrl Del>.

If you reconfigure the key assignments for commands that are displayed on the Menu, the Menu is automatically updated. (For more information on configuring key assignments, see "

[Adding or Changing Key Assignments](#)" in the chapter on "[Fully Customizing the Editor](#).")

Keyboard Macros

[Keyboard Macros](#) allow you to save keystrokes and execute them as needed. A Keyboard Macro is a series of keystrokes that are recorded as you type. The series of keystrokes is assigned to a key of your choice. By pressing the assigned key, the Keyboard Macro can be re-played, causing the commands and operations executed by those keystrokes to be repeated, just as if the keystrokes were individually re-typed on the keyboard. Keyboard Macros can be recorded for use only in the current editing session (and automatically discarded when the editor is terminated), or they can be saved for use in later editing sessions. A single Keyboard Macro can be as large as 64k in size. Up to twenty [Keyboard Macros](#) can be active at any one time in a session. Because [Keyboard Macros](#) record your actual keystrokes, any changes made to your keyboard configuration may affect the results when a Keyboard Macro is re-played. Thus, the behavior of Keyboard Macros may be affected by command key assignments, as well as by the setting of various modes. For example, if Insert mode was ON while a Keyboard Macro was being recorded, but is turned OFF at the time the Keyboard Macro is executed, the macro may not behave as anticipated. [Recording Keyboard Macros](#) ----- To create a Keyboard Macro, you must first turn MacroRecord mode ON. The editor prompts you for the key to which the Keyboard Macro is to be assigned, and then begins recording the keystrokes that you enter. Once you have completed the series of keystrokes, turn MacroRecord mode OFF. To toggle MacroRecord mode ON and OFF, use the RecordKeyMacro command. * RecordKeyMacro (<Ctrl M>, or "Record [Key Macro](#)" under Macro [Menu](#)) This command toggles MacroRecord mode [ON](#) and [OFF](#). Execute this command to initially turn MacroRecord [ON](#), and then execute it again (when macro recording is complete) to turn MacroRecord [OFF](#). When MacroRecord mode is ON, an "R" displays on [the StatusLine](#). (On many monitors, the "R" flashes while this mode is active.) To begin recording a Keyboard Macro, execute RecordKeyMacro to activate MacroRecord mode. The editor prompts for a key to which to assign the Keyboard Macro, with the following: Assign to [key](#): (<Enter> for scrap, <Escape> [to](#) cancel) To cancel MacroRecord at this point, press <Escape>. Otherwise, press the key to which that macro is to be assigned. This must be one of the single keys that can be configured. Note that combining <Shift>, <Alt>, and/or <Ctrl> with one other key is still considered a single key (such as <Ctrl A>). If the key you selected for assignment of the Keyboard Macro is already assigned to another macro or to a command, the editor verifies whether you want to override the existing key assignment for that key with the following [selection menu](#): + Overwrite existing command? + *Yes*****|| No || Cancel | +-----+ (If you override the key assignment of another Keyboard Macro, the other Keyboard Macro cannot be restored.) Once you indicate the desired key to which to assign the Keyboard Macro, the editor begins recording your subsequent keystrokes. Your editing session appears as usual during this time, except that the letter "R" displays (and possibly flashes, depending upon your monitor) on [the StatusLine](#). To end macro recording, turn MacroRecord [OFF](#) by re-executing RecordKeyMacro. As a handy option for recording and re-playing a Keyboard Macro, the editor offers a Scrap Macro Buffer. You assign a macro to the Scrap Macro Buffer by pressing <Enter> at the "Assign to key:" prompt. To execute the Scrap Macro, press <Ctrl Enter>, or choose "Run Scrap Macro" under the Macro Menu. (See "[Re-playing Keyboard Macros](#)" below.) When a Macro is assigned to the Scrap Macro Buffer, it replaces any macro which may have been previously assigned to this area. [Re-Playing Keyboard Macros](#) ----- To re-play, or execute, a Keyboard Macro, press the assigned key. To re-play a Scrap Macro, execute the ExecScrapMacro command. * ExecScrapMacro (<Ctrl Enter>, or "Run Scrap Macro" under Macro [Menu](#)) This command executes a Scrap Macro (a Keyboard Macro assigned to <Ctrl Enter>). To execute Keyboard Macros that were created and saved in a previous editing session, those macros must first be loaded into the current editing session. (See "[Loading Keyboard Macros Saved in a Previous Session](#)" later in this chapter.) Remember that executing a Keyboard Macro simply re-plays the actual

keystrokes that were recorded. Changes that you have made to the keyboard configuration, to edit mode settings (such as [Insert](#)), or to command behavior, may possibly affect the results of the Keyboard Macro. For certain macros, the position of the cursor in the file may also have an effect on the macro's operation. [Saving Keyboard Macros for Use in a Later Session](#) ----- Unless you specifically save the Keyboard Macros that are

recorded during an editing session, they are discarded once you exit the editor. To save your Keyboard Macros for use in future sessions, you must write them to a Keyboard Macro file. The editor convention for the filename extension for Keyboard Macro files is ".KBD". To create a Keyboard Macro file, use the SaveKeyMacro command. *

SaveKeyMacro ("Save [Key](#) Macro" under Macro [Menu](#)) This command saves the current set of Keyboard Macros by writing them to a user-specified file on disk. The Keyboard Macro file that is saved contains the Keyboard Macros recorded during the current editing session, along with any others that were loaded (and not purged) during the same session. (See "[Loading Keyboard Macros Saved in a Previous Session](#)" below.) When you execute SaveKeyMacro, the editor asks for the name of the Keyboard Macro file to which to write with the prompt, "Save key-macros to file:". Enter the desired filename. If the filename extension is omitted, the editor assigns the extension ".KBD". If the filename you specify includes path information, the Keyboard Macro file is written to the specified directory. If you specify only a filename (with no path), the editor writes the file to the directory indicated by the KbdPath variable. (For more information on [KbdPath](#), see "Path for Keyboard Macro Files" under "[System/File Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") If the filename you specify already exists in the applicable directory, the file is overwritten (overlaid) with the current set of Keyboard Macros. However, before overlaying the file, the editor prompts for verification with the following selection menu: + Overwrite existing file? + |

*Yes*****| No | | Cancel | +-----+ Select "Yes" to overwrite the existing file with the current set of Keyboard Macros. Select "No" to cause the editor to re-display the "Save key-macros to file:" prompt, allowing you to enter another filename. Select "Cancel" to cancel the SaveKeyMacro command without writing a new Keyboard Macro file. [Loading Keyboard Macros Saved in a Previous Session](#)

----- To use Keyboard Macros that were recorded and saved in a previous session, you must first load the Keyboard Macro file containing those macros. To do this, execute the LoadKeyMacro command. * LoadKeyMacro ("Load [Key](#) Macro" under Macro [Menu](#)) This command loads a user-specified Keyboard Macro file from disk, making the Keyboard Macros in that file available in the current editing session. When you load a Keyboard Macro file, any and all other Keyboard Macros that were previously recorded or loaded during the current editing session, are automatically PURGED by the editor. That is, when you load a Keyboard Macro file, the editor first purges all current Keyboard Macros, and then loads the new set of Keyboard Macros into the session. (See "[Purging Keyboard Macros from the Current Session](#)" below for more information on the effects of purging.) When you execute LoadKeyMacro, the editor prompts you for the name of the Keyboard Macro file to load with "[Key](#)-macro file to load:". Enter the desired Keyboard Macro filename. If you omit the filename extension, the editor assumes ".KBD". If the filename you specify includes path information, the editor attempts to load the Keyboard Macro file from the specified directory. If you specify only a filename (with no path), the editor first attempts to load the file from the current directory. If the specified filename does not exist in the current directory, then the editor attempts to load the file from the directory indicated by the KbdPath variable. (For more information on the [KbdPath](#) variable, see "Path for Keyboard Macro Files" under "[System/File Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") In addition to loading a Keyboard Macro file from within the editor, you can also load one when you start the editor. For more information, see "[Load Keyboard Macro](#)" under "[Command-Line Options](#)" in the chapter on "[Starting the Editor](#)." +----- Tip -----+ | To add to or change Keyboard Macros in an existing Keyboard Macro | | file: | | | 1. Load that Keyboard Macro file. | | 2. Record any new or modified [Keyboard](#)

[Macros](#). | | 3. Save the new Keyboard Macro file. | +-----+

[Purging Keyboard Macros from the Current Session](#) ----- The Keyboard Macros that have been recorded or loaded during an editing session can be removed (purged) from the session using the PurgeKeyMacro command. * PurgeKeyMacro ("Purge [Key](#) Macro" under Macro [Menu](#)) This command purges [Keyboard Macros](#) from your current editing session. ALL Keyboard Macros that were recorded or loaded during the current session are removed from memory. Once purged, each key to which a Keyboard Macro was assigned no longer causes that macro to be re-played. When you execute PurgeKeyMacro, the editor immediately removes all Keyboard Macros from the editing session. Any keys affected by the Keyboard Macros now resume their previous function.

KeyCode

Converts a key name to an integer value.

Syntax: [INTEGER KeyCode](#)([STRING](#) key_name) - key_name is the key name to be converted to its integer value. Returns: The [integer](#) value of key_name. Notes: key_name should be any valid editor key name, in the format "editor key-name" (as shown by ShowKey). The [KeyCode\(\)](#) and [KeyName\(\)](#) commands are complementary functions. **Examples:** [integer key key](#) = [KeyCode](#)("Enter") See Also: [GetKey\(\)](#), [KeyName\(\)](#), [isKeyAssigned\(\)](#)

Keydef

The

[keydef](#) construct defines [key](#) definitions which may be dynamically enabled and disabled within a macro using the [Enable\(\)](#) and [Disable\(\)](#) commands. Declare a key definition as follows: [keydef](#) keydef_name <key_list> command ... <key_list> command [end](#) [keydef_name] where: - keydef specifies this as a key definition. - keydef_name is a unique name associated with this keydef. - key_list is a key (or two-key) name. - command is a native or user-written command. - end specifies the end of the key definition. - The keydef_name may optionally be repeated after the end statement on the same line. When a particular keydef is enabled via the [Enable\(\)](#) command, the keys and their associated functions are activated when the specified key is pressed. For example, to allow using the WordStar-style cursor keys for line drawing, the following keydef could be used: [keydef](#) LineKeys <Ctrl D> [LineDraw](#)(_RIGHT_) <Ctrl S> [LineDraw](#)(_LEFT_) <Ctrl E> [LineDraw](#)(_UP_) <Ctrl X> [LineDraw](#)(_DOWN_) <F1> [Disable](#)(LineKeys) [end](#) <F1> [Enable](#)(LineKeys) <F1> could be used to temporarily enable and disable line drawing on the [cursor](#) keys. A special [PromptKeys keydef](#) is also available to allow you to automatically enable a key so that it executes one of the limited commands available within the editor's command prompts. The following section describes use of the [PromptKeys keydef](#). (For more information on using keydefs, see the descriptions of the [Enable\(\)](#) and [Disable\(\)](#) commands in the chapter on "[Editor Commands](#)." For more information on assigning commands to keys in the editor, see "[Adding or Changing Key Assignments](#)" in the chapter on "[Fully Customizing the Editor](#)" in [the User's Guide](#).)

KeyName

Converts an integer keycode to a key name.

Syntax: [STRING KeyName](#)([INTEGER](#) key_code) - key_code is the integer value for a key. Returns: The [key](#) name of key_code. Notes: The [KeyCode\(\)](#) and [KeyName\(\)](#) commands are complementary functions. **Examples:** [string keyst](#) [40] keyst = [KeyName](#)([GetKey](#)()) See Also: [GetKey\(\)](#), [KeyCode\(\)](#), [isKeyAssigned\(\)](#)

KeyPressed

Determines if a key or mouse button has been pressed.

Syntax: [INTEGER Keypressed](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: [KeyPressed\(\)](#) returns non-zero if there is a key waiting at the top of the key stack. Keys are placed on the key stack by any of the

following methods: - pressing a key at the keyboard - pressing a mouse button - pushing a key on the stack with the [PushKey\(\)](#) command If the key stack is empty, then [KeyPressed\(\)](#) will return zero. See Also: [GetKey\(\)](#), [PushKey\(\)](#), [PushKeyStr\(\)](#), [PressKey\(\)](#), [GetKeyFlags\(\)](#), [WaitForKeyPressed\(\)](#)

KillBlock

Deletes the marked block if it exists in the current file.

Syntax: [INTEGER KillBlock\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: [KillBlock\(\)](#) will not delete a block outside the current file. If no block is marked in the current file or there was not enough memory to complete the operation, no action occurs and zero is returned. This command is similar to the [DelBlock\(\)](#) command, except that deleted blocks are not stored in the Deletion Buffer, and can not be retrieved once deleted. Use with caution. If you want to save the deleted block in the Deletion Buffer, use the [DelBlock\(\)](#) command instead. See Also: [DelBlock\(\)](#), [isBlockInCurrFile\(\)](#), [isBlockMarked\(\)](#), [KillLine\(\)](#), [KillToEol\(\)](#) **Variables:** [BlockId](#)

KillFile

Erases the current file from disk.

Syntax: [INTEGER KillFile\(\)](#) Returns: Non-zero if the current file could be erased from disk; otherwise, zero ([FALSE](#)). Notes: The modified flag for the current buffer is set to TRUE. See Also: [EraseDiskFile\(\)](#)

KillLine

Deletes the current line.

Syntax: [INTEGER KillLine\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Upon deletion of the current line, the cursor is placed on the following line with the cursor column remaining unchanged. Similar to [DelLine](#), except that the deleted line is not placed in the Deletion Buffer. See Also: [KillToEol\(\)](#), [DelLine\(\)](#), [KillBlock\(\)](#)

KillLocation

Removes the last

[PushLocation\(\)](#) from the saved location stack, without going to that location. **Syntax:** [INTEGER KillLocation\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: The editor maintains a stack of up to 100 saved locations that can be saved, restored, and deleted with the [PushLocation\(\)](#), [PopLocation\(\)](#) and [KillLocation\(\)](#) commands. If there are no locations to remove from the location stack, no action occurs and zero is returned. It is considered a logic error to "push" a location on the stack with [PushLocation\(\)](#) and to not remove it with either [KillLocation\(\)](#) or [PopLocation\(\)](#). Note that each [PopLocation\(\)](#) or [KillLocation\(\)](#) removes one location from the stack. Locations are different than positions (used by [PushPosition\(\)](#), [PopPosition\(\)](#), and [KillPosition\(\)](#)). Positions maintain a relative position within a file, adjusting for insertions and deletions. Locations maintain an absolute location within a file, with no adjustments for insertions or deletions. For example, using locations, if one pushes a location (via [PushLocation\(\)](#)) at line 15, and subsequently deletes lines 1, 2, and 3; when the location is visited (via [PopLocation\(\)](#)), line 15 will be made the current line. This is because locations only maintain a line and column number. Locations do

not track insertions or deletions. On the other hand, using positions, if one pushes a position (via [PushPosition\(\)](#)) at line 15, and subsequently deletes lines 1, 2, and 3; when the position is visited (via [PopPosition\(\)](#)), line 12 (rather than line 15) will be made the current line. This is because positions remain on the line and column on which they were originally set. Positions track insertions and deletions, to keep the position set at the proper place. See Also: [PushLocation\(\)](#), [PopLocation\(\)](#), [PushPosition\(\)](#), [PopPosition\(\)](#), [KillPosition\(\)](#)

KillMax

The maximum number of items that can be stored in the Deletion Buffer.

Values: [INTEGER](#) - [0..1000] **Default:** 1000 **Notes:** Setting [KillMax](#) to 0 prevents deleted items from being stored in the Deletion Buffer. **Affects:** [Undelete\(\)](#), [GlobalUndelete\(\)](#), [PasteUndelete\(\)](#)

KillPosition

Removes the last

[PushPosition\(\)](#) from the saved position stack, without going to that position. **Syntax:** [INTEGER KillPosition\(\)](#) **Returns:** Non-zero if successful; zero ([FALSE](#)) on failure. **Notes:** The editor maintains a stack of up to 10 saved positions that can be saved, restored, and deleted with the [PushPosition\(\)](#), [PopPosition\(\)](#) and [KillPosition\(\)](#) commands. If there are no positions to remove from the position stack, no action occurs and zero is returned. It is considered a logic error to "push" a position on the stack with [PushPosition\(\)](#) and to not remove it with either [KillPosition\(\)](#) or [PopPosition\(\)](#). Note that each [PopPosition\(\)](#) or [KillPosition\(\)](#) removes one position from the stack. See Also: [KillLocation\(\)](#), [GetPositionInfo\(\)](#), [SwapPosition\(\)](#), [PushPosition\(\)](#), [PopPosition\(\)](#), [PushLocation\(\)](#), [PopLocation\(\)](#), [PlaceMark\(\)](#), [GotoMark\(\)](#)

KillToEol

Deletes the text from the cursor position to the end of the current line.

Syntax: [INTEGER KillToEol\(\)](#) **Returns:** Non-zero if successful; zero ([FALSE](#)) on failure. **Notes:** The character at the cursor position is included with the text that is deleted. This command does not cause lines to be joined. This command is very similar to the [DelToEol\(\)](#) command, except that the deleted text is not saved in the Deletion Buffer. See Also: [KillLine\(\)](#), [DelToEol\(\)](#), [KillBlock\(\)](#)

LastKey

The key or mouse key that was pressed when the next to last

[GetKey\(\)](#) was called, either internally by the editor or by a user-written macro. **Values:** [INTEGER](#) **Default:** Undefined **Limits:** Cannot be used within a config/endconfig construct. **Notes:** This variable can be used to see if the same key has been pressed twice in succession. See the Key variable for additional information. See Also: [Key](#), [MouseKey](#), [LastMouseKey](#), [LastMouseX](#), [LastMouseY](#) **Commands:** [GetKey\(\)](#)

LastMouseKey

The mouse key that was pressed when the next to last

[MouseStatus\(\)](#) was called, either internally by the editor or by a user-written macro. [Values:](#) [INTEGER](#) [Default:](#) Undefined Limits: Cannot be used within a config/endconfig construct. Notes: This variable can be used to see if the same mouse key has been pressed twice in succession. See the [MouseKey](#) variable for additional information. See Also: [MouseKey](#), [Key](#), [LastKey](#), [LastMouseX](#), [LastMouseY](#) Commands: [MouseStatus\(\)](#)

LastMouseX

The column number (relative to the entire display) of the mouse cursor the next to last time

[GetKey\(\)](#) was called for a mouse key, or [MouseStatus\(\)](#) was called to obtain the current mouse status. [Values:](#) [INTEGER](#) [Default:](#) Undefined Limits: Cannot be used within a config/endconfig construct. See Also: [MouseX](#), [LastMouseY](#), [LastMouseKey](#) Commands: [MouseStatus\(\)](#), [GetKey\(\)](#)

LastMouseY

The row number (relative to the entire display) of the mouse cursor the next to last time

[GetKey\(\)](#) was called for a mouse key, or [MouseStatus\(\)](#) was called to obtain the current mouse status. [Values:](#) [INTEGER](#) [Default:](#) Undefined Limits: Cannot be used within a config/endconfig construct. See Also: [MouseY](#), [LastMouseX](#), [LastMouseKey](#) Commands: [MouseStatus\(\)](#), [GetKey\(\)](#)

IDos

Passes a command to the operating system for execution.

This command is intended for ADVANCED USAGE.

[Syntax:](#) [INTEGER](#) [IDos](#)([STRING](#) cmd, [STRING](#) cmdline [, [INTEGER](#) flags]) - cmd is the command to pass to the operating system. - cmdline is the command line to pass to cmd. - flags is an optional integer value. Supported values for flags are: - [_DONT_CLEAR_](#) prevents the editing screen from being cleared. - [_DONT_PROMPT_](#) prevents the prompt, "Press a key to continue", from being issued. - [_RETURN_CODE_](#) causes the return code of cmd to be trapped, so that it can be returned via the [DosIOResult\(\)](#) command. - [_TEE_](#) causes the output of cmd to be displayed on the screen, regardless of whether the output is redirected. - [_RUN_DETACHED_](#) causes cmd to be run as a detached process. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Multiple flags values can be combined with the bitwise logical OR (|) operator. If cmd does not include the complete path, the operating system will search for it. The editing screen is saved and restored, as long as the number of rows and columns can be restored after calling the operating system. [Examples:](#) The implementation of the editor's [Shell\(\)](#) command is: [IDos](#)([GetEnvStr](#)("COMSPEC"), "") The implementation of the editor's [Dos\(\)](#) command is: [IDos](#)([GetEnvStr](#)("COMSPEC"), " /c " + cmdline) To create a shell that has a 4k environment space: [IDos](#)([GetEnvStr](#)("COMSPEC"), " /e:4096") To run the SAL compiler program directly, without loading a copy of the command processor: [IDos](#)("sc32.exe", "macro.s") To run a program as a separate process (that is, one that is not connected to the current editing session): [IDos](#)("notepad.exe", "some.file", [_RUN_DETACHED_](#)) See Also: [Dos\(\)](#), [Shell\(\)](#), [StartPgm\(\)](#)

Left

Moves the cursor one character to the left.

Syntax: [INTEGER Left\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command does not wrap to the previous line when the cursor is in column 1. Use the [PrevChar\(\)](#) command to wrap to the previous line at column 1. Zero is returned when this command is executed with the cursor in column 1. See Also: [Right\(\)](#), [NextChar\(\)](#), [PrevChar\(\)](#)

LeftMargin

This is the left margin used when the editor creates a new line.

Values: [INTEGER](#) - [0..16,000] **Default:** 0 (in other words, none) Notes: See the [CReturn\(\)](#) and [WrapPara\(\)](#) commands for details on the effect of the LeftMargin setting. [AutoIndent](#) takes precedence over [LeftMargin](#) when [CReturn\(\)](#) (normally bound to the <Enter> key) is invoked. That is, if AutoIndent is ON or STICKY, and a LeftMargin is set, [CReturn\(\)](#) will ignore the left margin. For [CReturn\(\)](#) to use the left margin, [AutoIndent](#) must be turned [OFF](#). **Affects:** [CReturn\(\)](#), [WrapPara\(\)](#) See Also: [RightMargin](#), [AutoIndent](#), [WordWrap](#)

LeftStr

Returns the specified left-most characters of a string.

Syntax: [STRING LeftStr](#)([STRING](#) s, [INTEGER](#) len) - s is the string from which the characters are to be retrieved. - len is the number of characters to retrieve, starting from the left end of the string. Returns: The [string](#) characters requested. Notes: This command is a convenient short-cut for the slice feature, when you need to get the left-most len characters of a string. The entire [string](#) will be returned if len is greater than or equal to the length of s. **Examples:** To return the n leftmost characters of s: s2 = [LeftStr](#)(s, n) Using slices: s2 = s[1:n] Given: [string](#) abc[11] = "catastrophe" then: [LeftStr](#)(abc, 3) //returns "cat" See Also: [RightStr\(\)](#), [Pos\(\)](#), [DelStr\(\)](#), [InsStr\(\)](#), [SubStr\(\)](#), [StrCount\(\)](#), [StrFind\(\)](#), [StrReplace\(\)](#)

Length

Returns the length of the passed string.

Syntax: [INTEGER Length](#)([STRING](#) s) - s is the string whose length is to be measured. Returns: The length of s. See Also: [SizeOf\(\)](#)

IFind

Performs a low-level

[Find\(\)](#). **Syntax:** [INTEGER IFind](#)([STRING](#) find_str, [STRING](#) opt_str) - find_str is the string to search for. - opt_str

contains the options to use in the search. Returns: Non-zero if the string was found; otherwise, zero (FALSE). Notes: This command is specifically designed for macro use. It should not be bound to a key by itself. It allows a macro to find strings without all of the normal screen updating associated with the higher-level [Find\(\)](#) command; thus, potentially giving the macro a more polished look when invoked. As a side effect, in a macro performing many finds, the macro will actually run faster. Unlike the normal [Find\(\)](#), this command: - does not prompt for input; the parameters are required. - does not highlight the found string. - does not do any setup for the [RepeatFind\(\)](#) command. This command can be used in conjunction with the [lRepeatFind\(\)](#) command, to search multiple times. Note that this command has no effect on the normal [Find\(\)](#) and [RepeatFind\(\)](#) commands, and can be used in [macros](#) without changing the former's histories and/or current targets and options. For a list of options supported by [lFind\(\)](#), see the [Find\(\)](#) command. See Also: [lRepeatFind\(\)](#), [lReplace\(\)](#), [Find\(\)](#)

LineDraw

Draws the appropriate line-drawing character at the cursor location then moves in the direction specified.

Syntax: [LineDraw](#)(direction) - direction specifies the direction to move the cursor and may be one of the following values: - [_LEFT_](#) - [_RIGHT_](#) - [_UP_](#) - [_DOWN_](#) Returns: Nothing. Notes: [LineDraw\(\)](#) is useful for creating boxes and designs with line-drawing characters in the macro language. The type of line character that will be drawn is determined by the current line type, which is set using the [LineTypeMenu\(\)](#) command. **Examples:** This example draws a box in the current file that looks like a folder.

```
proc mLineDrawN(INTEGER dir, INTEGER len) integer i = len while i > 0 LineDraw(dir) i = i - 1 endwhile end proc Main() mLineDrawN(\_RIGHT\_, 10) LineDraw(\_DOWN\_) mLineDrawN(\_RIGHT\_, 15) mLineDrawN(\_DOWN\_, 5) mLineDrawN(\_LEFT\_, 25) mLineDrawN(\_UP\_, 6) LineDraw(\_RIGHT\_) end
```

 See Also: [LineTypeMenu\(\)](#) **Variables:** [LineDrawChar](#), [LineDrawType](#)

LineDrawChar

The ASCII character to be used when

[LineDrawing](#) is [ON](#) and [LineDrawType](#) is set to the user-defined character. **Values:** [INTEGER](#) **Default:** [Asc\(254\)](#) or ["*"](#) Notes: If [LineDrawing](#) is [ON](#), the cursor keys will input line-drawing characters into the current file. If [LineDrawType](#) is set to "user-defined character," then the character used is determined by [LineDrawChar](#). **Affects:** [LineTypeMenu\(\)](#), [LineDraw\(\)](#) See Also: [LineDrawType](#), [LineDrawing](#)

LineDrawing

Forces the cursor keys to input line-drawing characters in the current file.

Values: [BOOLEAN](#) **Default:** [OFF](#) Notes: When [LineDrawing](#) is [ON](#), the keypad cursor keys can be used to draw lines, based on the last line type set via the [LineTypeMenu\(\)](#) command. Setting [LineDrawing OFF](#) (usually toggled via [Toggle\(LineDrawing\)](#)) will restore the keypad cursor keys to their previous assignments. See Also: [LineDrawType](#), [LineDrawChar](#)

LineDrawType

The "drawing" character to be used when

[LineDrawing](#) is [ON](#). [Values](#): [INTEGER](#) - 1 - single-line characters - 2 - single-line side, double-line top characters - 3 - double-line side, single-line top characters - 4 - double-line characters - 5 - ASCII + - | - 6 - eraser - 7 - user-defined character (see [LineDrawChar](#) variable) [Default](#): 1 [Notes](#): If [LineDrawing](#) is ON, the cursor keys will input line-drawing characters into the current file. The characters that will be used are determined by the value of [LineDrawType](#). [Affects](#): [LineTypeMenu\(\)](#), [LineDraw\(\)](#) See Also: [LineDrawChar](#), [LineDrawing](#)

LineTypeMenu

Presents a menu which allows the user to select the line-drawing character to be used when

[LineDrawing](#) is [ON](#). [Syntax](#): [LineTypeMenu\(\)](#) [Returns](#): Nothing. [Notes](#): The line-drawing character is also used with the [LineDraw\(\)](#) command. The editor offers several line-drawing styles, including: - single-line characters - single-line side, double-line top characters - double-line side, single-line top characters - double-line characters - ASCII characters - eraser - user-defined character See Also: [LineDraw\(\)](#) [Variables](#): [LineDrawChar](#), [LineDrawType](#), [LineDrawing](#)

LinkSynFile

Links the information from the associated SyntaxHilite mapping (".syn") file, to the current file.

This command is intended for ADVANCED USAGE.

[Syntax](#): [INTEGER](#) [LinkSynFile](#)([STRING](#) fn [, [INTEGER](#) flag]) - fn is the name of the ".syn" mapping file to associate with the current file. - flag is an optional parameter. Valid flag values are: - [_ALWAYS_LOAD_](#) specifies that a copy of fn will be loaded from disk, even if a copy is already in memory. - [_DONT_PROMPT_](#) instructs the editor to suppress warning or informational messages that may be issued for any problems that occur when this command is executed. [Returns](#): Non-zero if fn was either found in memory, or loaded into the editor; [otherwise](#), zero ([FALSE](#)). [Notes](#): Unless [_ALWAYS_LOAD_](#) is specified, [LinkSynFile\(\)](#) searches to see if fn has already been loaded (in a previous call), and if so, links the copy in memory to the current file. If the extension for fn is not specified, ".syn" is assumed. If the path is not specified, fn is searched for in the following places: the current directory, the SYNHI\ editor subdirectory, then the editor load directory. This command is called internally by [InitSynhiCurrFile\(\)](#). This command can be used to dynamically load a custom ".syn" file for the current file. [Examples](#): // link "sal.syn" to the current file [LinkSynFile](#)("sal") See Also: [InitSynhiCurrFile\(\)](#), [PurgeSynhi\(\)](#), [GetSynFilename\(\)](#), [GetSynLanguageType\(\)](#), [LoadSynhiAssoc\(\)](#), [ReplaceSynFile\(\)](#) [Variables](#): [ShowSyntaxHilite](#)

List

Displays the current file in a read-only browse window.

[Syntax](#): [INTEGER](#) [List](#)([STRING](#) title, [INTEGER](#) width) - title is the title for the List window; can be an empty [string](#). - width is the width of the List window. [Returns](#): Non-zero if [List\(\)](#) is exited via <Enter>. Zero if [List\(\)](#) is exited via <Escape>. [Notes](#): [List\(\)](#) is similar to [IList\(\)](#) in operation, but [IList\(\)](#) allows the height of the List window to be specified, along with the ability to configure several options through the flag parameter. While the List window is

displayed, the following commands are active: [Up\(\)](#), [Down\(\)](#), [BegFile\(\)](#), [CReturn\(\)](#), [Escape\(\)](#), [EndFile\(\)](#), [PageUp\(\)](#), [PageDown\(\)](#). SpeedSearch is automatically enabled. Once [List\(\)](#) is exited, the current cursor line in the file will be the same as the cursor line that was in use in [List\(\)](#) before [List\(\)](#) was exited. This functionality allows [List\(\)](#) to be used as a simple "picker". [List\(\)](#) automatically resizes the List window bottom line for small files. [List\(\)](#) does not automatically reposition the listed file at line 1. If this is desired, the user must do it. The List window is centered on the screen. To specify a new location, use [Set\(X1\)](#) and/or [Set\(Y1\)](#) as discussed in the chapter on "[Editor Variables](#)." The behavior of [List\(\)](#) can be modified by the use of the hooked events `_LIST_STARTUP_` and `_LIST_CLEANUP_`. See Also: [PickFile\(\)](#), [IList\(\)](#), [Hook\(\)](#), [ListFooter\(\)](#), [ListHeader\(\)](#) [Variables: X1, Y1](#)

ListFooter

Displays a HelpLine string at the bottom of the current List window.

Syntax: [ListFooter](#)([STRING](#) text) - text is the string (coded in standard HelpLine format) to be displayed. Returns: Nothing. Notes: This command is used in conjunction with the [List\(\)](#) and [IList\(\)](#) commands to display a HelpLine at the bottom of a List window. (To display a HelpLine string at the bottom of a pop-up window, via the [PopWinOpen\(\)](#) or [Window\(\)](#) commands, use the [WindowFooter\(\)](#) command.) Typically, this command is called from a procedure activated by the `_LIST_STARTUP_` event. **Examples:** `proc ListStartup() Unhook(ListStartup) ListFooter("This is a help-line") end proc main() Hook(_LIST_STARTUP_, ListStartup) List("Simple List", Query(ScreenCols)) end` See Also: [List\(\)](#), [IList\(\)](#), [ListHeader\(\)](#), [PickFile\(\)](#), [WindowFooter\(\)](#)

ListHeader

Displays a HelpLine string at the top of the current List window.

Syntax: [ListHeader](#)([STRING](#) text) - text is the string (coded in standard HelpLine format) to be displayed. Returns: Nothing. Notes: This command is used in conjunction with the [List\(\)](#) and [IList\(\)](#) commands to display a HelpLine at the top of a List window. Typically, this command is called from a procedure activated by the `_LIST_STARTUP_` event. **Examples:** `proc ListStartup() Unhook(ListStartup) ListHeader("This is a (header) help-line") end proc main() Hook(_LIST_STARTUP_, ListStartup) List("", Query(ScreenCols)) // overwrite the title end` See Also: [List\(\)](#), [IList\(\)](#), [ListFooter\(\)](#), [PickFile\(\)](#)

Listing All Occurrences of a Specified Text String

* CompressView (<Alt V>, or "Compressed View" under Search

[Menu](#)) This command displays a PickList of all lines that contain a specified string of text. This command is a specialized version of the Find command, that forces the [V] option. When you execute CompressView, the editor displays the prompt, "[String](#) to list all occurrences of:" Enter the desired text string (which can be a Regular Expression) for which you want to view a list of all occurrences. Or, to select the default compressed-view string, simply press <Enter> without typing a selection string. (The default string is the set of all alphabetic characters (a through z, or A through Z) or the underscore character ("_"), located in column 1.) Once you enter a string, the editor displays the prompt: +-----+ |Search options [IWXA] (Ignore-[case](#) Words reg-eXp All-files):| | +-----+ Enter the desired options. (For more information on the available search options, see "[Basic Find and Replace Options](#)" earlier in this chapter.) A PickList is then displayed of all lines in the designated text that contain the indicated string. This list includes the line number

and the text for every matching line in the file, and indicates the name of the file in which the matching lines are located. At the bottom of the screen (or at the bottom of the list, if the list does not fill the entire screen), the following is displayed: Enter-Go to line Escape-Cancel Alt E-Edit this list The following options are available: - Scroll through the entries in the CompressView PickList. (For more information on scrolling within a PickList, see "[Using PickLists](#)" in the chapter on "[Command Prompts and PickLists](#).") - Press <Enter> on a line and the editor will place you into the file on the selected line. - Press <Escape> to terminate CompressView and [return](#) to your position in the current file. - Press <Alt E> to create a temporary file containing the entries in the list. You are immediately placed in that temporary COMPRESS file so that it becomes the current file, and CompressView is terminated. The COMPRESS file remains in the Ring of files until you execute the CompressView command again or you exit from the editor. (You cannot save the temporary COMPRESS file unless you first rename it using the ChangeCurrFilename command.) Initial placement of the cursor bar in the CompressView list is based on the position of the cursor in the current file. If the current line contains the specified string, then when the CompressView list is initially displayed, the cursor bar is located on that line in the list. If the current line did not contain the indicated string, then the cursor bar is located on the first line preceding the current line that does contain the [string](#). To move between the entries in the CompressView list and the text in the appropriate file, you can do the following: - Press <Alt E> to create a temporary file of CompressView entries. - Split the screen into two windows, with the CompressView list in one, and an appropriate file in the other. - Scroll the entries in the CompressView list to display entries you wish to view. - Switch to the window containing the indicated file. - Execute the GotoLine command, using the line number identified in the CompressView list, for each line you wish to view in the indicated file. To re-display the CompressView PickList once it has been removed from the screen, see the RedisplayViewPickList command later in this section. * FunctionList (<Ctrl G>, or "Function List" under Search [Menu](#)) This command is a specialized CompressView feature, for use when editing a program source file. FunctionList lists all function/procedure declaration lines in the current file. Function/procedure declaration lines are identified based on the current filename's extension, as follows: - If the extension is ".c" or ".cpp", declaration lines for C and C++ functions are listed. - If the extension is ".s" or ".ui", declaration lines for SAL procedures are listed. - If the extension is ".pas", declaration lines containing Pascal functions and procedures are listed. - If the extension is ".prg", ".spr", ".mpr", ".qpr", ".fmt", ".frg", ".lbg", or ".ch", declaration lines for xBase functions and procedures are listed. When you execute FunctionList, a CompressView PickList of all declaration lines for the appropriate functions or procedures is displayed. The format and processing options available in the PickList are the same as that described for CompressView. If you execute FunctionList from a file whose extension is not supported by the command, the CompressView PickList displays all lines that contain an alphabetic character (a through z, or A through Z) or the underscore character ("_"), in column 1. * RedisplayViewPickList (<AltShift V>, or "Redisplay View PickList" under Search [Menu](#)) This command re-displays the PickList that was created by the last search operation in which the "view" option was specified (that is, the last Find operation using the [V] option, or the last execution of CompressView or FunctionList). The PickList that is displayed when you execute RedisplayViewPickList does not reflect changes made to the text after the last "view-type" search was performed.

Literal

The next keypress typed after calling this command is taken as the literal ASCII character that the key represents.

Syntax: [INTEGER Literal](#)() Returns: Non-zero. Notes: This command prevents the editor from translating the next key press into an editor command. Use this command to input the actual character that is represented by a key. For example, to enter a NULL character, press <Ctrl 2> at the "Literal:" prompt displayed by the [Literal](#)() command. To determine what ASCII value a key returns in the editor, the user can run the ShowKey macro (located in the MAC\ editor subdirectory). See Also: [SelfInsert](#)()

lList

Displays the current file in a read-only browse window.

Syntax: [INTEGER IList](#)([STRING](#) title, [INTEGER](#) width , [INTEGER](#) height, [INTEGER](#) flag) - title is the title for the List window; can be an empty [string](#). - width is the width of the List window. - height is the height of the List window. - flag can be any combination of the following (OR'ed together to obtain the desired result): - [_ENABLE_SEARCH_](#) enables SpeedSearch in [IList](#)(). - [_BLOCK_SEARCH_](#) [limits](#) SpeedSearch within marked block of file being listed. Block MUST exist! - [_ANCHOR_SEARCH_](#) anchors search to beginning of line or beginning of block if [_BLOCK_SEARCH_](#) specified. - [_ENABLE_HSCROLL_](#) enables horizontal movement commands. Returns: Non-zero if [IList](#)() is exited via <Enter>. Zero if [IList](#)() is exited via <Escape>. Notes: [IList](#)() is similar to [List](#)() in operation, but [IList](#)() allows the height of the List window to be specified, and provides the ability to configure several options through the flag parameter. While the IList window is displayed, the following commands are active: [Up](#)(), [Down](#)(), [BegFile](#)(), [CReturn](#)(), [Escape](#)(), [EndFile](#)(), [PageUp](#)(), [PageDown](#)(). Once [IList](#)() is exited, the current cursor line in the file will be the same as the cursor line that was in use in [IList](#)() before [IList](#)() was exited. This functionality allows [IList](#)() to be used as a simple "picker." [IList](#)() automatically resizes the List window bottom line for small files. [IList](#)() does not automatically reposition the listed file at line 1. If this is desired, the user must do it. The IList window is centered on the screen. To specify a new location, use [Set](#)(X1) and/or [Set](#)(Y1) as discussed in the chapter on "[Editor Variables](#)." The behavior of [IList](#)() can be modified by the use of the hooked events [_LIST_STARTUP_](#) and [_LIST_CLEANUP_](#). See Also: [List](#)(), [PickFile](#)(), [Hook](#)(), [ListFooter](#)(), [ListHeader](#)() **Variables:** [X1](#), [Y1](#)

LoadBuffer

Deletes all the text in the current buffer, and then loads a file into it. No hooks are called.

Syntax: [INTEGER LoadBuffer](#)([STRING](#) fn [, [INTEGER](#) reclen]) - fn is the name of the file to load. - reclen is an optional integer parameter. If passed and non-zero, the file is set to binary mode with a line length of reclen. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command empties the current buffer and loads fn into it. If reclen is specified, the file is loaded in binary mode with a line length of reclen. No hooks are called. This is a common operation performed by macros, that save and restore configuration or other types of information. Its operation is comparable to the following: [EmptyBuffer](#)() [BinaryMode](#)(reclen) [PushBlock](#)() [InsertFile](#)(fn, [_DONT_PROMPT_](#)) [PopBlock](#)() See Also: [CreateBuffer](#)(), [CreateTempBuffer](#)(), [EditFile](#)(), [EditThisFile](#)(), [InsertFile](#)(), [EditBuffer](#)()

LoadDir

Returns the directory from which the editor was started.

Syntax: [STRING LoadDir](#)() Returns: A string containing the directory from which the editor was started. Notes: The string will contain the full path of the directory where the editor executable program resides. The path will include the trailing slash but not the name of the editor. See Also: **Variables:** [TSEPath](#), [DosCmdLine](#)

LoadHistory

Loads a history file from disk into the History Buffer.

Syntax: [INTEGER LoadHistory\(\[STRING fn\]\)](#) - fn is the optional name of the history file to be loaded. If fn is not specified, the editor attempts to load TSEHIST.DAT from the editor load directory. If fn is specified, but no path information is supplied, the current directory is assumed. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: The current contents of the History Buffer are replaced by the contents of the specified history file. [Examples:](#)
[LoadHistory\(\)](#) // Loads history from TSEHIST.DAT located in // the editor load directory [LoadHistory\("tsehist.dat"\)](#) // Loads history from TSEHIST.DAT // located in current directory [LoadHistory\("c:\foo\abc.dat"\)](#) See Also: [SaveHistory](#)
 ()

Loading, Processing, and Saving Files

This chapter discusses how files are loaded and managed by the editor. It also describes file-processing operations available during an editing session, and explains how to save and close files, and how to exit from the editor. The following topics are covered:

-

[Managing multiple files and maintaining the file Ring](#) - [Adding files to the Ring: specifying and opening files](#) (EditFile, NewFile) - [Locking files](#) (FileLocking) - [Moving between files in the Ring](#) (EditFile, ListOpenFiles, ListRecentFiles, NextFile, PrevFile) - [Saving and closing files, and exiting from the editor](#) (SaveFile, SaveAs, SaveAllFiles, QuitFile, SaveAndQuitFile, Exit, SaveAllAndExit) - [Saving/restoring the current editing session](#) ([PersistentHistory](#), [PersistentRecentFiles](#), [SaveState](#)) - [Alternative modes for viewing files](#) ([Hex mode](#), [Binary mode](#), [Browse mode](#)) - [Performing other file-processing operations](#) (FileManager, ChangeDir, ShowFileInfo, ChangeCurrFilename, InsertFile, KillFile) The chapter on "[Splitting the Screen into Windows](#)" describes two additional commands for opening and closing files. The OpenFileAndWindow command allows you to open a new file at the same time that you open a new window. The CloseFileAndWindow command allows you to close a file, along with the window in which the file is displayed. (Many of the file-related commands described in this chapter take advantage of the editor's PickList feature. For more information on using a file PickList, see "[Obtaining a File PickList within a Prompt Box](#)" in the chapter on "[Command Prompts and PickLists](#).")

Loading/Executing Macros

In order to execute an external Compiled Macro (.MAC) file, it must first be loaded into the editor. The

[ExecMacro\(\)](#) command automatically loads the Compiled Macro file into the editor and, if the load is successful, it then executes the macro (that is, it executes the Main macro, if one exists in the macro file). There are times when you only want to load a macro (which may contain many macro commands, [key](#) definitions, [menus](#), and help text) but [not](#) immediately execute it. This can be accomplished by using the [LoadMacro\(\)](#) command. The [LoadMacro\(\)](#) command can be invoked with or without passing an external macro filename. If no parameter is passed, [LoadMacro\(\)](#) prompts for the name of the macro file to load. If you pass a parameter to the [LoadMacro\(\)](#) command, [LoadMacro\(\)](#) attempts to load the macro filename passed to it. If you attempt to load a macro file that has the same name as a macro file that is already loaded, the existing macro file is replaced by the one loaded from disk. If a macro in the existing macro file is currently executing, the new macro file is not loaded and the following message is displayed: "Cannot overlay running macro". Like the [LoadMacro\(\)](#) command, the [ExecMacro\(\)](#) command can also be invoked with or without passing a macro filename. It prompts for the macro file to execute if you do not pass a filename parameter. When you execute the [ExecMacro\(\)](#) command, the editor first checks to see if the macro file has previously been loaded, and if so,

executes the macro file (that is, it executes the Main macro, if one exists in the macro file). Otherwise, the editor first calls [LoadMacro\(\)](#) to load the macro file into the editor, and if [LoadMacro\(\)](#) is successful, [ExecMacro\(\)](#) then executes the newly-loaded macro file. [ExecMacro\(\)](#) leaves the macro file loaded in memory, so the next time you execute that macro file, it does not have to be re-loaded from disk. In addition to specifying a macro filename when you execute the [ExecMacro\(\)](#) command, you can optionally include a command-line string to pass to the macro(s) included in the macro file. (For more information on passing a command line to a macro, see "[Using Command Lines in Macros](#)" in the chapter on "[Using Macro Language Features](#).") (For more information on loading and executing [macros](#), see "[Introduction to External Compiled Macros](#)" in the chapter on "[Creating and Using Macros](#)" in [the User's Guide](#). Also see the [LoadMacro\(\)](#) and [ExecMacro\(\)](#) commands in the chapter on "[Editor Commands](#)." Note that there are other ways to invoke macros, once the macro file is loaded. [Macros](#) can be assigned to keys and [menu](#) selections. (For more information on keydefs and menus, see the chapter on "[Using Macro Language Features](#)." If you find that you are constantly loading/executing an external macro, you should consider making it more readily available. You can have the editor automatically load the macro in question each editing session by adding the macro to [the TSELOAD.DAT file](#), using the [EditAutoLoadList\(\)](#) command. Or, if you do not use the macro quite as often, you can add it to the Potpourri menu. Finally, you can even add the macro to the user-interface (UI) macro file. (For more information on these options, see the chapter on "[Fully Customizing the Editor](#)" in [the User's Guide](#).)

LoadKeyMacro

Loads a Keyboard Macro from disk into memory.

Syntax: [INTEGER LoadKeyMacro](#)([\[STRING keymac_filename\]](#)) - keymac_filename is an optional string that contains the name of the Keyboard Macro file to be loaded. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If keymac_filename does not contain an extension, ".kbd" is used. If it does not contain a drive or directory, the editor searches, as follows, to determine from where to load the macro: - The current directory is searched first. - If the macro is not found there, and the KbdPath variable is set, then the directory specified by KbdPath will be searched. - Finally, if the macro is not found there, the load directory of the editor will be searched. If keymac_filename contains wildcard characters, a PickList of matching filenames in the current directory will be displayed. Before the Keyboard Macro is loaded, any existing Keyboard Macros will be purged. This command includes a history list of previous entries passed to it. See [AddHistoryStr\(\)](#) for additional information. See Also: [PurgeKeyMacro\(\)](#), [RecordKeyMacro\(\)](#), [SaveKeyMacro\(\)](#), [AddHistoryStr\(\)](#) **Variables:** [KbdPath](#)

LoadMacro

Loads a Compiled Macro file from disk into memory.

Syntax: [INTEGER LoadMacro](#)([\[STRING macro_filename\]](#)) - macro_filename is the name of the Compiled Macro file to load. May [include](#) drive, path, and wildcards. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If macro_filename does not contain an extension, ".mac" is used. If it does not contain a drive or directory path, the following algorithm is used to determine from where to load the Compiled Macro file: - The current directory is searched first. If the macro file is not found there, and the TSEPath variable is set, the indicated directories are searched (along with any MAC\ subdirectories). Finally, if the macro still has not been found, the directory where the editor resides is searched (followed by the MAC\ subdirectory, if one exists). If macro_filename contains wildcard characters, a standard PickList is used, using the current directory. The requested Compiled Macro file, if it exists, is loaded from disk into memory. A maximum of 40 macro files can be loaded at any given time. If a macro file of the same name has already been loaded, it is purged. Note that two macro files in different directories, but having the same name, are considered to be the same macro file. The name of the Compiled Macro file is entered into the Public macro

table. The name used is macro_filename, less any extension, drive and path. Selecting this entry during [ExecMacro\(\)](#) executes the Main() procedure, if one exists. If the macro file has any [Public macro procedures](#), they are also entered into the Public table and are shown under the filename in the [ExecMacro\(\)](#) PickList. Finally, if the macro file contains a WhenLoaded() macro, it is executed. This command includes a history list of previous entries passed to it. See [AddHistoryStr\(\)](#) for additional information. See Also: [ExecMacro\(\)](#), [isMacroLoaded\(\)](#), [ExecLoadedMacro\(\)](#), [PurgeMacro\(\)](#), [AddHistoryStr\(\)](#) [Variables: TSEPath](#)

LoadProfileSection

Makes the requested section from the initialization file the current section for use

[by](#) calls [to GetNextProfileItem\(\)](#). **Syntax:** [INTEGER LoadProfileSection](#)([STRING](#) section [, [STRING](#) profile_name]) - section is the section of an initialization file to load. - profile_name is the optional file to search. If not specified, the tse.ini file located in the editor's load directory is searched. If profile_name does not include a path, the profile_name that resides in the WINDOWS directory (specified by the windir environment variable) is searched. Returns: Non-zero on success; zero ([FALSE](#)) on failure. Notes: section is not case sensitive. If the section is not found or it is found, but contains no items, then zero ([FALSE](#)) is returned. Each time this command is called, the item pointer is set to the topmost entry in the item list. Each call to [GetNextProfileItem\(\)](#) causes the item pointer to move to the next item in the list. **Examples:** See the [GetNextProfileItem\(\)](#) command example. See Also: [GetNextProfileItem\(\)](#), [LoadProfileSectionNames\(\)](#), [GetNextProfileSectionName\(\)](#), [GetProfileInt\(\)](#), [GetProfileStr\(\)](#), [WriteProfileInt\(\)](#), [WriteProfileStr\(\)](#), [RemoveProfileItem\(\)](#), [RemoveProfileSection\(\)](#), [FlushProfile\(\)](#)

LoadProfileSectionNames

Loads the section names of an initialization file that will be retrieved by repetitive calls to

[GetNextProfileSectionName\(\)](#). **Syntax:** [INTEGER LoadProfileSectionNames](#)([[STRING](#) profile_name]) - profile_name is an optional initialization file's name. If not specified, the tse.ini file, located in the editor's load directory, is assumed. If profile_name is specified, and does not include a path, the profile_name that resides in the WINDOWS directory (specified by the windir environment variable) is assumed. Returns: Non-zero. Notes: Once this command has been called for a particular initialization file, use repetitive calls to [GetNextProfileSectionName\(\)](#) to retrieve each section name of that initialization file. Note that each call to [LoadProfileSectionNames\(\)](#) resets the section list pointer to the top of the section name list. **Examples:** // The following loads the section names from the tse.ini file // (located in the editor's load directory) into memory and then // displays the name of each section name. [proc main\(\)](#) [integer](#) counter = 0 [string](#) section_name[255] = "" [LoadProfileSectionNames\(\)](#) [while](#) [GetNextProfileSectionName](#) (section_name) counter = counter + 1 [Warn](#)("Section ", counter, " name is ", section_name, " [in](#) ", [Query](#)(LoadDir) + "tse.ini") [endwhile](#) [Warn](#)("Total of ", counter, " section names found in ", [Query](#)(LoadDir) + "tse.ini") [end](#) See Also: [GetNextProfileSectionName\(\)](#), [LoadProfileSection\(\)](#), [GetNextProfileItem\(\)](#), [GetProfileStr\(\)](#), [GetProfileInt\(\)](#), [WriteProfileStr\(\)](#), [WriteProfileInt\(\)](#), [RemoveProfileItem\(\)](#), [RemoveProfileSection\(\)](#), [FlushProfile\(\)](#)

LoadStartupMacros

Loads and processes startup

[macros](#). This command is intended for ADVANCED USAGE. **Syntax:** [LoadStartupMacros\(\)](#) Returns: Nothing. Notes:

The [LoadStartupMacros\(\)](#) command runs [TSESTART.MAC](#), if one is found, and then loads any [macros](#) found in [TSELOAD.DAT](#). This command is intended to be executed one time only during an editing session, at the time the editor is initially loaded. The command should be included in the editor user-interface (.UI) file.

LoadSynhiAssoc

Loads SyntaxHilite extension-association information.

This command is intended for ADVANCED USAGE.

Syntax: [INTEGER LoadSynhiAssoc\(\)](#) Returns: Non-zero if the SyntaxHilite extension-association file (TSESYNHI.DAT) was loaded and is valid; [otherwise](#), zero ([FALSE](#)). Notes: The editor calls this command during editor startup. It loads the TSESYNHI.DAT database file which consists of extensions as keys, and the names of SyntaxHilite mapping (".syn") files as the data. TSESYNHI.DAT must reside in the editor load directory. Macros that modify the TSESYNHI.DAT file can call this command to load the information from the modified TSESYNHI.DAT file. See Also: [InitSynhiCurrFile\(\)](#), [PurgeSynhi\(\)](#), [GetSynFilename\(\)](#), [GetSynLanguageType\(\)](#), [LinkSynFile\(\)](#), [ReplaceSynFile\(\)](#)
Variables: [ShowSyntaxHilite](#)

LoadWildFromDOS

Controls the editor file loading when a wildcarded file specification is given at the command line.

Values: BOOLEAN **Default:** [ON](#) **Limits:** Has effect at startup only. Notes: When LoadWildFromDOS is set ON, the editor will startup and load all files that match the wildcard specification. When LoadWildFromDOS is set OFF the editor displays a PickList of matching files from which to choose. You can override LoadWildFromDOS with the -a (all) command-line parameter: - a forces the editor to load all the files that match the specified wildcard. - -a- forces the editor to display a PickList of the matching files. See Also: [LoadWildFromInside](#)

LoadWildFromInside

Controls the editor file loading when a wildcarded file specification is given in the

[EditFile\(\)](#) or [EditThisFile\(\)](#) prompt. **Values:** BOOLEAN **Default:** [OFF](#) Notes: When LoadWildFromInside is set OFF, the editor will display a PickList of matching files to choose from. When LoadWildFromInside is set ON, the editor will load all files that match the wildcard specification. You can override LoadWildFromInside with the -a (all) command-line parameter: - a forces the editor to load all the files that match the specified wildcard. - -a- forces the editor to display a PickList of the matching files. **Affects:** [EditFile\(\)](#), [EditThisFile\(\)](#) See Also: [LoadWildFromDos](#)

LoByte

Returns the lower byte from the lower 16 bits of a specified integer value.

Syntax: [INTEGER LoByte\(INTEGER i\)](#) - i is the integer value from which to return the lower byte of the lower 16

bits. Returns: The lower byte of the lower 16 bits of the specified integer value. Notes: SAL utilizes 32-bit integers. [LoByte\(\)](#) returns the lower byte of the lower 16 bits. The value is always interpreted as positive, and will always be in the range 0 to 255. [Examples](#): See the [HiByte\(\)](#) command. See Also: [HiByte\(\)](#), [LoWord\(\)](#), [HiWord\(\)](#)

LockCurrentFile

Locks the current file, to prevent it from being updated by other processes.

Syntax: [INTEGER LockCurrentFile](#)([INTEGER](#) how_to_lock) - how_to_lock should be one of _READONLY_LOCKING_ or _HANDLE_LOCKING_. Returns: 0 on success, [otherwise](#): _FILE_ALREADY_LOCKED_ _CANT_CREATE_ _CANT_LOCK_FILE_ Notes: If how_to_lock is [not](#) _READONLY_LOCKING_ or _HANDLE_LOCKING_, this command does nothing and returns 0. If the current buffer is not a file buffer, or the current file is already locked, this command does nothing and returns 0. Only a limited number of files can be locked via _HANDLE_LOCKING_ - this limit is operating system dependant. This limit may be removed in a future version. There is no limit to the number of files that can be locked via _READONLY_LOCKING_. See [Locking Files](#) for more information. See Also: [UnLockCurrentFile\(\)](#) [Variables](#): [FileLocking](#)

Locking Files

The editor includes an option to "lock" the files that have been opened for editing in the current session. When files are being edited in a multi-user or multi-tasking environment, it is possible that more than one user or process may attempt to modify the same file at the same time. The FileLocking variable allows you to reduce or eliminate the likelihood that another program overwrites a file (on disk) that you are currently editing.

The editor provides two different methods for

[locking files](#): Read-Only and Handle. - If Read-Only locking is in effect, the editor sets the read-only attribute of each file that is opened in an editing session. As each file is closed (removed from the editing session), the read-only attribute is removed. With Read-Only locking, there is no limit to the number of opened files that can be locked. As many files as can be loaded into the editor, can also be locked. It should be noted that other users or processes can remove the read-only attribute of a file at any time, thereby allowing the file to be written to by another program. Though this does [not](#) typically occur in multi-user or multi-tasking environments, the potential for this should be assessed when selecting the file-locking method. You should also note that if the editor is terminated abnormally (as occurs with a system failure), the editor would not have the opportunity to remove the read-only attribute from the files that it locked under the Read-Only locking method. In the event this occurs, the read-only attribute must be manually removed from each affected file. (The editor's FileManager utility can be used to do this.) - If Handle locking is in effect, the editor allocates a handle (a file-identifier assigned by the operating system) for each file that is opened in an editing session, and then retains that handle until the file is closed, and thereby removed from the editing session. (Otherwise, when a file is opened in an editing session, the editor allocates a handle for the file, reads the file into memory, and then immediately de-allocates the assigned handle.) As long as the editor retains the handle for a file, the file can generally be read by another user or process (as long as the other program opens the file in an appropriate mode), but it cannot be written to. If the editor is terminated abnormally, the system will take care of de-allocating file handles. Thus, it should [not](#) be necessary to manually release files that were locked at the time of the system failure. (However, depending on your system, there may be a short delay before the allocated file handles are de-allocated.) To activate file-locking, the FileLocking variable must be set to HANDLE or READONLY. (For more information on configuring the FileLocking variable, see "File Locking" under "[System/File Options](#)" in the chapter on "[Configuring](#)

[the Editor Interactively](#)." A file is locked only when it is actually opened by the editor (that is, when the file is initially made the current file.) Files are locked according to the currently-selected file-locking method. If a particular file cannot be locked, the editor issues a warning message, then goes ahead and loads the file for editing; however, you will not be allowed to save the file back to disk under its original filename. (The editor cannot lock a file that is already locked by another program, or that is protected in some other way.) For any file that has been locked by the editor, a "+" displays on the StatusLine when that file is the current file. For the most effective use of file-locking, the appropriate file-locking option should be in effect when the editor is started. Though you can activate file-locking after an editing session has started, this would compromise the protection that this option is attempting to provide. Only those files opened with file-locking activated, will be locked. If file-locking is de-activated during an editing session, those files already opened and locked, will remain locked; but any other files that are subsequently opened will [not](#) be locked.

LogDrive

Changes to the specified drive.

Syntax: [INTEGER LogDrive](#)([STRING](#) s) - s is the drive to which to change. Do not include a colon. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: s should be in the range A through Z, case insensitive. See Also: [GetDrive\(\)](#), [CurrDir\(\)](#), [GetDir\(\)](#)

LongestLineInBuffer

Returns the length of the longest line in the current buffer.

Syntax: [INTEGER LongestLineInBuffer](#)() Returns: The length of the longest line in the current buffer. See Also: [CurrLineLen](#)()

Lower

Changes alphabetic characters to lower

case. **Syntax:** [STRING Lower](#)([STRING](#) s) - s is an optional string to convert. or: [Lower](#)() Returns: If the first Syntax format is used, the resulting lower-cased [string](#) is returned; [otherwise](#), nothing is returned. Notes: [Lower](#)() "lower-cases" the target characters - all upper-case alphabetic characters [A..Z] are converted to lower [case](#) [a..z]. Other characters are [not](#) changed. If the first Syntax format is used, s is lower-cased. If the second Syntax format is used: if the cursor is in a marked block, the characters in the block are lower-cased; otherwise, the character at the cursor position is lower-cased. **Examples:** Given: [string](#) s[3], = "aBc", s2[3] then: s2 = [Lower](#)(s) // s2 is "abc" s2 = [Lower](#)("Def") // s2 is "def" s2 = [Lower](#)(s2) // s2 is "def" // If the cursor is on a word, lower case it [proc LowerWord](#)() [PushBlock](#)() // save block state if [MarkWord](#)() // if on a word, mark it [Lower](#)() // convert to lower [case](#) [endif PopBlock](#)() // restore block state [end](#) See Also: [Upper](#)(), [Flip](#)()

LoWord

Returns the lower 16-bit word from a specified integer value.

Syntax: [INTEGER LoWord](#)([INTEGER](#) i) - i is the integer value from which to return the lower 16-bit word. Returns: The lower 16 bits of the specified integer value. Notes: SAL utilizes 32-bit integers. [LoWord\(\)](#) returns the lower 16-bit word. The value is always interpreted as positive, and will always be in the range 0 to 65535. See Also: [LoByte\(\)](#), [HiByte\(\)](#), [HiWord\(\)](#)

IRead

Gets keyboard input from the user, allowing a maximum width to be specified.

Syntax: [INTEGER IRead](#)([VAR STRING](#) s, [INTEGER](#) maxlen [, [INTEGER](#) history_num]) - s is the string that the response is placed into, and should always be initialized to some value. - maxlen is the length of the input prompt area as well as the maximum length allowed for the response string. This length must not exceed the length of s. - history_num is an optional history number. If passed, history will be in effect, using the passed history number. Returns: A non-zero value if the user terminated input by pressing <Enter>; zero (FALSE) if the user terminated input by pressing <Escape>. Notes: When invoked, [IRead\(\)](#) will display a prompt area the size passed in by maxlen. The description for [AddHistoryStr\(\)](#) contains a list of pre-defined history_num [values](#). See Also: [Read\(\)](#), [ReadNumeric\(\)](#), [IReadNumeric\(\)](#), [AddHistoryStr\(\)](#)

IReadNumeric

Gets numeric keyboard input (with a maximum specified length) from the user.

Syntax: [INTEGER IReadNumeric](#)([VAR STRING](#) s, [INTEGER](#) maxlen [, [INTEGER](#) history_num]) - s is the string in which the user input (response) is placed. s should always be initialized to some value. - maxlen is the length of the input prompt area, as well as the maximum length allowed for the response string. This length must not exceed the length of s. - history_num is an optional history number. If specified, history will be in effect, using the passed history number. Returns: A non-zero value if the user terminated input by pressing <Enter>; otherwise, zero (FALSE) if the user terminated input by pressing <Escape>. Notes: When invoked, this command displays a prompt area that is the size specified by maxlen. See the [AddHistoryStr\(\)](#) command for a listing of the pre-defined [values](#) for history_num. See Also: [IRead\(\)](#), [Read\(\)](#), [ReadNumeric\(\)](#), [AddHistoryStr\(\)](#)

IRepeatFind

Repeats the previous

[IFind\(\)](#) or [IReplace\(\)](#) command, whichever was executed last. **Syntax:** [INTEGER IRepeatFind](#)([[INTEGER](#) direction]) - direction is an optional integer that forces the repeated find or replace to proceed in the indicated direction. Supported [values](#) for direction are: - [_BACKWARD_](#) forces the search to proceed backward, towards the beginning of the file. - [_FORWARD_](#) forces the search to proceed forward, towards the end of the file. - [_REVERSE_](#) forces the search to proceed in the opposite direction than the last find or replace. - [_SEARCH_INCLUSIVE_](#) forces the search to proceed starting from the current position, rather than one character beyond the current cursor position, for a forward search (as is normally done). Returns: Non-zero if the find string was found; otherwise, zero (FALSE). Notes: For a forward search, the search starts one character beyond the current cursor position. For a backward search, the search starts one character before the current cursor position. [IRepeatFind\(\)](#) is intended for use in [macros](#). It does [not](#) affect the histories and options of the high-level [Find\(\)](#), [Replace\(\)](#), and [RepeatFind\(\)](#). Unlike [RepeatFind\(\)](#), [IRepeatFind\(\)](#) does [not](#) issue any messages and does not highlight the found string. See Also: [IFind\(\)](#), [IReplace\(\)](#)

IReplace

Performs a low-level

[IReplace\(\)](#) command. **Syntax:** [INTEGER IReplace](#)([STRING](#) find_str, [STRING](#) replace_str, [STRING](#) opt_str) - find_str is the string to search for. - replace_str is the string that will replace find_str, if find_str is found. - opt_str contains the options to use in the search/replace. Returns: Non-zero if the string was found; otherwise, zero (FALSE). Notes: This command is intended for use within [macros](#), and should [not](#) be bound to a key by itself. Unlike the normal [Replace\(\)](#), this command: - does not prompt for input; the parameters are required. - does not highlight the found string. - does not do any setup for (or interfere with) the [RepeatFind\(\)](#) command. This command can be used in conjunction with the [IRepeatFind\(\)](#) command, to search/replace multiple times. Note that this command has no effect on the normal [Replace\(\)](#) and [RepeatFind\(\)](#) commands, and can be used in [macros](#) without changing the former's histories and/or current targets and options. For a list of options supported by [IReplace\(\)](#), see the [Replace\(\)](#) command. See Also: [IRepeatFind\(\)](#), [IFind\(\)](#), [Replace\(\)](#)

IShowEntryScreen

Displays the screen that was present at the time the editor was invoked, and returns control to the calling macro.

Syntax: [INTEGER IShowEntryScreen\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: A low-level version of the [ShowEntryScreen\(\)](#) command, this command simply displays the entry screen, and returns to the calling macro. **Examples:** // The [ShowEntryScreen\(\)](#) command [proc ShowEntryScreen\(\)](#) [integer](#) save_cursor [if](#) [IShowEntryScreen\(\)](#) save_cursor = TurnCursorOff() [GetKey\(\)](#) [Set\(Cursor](#), save_cursor) [endif end](#) See Also: [ShowEntryScreen\(\)](#)

LTrim

Trims leading white space from a string.

Syntax: [STRING LTrim](#)([STRING](#) s) - s is the input string. Returns: A copy of s, with leading white space removed. Notes: The passed [string](#) itself is [not](#) modified. **Examples:** [string](#) s[10] = " test ", s2[10] s2 = [LTrim](#)(s) // s2 is "test ", // s is still " test " s = [LTrim](#)(s) // s is "test " s = [LTrim](#)(" abc ") // s is "abc " See Also: [RTrim\(\)](#), [Trim\(\)](#)

IVersion

Displays information about the editor, including the version number, the serial number, the directory in which the editor program is located, and current memory usage.

Syntax: [IVersion\(\)](#) Returns: Nothing. See Also: [Version\(\)](#)

Macro File Search Order and the Tsepath Variable

External Compiled Macros should be maintained in an appropriate directory for subsequent loading into the editor. To determine the appropriate directory, consider the following description of the "

[macro file search order](#)" used by the editor. When you attempt to load an external Compiled Macro file, the editor successively searches the following directories to locate the file to load: 1. The current directory 2. The directories indicated by the TSEPath variable, as well as the subdirectory MAC\ (if one exists) of each directory indicated by [TSEPath](#) 3. The editor load directory, as well as the subdirectory MAC\ (if one exists) of the editor load directory As indicated in this "[macro file search order](#)," the TSEPath editor variable is used by the editor to locate external Compiled Macro object files. By setting this variable to the location of your Compiled Macro files, you will not need to specify the full path when attempting to load or execute macros. The recommended directory to create for your own [external macro files](#) is \TSE\USERMAC\ . By keeping your source (.S) [and](#) object (.MAC) external macro files in a dedicated, separate directory, it will be easier to manage your macros and to later upgrade to newer versions of the editor. (For more information, see the TSEPath description in the chapter on "[Editor Variables](#).")

Macro Language Syntax

A macro is generally composed of one or more procedures, written according to keywords and syntax defined in SAL. A procedure is identified by the keyword "

[proc](#)," and incorporates a number of other optional elements, including [identifiers](#), [reserved words](#), [strings](#), [numbers](#), [constants](#), [variables](#), and statements. A procedure itself can be considered a macro. This chapter discusses the many elements used to create macro procedures, and describes the keywords and syntax required by each element. It then presents the structure for combining those elements into an actual macro procedure. The following topics are covered: - [Identifiers](#) - [Reserved words](#) - [Strings](#) - [Numbers](#) - [Constants](#) - [Pre-defined constants](#) - [Variables](#) - [Integer variables](#) - [String variables](#) - [White characters](#) - [Comments](#) - [Statements](#) - [Variable assignment statements](#) - [Expressions](#) - [Precedence and order of evaluation in expressions](#) - [Type conversions within expressions](#) - [Control statements](#) - [Function call statements](#) - [Macro procedures](#) - [Scope](#) - [Forward procedures](#) - [Main](#) - [Global vs. local variables](#) - [Returning values](#) - [Arguments - passing and retrieving parameters](#) - [Public](#) - [Using DLLs](#) - [Conditional compilation directives](#) In addition to procedures, SAL provides the ability to embed data within a macro (using the [datadef](#) construct), to create help information (using the [helpdef](#) construct), to build menus and menubars (using the [menu](#) and [menubar](#) constructs), and to assign commands, [macros](#), [menus](#), and help to keys (using the [keydef](#) construct). The chapter on "[Using Macro Language Features](#)" provides information on defining [datadef](#), [helpdef](#), [menu](#), [menubar](#), and [keydef](#) constructs.

Macro Procedures

SAL provides the ability to create user-written macro procedures that the editor will execute just like its own built-in commands. Procedures are the building blocks of SAL, and provide a convenient way to create simple or complex functions, which can then be used without concern for the procedure's internal composition. Procedures

[break](#) large computing tasks into smaller ones, and enable the re-use of code without starting from scratch each time. Once a procedure has been written to do a task, you can call it from another procedure, call it from a menu, or even assign the procedure to be executed when a specified key is pressed. This last feature is the means by which your own new commands can be assigned to keys. Features of procedures [include](#): - local [variables](#) - parameters (passed by value

and passed by reference) - [return values](#) The "[proc](#)" statement is used to declare a procedure, and is terminated by "[end](#)". The basic [syntax](#) is: [proc](#) proc_name() local variable declarations [statements](#) [end](#) where: - proc identifies proc_name() as a procedure. - proc_name is the name of the procedure. - local variable declarations are one or more optional local [variables](#) as described in the previous sections. - statements are one or more optional statements as described in the previous sections. - end signifies the end of the procedure. A simple example of a procedure is: [proc](#) Hello() [Message](#)('Hello world!') [end](#) [Scope](#) ----- While native commands can be called from any macro file, user-written procedures can only be called from within the source file in which they are declared. If a user-written procedure from another source file is required within the current source file, the procedure must be copied into the current source file before it can be used. An alternative to physically copying the procedure into the current source file is to use the "#include" directive, along with the name of the source file that contains the procedure. (For more information on include files, see "[Include Files](#)" in the chapter on "[Using Macro Language Features](#)".) SAL requires entities (procedures and [variables](#)) to be declared before they are used. If procedure A will call procedure B before procedure B has been defined, use the "[Forward](#)" reference to B before procedure A. (See "[Forward Procedures](#)" below.) [Forward Procedures](#) ----- To call a procedure that is declared later in the same file, it is not necessary to move the entire procedure above the current procedure. All that is needed is to declare it as "[forward](#)". For example: [forward](#) [string](#) [proc](#) WordAtCursor() By using forward to identify a procedure and its parameters, the procedure may be called at any point after the forward declaration. [Main](#) ----- Macro files can contain a special procedure named "[main](#)". If a macro file contains a procedure called "[main](#)", then that Main procedure will be executed when the name of the macro file is supplied to the [ExecMacro](#)() command. The macro file can also be executed on start-up from the command line. Example: If MYMACRO.S contains the following (and the compiled name is MYMACRO.MAC): [proc](#) [main](#)() [statements](#) [end](#) then, [ExecMacro](#)("MyMacro.mac") will load MYMACRO and then execute the main() procedure. Using the following command-line specification to start the editor: g32 file1 file2 -eMyMacro.mac will load file1, file2, and execute the main() procedure in MYMACRO. If a macro file is selected from the editor's list of Public macros and the macro contains a Main procedure, the Main procedure will be executed. If a macro file is selected from the Public macros list that does not have a Main procedure, nothing will be executed. (For more information on [Public macros](#), see "[Public](#)" later in this chapter.) [Global vs. Local Variables](#) ----- Variables can be declared within a procedure, and, as such, are referred to as local variables. These variables have a limited lifetime, and are created upon entry to the procedure, and deleted when the procedure terminates. Local variables can not be directly referenced outside the procedure in which they are declared. Local variables do not have a default value, unless they are given an initial value when they are created. [Variables](#) can also be declared outside of procedures. These [variables](#) are referred to as global [variables](#), and are available to any procedures that follow the variable declaration within the same macro source file. The values of global variables are retained until the macro containing them is purged from memory, or until the editor session is terminated. These variables, if not given an initial value, default to zero for integer variables, and to an empty [string](#) for [string variables](#). An example that uses both types of [variables](#) is: integer count // this is a global variable /
***** This procedure will [loop](#) 10 times and create 10 lines containing "test". *****/ [proc](#) AddSomeLines()
() [integer](#) i = 10 // [loop](#) counter [while](#) i [addline](#)("test") i = i - 1 [endwhile](#) count = count + 1 [Message](#)("AddSomeLines invoked ", count, " time(s)") [end](#) As long as this macro is loaded into memory, the value of "count" is retained. Each time AddSomeLines() is invoked, the value of "count" is increased by 1. The value of "i", however, is set to 10 each time the macro is invoked. It is used in the "[while](#)" loop to count down to 0, causing 10 lines with the text "test" to be placed in the file starting at the current position. Once the macro is complete, the variable "i" is deleted, and is no longer available. [Returning Values](#) ----- Procedures may optionally be declared as returning a specified type of either integer or string. Procedures that return a type may be used in expressions and assignment statements. Native commands that return a value include [NumFiles](#)() and [CurrFilename](#)(). For example: [integer](#) n [string](#) fn[80] n = [NumFiles](#)() // n is set to number of files loaded fn = [CurrFilename](#)() // fn is set to name of current file The syntax for declaring procedures that return a type is: [integer](#) [proc](#) proc_name() [statements](#) [end](#) or: [string](#) [proc](#) proc_name() [statements](#) [end](#) where: - integer proc identifies this as a procedure that must return an [integer](#) value. - string proc

identifies this as a procedure that must return a string value. - proc_name is the name of the procedure. - [statements](#) are one or more optional [statements](#) as described in previous sections. - end signifies the end of the procedure. The return() statement is used to specify the value returned by the procedure. For example: /

***** First, a simple example: a procedure that returns the value 5. Not useful for anything except an example.

*****/ [integer proc](#) Five() [return](#)(5) [end](#) /

***** Now a slightly more complicated example: return the word at the cursor as a string. *****/

[string proc](#) GetWordAtCursor() [string](#) word[80] = " // Init to empty, in [case](#) no // word [PushBlock](#)() // Save current block status if [MarkWord](#)() // Mark the word word = [GetMarkedText](#)() // Get it [endif](#) [PopBlock](#)() // Restore block status [return](#)(word) // And return the word at the end // cursor or the empty string [Arguments - Passing and Retrieving Parameters](#) ----- [Passing and Retrieving Parameters by Value](#) Procedures can also

have [variables](#) passed to them. Both integers and [strings](#) can be passed to procedures, and both can be passed by value and by reference. When a variable is passed by value, the called procedure gets a copy of the actual variable. Any modifications made to the passed variable are made to the copy. Variables passed by value should be considered as constants with possibly different initial values each time the procedure is invoked. [Syntax](#): [proc](#) proc_name([integer](#) age, [string](#) name) where: - proc_name is the name of the procedure. - age is an integer variable passed by value. - name is a string variable passed by value. Consider, for example, a procedure that goes to a specified line and column: [proc](#) GotoLineCol([integer](#) line, [integer](#) column) [GotoLine](#)(line) [GotoColumn](#)(column) [end](#) This procedure would be called as: GotoLineCol(50, 37) or: [integer](#) line, column // assume line and column have been set to some value GotoLineCol (line, column) [Passing and Retrieving Parameters by Reference](#) To indicate that a parameter is to be passed by reference, it must be prefaced with "[var](#)". The called procedure then gets a pointer to the actual parameter. Any modifications made to the passed parameter are made to the actual parameter. Parameters passed by reference should be used with care, since changing them will also change their values in the calling procedure. [Syntax](#): [proc](#) proc_name ([var integer](#) age, [var string](#) name) where: - proc_name is the name of the procedure. - age is an integer variable passed by reference. - name is a string variable passed by reference. For an example of a procedure that takes a variable and changes it, consider the following procedure which gets the word at the cursor position, and returns the word by altering the string that was passed to the procedure. /

***** Example call: WordAtCursor(my_string)

When this procedure is called with my_string, it will change the value of my_string to the word at the cursor, or an empty (null) string if no word is at the cursor position.

*****/ [proc](#) WordAtCursor([var string](#) word)

[PushPosition](#)() // save our current position word = "" // make sure word is blank [if isWord](#)() while [isWord](#)() // while the current character if not [Left](#)() // is in the word set go left // if you can [break](#) // if you cannot go left, // break out of the loop [endif](#) [endwhile](#) if not [isWord](#)() // if not on a character [Right](#)() // move to the right [endif](#) [repeat](#) // start of [loop](#) word = word + [chr](#)([CurrChar](#)()) // add next letter // to word [if not Right](#)() // move to right if you can [break](#) // if not, [break](#) out of [loop](#) [endif](#) until not [isWord](#)() // loop until the current char // is not in the word set [endif](#) [PopPosition](#)() // [return](#) to where we started [end](#) You could change the above example so that the procedure returns the word at the cursor position as a string. Then to use it you would set a string variable equal to the procedure name. For example: /

***** Example call: my_string = WordAtCursor()
() When this procedure is called, it will return a string that is null (cursor not on a word), or the word at the cursor position. My_string will then be assigned the value returned by WordAtCursor().

*****/ [string proc](#) WordAtCursor() [string](#) word [40] [PushPosition](#)() // save our current position word = "" // make sure word is blank [if isWord](#)() while [isWord](#)() // while the current character if not [Left](#)() // is in the word set go left // if you can [break](#) // if you cannot go left, // break out of the loop [endif](#) [endwhile](#) if not [isWord](#)() // if not on a character [Right](#)() // move to the right [endif](#) [repeat](#) // start of [loop](#) word = word + [chr](#)([CurrChar](#)()) // add next letter // to word [if not Right](#)() // move to right if you can [break](#) // if not, [break](#) out of [loop](#) [endif](#) until not [isWord](#)() // [loop](#) until current char is // not in the word set [endif](#) [PopPosition](#)() // [return](#)

to where we started [return](#) (word) [end](#) Note that the preceding two example procedures are very similar in function to the [GetWord\(\)](#) command. +----- Tip -----+ | If a procedure will return a single value, it should return it via | a "[return](#)" statement. If a procedure will return more than a | single value, some of the variables must be passed by reference so | that the procedure can change them directly. | | | An alternative is to use global [variables](#), rather than passing | parameters. This may be easier in many cases, but keeping track of | all the global variables, and of the procedures that change them, | may become difficult to manage in large macro files. | +-----+ The following are examples of the use of [return\(\)](#). If returning a single value from a procedure is sufficient, and given: [integer proc](#) abc() ... [statements](#) ... [return](#)(answer) [end](#) then, abc() will return "answer" as an integer value. To handle multiple variables, and have the procedure manipulate the [variables](#) directly, use: [proc](#) abc([var integer](#) a, [var integer](#) b) ... [statements](#) ... [return\(\)](#) [end](#) abc(), in this example, does not return a value, but integers "a" and "b" are directly manipulated by the procedure. To handle multiple variables, and have the procedure manipulate those variables directly and also return a value, use: [integer proc](#) abc([var integer](#) a, [var integer](#) b) [integer](#) answer ... [statements](#) ... [return](#)(answer) [end](#) abc(), in this example, does return a value, and integers "a" and "b" are directly manipulated by the procedure. An example use of multiple [return\(\)](#) [statements](#) is: [integer proc](#) doit() ... [if](#) expression [return](#)(TRUE) [endif](#) [return](#)(FALSE) [end](#) doit(), in this example, returns an integer value. The value returned is either 1 (TRUE) or 0 (FALSE), depending on the outcome of expression. Note about the use of [return\(\)](#): A [return\(\)](#) should be placed in the procedure at the point where execution should cease and have control passed back to the caller. If the procedure is to pass back a value, use [return](#)(value) at the appropriate location(s). If passing back a value, be sure to declare the procedure as returning the "type" of value that is being passed back to the caller. The [return\(\)](#) statement may be used in as many locations within a procedure as necessary. [Examples](#): [proc](#) abc() is a procedure that DOES NOT return a value. The use of [return\(\)](#) within the procedure simply signifies that the procedure is finished, once [return\(\)](#) is encountered. [integer proc](#) abc() is a procedure that DOES return an integer value. The user would use [return](#)(value) at the appropriate location(s). [Public](#) ----- The "[public](#)" directive identifies the procedure as one whose name is to be inserted into the editor's list of Public names. The editor can give the user a list of Public procedures, facilitating their execution without having to bind them directly to a key. The use of the "[public](#)" identifier is optional. If a procedure is NOT defined as "[public](#)", it cannot be executed directly, unless it is assigned to a key or called from within another procedure in the same macro file. (For more information on Public macros, see the descriptions of the [ExecMacro\(\)](#) and [LoadMacro\(\)](#) commands in the chapter on "[Editor Commands](#).") [Using DLLs](#) ----- The "DLL" directive allows the user to extend the editor to include additional procedures which cannot be written using the macro language. Since this is a very advanced feature, great care must be exercised when [using DLLs](#). Basically, DLLs are procedures written using another programming language, but which can interface with the editor's calling convention. DLLs must be written in a specific format to properly interface with the editor. DLLs can be written in C, C++, Pascal, Delphi, or any other language that allows the creation of DLLs. The [syntax](#) to declare DLL procedures is: DLL "dll_file" type [proc](#) [PASCAL] Proc1(param_list) [: import_name] type [proc](#) [PASCAL] Proc2(param_list) [: import_name] ... type [proc](#) [PASCAL] ProcN(param_list) [: import_name] [end](#) where: - DLL specifies that the procedures which follow reside in the named DLL. - dll_file is the name of the file containing the DLL procedures. If dll_file is enclosed within <>, the editor allows the operating system to search the default DLL search path. Thus, a dll_file of "<kernel32.dll>" would be found in the Windows system directory. If dll_file is simply enclosed in quotes, the editor searches for the DLL using the same search method employed when searching for macros. - type is an integer or nothing. DLL string procedures are NOT currently supported. - proc specifies that a procedure follows. - Proc1...ProcN are descriptive name(s) of each procedure. The procedure name is used within the editor macro to identify (that is, to call) the specified function. This name is case sensitive and must match the DLL export name exactly unless the import_name directive is used. - PASCAL is an optional modifier that specifies that this DLL is compiled using the PASCAL calling convention. For cdecl and syscall calling conventions, omit this modifier. For more information on calling conventions, see your vendors compiler reference manual. - param_list is a comma-separated list of parameters. (See "[DLL Parameters and Return Values](#)" below, and see "[Arguments - Passing and Retrieving Parameters](#)" earlier in this chapter.) - import_name is the optional name of the DLL entry point for this procedure, if different from the procedure name. In many cases the procedure name is not exactly the same as the DLL name. In this case the import_name may be used to specify the name within

the DLL. For example, Windows includes a function called MessageBox, but the actual DLL export name is either MessageBoxA or MessageBoxW. You could declare this as: [integer proc](#) MessageBoxA(...) and always call MessageBoxA(). Or you could declare it as: [integer proc](#) MessageBox(...) : "MessageBoxA" and then just call MessageBox() to invoke it. This parameter is also useful for PASCAL-style functions whose internal DLL name is often upper-cased. For example: [integer proc](#) Operate(...) : "OPERATE" - end signifies the end of the DLL procedures in the named DLL. [DLL Parameters and Return Values](#) To interface with the internal types, integer and string, it is necessary to understand their formats. Integers are stored using 32 bits, or 4 bytes. In the C programming language, the macro language declaration "[integer](#) n" is simply expressed as: int n or long n The format of a string is a structure as follows: bytes 1-2: length of [string](#) (0 to 255) byte 3..n: actual [string](#) data In the C programming language, string s may be expressed as a structure similar to: struct { unsigned short len; char data[maximum [string length](#)]; } s; The len field indicates the length of data. Parameters are passed as follows: [integer](#) n 32-bit [integer](#) n (4 bytes) - by value [var integer](#) n 32-bit pointer to [integer](#) n (4 bytes) - by address [string](#) s 32-bit pointer to [string](#) structure discussed above [var string](#) s 32-bit pointer to [string](#) structure discussed above, and a 32-bit integer specifying string's maximum length To improve interfacing with DLLs, the macro language allows an additional optional modifier to be used in a DLL parameter list, as follows: [[VAR](#)] [INTEGER](#) integer_name [: modifier] [[VAR](#)] [STRING](#) string_name [: modifier] To modify the parameters, the following keywords are available (some do not make sense in all contexts): byte passes signed 8-bit char as 32-bit [integer](#) word passes signed 16-bit word as 32-bit [integer](#) long passes signed 32-bit word as 32-bit [integer](#) strval passes 32-bit pointer to [string](#) data followed by 32-bit [integer](#) length strptr passes 32-bit pointer to [string](#) data (no length byte) strptrref passes 32-bit pointer to [string](#) data followed by 32-bit integer specifying the string's maximum length strref passes 32-bit pointer to SAL-style [string](#) followed by 32-bit specifying the string's maximum length cstrval creates a local copy of an ASCIIZ string into which the editor copies and null-terminates the string before passing a 32-bit pointer to the copy of the string. Use this modifier for ASCIIZ string parms to ensure a null terminator is placed on the end of the string. When a DLL procedure returns nothing or returns an integer, the parameters are first passed as described above. Then, the return address is pushed on the stack and control is passed to the DLL procedure at the specified location within the DLL. Any DLL procedure that returns an integer must place the result in the EAX register. In most programming languages, this is performed with a simple [return](#) statement. Only 32-bit DLLs are supported. Example: dll "<user32.dll>" [integer proc](#) MessageBox([integer](#) hwndOwner, //handle of owner window [string](#) lpzText : cstrval, //address of text in message box [string](#) lpzTitle: cstrval, //address of title of message box [integer](#) fuStyle //style of message box) : "MessageBoxA" [end](#) [define](#) NULL 0 // some sample styles [define](#) MB_OK 0x00000000 [define](#) MB_OKCANCEL 0x00000001 [define](#) MB_ABORTRETRYIGNORE 0x00000002 [define](#) MB_YESNOCANCEL 0x00000003 [define](#) MB_YESNO 0x00000004 [define](#) MB_RETRYCANCEL 0x00000005 [define](#) MB_ICONHAND 0x00000010 [define](#) MB_ICONQUESTION 0x00000020 [define](#) MB_ICONEXCLAMATION 0x00000030 [define](#) MB_ICONASTERISK 0x00000040 [proc](#) PopMsg([string](#) title, [string](#) text) MessageBox([GetWinHandle](#)(), text, title, MB_OK) [end](#) <F12> PopMsg("Title goes here", "This is the text" + [chr](#)(10) + "of the message") For an additional example of a DLL procedure, see the TabUtil macro (TABUTIL.S), and the associated DLL source file (ENTAB.C), in the MAC\ editor subdirectory.

Macro Source Files Vs. Macro Object Files

There are two files related to the creation of every macro. One is the source file, which you create. It is written in the English-like macro language. The source file can have any valid filename, but the extension must be ".S".

The other file associated with a macro is the macro object file (also called the "Compiled Macro" file). This file is created by the SAL compiler program, and is in a format that is executable by the editor. The compiler program compiles the source file into an object file with the same filename as the source file, but with an extension of ".MAC". When the editor executes a macro, the compiled object file is executed. This allows the

macro to run much faster than if the source file was executed directly.

Macro Syntax

The minimal syntax required for a macro is:

[proc](#) macro_name() [statements](#) [end](#) where: - proc designates that the following is a macro procedure. - macro_name specifies the name of the macro procedure. - statements are one or more optional statements. Statements are the "guts" of the macro. They are where the macro does its work. - end designates the end of the macro procedure. As you can imagine, these [macros](#) can become quite complex!

MacroCmdLine

Contents of the macro command line.

[Values](#): [STRING](#) (maximum of 128 characters) [Default](#): "" Limits: Cannot be used within a config/endconfig construct. Notes: This variable can be used to pass information to and from an external macro. If MacroCmdLine is used to pass information back to the calling procedure, the return code of [ExecMacro\(\)](#) should first be checked. If the macro cannot execute, MacroCmdLine is undefined. [Examples](#): //execute macro "foo" and pass data "This is the command line" [ExecMacro](#)("foo This is the command line") //in [main\(\)](#) of macro "foo" [string](#) s[128] s = [Query](#) (MacroCmdLine) //s is now: "This is the command line" See Also: Commands: [ExecMacro\(\)](#)

Macros

+----- From

[the User's Guide](#) -----+ || [Creating and Using Macros](#) || - [KeyBoard Macros](#) || - [Introduction to External Compiled Macros](#) || [Customization using SAL Compiled Macros](#) || +----- From [the Macro Reference Guide](#) -----+ || [Overview of the Macro Programming Language](#) || [Macro Language Syntax](#) || [Using Macros](#) || [Using Macro Language Features](#) || [Editor Commands](#) || [Editor Variables](#) || || [Appendix B: Command Implementation](#) || [Appendix C: Commands by Category](#) || [Appendix D: Variables by Category](#) | +-----+

MacroStackAvail

Returns the amount of stack space available to the currently-executing Compiled Macro.

[Syntax](#): [INTEGER](#) [MacroStackAvail\(\)](#) Returns: The number of bytes available in the macro stack. Notes: This command is useful for testing and debugging complex [macros](#). Since the size of the editor's macro stack is finite (approximately 16k), heavily-recursive or [string](#)-intensive [macros](#) could exceed the stack space. This command can be used to display the stack usage while the macro is running. [Examples](#): [if](#) [MacroStackAvail\(\)](#) < 1000 [Warn](#)("Available macro space less than 1000 bytes!") [endif](#)

MakeBackups

Specifies that a backup of the original file be created whenever a file is saved.

Values: BOOLEAN **Default:** [ON](#) **Notes:** When a file is saved, the editor uses the following method to make a backup copy of the original file. The copy of the file on the disk is first renamed. The base name is the same, but the extension specified by BackupExt will be used. Next, the copy of the file in memory (the changed one), will be written to disk using the current filename. When MakeBackups is ON, the ProtectedSaves setting is ignored and treated as though [ProtectedSaves](#) is [ON](#). When [BackupPath](#) is set, [BackupExt](#) is ignored. **Affects:** [SaveAllAndExit\(\)](#), [SaveAllFiles\(\)](#), [SaveAndQuitFile\(\)](#), [SaveFile\(\)](#) See Also: [BackupExt](#), [BackupPath](#), [ProtectedSaves](#)

MakeTempName

Returns a unique filename.

Syntax: [STRING](#) [MakeTempName](#)([STRING](#) path [, [STRING](#) extension]) - path is the location where the filename is to be generated. - extension is the optional extension for the file. Returns: A unique filename. **Examples:** [string](#) temp [80] // generate a unique filename in the c:\foo directory temp = [MakeTempName](#)("c:\foo") See Also: [fCreate\(\)](#), [SplitPath\(\)](#)

Managing Multiple Files and Maintaining the File Ring

The editor allows you to edit multiple files during a session. The number of files that can be edited in a session is limited only by available memory.

Though multiple files may be available in a session for editing, editing tasks can be performed only in the file in which the cursor is positioned. The term "current" is used to indicate the file, line, column, character, position, or window in which the cursor is located, and, thus, where editing occurs. For example, the file currently being edited is the "current file," and the line on which the cursor is positioned in that file is the "current line" (also called the "

[cursor](#) line"). If the screen is split into multiple windows, the window containing the current file is the "current window." The term "current" is also used to identify the default path, including the drive and directory, recognized by the editor for various operations when you do not specify the full filename path. When the editor is initially started, the current path is the drive and directory in which you are located when you start the editor. This remains the current path until you change the drive or directory from a file PickList (unless the editor is configured otherwise). You can also change the drive or directory via the ChangeDir command. For more information, see the ChangeDir command later in this chapter. For managing the files that are selected for editing in a session, information on each file is maintained in a file-management Ring. For each file that is selected, an entry for that file is added to the Ring. This information includes the full path and filename for the file, pointers to the next and previous files in the Ring, and specific control information about the file. Files are added to the Ring following the current file, in the order in which they are specified. To edit a file, it must be added to the Ring and opened by the editor. A file is opened by loading an existing file from disk into memory or by creating a new file in memory, in an area called a buffer. For each file that is opened, a memory buffer is created. For existing files, a copy of the file on disk is read into a buffer. Any changes or additions

that are made to a file are handled in its memory buffer area. The copy of the file on disk is not altered or updated until that file is saved (written to disk), using the contents of the associated buffer. When an existing file is loaded into the editor, the file is split into lines. A new line is started at each occurrence of a carriage return, a linefeed, or a carriage return/linefeed pair. A new line is also started at the editor's maximum allowable line length, by splitting any lines which exceed this length. (If the file is subsequently saved, the split lines are saved accordingly. See "[Saving and Closing Files, and Exiting from the Editor](#)" below.) In the copy of a file that is in a memory buffer, physical line-termination characters are not maintained by the editor. Such characters are removed from the file when it is loaded. When the file is subsequently saved, termination characters are placed at the end of each line of the file (including the last line of the file, unless configured otherwise). (See "[Saving and Closing Files, and Exiting from the Editor](#)" below.) If changes are made to a file, the Modified File Indicator (an asterisk ("*") preceding the filename) is displayed on [the StatusLine](#). When the file is saved, this indicator is removed. When a file is no longer needed, it can be discarded, or removed from memory, by closing the file. A file is closed by quitting that file and unloading, or emptying, its memory buffer. Once a file is closed, it is taken out of the file Ring and is no longer available for editing (unless it is subsequently re-opened). To end an editing session and exit from the editor, all files must be closed. The editor ends automatically when the last (or only) open file is closed, unless you execute QuitFile or SaveAndQuitFile to close the last file, and the QuitToPrompt variable is ON. (For more information on the QuitToPrompt variable, see "EditFile Prompt on Quit" under "[Command/Format Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") To quickly end an editing session, commands are available to close ALL open files and exit. When you exit from the editor, you are returned to the operating system (or to the application from which you invoked the editor). The editor places you back in the original drive and directory that you were in when you started the editor.

Manipulating Blocks Via Clipboards

As an alternative to the CopyBlock and MoveBlock operations described earlier in this chapter, blocks can also be stored in and retrieved from temporary text buffers within the editor called ClipBoards. ClipBoards are similar to scratch pads, offering intermediate storage areas for blocks of text during an editing session. The contents of ClipBoards are discarded when the editor is terminated.

Two temporary internal editor ClipBoard options are available: the System ClipBoard and Named ClipBoards. The System ClipBoard is a single storage area that you can readily access using the ClipBoard commands described in this section. Named ClipBoards differ from the System ClipBoard in that you can create hundreds of these individual [ClipBoards](#), each uniquely identified by a user-defined name. The commands Copy, CopyAppend, Cut, CutAppend, Paste, and PasteOver allow you to store blocks in [ClipBoards](#), then later retrieve those blocks. The Empty command allows you to remove the contents of a ClipBoard. In addition to the editor's internal ClipBoards, you can also access the Microsoft Windows Clipboard, which is external to the editor. Commands are available that allow you store text in or retrieve text from the Microsoft [Windows](#) Clipboard. [The ClipBoard Copy Commands](#) ----- Use [the ClipBoard Copy commands](#) to copy a block in the current file to a ClipBoard, and overwrite (replace) any current contents of that ClipBoard. The contents of the block are stored in the ClipBoard according to the type of block (line, column, or character) that is marked. Once the block is copied, it is then unmarked. * Copy to the System ClipBoard (<Grey+>, or "Copy" under Clip [Menu](#)) This command copies the block in the current file to the System ClipBoard, and replaces any text already in that ClipBoard. * Copy to a Named ClipBoard ("Copy" under "[Named ClipBoards](#)" of Clip [Menu](#))* This command copies the block in the current file to a Named ClipBoard. When you execute this command, the editor prompts with "Copy to ClipBoard:". Enter the desired name of the ClipBoard. The block is then copied to the designated ClipBoard, and replaces any text already in that ClipBoard. [The ClipBoard CopyAppend Commands](#) ----- Use [the ClipBoard CopyAppend commands](#) to copy the block in the current file to a ClipBoard, and append it to any current contents of that ClipBoard. The contents of the block are stored in the ClipBoard according to the type of block (line, column, or character) that is marked. (If you append different block

types into the same Clipboard, the block type of the last block appended determines the block type of the text in that Clipboard.) Once the block is copied, it is then unmarked. * CopyAppend to the System Clipboard (<Ctrl Grey+>, or "Copy Append" under Clip [Menu](#)) This command copies the block in the current file to the System Clipboard, and appends it to any text already in that Clipboard. * CopyAppend to a Named Clipboard ("Copy Append" under "[Named ClipBoards](#)" of Clip [Menu](#))* This command copies the block in the current file to a Named Clipboard. When you execute this command, the editor prompts with "Copy Append to Clipboard:". Enter the desired name of the Clipboard. The block is then copied to the designated Clipboard, and is appended to any text already in that Clipboard. [The Clipboard Cut Commands](#) ----- Use [the Clipboard Cut commands](#) to move the block in the current file to a Clipboard, and overwrite (replace) any current contents of that Clipboard. The contents of the block are stored in the Clipboard according to the type of block (line, column, or character) that is marked. Once the block is moved, it is deleted from the current file. * Cut to the System Clipboard (<Grey->, or "Cut" under Clip [Menu](#)) This command cuts the block in the current file to the System Clipboard, and replaces any text already in that Clipboard. * Cut to a Named Clipboard ("Cut" under "[Named ClipBoards](#)" of Clip [Menu](#))* This command cuts the block in the current file to a Named Clipboard. When you execute this command, the editor prompts with "Cut to Clipboard:". Enter the desired name of the Clipboard. The block is then cut to the designated Clipboard, and replaces any text already in that Clipboard. [The Clipboard CutAppend Commands](#) ----- Use [the Clipboard CutAppend commands](#) to move the block in the current file to a Clipboard, and append it to any current contents of that Clipboard. The contents of the block are stored in the Clipboard according to the type of block (line, column, or character) that is marked. (If you append different block types into the same Clipboard, the block type of the last block appended determines the block type of the text in that Clipboard.) Once the block is moved, it is deleted from the current file. * CutAppend to the System Clipboard (<Ctrl Grey->, or "Cut Append" under Clip [Menu](#)) This command cuts the block in the current file to the System Clipboard, and appends it to any text already in that Clipboard. * CutAppend to a Named Clipboard ("Cut Append" under "[Named ClipBoards](#)" of Clip [Menu](#))* This command cuts the block in the current file to a Named Clipboard. When you execute this command, the editor prompts with "Cut Append to Clipboard:". Enter the desired name of the Clipboard. The block is then cut to the designated Clipboard, and appended to any text already in that Clipboard. +----- Tip -----+ | The editor can be configured to Copy/CopyAppend or Cut/CutAppend | | the current line to a Clipboard, if a block is not marked. (For | | more information, see "Use Current Line If No Block" under | | "[Text Entry/Block Options](#)" in the chapter on | | "[Configuring the Editor Interactively](#).") | +-----+ [The Clipboard Paste Commands](#) ----- Use [the Clipboard Paste commands](#) to copy the contents of a Clipboard into the current file. The block contents are inserted into the file at the cursor location and are then unmarked. (The editor can be configured so that it does not unmark the block contents once pasted into the file. For more information, see "UnMark After Paste" under "[Text Entry/Block Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") The type of block (line, column, or character) stored in the Clipboard determines how the text is inserted into the file. This behavior is the same as that described earlier in this chapter for the CopyBlock command. If different types of blocks were appended to the same Clipboard, the block type for the entire contents of the Clipboard is based on the type of the last block appended to that Clipboard. The Paste commands do not delete the contents of the Clipboard; thus, the same text can be copied to multiple locations. * Paste from the System Clipboard (<Grey*>, or "Paste" under Clip [Menu](#)) This command pastes the contents of the System Clipboard into the current file at the cursor location. * Paste from a Named Clipboard ("Paste" under "[Named ClipBoards](#)" of Clip [Menu](#))* This command pastes the contents of a Named Clipboard into the current file. When you execute this command, the editor prompts with "Paste from Clipboard:". Enter the desired name of the Clipboard. A copy of the contents of the designated Clipboard are inserted into the file at the [cursor](#) location. [The Clipboard PasteOver Commands](#) ----- Like [the Clipboard Paste commands](#), use [the Clipboard PasteOver commands](#) to copy the contents of a Clipboard into the current file. However, the PasteOver commands are intended for use with column blocks. These commands function just like the Paste commands for line and character blocks. For column blocks, they behave the same as the Paste command, except that existing text on the affected lines is not shifted to the right; rather, it is overlaid by the contents of the Clipboard. * PasteOver from the System Clipboard (<Ctrl Grey*>, or "Paste Over" under Clip [Menu](#)) This command pastes the contents of the System

Clipboard into the current file at the cursor location. * PasteOver from a Named Clipboard ("Paste Over" under ["Named ClipBoards"](#) of Clip [Menu](#)) * This command pastes the contents of a Named Clipboard into the current file. When you execute this command, the editor prompts with "Paste Over from Clipboard:". Enter the desired name of the Clipboard. A copy of the contents of the designated Clipboard are placed into the file at the cursor location. [The Clipboard Empty Command](#) ----- Use [the Clipboard Empty command](#) to remove the current contents of a Clipboard from the Clipboard itself. * Empty the System Clipboard ("Empty" under Clip [Menu](#)) This command deletes the current contents of the System Clipboard. * Empty a Named Clipboard ("Empty" under ["Named ClipBoards"](#) of Clip [Menu](#)) * This command deletes the current contents of a Named Clipboard and deletes that Clipboard. When you execute this command, the editor prompts with "Delete which Clipboard:". Enter the desired name of the Clipboard. * As an alternative to using the pull-down Menu to execute the Named Clipboard commands, press <Ctrl B> to pop up the following selection menu: + [Named ClipBoards](#) + | *Cut...***** | | Cut Append... | | Copy... | | Copy Append... | +-----+ | Paste... | | Paste Over... | +-----+ | Empty... | +-----+ + [The Microsoft Windows Clipboard Commands](#) ----- The following commands allow you to copy, cut, or paste a block of text between the current file and the Microsoft Windows Clipboard. The contents of the block are stored in the Windows Clipboard according to the type of block (line, column, or character) that is marked; text that is retrieved from the Windows Clipboard is inserted into the file based on the stored block type. (However, note that if a column block is pasted into another application, the block will be pasted as a normal Windows-style block, with each row being a separate line.) * CopyToWinClip (<CtrlShift Grey+>, or "Copy ([Windows Clipboard](#))" under Clip [Menu](#)) This command copies the block in the current file to the Microsoft Windows Clipboard, and replaces any text already in that Clipboard. If the configuration option "Use Current Line If No Block" is set ON, then the editor copies the current line to the Windows Clipboard, if a block is [not](#) currently marked. * CopyAppendToWinClip ("Copy Append ([Windows Clipboard](#))" under Clip [Menu](#)) This command copies the block in the current file to the Microsoft Windows Clipboard, and appends it to any text already in that Clipboard. If the configuration option "Use Current Line If No Block" is set ON, then the editor appends the current line to the Windows Clipboard, if a block is [not](#) currently marked. * CutToWinClip (<CtrlShift Grey->, or "Cut ([Windows Clipboard](#))" under Clip [Menu](#)) This command moves, or "cuts," the block in the current file to the Microsoft [Windows Clipboard](#), and replaces any text already in that Clipboard. Once the block is moved, it is deleted from the current file. If the configuration option "Use Current Line If No Block" is set ON, then the editor cuts the current line to the Windows Clipboard, if a block is [not](#) currently marked. * CutAppendToWinClip ("Cut Append ([Windows Clipboard](#))" under Clip [Menu](#)) This command moves, or "cuts," the block in the current file to the Microsoft [Windows Clipboard](#), and appends it to any text already in that Clipboard. Once the block is moved, it is deleted from the current file. If the configuration option "Use Current Line If No Block" is set ON, then the editor cuts the current line to the Windows Clipboard, if a block is [not](#) currently marked. * PasteFromWinClip (<CtrlShift Grey*>, or "Paste ([Windows Clipboard](#))" under Clip [Menu](#)) This command pastes the contents of the Microsoft Windows Clipboard into the current file at the cursor position. * PasteOverFromWinClip ("Paste Over ([Windows Clipboard](#))" under Clip [Menu](#)) This command pastes the contents of the Microsoft Windows Clipboard into the current file at the cursor position. For column blocks, existing text on the affected lines is not shifted to the right; rather, it is overlaid by the contents of the Clipboard. * EmptyWinClip ("Empty ([Windows Clipboard](#))" under Clip [Menu](#)) This command deletes the current contents of the Windows Clipboard.

Manipulating Text

The editor offers many commands for

[manipulating text](#). This chapter describes the following text-manipulation commands, which do not require that a block be marked in order to operate. It also discusses the function of the Deletion Buffer, which allows deleted text to be recovered. - [Commands to add text](#) (AddLine, InsertLine, Literal) - [Commands to copy text](#) (DupLine, CopyCharAbove, CopyCharAboveToEol) - [Commands to move text](#) (CReturn, JoinLine, SplitLine, SwapLines) - [Commands to format text](#) (CenterLine, WrapPara, QuoteLine) - [Commands to delete text](#) (BackSpace, DelChar,

DelLine, DelLeftWord, DelRightWord, DelToEol) - [Commands and features that allow recovery of text](#) (RestoreCursorLine, Deletion Buffer, UnDelete, GlobalUnDelete, PasteUnDelete) Additional text-manipulation commands are discussed in other chapters. The chapter on "[Block Operations](#)" presents commands that operate on blocks of text. Included are the following commands that act on the current line, if the cursor is not in a marked block: Shift, ShiftLeft, and ShiftRight; the [case](#)-change commands of Flip, Upper, Lower, and Capitalize (which can also act on just the current word); and the ClipBoard Cut and Copy commands. The chapter on "[Editing Modes](#)" describes modes that you can use to affect the way certain commands operate when you are entering text. The chapter on "[Search Features: Finding and Replacing Text](#)" explains use of the Replace and FindAndDo commands for changing strings of text. The chapter on "[Checking Your Spelling](#)" provides information on how to use SpellCheck to correct the spelling of words in a file. The chapter on "[Special Commands and Features](#)" discusses other utilities for modifying text, like [LineDrawing](#) mode, Sort, pop-up AsciiChart, and DateTimeStamp. NOTE: Many of the text manipulation commands described in this chapter are affected by the status of Insert mode. (For more information on these effects, see "[Insert Mode](#)" in the chapter on "[Editing Modes](#).")

Mark

Marks a block.

Syntax: [Mark](#)([INTEGER](#) block_type) - block_type is the type of block to mark, and can be: - [_INCLUSIVE_](#) for an inclusive character block - [_NONINCLUSIVE_](#) for a non-inclusive character block - [_LINE_](#) for a line block - [_COLUMN_](#) for a column block Returns: Nothing. Notes: This is a low-level version of the other marking commands. For example: [BegLine\(\)](#) [MarkColumn\(\)](#) [EndLine\(\)](#) [MarkColumn\(\)](#) and [BegLine\(\)](#) [Mark](#)([_COLUMN_](#)) [EndLine\(\)](#) [Mark](#)([_COLUMN_](#)) are functionally identical. See Also: [MarkColumn\(\)](#), [MarkLine\(\)](#), [MarkChar\(\)](#), [MarkStream\(\)](#), [MarkToEOL\(\)](#), [isBlockInCurrFile\(\)](#), [isBlockMarked\(\)](#), [isCursorInBlock\(\)](#) **Variables:** [Marking](#)

MarkBlockBegin

Used to mark the beginning of a character block.

Syntax: [INTEGER](#) [MarkBlockBegin](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command is mainly used in [macros](#) and in emulating other editors. Blocks marked with this command are not marked until a corresponding [MarkBlockEnd\(\)](#) command is issued. See Also: [MarkBlockEnd\(\)](#), [MarkChar\(\)](#), [MarkStream\(\)](#), [MarkLine\(\)](#), [MarkColumn\(\)](#)

MarkBlockEnd

Used to mark the end of a character block.

Syntax: [INTEGER](#) [MarkBlockEnd](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command is mainly used in [macros](#) and in emulating other editors. Blocks marked with this command are not marked until a corresponding [MarkBlockBegin\(\)](#) command is issued. See Also: [MarkBlockBegin\(\)](#), [MarkChar\(\)](#), [MarkStream\(\)](#), [MarkLine\(\)](#), [MarkColumn\(\)](#)

MarkChar

Marks a character block which does not include the cursor position.

Syntax: [INTEGER MarkChar\(\)](#) Returns: Non-zero. Notes: This block type is often referred to as a non-inclusive style block, as the cursor position is not included in the block. This is in contrast to [MarkStream\(\)](#), which does include the cursor position in the block. The commands [isBlockInCurrFile\(\)](#), [isBlockMarked\(\)](#), and [isCursorInBlock\(\)](#) return [_NONINCLUSIVE_](#) for this block type. Initially executing this command marks one end of a character block at the cursor position. As the cursor is moved, the block grows with it. Executing the command again marks the other end of the block at the cursor position. If [MarkChar\(\)](#) is executed outside of a marked block, the block is extended to the current cursor position. The block type will be a character block, even if it was a different type of block before the [MarkChar\(\)](#) command was issued. If [MarkChar\(\)](#) is executed with the cursor inside a block, the existing block is unmarked and a new block marking action is begun. See Also: [Mark\(\)](#), [MarkBlockBegin\(\)](#), [MarkBlockEnd\(\)](#), [MarkColumn\(\)](#), [MarkLine\(\)](#), [MarkStream\(\)](#), [MarkToEOL\(\)](#), [isBlockInCurrFile\(\)](#), [isBlockMarked\(\)](#), [isCursorInBlock\(\)](#)
Variables: [Marking](#)

MarkColumn

Marks a column block.

Syntax: [INTEGER MarkColumn\(\[INTEGER start_line, INTEGER start_column, INTEGER end_line, INTEGER end_column\]\)](#) - start_line is the optional block starting line. - start_column is the optional block starting column. - end_line is the optional block ending line. - end_column is the optional block ending column. Returns: Non-zero. Notes: The commands [isBlockInCurrFile\(\)](#), [isBlockMarked\(\)](#), and [isCursorInBlock\(\)](#) return [_COLUMN_](#) for this block type. Initially executing this command marks one end of a column block at the cursor position. As the cursor is moved, the block grows with it. Executing the command again marks the other end of the block at the cursor position. If [MarkColumn\(\)](#) is executed outside of a marked block, the block is extended to the current cursor position. The block type will be a column block, even if it was a different type of block before the [MarkColumn\(\)](#) command was issued. If [MarkColumn\(\)](#) is executed with the cursor inside a block, the existing block is unmarked and a new block marking action is begun. See Also: [MarkBlockBegin\(\)](#), [MarkBlockEnd\(\)](#), [MarkStream\(\)](#), [MarkChar\(\)](#), [MarkLine\(\)](#), [MarkWord\(\)](#), [Mark\(\)](#), [MarkToEOL\(\)](#), [isBlockInCurrFile\(\)](#), [isBlockMarked\(\)](#), [isCursorInBlock\(\)](#) **Variables:** [Marking](#)

MarkFoundText

Marks the results of the immediately preceding find.

Syntax: [MarkFoundText\(\[INTEGER tag_number\]\)](#) - tag_number is the optional tagged pattern number to mark in a regular expression search string. If not passed, the entire found [string](#) is marked. Returns: Nothing. Notes: For this command to work properly, it should immediately follow a find command, before any other commands that might change the current position are invoked. The optional tag_number parameter only applies if a regular expression find was performed. For more information on regular expressions, see the chapter on "[Search Features: Finding and Replacing Text](#)" in the User's Guide. **Examples:** [string](#) s[80] [if](#) [IFind](#)("^a.*z\$", "x") [MarkFoundText](#)() // mark found text s = [GetMarkedText](#)() // get it in s [endif](#) // Second example, this time marking the 2nd tagged // expression found. [string](#) s[80] [if](#) [IFind](#)("{^a}{.*}{z\$}", "x") [MarkFoundText](#)(2) // mark tag #2 s = [GetMarkedText](#)() // store it in s [endif](#) See Also: [IFind\(\)](#), [GetMarkedText\(\)](#), [GetFoundText\(\)](#), [HiLiteFoundText\(\)](#) **Variables:** [CenterFinds](#)

Marking

Used to start/stop block marking with the current block type.

Values: BOOLEAN **Default:** [OFF](#) **Limits:** Cannot be used within a config/endconfig construct. **Notes:** You should save the current state of Marking if you use it within a macro, and restore it upon exiting the macro. Blocks are said to be either open or closed. An open block is extended as the cursor is moved. A closed block is not extended. Setting Marking On, opens the block, and allows it to be extended by moving the cursor. Setting Marking Off, closes the block, and keeps it from being extended. The current status of a block, whether it is opened or closed, can be determined by querying [Marking](#). For instance: `if Query(Marking) // if block is open ... if not Query(Marking) // block is either closed, or there is no block. // need to call one of the isBlock????() routines // to determine for sure.` **Affects:** [MarkLine\(\)](#), [MarkColumn\(\)](#), [MarkStream\(\)](#), [MarkChar\(\)](#), [PushBlock\(\)](#), [PopBlock\(\)](#) See Also: [BlockId](#) Commands: [Unmarkblock\(\)](#), [isBlockMarked\(\)](#), [isBlockInCurrFile\(\)](#), [isCursorInBlock\(\)](#), [isCurrLineInBlock\(\)](#), [MarkToEOL\(\)](#)

Marking a Block

This section describes how to use the mouse to mark a block of text.

To mark a single line:

Place the mouse pointer on the line to be marked. Click the <LeftBtn> three times (without moving the mouse pointer).

To mark multiple lines:

Place the mouse pointer on the beginning line to be marked. Press and hold <Alt> (on the keyboard) and the <LeftBtn>. Then drag the mouse to the desired ending line, and release <Alt> and the <LeftBtn>.

To mark a stream of characters:

Place the mouse pointer on the beginning character to be marked. Press and hold the <LeftBtn>. Then drag the mouse to the desired ending character, and release the <LeftBtn>.

To mark a column:

Place the mouse pointer on the beginning character to be marked. Press and hold <Ctrl> (on the keyboard) and the <LeftBtn>. Then drag the mouse to the desired ending position, and release <Ctrl> and the <LeftBtn>.

To mark a word:

Place the mouse pointer on a character in the word to be marked. Click the <LeftBtn> twice (without moving the mouse pointer).

When the ending position for a block is not displayed on the screen, you can scroll the text in the necessary direction by dragging the mouse pointer outside the current window on the appropriate side. Then drag the mouse

pointer back inside the current window to stop scrolling and continue marking. (To drag the mouse pointer outside a window, there must be a border, other editing window, StatusLine, or HelpLine onto which to move the mouse pointer; otherwise, the file cannot be scrolled in that direction.)

```
+----- Tip -----+
| To unmark a block using the mouse, place the mouse pointer on a |
| blank character on the screen and click the <LeftBtn> twice. The |
| editor attempts to mark a word at that location. Because the blank |
| character is not a word, the editor fails to mark a block, though |
| it succeeds in first unmarking the existing block.                |
+-----+
```

(For more information on blocks, see the chapter on "

[Block Operations.](#)")

Marking and Manipulating a Block of Text

[Opening and Closing a Block](#) ----- A block is a contiguous portion of text that is marked using one of the five [block-marking commands](#) described later in this section under "Block-[Marking](#) Commands." One of three types of blocks can be marked, depending on the specific [marking](#) command used: - a line block, which is a group of entire lines; - a character block, which is a continuous stream of characters that can span multiple lines; or - a column block, which is one or more columns of text that extend across one or more lines. To mark a block of text in the current file: 1. Execute the appropriate block-marking command. This opens the block and begins [marking](#). 2. Move the cursor in the direction of the text to be marked. As the cursor moves, the text included in the block displays on the screen according to the color attribute specified for blocked text. (For more information on how to specify the desired color attribute for blocked text, see "Text - Blocked" of "[Set Colors](#)" under "[Display/Color Options](#)" in the chapter on "[Configuring the Editor Interactively](#)".) If you move to another window that also contains the current file, the text through the cursor position in the new window is included in the block. 3. Position the cursor on the last line and/or character of the block, as appropriate. Then re-execute the same block-marking command. This closes the block and ends marking. If the block-marking command you execute to close a block is different from the one you executed to open the block, the block type changes to the type used to close the block. The marked area is then adjusted to accommodate the new block type, based on the cursor positions where the block was opened and later closed. It is not always necessary to explicitly close the block. Specifically closing the block is required only when the block is to be copied or moved to another location within the same file, or when you wish to move the cursor away from the closing position of the block. In these cases, you must close the block so that movement of the cursor does not affect the bounds of the block. If you execute a block-manipulation command when the block has not been closed, the current position of the cursor designates the closing position of the block. (Execution of either GotoBlockBegin or GotoBlockEnd forces the block to be closed. See the discussion of these two commands under "[Moving to the Beginning or End of a Block](#)" later in this chapter.) You can only mark a block in the current file. However, if you mark a block in one file, then move to another file, the block (whether open or closed) remains marked in the first file. Only one block at a time can be marked. If you mark a block in one file, then move to another file and begin [marking a block](#), the block in the first file is automatically unmarked. [Extending a Block](#) ----- Once a block is closed, you can extend it without having to re-mark the entire block. Simply place the cursor at the desired new position outside the block, then press the appropriate block-marking command. The block is extended to the position where the cursor is located, then re-closed. If the marking command you use to extend the block is different than the one used to previously close the block, the blocked area is adjusted, as necessary, to accommodate the new block type. If you

execute the marking command on a line that passes through or is contained in the current block, AND the location of the cursor is not a logical location to which to extend the block, then the editor unmarks the current block. The editor opens a new block at the current position. If you execute the marking command from another window containing the same (current) file, the block is extended to the position of the cursor in that new window. [Block-Marking Commands](#)

----- There are a number of different commands available for marking blocks. To use any of these commands, position the cursor at the desired location in the text, and execute the command. A line, column, or character block is opened, depending upon the marking command that you use. With the first group of commands, you then move the cursor within the text to mark the area to be included in the block. The marked area grows (or shrinks) as you move the cursor. To explicitly close the block at the new cursor position, re-execute the block-marking command. With the second group of commands, the block is immediately closed once you execute the command. The ending location of the block depends on the command that is executed. * MarkColumn (<Alt K>, or "Mark Column" under Block [Menu](#)) This command opens a column block. It marks by columns, across lines, to identify a rectangular block of text. The character at the cursor position is included in the block. The block begins with the column where you open the block, and extends through the column where you subsequently position the cursor. The block includes the text between these columns on all lines from the beginning line to the ending line of the block. * MarkLine (<Alt L>, or "Mark Line" under Block [Menu](#)) This command opens a line block. It marks entire lines, regardless of the position of the cursor on a line. The block begins with the line where you open the block, and extends through the line on which you subsequently position the cursor. The block includes the entire contents of all lines from the beginning line through the ending line of the block. * MarkStream (<Alt A>, or "Mark Character" under Block [Menu](#)) This command opens a character block. It marks a stream of characters that includes all characters from the beginning to the ending position of the block. The character at the cursor position is included in the block. The block begins with the character where you open the block, and extends through the character on which you subsequently position the cursor. The block includes the entire contents of all lines between the beginning line and the ending line of the block. For a similar command, but one that does NOT include the character at the [cursor](#) position, see MarkChar. * MarkChar (<Alt I>) Similar to the MarkStream command, this command opens a character block. It marks a stream of characters that includes all characters from the beginning to the ending position of the block. However, the inclusion of the character at the cursor position is handled in a different manner for this command than for MarkStream, as follows: - If you open a block, then move the cursor forward in the text, the block begins with the character where you opened the block, and extends through the character preceding the cursor position. - If you open a block, then move the cursor backward in the text, the block begins with the character preceding the one where you opened the block, and extends through the character at the cursor position. Note that highlighting of the block does not begin until you move the cursor from the character where you opened the block. As with MarkStream, the block includes the entire contents of all lines between the beginning line and the ending line of the block. * MarkWord (<Ctrl K T>) This command opens and immediately closes a character block. It marks the word in which the cursor is positioned. The block begins with the first character of the word in which you execute the command, and ends at the last character of the word. Regardless of your position within the word when you execute MarkWord, the entire word is marked. (For more information on how the editor defines a word, see "[WordSet Characters](#)" under "[Command/Format Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") * MarkToEOL (<Alt Y>, or "Mark to EOL" under Block [Menu](#)) This command opens and immediately closes a character block. It marks a block that begins at the cursor position, and ends at the last character on the line. * MarkParagraph ("Mark Paragraph" under Block [Menu](#)) This command opens and immediately closes a line block. It marks the paragraph in which the cursor is positioned. The block begins with the first line of the paragraph in which you execute the command, and ends on the last line of the paragraph. Regardless of your position within the paragraph when you execute MarkParagraph, the entire paragraph is marked. A paragraph, as determined by this command, depends on the file type (extension) of the current file: - For certain programming-language file types, paragraphs are equated with functions/procedures. Supported file types [include](#) SAL (.s, .ui), C (.c, .h), C++ (.cpp, .hpp), Pascal (.pas), and xBase (.prg, .spr, .mpr, .qpr, .fmt, .frg, .lbg, .ch). - For non-programming-language file types (or other non-supported programming-language file types), a paragraph is any contiguous group of non-blank lines.

[Unmarking a Block](#) ----- Once you have finished with a particular block of text, it can be unmarked so that it is no longer selected as a block, using the following command. * UnMarkBlock (<Alt U>, or "UnMark" under Block [Menu](#)) This command unmarks the block that is currently marked. The block is unmarked even if it is not located in the current file. When you execute UnMarkBlock, the cursor remains at its current position in the current file,

regardless of where the block is located. Generally, a block remains marked until you unmark it. If you mark a block in one file, then move to another file, the block in the first file remains marked. However, if you begin to mark a block in another file, the block in the first file is automatically unmarked. If you transfer (copy, move, or cut) a block to another location, the block marking moves with the text. The Paste command, though, can be configured to unmark the block after the Paste operation. (See "[The Clipboard Paste Commands](#)" under "[Manipulating Blocks via ClipBoards](#)" later in this chapter.) If a block is marked and closed in the current file, and you then issue a conflicting marking command on a line contained within the block, the closed block is unmarked. The editor opens a new block at the current position.

[Manipulating Text in a Block](#) ----- This section describes commands to: copy, delete, or move a block; save (write) a block to disk; fill a block with a specified character; shift text within a block; and change the case of characters in a block. To use these commands, you first mark the desired block of text. If you are copying or moving the block, you then position the cursor at the desired location. Finally, you execute the appropriate block command. *

CopyBlock (<Alt C>, or "Copy" under Block [Menu](#)) This command makes a copy of the block that is marked, then inserts that copy into the current file at the cursor location. The block does not have to be located in the current file; thus, you can copy a block of text from one file to another. Once the block is copied, the new copy is marked as a block. The type of block that you copy (line, column, or character) determines how the editor inserts the block of text at the new location, as follows:

- A line block is inserted according to the setting of the configuration option, "[Insert Line Blocks Above](#)":
 - If "[Insert Line Blocks Above](#)" is OFF, the block is inserted BELOW the current line. Any lines that follow the current line are shifted downward. (This is the standard setting for this option.)
 - If "[Insert Line Blocks Above](#)" is ON, the block is inserted ABOVE the current line. The current line, along with any lines that follow, are shifted downward.
- A column block is inserted beginning at the cursor column, and extending across the current line and down the lines that follow, to accommodate the text in the block. Any text on each of the affected lines, from and to the right of the current cursor column, is shifted to the right.
- A character block is inserted beginning at the cursor position. Line breaks occur just as they were when the original text was placed into the block. Any text on the current line at and to the right of the cursor position, is moved to the end of the last line of text inserted from the block. Any lines below the current line are shifted downward, following the last line of text inserted from the block. *

OverlayColumnBlock (<Alt Z>) This command is intended for use with column blocks. It functions the same as CopyBlock for line and character blocks. For column blocks, it behaves the same as CopyBlock, except that text on the affected lines is not shifted to the right; rather, it is overlaid by the text inserted from the block. *

DelBlock (<Alt G>, or "Delete" under Block [Menu](#)) This command deletes the text in the block that is in the current file. The deleted block is placed in the Deletion Buffer. (For more information on the Deletion Buffer, see "[The Deletion Buffer: A Safety Net](#)" in the chapter on "[Manipulating Text](#).") If a block is not marked in the current file when you execute DelBlock, nothing is deleted. When the block is deleted, text to the right of or below the deleted text is shifted, according to the type of block that is deleted:

- If a line block is deleted, any lines below the block are shifted upward.
- If a column block is deleted, any text to the right of the block is shifted to the left.
- If a character block is deleted, any text to the right of the final character in the block is shifted left. If any entire lines were included in the deleted block, then the text below the block is shifted upward. If the block extended beyond the last character on the last line of the block, then the text from the line below the block is joined with the line where the block began. *

FillBlock (<Ctrl K L>, or "Fill" under Block [Menu](#)) This command fills the block in the current file with a specified character string. If you execute FillBlock, and a block exists in the current file, the editor prompts with "Fill block with:". You can specify a string of one or more characters at this prompt. Once you press <Enter>, the block is filled with the specified character string. This command is useful for filling an area of text with spaces. *

MoveBlock (<Alt M>, or "Move" under Block [Menu](#)) This command is similar to the CopyBlock command. However, MoveBlock deletes the text in the block from its original location, then inserts it at the cursor location. Each block type (line, column, and character) is inserted at the new location just as it is with CopyBlock. Text to the right of or below the deleted text is shifted, according to the type of block that you moved (similar to the DelBlock command).

+-----+ Tip
 -----+ | MoveBlock can be configured to overlay existing text when a column | | block is moved, similar to the way a block is copied when you | | execute OverlayColumnBlock. (For more information on how to | | configure the behavior of a command, see | | "[Creating an External Compiled Macro](#)" and | | "[Adding Macros to the User-Interface \(UI\) Macro File](#)" in the | | chapter on "[Fully Customizing the Editor](#).") |

+-----+ * **SaveBlock** (<Alt W>, or "Write to File" under Block

[Menu](#)) This command saves, or writes, the block in the current file to a file on disk. The editor prompts for the name of the file with "Save block to:". If you enter a filename that already exists on disk, you are given the option to overlay the existing file, append to the existing file, or cancel your response. If a block is not marked in the current file when you execute SaveBlock, the prompt "Save as:" is displayed. The editor then attempts to write the entire contents of the current file to disk, under the name specified. If the filename that you enter already exists on disk, you are given the option to overlay the existing file, append to the existing file, or cancel your response. * SaveAs (<Alt W>, or "Write to File" under Block [Menu](#) w/NO BLOCK MARKED) This command issues the prompt, "Save as:", and then saves the current file. The buffer contents of the current file are written to disk, using the name you specify at the prompt. Press <Escape> at the "Save as:" prompt to cancel this command. If you enter a filename that already exists on disk, the following selection [menu](#) is displayed: + File exists + |*Append*****| | Overlay | | Cancel | +-----+ You then choose whether to append the current file to the existing file, overlay the existing file with the current file, or cancel your response and re-display the "Save as:" prompt. Following this command, the file remains loaded in memory and included in the Ring. The name of the file in memory is not changed. * Shift (<Ctrl F7>, or "Shift" under Block [Menu](#)) This command shifts the text within the block in the current file, if the cursor is located in the block. If the cursor is not located inside a block, the current line is shifted. When you execute Shift, the editor prompts with: <Left>,<Right> or <Tab>,<Shift Tab> to shift text; <Enter> when done To shift the text to the right or left by one character at a time, press the right or left cursor key, respectively. To shift the text right or left by the number of characters in the TabWidth, press <Tab> or <Shift Tab>, respectively. Press <Enter> or <Escape> to terminate this command. Note that text which you shift beyond the edge of the block (or beyond the beginning or end of the line) is deleted and cannot be recovered. * ShiftLeft (<Shift F7>) This command is a variation of Shift. ShiftLeft shifts the text left by one character at a time. If the cursor is located in a block in the current file, the entire text within the block is shifted left. If the cursor is not located inside a block, the current line is shifted left. * ShiftRight (<Shift F8>) This command is a variation of Shift. ShiftRight shifts the text right by one character at a time. If the cursor is located in a block in the current file, the entire text within the block is shifted right. If the cursor is not located inside a block, the current line is shifted right. [Changing the Case of Characters in a Block](#) ----- A group of commands are available that allow you to change the case of alphabetic characters in a block. These case-change commands are Upper, Lower, Flip, and Capitalize. You can also use these commands to change the case of characters on the current line, or in the word at the cursor position, without having to explicitly mark a block. When executed, the case-change commands display a selection menu with the following options: +-----+ |*Word*at*[Cursor](#)*| | Current Line | | Block | +-----+ To change the case of characters in the word in which the cursor is positioned, choose "Word at Cursor" from the selection menu. This option does not require that the word be marked as a block. The cursor can be located anywhere within the word. (For more information on how the editor defines a word, see "[WordSet](#) Characters" under "[Command/Format Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") To change the case of characters on the current line, choose "Current Line" from the selection menu. This option does not require that the line be marked as a block. To change the case of characters in a block, position the cursor in the block before you execute the desired case-change command. Then choose "Block" from the selection menu. No action occurs if "Block" is selected and either no block exists in the current file, or the cursor is outside the block. Following are the available case-change commands. * Upper (<Shift F4>, or "Upper" under Block [Menu](#)) This command changes ALL indicated characters to upper [case](#). * Lower (<Alt F4>, or "Lower" under Block [Menu](#)) This command changes ALL indicated characters to lower [case](#). * Flip ("Flip" under Block [Menu](#)) This command reverses the case of ALL indicated characters; that is, lower-[case](#) characters are changed to upper [case](#), and upper-[case](#) characters are changed to lower [case](#). * Capitalize ("Capitalize" under Block [Menu](#)) This command capitalizes each word in the indicated text; that is, the FIRST character of each word is changed to upper [case](#), and every other letter of each word is changed to lower [case](#). [Moving to the Beginning or End of a Block](#) ----- The following two commands allow you to move the cursor to the beginning or the end of a block. * GotoBlockBegin (<Ctrl >) This command moves the cursor to the beginning of the currently marked block. If the block is not in the current file, the file containing the block becomes the current file. In a character or column block, the cursor moves to the first column position on the first line of the block. In a line block, the cursor is placed on the first line of the block, but the column position of the cursor does not change. If you execute GotoBlockBegin before you have closed the block, the editor first closes the block at the cursor position, then moves the cursor to the beginning of the block. * GotoBlockEnd

(<Ctrl]>) This command moves the cursor to the end of the currently marked block. If the block is not in the current file, the file containing the block becomes the current file. In a character or column block, the cursor moves to the last column position on the last line of the block. In a line block, the cursor is placed on the last line of the block, but the column position of the cursor does not change. If you execute GotoBlockEnd before you have closed the block, the editor first closes the block at the cursor position, then moves the cursor to the end of the block.

MarkLine

Marks a line block.

Syntax: [INTEGER MarkLine](#)([INTEGER](#) start_line, [INTEGER](#) end_line) - start_line is the optional block starting line. - end_line is the optional block ending line. Returns: Non-zero. Notes: The commands [isBlockInCurrFile](#)(), [isBlockMarked](#)(), and [isCursorInBlock](#)() [return](#) _LINE_ for this block type. Initially executing this command marks one end of a line block at the cursor position. As the cursor is moved, the block grows with it. Executing the command again marks the other end of the block at the cursor position. If [MarkLine](#)() is executed outside of a marked block, the block is extended to the current cursor line. The block type will be a line block, even if it was a different type of block before the [MarkLine](#)() command was issued. If [MarkLine](#)() is executed with the cursor inside a block, the existing block is unmarked and a new block marking action is begun. See Also: [MarkBlockBegin](#)(), [MarkBlockEnd](#)(), [MarkStream](#)(), [MarkChar](#)(), [MarkColumn](#)(), [MarkWord](#)(), [Mark](#)(), [MarkToEOL](#)(), [isBlockInCurrFile](#)(), [isBlockMarked](#)(), [isCursorInBlock](#)() **Variables:** [Marking](#)

MarkStream

Marks a character block that includes the cursor position.

Syntax: [INTEGER MarkStream](#)() Returns: Non-zero. Notes: Unlike [MarkChar](#)(), this block command includes the cursor position in the block, and is commonly referred to as an inclusive style block. The commands [isBlockInCurrFile](#)(), [isBlockMarked](#)(), and [isCursorInBlock](#)() [return](#) _INCLUSIVE_ for this block type. Initially executing this command marks one end of a character block at the cursor position. As the cursor is moved, the block grows with it. Executing the command again marks the other end of the block at the cursor position. If [MarkStream](#)() is executed outside of a marked block, the block is extended to the current cursor position. The block type will be a character block, even if it was a different type of block before the [MarkStream](#)() command was issued. If [MarkStream](#)() is executed with the cursor inside a block, the existing block is unmarked and a new block marking action is begun. See Also: [MarkBlockBegin](#)(), [MarkBlockEnd](#)(), [MarkChar](#)(), [MarkColumn](#)(), [MarkLine](#)(), [MarkWord](#)(), [Mark](#)(), [MarkToEOL](#)(), [isBlockInCurrFile](#)(), [isBlockMarked](#)(), [isCursorInBlock](#)() **Variables:** [Marking](#)

MarkToEOL

Marks a character block from the cursor position to the last character on the current line.

Syntax: [INTEGER MarkToEOL](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If the cursor is at or past the end-of-line position, this command returns [FALSE](#). (The end-of-line position is one position past the last character on the line.) The end-of-line position is not included in the block. See Also: [MarkLine](#)(), [MarkColumn](#)(), [MarkChar](#)(), [MarkStream](#)() **Variables:** [Marking](#)

MarkWord

Marks the current word as a block.

Syntax: [INTEGER](#) [MarkWord](#)([STRING](#) bit_encoded_32_char) - bit_encoded_32_char is an optional bit-encoded 32-character string denoting a user-defined WordSet. Use the [ChrSet\(\)](#) command to generate this [string](#). Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: The WordSet variable determines what constitutes a word. If the cursor is not positioned on a character in the current [WordSet](#), no action occurs and zero is returned. **Examples:**

[MarkWord](#)() // marks the current word as a block /

***** marks the current word as a block, but uses 0-9 as the character set. So, only a word containing all numbers will be marked.

*****/ [MarkWord](#)([ChrSet](#)("0-9")) See Also: [isWord\(\)](#), [MarkStream\(\)](#), [MarkChar\(\)](#), [MarkColumn\(\)](#), [MarkLine\(\)](#), [MarkToEOL\(\)](#), [Mark\(\)](#), [GetWord\(\)](#) **Variables:** [WordSet](#), [Marking](#)

MatchFilename

Checks a filename against an ambiguous file specification.

Syntax: [INTEGER](#) [MatchFilename](#)([STRING](#) fn, [STRING](#) mask) - fn is the filename to compare. - mask is the filename specification against which fn is compared. It can contain the wildcard characters "*" and "?". Returns: Non-zero if fn matches mask; [otherwise](#), zero ([FALSE](#)). Notes: Neither fn nor mask should include a drive and/or directory specification. This command is used by the [BuildPickBufferEx\(\)](#) command. **Examples:** [MatchFilename](#)("abcdef", "abc*") //returns non-zero (MATCH) [MatchFilename](#)("abcdef", "x*") //returns zero ([FALSE](#)) [MatchFilename](#)("e.exe", "?*.exe") //returns non-zero (MATCH) [MatchFilename](#)("anyname", "*.*) //returns non-zero (MATCH) See Also: [BuildPickBufferEx\(\)](#)

Max

Returns the greater of two specified numbers.

Syntax: [INTEGER](#) [Max](#)([INTEGER](#) a, [INTEGER](#) b) - a is any integer expression to compare with b. - b is any [integer](#) expression to compare with a. Returns: The larger of the two numbers, a and b. **Examples:** [integer](#) largest = [Max](#)(100, [NumLines](#)()) See Also: [Min](#)()

MaxHistoryPerList

Specifies the maximum number of items stored in each history list.

Values: [INTEGER](#) - [1..1000] **Default:** 50 Notes: MaxHistoryPerList applies to the list of history items maintained [for](#) each particular command (or command group) prompt. (The related variable, MaxHistorySize, applies to the total number of items in the History Buffer for ALL prompts.) If PersistentHistory is set ON, the items stored in the History Buffer are maintained across editing sessions. The History Buffer then has the potential to grow quite large.

[MaxHistoryPerList](#), along with [MaxHistorySize](#), can be used to limit the History Buffer to a manageable size. [Affects: AddHistoryStr\(\)](#) [and](#) any command that prompts for input. See Also: [MaxHistorySize](#), [PersistentHistory](#)

MaxHistorySize

Specifies the maximum number of items that can be stored in the History Buffer.

Values: [INTEGER](#) - [1..10,000] **Default:** 1000 **Notes:** MaxHistorySize specifies the total number of items for all prompts. (The related variable, [MaxHistoryPerList](#), applies to the list of history items maintained for each particular command (or command group) prompt.) If you do [not](#) want history for your commands, except for the last item entered, set MaxHistorySize to 1 (or set [MaxHistoryPerList](#) to 1). If PersistentHistory is set ON, the items stored in the History Buffer are maintained across editing sessions. The History Buffer then has the potential to grow quite large. [MaxHistorySize](#), along with [MaxHistoryPerList](#), can be used to limit the History Buffer to a manageable size. [Affects: AddHistoryStr\(\)](#) [and](#) any command that prompts for input. See Also: [MaxHistoryPerList](#), [PersistentHistory](#)

MaxRecentFiles

Specifies the maximum number of files that can be stored in the recently-edited-files list.

Values: [INTEGER](#) - [1..5000] **Default:** 200 **Notes:** The editing command ListRecentFiles (implemented as the mListRecentFiles() macro) uses this variable to determine the number of recently-edited files to display in the PickList. See Also: [PersistentRecentFiles](#)

MenuKey

Returns the character code and scan code of the key pressed to exit the most recently used

[menu](#). **Syntax:** [INTEGER MenuKey\(\)](#) **Returns:** An integer comprised of the character and scan code of the key that was pressed to exit the most recently used menu. **Notes:** Key names in the format <editor key-name> (as shown by the ShowKey macro, located in the MAC\ editor subdirectory) can be compared with the results returned from [MenuKey\(\)](#) to determine the key used to exit the menu. The numerical codes returned by ShowKey can also be used. [Menu](#) options declared with _MF_DONT_CLOSE_, _MF_SKIP_, or _MF_DIVIDE_ menu flags cannot be used to exit the menu, therefore [MenuKey\(\)](#) will [not return](#) those keys. See [GetKey\(\)](#) for techniques to isolate the character portion and the scan code portion of the key. For more information on menus, see "[Menus](#)" in the chapter on "[Using Macro Language Features](#)." **Examples:** [Menu](#) TabTypeMenu() // [Menu](#) includes "quick-keys" for // each selection "&Hard" "&Soft" "S&mart" "&Variable" [end proc](#) SetTabs() TabTypeMenu() // call the menu [case MenuKey\(\)](#) [when](#) <h>, <Shift H> // if "Hard tabs" [Set\(TabType, _HARD_\)](#) [when](#) <s>, <Shift S> // if "Soft tabs" [Set\(TabType, _SOFT_\)](#) [when](#) <m>, <Shift M> // if "Smart tabs" [Set\(TabType, _SMART_\)](#) [when](#) <v>, <Shift V> // if "Variable tabs" [Set\(TabType, _VARIABLE_\)](#) [when](#) <Escape> // if escape pressed [return\(\)](#) // just [return when](#) <Enter> // CR, use [MenuOption\(\)](#) // to set [tabtype Set\(TabType, MenuOption\(\) - 1\)](#) [endcase end](#) See Also: [MenuStr\(\)](#), [MenuOption\(\)](#), [GetKey\(\)](#)

MenuOption

Returns the option number of the last menu option selected.

Syntax: [INTEGER MenuOption\(\)](#) Returns: The number of the last menu option that was selected. Options are numbered sequentially, beginning with 1 and ending with the number of items in the menu. Notes: Options flagged with `_MF_SKIP_` or `_MF_DIVIDE_` [menu](#) flags are counted as [menu](#) options, even though they cannot be selected. For more information on menus, see "[Menus](#)" in the chapter on "[Using Macro Language Features](#)." **Examples:** Macro `SetTabs()` works in this example because the second parameter for `Set(TabType, type)` is a number from 0 to 3, corresponding to the order of options in `TabTypeMenu()`. [Menu](#) `TabTypeMenu() "&Hard" "&Soft" "S&mart" "&Variable"` [end proc](#) `SetTabs() TabTypeMenu() // call the menu Set(TabType, MenuOption() - 1) // set tabtype end` See Also: [MenuStr\(\)](#), [MenuKey\(\)](#)

Menus

The menu system has many features. You can even redesign the standard pull-down Menu system that comes with the editor. Within the menu system, you can design both pull-down and pop-up menus. Menus can be used to simply display options from which the user can make a selection, or they can be used as a complex series of menus within menus. Pull-down menus are used in conjunction with a menubar. To use this type of menu, you first activate the menubar. The menubar then displays on the top line of the screen. It consists of one or more menu items that can be selected. Each menu item on the menubar is linked to a separate menu definition. To select one of the menu items, you position the cursor on the desired item and press <Enter> (or press the quick-key, if one is assigned). This "pulls down" the menu called by that menu item. Pop-up menus are called by a macro or are assigned to a key. Pop-up menus are displayed on the screen either at a specified location (if designated in the menu definition) or simply near the cursor position. This type of menu "pops up" on the screen when it is invoked by the associated macro or keystroke.

[Defining a Pull-Down or Pop-Up Menu](#) ----- Following is the syntax for [defining a pull-down or pop-up menu](#). (The syntax for [defining a menubar](#) is described later in this section.) [menu](#) menu_name() [title = constant_string] [width = constant_integer] [x = constant_integer] [y = constant_integer] [history [= integer]] [command = function] [CheckBoxes] [NoEscape] [NoKeys] option_string [var_text_function : width], function, flag, message ... option_string [var_text_function : width], function, flag, message [end](#) [menu_name] where: - menu specifies this as a menu definition. - menu_name is a unique name associated with this pull-down or pop-up [menu](#). - A number of setup options for the menu can optionally be set. The first options listed define the menu display area, which the editor encloses within a box. The remaining options set other parameters that apply to the entire menu definition. You can include any or all of the following options. The order of these options is [not](#) important, but they **MUST** be declared BEFORE any option_strings are declared. - title indicates the title of the menu, and is specified as a constant string. It displays within the top border of the menu, and is centered over the menu area. - width indicates the width of the menu area, and is specified as a constant integer. The editor draws a box around the menu area, based on the width of the menu area and the number of entries defined in the menu. If no width is specified, or the specified width is too small, the editor determines a width that accommodates the longest menu item entry. - x indicates the left column boundary of the menu, and is specified as a constant integer. This indicates the leftmost column position at which the menu displays on the screen. If not specified, the menu displays near the cursor column position. For pull-down menus, this option should not be specified (thus causing the menu to display immediately under the associated menubar item). - y indicates the top boundary of the menu, and is specified as a constant integer. This indicates the top position at which the menu displays on the screen. If not specified, the menu displays near the [cursor](#) line position. For pull-down [menus](#), this option should [not](#) be specified (thus causing the menu to display immediately under the associated [menubar](#) item). - history, if specified, instructs the menu to keep an historical indicator of the last menu item selected. It also allows you to specify an integer (from 1 through the number of menu items included in the menu definition) that identifies the initial menu item entry position. - If history is not specified for a menu, then each time the menu is called, the cursor bar is positioned on the first menu item. - If history is specified, but not set to an integer value, then each call to the menu causes the cursor bar to be positioned on the item that was selected the last time the

menu was called. - If history is set to a constant integer, then the specified value determines the menu item on which the cursor bar is positioned the FIRST time the menu is called during an editing session. Each subsequent call to the menu causes the cursor bar to be positioned on the item that was selected the last time the menu was called. - If history is set to a variable integer, then this variable determines the menu item on which the cursor bar is positioned EACH time the menu is called. - command indicates the default native or user-defined command to execute when a menu item is selected for which a function (described below) is [not](#) specified. - CheckBoxes, if specified, instructs the menu to allocate a two-column checkbox area at the beginning of each indicated menu item. This provides an area to display a check mark for any item(s) on which the `_CHECKED_` flag option (described below) is specified. - NoEscape, if specified, dictates that pressing the <Escape> key does not cancel the menu. Normally, when you press <Escape> in a menu, the current menu is cancelled and is removed from the screen. - NoKeys, if specified, instructs the menu not to display key assignments that exist for any functions (described below) specified in [menu](#) items. - option_string MUST be a constant string. At least one option_string must be specified in a menu definition. Each option_string is displayed as a menu item. You can specify as many option_strings as will fit on the display screen. Within the constant string, an ampersand (&) can be placed before a specific letter, which forces that letter to be highlighted in the menu. The letter then serves as a "quick-key" that can be used as a shortcut to select that menu item. Pressing the highlighted letter from the keyboard is equivalent to moving the cursor bar to the menu item and pressing <Enter>. Avoid specifying the same letter for more than one menu option; if the same letter is used in more than one menu option, only the first menu option that uses the letter will be selected when the letter is pressed. (If you wish to display an & as an actual character in the menu string, specify it as &&.) - [var_text_function : width] is optional. The square bracket ([]) delimiters are required if this option is specified; a comma does NOT separate the option_string and this option. This option is used to display the current setting or value of the menu item (identified by option_string) that you are presenting to the user [for](#) modification. The value is displayed in square brackets ([]) [to](#) the right of option_string. The user can then select this item and modify the value or setting indicated within []. - var_text_function can be either a string or integer function to call to obtain the current setting or value of the menu item, or it can be a variable. The [Query\(\)](#) command is a useful function to call with this option. - width is the maximum width needed by the var_text_function. If the specified width does not accommodate the value of var_text_function, the editor displays what it can, and places a ">" at the end to indicate that the value is longer than what is displayed. - function is an optional native or user-defined command to be executed if the user selects the associated menu item. If function has been assigned to a key, the key assignment will be displayed for this menu entry (unless NoKeys is specified). If function is [not](#) specified, the default command (specified at the beginning of the menu declaration) is used. - flag is an optional parameter indicating how this menu item is to be processed. Option [values](#) for this parameter can be specified explicitly within the menu definition; or they can be specified using an integer expression, which allows you to dynamically set the desired options. (NOTE: The editor supports certain previous [values](#) for flag, as identified below; however, these previous [values](#) cannot be referenced outside a menu definition, so they cannot be used to dynamically set flag options.) If nothing is specified for flag, `_MF_CLOSE_ALL_BEFORE_` and `_MF_ENABLED_` are assumed. Valid options and the associated processing operations are: - `_MF_CLOSE_BEFORE_` instructs the editor to close the current menu before calling function. (The previous value, CloseBefore, is still supported, but cannot be referenced outside a menu definition.) - `_MF_CLOSE_ALL_BEFORE_` instructs the editor to close ALL currently displayed menus before calling function. This is the default if no flag value is specified. Select this option for [menus](#) that change the current window. (The previous value, CloseAllBefore, is still supported, but cannot be referenced outside a menu definition.) The effect of this option is to immediately remove all [menus](#) currently displayed on the screen, and then process the indicated function. Once the operation of the function is complete, the user is returned to the editing state that was in effect before the menu, or system of [menus](#), was invoked. - `_MF_CLOSE_AFTER_` instructs the editor to close the current menu after calling function. (The previous value, CloseAfter, is still supported, but cannot be referenced outside a menu definition.) - `_MF_CLOSE_ALL_AFTER_` instructs the editor to close ALL currently displayed [menus](#) after calling function. (The previous value, CloseAllAfter, is still supported, but cannot be referenced outside a menu definition.) - `_MF_DONT_CLOSE_` instructs the editor NOT to close the menu until the user presses <Escape>. This allows you to leave the menu on the screen after this item is processed, so that another [menu](#) item can then be selected. (The previous value, DontClose, is still supported, but cannot be referenced outside a menu definition.) - `_MF_SKIP_` instructs the editor to display the indicated option_string, but not allow the user to select this item from

the [menu](#). This option is useful for displaying additional information [about menu](#) item entries. (The previous value, Skip, is still supported, but cannot be referenced outside a menu definition.) - `_MF_DIVIDE_` instructs the editor to center the indicated option_string. A line extends from either side of the displayed option_string, out to the left and right borders of the menu. If option_string is empty, a solid dividing line is displayed. The user is not allowed to select this option from the menu. (The previous value, Divide, is still supported, but cannot be referenced outside a menu definition.) This option provides a way to group items on the menu, and even include a title over a group of items. - `_MF_ENABLED_` instructs the editor to enable the associated function for this [menu](#) item. Thus, if this [menu](#) item is selected, function will be called. This option is assumed unless `_MF_GRAYED_` or `_MF_DISABLED_` is specified. - `_MF_GRAYED_` instructs the editor to disable the associated function for this menu item, and to display this item using the "gray" [menu](#) color attributes. If this [menu](#) item is selected, function will **NOT** be called. - `_MF_DISABLED_` instructs the editor to disable the associated function for this [menu](#) item. If this [menu](#) item is selected, function will **NOT** be called. - `_MF_CHECKED_` instructs the editor to place a check mark to the left of this [menu](#) item. If CheckBoxes has also been specified for this menu, then the check mark will be placed within the checkbox area. - `_MF_UNCHECKED_` instructs the editor to remove the check mark to the left of this menu item (if a check mark is present). - message is an optional constant string that is displayed at the bottom of the screen when the associated menu item is highlighted. The specified string can be as long as the screen is wide. (If the string is longer than the width of the screen, the string is truncated to fit.) This option is useful for providing the user with a help message for the [menu](#) item. - end signifies the end of the menu definition. - The menu_name may optionally be repeated after the end statement on the same line. A note [about](#) setting flags: You should exercise care when setting flag options within a multi-menu system. A secondary menu item flag setting may override the calling menu's flag setting. For example, if a primary menu has set the flag for an item to `_MF_DONT_CLOSE_`, but that menu item invokes another menu and the selected item from the secondary menu sets the flag to `_MF_CLOSE_ALL_BEFORE_`, then the primary menu is not re-displayed after the item from the secondary menu is processed. Following are example menu definitions, similar to menus included in the Main pull-down Menu system included in the standard editor user-interface (TSE.UI). The first example is similar to the File Menu. [Menu](#) FileMenu() history "&Open...", [EditFile\(\)](#) "&Insert...", [InsertFile\(\)](#) "&File Manager...", [ExecMacro\(\)](#)("f *.*") "", , Divide "&Next", [NextFile\(\)](#) "&Previous", [PrevFile\(\)](#) "Lis&t Open...", [ExecMacro\(\)](#)("listopen") "List &Recent...", mListRecentFiles() "Current File" , , Divide "&Save", [SaveFile\(\)](#) "Save &As...", [SaveAs\(\)](#) "Save && C&lose", [SaveAndQuitFile\(\)](#) "&Close", [QuitFile\(\)](#) "Chan&ge Name...", [ChangeCurrFilename\(\)](#) "S&how File Info", [ExecMacro\(\)](#)("fileinfo") "All Files" , , Divide "Sa&ve All", [SaveAllFiles\(\)](#) "Save All && &Exit", [SaveAllAndExit\(\)](#) "E&xit", [Exit\(\)](#) end You will notice that all of the items on the FileMenu() call a function, except the options with the Divide flag. mListOpenFiles() and mListRecentFiles() are macros defined in TSE.UI, while the other functions are native [editor commands](#). The second example is similar to the more complicated Util Menu, contained in the TSE.UI user-interface macro file. It has several enhanced features in it, including calling another menu. It also includes items that display the current setting within [] characters, and allow the user to modify the setting. [Menu](#) UtilMenu() history "&Line Drawing" [OnOffStr([Query\(\)](#)(LineDrawing)):3], [Toggle\(\)](#)(LineDrawing), DontClose "Li&ne Type >", [LineTypeMenu\(\)](#), DontClose "", , Divide "&Sort", [ExecMacro\(\)](#)("sort " + [Str\(\)](#)(sort_flags)) "Sort &Order" [ShowSortFlag() : 10], ToggleSortFlag(_DESCENDING_), DontClose "&Case-Sensitive Sort" [OnOffStr((sort_flags & _IGNORE_CASE_) == 0):3], ToggleSortFlag(_IGNORE_CASE_), DontClose "", , Divide "S&pell Check >", [ExecMacro\(\)](#)("SpellChk") "&ASCII Chart...", mAsciiChart() "&Date/Time Stamp", mDateTimeStamp() "S&hell", [Shell\(\)](#) "Captu&re OS Output", [ExecMacro\(\)](#)("Capture") "Potpo&urri...", [ExecMacro\(\)](#)("Potpourr") "", , Divide "Change &Video Mode >", [ExecMacro\(\)](#)("Video"), DontClose "He&x View" [OnOffStr([DisplayMode\(\)](#) == _DISPLAY_HEX_):3], [DisplayMode\(\)](#)(if([DisplayMode\(\)](#) == _DISPLAY_TEXT_, _DISPLAY_HEX_, _DISPLAY_TEXT_)) "&Template Expansion" [OnOffStr(misTemplateLoaded()):3], mToggleTemplateExpand(), DontClose end OnOffStr(), ShowSortFlag(), ToggleSortFlag(), mDateTimeStamp(), mAsciiChart(), VideoModeMenu(), misTemplateLoaded(), and mToggleTemplateExpand() are [macros](#) defined in TSE.UI. The other functions are native [editor commands](#). [Defining a Menubar](#) ----- Menubars are basically a subset of menus. Whereas a menu can call commands or other menus, a menubar can call ONLY menus (which become pull-down menus). Menubars recognize only one of the setup options; namely, the history option. And menubars do **not include** any flag options or allow you to specify a message. The syntax for a

menubar is: [menubar](#) menubar_name() [history [= [integer](#)]] option_string, menu_name() ... option_string, menu_name() [end](#) [menubar_name] where: - menubar specifies this as a menubar definition. - menubar_name is a unique name associated with this menubar. - menu_name is the name of the menu to call (which becomes a pull-down [menu](#)). - All other indicated options are the same as shown for the menu syntax. A menubar is simple to define. Following is an example menubar, similar to the Main pull-down menubar in the standard editor user-interface file (TSE.UI): [MenuBar](#) MainMenu() history "&File" , FileMenu() "&Block" , BlockMenu() "&Text" , TextMenu() "&Search" , SearchMenu() "&Window" , WindowMenu() "&Clip" , ClipBoardMenu() "&Macro" , MacroMenu() "&Print" , PrintMenu() "&Util" , UtilMenu() "&Options" , ReconfigMenu() "&Help" , HelpMenu() [end](#) The quoted area is the text that displays on the menu. The "&" tells the editor to highlight the following letter, and make it the quick-key for that [menu](#) item. All of the functions executed are pull-down menus that are defined in the TSE.UI user-interface macro file.

MenuStr

Returns the specified menu option string from the named menu.

Syntax: [STRING](#) MenuStr([MENU](#) menu_name, [INTEGER](#) option_num) - menu_name is the name of a menu. It must be defined in the same macro file that contains the [MenuStr\(\)](#) command. - option_num is the desired option number from the menu. Returns: A string corresponding to the option specified for the named menu. If option_num is less than 1 or greater than the number of options on the menu, a null string is returned. Replaceable text used as part of a menu option is not returned. Text is returned from options flagged with _MF_DIVIDE_ or _MF_SKIP_ [menu](#) flags. Notes: For more information on menus, see "[Menus](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [MenuOption\(\)](#), [MenuKey\(\)](#)

Message

Writes the specified expression(s) on the editor StatusLine.

Syntax: [Message](#)([[STRING](#)] | [[INTEGER](#)] expr [: [INTEGER](#) width [: [STRING](#) pad_char [: [INTEGER](#) base]]] [, [[STRING](#)] | [[INTEGER](#)] expr [: [INTEGER](#) width [: [STRING](#) pad_char [: [INTEGER](#) base]]] ...) - expr represents each expression. Multiple [expressions](#) must be separated by commas. [Expressions](#) can be of type [integer](#) or [string](#). - width is the optional format width. A negative value will left justify the expression, whereas a positive value will right justify the expression. - pad_char is the optional character that is used to pad the expression to fit the specified width. The space character is used if this character is [not](#) specified. - base is the optional base to use when converting numeric [expressions](#). Base 10 is used if this value is [not](#) specified. Returns: Nothing. Notes: The message remains on [the StatusLine](#) until the user does something (for example, moves the cursor) that causes the editor to update [the StatusLine](#). Specifying any [optional parameters](#) requires specifying all [optional parameters](#) which precede it. Thus, to specify base requires that width and pad_char be specified. If a numeric expression yields a negative result and the expression is to be right justified (width is a positive value), then the pad_char character is placed between the negative sign "-" (which is left justified) and the actual number. base can only be specified for [numeric expressions](#). If an expression will not fit within the specified width, the expression is truncated to width less 1, and the ">" character is placed in the last position. The [Message\(\)](#) command supports the same expression format as the [Format\(\)](#) command. See Also: [Warn\(\)](#), [Format\(\)](#)

Methods For Interactively Configuring the Editor

The editor provides interactive access to configuration options through two avenues: by including certain options directly on the pull-down Menu, and by making a comprehensive set of editing options available in IConfig. Changes made to configuration options are effective for the current editing session, and can also be permanently applied to the editor.

A small set of the editor's configuration options are readily available from the pull-down Menu. These represent options that are likely to be changed frequently during an editing session. The majority of the printing options are included under the Print Menu, in the "Set Options" selection. A number of miscellaneous editing options (such as

[AutoIndent](#), [WordWrap](#), [RightMargin](#), LeftMargin, TabType, TabWidth, Backups, and SaveState) are included under the Options [Menu](#). To allow full access to the many editing options, the menu-driven Interactive Configuration facility (IConfig) is provided. To access IConfig, select "[Full Configuration](#)" under the Options Menu. The following IConfig menu pops up, identifying categories (described later in this chapter) of configuration options that can be selected:

```
+----- IConfig -----+ | Change Settings: | |***System/File*Options**>*****| | Keyboard/Mouse
Options > | | Command/Format Options > | | Text Entry/Block Options > | | Tab/Margin Options > | | Print Output
Options > | | Display/Color Options > | +-----+ | Write Settings to ASCII File... |
+-----+ Categories and options in IConfig are selected by placing the cursor bar over the
desired item and pressing <Enter>, or by pressing the highlighted quick-key. Pressing the <Escape> key allows you to
cancel a selection within a prompt, move back to the previous level within IConfig, or exit entirely from IConfig
(depending on your current location within IConfig). The option "Write Settings to ASCII File" allows you to create a
Settings File that contains your current configuration settings. See "Writing Configuration Settings to a Settings
File" (later in this chapter) for information about this.
```

Min

Returns the lesser of two specified numbers.

Syntax: [INTEGER](#) [Min](#)([INTEGER](#) a, [INTEGER](#) b) - a is any integer expression to compare with b. - b is any [integer](#) expression to compare with a. Returns: The smaller of the two numbers, a and b. **Examples:** [integer](#) smallest smallest = [Min](#)(100, [NumLines](#)()) See Also: [Max](#)()

Miscellaneous Questions

- Q: When should a macro be placed in the user-interface macro file versus an external macro file?
- A: There are not any specific rules, but the following guidelines should help you decide:
- If you use the macro frequently, place it in the user-interface macro file.
 - In most cases, it is easiest to develop a macro in an external file so that you can compile it without exiting the editor or installing a new configuration each time you change it. Once you have decided that the macro is working the way you want it, you can place it in the

user-interface macro file to be installed in the editor.

- If you have a large macro, or possibly a related series of macros, that you use for special purposes, put them together in an external macro file. You can think of that file as a "library" of macros.
- If you build a macro that uses a lot of low-level, simple macros already defined in your user-interface file, you have two choices:
 - You can put the macro in your user-interface file. Be aware that changes you might later make to your low-level

[macros](#) will impact the macro, for better or for worse. - Or you can put the macro in an external macro file, along with a copy of each of the low-level macros. Q: From a macro in one file, can you call a macro procedure in another file, or use a variable defined in another macro file? A: Yes and No. [Macros](#) may ONLY call: - native [editor commands](#), or - macros procedures defined in the same macro file, or - other macros (.MAC) files via the [ExecMacro\(\)](#) command. In this case, the executed macro is run per the [ExecMacro\(\)](#) specifications. Note that the [ExecMacro\(\)](#) command allows you to pass a command line using the MacroCmdLine variable. [Macros](#) may only access [variables](#): - that are local to the macro, or - that are global, but are in the same macro file. Q: What happens if two macros have the same name but are in different macro files? A: The editor handles them separately. Any [macros](#) in file A that call that macro name will use the copy of the macro from file A. Any macros in file B that call that macro name will use the copy from file B. If they are both Public macros, they are both listed in the Public macros PickList. If you use the [ExecMacro\(\)](#) command, the first one in the Public macros list is the one executed. Since all Public macros are placed at the beginning of the Public macros list when they are loaded, the last one loaded will be at the beginning of the Public macros list. Q: When should I use the TSELoad.dat feature versus [the TSEStart macro](#)? A: If you have external [macros](#) that you want loaded every editing session, use the TSELoad.dat feature. If, on the other hand, you want certain macros loaded depending on the directory you are in or other circumstances, use [the TSEStart macro](#).

MkDir

Create a directory.

Syntax: [INTEGER](#) [MkDir](#)([STRING](#) dir) - dir is the directory that will be created. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If dir does not include a path, the current directory is assumed. **Examples:** [MkDir](#)("some-dir") [MkDir](#)("c:\[windows](#)\some-dir") See Also: [RmDir\(\)](#), [ChDir\(\)](#), [CurrDir\(\)](#), [GetDir\(\)](#)

Mouse Equipment

To fully use the mouse capabilities, the editor must be configured to display a full border around each window on the screen. This is because the mouse uses the border area to scroll text and perform many window operations. In the standard configuration of the editor, full window borders are displayed whenever the mouse is enabled. (The variable DisplayBoxed determines whether borders are displayed around windows. For full use of the mouse, this variable must be set to "Mouse"

[or](#) "Always". The standard setting is "Mouse".) When the editor detects that a mouse is in use, the following special mouse-sensitive areas are included in the window borders. These areas (called mouse "hot spots") are used by the

mouse to perform particular operations, described later in this chapter. - In the current window, a horizontal scrollbar is placed in the bottom border, and a vertical scrollbar is placed in the right-hand border. - The < symbol is located at the left of the horizontal scrollbar (in the current window), and the > symbol is located at the right. - The ^ symbol is located at the top of the vertical scrollbar (in the current window), and the v symbol is located at the bottom. - Within the horizontal scrollbar (in the current window), an elevator shaft runs between the < and > symbols, and includes an * elevator symbol. This horizontal elevator moves within the elevator shaft according to your relative column position in the current file. - Within the vertical scrollbar (in the current window), another elevator shaft runs between the ^ and v symbols, and includes an * elevator symbol. This vertical elevator moves within the elevator shaft according to your relative line position in the current file. - The [^] or [v] symbol is located on the right-hand side of the title line of each window when multiple windows are open. When the windows are non-zoomed, the [^] symbol is displayed in each window. When a window is zoomed, the [v] symbol is displayed in the zoomed window. - The [*] symbol is located on the left-hand side of the title line of each window when multiple [windows](#) are open. (For more information on setting the variables that affect the use of the mouse, see "[Keyboard/Mouse Options](#)" and "[Display/Color Options](#)" in the chapter on "[Configuring The Editor Interactively](#)".)

Mouse Symbols and Scrollbars

If you are

[using a mouse](#) with the editor, special mouse-sensitive areas are included in the window borders of the screen. These areas include scrollbars and special symbols that allow you to use the mouse to scroll text and perform window operations. (For more information about using the mouse in the editor, see the chapter on "[Mousing Around the Editor](#)".)

MouseEnabled

Determines if the mouse is currently enabled.

[Values](#): BOOLEAN [Default](#): Undefined Limits: Can only be queried. Cannot be used within a config/endconfig construct. Notes: MouseEnabled is TRUE if the mouse is currently active; otherwise, [MouseEnabled](#) is [FALSE](#). See Also: [DisplayBoxed](#) Commands: [HideMouse\(\)](#), [ShowMouse\(\)](#)

MouseHoldTime

The time delay that is used to determine if a mouse key is simply clicked, or pressed and held.

[Values](#): INTEGER - [1..36] [Default](#): 9 (approximately 1/2 second) Notes: [MouseHoldTime](#) is set in 1/18-second increments. [MouseHoldTime](#) is used by [WaitForMouseEvent\(\)](#)'s _MOUSE_HOLD_TIME_ option, which is commonly used to determine if a mouse key is simply clicked, or pressed and held. Scrollbar events activate only once per click. But if the mouse key is pressed and held for a duration longer than MouseHoldTime, the scrollbar event automatically repeats. [Affects](#): [WaitForMouseEvent\(\)](#) See Also: [MouseRepeatDelay](#)

MouseHotSpot

The mouse hot spot at the location where the mouse key is clicked or pressed.

Syntax: [INTEGER MouseHotSpot\(\)](#) Returns: The current mouse hot spot (if any) at the location where a mouse key is clicked or pressed, based on the current values of [MouseX](#), [MouseY](#), and [MouseKey](#). The value returned is one of the following: Hot Spot [Return](#) Value on which Mouse [Key](#) Pressed _____

_____ _NONE_ None _MOUSE_MARKING_ Within Window (to change [windows](#), move [cursor](#), and mark text while dragging mouse) _MOUSE_CLOSE_ Close Window Symbol [*] _MOUSE_ZOOM_ Zoom Window Symbol [^] or [v] _MOUSE_UP_ Scroll Up Symbol (^) _MOUSE_DOWN_ Scroll Down Symbol (v) _MOUSE_PAGEUP_ Above Vertical Elevator (page up) _MOUSE_PAGEDOWN_ Below Vertical Elevator (page down) _MOUSE_VELEVATOR_ Vertical Elevator (*) (page up/down while mouse held) _MOUSE_LEFT_ Scroll Left (<) _MOUSE_RIGHT_ Scroll Right (>) _MOUSE_TABLEFT_ Left of Horizontal Elevator (scroll tab left) _MOUSE_TABRIGHT_ Right of Horizontal Elevator (scroll tab right) _MOUSE_HELEVATOR_ Horizontal Elevator (*) (scroll tableft/tabright while mouse held) _MOUSE_VRESIZE_ Left Window Border (resize vertical window while dragging mouse) _MOUSE_HRESIZE_ Top Window Border (resize horizontal window while dragging mouse) _MOUSE_VWINDOW_ Top Border (create vertical window at mouse [cursor](#) by clicking mouse twice) _MOUSE_HWINDOW_ Left Border (create horizontal window at mouse [cursor](#) by clicking mouse twice) If the mouse is clicked on the top or left border and not dragged, then _MOUSE_VRESIZE_ or _MOUSE_HRESIZE_ is returned, but [not](#) activated unless the mouse key is held down. Use [MouseWindowId\(\)](#) to determine in which window the mouse is clicked. To activate the mouse hot spot, call [ProcessHotSpot\(\)](#). See Also: [ProcessHotSpot\(\)](#), [MouseWindowId\(\)](#), [MouseMarking\(\)](#), [MouseStatus\(\)](#), [MouseKeyHeld\(\)](#) [Variables:](#) [MouseKey](#), [MouseX](#), [MouseY](#)

MouseKey

The mouse key pressed the last time

[GetKey\(\)](#) was called, or the state of the mouse keys the last time [MouseStatus\(\)](#) was called. [Values:](#) [INTEGER](#) [Default:](#) Undefined Limits: Cannot be used within a config/endconfig construct. See Also: [Key](#), [LastMouseKey](#) Commands: [MouseStatus\(\)](#), [GetKey\(\)](#)

MouseKeyHeld

Determines if a mouse key is pressed and held.

Syntax: [INTEGER MouseKeyHeld\(\)](#) Returns: The value of the [MouseKey](#) variable if the mouse key is pressed and held; [otherwise](#), zero ([FALSE](#)). Notes: If a mouse key is pressed and held, this command returns the mouse [key](#) being held. If a new key is pressed before the original key(s) are released, [MouseKeyHeld\(\)](#) returns zero. If any mouse events are pending, [MouseKeyHeld\(\)](#) also returns zero. Thus, this command will [return](#) zero if any mouse keys have been internally stacked. [MouseKeyHeld\(\)](#) internally calls [MouseStatus\(\)](#) which updates the [MouseX](#), [MouseY](#), and [MouseKey](#) variables. See Also: [MouseStatus\(\)](#), [GetKey\(\)](#), [MouseWindowId\(\)](#) [Variables:](#) [MouseKey](#), [MouseX](#), [MouseY](#)

MouseMarking

Moves the text cursor to the mouse cursor, changes windows, and marks blocks with the mouse movements.

Syntax: [MouseMarking](#)([INTEGER](#) block_type) - block_type is the type of block marking to use. Supported [values](#) for block_type are: - [_INCLUSIVE_](#) for an inclusive character block. - [_NONINCLUSIVE_](#) for a non-inclusive character block. - [_LINE_](#) for a line block. - [_COLUMN_](#) for a column block. Returns: Nothing. Notes: MouseMarking allows the user to move the text cursor, switch windows, and mark blocks with the mouse movements. [MouseMarking\(\)](#) works as follows: - Clicking within the current editing window moves the text cursor to the mouse cursor. - Clicking the mouse cursor twice on the same spot marks the word at that location. - Clicking the mouse cursor three times on the same spot marks the line. - Clicking within another window makes the new window the current window, without moving the text cursor. - Pressing and holding the mouse button in another window will first change windows, and if MouseHoldTime occurs, the text cursor will be moved to the mouse cursor. - If the mouse is clicked and dragged within a window, block [marking](#) will be initiated. The type of block [marking](#) used is based on the block_type passed to [MouseMarking\(\)](#). For a multiple-click hot spot, the speed at which the clicks occur is not a factor. See Also: [MouseHotSpot\(\)](#), [isBlockMarked\(\)](#), [isBlockInCurrFile\(\)](#), [isCursorInBlock\(\)](#) Variables: [MouseHoldTime](#), [Marking](#)

MouseRepeatDelay

The relative delay to use between the repetitions of a repeatable mouse event due to a mouse key being held.

Values: [INTEGER](#) **Default:** 0 (no delay) Notes: The amount of time represented by the value of MouseRepeatDelay varies depending on the hardware and operating system. This value is simply a relative number which should be increased to slow down repeating scrollbar events. Decrease this number to increase the [repeat](#) rate of scrollbar events. See Also: [MouseHoldTime](#)

MouseStatus

Updates the MouseKey, MouseX, and MouseY variables with the current state of the mouse.

Syntax: [INTEGER](#) [MouseStatus](#)() Returns: The value of [MouseKey](#). Notes: [MouseStatus\(\)](#) determines the CURRENT state of the mouse cursor location and keys. Unlike [GetKey\(\)](#), [MouseStatus\(\)](#) does [not](#) wait for a mouse key to be pressed. If a mouse key is not being pressed at the time [MouseStatus\(\)](#) is called, MouseKey is set to zero. [MouseStatus\(\)](#) is used internally during most mouse processing. When a key definition is executed, [MouseStatus\(\)](#) is called, internally, for all pending mouse events before passing control [to](#) the key definition. Generally, a macro that is assigned to a mouse [key](#) does [NOT](#) need to call [MouseStatus\(\)](#). **Examples:** The following procedure will display the Mouse coordinates on the StatusLine as the mouse is moved, and stop once a mouse key is pressed: [integer](#) prev_mouse_x = -1, prev_mouse_y = -1 [if](#) [Query](#)([MouseEnabled](#)) [repeat](#) [MouseStatus](#)() // get current state [if](#) ([Query](#)([MouseX](#)) <> prev_mouse_x or [Query](#)([MouseY](#)) <> prev_mouse_y) [Message](#)([Query](#)([MouseX](#)):5,[Query](#)([MouseY](#)):5) [endif](#) prev_mouse_x = [Query](#)([MouseX](#)) prev_mouse_y = [Query](#)([MouseY](#)) [until](#) [Query](#)([MouseKey](#)) <> 0 [GetKey](#)() // remove mouse [key](#) from keyboard buffer [endif](#) See Also: [GetKey\(\)](#) Variables: [MouseKey](#), [MouseX](#), [MouseY](#)

MouseWindowId

The window id where the mouse was last clicked.

Syntax: [INTEGER MouseWindowId\(\)](#) Returns: Returns the window id in which the mouse was clicked based on the current [values](#) of [MouseX](#), [MouseY](#), and [MouseKey](#); [otherwise](#), zero (FALSE) if the mouse was clicked outside a window. See Also: [MouseHotSpot\(\)](#)

MouseX

The column number (relative to the entire display) of the mouse cursor the last time

[GetKey\(\)](#) was called for a mouse key or [MouseStatus\(\)](#) was called to obtain the current mouse status. [Values:](#) [INTEGER Default:](#) Undefined Limits: Cannot be used within a config/endconfig construct. See Also: [MouseY](#), [MouseKey](#), [LastMouseX](#), [LastMouseY](#) Commands: [MouseStatus\(\)](#), [GetKey\(\)](#)

MouseY

The row number (relative to the entire display) of the mouse cursor the last time

[GetKey\(\)](#) was called for a mouse key or [MouseStatus\(\)](#) was called to obtain the current mouse status. [Values:](#) [INTEGER Default:](#) Undefined Limits: Cannot be used within a config/endconfig construct. See Also: [MouseX](#), [MouseKey](#), [LastMouseX](#), [LastMouseY](#) Commands: [MouseStatus\(\)](#), [GetKey\(\)](#)

Mousing around the Editor

The mouse can be used for editing tasks, as well as for a wide range of functions, including accessing

[menu](#) selections and PickLists, navigating through [the HelpSystem](#), positioning the cursor on the screen, scrolling text, [marking](#) and manipulating blocks, using [ClipBoards](#), and creating and moving between [windows](#). This chapter provides the following information on using the mouse: - [Mouse equipment](#) - [General use of the mouse](#) - [Using the pull-down Menu system](#) - [Positioning the cursor and scrolling text](#) - [Marking a block](#) - [Using the pop-up Mouse menu](#) - [Working with windows](#) - [Summary of mouse operations](#)

MoveBlock

Moves the marked block to the current cursor position.

Syntax: [INTEGER MoveBlock](#)([INTEGER](#) overwrite) - overwrite is an optional integer that specifies whether [MoveBlock\(\)](#) should overwrite or [insert](#) column blocks. If [not](#) specified, [_DEFAULT_](#) is used. overwrite has no effect if the block being moved is not a column block. Supported values for overwrite are: - [_DEFAULT_](#) causes column blocks to be inserted. - [_OVERWRITE_](#) causes column blocks to overwrite text at the current [cursor](#) position. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command fails if there is [not](#) enough memory to successfully complete the operation. If no block is marked, no action occurs and this command returns [FALSE](#). Character blocks are always inserted at the current cursor position. Column blocks are inserted or overlaid based on the value of the overwrite parameter. If the optional parameter is not specified, [_DEFAULT_](#) is used. Where line blocks are placed depends on the value of the [InsertLineBlocksAbove](#) variable. If set [ON](#), line blocks are inserted above the

current cursor line. If set OFF, line blocks are added below the current cursor line. This command will move blocks between files. See Also: [CopyBlock\(\)](#), [DelBlock\(\)](#), [isBlockMarked\(\)](#) Variables: [InsertLineBlocksAbove](#), [BlockId](#)

MoveFile

Moves a disk file from one location to another.

Syntax: [INTEGER](#) [MoveFile](#)([STRING](#) src, [STRING](#) dst [, [INTEGER](#) overwrite]) - src is the name of the file to move. - dst is the name of the moved file. - overwrite defaults to [FALSE](#). In this [case](#), [MoveFile\(\)](#) fails if dst exists. If overwrite is set to [TRUE](#), dst will be overwritten if it exists. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Wildcards cannot be specified with this command. Only single files may be moved. If path information is not specified, the current directory is assumed. The original file (src) is deleted at the completion of this command. **Examples:** [MoveFile](#)("foo", "bar") [MoveFile](#)("c:\dir\foo", "d:\path\bar") // assume the existence of two strings, s1 and s2 [MoveFile](#)(s1, s2) // fails if s2 exists [MoveFile](#)(s1, s2, [TRUE](#)) // s2 will be overwritten See Also: [CopyFile\(\)](#), [EraseDiskFile\(\)](#), [RenameDiskFile\(\)](#)

Moving Between Files in the Ring

Once multiple files are selected for editing and placed in the Ring, you can move to any one of those files, making it the current file. If the new current file was

[not](#) previously opened, it is now opened. You can move directly to a specific file by executing the [EditFile](#) command and entering the name of the file that you wish to edit. Or you can display a [PickList](#) of files in the Ring, and choose the desired file from that list, with the following commands. * [ListOpenFiles](#) (<Alt O>*, or "List Open" under File [Menu](#)) This command displays a [PickList](#) of all files that have been selected for editing (and have not been closed) during the current session. You can choose a file from this list and make it the current file. You can also save and/or close files in the list. To terminate the command without changing the current file position, press <Escape>. * <Alt O> indicates the number zero, not the alphabetic character "O". * [ListRecentFiles](#) (<Ctrl R>*, or "List Recent" under File [Menu](#)) This command displays a [PickList](#) of the files that were most recently accessed, and allows you to select a file from the [PickList](#) to be edited. The filenames are listed from top to bottom in order of most recent access, with the current file listed first. Closing a file does not remove it from the Recent Files list. The maximum number of files that are maintained in the Recent Files list is controlled by the [MaxRecentFiles](#) variable. (For more information on setting the [MaxRecentFiles](#) variable, see "Max Recent Files" under "[Command/Format Options](#)" in the chapter on "[Configuring the Editor Interactively](#)".) When the [PickList](#) is displayed, the cursor bar is positioned on the second file in the list (or on the first file if there is only one file in the Recent Files list). You can choose a file from this list and make it the current file. (If the selected file is not currently loaded, it will be loaded by this command.) You can also delete an entry from the list itself. To terminate the command without changing the current file position, press <Escape>. You can optionally configure the editor to save the entries in the Recent Files list from one editing session to the next. For more information, see "[Saving/Restoring Entries in the Recent Files List](#)" later in this chapter. * <Ctrl R> indicates the number zero, not the alphabetic character "O". As an alternative method for moving to a new file, you can move forward or backward through the Ring of files. The following commands allow you to move to the next or previous file in the Ring, thus making that file the current file. * [NextFile](#) (<Alt N>, or "Next" under File [Menu](#)) This command moves forward in the Ring to select the next file, and makes that file the current file. * [PrevFile](#) (<Alt P>, or "Previous" under File [Menu](#)) This command moves backward in the Ring to select the previous file, and makes that file the current file. +----- Tip -----+ | To cause the editor to execute specific commands upon loading a || file or moving (switching) to a new file, you can create special || "event" macros. (For more information, see "[Event Macros](#)" in the || chapter on "[Using Macro Language Features](#)" in the || Macro

Moving Through Text in a File

This chapter describes the following commands that allow you to move through text in a file or scroll text in the current window:

-

[Changing position by one character or line](#) (CReturn, Down, Up, Left, Right, NextChar, PrevChar) - [Changing position by multiple characters or lines](#) (BegLine, EndLine, BegFile, EndFile, BegWindow, EndWindow, GotoColumn, GotoLine, PrevPosition, WordLeft, WordRight, BegPara, EndPara, NextPara, PrevPara) - [Scrolling forward or backward in a file](#) (PageDown, PageUp, RollDown, RollUp, RollLeft, RollRight, ScrollDown, ScrollUp, ScrollLeft, ScrollRight) - [Scrolling the current line](#) (ScrollToCenter, ScrollToTop) Other chapters describe additional commands and features available to move to the desired position in a file. The chapter on "[Mousing Around the Editor](#)" explains how to use the mouse to move through a file or scroll text. The chapters on "[Loading, Processing, and Saving Files](#)" and "[Splitting the Screen into Windows](#)" describe how to move to another file or window. The chapter on "[Using Tabs](#)" discusses the behavior of the TabRight and TabLeft commands, allowing you to move to the next or previous tab stop on a line. The chapter on "[Block Operations](#)" describes the commands GotoBlockBegin and GotoBlockEnd, which allow you to move to the beginning or the end of a block. The chapter on "[Search Features: Finding and Replacing Text](#)" explains how to locate and position the cursor on a specific or non-specific string of characters, using the Find command. It also describes the FindAndDo, FindWordAtCursor, and Match commands. The chapter on "[Bookmarks](#)" presents the editor's handy facility for marking bookmark positions in a file to which you can return later. NOTE: When editing a file containing Hard ("real") Tabs, with ExpandTabs ON, the position of the cursor may be affected when the cursor moves to a new line. (For more information, see the chapter on "[Using Tabs](#).")

MsgBox

Displays a message box on the screen, with a specified message and selected user-response options.

Syntax: [INTEGER MsgBox](#)([STRING](#) msg) - msg is the message that appears as the main text of the message box. or: [INTEGER MsgBox](#)([STRING](#) title, [STRING](#) msg [, [INTEGER](#) flag]) - title is the title to be displayed along the top border of the message box. - msg is the message that appears as the main text of the message box. - flag is an optional integer that determines the user-response selections. If [not](#) specified, [_OK_](#) is used. Supported [values for](#) flag are: - [_YES_NO_](#) causes "Yes" [and](#) "No" selections to be included in the message box. - [_YES_NO_CANCEL_](#) causes "Yes", "No", and "Cancel" selections to be included in the message box. - [_OK_](#) causes just an "OK" selection to be included in the message box. Returns: The number of the selected item, or zero (FALSE) if <Escape> is pressed. Notes: If the first format is used, no title is displayed, and an "OK" selection is used in the message box. [Otherwise:](#) This command allows you to select one of three formats for the user-response options of the message box: Yes/No, Yes/No/Cancel, or OK. The following values are returned by [MsgBox\(\)](#) (for the relevant choices and keys, based on flag): - 0 - <Escape> is pressed; or <c> is pressed or "Cancel" is selected. - 1 - <y> or <o> is pressed, or "Yes" [or](#) "OK" is selected. - 2 - <n> is pressed or "No" is selected. If title is longer than what can be displayed on the screen, title is truncated accordingly. The text of msg will wrap to a new line, as necessary, if the length is longer than the width of the message box. To force the text to wrap after a particular character, include a carriage-return character at that position in the text. With this command, the length of the displayed message, as contained in msg, is limited to the maximum string length. For a similar command that allows you to display a message longer than the maximum string

length, see [MsgBoxBuff\(\)](#). The pop-up menu generated by [MsgBox\(\)](#) is centered on the screen. To specify a different location, use the [Set\(\)](#) command to set the X1 and/or Y1 variables, as desired. Another similar command, [MsgBoxEx\(\)](#), allows you to customize the user-response options. **Examples:** [MsgBox](#)("This is a message") // No title, ok used
[MsgBox](#)("", "Hello, World!") // One line for msg [MsgBox](#)("", "Hello, " + [chr](#)(13) + "World!") // Two lines for msg
[MsgBox](#)("Error!", "Quit now?", _YES_NO_CANCEL_) // Three choices The following macro displays a message box (whose title is the current Macro name, and whose message is "Finished!") in the upper left corner of the current editing window. The [OK] choice is displayed. [proc](#) DisplayFinish() [integer](#) x = [Set](#)(X1,1) // save current X1 value, set new one [integer](#) y = [Set](#)(Y1,1) // save current Y1 value, set new one [MsgBox](#)([SplitPath](#)([CurrMacroFilename](#)()), _NAME_), "Finished!") [Set](#)(X1,x) // restore saved X1 value [Set](#)(Y1,y) // restore saved Y1 value [end](#) See Also: [MsgBoxBuff\(\)](#), [MsgBoxEx\(\)](#), [YesNo\(\)](#), [Warn\(\)](#) **Variables:** X1, Y1

MsgBoxBuff

Displays a message box on the screen, with a message (from a specified buffer)

[and](#) selected user-response options. **Syntax:** [INTEGER](#) [MsgBoxBuff](#)([STRING](#) title [, [INTEGER](#) flag [, [INTEGER](#) buffer_id]]) - title is the title to be displayed along the top border of the message box. - flag is an optional integer that determines the user-response selections. If [not](#) specified, _OK_ is used. Supported [values for](#) flag are: - _YES_NO_ causes "Yes" [and](#) "No" selections to be included in the message box. - _YES_NO_CANCEL_ causes "Yes", "No", and "Cancel" selections to be included in the message box. - _OK_ causes just an "OK" selection to be included in the message box. - buffer_id is the optional buffer from which the main text of the message box is taken. If not passed, the current buffer is assumed. Returns: The number of the selected item, or zero (FALSE) if <Escape> is pressed. Notes: This command allows you to select one of three formats for the user-response options of the message box: Yes/No, Yes/No/Cancel, or OK. The following values are returned by [MsgBox\(\)](#) (for the relevant choices and keys, based on flag): - 0 - <Escape> is pressed; or <c> is pressed or "Cancel" is selected. - 1 - <y> or <o> is pressed, or "Yes" [or](#) "OK" is selected. - 2 - <n> is pressed or "No" is selected. If title is longer than what can be displayed on the screen, title is truncated accordingly. The text in buffer will wrap to a new line, as necessary, if the length is longer than the width of the message box. To force the text to wrap after a particular character, include a carriage-return character at that position in the text. In contrast to the [MsgBox\(\)](#) command, the length of the message is not limited to the maximum string length. The entire contents of the specified buffer will be displayed, provided the editing window is large enough to accommodate it. The pop-up menu generated by [MsgBoxBuff\(\)](#) is centered on the screen. To specify a different location, use the [Set\(\)](#) command to set the X1 and/or Y1 variables, as desired. **Examples:** [proc](#) main() [CreateTempBuffer](#)() [AddLine](#)("This is a test to see if this thing will not") [AddLine](#)("rewrap the way I hope it won't.") [AddLine](#)("Well, here goes nothing (or, famous last words!") [MsgBoxBuff](#)("Testing", _YES_NO_CANCEL_, [GetBufferId](#)()) [end](#) See Also: [MsgBox\(\)](#), [MsgBoxEx\(\)](#), [YesNo\(\)](#), [Warn\(\)](#) **Variables:** X1, Y1

MsgBoxEx

Displays a message box on the screen, with a specified message and custom response buttons.

Syntax: [INTEGER](#) [MsgBoxEx](#)([STRING](#) title, [STRING](#) msg, [STRING](#) buttons) - title is the title to be displayed along the top border of the message box. - msg is the message that appears as the main text of the message box. - buttons is a string containing the different response choices. Returns: The number of the selected item, or zero (FALSE) if <Escape> is pressed or "Cancel" is selected (under the condition described below). Notes: buttons is a list of user-response choices, each separated by a semi-colon (";"). For each choice, the ampersand ("&") can be used to designate

a shortcut key. For example, "[&Append];[&Overwrite];[&Cancel]" would display 3 buttons, with "A", "O", and "C" being the shortcut keys. Each choice included in buttons is assigned a return number, beginning with 1. If the last button designation is "Cancel", instead of the button number being returned, 0 is returned. If title is longer than what can be displayed on the screen, title is truncated accordingly. The text of msg will wrap to a new line, as necessary, if the length is longer than the width of the message box. To force the text to wrap after a particular character, include a carriage-return character at that position in the text. With this command, the length of the displayed message, as contained in msg, is limited to the maximum string length. For a similar command that allows you to display a message longer than the maximum string length, see [MsgBoxBuff\(\)](#). The pop-up menu generated by [MsgBoxEx\(\)](#) is centered on the screen. To specify a different location, use the [Set\(\)](#) command to set the X1 and/or Y1 variables, as desired. Similar commands, [MsgBox\(\)](#) and [MsgBoxBuff\(\)](#), display message boxes with predefined user-response options. [Examples](#): [MsgBoxEx\("File changed", "Save changes?", "\[&Yes\];\[&No\];\[&Cancel\]"\)](#) See Also: [MsgBox\(\)](#), [MsgBoxBuff\(\)](#), [YesNo\(\)](#), [Warn\(\)](#) [Variables](#): [X1](#), [Y1](#)

MsgLevel

Controls which internal messages the editor will display.

[Values](#): [INTEGER](#) - [_ALL_MESSAGES_](#) - [_WARNINGS_ONLY_](#) - [_NONE_](#) [Default](#): [_ALL_MESSAGES_](#)
Limits: Cannot be used within a config/endconfig construct. Notes: [MsgLevel](#) is intended for macro use only. - [_ALL_MESSAGES_](#) causes the editor to issue all messages and warnings. - [_WARNINGS_ONLY_](#) causes the editor to issue warnings only. A warning is defined as any editor message that requires the user to press <Escape>. If you have a macro that loads or inserts a large file, you can use [_WARNINGS_ONLY_](#) to keep the screen from being updated. - [_NONE_](#) causes the editor to omit all warnings and messages. This setting should only be used for macro operations. This command does not affect warnings/messages generated by the [Warn\(\)](#) and [Message\(\)](#) commands. Once a macro "runs to completion," MsgLevel is automatically set to [_ALL_MESSAGES_](#). Affects: All commands that send a message to the user

NewFile

Creates a new editing buffer, with a name in the format <unnamed-#> (where # represents an actual number).

[Syntax](#): [INTEGER NewFile\(\)](#) Returns: The non-zero id number of the buffer just created, if successful; [otherwise](#), zero ([FALSE](#)). Notes: The newly-created buffer becomes the current file. This command fails if there is [not](#) enough memory available to create the new buffer. See Also: [CreateBuffer\(\)](#), [CreateTempBuffer\(\)](#)

NextChar

Moves the cursor to the next character in the file, traversing lines if necessary.

[Syntax](#): [INTEGER NextChar\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command does [not](#) stop on end-of-line boundaries. It wraps to the first character of the next line when issued at the [end](#)-of-line position or beyond. Zero is returned if [NextChar\(\)](#) is executed on or past the end-of-line position of the last line of the file. Idioms: // To repeat an operation on each character in the file // (assuming [BegFile\(\)](#) has been issued): [repeat if](#)

[CurrChar\(\)](#) >= 0 //only process if [not](#) at end-of-line //do something [endif until not NextChar\(\)](#) See Also: [PrevChar\(\)](#), [Right\(\)](#), [Left\(\)](#), [CurrChar\(\)](#)

NextFile

Switches to the next file in the editor's Ring of files.

Syntax: [INTEGER NextFile](#)([[INTEGER](#) flag]) - flag is an optional integer which specifies whether the file being switched to is actually loaded into the editor, if it has [not](#) already been loaded. If [not](#) specified, [_DEFAULT_](#) is used. Supported [values](#) for flag are: - [_DEFAULT_](#) causes the next file to be loaded from disk (if it has not already been loaded), the first time it is switched to. - [_DONT_LOAD_](#) causes the next file not to be loaded from disk (if it has [not](#) already been loaded). Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command returns a non-zero value if it changes to another file. If there is [not](#) another file loaded, it will [return](#) zero. If an [_ON_FIRST_EDIT_](#) event exists, it will be called the first time a file is switched to via this command. If an [_ON_CHANGING_FILES_](#) event exists, it is also called. If the [_DONT_LOAD_](#) flag is passed, these events are [NOT](#) called. The flag setting of [_DONT_LOAD_](#) is intended for macro use, for the cases when a macro needs to traverse the files in the editor's internal Ring of files, without actually editing the file. It is not intended for switching to a file that the user can edit, without first issuing an [EditFile](#)([CurrFilename\(\)](#)) command from the same macro. If there is only one file, [NextFile\(\)](#) has no effect. See Also: [PrevFile\(\)](#), [Hook\(\)](#), [UnHook\(\)](#), [EditFile\(\)](#), [EditThisFile\(\)](#), [BufferType\(\)](#)

NextWindow

Makes the next editing window the current window.

Syntax: [INTEGER NextWindow](#)() Returns: Zero ([FALSE](#)) if there was only one window or the current window is zoomed; [otherwise](#), non-zero. Notes: If the current window is the last one in the sequence, [NextWindow\(\)](#) switches to the first window and makes it current. If an [_ON_CHANGING_FILES_](#) event exists, it is called after this command executes, unless next window contains the same file as the previous window. Each window's sequence number is shown in the window's border. These numbers are assigned in the order windows are opened. If a window is closed, its number is reused. Hence the sequence of numbers does not always reflect the order in which windows are created. If there is only one window, or if [windows](#) are zoomed, no action occurs and zero is returned. See Also: [PrevWindow\(\)](#), [GotoWindow\(\)](#), [Hook\(\)](#), [UnHook\(\)](#)

NoOp

Does nothing.

Syntax: [NoOp](#)() Returns: Nothing. Notes: This provides a no-operation command. It can be handy as a place holder when you need to do something that causes the editor to run a command, but you do not want the command to do anything. This command can also be useful if you need to override a key assignment.

Numbers

[Constant numbers](#) consist of one or more digits, and possibly letters, concatenated together. All numeric representations must begin with a digit 0 (zero) through 9. By [default](#), [numbers](#) are specified as decimal (base 10)

representations. Binary and hexadecimal numbers may also be specified using a special notation. Decimal numbers consist of the digits 0 (zero) through 9. No letters are allowed. Binary numbers consist of a series of the digits 0 (zero) and 1, followed by the letter "b". Hexadecimal (hex) numbers consist of a series of digits 0 (zero) through 9, and letters A through F, followed by the letter "h". Since all numbers must begin with a digit 0 (zero) through 9, the digit 0 (zero) must precede a hex number that begins with a letter. Alternatively, hex [numbers](#) can be represented as 0X (this is zero followed by X), followed by a series of digits 0 (zero) through 9, and letters A through F. [Examples](#): 32 9 100 255 // decimal 20h 9h 64h 0ffh // hex 0x20 0x9 0x64 0xff // alternate hex 100000b 1001b 1100100b 11111111b // binary The following are [not](#) valid: ffh // hex [numbers](#) must begin with number (example: 0ffh) 12200003b // binary [numbers](#) may only use digits 0 [and](#) 1 Numbers must be within the range of -2,147,483,648 to +2,147,483,647. MAXINT is a reserved constant which is equal to the maximum allowable integer value of +2,147,483,647. MININT is a reserved constant which is equal to the minimum allowable integer value of -2,147,483,648.

NumFiles

Returns the number of files in the editor's Ring of files.

[Syntax](#): [INTEGER NumFiles\(\)](#) Returns: The number of files in the editor's Ring of files excluding `_SYSTEM_` and `_HIDDEN_` [buffers](#). Notes: If an operation that begins at the current editing buffer requires traversing all the files that are accessible with [NextFile\(\)](#) and [PrevFile\(\)](#), the buffer type of the current editing buffer has to be taken into account. The reason for this is that System and hidden [buffers](#) are [not](#) accessible with [NextFile\(\)](#) [and](#) [PrevFile\(\)](#), and are not included in the value returned by [NumFiles\(\)](#). Idioms: In this example, if the current buffer is a `_SYSTEM_` or `_HIDDEN_` buffer, the number of necessary [NextFile\(\)](#) repetitions is increased by 1 over the value returned by [NumFiles\(\)](#). //Perform an operation on every file loaded into the editor. [integer](#) n = [NumFiles\(\)](#) + ([BufferType\(\)](#) <> `_NORMAL_`), id = [GetBufferId\(\)](#) [while](#) n //do some operation on the file [NextFile\(\)](#) n = n - 1 [endwhile](#) [GotoBufferId](#) (id) // Return to the starting file/buffer See Also: [NextFile\(\)](#), [PrevFile\(\)](#), [BufferType\(\)](#)

NumFileTokens

Determines the number of filename tokens (elements) in a specified string.

[Syntax](#): [INTEGER NumFileTokens\(STRING s\)](#) - s is the string in which to count the number of filename tokens. Returns: The number of filename tokens in the string. [Examples](#): See the [GetFileToken\(\)](#) command. See Also: [GetFileToken\(\)](#), [GetToken\(\)](#), [NumTokens\(\)](#), [SubStr\(\)](#)

NumHistoryItems

Returns the number of items in a specified history list.

[Syntax](#): [INTEGER NumHistoryItems\(INTEGER history\)](#) - history is an integer specifying the history list to use. history can be: - A pre-defined constant value representing one of the built-in histories. See the [AddHistoryStr\(\)](#) command for a listing of these [constants](#). - A number in the range 1..127. - A number returned by [GetFreeHistory\(\)](#), which will be in the range 1..127. Returns: The number of items in the specified history list. [Examples](#): See the [GetHistoryStr\(\)](#) command. See Also: [DelHistoryStr\(\)](#), [GetHistoryStr\(\)](#), [GetFreeHistory\(\)](#), [AddHistoryStr\(\)](#) [Variables](#): [MaxHistoryPerList](#), [PersistentHistory](#)

NumLines

Returns the number of lines in the current file.

Syntax: [INTEGER NumLines\(\)](#) Returns: The number of lines in the current file. Notes: In a new file, an empty file, or a file in which all lines have been deleted, [NumLines\(\)](#) can equal 0. See Also: [CurrLine\(\)](#)

NumTokens

Determines the number of tokens (elements) in a specified string.

Syntax: [INTEGER NumTokens\(STRING s, STRING delimiter\)](#) - s is the string in which to count the number of tokens. - delimiter is the delimiting character (or set of characters) that separate each token. Returns: The number of tokens in the string. **Examples:** See the [GetToken\(\)](#) command. See Also: [GetToken\(\)](#)

NumWindows

Returns the number of editing windows currently open.

Syntax: [INTEGER NumWindows\(\)](#) Returns: The number of editing [windows](#) (horizontal and vertical) that are open. Notes: The number returned does not include pop-up [windows](#) ([windows](#) opened with [PopWindow\(\)](#)). **Examples:** [integer](#) n [OneWindow\(\)](#) n = [NumWindows\(\)](#) // n is 1 [HWindow\(\)](#) n = [NumWindows\(\)](#) // n is 2 [VWindow\(\)](#) n = [NumWindows\(\)](#) // n is 3 See Also: [OneWindow\(\)](#), [HWindow\(\)](#), [VWindow\(\)](#), [GotoWindow\(\)](#)

Ofs

Returns the offset address of a specified variable.

This command is intended for ADVANCED USAGE.

Syntax: [INTEGER Ofs\(VAR INTEGER | VAR STRING variable_name\)](#) - variable_name is the string or integer variable for which to return the offset address. Returns: The offset address of the specified variable. Notes: 32-bit pointers consist of a segment and an offset. The segment is the upper 16 bits, while the offset is the lower 16 bits. **Examples:** See the [Intr\(\)](#) command. See Also: [Seg\(\)](#), [Addr\(\)](#), [AdjPtr\(\)](#)

OneWindow

Expands the current editing window to full-screen size and closes all other editing

[windows](#). **Syntax:** [OneWindow\(\)](#) Returns: Nothing. See Also: [HWindow\(\)](#), [VWindow\(\)](#), [ZoomWindow\(\)](#), [CloseWindow\(\)](#), [DelWindow\(\)](#)

Opening Windows

Additional windows are opened by splitting the current window horizontally or vertically.

The following two commands open a new window, and display in it a view of a file that is already open.

* HWindow (<Ctrl O H>, or "Split Horizontally" under Window

[Menu](#)) This command splits the current window horizontally. * VWindow (<Ctrl O V>, or "Split Vertically" under Window [Menu](#)) This command splits the current window vertically. The above commands split the current window into two equal windows, with no effect on any other [window](#)(s) already on the screen. Each newly-created window has its own title line containing the window number and the name of the file that is displayed in the window. The cursor moves either down or to the right, into the new window, making it the current window. [The StatusLine](#) is updated to reflect information on the file in the current window. If you are editing only one file, the newly-created window only provides an additional view of the file. The editor will not load more than one copy of the same file. If you are editing multiple files, the newly-created window provides a view of the next file in the Ring. (For more information on how multiple files are handled, see the chapter on "[Loading, Processing, and Saving Files](#).") The following command can be used to open a new window, and load a new file into it. * OpenFileAndWindow ("Open File & Window" under Window [Menu](#)) This command prompts for the name of the file to open by issuing the prompt, "File(s) to edit:". You can enter the complete or abbreviated filename(s) to open, or you can obtain a PickList of files from which to choose. (For more information on selecting files from the "File(s) to edit:" prompt, see the EditFile command in the chapter on "[Loading, Processing, and Saving Files](#).") The editor opens a new window by splitting the current window horizontally, and then loads the first (or only) file that you specified at the prompt. This new window and file become the current window and file.

Other Special-Purpose Search Features

* FindAndDo ("Find And Do" under Search

[Menu](#)) This command is a variation of the Find command that allows you to search for a specified string, then perform one of several pre-defined operations. When you execute FindAndDo, the editor invokes the Find command and displays the Find prompt, "Search for:". Once you enter the desired search string, the editor displays the Find "Options" prompt. All of the Find command search options are available except the [A] and [V] options. (For more information on the Find command, see "[Basic Find and Replace Options](#)" earlier in this chapter.) After you enter the appropriate Find options, the editor displays the following [selection menu](#): + After Find, Do + |*Delete*Line****| | Cut Append | | Copy Append | | Count | +-----+ From this menu, select the operation that you want the editor to perform, based on each occurrence of the search string and Find options you entered. The functions performed by the available selections are as follows: - "Delete Line" instructs the editor to delete all lines on which the specified search [string](#) is located. - "Cut Append" instructs the editor to append each line on which the specified search string is located to the System Clipboard. It copies the located lines to the Clipboard, then deletes ("cuts") those lines from the file. (For more information on the CutAppend command and the System Clipboard, see "[Manipulating Blocks via ClipBoards](#)" in the chapter on "[Block Operations](#).") - "Copy Append" instructs the editor to append each line on which the specified search string is located to the System Clipboard, by making a copy of the located lines in the Clipboard. (For more information on the CopyAppend command and the System Clipboard, see "[Manipulating Blocks via ClipBoards](#)" in the chapter on "[Block Operations](#).") - "Count" instructs the editor to count the number of times the specified search [string](#) is located. Once you select the desired operation, the editor immediately performs that operation within the indicated portion of the file, for each occurrence of the search string (or for each line on which the search

string is located, according to the operation selected). When complete, the editor displays the following message on [the StatusLine](#): # occurrence(s) found where "#" represents the number of times (or lines on which) the search [string](#) was located. * FindWordAtCursor (<Alt =>, or "Find Word at [Cursor](#)" under Search [Menu](#)) This command searches for the next occurrence of the word on which the cursor is currently positioned. When you execute FindWordAtCursor, the editor invokes the Find command (described earlier in this chapter) using that word. The Find options in effect are the Initial Find Options. When located, the cursor is placed in the text on the first character of the word and the word is highlighted. If the cursor is not located on a word when you execute FindWordAtCursor, the editor executes the Find command itself, and issues the standard Find prompt. The editor also offers the following variations of FindWordAtCursor: - Press <Ctrl => to execute FindWordAtCursor with the "Backward" Find option included. - Press <Ctrl \> to execute FindWordAtCursor with the "Global" Find option included. (For more information on how the editor defines a word and on configuring the Initial Find Options, see "[WordSet Characters](#)" and "Initial Find Options" under "[Command/Format Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") * Match (<Alt F3>, or "Match" under Search [Menu](#)) This command searches for the matching character in the following sets of bracketing pairs: (), { }, < >, and []. When you position the cursor on one of these characters and execute the Match command, the editor attempts to locate the matching character. The editor searches forward in the current file to find a closing match for an opening character, and backward to find an opening match for a closing character. If the character pairs are nested, the editor attempts to locate the logically-matching character. If no match for the character is found, the message, "Match not found Press <Escape>", is displayed on [the StatusLine](#). If the cursor is not positioned on one of the indicated bracketing characters when you execute Match, the editor searches forward in the file for the first occurrence of one of these characters. If one is found, the [cursor](#) is positioned on that character. If none of these characters are located, the message, "Match not found Press <Escape>", is displayed on the StatusLine.

OtherWinBorderType

The type of line that outlines the non-current windows.

[Values](#): [INTEGER](#) - 1 - single-line characters - 2 - single-line side, double-line top characters - 3 - double-line side, single-line top characters - 4 - double-line characters - 5 - small block characters - 6 - large block characters - 7 - ASCII - 8 - 3D [Default](#): 2 Notes: OtherWinBorderType is used to specify the type of border characters to use for all editing windows that are not the current window. See Also: [OtherWinBorderAttr](#), [CurrWinBorderType](#), [CurrWinBorderAttr](#)

Overview of the Editor

To help you become familiar with the editor, this chapter provides a basic overview of the following topics:

-

[Starting an editing session](#) - [Working in the editor](#) - [Saving files and exiting from the editor](#) - [Customizing the editor](#)

This chapter does not attempt to cover all the commands and features available within the editor. Detailed information about the editor's capabilities and options can be found in the remaining chapters of this manual.

Overview of the Macro Programming Language

The editor includes its own macro programming language, called the SemWare Applications Language (SAL), which is used to create Compiled

[Macros](#). This macro programming language is fully integrated into the editor, and is also fully available for your own use in [customizing the editor](#). Much of the editor itself has been implemented internally using the macro language. Many of the built-in native commands in the editor are actually [macros](#) written in SAL. And you can use SAL to write and compile your own [macros](#). Using Compiled [Macros](#), you can create or customize your own commands, routines, menus, or help information, which are processed by the editor just like its own built-in commands. In its simplest form, a macro is a series of editor commands grouped together to perform a specific task. Compiled Macros can be used to: - Add customized commands to the editor to fit your particular needs. - Enhance or modify the operation of native editor commands. - Create your own editing environment. - Manipulate text files in just [about](#) any way imaginable. This chapter provides an overview of how to use SAL to write your own editor macros, and covers the following topics: - [Creating macros \(overview\)](#) - [Compiling macros \(overview\)](#) - [Compiling macros from inside the editor](#) - [Compiling macros from outside the editor](#) - [Executing macros \(overview\)](#) - [Macro source files vs. macro object files](#) - [Macro syntax](#) - [Some simple macro examples](#) - [Obtaining useful information within a macro](#) - [Adding a variable](#) - [Prompting the user](#) - [Flow of control](#) - [Assigning a macro to a key](#) - [Creating a menu](#) You may also find it helpful to review the following in [the User's Guide](#): "[Introduction to External Compiled Macros](#)" in the chapter on "[Creating and Using Macros](#)," and the chapter on "[Fully Customizing the Editor](#)."

OverwriteCursorSize

Sets the size (height) of the text cursor for overwrite mode. Size is measured in units equal to 1/8th of the maximum cursor height.

Values: [INTEGER](#) - 0 - no visible [cursor](#) - 1..8 - size of [cursor](#) - 9 - A vertical 'bar' (line) [cursor](#). - 255 - [cursor](#) size is [not](#) changed **Default:** 2 Notes: 8 gives the maximum cursor size, 1 gives the minimum cursor size. 2..7 give increasing [cursor](#) sizes. Because of differences in video card resolution, the cursor size on different machines may differ slightly. **Affects:** [ToggleInsert\(\)](#) See Also: [Cursor](#), [Insert](#), [InsertCursorSize](#)

PageDown

Scrolls text toward the end of the file in increments of one line less than the number of lines in the current window. The cursor remains stationary, except when end of file is reached.

Syntax: [INTEGER PageDown\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. See Also: [PageUp\(\)](#)

PageUp

Scrolls text toward the beginning of the file in increments of one line less than the number of lines in the current window. The cursor remains stationary, except when beginning of file is reached.

Syntax: [INTEGER PageUp\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. See Also: [PageDown\(\)](#)

ParaEndStyle

Determines where the

[WrapPara\(\)](#) command stops executing. [Values:](#) [INTEGER](#) - 0 - wrap until a blank line or EOF - 1 - same as 0, or until encountering a line indented the same as the first line of the paragraph. [Default:](#) 0 [Notes:](#) Using a value of 1 allows for indented paragraphs that are not separated by blank lines. [Affects:](#) [WrapPara\(\)](#)

Paste

Inserts the contents of the Clipboard at the cursor position.

[Syntax:](#) [INTEGER Paste](#)([[INTEGER](#) overwrite]) - overwrite is an optional integer that specifies whether [Paste\(\)](#) should overwrite or [insert](#) column blocks. If [not](#) specified, [_DEFAULT_](#) is used. Supported [values](#) for overwrite are: - [_DEFAULT_](#) causes column blocks to be inserted. - [_OVERWRITE_](#) causes column blocks to overwrite text at the current [cursor](#) position. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command fails if there is [not](#) enough memory to successfully complete the operation. If the Clipboard is empty, no action occurs and zero is returned. Text is placed into the Clipboard using the [Cut\(\)](#) and [Copy\(\)](#) commands. The type of block last cut or copied to the Clipboard determines the type of block in the Clipboard. - Character blocks are always inserted at the current cursor position. - Column blocks are inserted or overlaid based on the value of the overwrite parameter. - Where line blocks are placed depends on the value of the [InsertLineBlocksAbove](#) variable. If set [ON](#), line blocks are inserted above the current cursor line. If set OFF, line blocks are added below the current cursor line. The contents of the Clipboard remain unchanged and can be used with subsequent [Paste\(\)](#) commands. Macros that depend on the placement of line blocks, should first save the [InsertLineBlocksAbove](#) setting, set it to the desired value, perform the macro, and restore the setting at the end of the macro. The [cursor](#) position does [not](#) change. Normally, a pasted block remains as a marked block, unless the [UnMarkAfterPaste](#) variable is set [ON](#). See Also: [PasteReplace\(\)](#), [Copy\(\)](#), [CopyAppend\(\)](#), [Cut\(\)](#), [CutAppend\(\)](#) [PasteFromWinClip\(\)](#), [PasteReplaceFromWinClip\(\)](#), [CopyToWinClip\(\)](#), [CopyAppendToWinClip\(\)](#), [CutToWinClip\(\)](#), [CutAppendToWinClip\(\)](#) [CopyBlock\(\)](#), [DelBlock\(\)](#), [MoveBlock\(\)](#)
[Variables:](#) [InsertLineBlocksAbove](#), [UnMarkAfterPaste](#)

PasteFromWinClip

Inserts the contents of the Windows Clipboard at the cursor position.

[Syntax:](#) [INTEGER PasteFromWinClip](#)([[INTEGER](#) overwrite]) - overwrite is an optional integer that specifies whether [PasteFromWinClip\(\)](#) should overwrite or [insert](#) column blocks. If [not](#) specified, [_DEFAULT_](#) is used. Supported [values](#) for overwrite are: - [_DEFAULT_](#) causes column blocks to be inserted. - [_OVERWRITE_](#) causes column blocks to overwrite text at the current [cursor](#) position. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: The [cursor](#) position does [not](#) change. See Also: [PasteReplaceFromWinClip\(\)](#), [CopyToWinClip\(\)](#), [CopyAppendToWinClip\(\)](#), [CutToWinClip\(\)](#), [CutAppendToWinClip\(\)](#) [Paste\(\)](#), [PasteReplace\(\)](#), [Copy\(\)](#), [CopyAppend\(\)](#), [Cut\(\)](#), [CutAppend\(\)](#) [CopyBlock\(\)](#), [DelBlock\(\)](#), [MoveBlock\(\)](#) [isWinClipAvailable\(\)](#), [EmptyWinClip\(\)](#) [GetStrFromWinClip\(\)](#)

PasteReplace

Inserts the contents of the editor's Clipboard at the cursor position, replacing a block, if it exists.

[Syntax:](#) [INTEGER PasteReplace](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Before text is

pasted into the editor, if a block exists, and the cursor is inside the block, the block is deleted. If a block exists, but the cursor is not inside the block, the block is not deleted. The cursor is placed at the end of the inserted text. See the [Paste\(\)](#) command for additional usage notes. See Also: [Paste\(\)](#), [Copy\(\)](#), [CopyAppend\(\)](#), [Cut\(\)](#), [CutAppend\(\)](#), [PasteReplaceFromWinClip\(\)](#), [PasteFromWinClip\(\)](#), [CopyToWinClip\(\)](#), [CopyAppendToWinClip\(\)](#), [CutToWinClip\(\)](#), [CutAppendToWinClip\(\)](#) Variables: [InsertLineBlocksAbove](#), [UnMarkAfterPaste](#)

PasteReplaceFromWinClip

Inserts the contents of the Windows Clipboard at the cursor position, replacing a block, if it exists.

Syntax: [INTEGER PasteReplaceFromWinClip\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Before text is pasted into the editor, if a block exists, and the cursor is inside the block, the block is deleted. If a block exists, but the cursor is not inside the block, the block is not deleted. The cursor is placed at the end of the inserted text. See the [PasteFromWinClip\(\)](#) command for additional usage notes. See Also: [PasteFromWinClip\(\)](#), [CopyToWinClip\(\)](#), [CopyAppendToWinClip\(\)](#), [CutToWinClip\(\)](#), [CutAppendToWinClip\(\)](#), [PasteReplace\(\)](#), [Paste\(\)](#), [Copy\(\)](#), [CopyAppend\(\)](#), [Cut\(\)](#), [CutAppend\(\)](#), [CopyBlock\(\)](#), [DelBlock\(\)](#), [MoveBlock\(\)](#), [isWinClipAvailable\(\)](#), [EmptyWinClip\(\)](#) Variables: [InsertLineBlocksAbove](#), [UnMarkAfterPaste](#)

PasteUndoDelete

Pastes the last deletion from the global Deletion Buffer at the cursor position. The Deletion Buffer contents remain unchanged.

Syntax: [INTEGER PasteUndoDelete\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command is similar to the [Paste\(\)](#) command for the Clipboard, in that the contents of the Deletion Buffer are not changed. This is useful for making multiple copies of the last deleted line, word, or block. See Also: [UndoDelete\(\)](#), [GlobalUndoDelete\(\)](#), [isBlockMarked\(\)](#)

PBAttribute

Returns the attributes of the current file entry in a PickBuffer (created by the

[BuildPickBufferEx\(\)](#) command). This command is intended for ADVANCED USAGE. **Syntax:** [INTEGER PBAttribute\(\)](#) Returns: The attributes of the indicated PickBuffer file entry. Notes: This command works in conjunction with the [BuildPickBufferEx\(\)](#) command. The current buffer should be one filled with file entries via the [BuildPickBufferEx\(\)](#) command. The current line in the buffer is the file entry for which file attribute information is returned. Supported attribute [values](#) returned by this command are: - [_NORMAL_](#) indicates files that can be read from or written to. - [_READONLY_](#) indicates files that can be read from but not written to. - [_HIDDEN_](#) indicates files that are hidden and do not appear in file directory listings. - [_SYSTEM_](#) indicates files that are system files. - [_VOLUME_](#) indicates the volume label entry. - [_DIRECTORY_](#) indicates directory entries. - [_ARCHIVE_](#) indicates files that are marked for archiving. **Examples:** [integer](#) attribute [if CreateTempBuffer\(\)](#) [and BuildPickBufferEx\("**.*"\)](#) attribute = [PBAttribute\(\)](#) [endif](#) See Also: [BuildPickBufferEx\(\)](#), [PBName\(\)](#), [PBSize\(\)](#), [PBDate\(\)](#), [PBDateStr\(\)](#), [PBTime\(\)](#), [PBTimeStr\(\)](#)

PBDate

Returns the date, in encoded format, of the current file entry in a PickBuffer (created by the

[BuildPickBufferEx\(\)](#) command). This command is intended for ADVANCED USAGE. **Syntax:** [INTEGER PBDate\(\)](#)
Returns: The date of the indicated PickBuffer file entry. Notes: The date is returned in an encoded format that is suitable for comparison of dates returned by this command and the [FFDate\(\)](#) command. **Examples:** [integer](#) date [if](#) [CreateTempBuffer\(\)](#) [and](#) [BuildPickBufferEx\("*.*\)"](#) date = [PBDate\(\)](#) [endif](#) See Also: [BuildPickBufferEx\(\)](#), [PBName\(\)](#), [PBAttribute\(\)](#), [PBSize\(\)](#), [PBDateStr\(\)](#), [PBTime\(\)](#), [PBTimeStr\(\)](#)

PBDateStr

Returns the date, in displayable format, of the current file entry in a PickBuffer (created by the

[BuildPickBufferEx\(\)](#) command). This command is intended for ADVANCED USAGE. **Syntax:** [STRING PBDateStr\(\)](#)
Returns: The date of the indicated PickBuffer file entry. Notes: The date is returned in a string format that is suitable for displaying. **Examples:** [string](#) date[8] [if](#) [CreateTempBuffer\(\)](#) [and](#) [BuildPickBufferEx\("*.*\)"](#) date = [PBDateStr\(\)](#) [endif](#) See Also: [BuildPickBufferEx\(\)](#), [PBName\(\)](#), [PBAttribute\(\)](#), [PBSize\(\)](#), [PBDate\(\)](#), [PBTime\(\)](#), [PBTimeStr\(\)](#)

PBName

Returns the filename of the current file entry in a PickBuffer (created by the

[BuildPickBufferEx\(\)](#) command). This command is intended for ADVANCED USAGE. **Syntax:** [STRING PBName\(\)](#)
Returns: The filename of the indicated PickBuffer file entry. Notes: The filename that is returned includes the name and the extension, if any, and possibly the full path, depending on the flags that were passed to [BuildPickBufferEx\(\)](#). **Examples:** [string](#) fn[[_MAXPATH_](#)] [if](#) [CreateTempBuffer\(\)](#) [and](#) [BuildPickBufferEx\("*.*\)"](#) fn = [PBName\(\)](#) [endif](#) See Also: [BuildPickBufferEx\(\)](#), [PBAttribute\(\)](#), [PBSize\(\)](#), [PBDate\(\)](#), [PBDateStr\(\)](#), [PBTime\(\)](#), [PBTimeStr\(\)](#)

PBSize

Returns the size of the current file entry in a PickBuffer (created by the

[BuildPickBufferEx\(\)](#) command). This command is intended for ADVANCED USAGE. **Syntax:** [INTEGER PBSize\(\)](#)
Returns: The size of the indicated PickBuffer file entry. Notes: This command is valid only for files less than 2 gigabytes in size. **Examples:** [integer](#) size [if](#) [CreateTempBuffer\(\)](#) [and](#) [BuildPickBufferEx\("*.*\)"](#) size = [PBSize\(\)](#) [endif](#) See Also: [BuildPickBufferEx\(\)](#), [PBName\(\)](#), [PBAttribute\(\)](#), [PBDate\(\)](#), [PBDateStr\(\)](#), [PBTime\(\)](#), [PBTimeStr\(\)](#)

PBTime

Returns the time, in encoded format, of the current file entry in a PickBuffer (created by the

[BuildPickBufferEx\(\)](#) command). This command is intended for ADVANCED USAGE. **Syntax:** [INTEGER PBTime\(\)](#)
Returns: The time of the indicated PickBuffer file entry. Notes: The time is returned in an encoded format that is

suitable for comparison of dates returned by this command and the [FFTime\(\)](#) command. [Examples](#): [integer](#) time [if](#) [CreateTempBuffer\(\)](#) [and](#) [BuildPickBufferEx](#)("*.*") time = [PBTime\(\)](#) [endif](#) See Also: [BuildPickBufferEx\(\)](#), [PBName\(\)](#), [PBAttribute\(\)](#), [PBSize\(\)](#), [PBDate\(\)](#), [PBDateStr\(\)](#), [PBTimeStr\(\)](#)

PBTimeStr

Returns the time, in displayable format, of the current file entry in a PickBuffer (created by the

[BuildPickBufferEx\(\)](#) command). This command is intended for ADVANCED USAGE. [Syntax](#): [STRING](#) [PBTimeStr\(\)](#)
Returns: The time of the indicated PickBuffer file entry. Notes: The time is returned in a string format that is suitable for displaying. [Examples](#): [string](#) time[8] [if](#) [CreateTempBuffer\(\)](#) [and](#) [BuildPickBufferEx](#)("*.*") date = [PBTimeStr\(\)](#) [endif](#) See Also: [BuildPickBufferEx\(\)](#), [PBName\(\)](#), [PBAttribute\(\)](#), [PBSize\(\)](#), [PBDate\(\)](#), [PBDateStr\(\)](#), [PBTime\(\)](#)

PeekByte

Returns the 8-bit value (byte) stored at a specified memory location.

This command is intended for ADVANCED USAGE.

[Syntax](#): [INTEGER](#) [PeekByte](#)([INTEGER](#) address) - address is the 32-bit pointer to the desired memory location.
Returns: The contents of the byte at the specified address. [Examples](#): Read the keyboard flags from the BIOS area at 0040:0017h. [integer](#) flags flags = [PeekByte](#)([Ptr](#)(40h, 17h)) See Also: [PeekWord\(\)](#), [PeekLong\(\)](#), [PokeByte\(\)](#), [PokeWord\(\)](#), [PokeLong\(\)](#), [Ptr\(\)](#), [Seg\(\)](#), [Ofs\(\)](#), [AdjPtr\(\)](#)

PeekLong

Returns the 32-bit value (long) stored at a specified memory location.

This command is intended for ADVANCED USAGE.

[Syntax](#): [INTEGER](#) [PeekLong](#)([INTEGER](#) address) - address is the 32-bit pointer to the desired memory location.
Returns: The contents of the 32-bit value at the specified address. [Examples](#): See the [PeekByte\(\)](#) command for an example of similar usage. See Also: [PeekByte\(\)](#), [PeekWord\(\)](#), [PokeByte\(\)](#), [PokeWord\(\)](#), [PokeLong\(\)](#), [Ptr\(\)](#), [Seg\(\)](#), [Ofs\(\)](#), [AdjPtr\(\)](#)

PeekWord

Returns the 16-bit value (word) stored at a specified memory location.

This command is intended for ADVANCED USAGE.

[Syntax](#): [INTEGER](#) [PeekWord](#)([INTEGER](#) address) - address is the 32-bit pointer to the desired memory location.
Returns: The contents of the 16-bit value at the specified address. [Examples](#): See the [PeekByte\(\)](#) command for an example of similar usage. See Also: [PeekWord\(\)](#), [PeekLong\(\)](#), [PokeByte\(\)](#), [PokeWord\(\)](#), [PokeLong\(\)](#), [Ptr\(\)](#), [Seg\(\)](#), [Ofs\(\)](#)

()), [AdjPtr\(\)](#)

Performing Other File-Processing Operations

The editor includes other file-processing operations to:

- Manage and manipulate files on disk.
- Change the current drive and/or directory.
- Display information about the current file.
- Change the name of the current file.
- Insert a copy of another file into the current file.
- Erase a file from disk.

The following commands and options are available for these operations.

* FileManager ("File Manager" under File

[Menu](#)) This command allows you to manage and manipulate the files located on disk, all from within the editor. When you execute this command, the editor places you in the FileManager utility and forces certain key assignments to be active while you are in the FileManager. A PickList of all files and directories in the current directory is displayed. Also displayed are: a menubar at the top of the screen, indicating the categories of options that are available; and a HelpLine at the bottom of the screen, indicating key assignments for commonly-used options of the FileManager. To access the FileManager menu, press <F10> and select the desired menu category (or press <Alt> plus the quick-key of the desired menu category). To exit from the FileManager, press <Escape> (or select "Exit" from the "File" [menu](#) category). The FileManager allows you to execute file- and directory-related commands and operations, including: open, move, copy, delete, rename, print, set the date and time, and change file attributes. You can sort the entries in the directory PickList. You can execute a command on a single file; or you can tag multiple files, and then execute a command on all of the tagged entries. You can even view/manipulate compressed (archived) files. The "File" and "Dir" menus within the FileManager allow you to select the file or directory operation to perform. You can sort the entries in the PickList based on selected keys using the "Sort" menu. The "Print" menu allows you to print one or more files. From the "Execute" menu you can run an executable file. To execute a command for a specific file or directory, first position the cursor bar on the desired file or directory. Then select the command that you wish to execute. To execute a command for a group of files, first tag the desired files or directories. You can tag an entry in the PickList by placing the cursor bar on the entry and pressing the <Spacebar>. Or you can tag (or untag) a group of files/directories according to selected criteria, using options on the "Tag" menu. Once the appropriate files or directories are tagged, select the command that you wish to execute. * ChangeDir ("Change Dir" under File [Menu](#)) This command provides a facility for changing the current drive and/or directory. You can indicate a specific drive/directory to which to change, or optionally select a new directory from a database containing the drives and directories on your system. When you execute this command, the current directory is displayed on the StatusLine; and a prompt box is displayed that contains the directory of the current file, along with the message "Change to directory:". To change to the directory of the current file (as displayed in the prompt), just press <Enter>. To change to another directory, you can type the entire path name of the desired directory and press <Enter>. Unless a drive is included in the path you specify, the macro looks for the indicated directory on the current drive. You can also change to another directory using the optional "database directory" feature included in the macro. This allows you to select a directory based on a partial or ambiguous directory name specification. Use this macro to build and maintain a database of directories on your

machine. Once this database has been created, you can specify a partial directory name to change to; or you can enter an ambiguous directory entry, and then select the desired directory from a PickList displayed by the macro. If the database directory exists, ChangeDir processes the entries you make at the "Change to directory:" prompt as follows: - The editor first attempts to match the specified directory path exactly. If a matching directory exists, that directory becomes the current directory and the command terminates. - If an ambiguous entry is made (that is, one which contains the wildcard character "*"), the editor searches the database for matching directory entries, and displays a PickList of those entries. You can select the desired directory by moving the cursor bar to that directory entry and pressing <Enter>. The selected directory becomes the current directory and the command terminates. - Otherwise, the editor treats your entry as a partial entry, and searches the database for a directory path(s) that begins with the string entered at the prompt. - If only one directory is located that matches the partial entry, that directory becomes the current directory and the command terminates. - If more than one matching directory is located, the editor displays a PickList of matching entries from which you can select the desired directory. The selected directory becomes the current directory and the command terminates. - If a match still has not been found, the editor then searches the database for any directory that contains the indicated string, anywhere in the directory name. If only one match is found, that directory becomes the current directory and the command terminates. If multiple matching entries are found, a PickList of matching entries is displayed and the command terminates once you select an entry. - Finally, if no match is found, the editor issues a message that it could not find any matches, and the command terminates. To initially create or to update the directory database, press <Alt U> once the "Change to directory:" prompt is displayed. The editor then builds, or rebuilds, a database of all the directory entries that are currently on the selected drives. The directory entries are stored in the ChangeDir database file, called "TSECD.DAT". (This file is maintained in the editor [load directory](#).) You can configure which drives are to be searched when the directory database is built (or rebuilt). Press <Alt E> once the "Change to directory:" is displayed, and enter the desired drives at the "Drive letters:" prompt. The order in which the drives are listed will control the order in which the associated directories are listed in the directory database. Any drives that are [not](#) listed will be ignored. * ShowFileInfo ("Show File Info" under File [Menu](#)) This command displays selected information about the current file, if the file exists on disk. When you execute ShowFileInfo, an information box is displayed on the screen that includes the following about the current file: the complete path; the number of lines currently in memory within the editor; the size of the file that is located on disk; the date and time the file was last written to disk; the disk file attributes; and the current read/write permission status. * ChangeCurrFilename (<Alt O>, or "Change Name" under File [Menu](#)) This command changes the name of the current file, but only for the copy of the file that is in memory. The Modified File Indicator is displayed on the StatusLine once the filename has been changed. The file with the new name is not written to disk until you save the newly-named file. This command does not affect the copy of the file on disk under the original name. When you execute ChangeCurrFilename, the editor issues the prompt, "New filename:", and then changes the name of the file in the Ring to the name you indicate at the prompt. To terminate this command, press <Escape> at the prompt. If the filename you enter at the "New filename:" prompt represents a file that is already in the Ring, the message, "Filename in use Press <Escape>" is displayed. The name of the current file is not changed. If the filename you enter at the "New filename:" prompt already exists on disk, the following selection menu is displayed: + Overwrite existing file? + | *Yes*****| No | | Cancel | +-----+ Select "Yes" to complete the name change. (The file on disk is not actually overlaid until the file is saved under the new name.) Select "No" to cancel the specified filename (and re-display the "New filename:" prompt). Select "Cancel" (or press <Escape>) to terminate the ChangeCurrFilename operation. * InsertFile (<Alt R>, or "[Insert](#)" under File [Menu](#)) This command makes a copy of a file on disk and inserts it into the current file. The copy is placed in the current file at the current cursor position as a marked character block. When you execute InsertFile, the prompt "File to insert:" is issued. You can enter the complete filename to be inserted; or you can make an empty, abbreviated, or ambiguous entry, or specify the "[Search SubDirs](#)" (-s) option, allowing you to obtain a PickList of files from which to choose. (For more information on the "[Search SubDirs](#)" option, see "[Command-Line Options](#)" in the chapter on "[Starting the Editor](#).") If you specify a filename that does not exist on disk, the message, "File [not](#) found:filename Press <Escape>", is displayed on the [StatusLine](#). When you press <Escape>, the "File to insert:" prompt is re-displayed. * KillFile (<Ctrl K Z>) This command erases the current file from disk, but keeps a copy of the file in memory in the current editing session. When you execute KillFile, the copy of the current file on disk is immediately erased, and the message, "Current file erased from disk", is displayed on the [StatusLine](#). Once you press a key, the normal StatusLine is re-displayed, including the

Modified File Indicator.

PersistentHistory

Determines whether the items stored in the History Buffer are maintained across editing sessions.

Values: BOOLEAN **Default:** [ON](#) **Notes:** If ON, History Buffer items are maintained in the file TSEHIST.DAT in the editor load directory. The maximum number of items that are kept in this file is governed by the values of [MaxHistorySize](#) and [MaxHistoryPerList](#). See Also: [MaxHistorySize](#), [MaxHistoryPerList](#)

PersistentRecentFiles

Determines whether the items stored in the recently-edited-files list are maintained across editing sessions.

Values: BOOLEAN **Default:** [ON](#) **Notes:** If ON, the recently-edited-files items are maintained in the file TSEFILES.DAT in the editor load directory. The maximum number of items that are kept in this file is governed by the value of [MaxRecentFiles](#). See Also: [MaxRecentFiles](#)

PickDrive

Allows the user to select a drive from a list and returns the selected drive.

Syntax: [STRING](#) [PickDrive\(\)](#) **Returns:** The selected drive, or a zero-length string if <Escape> was pressed. **Notes:** This is the same PickList used in the built-in [PickFile\(\)](#) command, when a drive-list is requested. See Also: [PickFile\(\)](#)

PickFile

Allows the user to select a file from a list and returns the filename.

Syntax: [STRING](#) [PickFile](#)([STRING](#) fn [, [INTEGER](#) flags]) - fn is a string that may include wildcards to specify files to include in the PickList. It may also include a specific drive and/or directory. If these are omitted from the string, the current drive and/or directory are used. - flags is optional. Supported [values](#) for flags are: - [_INSERT_PATH_](#) causes [PickFile\(\)](#) to insert the entire path, rather than just the name of the file. - [_SEARCH_SUBDIRS_](#) causes [PickFile\(\)](#) to also search any subdirectories that are under the current search path. **Returns:** The selected filename (including full path), or a zero-length [string](#) if <Escape> was pressed. **Notes:** If fn is the name of a directory or of a filename containing wildcard characters, a list of all matching files is presented [for](#) selection. This is the same PickList used in the built-in [EditFile\(\)](#) command. See Also: [EditFile\(\)](#), [EditThisFile\(\)](#), [PickDrive\(\)](#) **Variables:** [PickFileFlags](#), [PickFileSortOrder](#)

PickFileChangesDir

Determines whether the editor changes the current directory when a new

directory is selected from a pop-up list of files and directories (the file `PickList`).

Values: `BOOLEAN` Default: `Off` Affects: [ChangeCurrFilename\(\)](#), [EditFile\(\)](#), [EditThisFile\(\)](#), [InsertFile\(\)](#), [PickFile\(\)](#), [SaveAs\(\)](#), [SaveBlock\(\)](#) See Also: [PickFileFlags](#), [PickFileSortOrder](#)

PickFileFlags

Controls the treatment of directories in file-related `PickLists` displayed within the editor.

Values: `INTEGER` - `_ADD_DIRS_` includes all directories in the directory being searched, regardless of the file specification. - `_ADD_SLASH_` adds slashes in front of directories. - `_DIRS_AT_TOP_` places directories at top of `PickList`. - `_NONE_` sets all other flags OFF. Default: `_ADD_DIRS_ | _ADD_SLASH_ | _DIRS_AT_TOP_` Notes: Flags may be OR'ed together to produce the desired effect. To set the flags OFF, use: [Set](#)(`PickFileFlags`, `_NONE_`) Affects: [PickFile\(\)](#), [BuildPickBufferEx\(\)](#), [EditFile\(\)](#), [EditThisFile\(\)](#), [InsertFile\(\)](#), [SaveAs\(\)](#), [SaveBlock\(\)](#), [SaveFile\(\)](#) See Also: [PickFileSortOrder](#)

PickFileSortOrder

Determines the sort key and sort order for file-related `PickLists` within the editor.

Values: `STRING` (maximum of 7 characters) - a (or A) - attribute - d (or D) - date - t (or T) - time - s (or S) - size - n (or N) - name (filename without extension) - e (or E) - extension - f (or F) - full filename, including extension - p (or P) - full path Default: "f" (full filename including extension, in ascending order) Notes: The sort key is indicated by the appropriate letter (as identified above). The sort order is indicated by the case of the sort-key letter: a lower-case letter specifies an ascending sort; an upper-case letter specifies a descending sort. Multiple keys can be specified. Affects: [PickFile\(\)](#), [BuildPickBufferEx\(\)](#), [EditFile\(\)](#), [EditThisFile\(\)](#), [InsertFile\(\)](#), [SaveAs\(\)](#), [SaveBlock\(\)](#), [SaveFile\(\)](#) See Also: [PickFileFlags](#)

PlaceMark

Sets a bookmark at the current cursor position.

Syntax: `INTEGER PlaceMark`(`STRING` mark) - mark is the optional single-character bookmark name to be set. Returns: Non-zero if successful; zero (`FALSE`) on failure. Notes: Returns a non-zero value when a bookmark is set, or zero if the user presses <Escape> in response to the prompt. [PlaceMark\(\)](#) will prompt for the bookmark name if the mark is not passed. The editor allows 26 [bookmarks](#). Bookmark names are A through Z, and are [case](#) insensitive. See Also: [GotoMark\(\)](#), [DelAllBookMarks\(\)](#), [DelBookMark\(\)](#), [GetBookMarkInfo\(\)](#), [isBookMarkSet\(\)](#), [Hook\(\)](#), [UnHook\(\)](#), [PushPosition\(\)](#), [PopPosition\(\)](#)

PokeByte

Writes an 8-bit value (byte) to a specified memory location.

This command is intended for ADVANCED USAGE.

Syntax: [INTEGER PokeByte](#)([INTEGER](#) address, [INTEGER](#) byte) - address is the 32-bit pointer to the desired memory location. - byte is an 8-bit value (byte) to write to the specified address. Returns: The contents of the byte at the specified address, before being changed to the new byte value. **Examples:** Set CAPS LOCK on by writing to the BIOS keyboard flags at 0040:0017h. // set CAPS-LOCK bit in BIOS area [PokeByte](#)([Ptr](#)(40h,17h), 0x40) To toggle CAPS LOCK: [integer](#) flags = [PeekByte](#)([Ptr](#)(40h,17h)) [PokeByte](#)([Ptr](#)(40h,17h), flags ^ 0x40) See Also: [PokeWord](#)(), [PokeLong](#)(), [PeekByte](#)(), [PeekWord](#)(), [PeekLong](#)(), [Ptr](#)(), [Seg](#)(), [Ofs](#)(), [AdjPtr](#)()

PokeLong

Writes a 32-bit value (long) to a specified memory location.

This command is intended for ADVANCED USAGE.

Syntax: [INTEGER PokeLong](#)([INTEGER](#) address, [INTEGER](#) long) - address is the 32-bit pointer to the desired memory location. - long is a 32-bit value to write to the specified address. Returns: The contents of the 32-bit value at the specified address, before being changed to the new long value. **Examples:** See the [PokeByte](#)() command for an example of similar usage. See Also: [PokeByte](#)(), [PokeWord](#)(), [PeekByte](#)(), [PeekWord](#)(), [PeekLong](#)(), [Ptr](#)(), [Seg](#)(), [Ofs](#)() , [AdjPtr](#)()

PokeWord

Writes a 16-bit value (word) to a specified memory location.

This command is intended for ADVANCED USAGE.

Syntax: [INTEGER PokeWord](#)([INTEGER](#) address, [INTEGER](#) word) - address is the 32-bit pointer to the desired memory location. - word is a 16-bit value to write to the specified address. Returns: The contents of the 16-bit value at the specified address, before being changed to the new word value. **Examples:** See the [PokeByte](#)() command for an example of similar usage. See Also: [PokeByte](#)(), [PokeLong](#)(), [PeekByte](#)(), [PeekWord](#)(), [PeekLong](#)(), [Ptr](#)(), [Seg](#)(), [Ofs](#)() , [AdjPtr](#)()

PopBlock

Unmarks the current block and marks the block indicated by the topmost entry in the marked block stack. The state of the newly-marked block, as originally saved, is restored. For example, if a block was opened, but not closed, it will still be in the marking process when popped.

Syntax: [PopBlock](#)() Returns: Nothing. Notes: The marked block stack holds up to 10 entries. This command removes the topmost entry from the stack. There should always be one and only one corresponding [PopBlock](#)() [for](#) each

[PushBlock\(\)](#). It is a logic error to "push" a block on the stack with [PushBlock\(\)](#) and then not remove it with a corresponding [PopBlock\(\)](#). See Also: [PushBlock\(\)](#) [Variables:](#) [Marking](#)

PopLocation

Moves the cursor to the location specified by the topmost entry in the location stack.

Syntax: [PopLocation\(\)](#) Returns: Nothing. Notes: The editor maintains a stack of up to 100 saved locations that can be saved, restored, and removed with the [PushLocation\(\)](#), [PopLocation\(\)](#) and [KillLocation\(\)](#) commands. [PopLocation\(\)](#) removes the last entry placed on the saved location stack by [PushLocation\(\)](#), and moves the cursor to that saved location. This command may cause the editor to make another file the current file. If the location is in another file, the `_ON_CHANGING_FILES_` event is called, if it exists. If a different file does become the current file, it does so in the current window, which might be different from the original window the file was located in. There should always be one, and only one, corresponding [PopLocation\(\)](#) (or [KillPosition\(\)](#)) for each [PushLocation\(\)](#). It is considered a logic error to "push" a location on the stack with [PushLocation\(\)](#) and then not remove it with either [KillLocation\(\)](#) or [PopLocation\(\)](#). Locations are different than positions (used by [PushPosition\(\)](#), [PopPosition\(\)](#), and [KillPosition\(\)](#)). Positions maintain a relative position within a file, adjusting for insertions and deletions. Locations maintain an absolute location within a file, with no adjustments for insertions or deletions. For example, using locations, if one pushes a location (via [PushLocation\(\)](#)) at line 15, and subsequently deletes lines 1, 2, and 3; when the location is visited (via [PopLocation\(\)](#)), line 15 will be made the current line. This is because locations only maintain a line and column number. Locations do not track insertions or deletions. On the other hand, using positions, if one pushes a position (via [PushPosition\(\)](#)) at line 15, and subsequently deletes lines 1, 2, and 3; when the position is visited (via [PopPosition\(\)](#)), line 12 (rather than line 15) will be made the current line. This is because positions remain on the line and column on which they were originally set. Positions track insertions and deletions, to keep the position set at the proper place. See Also: [PushLocation\(\)](#), [KillLocation\(\)](#), [KillPosition\(\)](#), [PushPosition\(\)](#), [PopPosition\(\)](#)

PopPosition

Moves the cursor to the position specified by the topmost entry in the position stack.

Syntax: [PopPosition\(\)](#) Returns: Nothing. Notes: The editor maintains a stack of up to 10 saved positions that can be saved, restored, and removed with the [PushPosition\(\)](#), [PopPosition\(\)](#) and [KillPosition\(\)](#) commands. [PopPosition\(\)](#) removes the last entry placed on the saved position stack by [PushPosition\(\)](#), and moves the cursor to that saved position. This command may cause the editor to make another file the current file. If the position is in another file, the `_ON_CHANGING_FILES_` event is called, if it exists. If a different file does become the current file, it does so in the current window, which might be different from the original window the file was located in. There should always be one, and only one, corresponding [PopPosition\(\)](#) (or [KillPosition\(\)](#)) for each [PushPosition\(\)](#). It is considered a logic error to "push" a position on the stack with [PushPosition\(\)](#) and then not remove it with either [KillPosition\(\)](#) or [PopPosition\(\)](#). See Also: [GetPositionInfo\(\)](#), [KillPosition\(\)](#), [PushPosition\(\)](#), [SwapPosition\(\)](#), [PlaceMark\(\)](#), [GotoMark\(\)](#), [Hook\(\)](#), [UnHook\(\)](#)

PopWinClose

A video output command that closes a pop-up video window and restores the

screen area to its previous contents.

Syntax: [PopWinClose\(\)](#) Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. There should always be one and only one corresponding [PopWinClose\(\)](#) for each [PopWinOpen\(\)](#). See Also: [PopWinOpen\(\)](#)

PopWinCols

The number of columns in the current video output window.

Values: [INTEGER Default:](#) Undefined Limits: Can only be queried. Cannot be used within a config/endconfig construct. Notes: PopWinCols returns the number of columns in the current video output window. The current video window is the current view port window, if one is open; otherwise, it is the entire screen. See Also: [PopWinX1](#), [PopWinY1](#), [PopWinRows](#), [WindowX1](#), [WindowY1](#), [WindowCols](#), [WindowRows](#), [ScreenCols](#), [ScreenRows](#)

PopWinOpen

This is a video output command that opens a pop-up window on the screen. The original screen contents are saved.

Syntax: [INTEGER PopWinOpen\(INTEGER x1, INTEGER y1, INTEGER x2, INTEGER y2, INTEGER boxtype, STRING title, INTEGER attribute\)](#) - x1 is the starting column for the window. - y1 is the starting row. - x2 is the ending column. - y2 is the ending row. - boxtype is the box-drawing character set to use. Supported [values](#) for boxtype are: - 1 - single-line characters - 2 - single-line side, double-line top characters - 3 - double-line side, single-line top characters - 4 - double-line characters - 5 - small block characters - 6 - large block characters - title is the window's title and may be an empty string. - attribute is the color for the window border. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. Whatever is behind the window is saved and is restored when [PopWinClose\(\)](#) is called. All windows popped open by a macro should be closed before the macro terminates or the editor will become confused. There should always be one and only one corresponding [PopWinClose\(\)](#) for each [PopWinOpen\(\)](#). This command paints the window border on the screen and defines the window for future video writing. [ClrScr\(\)](#) may be called after [PopWinOpen\(\)](#) to clear the created window before proceeding. Be sure to use [GotoXY\(\)](#) or [VGotoXY\(\)](#) to insure the current video output position is in the new window before calling [ClrScr\(\)](#). title is placed in the top center border of the window. See Also: [PopWinClose\(\)](#)

PopWinRows

The number of rows in the current video output window.

Values: [INTEGER Default:](#) Undefined Limits: Can only be queried. Cannot be used within a config/endconfig construct. Notes: PopWinRows returns the number of rows in the current video output window. The current video window is the current view port window, if one is open; otherwise, it is the entire screen. See Also: [PopWinX1](#), [PopWinY1](#), [PopWinCols](#), [WindowX1](#), [WindowY1](#), [WindowCols](#), [WindowRows](#), [ScreenCols](#), [ScreenRows](#)

PopWinX1

The left column coordinate of the current video output window, relative to the full display area.

Values: [INTEGER](#) Default: Undefined Limits: Can only be queried. Cannot be used within a config/endconfig construct. Notes: PopWinX1 contains the column coordinate of the top left-hand corner of the current video output window relative to the full screen display. See Also: [PopWinY1](#), [PopWinCols](#), [PopWinRows](#), [WindowX1](#), [WindowY1](#), [WindowCols](#), [WindowRows](#), [ScreenCols](#), [ScreenRows](#)

PopWinY1

The top row coordinate of the current video output window, relative to the full display area.

Values: [INTEGER](#) Default: Undefined Limits: Can only be queried. Cannot be used within a config/endconfig construct. Notes: PopWinY1 contains the row coordinate of the top left-hand corner of the current video output window relative to the full screen display. See Also: [PopWinX1](#), [PopWinCols](#), [PopWinRows](#), [WindowX1](#), [WindowY1](#), [WindowCols](#), [WindowRows](#), [ScreenCols](#), [ScreenRows](#)

Pos

Returns the starting position of one string within another string.

Syntax: [INTEGER](#) [Pos](#)([STRING](#) needle, [STRING](#) haystack) - needle is the string for which to search. - haystack is the string in which to search. Returns: The starting position of needle in haystack (in the range 1 through [Length](#) (haystack)); otherwise, zero ([FALSE](#)) if needle is not found in haystack. Examples: [integer](#) p p = [Pos](#)("World", "Hello World") // p == 7 See Also: [SubStr\(\)](#), [DelStr\(\)](#), [InsStr\(\)](#), [LeftStr\(\)](#), [RightStr\(\)](#) [StrCount\(\)](#), [StrFind\(\)](#), [StrReplace\(\)](#)

PosFirstNonWhite

Returns the position of the first non-white character in the current line.

Syntax: [INTEGER](#) [PosFirstNonWhite](#)() Returns: The position of the first non-white character in the current line (in the range 1 through [CurrLineLen](#)()); zero ([FALSE](#)) if line was empty or contained all [white characters](#). Notes: This is a physical position within the current line. Tabs are counted as a single character regardless of tab expansion. It is the same as the column number unless hard tab characters exist in the line and ExpandTabs is ON. [PosFirstNonWhite](#)() can be used to determine if a line is empty or contains only spaces and tabs, by checking for a return value of zero. Idioms:

```

/***** Position the cursor at the first non-white character (if it exists) on the current line. *****/
GotoPos(PosFirstNonWhite())
```

 See Also: [PosLastNonWhite](#)(), [GotoPos](#)(), [GotoColumn](#)(), [CurrPos](#)(), [CurrCol](#)()

Positioning the Cursor and Scrolling Text

You can use the mouse to position the cursor or scroll text on the screen, as described by the following. The special symbols, v ^ < >, and the horizontal and vertical elevators along the scrollbars are used for scrolling.

To position the editing cursor:

Place the mouse pointer in the text of the current window where you want the cursor to be positioned, and click the <LeftBtn>.

To scroll down in the file, by line:

Place the mouse pointer on the v symbol. To scroll down one line, click the <LeftBtn>. To repeat scrolling, press and hold the <LeftBtn>; scrolling continues until the button is released, or the end of the file is reached.

To scroll up in the file, by line:

Place the mouse pointer on the ^ symbol. To scroll up one line, click the <LeftBtn>. To repeat scrolling, press and hold the <LeftBtn>; scrolling continues until the button is released, or the beginning of the file is reached.

To scroll left in the text, by column:

Place the mouse pointer on the < symbol. To scroll left one column, click the <LeftBtn>. To repeat scrolling, press and hold the <LeftBtn>; scrolling continues until the button is released, or column 1 is reached.

To scroll right in the text, by column:

Place the mouse pointer on the > symbol. To scroll right one column, click the <LeftBtn>. To repeat scrolling, press and hold the <LeftBtn>; scrolling continues until the button is released, or the maximum line length is reached.

To scroll down in the file, by page:

Place the mouse pointer on the scrollbar below the vertical elevator (*), within the vertical elevator shaft. To scroll down one page, click the <LeftBtn>. To repeat scrolling, press and hold the <LeftBtn>, keeping the mouse pointer below the elevator; scrolling continues until the button is released, or the elevator joins the mouse pointer, or the end of the file is reached.

To scroll up in the file, by page:

Place the mouse pointer on the scrollbar above the vertical elevator (*), within the vertical elevator shaft. To scroll up one page, click the <LeftBtn>. To repeat scrolling, press and hold the <LeftBtn>, keeping the mouse pointer above the elevator; scrolling continues until the button is

released, or the elevator joins the mouse pointer, or the beginning of the file is reached.

To scroll left in the text, by TabWidth:

Place the mouse pointer on the scrollbar to the left of the horizontal elevator (*), within the horizontal elevator shaft. To scroll left by one TabWidth, click the <LeftBtn>. To repeat scrolling, press and hold the <LeftBtn>, keeping the mouse pointer to the left of the elevator; scrolling continues until the button is released, or the elevator joins the mouse pointer, or column 1 is reached.

To scroll right in the text, by TabWidth:

Place the mouse pointer on the scrollbar to the right of the horizontal elevator (*), within the horizontal elevator shaft. To scroll right by one TabWidth, click the <LeftBtn>. To repeat scrolling, press and hold the <LeftBtn>, keeping the mouse pointer to the right of the elevator; scrolling continues until the button is released, or the elevator joins the mouse pointer, or the maximum line length is reached.

To scroll up or down in the text as the vertical elevator is dragged:

Place the mouse pointer on the vertical elevator (*) in the scrollbar, and press and hold the <LeftBtn>. The elevator changes to the symbol while dragging is in effect. To scroll up, drag the elevator up. To scroll down, drag the elevator down. Scrolling continues while the elevator is dragged, and stops when the mouse is stationary. To terminate dragging, release the <LeftBtn>.

To scroll left or right in the text as the horizontal elevator is dragged:

Place the mouse pointer on the horizontal elevator (*) in the scrollbar, and press and hold the <LeftBtn>. The elevator changes to the symbol while dragging is in effect. To scroll left, drag the elevator to the left. To scroll right, drag the elevator to the right. Scrolling continues while the elevator is dragged, and stops when the mouse is stationary. To terminate dragging, release the <LeftBtn>.

PosLastNonWhite

Returns the position of the last non-white character in the current line.

Syntax: [INTEGER PosLastNonWhite\(\)](#) Returns: The position of the last non-white character in the current line (in the range 1 through [CurrLineLen\(\)](#)); otherwise, zero (FALSE) if [not](#) found. Notes: This is a physical position within the current line. Tabs are counted as a single character regardless of tab expansion. It is the same as the column number unless hard tab characters exist in the line and ExpandTabs is ON. [PosLastNonWhite\(\)](#) can be used to determine if a

line is empty or contains only spaces and tabs, by checking for a return value of zero. [Examples](#): This usage places the cursor just past the last non-white character of the current line; it is similar to the built-in [EndLine\(\)](#) command: [GotoPos\(PosLastNonWhite\(\) + 1\)](#) See Also: [PosFirstNonWhite\(\)](#), [GotoPos\(\)](#), [GotoColumn\(\)](#), [CurrPos\(\)](#), [CurrCol\(\)](#)

Pre-Defined Constants

The editor includes a number of pre-defined constants. Most pre-defined editor

[constants](#) are enclosed within underscore characters. The user is prohibited from creating numeric [constants](#) enclosed within underscores (as indicated under "[Reserved Words](#)," above). [Examples](#) of pre-defined editor [constants](#) are: [_EXCLUSIVE_](#), [_EDIT_HISTORY_](#), [_OVERWRITE_](#). A few [pre-defined constants](#) that are generally useful are [ON](#), [OFF](#), [TRUE](#), [FALSE](#), [MAXINT](#), [MININT](#), [MAXLINELEN](#), [MAXSTRINGLEN](#), [_MAXPATH_](#), [WIN32](#), and [EDITOR_VERSION](#). ON is the value 1. ON is equivalent to TRUE. OFF is the value 0. OFF is equivalent to FALSE. TRUE is the value 1. TRUE is equivalent to ON. FALSE is the value 0. FALSE is equivalent to OFF. MAXINT is a reserved constant which is equal to the maximum allowable integer value of +2,147,483,647. MININT is a reserved constant which is equal to the minimum allowable integer value of -2,147,483,648. MAXLINELEN is the editor's maximum line length; 16,000. MAXSTRINGLEN is the editor's maximum string length; 255. EDITOR_VERSION is a reserved constant that identifies the version of the editor. For example, a value of 0x4000 represents version 4.00. [_MAXPATH_](#) is intended to be used to set the maximum length of a SAL string that deals with files. Its pre-set value is the maximum allowable length for paths/filenames in the version of the editor that you are running. By using [_MAXPATH_](#) rather than an arbitrary number such as 80 or 255, you are assured that your macro will [not](#) inadvertently truncate long filenames and/or paths. Example: [string](#) fn[[_MAXPATH_](#)] WIN32 is intended to be used with the conditional compilation directives (discussed later in this chapter under "[Conditional Compilation Directives](#)") to write macros that are portable between the various versions of the editor. It indicates the 32-bit Windows version of the editor. In the following examples, any code specific to the 32-bit Windows version of the editor should be placed where the "[WIN32](#) specific code" comment is, and any code specific to the DOS version of the editor should be placed where the "MSDOS specific code" comment is. [Examples](#): [#ifdef WIN32](#) // [WIN32](#) specific code [#else](#) // MSDOS specific code [#endif](#) or [#ifndef WIN32](#) // MSDOS specific code [#else](#) // [WIN32](#) specific code [#endif](#)

PressKey

Executes the command assigned to the specified key, as if the key was pressed on the keyboard.

[Syntax](#): [INTEGER PressKey\(INTEGER key\)](#) - key is the key of which to execute the assigned command. Returns: The result of the command assigned to the specified key. See Also: [GetKey\(\)](#), [PushKey\(\)](#), [ChainCmd\(\)](#)

PrevChar

Moves the cursor to the previous character in the file, traversing lines if necessary.

[Syntax](#): [INTEGER PrevChar\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command does [not](#) stop on beginning-of-line boundaries. It wraps to the end-of-line position of the previous line when issued at

column 1. Zero is returned if [PrevChar\(\)](#) is executed at the beginning-of-file position. See Also: [NextChar\(\)](#), [Right\(\)](#), [Left\(\)](#)

PrevFile

Switches to the previous file in the editor's Ring of files.

Syntax: [INTEGER PrevFile](#)([INTEGER](#) flag) - flag is an optional integer which specifies whether the file being switched to is actually loaded into the editor, if it has [not](#) already been loaded. If [not](#) specified, `_DEFAULT_` is used. Supported [values](#) for flag are: - `_DEFAULT_` causes the previous file to be loaded from disk (if it has not already been loaded) the first time it is switched to. - `_DONT_LOAD_` causes the previous file to not be loaded from disk (if it has [not](#) already been loaded). Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command returns a non-zero value if it changes to another file. If only one file is loaded, it returns zero. If an `_ON_FIRST_EDIT_` event exists, it is called the first time a file is switched to via this command. If an `_ON_CHANGING_FILES_` event exists, it is also called. If `_DONT_LOAD_` is passed, these events are [NOT](#) called. The flag setting of `_DONT_LOAD_` is intended for macro use, for the cases when a macro needs to traverse the files in the editor's internal Ring of files, without actually editing the file. It is not intended for switching to a file that the user can edit, without first issuing an [EditFile](#)([CurrFilename](#)()) command from the same macro. If there is only one file, [PrevFile](#)() has no effect. See Also: [NextFile](#)(), [Hook](#)(), [UnHook](#)(), [EditFile](#)(), [EditThisFile](#)(), [BufferType](#)()

PrevHelp

Invokes Help on the last Help topic accessed.

Syntax: [INTEGER PrevHelp](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If Help was not invoked before [PrevHelp](#)(), the Table of Contents is loaded. See Also: [Help](#)(), [SearchHelp](#)()

PrevPosition

Positions the cursor at the last line from which the cursor was moved.

Syntax: [INTEGER PrevPosition](#)() Returns: Zero ([FALSE](#)) if there is not a previous position to go to; [otherwise](#), non-zero. Notes: The editor tracks changes in line or file position, [not](#) in column positions. While a Compiled Macro is active, the editor's tracking is turned off so that it ignores all line changes. The editor's tracking is not file specific; thus, it may cause a different file to become the current file. If a different file does become the current file, an `_ON_CHANGING_FILES_` event, if it exists, will be called. If a different file does become the current file, it does so in the current window, which might be different from the original window the file was located in. See Also: [PushPosition](#)(), [PopPosition](#)(), [PlaceMark](#)(), [GotoMark](#)(), [Hook](#)(), [UnHook](#)()

PrevWindow

Makes the previous editing window the current window.

Syntax: [INTEGER PrevWindow](#)() Returns: Zero ([FALSE](#)) if there was only one window or the current window is

zoomed; [otherwise](#), non-zero. Notes: If the current window is the first one in the sequence, [PrevWindow\(\)](#) switches to the last active window and makes it the current window. If an `_ON_CHANGING_FILES_` event exists, it is called after this command executes, unless previous window contains the same file as the next window. Each window's sequence number is shown in the window's border. These numbers are assigned in the order windows are opened. If a window is closed, its number is reused. Hence the sequence of numbers does not always reflect the order in which windows are created. If there is only one window, or if [windows](#) are zoomed, no action occurs and zero is returned. See Also: [NextWindow\(\)](#), [GotoWindow\(\)](#), [Hook\(\)](#), [UnHook\(\)](#)

PrintBlock

Prints the marked block.

Syntax: [INTEGER PrintBlock\(\)](#) Returns: Zero (FALSE) if the printer device returns an error status; [otherwise](#), non-zero. Notes: The block must be in the current file, otherwise, nothing is printed and zero is returned. If a `PrintInit` string is defined, it is sent to the printer before any text is printed. If an error occurs while printing, zero is returned. For more information, see the chapter on "[Printing from the Editor](#)" in [the User's Guide](#). See Also: [PrintFile\(\)](#), [PrintChar\(\)](#)
Variables: `PrintInit`, `PrintDevice`

PrintBotMargin

Specifies the number of blank lines to be left at the bottom of each printed page.

Values: [INTEGER](#) - [0..255] **Default:** 0 Notes: `PrintBotMargin` specifies the number of blank lines to be left at the bottom of each printed page. If a footer is specified by `PrintFooter`, it will be placed just above the bottom margin. For [PrintBotMargin](#) to be effective, [PrintLinesPerPage](#) must be set to a value greater than 0. The effective number of lines of text printed on each page is equal to: [PrintLinesPerPage](#) - ([PrintTopMargin](#) + [PrintBotMargin](#)) **Affects:** [PrintFile\(\)](#), [PrintBlock\(\)](#) See Also: [PrintTopMargin](#), [PrintLinesPerPage](#)

PrintChar

Sends a single character to the current print device.

Syntax: [INTEGER PrintChar\(\[STRING\]\(#\) c\)](#) - c is the single character to output. Returns: Zero (FALSE) if the printer device returns an error status; [otherwise](#), non-zero. Notes: The [default](#) print device is `lpt1`. This can be changed by setting the `PrintDevice` variable. If the passed parameter is longer than one character, it is truncated to one character. The `PrintInit` [string](#), even if defined, is [not](#) automatically sent to the printer for this command. If an error occurs while printing, zero is returned. See Also: [PrintFile\(\)](#), [PrintBlock\(\)](#) **Variables:** `PrintDevice`

PrintCopies

Specifies the number of copies to be printed.

Values: [INTEGER](#) - [1..32767] **Default:** 1 Notes: [PrintFile\(\)](#) and [PrintBlock\(\)](#) print the number of copies specified by

[PrintCopies](#). If PrintAddFF is set ON, a formfeed is sent between copies. [Affects](#): [PrintFile\(\)](#), [PrintBlock\(\)](#) See Also: PrintAddFF

PrinterFontFlags

Determines the initial printer font attributes.

[Values](#): [INTEGER](#) [_FONT_BOLD_](#) specifies a bold printer font. [_FONT_OEM_](#) specifies a printer font that uses the OEM character set. [Default](#): 0 - normal weight, ANSI character set. Limits: Must be a combination of the above specified values. Notes: It is [not](#) necessary to explicitly set this variable. It is set by the [SetPrinterFont\(\)](#) command. [Affects](#): Printing. See Also: [PrintUseScreenFont](#), [PrinterFontName](#), [PrinterFontSize](#) Commands: [SetPrinterFont\(\)](#)

PrinterFontName

Determines the initial printer font.

[Values](#): [STRING](#) (maximum of 32 characters) [Default](#): "Courier New" Limits: Must be an available Windows font. Notes: It is [not](#) necessary to explicitly set this variable. It is set by the [SetPrinterFont\(\)](#) command. [Affects](#): Printing See Also: [PrintUseScreenFont](#), [PrinterFontFlags](#), [PrinterFontSize](#) Commands: [SetPrinterFont\(\)](#)

PrinterFontSize

Determines the initial printer font size.

[Values](#): [INTEGER](#) [Default](#): 10 Limits: Must be a valid printer font size. Notes: If PrinterFontSize is <= 255, it is assumed to be a point size, and is applied appropriately. If [PrinterFontSize](#) is > 255, it is assumed to be a specific character width and height. For instance: PrinterFontSize 10 specifies a point size of 10. PrinterFontSize 0x080c specifies a width of 8, and a height of 0x0c, or 12. It is [not](#) necessary to explicitly set this variable. It is set by the [SetPrinterFont\(\)](#) command. See the [SetFont\(\)](#) command for an example use of specifying the size in hex. [Affects](#): Printing. See Also: [PrintUseScreenFont](#), [PrinterFontName](#), [PrinterFontFlags](#) Commands: [SetPrinterFont\(\)](#)

PrintFile

Prints the current file.

[Syntax](#): [INTEGER](#) [PrintFile\(\)](#) Returns: Zero (FALSE) if the printer device returns an error status; [otherwise](#), non-zero. Notes: Sends the entire current file to the current print device. If a PrintInit string is defined, it is sent to the printer before any text is printed. If an error occurs while printing, zero is returned. For more information, see the chapter on "[Printing from the Editor](#)" in [the User's Guide](#). See Also: [PrintBlock\(\)](#), [PrintChar\(\)](#) [Variables](#): PrintInit, PrintDevice

PrintFirstPage

The number of the page at which printing will begin.

Values: [INTEGER](#) - [0..65535] **Default:** 0 **Notes:** [PrintFirstPage](#) allows you to specify the page number from which to start printing. If set to 0, printing begins with the first page, just as if it were set to 1. For [PrintFirstPage](#) to be effective, [PrintLinesPerPage](#) must be set to a value greater than 0. **Affects:** [PrintFile\(\)](#), [PrintBlock\(\)](#) See Also: [PrintLastPage](#), [PrintLinesPerPage](#)

PrintFooter

Prints a footer line at the bottom of each printed page.

Values: [STRING](#) (maximum of 4 characters) Options that specify which of the following items to print on the footer line: - F - Places the file's name and extension, but not the path, at the left margin. The name is upper-cased. - D - Places the system date centered between the left and right margins. If the "T" option is also specified, the date and time are centered together with the date preceding the time. - T - Places the system time centered between the left and right margins. If the "D" option is also specified, the date and time are centered together with the date preceding the time. - P - Places the page number at the right margin. **Default:** "" **Notes:** The Footer string is printed before the bottom margin is added. A blank line is printed before the footer is printed. This means that two lines of text area are used to accommodate the footer. For [PrintFooter](#) to be effective, [PrintLinesPerPage](#) must be set to a value greater than 0 or formfeeds must exist within the file. If you specify a footer and [PrintLinesPerPage](#) is 0, and there are no formfeeds within the file, then the footer is printed on the the last page only. If [PrintRightMargin](#) is set to 0, a default of 72 will be used in determining where to place the Footer information. **Affects:** [PrintFile\(\)](#), [PrintBlock\(\)](#) See Also: [PrintLeftMargin](#), [PrintRightMargin](#), [DateFormat](#), [DateSeparator](#), [TimeFormat](#), [TimeSeparator](#), [PrintHeader](#), [PrintLinesPerPage](#)

PrintHeader

Prints the specified options at the top of each printed page.

Values: [STRING](#) (maximum of 4 characters) Options that specify which of the following items are to be printed on the header line. - F - filename - D - date - T - time - P - page number See [PrintFooter](#) for a detailed explanation of each option. **Default:** "" **Notes:** The Header string is printed after the top margin. A blank line is printed after the header is printed. This means that two lines of text area are used to accommodate the header. For [PrintHeader](#) to be effective, [PrintLinesPerPage](#) must be set to a value greater than 0 or formfeeds must exist within the file. If you specify a header and [PrintLinesPerPage](#) is 0, and there are no formfeeds within the file, then the header is printed on the top of the first page only. If [PrintRightMargin](#) is set to 0, a default of 72 is used in determining where to place the Header information. **Affects:** [PrintFile\(\)](#), [PrintBlock\(\)](#) See Also: [PrintFooter](#), [PrintLeftMargin](#), [PrintRightMargin](#), [DateFormat](#), [DateSeparator](#), [TimeFormat](#), [TimeSeparator](#), [PrintLinesPerPage](#)

Printing from the Editor

This chapter describes the features in the editor that allow you to print all or a portion of a file, and to format the text to be printed. The following sections identify the commands and options that are available for:

-

[Sending output and commands to the printer](#) (PrintFile, PrintBlock, SendFormfeed, SendInitString) - [Formatting print output](#) - [Setting print options](#) (PrintLeftMargin, PrintRightMargin, PrintLinesPerPage, PrintTopMargin, PrintBotMargin, PrintLineSpacing, PrintLineNumbers, PrintHeader, PrintFooter, PrintFirstPage, PrintLastPage, PrintCopies, PrintDevice, PrintInit, [PrintPause](#), PrintAddFF, PrintUseFF) - [Saving print options for future sessions](#)

PrintLastPage

The last page that will be printed.

Values: [INTEGER](#) - [0..65535] **Default:** 0 **Notes:** If [PrintLastPage](#) is set to 0, then [PrintFile\(\)](#) will continue until the end of the file is reached, and [PrintBlock\(\)](#) will continue until the end of the block is reached. For [PrintLastPage](#) to be effective, [PrintLinesPerPage](#) must be set to a value greater than 0. **Affects:** [PrintFile\(\)](#), [PrintBlock\(\)](#) See Also: [PrintFirstPage](#), [PrintLinesPerPage](#)

PrintLeftMargin

Controls the number of spaces added to the beginning of each line as it is printed.

Values: [INTEGER](#) - [0..255] **Default:** 0 **Notes:** PrintLeftMargin specifies the number of spaces that will be printed at the beginning of each printed line. This does not specify the column in which the printing will begin. For example, a setting of 5 will cause 5 spaces to be printed before the text from the line is printed. **Affects:** [PrintFile\(\)](#), [PrintBlock\(\)](#) See Also: [PrintRightMargin](#), [PrintLineNumbers](#), [PrintHeader](#), [PrintFooter](#)

PrintLineNumbers

Causes the line number to be printed at the beginning of each line.

Values: [BOOLEAN](#) **Default:** [OFF](#) **Notes:** The line number will use a minimum of 2 columns of the current printed line. The width taken up by the line number will be at least the width of the last line number printed plus one. If a PrintLeftMargin is set, the line number and remaining text is offset by the left margin setting. The number printed is the actual line number of the line in the file. For example, if a block is marked from line 12 to line 25 in a file, then the line [numbers](#) will begin at 12 instead of 1. **Affects:** [PrintFile\(\)](#), [PrintBlock\(\)](#) See Also: [PrintLeftMargin](#), [PrintRightMargin](#)

PrintLineSpacing

Specifies the line spacing to use (for instance, 1 for single-spaced output, 2

for double-spaced output, and so forth). **Values:** [INTEGER](#) - [1..256] **Default:** 1 **Notes:** Setting PrintLineSpacing to 2 will cause the text to be double spaced when printed. A setting of 3 will cause triple spacing, and so on. PrintLineSpacing only adds to the data that is being sent to the print device. The contents of the current file are [not](#) altered. **Affects:** [PrintFile\(\)](#), [PrintBlock\(\)](#)

PrintLinesPerPage

Sets the maximum number of lines to print on each page.

Values: [INTEGER](#) - [0..255] **Default:** 0 **Notes:** Set PrintLinesPerPage to the maximum number of lines that can be printed on one page. The number of text lines actually printed per page depends on the settings of PrintHeader, PrintFooter, [PrintLineSpacing](#), [PrintBotMargin](#), and [PrintTopMargin](#). Setting [PrintLinesPerPage](#) to 0 forces continuous printing, and causes many of the other print settings to have no effect. If this setting is greater than the actual number of printable lines per page recognized by the printer, then page breaks will not be where they are expected. The effective number of lines of text printed on each page is equal to: [PrintLinesPerPage](#) - ([PrintTopMargin](#) + [PrintBotMargin](#)) Dot Matrix users: Most Dot Matrix printers [default](#) to printing 6 lines per inch, which equates to 66 lines per page for standard 8.5-inch by 11-inch paper. Laser/InkJet Printer users: Most printers of this type [default](#) to 6 lines per inch, but only print 60 lines per page. This is because some of them have "unprintable zones" (that is, areas that cannot be printed in at all). If you set the editor to use a page length greater than 60, you will get a full page, then part of a second page before the editor sends the formfeed. **Affects:** [PrintFile\(\)](#), [PrintBlock\(\)](#)

PrintPause

Causes the editor to pause between printed pages. The message "Printing paused - press a key to continue" is given.

Values: [BOOLEAN](#) **Default:** [OFF](#) **Notes:** If you are using a printer that requires manual feeding, set PrintPause ON, so that the [PrintFile\(\)](#) and [PrintBlock\(\)](#) commands pause between each page. For PrintPause to be effective, PrintLinesPerPage must be set to a value greater than 0 or formfeeds must exist within the file. **Affects:** [PrintFile\(\)](#), [PrintBlock\(\)](#) See Also: [PrintLinesPerPage](#)

PrintRightMargin

Sets the maximum number of characters to be printed on a line. Lines with more than this number of characters are truncated.

Values: [INTEGER](#) - [0..255] **Default:** 0 **Notes:** The PrintRightMargin setting includes the PrintLeftMargin setting and PrintLineNumbers, if specified. For example, if the [PrintRightMargin](#) is 72, [PrintLeftMargin](#) is 12, and PrintLineNumbers is OFF, then the first 60 characters of each line are sent to the print device. If PrintLineNumbers is ON, the width taken up by the line numbers is equal to the width of the largest line number plus one. **Affects:** [PrintFile\(\)](#), [PrintBlock\(\)](#) See Also: [PrintLeftMargin](#), [PrintLineNumbers](#), [PrintHeader](#), [PrintFooter](#)

PrintTopMargin

Specifies the number of blank lines to place at the beginning of each page.

Values: [INTEGER](#) - [0..255] **Default:** 0 **Notes:** PrintTopMargin specifies the number of blank lines to be left at the top of each printed page. If a header is specified by PrintHeader, it is placed just below the top margin. For [PrintTopMargin](#) to be effective, [PrintLinesPerPage](#) must be set to a value greater than 0 or formfeeds must exist in the

file. If you specify a top margin and PrintLinesPerPage is 0, and there are no formfeeds within the file, then the top margin is placed on the top of the first page only. The effective number of lines of text printed on each page is equal to: [PrintLinesPerPage](#) - ([PrintTopMargin](#) + [PrintBotMargin](#)) [Affects:](#) [PrintFile\(\)](#), [PrintBlock\(\)](#) See Also: [PrintBotMargin](#), [PrintLinesPerPage](#)

PrintUseScreenFont

Determines which font the editor uses for printing.

[Values:](#) BOOLEAN [Default:](#) [ON](#) [Notes:](#) If PrintUseScreenFont is On, the editor ignores the user specified printer font (set via [SetPrinterFont\(\)](#)) when printing. In this case, the font used depends on the editor being used: In the GUI version of the editor, the current screen font is used in printing. In the Console version of the editor, the default printer font is used when printing. If PrintUseScreenFont is Off, the editor uses the user specified printer font (set via [SetPrinterFont\(\)](#)). [Affects:](#) Printing. See Also: [PrinterFontName](#), [PrinterFontFlags](#), [PrinterFontSize](#) [Commands:](#) [SetPrinterFont\(\)](#)

Process

Invokes another editor process.

[Syntax:](#) [INTEGER Process\(\)](#) [Returns:](#) The result of the [EndProcess\(\)](#) that terminated the call to [Process\(\)](#). If [EndProcess\(\)](#) is not passed a parameter, the result is 0. [Notes:](#) This command invokes the editor recursively. Control is not returned to the macro that originally invoked [Process\(\)](#) until the [EndProcess\(\)](#) command is executed. [Examples:](#) /
***** A simple browse only mode macro. In this example, the editor is re-entered at the call to [Process\(\)](#), and control is not returned to the Browse() macro until [EndProcess\(\)](#) is called - in this particular [case](#), by pressing <Escape>
*****/ [keydef](#) BrowseKeys <CursorUp> [Up\(\)](#)
<CursorDown> [Down\(\)](#) <Escape> [EndProcess\(\)](#) [end proc](#) Browse() [if](#) [Enable](#)(BrowseKeys, _EXCLUSIVE_) [Message](#)
("Press <Escape> to exit [browse mode](#)") [Process\(\)](#) [Disable](#)(BrowseKeys) [endif](#) [end](#) See Also: [EndProcess\(\)](#), [Enable\(\)](#), [Disable\(\)](#)

Process() and Keydefs

The editor offers a special manipulative feature in the

[Process\(\)](#) command. This command allows you, within a running macro, to give control of the editor back to the user. By also using keydefs, you can limit the keys that are available within the particular process. The [Enable\(\)](#) command can be used to enable your [keydef](#) and make it exclusive, if you want. For example, if you want to allow the user to move around using the arrow keys until he presses <Escape>, you could do the following: [keydef](#) ArrowKeys
<CursorRight> [Right\(\)](#) <CursorLeft> [Left\(\)](#) <CursorUp> [Up\(\)](#) <CursorDown> [Down\(\)](#) <Escape> [EndProcess\(\)](#) [end](#)
[proc](#) MoveAround() [Enable](#)(ArrowKeys, _EXCLUSIVE_) // Only enable [cursor](#) keys [Process\(\)](#) // Give control to user
[Disable](#)(ArrowKeys) // Turn off the keydef [end](#) In the above example, the macro is suspended until the user presses
<Escape> to invoke the [EndProcess\(\)](#) command. After that, the macro regains control at the statement immediately following the [Process\(\)](#) command.

ProcessHotSpot

Processes internally-defined mouse hot spots, such as scrollbars, zoom window, close window, horizontal window, vertical window, and mousemarking.

Syntax: [INTEGER ProcessHotSpot\(\)](#) Returns: A non-zero value if a hot spot exists at (MouseX, MouseY); [otherwise](#), zero ([FALSE](#)). **Examples:** To pull down the main menu if the mouse is clicked outside the pre-defined hot spots: [if not ProcessHotSpot\(\)](#) MainMenu() [endif](#) To pull down the main menu if the mouse is clicked outside the pre-defined hot spots as well as modify the default mousemarking mode to column block [marking: case MouseHotSpot\(\)](#) [when _NONE_ MainMenu\(\)](#) [when _MOUSE_MARKING_ MouseMarking\(_COLUMN_\)](#) [otherwise ProcessHotSpot\(\)](#) [endcase](#) See Also: [MouseHotSpot\(\)](#), [MouseWindowId\(\)](#)

ProtectedSaves

Specifies saving the file to disk by overlaying the existing file or by first saving under a temporary filename.

Values: BOOLEAN **Default:** [ON](#) Notes: For most circumstances, it is recommended to set [ProtectedSaves](#) ON. This is preferred, as the original file is not erased until the edited file has been safely written to disk. Setting ProtectedSaves OFF is somewhat faster, but at a loss of safety because the original file is immediately overwritten. If a hardware, power, or system failure occurs before the save operation is complete, both the original and the newly edited file could be lost or scrambled. If editing is being done on a shared or network drive, and the user does [not](#) have rename permission, then [ProtectedSaves](#) must be set OFF in order for the editor to be able to save files. If MakeBackups is set ON, then ProtectedSaves is ignored and the safe method is used. **Affects:** [SaveAllAndExit\(\)](#), [SaveAllFiles\(\)](#), [SaveAndQuitFile\(\)](#), [SaveFile\(\)](#) See Also: [MakeBackups](#), [BackupExt](#), [BackupPath](#)

Ptr

Builds a 32-bit pointer from specified segment and offset values.

This command is intended for ADVANCED USAGE.

Syntax: [INTEGER Ptr](#)([INTEGER](#) segment, [INTEGER](#) offset) - segment is the segment value (upper 16 bits) for the 32-bit pointer. - offset is the offset value (lower 16 bits) for the 32-bit pointer. Returns: 32-bit pointer composed of the specified segment and offset. Notes: This command is used to build a full 32-bit pointer, using the specified segment and offset [values](#). 32-bit pointers consist of a segment and an offset. The segment is the upper 16 bits, while the offset is the lower 16 bits. **Examples:** The BIOS Data Segment is at segment 40h. Some keyboard flags are at offset 17h. To read the current keyboard flags from the BIOS area: [integer](#) flags // read contents of 0040:0017h flags = [PeekByte](#)([Ptr](#) (40h, 17h)) This is equivalent to the [GetKeyFlags](#)() command. See Also: [Seg\(\)](#), [Ofs\(\)](#), [AdjPtr\(\)](#)

Pull-Down Menu Categories

The categories on the main Menu serve to group similar functions. This section describes each of the Menu categories, and identifies other chapters that discuss the individual functions in more detail.

The File

[Menu](#) displays commands used to create, load, list, select, save, close, rename, manage, and display information [about](#) files; as well as commands to exit the editor by closing and optionally saving all files. (For more information, see the chapter on "[Loading, Processing, and Saving Files](#).") The Block and Clip [Menus](#) provide access to commands used to mark and manipulate blocks of text, including commands to store and retrieve blocks using ClipBoards. (For more information, see the chapter on "[Block Operations](#).") The Text [Menu](#) displays commands that allow you to add, copy, move, format, delete, and recover text. (For more information, see the chapter on "[Manipulating Text](#).") The Search [Menu](#) includes commands to find, view, or replace text, as well as commands to locate specific positions in a file by line, column, or Bookmark. (For more information, see the chapters on "[Search Features: Finding and Replacing Text](#)," "[Moving Through Text in a File](#)," and "[Bookmarks](#).") The Window Menu provides a list of commands used to open, close, resize, and navigate through [windows](#), and to synchronize scrolling of files in [windows](#). (For more information, see the chapter on "[Splitting the Screen into Windows](#).") The Macro [Menu](#) displays commands that allow you to create and execute Keyboard and Compiled Macros. (For more information, see the chapter on "[Creating and Using Macros](#).") The Print Menu provides access to commands for printing all or a portion of a file, as well as options that allow you to format the text to be printed. This set of functions is a subset of the "[Print Output Options](#)" available in the Interactive Configuration facility (IConfig). (For more information about printing options, see the chapter on "[Printing from the Editor](#)." For more information on IConfig, see the chapter on "[Configuring the Editor Interactively](#)." The Util Menu includes commands for a variety of utility functions, including [drawing lines in the text](#), sorting text, checking the spelling of words in a file, displaying a pop-up ASCII chart, inserting a date/time stamp, executing external commands during an editing session, maintaining a miscellaneous collection ("[potpourri](#)") of readily-available supplemental macros, changing the current video mode, viewing/editing a file in hexadecimal format, and activating Template-expansion mode. (For more information, see the chapters on "[Special Commands and Features](#)," "[Checking Your Spelling](#)," "[Loading, Processing, and Saving Files](#)," and "[Editing Modes](#)." The Options Menu provides ready access to a select group of configuration options, including [AutoIndent](#), [WordWrap](#), [RightMargin](#), [LeftMargin](#), [TabType](#), TabWidth, Backups, and SaveState. It also gives you full access to the Interactive Configuration facility (IConfig) [and](#) allows you to save your current configuration options. (For more information, see the chapter on "[Configuring the Editor Interactively](#)." The Help Menu displays options available in the editor's context-sensitive, hypertext HelpSystem. (For more information, see the chapter on "[Getting Help in the Editor](#).")

PurgeKeyMacro

Purges any

[Keyboard Macros](#) that may exist. **Syntax:** [PurgeKeyMacro](#)() **Returns:** Nothing. **Notes:** This command is useful if [Keyboard Macros](#) have been loaded and/or recorded, and it is desired to recover the pre-existing key assignments. See Also: [LoadKeyMacro](#)(), [RecordKeyMacro](#)()

PurgeMacro

Purges a Compiled Macro file (and all Compiled Macros contained in that file) from memory.

Syntax: [INTEGER PurgeMacro](#)([\[STRING s\]](#)) - s is the name of the Compiled Macro file to purge from memory. **Returns:** Zero ([FALSE](#)) if macro file s was [not](#) found; [otherwise](#), non-zero. **Notes:** An executing macro will [not](#) be purged until it has finished execution. Key assignments set by the Compiled Macros within the macro file (and not

superseded by a subsequently-loaded macro file) are restored to their prior settings. See Also: [LoadMacro\(\)](#), [ExecMacro\(\)](#), [isMacroLoaded\(\)](#), [ExecLoadedMacro\(\)](#)

PurgeSynhi

Terminates SyntaxHilite mode for the current file.

This command is intended for ADVANCED USAGE.

Syntax: [PurgeSynhi\(\)](#) Returns: Nothing. Notes: To re-initiate SyntaxHilite mode for the current file, call [InitSynhiCurrFile\(\)](#). See Also: [InitSynhiCurrFile\(\)](#), [GetSynFilename\(\)](#), [GetSynLanguageType\(\)](#), [LinkSynFile\(\)](#), [LoadSynhiAssoc\(\)](#), [ReplaceSynFile\(\)](#) **Variables:** [ShowSyntaxHilite](#)

Purging Macros

The

[PurgeMacro\(\)](#) command removes an external Compiled Macro file from the editor and releases the virtual memory that it occupied. It is not necessary to purge macros, but there are times when you may want to free up the space, to be used by other [macros](#) or for editing. [Macros](#) that are [not](#) currently running, may be swapped out to virtual memory, and would [not](#) necessarily need to be purged. If you plan to use the macro repeatedly, it may be best to leave the macro loaded for the duration of the editing session. (For more information on [purging macros](#), see "[Introduction to External Compiled Macros](#)" in the chapter on "[Creating and Using Macros](#)" in [the User's Guide](#). Also see the [PurgeMacro\(\)](#) command in the chapter on "[Editor Commands](#).")

PushBlock

Saves information about the current block, placing the information on the marked block stack. This information is used to re-mark a block when the

[PopBlock\(\)](#) command is executed. **Syntax:** [PushBlock\(\)](#) Returns: Nothing. Notes: Up to 10 block marks can be saved on the marked block stack at any one time. When [PushBlock\(\)](#) is executed, the first nine existing entries on the stack are pushed down the stack. The 10th entry is pushed off the bottom of the stack. See Also: [PopBlock\(\)](#)

PushKey

Pushes a key onto the editor's internal keyboard stack. The key will be processed the next time the editor issues a read to the keyboard for keystrokes.

Syntax: [INTEGER PushKey\(INTEGER key\)](#) - key is the key name to place in the editor's internal keyboard stack. It can be one of the editor's pre-defined key names in the format <editor key-name> (as shown by the ShowKey macro, located in the MAC\ editor subdirectory) or the integer value corresponding to a key (also as shown by ShowKey). Returns: Zero (FALSE) if the keystroke stack is full; otherwise, non-zero. Notes: Up to 64 keys can be stored in the stack at any one time. Keys are processed the next time the editor requests a key from the keyboard. This is a LIFO (Last In, First Out) stack. **Examples:** This macro fragment shows the integer accepted being set if <Enter> or <Escape>

is pressed. If any other [key](#) is pressed, that key is placed in the editor's internal keystroke stack and macro execution is halted. The editor processes the key after halting the macro. [key](#) = [GetKey\(\)](#) [case key when](#) <Enter> accepted = [TRUE when](#) <Escape> accepted = [FALSE otherwise PushKey\(key\) halt endcase](#) See Also: [GetKey\(\)](#), [KeyPressed\(\)](#), [PushKeyStr\(\)](#), [PressKey\(\)](#)

PushKeyStr

Pushes multiple keys (limited to ASCII characters) onto the editor's internal keyboard stack. The keys will be processed the next time the editor issues a read to the keyboard for keystrokes.

Syntax: [INTEGER PushKeyStr\(STRING s\)](#) - s is a string of keys (ASCII characters) to push onto the editor's internal keyboard stack. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: When processed, the characters placed on the keyboard stack by this command are read just as if they were typed from the keyboard. For instance, calling [PushKeyStr\("abc"\)](#) would cause the next 3 characters read to be "a", "b", and "c", in that order. Only ASCII characters can be placed on the keyboard stack with this command. To place function keys (such as <Enter> or <F1>), use the [PushKey\(\)](#) command. The keyboard stack will hold up to 64 keys or characters at any one time. This command returns FALSE if the entire string can not be pushed onto the keyboard stack. **Examples:** Place a string in the keyboard stack: [PushKeyStr\("Live long and prosper"\)](#) See Also: [PushKey\(\)](#), [GetKey\(\)](#), [KeyPressed\(\)](#)

PushLocation

Saves a location onto the position stack.

Syntax: [PushLocation\(\)](#) Returns: Nothing. Notes: The editor maintains a stack of up to 100 saved locations that can be saved and/or restored with the [PushLocation\(\)](#), [PopLocation\(\)](#) and [KillLocation\(\)](#) commands. If 100 entries already exist on the saved location stack when [PushLocation\(\)](#) is executed, the oldest entry is removed from the stack. It is considered a logic error to "push" a location on the stack with [PushLocation\(\)](#), and then [not](#) remove it with either [KillLocation\(\)](#) or [PopLocation\(\)](#). Locations are different than positions (used by [PushPosition\(\)](#), [PopPosition\(\)](#), and [KillPosition\(\)](#)). Positions maintain a relative position within a file, adjusting for insertions and deletions. Locations maintain an absolute location within a file, with no adjustments for insertions or deletions. For example, using locations, if one pushes a location (via [PushLocation\(\)](#)) at line 15, and subsequently deletes lines 1, 2, and 3; when the location is visited (via [PopLocation\(\)](#)), line 15 will be made the current line. This is because locations only maintain a line and column number. Locations do not track insertions or deletions. On the other hand, using positions, if one pushes a position (via [PushPosition\(\)](#)) at line 15, and subsequently deletes lines 1, 2, and 3; when the position is visited (via [PopPosition\(\)](#)), line 12 (rather than line 15) will be made the current line. This is because positions remain on the line and column on which they were originally set. Positions track insertions and deletions, to keep the position set at the proper place. See Also: [PopLocation\(\)](#), [KillLocation\(\)](#), [KillPosition\(\)](#), [PopPosition\(\)](#), [PushPosition\(\)](#)

PushPosition

Saves a position onto the position stack.

Syntax: [PushPosition\(\)](#) Returns: Nothing. Notes: The editor maintains a stack of up to 10 saved positions that can be saved and/or restored with the [PushPosition\(\)](#), [PopPosition\(\)](#) and [KillPosition\(\)](#) commands. If 10 entries already exist

on the saved position stack when [PushPosition\(\)](#) is executed, the oldest entry is removed from the stack. It is considered a logic error to "push" a position on the stack with [PushPosition\(\)](#), and then [not](#) remove it with either [KillPosition\(\)](#) or [PopPosition\(\)](#). See Also: [GetPositionInfo\(\)](#), [KillPosition\(\)](#), [PopPosition\(\)](#), [SwapPosition\(\)](#), [GotoMark\(\)](#), [PlaceMark\(\)](#)

PutAttr

Re-displays existing characters on the screen with a new video attribute.

Syntax: [PutAttr](#)([INTEGER](#) attribute, [INTEGER](#) len) - attribute is the video attribute to use. - len is the number of characters to re-display with the new attribute. Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. The re-display begins at the current video output position. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [PutChar\(\)](#), [PutCharH\(\)](#), [PutCharV\(\)](#), [PutLine\(\)](#), [PutStr\(\)](#), [PutNCharAttr\(\)](#), [PutCtrStr\(\)](#)

PutAttrXY

Re-displays existing characters on the screen with a new video attribute, at the specified position.

Syntax: [PutAttrXY](#)([INTEGER](#) x, [INTEGER](#) y, [INTEGER](#) attribute, [INTEGER](#) len) - x is the column at which to start re-displaying characters. - y is the row at which to start re-displaying characters. - attribute is the video attribute to use. - len is the number of characters to re-display with the new attribute. Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [PutChar\(\)](#), [PutCharH\(\)](#), [PutCharV\(\)](#), [PutLine\(\)](#), [PutStr\(\)](#), [PutNCharAttr\(\)](#), [PutCtrStr\(\)](#)

PutChar

Writes a character to the screen, at the current video output position.

Syntax: [PutChar](#)([STRING](#) s) - s is the character that is written to the screen. Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. s is written at the current video output position. The attribute is determined by the value of the editor Attr variable. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [PutAttr\(\)](#), [PutCharH\(\)](#), [PutCharV\(\)](#), [PutLine\(\)](#), [PutStr\(\)](#), [PutNCharAttr\(\)](#), [PutCtrStr\(\)](#)
Variables: [Attr](#)

PutCharH

Writes a character to the screen one or more times, starting at the current video output position and continuing horizontally.

Syntax: [PutCharH](#)([STRING](#) s, [INTEGER](#) len) - s is the character that is written to the screen. - len is the number of times s is to be written. Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. Care should be taken to ensure that len is within the proper range. The attribute is determined by the value of the editor Attr variable. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [PutAttr](#)(), [PutChar](#)(), [PutCharV](#)(), [PutLine](#)(), [PutStr](#)(), [PutNCharAttr](#)(), [PutCtrStr](#)() **Variables:** [Attr](#)

PutCharHXY

Writes a character to the screen one or more times, starting at the specified position and continuing horizontally.

Syntax: [PutCharHXY](#)([INTEGER](#) x, [INTEGER](#) y, [STRING](#) s, [INTEGER](#) len) - x is the column at which to start writing. - y is the row at which to start writing. - s is the character that is written to the screen. - len is the number of times s is to be written. Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. Care should be taken to ensure that len is within the proper range. The attribute is determined by the value of the editor Attr variable. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [PutAttr](#)(), [PutChar](#)(), [PutCharV](#)(), [PutLine](#)(), [PutStr](#)(), [PutNCharAttr](#)(), [PutCtrStr](#)() **Variables:** [Attr](#)

PutCharV

Writes a character to the screen one or more times, starting at the current video output position and continuing vertically.

Syntax: [PutCharV](#)([STRING](#) s, [INTEGER](#) len) - s is the character that is written to the screen. - len is the number of times s is to be written. Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. The attribute is determined by the value of the editor Attr variable. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [PutAttr](#)(), [PutChar](#)(), [PutCharH](#)(), [PutLine](#)(), [PutStr](#)(), [PutNCharAttr](#)(), [PutCtrStr](#)() **Variables:** [Attr](#)

PutCharXY

Writes a character to the screen, at the specified video output coordinates.

Syntax: [PutCharXY](#)([INTEGER](#) x, [INTEGER](#) y, [STRING](#) s) - x is the column at which to start writing. - y is the row at which to start writing. - s is the character that is written to the screen. Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. The attribute is determined by the value of the editor Attr variable. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [PutAttr](#)(), [PutCharH](#)(), [PutCharV](#)(), [PutLine](#)(), [PutStr](#)(), [PutNCharAttr](#)(), [PutCtrStr](#)() **Variables:** [Attr](#)

PutCtrStr

Writes a string on the specified screen row, and centers it on that row.

Syntax: [PutCtrStr](#)([STRING](#) s, [INTEGER](#) row) - s is the string to write to the screen. - row is the screen row on which to display the specified string. row is relative to the current video window, which is typically the entire screen. Returns: Nothing. Notes: The specified string is centered based on the outer left and right columns of the current video output window. This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." Examples: [integer attr attr = Set](#)(Attr, [Query](#)(BlockAttr)) [PopWinOpen](#)(20, 10, 60, 12, 1, "", [Query](#)(CurrWinBorderAttr)) [ClrScr](#)() [PutCtrStr](#)("test", 1) // center test within window [WindowFooter](#)(" {Press any [key](#)} ") [GetKey](#)() [PopWinClose](#)() [Set](#)(Attr, attr) The result is:
+-----+ | test | +-----+ Press any [key](#) -----+ See Also: [PutAttr](#)(), [PutChar](#)(), [PutCharH](#)(), [PutCharV](#)(), [PutStr](#)(), [WriteLine](#)(), [Write](#)(), [PutLine](#)() Variables: [PopWinX1](#), [PopWinCols](#)

PutHelpLine

Displays a HelpLine string at the current video output position.

Syntax: [PutHelpLine](#)([STRING](#) text) - text is [the HelpLine](#) or message to be formatted and displayed at the current virtual cursor location. Returns: Nothing. Notes: This command is used internally by the editor to display the HelpLines that appear at the bottom of the editing window. See Also: [WindowFooter](#)(), [GotoXY](#)(), [VGotoXY](#)(), [VGotoXYAbs](#)(), [VHomeCursor](#)(), [WhereX](#)(), [WhereXAbs](#)(), [WhereY](#)(), [WhereYAbs](#)() Variables: [Attr](#)

PutLine

Writes a string to the screen, starting at the current video output position.

Syntax: [PutLine](#)([STRING](#) s, [INTEGER](#) field_width [, [INTEGER](#) attribute]) - s is the string that is written to the screen. - field_width is the maximum number of characters from s that will be written. - attribute is the optional attribute used to display s. If not passed, the value of the Attr variable is used. Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. If the length of s is greater than field_width, only field_width characters are written. If the length of s is less than field_width, it is right-padded with spaces to make it the length of field_width. The attribute is determined by the value of the Attr editor variable if the optional attribute flag is not passed. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [PutAttr](#)(), [PutChar](#)(), [PutCharH](#)(), [PutCharV](#)(), [PutStr](#)(), [WriteLine](#)(), [Write](#)(), [PutNCharAttr](#)(), [PutCtrStr](#)() Variables: [Attr](#)

PutLineXY

Writes a string to the screen, starting at the specified video output coordinates.

Syntax: [PutLineXY](#)([INTEGER](#) x, [INTEGER](#) y, [STRING](#) s, [INTEGER](#) field_width [, [INTEGER](#) attribute]) - x is the column at which to start writing. - y is the row at which to start writing. - s is the string that is written to the screen. - field_width is the maximum number of characters from s that will be written. - attribute is the optional attribute used to

display s. If not passed, the value of the Attr variable is used. Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. If the length of s is greater than field_width, only field_width characters are written. If the length of s is less than field_width, it is right-padded with spaces to make it the length of field_width. The attribute is determined by the value of the Attr editor variable if the optional attribute flag is not passed. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [PutAttr\(\)](#), [PutChar\(\)](#), [PutCharH\(\)](#), [PutCharV\(\)](#), [PutStr\(\)](#), [WriteLine\(\)](#), [Write\(\)](#), [PutNCharAttr\(\)](#), [PutCtrStr\(\)](#) Variables: [Attr](#)

PutNCharAttr

Displays a string of character/attribute pairs on the screen.

Syntax: [PutNCharAttr](#)([STRING](#) s, [INTEGER](#) len) - s is the string of character/attribute pairs to be displayed on the screen. - len is the number of character/attribute pairs from s that will be displayed. Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." **Examples:** A simple macro to display the letters "abc" in red, white, and blue, using a yellow background. [proc main\(\)](#) [string](#) s[6] = "a" + [Chr](#)([Color](#)(red on yellow)) + "b" + [Chr](#)([Color](#)(white on yellow)) + "c" + [Chr](#)([Color](#)(blue on yellow)) [VGotoxy](#)(1, 2) [PutNCharAttr](#)(s, [Length](#)(s) / 2) [end](#) See Also: [PutAttr\(\)](#), [PutChar\(\)](#), [PutCharH\(\)](#), [PutCharV\(\)](#), [PutStr\(\)](#), [WriteLine\(\)](#), [Write\(\)](#), [PutLine\(\)](#)

PutOemStrXY

Writes a string to the Window, at the specified coordinates. Characters outside of the range 0x20..0x7e are converted to OEM or 3D versions.

Syntax: [PutOemStrXY](#)([INTEGER](#) x, [INTEGER](#) y, [STRING](#) st [, [INTEGER](#) attribute [, [INTEGER](#) flags]]) - x is the column at which to start writing. - y is the row at which to start writing. - st is the string that is written to the screen. - attribute is the optional attribute used to display st. If not passed, the value of the Attr variable is used. - flags can be [_USE3D_](#) to use 3d characters. [Otherwise](#), if flags is 0 [or not](#) specified, OEM characters are used where appropriate. If [_USE3D_](#) is specified, line drawing characters are shown in a 3-dimensional format. Returns: Nothing. Notes: The ASCII character set defines characters in the range 0x20..0x7e. Characters outside of that range display according to the character set in effect. OEM stands for Original Equipment Manufacturer. The so-called OEM character is used in DOS boxes, and in [Windows](#) Console prompts. Windows uses the so-called ANSI character set. This command can be used to display OEM characters, even when an ANSI character set is currently in use. This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. st is clipped so that it will fit into the current video window, [if](#) needed. The attribute is determined by the value of the Attr editor variable if the optional attribute variable is not passed. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." In the Console version of the editor, no translation takes place, and this command simply calls [PutStrXY\(\)](#). **Examples:** [PutOemStrXY](#)(1, 1, "Hello, World!") See Also: [PutAttr\(\)](#), [PutChar\(\)](#), [PutCharH\(\)](#), [PutCharV\(\)](#), [PutLine\(\)](#), [WriteLine\(\)](#), [Write\(\)](#), [PutNCharAttr\(\)](#), [PutCtrStr\(\)](#) Variables: [Attr](#)

PutStr

Writes a string to the screen, starting at the current video output position.

Syntax: [PutStr](#)([STRING](#) s [, [INTEGER](#) attribute]) - s is the string that is written to the screen. - attribute is the optional attribute used to display s. If not passed, the value of the Attr variable is used. Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. s is clipped so that it will fit into the current video window, [if](#) needed. The attribute is determined by the value of the Attr editor variable if the optional attribute flag is not passed. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [PutAttr](#)(), [PutChar](#)(), [PutCharH](#)(), [PutCharV](#)(), [PutLine](#)(), [WriteLine](#)(), [Write](#)(), [PutNCharAttr](#)(), [PutCtrStr](#)() **Variables:** [Attr](#)

PutStrAttr

Writes a string of text and a string of attributes to the screen, starting at the current video output position.

Syntax: [PutStrAttr](#)([STRING](#) s, [STRING](#) attributes [, [INTEGER](#) attribute]) - s is the string that is written to the screen. - attributes is the string of attributes written to the screen for the corresponding characters in s. - attribute is the optional attribute used to display any character(s) [in](#) s for which there is no corresponding attribute in attributes. If not passed, the value of the [Attr](#) variable is used. Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. s is clipped so that it will fit into the current video window, [if](#) needed. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [PutAttr](#)(), [PutChar](#)(), [PutCharH](#)(), [PutCharV](#)(), [PutLine](#)(), [WriteLine](#)(), [Write](#)(), [PutNCharAttr](#)(), [PutCtrStr](#)() **Variables:** [Attr](#)

PutStrAttrXY

Writes a string of text and a string of attributes to the screen, starting at the specified video output coordinates.

Syntax: [PutStrAttrXY](#)([INTEGER](#) x, [INTEGER](#) y, [STRING](#) s, [STRING](#) attributes [, [INTEGER](#) attribute]) - x is the column at which to start writing. - y is the row at which to start writing. - s is the string that is written to the screen. - attributes is the string of attributes written to the screen for the corresponding characters in s. - attribute is the optional attribute used to display any character(s) [in](#) s for which there is no corresponding attribute in attributes. If not passed, the value of the [Attr](#) variable is used. Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. s is clipped so that it will fit into the current video window, [if](#) needed. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [PutAttr](#)(), [PutChar](#)(), [PutCharH](#)(), [PutCharV](#)(), [PutLine](#)(), [WriteLine](#)(), [Write](#)(), [PutNCharAttr](#)(), [PutCtrStr](#)() **Variables:** [Attr](#)

PutStrEOL

Writes a string to the screen, starting at the current video output position. The string is padded with spaces, if necessary, so that it extends to the end of the current video window.

Syntax: [PutStrEOL](#)([STRING](#) s [, [INTEGER](#) attribute]) - s is the string that is written to the screen. - attribute is the optional attribute used to display s. If not passed, the value of the Attr variable is used. Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. s is clipped so that it will fit into the current video window, [if](#) needed. If the string specified by s does not extend to the end of the current video window, it is padded with spaces to the end of the current video window. The attribute is determined by the value of the Attr editor variable if the optional attribute variable is not passed. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [PutAttr\(\)](#), [PutChar\(\)](#), [PutCharH\(\)](#), [PutCharV\(\)](#), [PutLine\(\)](#), [WriteLine\(\)](#), [Write\(\)](#), [PutNCharAttr\(\)](#), [PutCtrStr\(\)](#) [Variables:](#) [Attr](#)

PutStrEOLXY

Writes a string of text to the screen, starting at the specified video output coordinates. The

[string](#) is padded with spaces, if necessary, so that it extends to the end of the current video window. **Syntax:** [PutStrEOLXY](#)([INTEGER](#) x, [INTEGER](#) y, [STRING](#) s [, [INTEGER](#) attribute]) - x is the column at which to start writing. - y is the row at which to start writing. - s is the string that is written to the screen. - attribute is the optional attribute used to display s. If not passed, the value of the Attr variable is used. Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. s is clipped so that it will fit into the current video window, [if](#) needed. If the string specified by s does not extend to the end of the current video window, it is padded with spaces to the end of the current video window. The attribute is determined by the value of the Attr editor variable if the optional attribute variable is not passed. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [PutAttr\(\)](#), [PutChar\(\)](#), [PutCharH\(\)](#), [PutCharV\(\)](#), [PutLine\(\)](#), [WriteLine\(\)](#), [Write\(\)](#), [PutNCharAttr\(\)](#), [PutCtrStr\(\)](#) [Variables:](#) [Attr](#)

PutStrXY

Writes a string to the screen, starting at the specified video output coordinates.

Syntax: [PutStrXY](#)([INTEGER](#) x, [INTEGER](#) y, [STRING](#) s [, [INTEGER](#) attribute]) - x is the column at which to start writing. - y is the row at which to start writing. - s is the string that is written to the screen. - attribute is the optional attribute used to display s. If not passed, the value of the Attr variable is used. Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. s is clipped so that it will fit into the current video window, [if](#) needed. The attribute is determined by the value of the Attr editor variable if the optional attribute variable is not passed. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [PutAttr\(\)](#), [PutChar\(\)](#), [PutCharH\(\)](#), [PutCharV\(\)](#), [PutLine\(\)](#), [WriteLine\(\)](#), [Write\(\)](#), [PutNCharAttr\(\)](#), [PutCtrStr\(\)](#) [Variables:](#) [Attr](#)

Query

Returns the current value of the specified editor variable.

Syntax: [INTEGER Query](#)([INTEGER](#) editor_variable) or: [STRING Query](#)([STRING](#) editor_variable) - editor_variable

must be one of the pre-defined editor [variables](#). Returns: The value of the editor variable that was queried. Notes: For more information on all the variables in the editor, see the chapter on "[Editor Variables](#)." [Set\(\)](#) returns the same value as [Query\(\)](#) but also may change the value. See Also: [Set\(\)](#), [Toggle\(\)](#)

QueryEditState

Returns the current state of the editor.

This command is intended for ADVANCED USAGE.

Syntax: [INTEGER QueryEditState\(\)](#) Returns: Zero if the editor is in the main editing loop; otherwise, non-zero. Notes: This command is intended for use in macros triggered by the following editor events: - [_NONEDIT_IDLE_](#) - [_BEFORE_NONEDIT_COMMAND_](#) - [_AFTER_NONEDIT_COMMAND_](#) - [_ON_NONEDIT_UNASSIGNED_KEY_](#) - [_BEFORE_GETKEY_](#) - [_AFTER_GETKEY_](#) See the [Hook\(\)](#) command for a description of these events. These particular events can be triggered when a pop-up window is being displayed (such as a menu or prompt). Macros hooked to these events should not display text on the screen or read/write to the current buffer, if the result of [QueryEditState\(\)](#) is non-zero. **Examples:** An idle hook that is called by both [_IDLE_](#) and [_NONEDIT_IDLE_](#): [integer](#) idle_count // only display the message when safe to do so [proc](#) idle() idle_count = idle_count + 1 [if](#) [QueryEditState\(\)](#) == 0 [Message](#)("Idle count is ", idle_count) [endif](#) [end](#) See Also: [Hook\(\)](#)

QuickHelp

Displays the requested help screen(s).

Syntax: [INTEGER QuickHelp](#)(help_identifier) - help_identifier is the name of the help to display. Returns: Non-zero if [QuickHelp\(\)](#) is exited by pressing <Enter> or zero ([FALSE](#)) if exited by pressing <Escape>. Notes: Use [QuickHelp\(\)](#) to display specific help that has been created using the helpdef identifier. For more information on the helpdef identifier, see "[Helpdef](#)" in the chapter on "[Using Macro Language Features](#)." **Examples:** Given: [helpdef](#) MyHelp "Line1 of text help" "Line2 of text help" "LineN of text help" [end](#) then: [QuickHelp](#)(MyHelp) // displays MyHelp See Also: [InsertData\(\)](#)

QuitFile

Quits the current file,

[prompting the user](#) to save changes if the file has been modified. **Syntax:** [INTEGER QuitFile\(\)](#) Returns: Non-zero if the file was successfully saved. Zero ([FALSE](#)) if there was some error saving the file or if the user did not quit the file. Notes: This is also known as a protected quit. A modified file is not abandoned without first warning the user. The prompt given for a modified file is: + Save Changes? + |*Yes*****| | No | | Cancel | +-----+ Pressing <Escape> or selecting "Cancel" will cancel the operation. Selecting "Yes" will save and then quit the file. Selecting "No" will Quit the file, losing any modifications. If only a single file is loaded when executing this command, further action is determined by the setting of the QuitToPrompt variable. If set ON, the editor will prompt the user for another file. Escaping the "File(s) to edit:" prompt will exit the editor to the caller (the operating system or another application). If QuitToPrompt is set OFF, the editor immediately exits to the caller. See Also: [AbandonFile\(\)](#)

Variables: [QuitToPrompt](#)

QuitToPrompt

Determines if the editor prompts for another file to edit once the last file is quit via

[QuitFile\(\)](#) or [SaveAndQuitFile\(\)](#). **Values:** BOOLEAN **Default:** [OFF](#) **Notes:** If QuitToPrompt is ON, and the last remaining file is quit via [QuitFile\(\)](#) or [SaveAndQuitFile\(\)](#), the editor returns to the "File(s) to edit:" prompt; otherwise, control passes to the operating system. If [AbandonFile\(\)](#) is used to quit the last remaining file, then QuitToPrompt is ignored and the editor terminates. **Affects:** [QuitFile\(\)](#), [SaveAndQuitFile\(\)](#)

QuotePath

Encloses a string in quotes, if it contains spaces and is not enclosed in quotes.

Syntax: [STRING](#) [QuotePath](#)([STRING](#) path) - path is the string to (possibly) enclose in quotes. **Returns:** path, enclosed in quotes if applicable. **Examples:** [string](#) fn[_MAXPATH_] // assume fn is set to some user-entered [string](#) fn = [QuotePath](#)(fn) See Also: [ExpandPath\(\)](#), [SplitPath\(\)](#), [GetToken\(\)](#), [NumTokens\(\)](#)

Read

Gets keyboard input from the user.

Syntax: [INTEGER](#) [Read](#)([VAR STRING](#) s [, [INTEGER](#) history_num]) - s specifies where the user's response is returned. s should always be initialized to some value. - history_num is an optional history number. If passed, history will be in effect, using the passed history number. **Returns:** A non-zero value if the user terminated input by pressing <Enter>; otherwise, zero (FALSE) if the user terminated input by pressing <Escape>. **Notes:** When invoked, [Read\(\)](#) clears the screen line at the current cursor position an amount equal to the length of the string passed to [Read\(\)](#). **Examples:** A macro to prompt for a string on the status line. [integer proc](#) Prompt([string](#) msg, [var string](#) reply, [integer](#) history) [Message](#)(msg) [VGotoxy](#)([Wherex](#)(), [Wherey](#)()) [return](#) ([Read](#)(reply, history)) [end](#) Example usage: [string](#) name [40] = "" Prompt("First name:", name, 0) See Also: [Ask\(\)](#), [GetKey\(\)](#), [IRead\(\)](#)

ReadNumeric

Gets numeric keyboard input from the user.

Syntax: [INTEGER](#) [ReadNumeric](#)([VAR STRING](#) s [, [INTEGER](#) history_num]) - s is the string in which the user input (response) is placed. s should always be initialized to some value. - history_num is an optional history number. If specified, history will be in effect, using the passed history number. **Returns:** A non-zero value if the user terminated input by pressing <Enter>; otherwise, zero (FALSE) if the user terminated input by pressing <Escape>. **Notes:** When invoked, this command clears the screen line at the current cursor position an amount equal to the length of the passed [string](#). **Examples:** A macro to prompt for a string of numbers on the status line: [integer proc](#) Prompt([string](#) s, [var string](#) rep, [integer](#) hst) [Message](#)(s) [VGotoxy](#)([Wherex](#)(), [Wherey](#)()) [return](#) ([ReadNumeric](#)(rep, hst)) [end](#) Example usage: [string](#) serial_no[13] = "" Prompt("Serial number:", serial_no, 0) See Also: [Ask\(\)](#), [GetKey\(\)](#), [IRead\(\)](#), [IReadNumeric\(\)](#), [Read\(\)](#)

RecordKeyMacro

Begins and ends the recording of Keyboard Macros.

Syntax: [INTEGER RecordKeyMacro\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: When this command is executed, the editor prompts for a key to which to assign the macro. If the selected key already has an assignment, the user is given a choice of overwriting the existing assignment or cancelling the operation. Pressing <Enter> in response to this prompt assigns the macro to the scrap macro area, overwriting any previous scrap macro assignment. Once the user has chosen a key assignment, a flashing "R" will appear on the StatusLine and all subsequent [key](#) presses will be recorded. Executing RecordKeyMacro a second time turns OFF recording and the flashing "R" is removed from [the StatusLine](#). [RecordKeyMacro\(\)](#) records the actual keys that are pressed. It does not record any commands that are associated with the key via a key assignment. If key assignments are changed by loading a macro file that contains [key](#) assignments (or by assigning Keyboard Macros to a key used in other Keyboard Macros), the Keyboard Macro may [not](#) act as anticipated. Macros assigned to the scrap macro area cannot be saved with [SaveKeyMacro\(\)](#). [PurgeKeyMacro\(\)](#) will restore any previous [key](#) assignments that were in effect prior to the first call of [RecordKeyMacro\(\)](#). See Also: [ExecScrapMacro\(\)](#), [PurgeKeyMacro\(\)](#), [SaveKeyMacro\(\)](#) [Variables:](#) [KbdMacroRecording](#), [KbdMacroRunning](#)

Redo

Restores the last change cancelled by

[Undo\(\)](#). **Syntax:** [INTEGER Redo\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: The Redo list contains information needed to restore editing changes which were just cancelled by [Undo\(\)](#). The Redo list is cleared once the file is changed by execution of another command, besides [Undo\(\)](#). Undo/Redo information is [NOT](#) recorded for [_SYSTEM_ buffers](#). See Also: [Undo\(\)](#), [Redo\(\)](#), [SetUndoOn\(\)](#), [SetUndoOff\(\)](#), [ClearUndoRedoList\(\)](#), [RedoCount\(\)](#), [UndoMode\(\)](#) [Variables:](#) [KeepUndoBeyondSave](#)

RedoCount

Returns the number of Redo entries available for the current file.

Syntax: [INTEGER RedoCount\(\)](#) Returns: The number of Redo entries available for the current file. Notes: The count returned by this command does [not](#) necessarily reflect the number of outstanding events that can be redone. Many of the events may be grouped together as a single redoable entry. See Also: [UndoCount\(\)](#), [Redo\(\)](#), [Undo\(\)](#), [SetUndoOn\(\)](#), [SetUndoOff\(\)](#), [ClearUndoRedoList\(\)](#), [UndoMode\(\)](#) [Variables:](#) [KeepUndoBeyondSave](#)

RemoveProfileItem

Removes an item from a section of an initialization file.

Syntax: [INTEGER RemoveProfileItem](#)([STRING](#) section, [STRING](#) item [, [STRING](#) profile_name]) - section is the name of the section which contains the requested item that is to be removed from an initialization file. - item is the name of the item to remove from the requested section of an initialization file. - profile_name is an optional initialization file's name. If not specified, the tse.ini file, located in the editor's load directory, is assumed. If

profile_name is specified, and does not include a path, the profile_name that resides in the WINDOWS directory (specified by the windir environment variable) is assumed. Returns: Non-zero if the item is removed; otherwise, zero (FALSE). [Examples](#): See the [FlushProfile\(\)](#) command example. See Also: [RemoveProfileSection\(\)](#), [GetProfileStr\(\)](#), [GetProfileInt\(\)](#), [WriteProfileStr\(\)](#), [WriteProfileInt\(\)](#), [LoadProfileSection\(\)](#), [GetNextProfileItem\(\)](#), [LoadProfileSectionNames\(\)](#), [GetNextProfileSectionName\(\)](#), [FlushProfile\(\)](#)

RemoveProfileSection

Removes the requested section (and its items) from an initialization file.

Syntax: [INTEGER RemoveProfileSection](#)([STRING](#) section, [, [STRING](#) profile_name]) - section is the name of the section to remove from an initialization file. - profile_name is an optional initialization file's name. If not specified, the tse.ini file, located in the editor's load directory, is assumed. If profile_name is specified, and does not include a path, the profile_name that resides in the WINDOWS directory (specified by the windir environment variable) is assumed. Returns: Non-zero if the section is removed; otherwise, zero (FALSE). [Examples](#): See the [FlushProfile\(\)](#) command example. See Also: [RemoveProfileItem\(\)](#), [GetProfileStr\(\)](#), [GetProfileInt\(\)](#), [WriteProfileStr\(\)](#), [WriteProfileInt\(\)](#), [LoadProfileSection\(\)](#), [GetNextProfileItem\(\)](#), [LoadProfileSectionNames\(\)](#), [GetNextProfileSectionName\(\)](#), [FlushProfile\(\)](#)

RemoveTrailingSlash

Removes a trailing slash, if present, from a string.

Syntax: [STRING RemoveTrailingSlash](#)([STRING](#) path) - path is a string (typically a directory path) from which to remove a trailing slash. Returns: The string specified by path with the trailing slash removed (if one was present). Notes: If path did not include a trailing slash, path is returned unchanged. This function works with both [forward](#) slashes (/) [and](#) backslashes (\). [Examples](#): Assume dir is a string containing a directory path: dir = [RemoveTrailingSlash](#)(dir) See Also: [AddTrailingSlash\(\)](#), [isTrailingSlash\(\)](#)

RemoveTrailingWhite

Controls the stripping of white characters (spaces and tabs) from the end of edited lines.

Values: BOOLEAN **Default:** [ON](#) Notes: If set ON, any white characters that exist after the last non-white character on an edited line are removed when you move off of the line or execute the [EndLine\(\)](#) command. White characters are the tab (ASCII 9) and the space (ASCII 32). **Affects:** [EndLine\(\)](#)

RemoveUnloadedFiles

Removes all

[buffers](#) that have been allocated for files [not](#) yet loaded. **Syntax:** [RemoveUnloadedFiles\(\)](#) Returns: Nothing. Notes: When the editor is initially started using a file specification that includes multiple files, and/or when multiple files are specified using [EditFile\(\)](#), the editor allocates a buffer for each file found, but only reads the first listed file from disk

into the buffer. Additional files are only loaded when [NextFile\(\)](#), [PrevFile\(\)](#), or [EditFile\(\)](#) are used to switch to another buffer. [RemoveUnloadedFiles\(\)](#) deletes the buffers for all files not yet loaded. [Examples](#): Assume we load file1, file2, file3, file4, and file5 into the editor. We have only switched to file1 [and](#) file2, and decide we do not need to keep the rest of the files. If we call [RemoveUnloadedFiles\(\)](#); file3, file4, and file5 will be removed from the editor's buffer list. [NextFile\(\)](#) and [PrevFile\(\)](#) will show that file1 and file2 are the only files loaded. See Also: [EditFile\(\)](#), [EditThisFile\(\)](#), [AddFileToRing\(\)](#)

RenameAndSaveFile

Renames and saves the current buffer.

Syntax: [INTEGER RenameAndSaveFile\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: The name of the current buffer is changed to the specified name. If successful, the file is saved under the specified name. Contrast this with the [SaveAs\(\)](#) command, which simply writes the current buffer to disk using a specified name, but does not change the name of the current buffer. See Also: [ChangeCurrFilename\(\)](#), [SaveAs\(\)](#), [SaveFile\(\)](#)

RenameDiskFile

Renames the specified file on disk.

Syntax: [INTEGER RenameDiskFile](#)([STRING](#) oldname, [STRING](#) newname) - oldname is the name of the original file. - newname is the name of the new file. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command does not affect files loaded into the editor. It only [affects](#) files on disk. Wildcard characters are [not](#) allowed in either name. Fully specified paths are also supported by this command. A file can be moved to from one directory to another, as long as the files are on the same drive. [Examples](#): [RenameDiskFile](#)("my.file", "your.file") [string](#) fn1[12] = "first.file", fn2[12] = "second.file" [RenameDiskFile](#)(fn1, fn2) See Also: [ChangeCurrFilename\(\)](#), [EraseDiskFile\(\)](#)

RepeatCmd

Repeats the last command or Keyboard Macro the specified number of times.

Syntax: [RepeatCmd\(\)](#) Returns: Nothing. Notes: The user is prompted for the number of times to repeat the last command. This command includes a history list of previous entries passed to it. See [AddHistoryStr\(\)](#) for additional information. See Also: [AddHistoryStr\(\)](#)

RepeatFind

Repeats the most recent

[Find\(\)](#) or [Replace\(\)](#) command. **Syntax:** [INTEGER RepeatFind](#)([INTEGER](#) direction) - direction is an optional integer that forces the repeated find or replace to proceed in the indicated direction. Supported [values](#) for direction are: - [_BACKWARD_](#) forces the search to proceed backward, towards the beginning of the file. - [_FORWARD_](#) forces the search to proceed forward, towards the end of the file. - [_REVERSE_](#) forces the search to proceed in the opposite direction than the last find or replace. - [_SEARCH_INCLUSIVE_](#) forces the search to proceed starting from the

current position, rather than one character beyond the current cursor position, for a forward search (as is normally done). Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Returns zero if nothing was found. For a forward search, the search starts one character beyond the current cursor position. For a backward search, the search starts one character before the current cursor position. See Also: [Find\(\)](#), [Replace\(\)](#), [lFind\(\)](#), [lReplace\(\)](#), [lRepeatFind\(\)](#)
[Variables](#): [Beep](#)

Repeating a Character, Command, Or Macro

* RepeatCmd (<Ctrl Q Q>)

This command instructs the editor to repeat the last character, command, or macro entered from the keyboard. When you execute RepeatCmd, the editor displays the prompt "

[Repeat](#) how many times?". Enter the desired number. The editor then repeats the last keystroke entry the specified number of times. To terminate the RepeatCmd after it has begun, press <Ctrl Break>. Because RepeatCmd repeats the last KEYSTROKE entry, it should not be used to repeat a command that is executed from the pull-down Menu or a command that issues a prompt.

Replace

Finds and replaces specified text.

Syntax: [INTEGER Replace](#)([\[STRING](#) search, [STRING](#) replace, [STRING](#) options]) - search is the optional string for which to search. - replace is the optional string to be substituted for the search [string](#). - options is the optional string containing the search options; multiple options may be specified in any combination. Supported [values](#) for options are:
 - b - search backwards from the current position. (The search begins one character before the current cursor position.) - l - limit the search to the marked block. - i - ignore differences between upper-[case](#) and lower-[case](#) characters. - w - limit the search to whole words. (See the WordSet variable for what constitutes a word.) - x - recognize regular expression characters in the search [string](#). - g - start the search at the beginning of the file. - c - limit the search to the current line. - a - search all files currently loaded in the editor's Ring of files. The current file is searched from the current position, and all other loaded files are searched from the beginning of each file. - ^ - limit the search to the beginning of the line. - \$ - limit the search to the end of the line. - + - start the search at the next character, rather than starting at the current position. (This option is valid only with a forward search.) - (any number) - make this number of replacements; do [not](#) prompt for confirmation. - n - do [not](#) confirm each replacement. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Either all parameters must be passed, or no parameters can be passed. If no parameters are passed, [Replace\(\)](#) prompts for the search string, replace string and options. In the prompt for options, [Replace\(\)](#) initially uses the default options as specified by the ReplaceOptions variable. Subsequent replaces use the previously entered (or passed via a macro) options as the default for the prompt. Certain combinations of options affect each other, as follows: - If both b and g are specified, the search starts at the end of the file. - If both l and g are specified, the search starts at the beginning of the marked block. - If b, l, and g are specified together, the search starts at the end of the marked block. - If both c and g are specified, the search starts at the beginning of the current line. - If b, c, and g are specified together, the search starts at the end of the current line. - If both a and g are specified, all files (including the current file) are searched from the beginning of each file. Zero is returned if <Escape> is pressed at either prompt or if the search [string](#) is [not](#) found. Macros can use the [lReplace\(\)](#) command to search for text in a macro without the side effects of placing the search string and options string in the [Replace\(\)](#) histories, and setting the [RepeatFind\(\)](#) command's execution strategy. This command includes a history list of previous entries passed to it. See [AddHistoryStr\(\)](#) for additional information. For more information on regular expressions, see the chapter on "[Search](#)

[Features: Finding and Replacing Text](#)" in the User's Guide. [Examples](#): /

***** A [Replace\(\)](#) that ALWAYS uses the current ReplaceOptions configuration setting. *****/ [integer proc](#) mReplace() [string](#) needle[40], new_needle[40] needle = "" new_needle = "" [return](#)(iif([Ask](#)("Search for:", needle) and [Ask](#)("Replace with:", new_needle), [Replace](#)(needle, new_needle, [Query](#)([ReplaceOptions](#))), [FALSE](#))) [end](#) See Also: [Find\(\)](#), [RepeatFind\(\)](#), [lFind\(\)](#), [lReplace\(\)](#), [lRepeatFind\(\)](#), [AddHistoryStr\(\)](#) [Variables](#): [Beep](#), [ReplaceOptions](#), [WordSet](#)

ReplaceFile

Replaces the current buffer with the specified file.

Syntax: [INTEGER](#) [ReplaceFile](#)([[STRING](#) fn [, [INTEGER](#) flag]]) - fn is the optional name of the file that will replace the current buffer. If fn is not specified, the editor will prompt for the name of the file. - flag is an optional integer that specifies whether [ReplaceFile\(\)](#) will issue warning messages and other prompts, or simply return FALSE when an error condition is encountered. Supported [values](#) for flag are: - [_DEFAULT_](#) causes all warning messages and prompts to appear. - [_DONT_PROMPT_](#) causes [ReplaceFile\(\)](#) to simply [return FALSE](#) when an error condition is encountered. - [_OVERWRITE_](#) causes [ReplaceFile\(\)](#) to ignore any changes to the file in the current buffer. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If the file in the current buffer has been changed, [ReplaceFile\(\)](#) will prompt for confirmation before replacing the buffer (unless [_OVERWRITE_](#) has been specified). **Examples:** // Prompt user for a new file to replace the current one [ReplaceFile\(\)](#) // Replace the current file with "foo". If the current // file has changed, confirm replacing it. [ReplaceFile](#)("foo") // Replace the current file with "foo". Ignore any // changes to the current file. [ReplaceFile](#)("foo", [_OVERWRITE_](#)) // Replace the current file with "foo". Ignore any // changes to the current file. Do not issue any error // messages. [ReplaceFile](#)("foo", [_DONT_PROMPT_](#)|[_OVERWRITE_](#)) See Also: [EditFile\(\)](#), [EditThisFile\(\)](#), [ChangeCurrFilename\(\)](#) [Variables](#): [UseCommonDialogs](#)

ReplaceOptions

Defines the options for the initial execution of

[Replace\(\)](#). **Values:** [STRING](#) (maximum of 11 characters) - B - search backwards - G - global search - C - search current line - L - local search - I - ignore [case](#) - W - whole words only - N - no prompting for each replacement - X - regular [expressions](#) - ^ - tie [string](#) to beginning of line - \$ - tie [string](#) to end of line - + - start search/replace with the character following the current [cursor](#) position - 1..n - Number of non-prompted replacements **Default:** "" **Limits:** Has effect at startup only. Notes: The option is only used the first time [Replace\(\)](#) is executed. On subsequent executions, the previous option string is used. See the [Replace\(\)](#) command for explanations of each option. **Affects:** [Replace\(\)](#) See Also: [FindOptions](#)

ReplaceSynFile

Reloads a fresh copy of the specified SyntaxHilite mapping (".syn") file (if a copy of that ".syn" file is already loaded), and relinks the SyntaxHilite information to any associated files.

This command is intended for ADVANCED USAGE.

Syntax: [INTEGER ReplaceSynFile](#)([STRING](#) fn) - fn the name of the ".syn" file to be replaced. Returns: Non-zero if fn was loaded and is valid; [otherwise](#), zero ([FALSE](#)). Notes: This command should be called whenever a SyntaxHilite mapping (".syn") file is updated. This command has no effect if fn was [not](#) already loaded when this command is called. **Examples:** [ReplaceSynFile](#)("sal") See Also: [InitSynhiCurrFile](#)(), [PurgeSynhi](#)(), [GetSynFilename](#)(), [GetSynLanguageType](#)(), [LinkSynFile](#)(), [LoadSynhiAssoc](#)() **Variables:** [ShowSyntaxHilite](#)

Reserved Words

The macro language includes certain special names that have pre-defined meanings within

[macros](#). They are "reserved" by SAL and must be used in accordance with SAL [syntax](#). This means that you cannot use them to name your own [constants](#), [variables](#), procedures, and so forth. The following are macro language [reserved words](#): and [else](#) false [integer](#) [not](#) [shl](#) [elseif](#) for [keydef](#) off [shr](#) [break](#) end [forward](#) [loop](#) on [string](#) by [endcase](#) [goto](#) [maxint](#) or times [case](#) [endconfig](#) [halt](#) [maxlinelen](#) [otherwise](#) to [config](#) [enddo](#) [helpdef](#) [maxstringlen](#) [proc](#) true [constant](#) [endfor](#) if [menu](#) [public](#) until [datadef](#) [endif](#) [iif](#) [menubar](#) [var](#) [do](#) [endloop](#) in [minint](#) repeat when downto [endwhile](#) [include](#) [mod](#) [return](#) while [editor](#) [version](#) [win32](#) In addition, user variables or procedures cannot use the names of the editor's native commands (as described in the chapter on "[Editor Commands](#)"), or any names which are enclosed within underscore ("_") characters, as are the editor's [pre-defined constants](#).

ResizeFont

Increases or decreases the current font size.

This command has no effect in the Console version of the editor.

Syntax: [INTEGER ResizeFont](#)([INTEGER](#) size) - size determines by how much the current font will be resized. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Size refers to the width and height of displayed characters in pixels. To Increase the current font size, specify a positive value for size. To decrease the current font size, specify a negative value [for](#) size. This command changes the point size of the font, which may not cause an increase in both width and height of displayed characters. In fact, it may [not](#) cause any change at all, depending on the font currently in use. To get the current point size, see the [GetFont](#)() command. If the result of subtracting size from the current point size is less than 1, no resize takes place and [ResizeFont](#)() returns [FALSE](#). **Examples:** To gradually increase the size of the current font: <f11> [ResizeFont](#)(1) Pressing <f11> will "grow" the current font. To gradually decrease the size of the current font: <f11> [ResizeFont](#)(-1) Pressing <f11> will "shrink" the current font. See Also: [GetFont](#)(), [GetCharWidthHeight](#)(), [SetFont](#)(), [ChooseFont](#)()

ResizeWindow

Resizes the current editing window.

Syntax: [INTEGER ResizeWindow](#)([[INTEGER](#) which_side, [INTEGER](#) how_much]) - which_side is one of [_UP_](#), [_DOWN_](#), [_RIGHT_](#) or [_LEFT_](#), and specifies which window edge to move. - how_much is how many positions to increase (if positive) [or](#) decrease (if negative) the selected window edge. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If more than two editing [windows](#) are open and non-zoomed, this command prompts for

the edge of the current window to adjust. The edge to be moved is indicated by pressing the cursor key that "points" to that window. The selected edge must be adjacent to the edge of a second window. The same edge in the second window must touch only one other window, namely the window to be resized. If the optional parameters are passed, the indicated side is adjusted, if valid. After the edge to adjust is selected, the editor allows the edge to be interactively adjusted using the cursor keys until either <Enter> or <Escape> is pressed to end the adjustment. When a window is resized, the [window](#)(s) adjacent to it grows or shrinks to compensate. This command [affects](#) only [windows](#) created by [HWindow\(\)](#) and [VWindow\(\)](#), and not those created by the video output commands. Zero is returned if <Escape> is pressed, or there is only one window, or windows are zoomed, or the edge indicated is one that the editor cannot adjust. [Examples](#): [ResizeWindow](#)(_UP_, 6) [ResizeWindow](#)(_DOWN_, -5) // Reduce window by 5 lines, // by pulling bottom edge up. [proc](#) mSplitAndResize() [VWindow](#)() [ResizeWindow](#)(_LEFT_, 6) [end](#) See Also: [OneWindow](#)(), [HWindow](#)(), [VWindow](#)(), [SetWindowHeight](#)(), [SetWindowWidth](#)()

RestoreCursorLine

Restores the text on the cursor line to its previous contents.

[Syntax](#): [INTEGER](#) [RestoreCursorLine](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command restores the current line if the cursor has not been moved from the line. Once the cursor is moved off the line, the original line can only be restored if no editing has been performed and the cursor is restored to the original line. See Also: [UnDelete](#)(), [GlobalUnDelete](#)()

ReturnEqNextLine

Determines if

[CReturn](#)() should split the current line when Insert mode is ON. [Values](#): BOOLEAN [Default](#): [OFF](#) Notes: If [Insert](#) is [ON](#) and [ReturnEqNextLine](#) is [OFF](#), then [CReturn](#)() will split the current line at the cursor position. The newly-added line will contain the text that was moved from the previous line, and the cursor will be placed according to the value of [AutoIndent](#). If [Insert](#) is [OFF](#), this setting has no effect. [Affects](#): [CReturn](#)() See Also: [AutoIndent](#)

Right

Moves the cursor one position to the right.

[Syntax](#): [INTEGER](#) [Right](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command does [not](#) cross line boundaries; however, [NextChar](#)() does. Returns zero when the current column equals the maximum line length. Idioms: Repeat an operation on each character in the current line (Assumes [BegLine](#)() issued to start from the beginning) [while](#) [CurrChar](#)() >= 0 // do something [if](#) [not](#) [Right](#)() [break](#) [endif](#) [endwhile](#) See Also: [Left](#)(), [NextChar](#)()

RightMargin

Sets the right margin used by WordWrap and the

[WrapPara](#)() and [WrapLine](#)() commands. [Values](#): [INTEGER](#) - [0..16,000] [Default](#): 72 Notes: When WordWrap mode is ON, or you invoke the [WrapPara](#)() or [WrapLine](#)() commands, the column specified by RightMargin is used to determine where a line of text should be cut off and wrapped onto the next line. A RightMargin of 0 is handled the

same as a RightMargin of 1. [Affects: WrapPara\(\)](#), [WrapLine\(\)](#) See Also: [LeftMargin](#), [AutoIndent](#), [WordWrap](#)

RightStr

Returns the specified right-most characters of a string.

Syntax: [STRING RightStr](#)([STRING](#) s, [INTEGER](#) len) - s is the string from which the characters are to be retrieved. - len is the number of characters to retrieve, starting from the right end of the string. Returns: The [string](#) characters requested. Notes: This command is a convenient short-cut for the slice feature, when you need to get the right-most len characters of a string. With slicing, you must also compute the starting position, which involves an extra (sometimes error-prone) calculation. With [RightStr\(\)](#), you simply specify how many characters you want. The entire [string](#) will be returned if len is greater than or equal to the length of s. **Examples:** To return the n rightmost characters of s: s2 = [RightStr](#)(s, n) Using slices: s2 = s[[Length](#)(s) - n + 1:n] Given: [string](#) abc[11] = "catastrophe" then: [RightStr](#)(abc, 6) // returns "trophe" See Also: [LeftStr\(\)](#), [Pos\(\)](#), [DelStr\(\)](#), [InsStr\(\)](#), [SubStr\(\)](#), [StrCount\(\)](#), [StrFind\(\)](#), [StrReplace\(\)](#)

RmDir

Remove a directory.

Syntax: [INTEGER RmDir](#)([STRING](#) dir) - dir is the directory that will be removed. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If dir does not include a path, the current directory is assumed. If dir is not empty (it contains files or a directory) it will [not](#) be removed and [RmDir\(\)](#) will [return FALSE](#). **Examples:** [RmDir](#)("some-dir") [RmDir](#)("c:\windows\some-dir") See Also: [MkDir\(\)](#), [ChDir\(\)](#), [CurrDir\(\)](#), [GetDir\(\)](#)

RollDown

Scrolls the text one line at a time toward the end of the file while the cursor remains stationary on the screen.

Syntax: [INTEGER RollDown](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Zero is returned when the current line is the last line in the file. See Also: [RollUp\(\)](#), [Up\(\)](#), [Down\(\)](#), [ScrollUp\(\)](#), [ScrollDown\(\)](#), [ScrollToRow\(\)](#), [ScrollToCenter\(\)](#), [ScrollToTop\(\)](#)

RollLeft

Scrolls the view of text on the screen one position to the left, within the current window, while the cursor remains stationary on the screen.

Syntax: [INTEGER RollLeft](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. See Also: [RollRight\(\)](#), [ScrollLeft\(\)](#), [ScrollRight\(\)](#)

RollRight

Scrolls the view of text on the screen one position to the right, within the current window, while the cursor remains stationary on the screen.

Syntax: [INTEGER RollRight\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. See Also: [RollLeft\(\)](#), [ScrollLeft\(\)](#), [ScrollRight\(\)](#)

RollUp

Scrolls the text one line at a time toward the beginning of the file, while the cursor remains stationary on the screen.

Syntax: [INTEGER RollUp\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Zero is returned if the first line of the file is in the window. See Also: [RollDown\(\)](#), [Up\(\)](#), [Down\(\)](#), [ScrollUp\(\)](#), [ScrollDown\(\)](#), [ScrollToRow\(\)](#), [ScrollToCenter\(\)](#), [ScrollToTop\(\)](#)

RTrim

Trims trailing white space from a string.

Syntax: [STRING RTrim\(STRING s\)](#) - s is the input string. Returns: A copy of s, with trailing white space removed. Notes: The passed [string](#) itself is [not](#) modified. **Examples:** [string](#) s[10] = " test ", s2[10] s2 = [RTrim](#)(s) // s2 is " test", // s is still " test " s = [RTrim](#)(s) // s is " test" s = [RTrim](#)(" abc ") // s is " abc" See Also: [LTrim\(\)](#), [Trim\(\)](#)

Running Spellcheck

To start SpellCheck, select "Spell Check" under the Util Menu, or press <F12>. SpellCheck displays a menu with the following options:

```
+ Spell Check -+
|*File*****|
| Rest of File |
| Block       |
| Word        |
+-----+
```

The scope of spell checking in the current file is determined by the selection chosen from this

[menu](#), as follows: - To check all words in the file, select "File". Spell checking starts at the beginning of the file and continues until the end of the file is reached. - To check all words from the current cursor position to the end of the file, select "Rest of File". - To check only those words in the marked block, select "Block". Spell checking starts at the beginning of the block, and continues until the end of the block is reached. If no block is marked in the current file, an error message is displayed on [the StatusLine](#) and SpellCheck stops. - To check the word in which the cursor is located, select "Word". If the cursor is not on a word, the first word to the right of the cursor is checked. When SpellCheck encounters a word for which it cannot find a match in the applicable word [list](#)(s), a menu of suggested spellings, followed by a set of alternative actions, is displayed similar to the following: +----- "occurance" [not](#) found -----

+ |**[*occurrence*****]*| | [occupancy] | | [occurred] | | [occurrent] | | [occurring] | | [occupant] | | [occupants] | | [occlusive] | | [occupiers] | | [occurrences] | +-----+ |
Skip Once | | Ignore for Rest of Session | | Edit | | Add to Word List | | Tag | | Quit |
+-----+ To select a word from the suggestion list, move the cursor bar to the desired word, and press <Enter>. This occurrence of the word in question is replaced in the file by the selected suggestion, and spell checking proceeds. To determine suggested spellings, SpellCheck searches the standard word list only; it does not search [the user-defined word list](#). If no alternative spellings for the word can be found by SpellCheck, the message "not found" is displayed, and the list of suggested spellings is left empty. Other actions that are available for the word in question are listed below the suggested spellings. To select one of these alternatives, position the cursor bar on the desired action and press <Enter>, or type the highlighted capital letter of the desired option. The following is an explanation of these alternate options: - To skip the word in question for this occurrence only, select "Skip Once". Spell checking continues; however, SpellCheck will stop on the word if it is found again in the text. - To skip the indicated word for the current editing session, select "Ignore for Rest of Session". Spell checking continues. SpellCheck remembers this word until the editor is terminated, and will not stop on the word if found again during this editing session. - To enter a new spelling for the word, select "Edit". When selected, this prompt is displayed:
+-----+ |Change word: | |occurrence | +-----+ Type the new spelling and press <Enter>. The word is replaced in the file with the new spelling. Spell checking continues, starting with that new word. To cancel the "Edit" option, press <Escape>. The menu of suggested spellings and options is then re-displayed. - To add the word in question to your own supplementary user-defined word list, select "Add to Word List". SpellCheck places the word in the USER.LEX file, and spell checking continues. SpellCheck will not stop on that word if found again in any editing session. (For more information on [the user-defined word list](#), see "[The User-Defined Word List](#)" later in this chapter.) - To mark the indicated word for later reference after spell checking completes processing, select "Tag". The word is enclosed in double chevrons (* u) and can be located with the Find command. (The numeric keypad can be used to input the decimal value of a character that is not directly available from the keyboard. The decimal value of the left chevron is 174. For more information on using the numeric keypad to input characters, see the Tip under "[Commands to Add Text](#)" in the chapter on "[Manipulating Text](#)".) Be sure to remove the chevron characters before saving the file. - To end spell checking, select "Quit" (or press <Escape>). The current spell-checking process is terminated, and you are returned to your editing session. (For more information on how the editor defines a word, see "[WordSet](#) Characters" under "[Command/Format Options](#)" in the chapter on "[Configuring the Editor Interactively](#)".)

SaveAllAndExit

Saves all modified files and then exits the editor.

Syntax: [INTEGER](#) [SaveAllAndExit](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command does not save files that have not been modified. This command does not prompt; it immediately saves all modified files, and then exits to the operating system. Returns zero if there is an error during any of the save operations (and the command is halted at that point); returns a non-zero value if all saves are successfully completed. The `_ON_EXIT_CALLED_` event is called at the beginning of this command. (See the [Hook\(\)](#) command for a description of this event.) If the SaveState variable is ON, the editor's state is saved when this command begins execution. See Also: [Exit\(\)](#), [SaveAllFiles\(\)](#), [AbandonEditor\(\)](#) **Variables:** [SaveState](#)

SaveAllFiles

Saves all modified files; the current file and cursor location do not change.

Syntax: [INTEGER](#) [SaveAllFiles](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command is

commonly used prior to the [Dos\(\)](#) or [Shell\(\)](#) command to insure that modifications are not lost in the event a spawned process crashes the system. If successful, the modified flag for each file is set to FALSE. See Also: [SaveFile\(\)](#), [SaveAllAndExit\(\)](#), [Exit\(\)](#), [FileChanged\(\)](#)

SaveAndQuitFile

Saves the current file to disk, and then removes it from the editor.

Syntax: [INTEGER SaveAndQuitFile\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: The current file is saved regardless of its modified state. If only a single file is loaded when executing this command, further action is determined by the setting of the QuitToPrompt variable. If set ON, the editor prompts the user for another file. Escaping that prompt exits the editor to the operating system. If QuitToPrompt is set OFF, the editor immediately exits to the caller. **Examples:** To cause a file to be saved only if it is modified, the [QuitFile\(\)](#) command can be used. If no prompting is desired, the following can be used: [integer proc](#) SaveIfChangedAndQuitFile() [return](#)([iif](#)([FileChanged\(\)](#), [SaveAndQuitFile\(\)](#), [QuitFile\(\)](#))) [end](#) See Also: [AbandonFile\(\)](#), [QuitFile\(\)](#), [SaveFile\(\)](#), [SaveAs\(\)](#) **Variables:** [QuitToPrompt](#)

SaveAs

Saves the current buffer to disk under the specified name.

Syntax: [INTEGER SaveAs](#)([[STRING](#) filename [,[INTEGER](#) option]]) - filename is the optional filename to use. - option specifies how [SaveAs\(\)](#) works if a file of the same name already exists on disk. If [not](#) specified, [_DEFAULT_](#) is used. Supported [values](#) for option are: - [_DEFAULT_](#) prompts for whether to append the current file to the file on disk or to overwrite the file on disk with the current file. - [_OVERWRITE_](#) overwrites filename if it exists. - [_APPEND_](#) appends to filename if it exists. - [_DONT_PROMPT_](#) specifies that [SaveAs\(\)](#) should [not](#) display any warning or informational messages if a problem occurs. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If filename is not specified, the editor will prompt for that information. If the file already exists on disk, and option is [_DEFAULT_](#), the editor will confirm appending or overwriting. If a directory name or a wildcarded filename is specified, the editor will display a PickList of matching files. This command includes a history list of previous entries passed to it. See [AddHistoryStr\(\)](#) for additional information. **Examples:** Save a file to "foo.bar", overwriting any existing file by that name. The user is [not](#) prompted. [SaveAs](#)("foo.bar", [_OVERWRITE_](#)) Save a file to "big.file", appending to the file if it already exists. The user is [not](#) prompted. [SaveAs](#)("big.file", [_APPEND_](#)) Save a file to "my.file", overwriting any existing file by that name. If there is an error (for instance, "my.file" is READONLY), do not display a message, simply return FALSE. [SaveAs](#)("my.file", [_OVERWRITE_|_DONT_PROMPT_](#)) See Also: [SaveFile\(\)](#), [SaveBlock\(\)](#), [AddHistoryStr\(\)](#) **Variables:** [UseCommonDialogs](#)

SaveBlock

Saves a block on disk as a file of the specified name.

Syntax: [INTEGER SaveBlock](#)([[STRING](#) filename [,[INTEGER](#) option]]) - filename is the optional filename to use for the saved block. - option specifies how [SaveBlock\(\)](#) works if a file of the same name already exists on disk. If [not](#) specified, [_DEFAULT_](#) is used. Supported [values](#) for option are: - [_DEFAULT_](#) prompts for whether to append the block to the file on disk or to overwrite the file on disk with the block. - [_OVERWRITE_](#) overwrites filename if it

exists. - `_APPEND_` appends to filename if it exists. - `_DONT_PROMPT_` specifies that [SaveBlock\(\)](#) should not display any warning or informational messages if a problem occurs. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If a block does not exist in the current file, this command uses the entire file, in essence behaving exactly like [SaveAs\(\)](#). If filename is not specified, the editor will prompt for that information. If the file already exists on disk, and option is `_DEFAULT_`, the editor will prompt whether to overwrite or append to the existing file or to cancel the operation. If a directory name or a wildcarded filename is specified, the editor will display a PickList of matching files. This command includes a history list of previous entries passed to it. See [AddHistoryStr\(\)](#) for additional information. [Examples](#): Save a block to file "foo.bar", overwriting any existing file. The user is not prompted. [SaveBlock](#)("foo.bar", `_OVERWRITE_`) Save a block to a file, "big.file", appending to the file if it already exists. The user is not prompted. [SaveBlock](#)("big.file", `_APPEND_`) Save a block to a file, "my.file", overwriting any existing file by that name. If there is an error (for instance, "my.file" is READONLY), do not display a message, simply return FALSE. [SaveBlock](#)("my.file", `_OVERWRITE_|_DONT_PROMPT_`) See Also: [SaveFile\(\)](#), [SaveAs\(\)](#), [InsertFile\(\)](#), [AddHistoryStr\(\)](#) [Variables](#): [UseCommonDialogs](#)

SaveFile

Saves the current file to disk under its current name.

[Syntax](#): [INTEGER](#) [SaveFile](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Zero is returned if there was an error writing the current file to disk. If successful, the modified flag for the current file is set to [FALSE](#). See Also: [SaveAs\(\)](#), [SaveBlock\(\)](#), [QuitFile\(\)](#), [FileChanged\(\)](#)

SaveHistory

Saves the contents of the current History Buffer to a file.

[Syntax](#): [INTEGER](#) [SaveHistory](#)([\[STRING](#) fn]) - fn is the optional filename to use for the history file. If fn is not specified, the file TSEHIST.DAT is created in the editor [load directory](#). Returns: Non-zero if successful; zero ([FALSE](#)) on failure. See Also: [LoadHistory\(\)](#)

SaveKeyMacro

Writes the current Keyboard Macros to disk.

[Syntax](#): [INTEGER](#) [SaveKeyMacro](#)([\[STRING](#) keymac_filename]) - keymac_filename is an optional string that contains the name of the Keyboard Macro file to be saved. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If keymac_filename is specified, and does not contain an extension, ".kbd" is added. If keymac_filename is not specified, the user is prompted for the name of a file to store the Keyboard Macros to. The editor adds a file extension of ".kbd" if no extension is given. The scrap macro is not saved. This command includes a history list of previous entries passed to it. See [AddHistoryStr\(\)](#) for additional information. See Also: [LoadKeyMacro\(\)](#), [PurgeKeyMacro\(\)](#), [AddHistoryStr\(\)](#) [Variables](#): [KbdPath](#)

SaveSettings

Saves the current editor variable settings to the executable file.

Syntax: [INTEGER SaveSettings\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. See Also: [Set\(\)](#), [Query\(\)](#)

SaveState

Determines whether the

[Exit\(\)](#) and [SaveAllAndExit\(\)](#) commands save "state" information between editing sessions. **Values:** [INTEGER](#) - [OFF](#) - disables state-saving feature - [ON](#) - enables the state-saving feature, and saves state information in the editor load directory - [_LOCAL_](#) - enables the state-saving feature, and saves state information in the current directory **Default:** [OFF](#) **Limits:** The state is only saved automatically when the [Exit\(\)](#) or [SaveAllAndExit\(\)](#) command is invoked. **Notes:** The state is saved to a file named TSESTATE.DAT. The path used depends on the value of SaveState, as discussed above. State information that is saved includes the files (and corresponding file positions) in the editor file Ring, as well as [Bookmarks](#) and [windows](#). See Also: [PersistentRecentFiles](#), [PersistentHistory](#) **Commands:** [Exit\(\)](#), [SaveAllAndExit\(\)](#)

Saving and Closing Files, and Exiting from the Editor

Files that are changed during an editing session can be saved (written to disk) as necessary, either in the middle of a session or when a session is ended. Files that are no longer needed can be closed (or "quit"), releasing any memory used by those files. An editing session can be terminated by closing all files and exiting from the editor. A number of commands and options are available for saving and/or closing files, and for exiting from the editor.

[What Happens when a File Is Saved](#) ----- To save a copy of a file being edited, the contents of the memory buffer for that file must be written to disk. When a file is saved, a number of operations are performed by the editor, as described in the following paragraphs. An end-of-line string is placed at the end of each line in the file when it is saved. The standard setting for the line-termination string is a carriage return/linefeed pair. However, you can configure the editor to place only a carriage return or only a linefeed character at the end of each line. (For more information on configuring the end-of-line string, see "Line-Termination String" under "[System/File Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") If the original file contained lines longer than the maximum line length, those lines were split at the maximum length. When saved, the end-of-line string is added where each line was split. (If the file is loaded in Binary mode, no termination characters are added to the file. For more information on [Binary mode](#), see "[Alternative Modes for Viewing Files](#)" later in this chapter.) In the standard configuration of the editor, the line-termination string is also added to the LAST line of the file when it is saved. However, you can configure the editor to terminate the file (that is, the last line of the file) with only a Control-Z character, or with the end-of-line string AND a Control-Z character; or you can configure the editor so that it does not add ANY termination characters to the last line of the file. (For more information on configuring the end-of-file string, see "File-Termination String" under "[System/File Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") If the file in memory already exists on disk (that is, it is not a new file), the copy of the file in memory is initially written to disk under a temporary name. This temporary file is created in the same directory from which the file in memory was loaded. If the editor is successful in creating the temporary file, the original file on disk is then renamed by changing the extension to

".BAK", or to whatever backup extension [string](#) is configured. (If the same filename with a ".BAK" extension already exists on disk in the same directory, that older ".BAK" file is first erased.) This newly-created ".BAK" file provides a backup copy of the file as it was before it was last saved. The newly-written temporary file is then renamed to the original filename. The editor can be configured so that it does [not](#) create backups of files when they are saved. If so configured, once the editor has successfully written the file to disk under the temporary name, the original file on disk is erased and the newly-written temporary file is renamed to the original filename. Selecting "Backups" under the Options Menu allows you to indicate whether or [not](#) you want backups created when files are saved. Set this [ON](#) to have backups created; [otherwise](#), set this option [OFF](#). (For more information on suppressing the creation of backups or configuring the backup extension string, see "Create Backups" and "Backup Extension String" under "[System/File Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") You can also configure the editor to directly overwrite the original file on disk when a file is saved. In this case, the file is not written to a temporary filename and does not allow a backup file to be created. Although this is the fastest method for saving files, and prevents the editor from changing the order of files within a directory when files are saved to disk, it does not provide the safety of the normal method for saving files. (For more information on configuring this option, see "Protected Saves" under "[System/File Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") If the file in memory does not already exist on disk when saved, the copy in memory is simply written to disk under its designated name. No temporary or backup files are created. Once the file is saved, the Modified File Indicator ("*" preceding the filename) on [the StatusLine](#) is removed. NOTE: A file that has an attribute of "Read Only" can be loaded into the editor, and viewed or modified. You can even change the filename and save the file to that new name. However, you cannot save a "Read-Only" file to its original name. [Commands to Save and/or Close Files](#) ----- The following commands allow you to save the current file, without closing the file. * SaveFile (<Ctrl K S>, or "Save" under File [Menu](#)) This command saves the entire current file. The buffer contents for that file are written to disk, using the current name of the file. The file remains loaded in memory and included in the Ring. All or part of a file can also be saved using the SaveBlock command. (For more information, see "[Manipulating Text in a Block](#)" in the chapter on "[Block Operations](#).") * RenameAndSaveFile ("Save As" under File [Menu](#)) This command issues the prompt, "Rename and Save As:". The name of the current buffer is changed to the specified name, and then saved to disk. For more information, see the [RenameAndSaveFile\(\)](#) command. The editor's [SaveAs\(\)](#) command is available from the Block menu if "Write to File" is selected, when no block is currently marked. * SaveAllFiles ("Save All" under File [Menu](#)) This command saves all files in the current session that have been modified since last saved. The buffer contents of each modified file are written to disk, using the current name of each file. All modified files are automatically saved, without any prompting. The files remain loaded in memory and included in the Ring. The next group of commands allow you to quit files; that is, to close them and thereby remove them from the editor's memory. * QuitFile (<Ctrl K Q>, or "Close" under File [Menu](#)) This command prompts you to save the current file (if modified), and then closes (or "quits") the file. If the file was not saved since last modified, the following selection menu is given: + Save changes? + | *Yes***** | No | | Cancel | +-----+ Select "Yes" to first save the file, and then quit the file. Select "No" to quit the file without saving the changes. Select "Cancel" (or press <Escape>) to terminate this operation and continue editing the file. * SaveAndQuitFile (<Ctrl K X>, or "Save & Close" under File [Menu](#)) This command automatically saves the current file and then closes (or "quits") the file. The file is saved regardless of whether it was modified. +-----+ Tip -----+ | If only one file is open and the QuitFile or SaveAndQuitFile | | command is executed, the editor exits to the operating system (or | | the application that initially invoked the editor). Alternatively, | | the editor can be configured to display the "File(s) to edit:" | | prompt when the last (or only) file is closed using one of these | | commands. (For more information, see "EditFile Prompt on Quit" | | under "[Command/Format Options](#)" in the chapter on | | "[Configuring the Editor Interactively](#).") | +-----+ [Commands to Exit from the Editor](#) ----- The following commands allow you to quit (or close) all files and exit from the editor. * Exit (<Alt X>, or "Exit" under File [Menu](#)) This command quits (closes) all files that are open and exits the editor. The command operates by issuing the QuitFile command for each file in the Ring. As with QuitFile, you are prompted to save any files that were modified since they were last saved. If you select "Cancel" [or](#) press <Escape> at the "Save changes?" prompt for any file, the Exit command is terminated at that file, leaving that file open along with any other

files following it in the Ring. * [SaveAllAndExit](#) ("Save All & Exit" under File [Menu](#)) This command automatically saves all modified files, closes each file in the Ring, and exits the editor. No prompting to save files is given. Only those files that were modified are saved.

Saving and Restoring State Information

Many times, in order to accomplish a specific task, a macro may need to change the current editing state. Examples of this are changing to a different position in the current file,

[marking a block](#), or even changing to a different file or buffer. In order to be user friendly, [macros](#) should generally try to preserve the initially found states, and then restore them before the macro terminates. The editor provides commands to save and restore many of these different editing states. These commands [include](#):

- * [BinaryMode\(\)](#) Queries or sets binary or raw editing mode.
- * [BufferType\(\)](#) Queries or sets the current buffer's type.
- * [FileChanged\(\)](#) Queries or sets the current buffer's modified (changed) status.
- * [DisplayMode\(\)](#) Queries or sets the current display mode.
- * [GetBufferId\(\)](#) and [GotoBufferId\(\)](#) These two commands can be used to save the id of the current buffer, and make the buffer with the specified id the current buffer. [GotoBufferId\(\)](#) prevents `_ON_CHANGING_FILES_` hooks from being invoked, so it is sometimes the preferable method for switching back and forth between files within a macro.
- * [PushPosition\(\)](#), [PopPosition\(\)](#), and [KillPosition\(\)](#) These commands can be used to save the current position, and then to restore that position as needed. They are ideally used to mark a location when a macro needs to go to another location in the current file, do some processing, and then return back to the previously marked location in the file.
- * [PushBlock\(\)](#) and [PopBlock\(\)](#) These commands can be used to save the current block coordinates and status, and then restore the block status at a later time. They are ideally used when a macro needs to use a block in order to accomplish its task. With these commands, blocks that existed before the macro was invoked can be left undisturbed. An example illustrating use of these commands would be a macro to capitalize the word at the cursor position. This macro needs to save both the current location in the file, and the status of the marked block, since it changes positions and temporarily marks a new block.

```
proc CapitalizeWord()
  PushPosition\(\) // save current position
  PushBlock\(\) // save block coordinates/status if
  MarkWord\(\) // if cursor is on a word, mark it
  Lower\(\) // lower-case the word
  GotoBlockBegin\(\) // go to the first character
  InsertText\(Upper\(Chr\(CurrChar\(\)\)\), _OVERWRITE_)
end
PopBlock\(\) // restore block coordinates/status
PopPosition\(\) // restore position
end
```

State information can also affect the way certain commands operate. A macro may need to set a specific state, perform an operation, and then restore the state before terminating. Many [editor variables](#) are available in order to query and set these states. The states may be changed and/or set by using the [Query\(\)](#) and [Set\(\)](#) commands. Examples of these kinds of state information include the following editor variables:

- * [AutoIndent](#) Determines whether the [CReturn\(\)](#) command places the cursor at column one, or indents based on a previous line.
- * [CenterFinds](#) Determines whether the [Find\(\)](#) command changes the current cursor row.
- * [Cursor](#) Determines whether the cursor is displayed.
- * [EquateEnhancedKbd](#) Determines whether the duplicate grey keys are recognized as different from their keypad counterparts.
- * [ExpandTabs](#) Determines whether tabs are to be displayed as spaces or tab characters.
- * [Insert](#) Determines whether new text is to be inserted into the file, or to overwrite existing text.
- * [InsertLineBlocksAbove](#) Determines whether line blocks are to be added above or below the current [cursor](#) line. This [affects](#) [CopyBlock\(\)](#), [MoveBlock\(\)](#), and [Paste\(\)](#).
- * [KillMax](#) Determines whether the text deleted by the various delete commands is discarded or saved in the Deletion Buffer.
- * [LeftMargin](#) The left margin used by [WrapPara\(\)](#) and word wrapping.
- * [MakeBackups](#) Determines whether backup files are created when files are written to disk.
- * [Marking](#) Determines whether the current block is extended when the cursor position is changed.
- * [MaxHistorySize](#) Determines the number of entries that are stored in the history list.
- * [MsgLevel](#) Determines whether certain messages are displayed.
- * [ProtectedSaves](#) Determines whether the editor saves files in a safer (but slower) fashion, or in a faster (but more risky) fashion.
- * [RemoveTrailingWhite](#) Determines whether trailing spaces and tabs are removed when the cursor is moved from the current line.
- * [RightMargin](#) The right margin used by [WrapPara\(\)](#) and word wrapping.
- * [ShowStatusLine](#) Determines whether there is a visible StatusLine on the screen.

[StatusLineUpdating](#) Determines whether the editor updates [the StatusLine](#). * [TabType](#) The current tab type. * [TabWidth](#) The current tab width. * [UnMarkAfterPaste](#) Determines whether blocks pasted from the ClipBoard should be marked. * [UseCurrLineIfNoBlock](#) Determines whether the [Cut\(\)](#) and [Copy\(\)](#) commands use the current line if no block is marked. * [WordSet](#) The characters recognized by word-oriented commands ([WordRight\(\)](#), [WordLeft\(\)](#), [DelRightWord\(\)](#), and others). * [WordWrap](#) A mode in which lines wrap as the cursor goes past the RightMargin setting. Generally, if a macro changes an editor variable, it should first save the current value, and then restore it upon macro termination.

Saving Configuration Settings

Any changes that you make to editing options, either from the pull-down Menu or within IConfig, are automatically applied to the current session (except for a few indicated options that take effect only when the editor is restarted). To permanently apply your configuration changes to the editor, select "Save Current Settings" under the Options Menu. This installs in the editor the current state of all option settings.

Saving Files and Exiting from the Editor

File commands are provided that allow you to close individual files that are no longer needed during an editing session, and to save files that have been modified.

Once you are ready to end an editing session, you exit from the editor by

[terminating the editor](#) program. When you exit, all opened files are closed, and any modified files can optionally be saved. The easiest way to exit with an option to save your work is to execute the Exit command. To execute Exit, select "Exit" under File on the pull-down [Menu](#), or press <Alt X>. Each open file is closed, and for any file that has been modified, the editor asks whether to save the changes for that file. The editor then terminates. You can also exit by instructing the editor to automatically save all changed files. To do this, select "Save All & Exit" under File on the pull-down Menu. Without any prompting, the editor saves all modified files, and immediately terminates. If desired, you can instruct the editor to save information about the current editing session so that it can be restored in a future session. (For more information on [saving files and exiting from the editor](#), see the chapter on "[Loading, Processing, and Saving Files](#).")

Saving Print Options For Future Sessions

If desired, you can update the editor executable program with your print output settings to save them for subsequent editing sessions. To do so, select "Save Current Settings" under the Options Menu. The editor program is immediately updated with the current settings of all configurable editing options (including non-printing options). The next time the editor is invoked, the option settings that you saved will be in effect.

Saving/Restoring the Current Editing Session

After you have exited from the editor (thereby ending the editing session),

you may later wish to resume editing and "pick up where you left off." To make this possible, the editor gives you the option to save certain information and settings from the current editing session so that they can be restored for the next editing session. Three separate options are available: you can elect to save and restore the entries in the History Buffer; you can elect to save and restore the entries in the Recent Files list; and/or you can elect to save certain editing environment information so that it can be restored. You can choose to take advantage of any or all of the available options.

[Saving/Restoring Entries in the History Buffer](#) ----- If you want the editor to save the entries in the prompt-box History Buffer from one editing session to the next, set the PersistentHistory variable ON. (For more information on the History Buffer, see "[History Buffer for Prompt Boxes](#)" in the chapter on "[Command Prompts and PickLists](#)." For more information on setting the [PersistentHistory](#) variable, see "Persistent History" under "[Command/Format Options](#)" in the chapter on "[Configuring the Editor Interactively](#)." If the PersistentHistory variable is ON when you exit from the editor, the prompt-box entries in the current History Buffer will be saved in a file called "TSEHIST.DAT". (This file is maintained in the editor load directory.) The next time you start an editing session, the editor will automatically load the entries from TSEHIST.DAT into the History Buffer for the new editing session. Saving History Buffer entries across editing sessions can cause the number of entries in the History Buffer to potentially grow very large. To control the number of entries that are maintained in the History Buffer, two variables are available: [MaxHistorySize](#) and [MaxHistoryPerList](#). [MaxHistorySize](#) allows you to set the maximum number of History entries that are maintained, in total, for ALL prompts. MaxHistoryPerList allows you to set the maximum number of History entries that are maintained for each particular command (or command group) prompt (that is, for each History "list"). (For more information on setting these [variables](#), see "Max History Entries" [and](#) "Max History Per List" under "[Command/Format Options](#)" in the chapter on "[Configuring the Editor Interactively](#)." [Saving/Restoring Entries in the Recent Files List](#) ----- To cause the editor to save the entries in the

Recent Files list, set the [PersistentRecentFiles](#) variable [ON](#). The maximum number of files that are maintained in the Recent Files list is controlled by the MaxRecentFiles variable. (For more information on setting the PersistentRecentFiles and [MaxRecentFiles](#) [variables](#), see "Persistent Recent Files" [and](#) "Max Recent Files" under "[Command/Format Options](#)" in the chapter on "[Configuring the Editor Interactively](#)." If the PersistentRecentFiles variable is ON when you exit from the editor, the filename entries in the Recent Files list will be saved in a file called "TSEFILES.DAT". (This file is maintained in the editor load directory.) The next time you start an editing session, the editor will automatically load the entries from TSEFILES.DAT into the Recent Files list for the new editing session.

[Saving/Restoring the Editing Environment](#) ----- You can instruct the editor to save additional information about the current editing environment ("state") so that it can be readily restored for a subsequent editing session. The editing environment that is restored from a previous session includes the file(s) that were loaded and the [window](#)(s) and file(s) that were displayed on the screen; the position of the cursor at the time the session was ended; the block (if any) that was marked; the bookmarks that were placed; and the settings of selected editing options (such as those on the "Options" Menu). With this feature, you can basically end an editing session, and then return to it at a later time, just as you left it. To take advantage of this capability, the SaveState variable must be set to ON or LOCAL for the session for which you want to save environment information. When you are ready to terminate that editing session and exit from the editor, execute either the Exit (<Alt X>) command or the SaveAllAndExit command. This causes the editor to save information about the current editing environment. Then when you start the next editing session, include the "[Restore State](#)" (-r) command-line option: e32 -r This will restore the editing session to what it was when you last exited the editor with the SaveState variable in effect. (No other editor options or filenames that you enter will be processed when you start the editor using the "[Restore State](#)" option. For more information on specifying the "Restore State" command-line option, see "[Command-Line Options](#)" in the chapter on "[Starting the Editor](#)." If desired, you can configure the editor to automatically restore the previously-saved editing environment each time you

start the editor without specifying any command-line parameters (that is, when you do [not include](#) any filenames or editing options on the command line). (For more information, see "[Starting the Editor with No Command-Line Parameters](#)" in the chapter on "[Starting the Editor](#).") To set the SaveState variable, select "Save State on Exit" under the Options Menu. If the SaveState variable is ON or LOCAL when you exit from the editor (using either the Exit or SaveAllAndExit command), the editing environment information will be saved in a file called "TSESTATE.DAT". If the variable is set to ON, the "TSESTATE.DAT" file is saved in the editor load directory. If set to LOCAL, the "TSESTATE.DAT" file is saved in the current directory. (If you save environment information using the LOCAL option, you must re-start the editor from the same directory in order to restore that environment.) In addition to saving/restoring the environment when you end/begin an editing session, you can also save and restore the editing environment interactively without leaving the editor. For more information on interactively saving and restoring the editing environment, see the State macro under "[Executing Supplemental Macros from the Potpourri](#)" in the chapter on "[Special Commands and Features](#)."

ScreenCols

The current number of columns on the screen.

[Values](#): [INTEGER](#) [Default](#): Undefined Limits: Can only be queried. Cannot be used within a config/endconfig construct. Notes: ScreenCols returns the current number of columns on the screen. See Also: [ScreenRows](#), [WindowCols](#), [WindowRows](#), [WindowX1](#), [WindowY1](#), [PopWinCols](#), [PopWinRows](#)

ScreenRows

The current number of rows on the screen.

[Values](#): [INTEGER](#) [Default](#): Undefined Limits: Can only be queried. Cannot be used within a config/endconfig construct. Notes: ScreenRows returns the current number of rows on the screen. See Also: [ScreenCols](#), [WindowRows](#), [WindowCols](#), [WindowX1](#), [WindowY1](#), [PopWinCols](#), [PopWinRows](#)

ScrollDown

Scrolls the text one line at a time toward the end of the file.

[Syntax](#): [INTEGER](#) [ScrollDown](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: The cursor remains on the same line of text until it reaches the top of the window. See Also: [Up](#)(), [Down](#)(), [RollUp](#)(), [RollDown](#)(), [ScrollUp](#)(), [ScrollToRow](#)(), [ScrollToCenter](#)(), [ScrollToTop](#)()

Scrolling Text

These commands scroll the text in the current window. Many of them move just the text, and leave the cursor position unchanged relative to the screen.

[Scrolling Forward or Backward in a File](#) ----- These commands allow you to scroll the text

forward, backward, left, or right. * **PageDown** (<PgDn>) This command scrolls down a page in the current file. The cursor does not change its position relative to the screen. The amount of text scrolled is one line less than the number of rows in the current window. This causes the last line in the window to be the first line displayed when the text is scrolled. If you execute PageDown when the last line of the file is displayed in the window, then the cursor moves to the last line of the file without scrolling the text. If the cursor is already positioned on the last line of the file when you execute PageDown, then the last line of the file scrolls to the top of the window, and the cursor moves with the line. * **PageUp** (<PgUp>) This command scrolls up a page in the current file. The cursor does not change its position relative to the screen. The amount of text scrolled is one line less than the number of rows in the current window. This causes the first line in the window to be the last line displayed when the text is scrolled. If you execute PageUp when the first line of the file is displayed in the window, the cursor moves to the first line of the file without scrolling the text. * **RollDown** (<Ctrl CursorDown>) This command scrolls down one line in the current file. The cursor does not change its position relative to the screen, so that the line following the current line becomes the new current line. (For a similar command, see ScrollDown below.) If you execute RollDown with the cursor on the last line of the file, no action occurs. * **RollUp** (<Ctrl CursorUp>) This command scrolls up one line in the current file. The cursor does not change its position relative to the screen, so that the line above the current line becomes the new current line. (For a similar command, see ScrollUp below.) If you execute RollUp with the first line of the file at the top of the current window, no action occurs. * **RollLeft** (<Alt F5>) This command scrolls left in the text by one column. The cursor does not change its position relative to the screen. (For a similar command, see ScrollLeft below.) If you execute RollLeft with column 1 at the left edge of the current window, no action occurs. * **RollRight** (<Alt F6>) This command scrolls right in the text by one column. The cursor does not change its position relative to the screen. (For a similar command, see ScrollRight below.) If you execute RollRight with the maximum line length column at the right edge of the current window, no action occurs. * **ScrollDown** (<Ctrl Z>) Like RollDown, this command scrolls down one line in the current file. However, the cursor remains on the same line of text, moving the cursor line upward as ScrollDown is executed, until it becomes the top line of the window. * **ScrollUp** (<Ctrl W>) Like RollUp, this command scrolls up one line in the current file. However, the cursor remains on the same line of text, moving the cursor line downward as ScrollUp is executed, until it becomes the bottom line of the window. If you execute ScrollUp with the first line of the file at the top of the current window, no action occurs. * **ScrollLeft** Like RollLeft, this command scrolls left in the text by one column. However, the cursor remains on the same character in the text, moving the cursor position to the right as ScrollLeft is executed, until it reaches the right edge of the window. If you execute ScrollLeft with column 1 at the left edge of the current window, no action occurs. * **ScrollRight** Like RollRight, this command scrolls right in the text by one column. However, the cursor remains on the same character in the text, moving the cursor position to the right as ScrollRight is executed, until it reaches the left edge of the window. If you execute ScrollRight with the maximum line length column at the right edge of the current window, no action occurs. [Scrolling the Current Line](#)

----- These commands scroll the text and move the cursor to re-position the current line. *

ScrollToCenter (<Shift F5>) This command scrolls the text either up or down, as necessary, so that the current line is positioned at the center of the window. The column position of the cursor does not change. * **ScrollToTop** (<F5>) This command scrolls the text up so that the current line is positioned at the top of the window. The column position of the cursor does not change.

ScrollLeft

Scrolls the view of text on the screen one position to the left, within the current window, while the cursor remains on the same character.

Syntax: [INTEGER ScrollLeft\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: The cursor character does not change unless the cursor reaches the edge of the screen. See Also: [ScrollRight\(\)](#), [RollLeft\(\)](#), [RollRight\(\)](#)

ScrollRight

Scrolls the view of text on the screen one position to the right, within the

current window, while the cursor remains on the same character.

Syntax: [INTEGER ScrollRight\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: The cursor character does not change unless the cursor reaches the edge of the screen. See Also: [ScrollLeft\(\)](#), [RollLeft\(\)](#), [RollRight\(\)](#)

ScrollToBottom

Moves the current line to the bottom row of the window by moving the file within the current window, without changing the current line number.

Syntax: [INTEGER ScrollToBottom\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command scrolls the current line, without the cursor leaving that line. See Also: [GotoRow\(\)](#), [CurrRow\(\)](#), [ScrollToRow\(\)](#), [ScrollToCenter\(\)](#), [ScrollToTop\(\)](#)

ScrollToCenter

Moves the current line to the center row of the window by moving the file within the current window, without changing the current line number.

Syntax: [INTEGER ScrollToCenter\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command scrolls the current line, without the cursor leaving that line. See Also: [GotoRow\(\)](#), [CurrRow\(\)](#), [ScrollToBottom\(\)](#), [ScrollToRow\(\)](#), [ScrollToTop\(\)](#)

ScrollToRow

Moves the current line to the indicated window row by moving the file within the current window, without changing the current line number.

Syntax: [INTEGER ScrollToRow\(INTEGER y\)](#) - y is the row number to which to scroll. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command scrolls the current line, without the cursor leaving that line.

Examples: `***** Scrolls the screen, so that the current line becomes the first line in the current window.`

`*****/ ScrollToRow(1) /`

`***** Scrolls the screen, so that the current line becomes the last line in the current window. *****/`

`ScrollToRow(Query(WindowRows)) /***** Scrolls the screen, so that the current line moves to the center of the current window.`

`*****/ ScrollToRow(Query(WindowRows)/2) See`

Also: [GotoRow\(\)](#), [CurrRow\(\)](#), [ScrollToBottom\(\)](#), [ScrollToCenter\(\)](#), [ScrollToTop\(\)](#)

ScrollToTop

Moves the current line to the top row of the window by moving the file within

the current window, without changing the current line number.

Syntax: [INTEGER ScrollToTop\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command scrolls the current line, without the cursor leaving that line. See Also: [GotoRow\(\)](#), [CurrRow\(\)](#), [ScrollToBottom\(\)](#), [ScrollToRow\(\)](#), [ScrollToCenter\(\)](#)

ScrollUp

Scrolls the text one line at a time toward the beginning of the file.

Syntax: [INTEGER ScrollUp\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: The cursor remains on the same line of text until it reaches the bottom of the screen. If the screen cannot be scrolled, zero is returned. See Also: [Up\(\)](#), [Down\(\)](#), [RollUp\(\)](#), [RollDown\(\)](#), [ScrollDown\(\)](#), [ScrollToRow\(\)](#), [ScrollToCenter\(\)](#), [ScrollToTop\(\)](#)

Search Features: Finding and Replacing Text

A number of search options are available within the editor to allow you to Find a string of text in a file, and to Replace one string with another. A search string can be an exact string of characters to be located; or it can be an ambiguous pattern of characters, using wildcard characters and other special symbols to create a Regular Expression. A search can move forward or backward from the current position; can be limited to the current file or to a block marked within the current file, or can span all files in the Ring; can be located at any position on a line, or tied to the beginning or the end of a line; can be case-sensitive or case-insensitive; can be identified as a "word" so that it will match only against whole words in the text; and can be set to display a PickList of only those lines that contain a specified string.

Additional search features included are: IncrementalSearch, to progressively narrow a search as a string is entered one character at a time; CompressView and FunctionList, to list all occurrences of selected text

[strings](#); RedisplayViewPickList, to redisplay the list of occurrences of a specified string; FindAndDo, to find a string and perform one of a group of pre-defined operations; FindWordAtCursor, to find the word at the cursor position; and Match, to locate a matching character for a select group of bracketing pairs. This chapter discusses: - [Basic Find and Replace options](#) (Find, Replace, Again, AgainReverse) - [Searching with Regular Expressions](#) - [IncrementalSearch feature](#) (IncrementalSearch) - [Listing all occurrences of a specified text string](#) (CompressView, FunctionList, RedisplayViewPickList) - [Other special-purpose search features](#) (FindAndDo, FindWordAtCursor, Match)

SearchHelp

Allows searching of

[the HelpSystem](#) for a particular string. **Syntax:** [SearchHelp\(\)](#) Returns: Nothing. See Also: [Help\(\)](#)

Searching

The following three commands are included specifically for search operations within

[macros](#): [IFind\(\)](#), [IReplace\(\)](#), and [IRepeatFind\(\)](#). These commands do not use or disturb any options set by the user-callable commands, [Find\(\)](#), [Replace\(\)](#) and [RepeatFind\(\)](#). Also, they do [not](#) prompt for parameters. Any required parameters must be fully specified in the call to the command. The use of these "low-level" commands avoids unnecessary screen updating and user prompts, thus giving the operation of the macro a more polished appearance.

Searching With Regular Expressions

In addition to searching for a simple, precise string (or pattern) of characters, the editor also allows you to indicate a complex or ambiguous search pattern, called a Regular Expression. The [X] (regular eXpression) search option allows you to define a Regular Expression search pattern.

By using Regular

[Expressions](#), you could, for example, find all occurrences of words in a file beginning with "b" or "B", and ending with "ing" (such as being, Beginning, bring). Or you could change a group of names listed as first name followed by last name (example: Kari Hood), to a list composed of last name followed by first name, and separated by a comma (example: Hood, Kari). Regular [Expressions](#) are composed by combining simple character patterns with special operators to create a complex search pattern. Operators in Regular Expressions allow you to: limit a match to specific characters or a Class of characters; broaden a match to any character; optionally match a pattern; indicate a repeated pattern; and specify alternate patterns to match. Matches across line boundaries, however, are [not](#) supported. The following list identifies and describes the operation of the operator symbols used to create Regular [Expressions](#). (A "Summary List of Regular Expression Operators" is provided at the end of this section.) [Regular Expression Operators](#)

Symbol	Regular Expression	Operation
.		In a search pattern, matches any single character. (This does not match the end-of-line position.) Example: Search pattern: wh.t matches a string beginning with the letters wh, followed by any single character, followed by the letter t (such as, what or whet or wh t (all on one line), but NOT wht or wheat)
^		In a search pattern, anchors the search for the sub-pattern that follows, to the beginning of the line (column 1); or, if a block is marked and the [L] (Local) option is specified, anchors the search to the beginning column of the block on a line. Example: Search pattern: ^This matches the string This beginning in column 1
\$		In a search pattern, anchors the search for the preceding sub-pattern to the end of the line; or, if a block is marked and the [L] option is specified, anchors the search to the ending column of the block or to the end of the line, whichever comes first. Example: Search pattern: that\$ matches the string that occurring as the final text on a line
		"Or" operator: in a search pattern, matches the preceding or the following sub-pattern. Example: Search pattern: licens ce matches the strings license or licence The order of precedence in searching for the indicated patterns is the order in which they are listed in the search string. Thus, if one shorter pattern (such as what) is included in another longer "or" pattern (such as whatever), the longer pattern should be listed first. Example: Search pattern: {whatever} {what} matches the strings whatever and what, and locates occurrences of either string in the text (Use of { } symbols is explained later in this section.) HOWEVER: Search pattern: {what} {whatever} never locates the full string whatever, since the search is immediately satisfied by what
?		In a search pattern, optionally matches the preceding sub-pattern. Example: Search pattern: colour?r matches the strings color or colour
[]		In a search pattern, identifies a Class of characters against which to match a single character. Within a Class, the case of characters is significant. (The [I] Ignore-case search option does not apply to a Class.) If upper-case and lower-case characters are to be included in a Class, both cases must be specified. Example: Search pattern: [123aAbC] matches any one of the following single characters: 1 2 3 a A b C
[-]		In a search pattern, indicates a range of characters (based on ASCII sequence) when used BETWEEN characters in a Class. The "-" symbol has no special meaning when it occurs as the first or last character within the "[]" Class notation, or when used outside the Class notation. In such cases, it is treated as the dash ("-")

character. Example: Search pattern: [0-9abc-] matches any one of the following single characters: 0 1 2 3 4 5 6 7 8 9 a b c - [~] In a search pattern, identifies a complement Class of characters to match against a single character, when "~" is used as the first character within the Class notation (immediately following the "[" symbol). It matches against the characters that ARE NOT in the specified Class of characters. The "~" symbol has no special meaning when it DOES NOT occur as the FIRST character within the Class notation, or when used outside the Class notation. In such cases, it is treated as the tilde ("~") character. Example: Search pattern: [~0-2a=] matches any single character OTHER than: 0 1 2 a = Example: Search pattern: [~] (This is ~ followed by a space.) matches any single character OTHER than a space character *

In a search pattern, matches 0 or more occurrences of the preceding sub-pattern, with [minimum closure](#). (See "[Minimum/Maximum Closure](#)" below). Example: Search pattern: ba*c matches a string beginning with the letter b, followed by zero or more occurrences of the letter a, followed by the letter c (such as, bc, bac or baac) Example: Search pattern: wh.*t matches a string beginning with the letters wh, followed by zero or more occurrences of ANY character, followed by the letter t (such as, wh t (all on one line), wht, what or wheat) + In a search pattern, matches 1 or more occurrences of the preceding sub-pattern, with [minimum closure](#). (See "[Minimum/Maximum Closure](#)" below). Example: Search pattern: ba+c matches a string beginning with the letter b, followed by one or more occurrences of the letter a, followed by the letter c (such as, bac or baac, but **NOT** bc) @ In a search pattern, matches 0 or more occurrences of the preceding sub-pattern, with [maximum closure](#). (See "[Minimum/Maximum Closure](#)" below). Example: Search pattern: ba@c matches a string beginning with the letter b, followed by zero or more occurrences of the letter a, followed by the letter c (such as, bc, bac or baac) # In a search pattern, matches 1 or more occurrences of the preceding sub-pattern, with [maximum closure](#). (See "[Minimum/Maximum Closure](#)" below). Example: Search pattern: ba#c matches a string beginning with the letter b, followed by one or more occurrences of the letter a, followed by the letter c (such as, bac or baac, but **NOT** bc) Example (to find [identifiers](#) in many programming languages): Search pattern: [a-zA-Z0-9_]* matches any sequence of one or more characters in the Class consisting of all lower and upper [case](#) letters, all [numbers](#), and the underscore. { } In a search pattern, serves as a Tag to identify a sub-pattern within the full search pattern. Tagged patterns can be nested. Tags are used to define a group of characters as a sub-pattern so that an operator acts on more than one character or character Class. Tags are also used to identify a sub-pattern within a Regular Expression so the sub-pattern can be separately referenced in a subsequent replacement. Tagged sub-patterns are implicitly numbered from 1 through 9 based on the leftmost "{" symbol. The sub-pattern number can be used within a replacement string to reference a tagged sub-pattern, using the following format: \n where "n" is the actual sub-pattern number from 1 - 9 that represents the appropriate tagged sub-pattern. To identify the FULL search pattern, "n" is "0" (that is, \0). Example (defining groups of characters as sub-patterns): Search pattern: {Begin}|{End}File matches either of the strings BeginFile or EndFile NOTE (without the Tags): Search pattern: Begin|EndFile matches the strings BeginndFile or BegiEndFile Example (identifying sub-patterns for replacement): Search pattern: {flip}{flop}s Replace pattern: \2\1s changes the string flipflops to flopflips Example (identifying sub-patterns for replacement): Rearrange the following list into last name, first name: (1) Sammy Mitchell (8) Steve Watkins (15) Kevin Carr Search pattern: {[0-9]}+{[a-zA-Z]*}{[a-zA-Z]*} Replace pattern: \1\3, \2 changes the list to: (1) Mitchell, Sammy (8) Watkins, Steve (15) Carr, Kevin Example (identifying the full search pattern): Search pattern: ^.*\$ (any character(s) on entire line) Replace pattern: "\"0\" \" encloses entire line in quotes" \ In a search or replace pattern, serves as an Escape operator to override a Regular Expression operator so the operator is treated as a literal character. For a Regular Expression operator to be treated as a literal character in a search or replace pattern, precede the operator symbol with the Escape "\" symbol. Note that in a replace pattern, the only symbol recognized as a Regular Expression operator is the Escape symbol itself (as described in the following paragraph). In a search or replace pattern, the "\" symbol is also used with certain other letters and [numbers](#) to indicate specific characters or values, such as a formfeed character or a hexadecimal value (as listed below). Further, in a replace pattern, the "\" symbol is used to reference a Tagged sub-pattern (see the explanation of "{" }" (Tags) above). The following examples show use of the "\" symbol as an Escape operator. Example: Search pattern: abc** matches the string abc** Example: Search pattern: abc* matches the string abc* Example: Search pattern: abc* Replace pattern: \\abc* changes the string abc* to \abc* Note that within the "[]" Class notation in a search pattern, the only symbols that are recognized as operators are "\", "-", "~", and "]" (and then only when placed as indicated in their descriptions). All other operators can be used literally in a Class notation without the need for the Escape operator. Thus, to find the Class of characters consisting of the question mark, the comma, the dollar sign, and the dash, the Class can be designated as "[?,\$-]" and DOES **NOT** have to be designated as "[\\?\\\$\\-]". (Note in this example that the

dash has not been placed BETWEEN characters within the Class notation.) The following notations, each beginning with the "\" symbol, are used to indicate certain control characters or to identify a character by its numeric value. \a In a search or replace pattern, represents the alert (beep) character (^G or ASCII 7). Example: Search pattern: \axyz matches the string xyz following an alert character \b In a search or replace pattern, represents the backspace character (^H or ASCII 8). \c In a search pattern, designates the placement of the cursor in the located string when used with the Find command. By default, the cursor is positioned on the first character of the located string. However, if "\c" is used, the cursor is positioned on the character immediately following this operator. If multiple "\c" operators are included, the last one is used. This operator has no effect when used with the Replace command. Example: Search pattern (with Find): Hello \cWorld! matches the string Hello World! and positions the cursor on the character "W" \f In a search or replace pattern, represents the formfeed character (^L or ASCII 12). \n In a search pattern, represents the newline (line feed) character (^J or ASCII 10). This operator is intended for use in [Binary mode](#). It locates only imbedded newline characters; it does not represent the end-of-line position, nor does it cause a search to span lines. In a replace pattern, \n causes the line to be split (as if [SplitLine\(\)](#) was invoked) at the point in the replace string where \n is specified. \r In a search or replace pattern, represents the return character (^M or ASCII 13). This operator is intended for use in [Binary mode](#). It locates only imbedded return characters; it does not represent the end-of-line position, nor does it cause a search to span lines. \t In a search or replace pattern, represents the tab character (^I or ASCII 9). \v In a search or replace pattern, represents the vertical tab character (^K or ASCII 11). \xnn In a search or replace pattern, represents the character that is equivalent to the indicated hexadecimal value, where "nn" is a value from 00 through FF that must be specified as a 2-digit number. Example: Search pattern: ^\x40 matches the string @ located at the beginning of the line \dnnn In a search or replace pattern, represents the character that is equivalent to the indicated decimal value, where "nnn" is a value from 000 through 255 that must be specified as a 3-digit number. Example: Search pattern: \d064\$ matches the string @ located at the end of the line \onnn In a search or replace pattern, represents the character that is equivalent to the indicated octal value, where "nnn" is a value from 000 through 377 that must be specified as a 3-digit number. Example: Search pattern: \xyz\o100abc matches the string xyz@abc

[Minimum/Maximum Closure](#)

----- Within the editor's Regular Expression search feature, you can indicate that you want a search to be satisfied either by zero or more, or by one or more, occurrences of a pattern, based on minimum or maximum closure. Minimum closure is achieved as soon as a string is located which contains the minimum set of characters that match a specified search pattern. Maximum closure is not achieved until the maximum set of characters is located that matches a specified search pattern. To satisfy either minimum or maximum closure, the editor searches for a matching string that is entirely contained on a single line. The symbols "*" (zero or more) and "+" (1 or more) are [minimum closure](#) operators; "@" (0 or more) and "#" (1 or more) are maximum closure operators. To illustrate the difference between minimum and maximum closure in a search, consider the following: Given the text: This_is_the_issue. then: Search pattern: Thi.*is ([minimum closure](#), 0 or more occurrences) matches the string This_is and: Search pattern: Th.*is ([minimum closure](#), 0 or more occurrences) matches the string This HOWEVER: Search pattern: Thi.@is ([maximum closure](#), 0 or more occurrences) matches the string This_is_the_is and: Search pattern: Th.@is ([maximum closure](#), 0 or more occurrences) also matches the string This_is_the_is Note that if the LAST character of a Regular Expression is a MINIMUM closure operator, then the number of characters matched will be 0 characters for the "*" operator (0 or more occurrences), and 1 character for the "+" operator (1 or more occurrences). Typically, you would not use either "*" or "+" as the LAST character of a Regular Expression, since the same result can be achieved without the use of these operators. However, you may encounter situations where it is useful to include the MAXIMUM closure operators ("@" and "#") as the last character of a Regular Expression. For example, consider the following: Given the text: abbbxyz then: Search pattern: ab* ([minimum closure](#), 0 or more occurrences) matches the string a (because zero occurrences of b satisfies the minimum); thus, you can achieve the same result with the simple Search pattern: a and: Search pattern: ab+ ([minimum closure](#), 1 or more occurrences) matches the string ab (and not abbb, because one occurrence of b satisfies the minimum designated number of occurrences); thus, you can achieve the same result with the simple Search pattern: ab HOWEVER: Search patterns: ab@ or ab# ([maximum closure](#)) both match the string abbb (because either search is satisfied only by the maximum number of occurrences of b) ALSO NOTE: Search patterns: ab*x or ab+x or ab@x or ab#x ALL match the string abbbx Here is another example distinguishing minimum and [maximum closure](#): Given the text: They suggested we start singing something while sightseeing. then: Search pattern: s[a-z]*g ([minimum closure](#), 0 or more occurrences) matches the strings, sug, sing, something, sig, and seeing, as

indicated in bold in the following: They suggested we start singing something while sightseeing. **HOWEVER:** Search pattern: s[a-z]@g ([maximum closure](#), 0 [or](#) more occurrences) matches the strings, sugg, singing, something, and sightseeing, as indicated in bold by the following: They suggested we start singing something while sightseeing. [Additional Regular Expression Examples](#) ----- Following are additional examples illustrating the use of Regular Expressions. - To find any string enclosed in double quotes (""), use the following search pattern: "\".*" Use the following variation of this to position the cursor on the last character of the located string (so if you search Again, the search will begin immediately AFTER the quote at the end of the located string): "\".*\c" - To find any [string](#) enclosed in single (") OR double (") quotes, use the following search pattern: {\'.*\}|{\'.*\} - To find any white (tab or space) character, use either of the following search patterns: [\x09] [\t] - To find any non-white character, use either of the following search patterns: [~\x09] [~\t] - To find a blank line, use the following search pattern: ^\$ - To find blank lines, or lines that contain only [white characters](#), use either of the following search patterns: ^[\x09]@\$ ^[\t]@\$ [Summary List of Regular Expression Operators](#) ----- Symbol Regular Expression Operation _____

. Search: matches any single character (except end-of-line) ^ Search: anchors to beginning of line (or block) \$ Search: anchors to end of line (or block) | Search "[Or](#)" operator: matches preceding or following sub-pattern ? Search: optionally matches preceding sub-pattern [] Search: identifies a Class of characters [-] Search: indicates a range of characters in a Class [~] Search: identifies a complement Class * Search: matches 0 [or](#) more occurrences of preceding sub-pattern ([minimum closure](#)) + Search: matches 1 [or](#) more occurrences of preceding sub-pattern ([minimum closure](#)) @ Search: matches 0 [or](#) more occurrences of preceding sub-pattern ([maximum closure](#)) # Search: matches 1 [or](#) more occurrences of preceding sub-pattern ([maximum closure](#)) { } Search: Tags a sub-pattern \0..\9 Replace: references a Tagged search sub-pattern \ Search/replace: Escape operator (overrides Regular Expression operators) \a Search/replace: represents alert ([beep](#)) character (^G or ASCII 7) \b Search/replace: represents backspace character (^H or ASCII 8) \c Search: positions [cursor](#) within located [string](#) (with Find command) \f Search/replace: represents formfeed character (^L or ASCII 12) \n Search/replace: represents newline (line feed) character (^J or ASCII 10) if used in a "Search for:" specification. Causes a [SplitLine\(\)](#) operation to occur (at the specified position) if used in a "Replace with:" specification. \r Search/replace: represents [return](#) character (^M or ASCII 13) \t Search/replace: represents tab character (^I or ASCII 9) \v Search/replace: represents vertical tab character (^K or ASCII 11) \xnn Search/replace: represents hexadecimal value of equivalent character \dnnn Search/replace: represents decimal value of equivalent character \onnn Search/replace: represents octal value of equivalent character

SearchPath

Searches for a file along a specified path.

Syntax: [STRING](#) SearchPath([STRING](#) fn, [STRING](#) path [, [STRING](#) dir]) - fn is the name of the file for which to search. - path is the path(s) in which to search for fn. - dir is an optional directory to append to each search path. Returns: The fully-qualified name of the file, if found; otherwise, an empty [string](#). Notes: [SearchPath\(\)](#) searches for the file, using path. If dir is passed, [SearchPath\(\)](#) does additional [searching](#), including: - It first searches in the current directory. - After a path is searched, it adds dir to the path and searches it again. - After all paths have been searched, the editor load directory is searched, both without and with dir appended. The editor uses [SearchPath\(\)](#) internally in conjunction with TSEPath to load Compiled Macro files; and to find Help files, the spelling checker word lists, the potpourri data file, and other files. Note that if fn starts with a drive (d:) or slash, [SearchPath\(\)](#) limits the search to fn as passed, and simply returns the results of calling [ExpandPath](#)(fn). To cause [SearchPath\(\)](#) to search the current directory and the editor [load directory](#), but [not](#) add dir to each path searched, use "." as the optional dir. **Examples:** // To search for a file along the operating system PATH: [string](#) fn[80] fn = [SearchPath](#)("foo", [GetEnvStr](#)("PATH")) // Same as the above, but also include the current // directory and the editor load directory fn = [SearchPath](#)("bar", [GetEnvStr](#)("PATH"), ".") /***** Search for a macro file the same

way the editor does: This example will search for "my.mac" in the following places, assuming: [TSEPath](#) == "c:\foo1;d:\foo2\bar" [LoadDir](#)() = "e:\TSEPro" search for "my.mac" in the current directory search for "c:\foo1\my.mac" search for "c:\foo1\mac\my.mac" search for "d:\foo2\bar\my.mac" search for "d:\foo2\bar\mac\my.mac" search for "e:\TSEPro\my.mac" search for "e:\TSEPro\mac\my.mac" *****/
 fn = [SearchPath](#)("my.mac", [Query](#)([TSEPath](#)), "mac") See Also: [ExpandPath](#)(), [LoadDir](#)(), [GetEnvStr](#)(), [SplitPath](#)()
[Variables](#): [TSEPath](#)

Seg

Returns the segment address of a specified variable.

This command is intended for ADVANCED USAGE.

Syntax: [INTEGER Seg](#)([VAR INTEGER](#) | [VAR STRING](#) variable_name) - variable_name is the string or integer variable for which to return the segment address. Returns: The segment address of the specified variable. Notes: 32-bit pointers consist of a segment and an offset. The segment is the upper 16 bits, while the offset is the lower 16 bits.
Examples: See the [Intr](#)() command. See Also: [Ofs](#)(), [Addr](#)(), [AdjPtr](#)()

Selecting Files to Edit

When you start the editor, you can select one or more files to edit, as described in the preceding sections. Each file that is selected is added to the Ring of files maintained by the editor. The first (or only) file that is selected is then opened and is made the current file. (For more information on how multiple files are managed, see
 "

[Managing Multiple Files and Maintaining the File Ring](#)" in the chapter on "[Loading, Processing, and Saving Files](#).") When you specify filenames either on the command line or at a "File(s) to edit:" prompt, you can do so by entering the complete filename, by including full path information, or by entering an abbreviated specification. Multiple filenames are specified by separating each filename by a space. If the filename you specify is an abbreviated filename specification, the editor first establishes the complete path for the filename. If you do not specify a drive, the current drive is used. If you do not specify a path, the current path is used. Once the path for the filename is determined, the editor then resolves any ambiguous or partial specifications. Ambiguous filenames are those that contain wildcard characters ("*" [or](#) "?"). For filenames containing wildcard characters, the editor can be configured to either select all matching files or display a PickList of matching files from which you can choose. The LoadWildFromDOS variable determines the default behavior for handling filenames containing wildcard characters, for a filename specification included on the command line; the LoadWildFromInside variable determines the default behavior for handling filenames containing wildcard characters, for a filename specification entered at the "File(s) to edit:" prompt. The "-a" [and](#) "-a" [command-line options](#) (discussed in this chapter) can be used to override the default behavior. (For more information on configuring the LoadWildFromDOS and LoadWildFromInside variables, see "Load Wild from Command Line" [and](#) "Load Wild from Inside", respectively, under "[System/File Options](#)" in the chapter on "[Configuring the Editor Interactively](#)." For more information on selecting a file from a PickList, see "[Obtaining a File PickList Within a Prompt Box](#)" in the chapter on "[Command Prompts and PickLists](#).") A partial filename is one that does not include an extension and does not end with ".". If a partial filename specification does not match an existing file, the editor tries to locate a matching file using the list of default extensions specified by the DefaultExt variable. If a matching filename is located, that file is selected; if no match is found, the editor uses the filename as entered. (For more information on configuring the DefaultExt variable, see "[Default File Extensions](#)" under "[System/File Options](#)"

in the chapter on "[Configuring the Editor Interactively.](#)")

Selecting the Tab Type

There are four different tab types that can be used in the editor; three are positional and one is physical. The three positional tab types are Soft, Variable, and Smart Tabs. The physical tab type is Hard Tabs. You select the active tab type by setting the TabType option.

★

TabType ("Tab Type" under Options [Menu](#)) This option sets the tab type to be used to either Hard, Soft, Smart, or Variable. When you select this option, the following selection menu is displayed: +-----+ | Hard | |*Soft*****| | Smart | | Variable | +-----+ Move the cursor bar over the desired tab type and press enter. The standard default setting for TabType is Soft. Soft Tabs are the simplest and most commonly used type of tab. (For more information on how to change the default TabType, see "Tab Type" under "[Tab/Margin Options](#)" in the chapter on "[Configuring the Editor Interactively.](#)") The type of tab selected controls the setting of tab stops and the operation of the tab commands (TabRight and TabLeft). It also determines whether or not actual tab characters are placed in the file. The following sections describe the methods for setting tab stops and the effects with each type of tab.

Selecting the Video Mode

The number of lines and columns displayed on the screen is determined by the video mode in effect.

You can change the video mode in effect during an editing session by changing the CurrVideoMode variable (which affects only the current editing session). To change the current video mode, select "Change Video Mode" under the Util

[Menu](#). You can then choose from the following options to select your desired video mode (as supported by your screen resolution and current font): Selection Video Mode _____

_____ 25 Lines by 80 Columns 25 lines by 80 columns 43 Lines by 80 Columns 43 lines by 80 columns 50 Lines by 80 Columns 50 lines by 80 columns Max Lines Maximum lines supported by the current screen resolution and font Max Cols Maximum columns supported by the current screen resolution and font Max Lines/Cols Maximum lines/columns supported by the current screen resolution and font Custom Lines/Cols User-specified lines and columns Extended Modes Additional pre-defined video options When you start the editor, the effective video mode is determined by the value of the StartupVideoMode variable. You can choose to have the editor use the same video mode that is already in effect when the editor is started (this is the standard setting), or you can select one of the editor's pre-defined screen display options as your startup video mode (if supported by your system). For more information on setting the video mode that the editor uses each time it starts, see "Startup Video Mode" under "[Display/Color Options](#)" in the chapter on "[Configuring the Editor Interactively.](#)"

SelfInsert

Inserts the ASCII character associated with a key.

Syntax: [INTEGER SelfInsert\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command is only meaningful as part of a key definition for a typeable key. As the only definition for the key, it has the effect of removing any macro definition from the key. **Examples:** // Insert a Ctrl-L (formfeed character) when <Ctrl L> is // pressed <Ctrl L> [SelfInsert\(\)](#) // Insert an escape character when <Ctrl [> is pressed // (The ASCII code for <Ctrl [> is the same as that for // <Escape>) <Ctrl [> [SelfInsert\(\)](#) // A macro to re-format the current paragraph when // <Spacebar> is pressed [proc](#) mSpaceBar() [SelfInsert\(\)](#) [if](#) [Query](#)(WordWrap) [PushPosition\(\)](#) // Save position [WrapPara\(\)](#) [PopPosition\(\)](#) // Restore position [endif](#) [end](#) <Spacebar> mSpaceBar() See Also: [PressKey\(\)](#), [PushKey\(\)](#)

Sending Output and Commands to the Printer

Within the editor, you can use these commands to print the entire contents of the current file, or to print a block of text marked in the current file.

* [PrintFile](#) ("File" under [Print](#)

[Menu](#)) This command sends the entire current file to the print device. * [PrintBlock](#) ("Block" under [Print](#) [Menu](#)) This command sends the contents of a block in the current file to the print device. If no block is marked in the current file when [PrintBlock](#) is executed, the print operation fails and the following message is given: No block in this file Press <Escape> When you execute one of the above print commands, the print output and the page format of the text sent to the printer are affected by your current settings for the editor's print options. These options are described in the section, "[Setting Print Options](#)."

Session-Global Variables

The editor has a special feature called session-global variables. These variables, unlike regular global variables, are not lost when a macro is purged. They are retained for the entire editing session. They are very similar to environment

[variables](#), in that they may be shared by several macro files, and not just a single macro file (as are regular global variables). The commands that operate on [session-global variables](#) are [SetGlobalInt\(\)](#), [SetGlobalStr\(\)](#), [GetGlobalInt\(\)](#), and [GetGlobalStr\(\)](#). [SetGlobalInt\(\)](#) and [SetGlobalStr\(\)](#) assign a value to a session-global variable. [SetGlobalInt\(\)](#) is used for integers, and [SetGlobalStr\(\)](#) is used for [strings](#). Use the following basic format for these commands: [INTEGER](#) [SetGlobalInt](#)([STRING](#) name, [INTEGER](#) value) [INTEGER](#) [SetGlobalStr](#)([STRING](#) name, [STRING](#) value) Both commands return FALSE if the variable could not be created. [GetGlobalInt\(\)](#) and [GetGlobalStr\(\)](#) retrieve the value assigned to a session-global variable, for integers and strings, respectively. Use the following basic format for these commands: [INTEGER](#) [GetGlobalInt](#)([STRING](#) name) [STRING](#) [GetGlobalStr](#)([STRING](#) name) If the variable does not exist, [GetGlobalInt\(\)](#) returns 0, and [GetGlobalStr\(\)](#) returns the empty string "". [Session-global variables](#) are used when you need data to be accessible between [macros](#) in DIFFERENT macro files. These [variables](#) are "created," [and](#) must be created before they are used. For example: [SetGlobalInt](#)("foo", 0) In contrast, regular global [variables](#) are used when you need data to be accessible between macros in a SINGLE macro file. These variables are "declared," [and](#) must be declared before they are used. For example: [integer](#) bar = 0 (For more information on regular global variables, see "[Variables](#)" in the chapter on "[Macro Language Syntax](#).")

Set

Sets the specified editor variable to a new value. The previous value of the editor variable is returned.

Syntax: [INTEGER Set](#)([INTEGER](#) editor_variable, [INTEGER](#) new_value) or: [STRING Set](#)([STRING](#) editor_variable, [STRING](#) new_value) - editor_variable must be one of the pre-defined editor [variables](#). - new_value is the setting for the variable. Returns: The value of the passed editor variable, before being set. Notes: For more information on all the variables in the editor, see the chapter on "[Editor Variables](#)." [Set\(\)](#) returns the same value as [Query\(\)](#) but also may change the value. See Also: [Toggle\(\)](#), [Query\(\)](#)

SetBit

Sets the specified bit to 1.

This command is intended for ADVANCED USAGE.

Syntax: [SetBit](#)([STRING](#) bitset, [INTEGER](#) bit) - bitset is the bit-mapped string in which to set a particular bit. - bit is the specific bit number to be set. Returns: Nothing. Notes: Bit numbering starts at 0. The editor uses bit-mapped strings for both the WordSet variable and the [TabSet\(\)](#) command. [SetBit\(\)](#), [ClearBit\(\)](#), and [GetBit\(\)](#) can be used to modify these [values](#). Warning: No checks are done to verify that bit is valid. For example, given a string that is 10 characters long means that there are 80 bits available (8 bits per character/byte * 10 = 80, bits 0 - 79). Specifying a value greater than 79 would be out of bounds for this [string](#), but this boundary condition is [not](#) checked! **Examples:** Suppose you want to add the "_" character to the WordSet without changing the other WordSet characters: [string](#) word_set[32] word_set = [Query](#)(WordSet) [SetBit](#)(word_set, [ASC](#)('_')) [Set](#)(WordSet, word_set) See the WordSet variable for additional information. See Also: [ClearBit\(\)](#), [GetBit\(\)](#)

SetBufferInt

Sets the value of the specified buffer variable.

Syntax: [INTEGER SetBufferInt](#)([STRING](#) buffer_var_name, [INTEGER](#) value [, [INTEGER](#) buffer_id]) - buffer_var_name is the name of the buffer variable to set. - value is the INTEGER value buffer_var_name will be set to. - buffer_id is an optional buffer id, as returned by [GetBufferId\(\)](#). Returns: Non-zero if the operation is successful (buffer_var_name can be created); [otherwise](#), zero ([FALSE](#)). Notes: If buffer_id is not specified, the current buffer is assumed. See Also: [SetBufferStr\(\)](#), [GetBufferInt\(\)](#), [GetBufferStr\(\)](#), [ExistBufferVar\(\)](#), [DelBufferVar\(\)](#)

SetBufferStr

Sets the value of the specified buffer variable.

Syntax: [INTEGER SetBufferStr](#)([STRING](#) buffer_var_name, [STRING](#) value [, [INTEGER](#) buffer_id]) - buffer_var_name is the name of the buffer variable to set. - value is the STRING value buffer_var_name will be set to. - buffer_id is an optional buffer id, as returned by [GetBufferId\(\)](#). Returns: Non-zero if the operation is successful (buffer_var_name can be created); [otherwise](#), zero ([FALSE](#)). Notes: If buffer_id is not specified, the current buffer is assumed. See Also: [SetBufferInt\(\)](#), [GetBufferStr\(\)](#), [GetBufferInt\(\)](#), [DelBufferVar\(\)](#), [ExistBufferVar\(\)](#)

SetColorTableValue

Changes the specified color value that the editors uses.

This command is intended for ADVANCED USAGE.

Syntax: [INTEGER SetColorTableValue](#)([INTEGER](#) tab, [INTEGER](#) index, [INTEGER](#) rgb) - tab must be either `_FOREGROUND_` or `_BACKGROUND_` - index must be in `_BLACK_ .. _WHITE_`, where index represents the following editor color: Index Color-name [Default](#) Editor Color Value `_BLACK_ 0x000000` `_BLUE_ 0x000080` `_GREEN_ 0x008000` `_CYAN_ 0x008080` `_RED_ 0x800000` `_MAGENTA_ 0x800080` `_BROWN_ 0x808000` `_LIGHT_GRAY_ 0xc0c0c0` `_DARK_GRAY_ 0x808080` `_LIGHT_BLUE_ 0x0000ff` `_LIGHT_GREEN_ 0x00ff00` `_LIGHT_CYAN_ 0x00ffff` `_LIGHT_RED_ 0xff0000` `_LIGHT_MAG_ 0xff00ff` `_YELLOW_ 0xffff00` `_WHITE_ 0xffffffff` - rgb is the red-green-blue value. Individual colors can range from 0 to 255. Returns: The previous RGB (red-green-blue) value. Notes: The editor can use up to 16 foreground and 16 background colors at once. These [values](#) can be set to whatever color [values](#) desired. Since each color component (red-green-blue) can have a range of 0 to 255, this presents a possibility of 256*256*256 or 16,777,216 colors to choose from. An RGB color value is composed of three component bytes, representing the intensity of the colors red, green and blue in the resulting color. For instance, to give red an intensity of 30, green an intensity of 60, and blue and intensity of 90, one would first convert that [values](#) to hexadecimal: 1eh, 3ch, and 5ah, respectively. Next, simply combine those [values](#), as in: 0x1e3c5a to obtain an RGB color value that can be used with [SetColorTableValue\(\)](#). It is not required that the foreground and background colors have the same RGB value. **Examples:** The editors [default](#) blue color (index Color-name `_BLUE_`) is 0x000080 (a blue value of 128, a green and red value of 0). However, some people prefer a slightly different shade of blue, namely 0x0000a8 (a blue value of 168, a green and red value of 0). To use this new color instead of the editor's default blue color, one could place the following in the WhenLoaded of a startup or autoload macro: // set foreground table [SetColorTableValue](#)(`_FOREGROUND_`, `_BLUE_`, 0x0000a8) // set background table [SetColorTableValue](#)(`_BACKGROUND_`, `_BLUE_`, 0x0000a8) See Also: [GetColorTableValue\(\)](#)

SetCursorOff

Turns the display cursor off.

Syntax: [INTEGER SetCursorOff](#)() Returns: The [cursor](#) state when this command was issued, either [ON](#) or [OFF](#). Notes: This command provides a functional alternative to the Cursor variable. See Also: [SetCursorOn\(\)](#) [Variables: Cursor](#)

SetCursorOn

Turns the display cursor on.

Syntax: [INTEGER SetCursorOn](#)() Returns: The [cursor](#) state when this command was issued, either [ON](#) or [OFF](#). Notes: This command provides a functional alternative to the Cursor variable. See Also: [SetCursorOff\(\)](#) [Variables: Cursor](#)

SetFileAttr

Sets the attributes of a specified file.

Syntax: [INTEGER SetFileAttr](#)([STRING](#) filename, [INTEGER](#) attributes) - filename is the file whose attributes will be changed. - attributes are the file attributes to set. Supported values [for](#) attributes are: - [_NORMAL_](#) specifies the file can be read from or written to. - [_READONLY_](#) specifies the file can be read from but not written to. - [_HIDDEN_](#) specifies the file is hidden and does not appear in a file directory listing. - [_SYSTEM_](#) specifies the file is a system file. - [_ARCHIVE_](#) specifies the file is marked for archiving. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: To determine the current attributes for a file, use the [FileExists\(\)](#) command. **Examples:** Given: integer a To remove the READONLY attribute (but preserve all other existing attributes) from a file: a = [FileExists](#)(fn) // Get existing attributes if a // If any (if 0, file does not exist) [SetFileAttr](#)(fn, a & ~ [_READONLY_](#)) // remove READONLY [endif](#) To add the READONLY attribute (but preserve all other existing attributes) to a file: a = [FileExists](#)(fn) // Get existing attributes if a // If any (if 0, file does not exist) [SetFileAttr](#)(fn, a | [_READONLY_](#)) // add READONLY [endif](#) See Also: [FileExists\(\)](#)

SetFont

Sets the name, size, and style of the font to use for the editing display.

This command has no effect in the Console version of the editor.

Syntax: [INTEGER SetFont](#)() or [INTEGER SetFont](#)([STRING](#) fontname, [INTEGER](#) pointsize, [INTEGER](#) flags) - fontname is the name of the font. Examples are "Fixedsys", and "Courier New". - pointsize identifies the font size. If pointsize is > 255, it is assumed to be a specific width and height (in pixels), rather than a point size. - flags can be any combination of [_FONT_BOLD_](#) and/or [_FONT_OEM_](#) or just 0. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If the first format is used, the standard Windows ChooseFont dialog box is displayed, allowing one to select a font. The second format is used to set a specific font without having to go through a dialog. This command can fail if the specified font is [not](#) available. **Examples:** To set the font to Terminal 8x12: (width 8, height 12) [SetFont](#) ("Terminal", 0x080c, [_FONT_OEM_](#)) The magic formula to use is: width * 256 + height, expressed in Hexadecimal. So, 7 x 15 (width 7, height 15) would be expressed as: 7 * 256 = 1792 1792 + 15 = 1807 1807 is 0x070f in Hexadecimal. Finally, note that you can skip the "* 256" part. This always creates a number whose Hexadecimal representation ends in two zeros. To set the font to Terminal, point size 9: [SetFont](#)("Terminal", 9, [_FONT_OEM_](#)) The former resolves to the same size font regardless of Windows system settings, whereas the latter may resolve to different font pixel sizes, depending on [Windows](#) system settings, particularly the setting of Large vs. Small fonts. To change the font to "Courier New", but use the same pixel size as the currently selected font: [integer](#) width, height, pointsize, flags [string fontname](#)[255] [GetFont](#)(fontname, pointsize, flags) [if fontname](#) <> "Courier New" [GetCharWidthHeight](#)(width, height) [SetFont](#)("Courier New", width * 256 + height, flags) [endif](#) See Also: [ChooseFont](#) (), [GetFont](#)(), [GetCharWidthHeight](#)(), [ResizeFont](#)()

SetGlobalInt

Creates and/or sets the value of a session global integer variable.

Syntax: [INTEGER SetGlobalInt](#)([STRING](#) integer_name, [INTEGER](#) value) - integer_name is the name of the session global integer variable to create/set. - value is the value to which integer_name will be set. Returns: Zero ([FALSE](#)) if integer_name could [not](#) be created; [otherwise](#), non-zero. Notes: This facility can be used to permit communication between macro files. To retrieve the value of a session global integer set with [SetGlobalInt](#)(), use [GetGlobalInt](#) (integer_name). **Examples:** // create "secret_code" session global variable. [SetGlobalInt](#)("secret_code", 324693) // later on, in another macro file: [integer](#) code // sets code to 324693 code = [GetGlobalInt](#)("secret_code") See Also:

[SetGlobalStr\(\)](#), [GetGlobalInt\(\)](#), [GetGlobalStr\(\)](#), [DelGlobalVar\(\)](#), [ExistGlobalVar\(\)](#)

SetGlobalStr

Creates and/or sets the value of a session global string variable.

Syntax: [INTEGER SetGlobalStr](#)([STRING](#) string_name, [STRING](#) value) - string_name is the name of the session global string variable to set/create. - value is the value to which string_name will be set. Returns: Zero ([FALSE](#)) if string_name could [not](#) be created; [otherwise](#), non-zero. Notes: This facility can be used to permit communication between macro files. To retrieve the value of a session global string set with [SetGlobalStr\(\)](#), use [GetGlobalStr](#) (string_name). **Examples:** // create "password" session global variable. [SetGlobalStr](#)("password", "QUARK") // later on, in another macro file: [string](#) pw[12] // sets pw to "QUARK" pw = [GetGlobalStr](#)("password") See Also: [SetGlobalInt\(\)](#), [GetGlobalInt\(\)](#), [GetGlobalStr\(\)](#), [DelGlobalVar\(\)](#), [ExistGlobalVar\(\)](#)

SetHookState

Sets the hook state of an event.

Syntax: [INTEGER SetHookState](#)([INTEGER](#) state [, [INTEGER](#) event]) - state is the hook state (enabled or disabled) to set the specified editor event to. If non-zero (TRUE), the hook state of the specified event is enabled; if zero (FALSE), the hook state is disabled. - event is the optional editor event to enable/disable. If event is not specified, the "global" editor event state is set. See the [Hook\(\)](#) command for a list of editor events. Returns: The state of the event prior to execution of this command. Notes: This command allows you to specify whether the hooks for a particular editor event are currently enabled or disabled. If the hooks are enabled, the editor event will trigger any event macros that are hooked to the event. If disabled, the event will [not](#) trigger [event macros](#) hooked to it. This command can be used within macros to disable the hooks for an event, do some required processing, and then re-enable the previous hooks. With this command, you do not need to use the [BreakHookChain\(\)](#) command to prevent execution of an event macro that has been hooked to the event. However, care must be taken to restore the previous hook state after using this command. The editor maintains a "global" event state. When the editor is started, the global event state is enabled, which means that the hooks for all editor events are enabled. Specific events can be individually disabled/enabled using the [SetHookState\(\)](#) command. The global event state can also be disabled using the [SetHookState\(\)](#) command, forcing all editor events to be disabled. When the global event state is re-enabled (again using the [SetHookState\(\)](#) command), each editor event is then restored to its previous hook state (just prior to the disabling of the global state). **Examples:** // in the following example, the file-changing hook will // not be called, because the hook state has been disabled // by [SetHookState\(\)](#) [integer](#) state, cid cid = [GetBufferId\(\)](#) state = [SetHookState](#)(OFF, _ON_CHANGING_FILES_) [repeat](#) [NextFile\(\)](#) DoSomething() [until](#) cid == [GetBufferId\(\)](#) [SetHookState](#)(state, _ON_CHANGING_FILES_) See Also: [BreakHookChain\(\)](#), [ExecHook\(\)](#), [GetHookState\(\)](#), [Hook\(\)](#), [UnHook\(\)](#), [TerminateEvent\(\)](#)

SetMenuBar

Sets the main menubar to the specified menubar.

Syntax: [INTEGER SetMenuBar](#)([MENUBAR](#) menubar) - menubar is the name of the menubar that is to be assigned as the new main menubar. or: [INTEGER SetMenuBar](#)([INTEGER](#) menubar_id) - menubar_id is the id of the menubar to

specify as the new main menubar. This id is obtained from a previous call to this command. Returns: The identifier of the main menubar as it was just before being set to the specified new menubar. Notes: The purpose of this command is to provide a means to temporarily change the active main menubar, and then later restore it. [Examples](#): To temporarily set the main menubar, assuming MyMenu is a [menubar](#): `integer oldmenubar = SetMenuBar(MyMenu) // save current menubar` // and set new [menubar](#) // do whatever here... To restore the previous main menubar: `SetMenuBar(oldmenubar)` // restore previous [menubar](#) See Also: [Variables](#): [ShowMainMenu](#)

SetPrinterFont

Sets the font to be used when printing.

Syntax: `INTEGER SetPrinterFont()` Returns: Non-zero if successful; zero ([FALSE](#)) on failure. [FALSE](#) is returned if the ChooseFont dialog is canceled, for instance. Notes: This command displays the standard Windows ChooseFont dialog, allowing the user to select a font. The selected font is assigned as the font to use when printing. In order for the printer font to actually be used when printing, the PrintUseScreenFont editor variable must be set to Off. See Also: [PrintFile\(\)](#), [PrintBlock\(\)](#), [PrintChar\(\)](#) **Variables:** [PrintUseScreenFont](#), [PrinterFontFlags](#), [PrinterFontName](#), [PrinterFontSize](#)

Setting Print Options

This section describes the print options provided in the editor. You can access them by selecting "Set Options" under the Print Menu. Alternatively, you can set these options using IConfig (the Interactive Configuration facility) by selecting "

[Full Configuration](#)" under the Options Menu, and then choosing "[Print Output Options](#)." (For more information on using IConfig, see the chapter on "[Configuring the Editor Interactively](#).") * [PrintLeftMargin](#) ("Left Margin" under "Set Options" of Print [Menu](#)) Indicates the number of spaces added to the beginning of each line that is printed. The number of characters that can be printed on each line, as determined by [PrintRightMargin](#) (if specified), is reduced by the value of the [PrintLeftMargin](#) setting. This option also affects placement of printed line [numbers](#) (see [PrintLineNumbers](#)), and [PrintHeader](#) and [PrintFooter](#) information, if these options are selected. Valid [values](#) are 0 through 255. * [PrintRightMargin](#) ("Right Margin" under "Set Options" of Print [Menu](#)) Indicates the maximum number of characters printed for each line. Any text beyond this maximum is truncated. The [PrintRightMargin](#) value includes the value of [PrintLeftMargin](#), as well as the number of characters required to [include](#) line [numbers](#) (see [PrintLineNumbers](#)), if these options are specified. For example, if [PrintRightMargin](#) is set to 72, [PrintLeftMargin](#) is set to 4, [PrintLineNumbers](#) is ON, and the number of characters required for printed line numbers is 3 (assuming the last printed line number is 2 digits), then the first 65 characters of each line are printed. The setting of this option also [affects](#) placement of [PrintHeader](#) and [PrintFooter](#) information, if specified. Valid values are 0 through 255. If the value you select exceeds the printable right margin of the page, your printer may wrap any excess text to the next line. * [PrintLinesPerPage](#) ("Lines Per Page" under "Set Options" of Print [Menu](#)) Indicates the number of lines available for print output per page. The value specified for this option includes any [values](#) set for [PrintTopMargin](#) and [PrintBotMargin](#), and any additional lines (2 each) required for [PrintHeader](#) or [PrintFooter](#) lines (if specified). The number of lines of actual text that can be printed within the available [PrintLinesPerPage](#) is also affected by the value of [PrintLineSpacing](#). If [PrintLinesPerPage](#) is set to a value greater than 0, and if [PrintTopMargin](#), [PrintBotMargin](#), [PrintHeader](#), and/or [PrintFooter](#) [values](#) are specified, the [PrintLinesPerPage](#) setting must be large enough to accommodate those [values](#); [otherwise](#), nothing will be printed. If [PrintLinesPerPage](#) is 0, and no formfeed characters exist in the file, text will be printed, but the [PrintTopMargin](#) value, along with any [PrintHeader](#) text to be printed, will

be ignored for all but the first printed page; and the PrintBotMargin value, along with any PrintFooter text to be printed, will be ignored for all pages. If PrintLinesPerPage is 0, but formfeed characters exist in the file, the PrintTopMargin value will be recognized, and the PrintFooter and PrintHeader information will be generated, at each occurrence of a formfeed. If the value for PrintLinesPerPage is greater than the actual number of printable lines per page recognized by the printer, page breaks will not occur as expected. Valid [values](#) are 0 through 255. A value of 0 causes this option to be ignored so that text is printed on all available lines as determined by the printer (or until a formfeed character is encountered in the file). * [PrintTopMargin](#) ("Top Margin" under "Set Options" of Print [Menu](#)) Indicates the number of blank lines to skip from the top of the page before printing text. If a PrintHeader is specified, it is printed on the first line following the PrintTopMargin value. The value specified for PrintTopMargin is taken out of the total number of print lines on a page, if PrintLinesPerPage is specified. For the PrintTopMargin setting to be effective, PrintLinesPerPage must be set to a value greater than 0, or formfeeds must be included in the file; otherwise, PrintTopMargin is IGNORED for all pages except the first printed page. Valid [values](#) are 0 through 255. * [PrintBotMargin](#) ("Bottom Margin" under "Set Options" of Print [Menu](#)) Indicates the number of blank lines to leave at the bottom of each page. The value specified for PrintBotMargin is taken out of the total number of print lines on a page, as specified by PrintLinesPerPage. For the PrintBotMargin setting to be effective, PrintLinesPerPage must be set to a value greater than 0; [otherwise](#), [PrintBotMargin](#) is IGNORED for all pages. Valid [values](#) are 0 through 255. * [PrintLineSpacing](#) ("Line Spacing" under "Set Options" of Print [Menu](#)) Indicates the number of lines to advance for each new line of text to be printed. Valid [values](#) are 1 through 256. A value of 1 signifies single-spaced output, 2 signifies double-spaced output, etc. * [PrintLineNumbers](#) ("Print Line [Numbers](#)" under "Set Options" of Print [Menu](#)) Toggles whether the editor adds line numbers to the print output. Set this option ON to have file line numbers printed at the beginning of each line; set this option OFF if you do not want line numbers added to the print output. The line numbers printed are the numbers of the lines in the file. For example, if you print a block that begins on line 25, the printed line numbers will begin with the number 25, rather than 1. If this option is ON, line numbers are printed to the right of the [PrintLeftMargin](#) (if specified). The number of characters reserved on each line for line numbers is one more than the number of digits required for the last printed line number. Thus, the minimum number of reserved characters is 2. The number of characters that can be printed on each line, as determined by PrintRightMargin (if specified), is reduced by the number of characters reserved for line [numbers](#). * [PrintHeader](#) ("Header" under "Set Options" of Print [Menu](#)) Identifies the optional Header text to be placed on the first printable line of each page, immediately following the PrintTopMargin (if specified). A blank line is skipped between the Header text and the first line of text printed on each page. Thus, to print a Header, two of the lines available for print output are used. If you do not specify a value for this option, no Header text is printed. The selected Header text is printed at the top of each page, as determined by the setting of PrintLinesPerPage and/or the existence of formfeed characters within the file. If PrintLinesPerPage is set to 0, and there are no formfeed characters in the file, the selected Header text is printed on the first page of output only. Enter one or more of the letters from the following chart (in upper or lower case, and in any order) to request the designated Header text option(s). (Positioning of several of the Header text options is dependent upon the PrintRightMargin setting. If PrintRightMargin is set to 0, the selected Header text is positioned as if [PrintRightMargin](#) is set to 72.) Selection Text to be Printed _____

_____ F Filename of the file being printed D Current system date T Current system time P Page number of the current printed page If "F" is selected, the filename and extension (without the path) are printed in upper-case characters, beginning at the PrintLeftMargin. The filename is printed even if it exceeds the PrintRightMargin. If "D" is selected, the current system date is centered between the PrintLeftMargin and PrintRightMargin. If the print margins are not wide enough for the date, the date is not printed. The format of the date is determined by the DateFormat and DateSeparator variables. (For more information on specifying the date format, see "Date Format" and "Date Separator" under "[Command/Format Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") If "T" is selected, the current system time is centered between the PrintLeftMargin and PrintRightMargin. If the print margins are not wide enough for the time, the time is not printed. The format of the time is determined by the TimeFormat and TimeSeparator variables. (For more information on specifying the time format, see "Time Format" and "Time Separator" under "[Command/Format Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") Note that if both "D" and "T" are selected, the current system date followed by the current system time is centered between the PrintLeftMargin and PrintRightMargin. If the print margins are not wide enough for both, then

only the date is printed (if sufficient space exists). If "P" is selected, the word "Page", followed by the page number, is printed. It is right-justified at the PrintRightMargin. If PrintRightMargin is not wide enough for the page number to fit within the margin, the PrintRightMargin is exceeded as much as necessary to print the full page notation. * [PrintFooter](#) ("Footer" under "Set Options" of Print [Menu](#)) Identifies the optional Footer text to be placed on the last printable line of each page, immediately preceding the PrintBotMargin (if specified). A blank line is skipped between the last line of text printed on each page and the Footer text. Thus, to print a Footer, two of the lines available for print output are used. If you do not specify a value for this option, no Footer text is printed. The selected Footer text is printed at the bottom of each page, as determined by the value of PrintLinesPerPage and/or the existence of formfeed characters within the file. If PrintLinesPerPage is set to 0, and there are no formfeed characters in the file, the selected Footer text is printed on the last page of output only. The valid text options and placement of text on the Footer line are the same as that described for the Header (under PrintHeader above). (As with the Header text, positioning of several of the Footer text options is dependent upon the PrintRightMargin setting. If PrintRightMargin is set to 0, the selected Footer text is positioned as if [PrintRightMargin](#) is set to 72.) * [PrintFirstPage](#) ("First Page" under "Set Options" of Print [Menu](#)) Indicates the print output page within the text from which to begin printing. This page is determined by the editor based on the settings of the various print options and/or the presence of formfeed characters in the file. This option is especially helpful for reprinting a portion of a file. For this option to operate effectively, PrintLinesPerPage must be set to an appropriate value greater than 0, or the file itself must contain formfeeds at appropriate intervals. Valid [values](#) are 0 through 65535. If [PrintFirstPage](#) is set to 0, printing starts at the beginning of the file or block, just as if PrintFirstPage were set to 1. * [PrintLastPage](#) ("Last Page" under "Set Options" of Print [Menu](#)) Indicates the last print output page within the text to be printed. This page is determined by the editor based on the settings of the various print options and/or the presence of formfeed characters in the file. This option is especially helpful for reprinting a portion of a file. For this option to operate effectively, PrintLinesPerPage must be set to an appropriate value greater than 0, or the file itself must contain formfeeds at appropriate intervals. Valid [values](#) are 0 through 65535. If [PrintLastPage](#) is set to 0, printing continues to the end of the file or block. * [PrintCopies](#) ("Number of Copies" under "Set Options" of Print [Menu](#)) Indicates the number of times the indicated file or block will be printed. Valid [values](#) are 1 through 65535. * [PrintPause](#) ("Pause Between Pages" under "Set Options" of Print [Menu](#)) Toggles whether the editor suspends printing between pages. Set this option ON for the editor to pause after each page is sent to the printer. The message "Printing paused - press a key to continue" is displayed. Once a key is pressed, the next page is sent to the printer. This allows you to manually feed paper to the printer, one sheet at a time. Set this option OFF for the editor to automatically send each subsequent page to the printer without pausing. For this option to operate effectively, PrintLinesPerPage must be set to an appropriate value greater than 0, or the file itself must contain formfeeds at appropriate intervals.

Setting Tab Stops

Tab stops identify the columns at which text is placed in the file when the tab commands are issued. Tab stops are set at a fixed interval for Soft and Hard Tabs, at varied intervals for [Variable Tabs](#), and at floating intervals based on existing text for [Smart Tabs](#). [Soft and Hard Tabs](#) ----- For Soft and Hard Tabs, tab stops are set at a fixed interval based on the value specified for TabWidth; and, in the Windows version, the beginning column position as indicated by [ZeroBasedTab](#). * [TabWidth](#) ("Tab Width" under Options [Menu](#)) This option identifies a fixed interval for placement of tab stops with Soft and Hard Tabs. When you select this option, you can enter a value from 1 through 16 to designate the tab width. Tab stops are spaced according to the specified interval beginning with column 1 or 0, depending on the setting of ZeroBasedTab. For example, if ZeroBasedTab is OFF, then a TabWidth value of "8" designates a tab stop every 8 columns, beginning with column 1, followed by column 9, column 17, and so on. * [ZeroBasedTab](#) ("Zero-Based Tab Stops" of "[Tab/Margin Options](#)" under "Full Configuration" of Options [Menu](#)) The setting of this option determines the column position from which to begin placing tab stops with [Soft and Hard Tabs](#). To have tab stops begin with column 1 (which is the standard default setting), set this option OFF. To have tab stops begin with column 0, set this option [ON](#). For example, if TabWidth is set to 4, and TabType is set to Soft, then the first 8 tab stops would be: - With [ZeroBasedTab](#) set [OFF](#): 5 9 13 17 21 25 29 33 - With [ZeroBasedTab](#) set [ON](#): 4 8 12 16 20

24 28 32 (For more information on setting the default TabWidth and ZeroBasedTab values, see "Tab Width" [and](#) "Zero-Based Tab Stops," respectively, under "Tab/Margin Options" in the chapter on "Configuring the Editor Interactively.") [Variable Tabs](#) ----- With Variable Tabs, you set tab stops at variable intervals on a line. You select the tab stop locations by specifying actual column numbers. Once the cursor moves beyond the last designated Variable Tab stop position on a line, the tab stops for the rest of the line are located according to the TabWidth value (relative to column 1 or 0, according to the setting of ZeroBasedTab), the same as Soft Tabs. (For more information on how to set the Variable Tab stop locations, see "Variable Tab Stops" under "Tab/Margin Options" in the chapter on "Configuring the Editor Interactively.") [Smart Tabs](#) ----- Smart Tabs provide a way for you to enter text and automatically position it under the text of the previous non-blank line. This can be very handy for creating tables or columns of text. When you execute TabRight with [Smart Tabs](#) selected, tab stops are aligned with the first character of each word (or continuous string of non-white characters) on the first previous non-blank line. For example, the tab stops for a line underneath the first line of this paragraph would be at the columns under "W" (in "When"), "y" (in "you"), "e" (in "execute"), and so on. The final Smart Tab stop would be at the end of the line. Once you go beyond the text of the previous non-blank line, or if there is no preceding non-blank line on which to base tab stops, then the Smart Tabs option behaves like Soft Tabs. When you execute TabLeft with Smart Tabs selected, the Smart Tabs option behaves like Soft Tabs, and the cursor moves to the tab stops defined by the [TabWidth](#).

SetUndoOff

Deactivates Undo processing for the current file, and clears the Undo/Redo information for that file.

Syntax: [SetUndoOff\(\)](#) Returns: Nothing. Notes: To reactivate Undo processing, use the [SetUndoOn\(\)](#) command. See Also: [Undo\(\)](#), [Redo\(\)](#), [SetUndoOn\(\)](#), [ClearUndoRedoList\(\)](#), [RedoCount\(\)](#), [UndoCount\(\)](#), [UndoMode\(\)](#) **Variables:** [KeepUndoBeyondSave](#)

SetUndoOn

Activates Undo processing for the current file.

Syntax: [SetUndoOn\(\)](#) Returns: Nothing. See Also: [Undo\(\)](#), [Redo\(\)](#), [SetUndoOff\(\)](#), [ClearUndoRedoList\(\)](#), [RedoCount\(\)](#), [UndoCount\(\)](#), [UndoMode\(\)](#) **Variables:** [KeepUndoBeyondSave](#)

SetVideoRowsCols

Sets the screen width and height.

Syntax: [INTEGER SetVideoRowsCols\(INTEGER rows, INTEGER cols\)](#) - rows is the number of rows to set. - cols is the number of columns to set. Returns: Non-zero if the number of rows or columns actually changed; zero (FALSE) if the number of rows or columns was unchanged. Notes: In full-screen mode, cols must be 80, and rows must be either 25, 43, or 50. If not in full-screen mode, the values for cols and rows can be anything allowed by the current font. The minimum value for cols is 6, and the minimum value for rows is 3. To determine the maximum values for cols and rows for the current font, use the [GetMaxRowsCols\(\)](#) command. **Examples:** // Set 60-line by 100-column mode [SetVideoRowsCols\(60, 100\)](#) See Also: [GetMaxRowsCols\(\)](#)

SetWindowHeight

Sets the height of the current window.

Syntax: [INTEGER SetWindowHeight\(INTEGER h\)](#) - h is the new requested height of the current window. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: The editor first attempts to resize the window from the bottom edge. If that fails, the top edge is then resized. A top-most or bottom-most window will only have one window from which to resize. Since only adjacent windows will be resized to accommodate the requested size, the new requested height can not always be satisfied, especially if it is greater than the current height, and there are more than two [windows](#) open. Zero is returned if the size of the specified window does not change. For instance, this command would fail if all adjacent [windows](#) are already at their minimum size, and the requested height is greater than the height of the current window. This command fails when [windows](#) are zoomed. See Also: [ResizeWindow\(\)](#), [SetWindowWidth\(\)](#)

SetWindowTitle

Sets the title bar string for the console window.

Syntax: [SetWindowTitle\(STRING title\)](#) - title is the desired title bar string. Returns: Nothing. Notes: The console window is the window in which the editor itself is running. **Examples:** [SetWindowTitle\("TSE Pro!"\)](#) See Also: [GetWindowTitle\(\)](#)

SetWindowWidth

Sets the width of the current window.

Syntax: [INTEGER SetWindowWidth\(INTEGER w\)](#) - w is the new requested width of the current window. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: The editor first attempts to resize the window from the right edge. If that fails, the left edge is then resized. A left-most or right-most window will only have one window from which to resize. Since only adjacent windows will be resized to accommodate the requested size, the new requested width can not always be satisfied, especially if it is greater than the current width, and there are more than two [windows](#) open. Zero is returned if the size of the specified window does not change. For instance, this command would fail if all adjacent [windows](#) are already at their minimum size, and the requested width is greater than the width of the current window. This command fails when [windows](#) are zoomed. See Also: [ResizeWindow\(\)](#), [SetWindowHeight\(\)](#)

Shell

Temporarily suspends the editor's operation and gives the user access to the command-line prompt through a "shell".

Syntax: [INTEGER Shell\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: To exit the shell, type "exit" at the command-line prompt. See Also: [Dos\(\)](#), [IDos\(\)](#), [StartPgm\(\)](#)

ShiftText

Shifts the text on the current line by the specified number of positions.

Syntax: [INTEGER ShiftText](#)([INTEGER](#) how_much [,[INTEGER](#) by_tab]) - how_much specifies the number of positions by which to shift the text. If how_much is a positive value, the text shifts right; if how_much is negative, the text shifts left. - by_tab is optional and may be specified as zero ([FALSE](#)) or non-zero. If non-zero, the text is shifted TabWidth positions, rather than the number of positions specified by how_much. The text shifts right or left, depending on whether how_much is positive or negative. If not specified, the default value is [FALSE](#). Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command fails if there is [not](#) enough memory to successfully complete the operation. If the cursor is within a COLUMN block when this command is executed, only the text within the column block on the current line is shifted. Text outside of the COLUMN block is not changed. The direction of the shift depends on the sign of how_much, as follows: - If how_much is negative, the text shifts to the left. - If how_much is positive, the text shifts to the right. - If how_much is zero, the text does not shift. For instance: - [ShiftText](#)(-6) or [ShiftText](#)(-6, [FALSE](#)) will cause a shift of the text to the left by 6 positions. - [ShiftText](#)(3) or [ShiftText](#)(3, [FALSE](#)) will cause a shift of the text to the right by 3 positions. If the current TabWidth is 4, then: - [ShiftText](#)(-2, [TRUE](#)) will cause a shift of the text to the left by 4 positions (the value of [TabWidth](#)). - [ShiftText](#)(5, [TRUE](#)) will cause a shift of the text to the right by 4 positions (the value of [TabWidth](#)). **Examples:** Shift text on the current line so as to be aligned with the cursor, for instance, move the cursor to the desired column, press the key this command is assigned to, and the text will be shifted such that it lines up with the cursor. After aligning the current line, the cursor moves to the next line. [proc](#) AlignAtCursor() [integer](#) p [if](#) [PosFirstNonWhite](#)() [PushBlock](#)() // ignore blocks! [UnMarkBlock](#)() p = [CurrPos](#)() [ShiftText](#)([CurrPos](#)() - [PosFirstNonWhite](#)()) [PopBlock](#)() // restore block if there [GotoPos](#)(p) [endif](#) [Down](#)() [end](#) See Also: [InsertText](#)()

ShowEntryScreen

Displays the screen that was present at the time the editor was invoked.

Syntax: [INTEGER ShowEntryScreen](#)() Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command returns zero if the current screen size (rows or columns) is smaller than the screen size that was current when the editor was invoked and the entry screen is not shown to the user.

ShowEOFMarker

Determines if the editor displays the "<*** End of File ***>" string after the last line of the current file.

Values: BOOLEAN **Default:** [ON](#) Notes: The attribute that is used to display "<*** End of File ***>" can be set via "[EOFMarkerAttr](#)". See Also: [EOFMarkerAttr](#)

ShowHelpLine

Determines if

the [HelpLine](#) is displayed on the editing screen. **Values:** BOOLEAN **Default:** [ON](#) Notes: If set ON, any HelpLine bars that are defined display at the top or bottom of the editing screen. The location will be the opposite location occupied by the [StatusLine](#). See Also: [StatusLineAtTop](#), [HelpLineDelay](#)

ShowLineNumbers

Causes line numbers to be displayed on the left side of the editing window.

Values: BOOLEAN Default: [OFF](#) Notes: To change the color line numbers are displayed in, see the [LineNumbersAttr](#) editor variable. Affects: See Also: [LineNumbersAttr](#)

ShowMainMenu

Determines whether the main pull-down Menu displays continuously on the editing screen.

Values: BOOLEAN Default: [ON](#) Notes: If set ON, the main pull-down Menubar displays continuously at the top of the screen. This option only has effect if there is a menubar named "MainMenu" in the user-interface (".UI") file that is currently installed in the editor (as is the case in the user-interface files supplied with the editor). If set OFF, the main Menubar displays only when activated. See Also: [ShowStatusLine](#), [ShowHelpLine](#)

ShowMouse

Shows the mouse cursor.

Syntax: [ShowMouse\(\)](#) Returns: Nothing. Notes: [ShowMouse\(\)](#) increments an internal mouse cursor flag. If the value of this flag is zero, the mouse reappears on the screen. If [HideMouse\(\)](#) has been called more than once (either by a macro or internally by the editor), and the internal mouse cursor flag is non-zero, the mouse cursor will not reappear until the flag is zero. For each use of [HideMouse\(\)](#), there should be a corresponding [ShowMouse\(\)](#) at the appropriate location. For example, if [HideMouse\(\)](#) is called 3 times, without any calls to [ShowMouse\(\)](#), then 3 [ShowMouse\(\)](#) calls will be necessary to cause the mouse to re-appear. See Also: [HideMouse\(\)](#)

ShowStatusLine

Determines whether the editor displays a StatusLine.

Values: BOOLEAN Default: [ON](#) Notes: If ShowStatusLine is [OFF](#), [the StatusLine](#) is [not](#) displayed. [Message\(\)](#) and [Warn\(\)](#) still display their message at the line where a StatusLine would be displayed. See Also: [ShowHelpLine](#), [StatusLineUpdating](#), [StatusLineAttr](#), [StatusLineFillChar](#), [StatusLineRow](#)

ShowSyntaxHilite

Determines whether active SyntaxHilite colors are displayed for applicable files during an editing session.

Values: BOOLEAN - [OFF](#) turns SyntaxHilite color display [OFF](#). - [ON](#) turns SyntaxHilite color display [ON](#). Default:

[ON](#) Notes: While [ShowSyntaxHilite](#) is [ON](#), SyntaxHilite colors are displayed [for](#) any files for which [SyntaxHilite mode](#) is active. While ShowSyntaxHilite is OFF, the editor suppresses the display of SyntaxHilite colors for all files in the editing session. See Also: Commands: [InitSynhiCurrFile\(\)](#), [PurgeSynhi\(\)](#), [GetSynFilename\(\)](#), [LinkSynFile\(\)](#), [LoadSynhiAssoc\(\)](#), [ReplaceSynFile\(\)](#)

SignOn

Displays the editor's signon screen.

Syntax: [SignOn\(\)](#) Returns: Nothing.

Simple Customization from Within the Editor

For many users, the full range of desired customization can be made from within the editor itself. You can change settings for editing options using the menu-driven Interactive Configuration facility (IConfig). You can record

[Keyboard Macros](#) to automate routine editing tasks. You can create templates to simplify entry of text that you type often or in a pre-defined format. You can even assign supplemental macros to the Potpourri PickList so that they are readily accessible during an editing session (as described later in this chapter under "[Adding an External Macro to the Potpourri](#)"). [Changing Settings for Editing Options Using IConfig](#) ----- To use the Interactive Configuration facility, select "[Full Configuration](#)" under the Options Menu. The entire range of configurable editing options is available for modification via menus. With this feature, you can change the editor's screen display colors, [WordSet](#) characters, date/time format, editing modes (such as [AutoIndent](#), [Insert](#), and [WordWrap](#)), tab styles and tab widths, right and left margins, printing options, and more. Would you prefer that the editor not display [the HelpLine](#)? Would you like [the StatusLine](#) to be located at the bottom of the screen rather than the top? Do you want the editor to start in 28-line, rather than 25-line, mode? For these and many other configurable settings, you can use IConfig. (For more information on IConfig and on saving your modified settings, see the chapter on "[Configuring the Editor Interactively](#).") A subset of the most common editing options are also directly available under the Options Menu, without having to select "[Full Configuration](#)". To save the settings you have changed using IConfig (or to save any other option settings available under the Options Menu), select "Save Current Settings" under the Options Menu. Then answer "Yes" to the prompt, "Overwrite existing config?". Once you save your settings, they are installed in the editor and are in effect each time you start the editor. Note that there is an alternate method for setting these same configuration options. That is, you can edit the user-interface (UI) macro source file and modify the settings for the editor variables in the configuration section of that file (as described below under "[Understanding User-Interface \(UI\) Macro Files](#)"), and then install the user-interface file in the editor using the SAL compiler program (as discussed below under "[Installing a User-Interface \(UI\) Macro File](#)"). However, setting the editor variables in the user-interface file is more complex than using IConfig to set configuration options. This method is [not](#) compatible with IConfig; thus, you should [not](#) use both methods to set and install editing options. It is highly recommended that you use IConfig for setting editing options, instead of using this alternate method. [Creating Keyboard Macros](#) ----- A Keyboard Macro is simply a series of keystrokes that have been recorded as they were typed at the keyboard. That series of keystrokes can then be re-issued at the touch of a single key. [Keyboard Macros](#) can be helpful as time-savers and to automate many mundane typing tasks. For instance, if you are having to enter the same series of keystrokes, over and over, you could let the editor "learn and remember" (that is, record) the keystrokes, and then have the editor "play" them back on command. You can even save the remembered keystrokes to a file, so that you can re-play them during a future editing session. Note that [Keyboard Macros](#) have certain limitations. If you need to perform different actions based on the text being edited, or if your macro must accept variable input, then a

Keyboard Macro would not be appropriate for the task. To accomplish such tasks, or almost any other task, you should consider writing a SAL macro, as described more fully in [the Macro Reference Guide](#). Keyboard macros are also very sensitive to their environment. If you record a Keyboard Macro with [Insert](#) and [AutoIndent ON](#), be sure that [Insert](#) and [AutoIndent](#) are ON when the macro is played back. Failure to do so can cause your macro to behave unpredictably. And because [Keyboard Macros](#) are simply recordings of the actual keys that are pressed, if you create and save a Keyboard Macro and then change your key assignments, the Keyboard Macro may not work properly. This is particularly true if any of the keys used in the original recording are reassigned after the Keyboard Macro is created. Even with these limitations, [Keyboard Macros](#) can still be useful. Time spent becoming familiar with them could pay off in time saved doing repetitive editing tasks. (For more information on creating and [using Keyboard Macros](#), see "[Keyboard Macros](#)" in the chapter on "[Creating and Using Macros](#).") [Creating Templates/Abbreviations](#)

----- Templates, and especially templates that you can create interactively, represent another time-saving feature of the editor. Templates can be used: - As a simple abbreviation facility. By using templates, you can avoid typing long or difficult words repetitively. Simply type the word (or phrase) once, and then assign it to an abbreviation (or keyword). After that, you type only the keyword abbreviation and press the template expansion key, and the template immediately replaces the keyword. - To build boiler-plate file headers. For example, you can type your company address and assign it to a suitable keyword. The next time you need to include your company address in a document, just type the keyword abbreviation and press the template expansion key, and the keyword is replaced with your company address. - To create text-entry forms. You can generate forms to guide the entry of text in a prescribed format. Just create the desired format using appropriate words or diagrams, and assign it to a keyword. To insert the guiding form into the text, just type the keyword and press the template expansion [key](#). - To save time writing programming code. "[If statements](#)", "[while statements](#)", or any regular construct in your favorite programming language can be turned into templates. The editor comes with templates for many of the standard language constructs in C, Pascal, xBase, and SAL. Or you can create your own. (For more information on creating and using templates, see "[Template Mode](#)" in the chapter on "[Editing Modes](#).")

SingleInstance

Determines whether the editor starts/uses a "shared" instance for editing purposes.

[Values](#): BOOLEAN [Default](#): [OFF](#) Notes: If SingleInstance is Off, the editor will always start as a separate, non-shared instance for editing purposes. If SingleInstance is On, a single shared instance of the editor is started, if a shared instance of the editor is not running. If a shared instance of the editor is running, then the existing instance of the editor is used for editing purposes. SingleInstance can be specified on the editor command line. See the "-1" (SingleInstance On) and "-1-" (SingleInstance Off) Command-line options. Note that "-1" is a minus one.

SizeOf

Determines the compile-time size of a specified string variable or menu name.

[Syntax](#): [INTEGER](#) [SizeOf](#)([\[STRING\]](#) | [\[MENU\]](#) ident) - ident is the name of the string or menu for which to obtain the size. Returns: The compile-time size of the specified string variable or menu name. Notes: For a string, compile-time size is the size specified in the declaration of the string; that is, the maximum length for that string. This is in contrast to [Length\(\)](#), which returns the length of the actual contents of the passed string. For a menu, compile-time size is the number of menu entries found, including those flagged with [_MF_DIVIDE_](#) or [_MF_SKIP_](#) [menu](#) options. [Examples](#): For example, given: [string](#) mystring[20]="this [string](#)" then: [SizeOf](#)(mystring) returns 20. [Length](#)(mystring) returns 11. See Also: [Length\(\)](#)

Some Simple Macro Examples

We start with the simple macro:

[proc Main\(\)](#) [Message](#)("Hello, World!") [end](#) that displays the message "Hello, World!" when you execute it. Now let us look at some other features of the macro language. [Obtaining Useful Information Within a Macro](#)

----- Let us change the macro to display the current time on the editor StatusLine. [proc Main\(\)](#) [Message](#)("The current time is ", [GetTimeStr](#)()) [end](#) When you execute this macro, it displays the following message on the StatusLine: "The current time is 12:22 pm" This macro illustrates the use of two native editor commands: the [Message](#)() command and the [GetTimeStr](#)() command. Over 400 native commands are built in to the editor, and are all available for use within macros. These native commands are covered in the chapter on [Editor Commands](#). [Adding a Variable](#) ----- Now we illustrate using a variable in a macro. [proc Main\(\)](#) [string](#) name

[40] name = "Bob" [Message](#)("Hello ", name) [end](#) In this macro, we declare "name" as a string variable, with a maximum length of 40 characters. Then we assign the string "Bob" to the string variable "name". When you execute this macro, "Hello Bob" is displayed on [the StatusLine](#). This particular macro illustrates the required ordering of a macro procedure: variables are defined first, then the macro statements follow. [Prompting the User](#) -----

Next we illustrate how to ask the user for some information, and display the user's response. [proc Main\(\)](#) [string](#) name [40] = "" [Ask](#)("What is your name?", name) [Message](#)("Hello ", name) [end](#) This macro prompts the user for a response by displaying the message, "What is your name?", in a dialog prompt box. [Ask](#)() takes the user's response and places it in the variable passed to it; in this case, "name". Note that we initialize "name" to an empty string ("") before calling [Ask](#)(), because [Ask](#)() uses the initial value of "name" as the [default](#) response. When you execute this macro, it displays the message "Hello Bob" on the StatusLine (assuming the user types "Bob" in response to the prompt). [Flow of Control](#) -----

The preceding macro is not quite complete, however. What if the user presses <Escape> in response to the prompt? The native [Ask](#)() command returns FALSE if <Escape> is pressed; [otherwise](#), [Ask](#)() returns TRUE. Now knowing this, we change our macro to: [proc Main\(\)](#) [string](#) name[40] = "" [if](#) [Ask](#)("What is your name?", name) [Message](#)("Hello ", name) [else](#) [Message](#)("Shy, aren't we?") [endif](#) [end](#) If the user types "Bob" in response to the prompt, "What is your name?", and then presses <Enter>, the message, "Hello Bob", displays on [the StatusLine](#). But if the user presses <Escape> instead, then the alternate message, "Shy, aren't we?", displays. What happens, though, if the user does not type any information in response to the prompt, "What is your name?", but just presses <Enter>? If the user does not type a response to an [Ask](#)() prompt, [Ask](#)() does not change the value of the reply [string](#) ("name", in this [case](#)). Since we initialized "name" to "" (an empty string; empty strings have a length of 0) before calling [Ask](#)(), then if [Ask](#)() returns TRUE with the length of "name" equal to 0, we can conclude that the user did not type any information in response to the [Ask](#)() prompt. The following final version of the macro checks for this case: [proc Main\(\)](#) [string](#) name[40] = "" [if](#) [Ask](#)("What is your name?", name) [and](#) [Length](#)(name) > 0 [Message](#)("Hello ", name) [else](#) [Message](#)("Shy, aren't we?") [endif](#) [end](#) The results are the same as the previous example, except this version also displays the message, "Shy, aren't we?" if the user presses <Enter> without typing a response. [Assigning a Macro to a Key](#) -----

Finally, we make our macro available by pressing a key. We edit the macro source file ("HELLO.S"), go to the end of the file, and add: <Ctrl F3> [Main](#)() following the last line of the existing macro. Now compile the macro again by pressing <Ctrl F9>. Once the compile successfully completes, select "Load" from the selection menu that the editor displays. Now, for the rest of this editing session or until you purge the "HELLO" macro, the macro will execute whenever you press <Ctrl F3>. [Creating a Menu](#) ----- Sometimes it becomes difficult to remember all the many different keystrokes that have been assigned in the editor, especially for seldom-used commands. To resolve this problem, the editor offers programmable menus. Menus are a convenient way to allow a user to select an item from a list of choices. In the macro language, menus are easy to create. Suppose, for example, we developed macros to delete sentences, paragraphs, the rest of the file, and the entire file (in memory). Further,

suppose we use these [macros](#) seldom enough that we can [not](#) easily remember [key](#) assignments. This is a perfect candidate for a menu. Following is an example: [proc](#) DeleteSentence() // wonderful macro code goes here [end proc](#) DeleteParagraph() // wonderful macro code goes here [end proc](#) DeleteToEOF() // wonderful macro code goes here [end proc](#) DeleteEntireFile() // wonderful macro code goes here [end menu](#) FancyDeleteMenu() "delete Sentence", DeleteSentence() "delete Paragraph", DeleteParagraph() "delete Rest of file", DeleteToEOF() "delete Entire file in memory", DeleteEntireFile() [end proc](#) [Main](#)() FancyDeleteMenu() [end](#) Enter the above text into a file called "MYMENU.S" and compile as described earlier in this chapter. Now, every time you execute the MYMENU macro, a menu will pop up with the selections listed under FancyDeleteMenu. Note that selecting the menu created in this example will not actually delete anything, as we have left filling in the actual macro code as an exercise for you.

Sort

Sorts lines spanned by a marked block.

Syntax: [INTEGER Sort](#)([[INTEGER](#) flag]) - flag is an optional integer. flag values can be OR'ed in order to achieve the desired result. If not specified, [_DEFAULT_](#) is used. Supported [values](#) for flag are: - [_DEFAULT_](#) results in an ascending, case-sensitive sort. - [_IGNORE_CASE_](#) results in the sort ignoring the case of alphabetic characters. - [_DESCENDING_](#) results in a descending sort. - [_PICKSORT_](#) causes other flags to be ignored, and the text being sorted is assumed to be in the form created by the [BuildPickBufferEx](#)() command. The sort [key](#) and sort order are taken from the PickFileSortOrder variable. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: A maximum of 65535 lines can be sorted. The sort key is determined by the block type. If a column block, up to the first 80 characters of the column block on each line is used as the key. For other block types, the first 80 characters on each line is used as the sort key. [Examples:](#) [Sort](#)() or [Sort](#)([_DEFAULT_](#)) will cause an ascending, case-sensitive sort. [Sort](#)([_DESCENDING_](#)) will cause a descending, case-sensitive sort. [Sort](#)([_IGNORE_CASE_](#)) will cause an ascending, case-insensitive sort. [Sort](#)([_DESCENDING_](#) | [_IGNORE_CASE_](#)) will cause a descending, [case](#)-insensitive sort. See Also: [BuildPickBufferEx](#)() [Variables:](#) [PickFileSortOrder](#)

Sorting Lines of Text

* Sort (<Shift F3>, or "Sort" under Util

[Menu](#)) This command sorts a range of lines in the current file, based on a block that you mark in the file (indicating the sort key and the group of lines to be sorted). Lines can be sorted in either ascending or descending order, and the sort key can be either case sensitive or insensitive. You identify the lines to be sorted by [marking a block](#) in the file that spans the range of lines to be included in the Sort. Only those lines included in the block are sorted. As the lines are sorted, the entire text for each affected line is moved as needed (regardless of whether the marked block includes the entire line). The order in which the sorted lines are arranged is based on the characters you specify for the sort key by appropriately [marking a block](#), as follows: - If you mark a column block, the sort key is limited to the characters within the column block. If the column block is wider than 80 characters, the first 80 characters of the column block are used. - If you mark a line or character block, the first 80 characters of each line are used as the sort key. To specify ascending or descending order for the Sort, select "Sort Order" under the Util Menu, and toggle this option to "Ascending" or "Descending," as needed. To specify whether the Sort is to be sensitive to the case of the characters in the sort key, select "[Case](#)-Sensitive Sort" under the Util Menu. Set this option ON if lines are to be sorted according to the case of alphabetic characters (a through z) included in the sort key; set this option OFF if the case of alphabetic characters should NOT affect the order in which lines are sorted. (A case-sensitive Sort operation is usually faster than a [case](#)-insensitive Sort.) The Sort algorithm used by the editor is stable, so the order of lines with matching sort keys is not changed. Thus, if you wish to sort a group of lines based on multiple sort keys, you can do so by performing multiple Sort operations on the same group of lines, as follows: - First, use a column block to mark the columns

containing the lowest-level sort key, and execute the Sort command. - Next, mark the columns containing the next higher sort key, and execute Sort again. - Continue marking from the lowest to the highest sort key, re-sorting the lines for each key. Once the final Sort is executed, the lines are arranged as specified. For example, suppose you have the following text and wish to sort lines 5 through 14 [in](#) ascending order by date (that is, by year, by month within year, and by day of the month within month/year):

Line No. Text _____ line 1: This text contains a group of items that are to be
 line 2: sorted in ascending, chronological order. The line 3: date field is in the format MM/DD/YY. line 4: line 5:
 07/25/90 This is the third date. line 6: 10/15/92 This is the eighth date. line 7: 07/04/92 This is the sixth date. line 8:
 07/04/91 This is the fourth date. line 9: 01/01/95 This is the tenth date. line 10: 10/15/91 This is the fifth date. line 11:
 07/04/90 This is the second date. line 12: 11/03/93 This is the ninth date. line 13: 01/31/90 This is the first date. line
 14: 10/03/92 This is the seventh date. - Select "Sort Order" under the Util Menu and set this option to "Ascending". -
 Select "[Case-Sensitive Sort](#)" under the Util Menu and set this option ON. (Since numbers do not have a case, this
 option has no effect on the final results of the Sort. But by specifying a case-sensitive Sort, the Sort speed is
 optimized.) - Using <Alt K>, mark the two columns containing the day of the month (lowest sort key), extending the
 block from line 5 through line 14. Press <Shift F3> to execute the Sort command. - Unmark the first column block,
 and then use <Alt K> to mark the two columns containing the month (next higher sort key), again extending the block
 from line 5 through line 14. Press <Shift F3> to execute the Sort command a second time. - Unmark the second
 column block, and then use <Alt K> to mark the two columns containing the year (highest sort key), once again
 extending the block from line 5 through line 14. Press <Shift F3> to execute Sort a third and final time. The text now
 appears as follows: Line No. Text _____ line 1:
 This text contains a group of items that are to be line 2: sorted in ascending, chronological order. The line 3: date field
 is in the format MM/DD/YY. line 4: line 5: 01/31/90 This is the first date. line 6: 07/04/90 This is the second date. line
 7: 07/25/90 This is the third date. line 8: 07/04/91 This is the fourth date. line 9: 10/15/91 This is the fifth date. line 10:
 07/04/92 This is the sixth date. line 11: 10/03/92 This is the seventh date. line 12: 10/15/92 This is the eighth date. line
 13: 11/03/93 This is the ninth date. line 14: 01/01/95 This is the tenth date. To terminate a long-running Sort, press
 <Ctrl Break>. Depending on the size of the text being sorted, it can take several minutes before the "[break](#)" request is
 acknowledged and the Sort terminated. Note that when the Sort operation is terminated, any lines already processed by
 the Sort are re-arranged in the file.

Sound

Turns on the PC speaker and causes it to sound at a specified frequency, for the specified duration.

Syntax: [Sound](#)([INTEGER](#) frequency, [INTEGER](#) duration) - frequency is the pitch of the tone to be issued, in hertz (cycles per second). - duration is the time, in milliseconds, for the tone to persist. Returns: Nothing. Notes: Under WINNT4/2000/XP, [Sound](#)() performs as stated. Under WIN95/98/Me, frequency and duration are ignored, and the following action occurs: If the system is configured with a sound card, the "[default](#) sound event" is played; otherwise, the "standard system beep" is sent to the PC speaker. **Examples:** /

***** Sound the PC speaker with a 1000-Hz signal for
 about 1 second (WINNT4/2000/XP ONLY) *****/
[Sound](#)(1000, 18) See Also: [Alarm](#)(), [Delay](#)()

Special Commands and Features

This chapter describes the following commands and features that are available

[for](#) various special editing functions: - [Drawing lines in the text](#) (LineDrawing mode) - [Sorting lines of text](#) (Sort) -
[Accessing the pop-up ASCII chart](#) (AsciiChart) - [Inserting the date and time into the text](#) (DateTimeStamp) - [Selecting](#)

[the video mode \(CurrVideoMode\)](#) - [Executing external commands from within the editor](#) (Dos, CaptureDosOutput, Shell) - [Repeating a character, command, or macro](#) (RepeatCmd) - [Executing supplemental macros from the Potpourri \(Potpourri\)](#) - [Compiling programs from within the editor](#) (Compile)

Special Handling For Certain Types of Text

Depending on the file extension of the file being spell checked, the scope of spell checking is automatically limited to or restricted from certain portions of the text being checked.

For HTML source files (such as those with an extension of .html, .htm, .shtml, or .shtm), SpellCheck only checks text that is outside HTML tags and

[comments](#). For these files, it also ignores any words that start with "&" [and](#) end with ";". For various programming language source files (such as those with an extension of .c, .h, .cpp, .hpp, .cc, .cxx, .hxx, .java, .js, .s, .si, or .ui), SpellCheck checks only text that is in [comments](#) or [strings](#). For all other file types, SpellCheck ignores text on any lines in which the text begins with a leading ">".

Special Programming-Language Modes

Two modes are provided specifically for entering code for certain programming languages. SmartIndent mode is useful with a variety of programming languages, including the SemWare Applications Language (SAL); C mode is useful [when](#) programming in C. [SmartIndent Mode](#) ----- SmartIndent mode affects the behavior of the BackSpace and CReturn commands to assist you in entering code for a group of programming languages. This mode is automatically activated by the editor if AutoIndent mode is ON or STICKY, and you are editing a file with one of the following filename extensions: .s, .ui, .c, .h, .asm, .cpp, .hpp, .pas, .inc, .prg When SmartIndent mode is active, the BackSpace and CReturn commands behave as follows: - If you execute BackSpace on a blank line or on the first non-white character of a line, the editor issues a TabLeft (based on the current TabType setting). If Insert mode is OFF, the cursor moves left one tab stop position. If Insert mode is ON, the cursor moves left one tab stop position and shifts left any text on the line. - If you execute BackSpace at column one, the cursor moves to the end of the line above and appends the current line to that line. - If you execute CReturn on a line beginning with one of the following key words: [if](#), [else](#), [elseif](#), while, repeat, [loop](#), for, switch, [case](#), when, [otherwise](#), [proc](#) the editor moves the cursor to [the effective left margin](#) of the next line and then immediately issues a TabRight (based on the current TabType setting). This results in an extra indentation on that line. [C Mode](#) ----- C mode is an extension of SmartIndent mode, designed specifically for entering code in the C programming language. This mode is automatically activated by the editor if AutoIndent mode is ON or STICKY, and if you are editing a file with one of the following filename extensions: .c, .h, .cpp, .hpp In addition to the behavior described for SmartIndent mode, C mode further modifies the behavior of the editor, as follows: - If you execute CReturn on a line that begins or ENDS with the character "{", the editor moves the cursor to [the effective left margin](#) of the next line and issues a TabRight (the same as it does for lines beginning with the key words identified for SmartIndent mode). - If you type the character "}", the editor issues a TabLeft, and places the "}" character at the previous tab stop. This mode facilitates C programming of the following style: void foo(void) { [if](#) (dit()) { tic(); tac(); toe(); } } [Not](#) only does this mode save keystrokes, it also keeps your code properly indented. For best results with [C mode](#), [AutoIndent mode](#) should be set to STICKY.

SpecialEffects

Determines what special effects the editor uses.

Values: [INTEGER](#) [_USE_3D_CHARS_](#) - The editor uses 3D characters when drawing Window Borders, [Menus](#), Lists, and Prompt Boxes. If this value is not set, the editor uses OEM characters for the respective display elements. This value has no effect in the Console version. [_DRAW_SHADOWS_](#) - The editor draws shadows on Popup [Windows](#). If this value is [not](#) set, no shadows are drawn. [_USE_3D_BUTTONS_](#) - The editor uses 3D buttons for the drawing of the "close window" symbol on Lists, Prompt Boxes, and editor "[windows](#)", and the "maximize" and "minimize" symbols of editor "[windows](#)". If this value is not set, then the editor uses OEM characters for the respective display elements. This value has no effect in the Console version. [_CENTER_POPUPS_](#) - The editor will attempt to center all popup prompts and [menus](#). **Default:** [_USE_3D_CHARS_](#) [_DRAW_SHADOWS_](#) **Affects:** The editor's display. See Also: Commands: [PutOemStrXY\(\)](#)

Speedsearch Feature

Within `PickLists` and certain other types of text displayed by commands or facilities (like

[the HelpSystem](#)), `SpeedSearch` allows you to quickly locate an entry. Once a `PickList` or other applicable text is displayed on the screen by a command or operation, follow these steps to use the `SpeedSearch` feature: - Type the first character of the desired entry. The cursor bar immediately moves to the first entry that begins with that character, and that character is highlighted. - Type the second character of the desired entry. The cursor bar moves to locate the first entry that matches the first and second characters typed. Those two `SpeedSearch` characters are highlighted in the cursor bar. - Continue typing successive characters. The [cursor](#) bar continues to move to locate the first matching entry. The string of `SpeedSearch` characters is highlighted in the cursor bar. (Note that you can execute `RepeatFind` (<Ctrl L>) to locate the next occurrence of the current `SpeedSearch` [string](#).) - Use the <BackSpace> key to remove the last typed character from the `SpeedSearch` string. The cursor bar moves to the first entry that now matches the `SpeedSearch` string, or to the top of the `PickList` if <BackSpace> is pressed on the first character in the string. - Once the cursor bar is located on the desired entry, press <Enter> to select it. - To deactivate the current `SpeedSearch` string, press any active cursor movement key. The current highlighting is removed and the cursor bar is moved according to the key that is pressed. A new `SpeedSearch` string can be entered by typing the appropriate character(s).

SplitLine

Splits the current line at the cursor position.

Syntax: [INTEGER](#) [SplitLine\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Splits the current line at the cursor position. Any text to the right of, and including the cursor position, is placed on a new line following the current line, beginning at position 1. ([AutoIndent](#) has no effect.) The [cursor](#) position does [not](#) change. If the cursor is past the last character on the line, a blank line is added following the current line. See Also: [JoinLine\(\)](#), [CReturn\(\)](#)

SplitPath

Returns the specified portion of the passed filename string.

Syntax: [STRING](#) [SplitPath](#)([STRING](#) fn, [INTEGER](#) flag) - fn is the path string to be split. - flag determines which parts of fn are returned. flag values can be OR'ed in order to achieve the desired result. Supported [values](#) for flag are: - [_DRIVE_](#) returns the drive letter including the colon (":") [for](#) paths. - [_PATH_](#) returns the path (less the drive portion) including the trailing slash. - [_NAME_](#) returns the filename excluding the extension and the period ("."). - [_EXT_](#) returns the extension, including the period ("."). Returns: The specified part of the input fn, or an empty string if the

specified parts did not exist in the input fn string. Notes: The case of the return string is the same as that of fn. It is not always necessary to enter all flag values to get the desired result. For instance: `_DRIVE_ | _PATH_ | _NAME_ | _EXT_` can be shortened to simply `_DRIVE_ | _EXT_`. **Examples:** Given: `string fn[] = "d:\path\filename.ext"` then: `s = SplitPath(fn, _DRIVE_) // s = "d:"` `s = SplitPath(fn, _PATH_) // s = "\path\"` `s = SplitPath(fn, _NAME_) // s = "filename"` `s = SplitPath(fn, _EXT_) // s = ".ext"` `s = "d:\path\"` `s = SplitPath(fn, _DRIVE_ | _PATH_) // s = "d:\path\filename"` `s = SplitPath(fn, _NAME_ | _EXT_) s = SplitPath(fn, _DRIVE_ | _PATH_ | _NAME_) // in the line above, s = "d:\path\filename"` // the following is a short cut for the above // line and will return the same thing `s = SplitPath(fn, _DRIVE_ | _NAME_) // thus, the following example will return the // full string passed in as fn` `s = SplitPath(fn, _DRIVE_ | _EXT_)` SplitPath will also handle UNC network path specifications: Given: `string netfn[] = "\\servername\volume\path\filename.ext"` then: `// s = ""` `s = SplitPath(netfn, _DRIVE_) s = SplitPath(netfn, _PATH_) // s = "\\servername\volume\path\"` `s = SplitPath(netfn, _DRIVE_ | _NAME_) // s = "\\servername\volume\path\filename"`
See Also: [ExpandPath\(\)](#), [CurrFilename\(\)](#), [SearchPath\(\)](#), [CurrExt\(\)](#)

Splitting the Screen Into Windows

Using

[windows](#), you can simultaneously view multiple files and display different views of the same file. This chapter provides information about: - [The screen layout with multiple windows](#) - [Opening windows](#) (HWindow, VWindow, OpenFileAndWindow) - [Switching between windows](#) (GotoWindow, GotoWindowNumber, NextWindow, PrevWindow) - [Changing window size and orientation](#) (ResizeWindow, ZoomWindow, ToggleHorizVertical) - [Synchronized Scrolling in Windows](#) (ToggleSyncScroll) - [Closing editing windows](#) (CloseWindow, DelWindow, OneWindow, CloseFileAndWindow) - [A walk-through of window commands](#) The chapter on "[Mousing Around the Editor](#)" explains how to use the mouse to create, change, resize, and close [windows](#).

SqueezePath

Truncates a path, as necessary, to fit in a specified field width.

Syntax: `STRING SqueezePath(STRING path, INTEGER max_width)` - path is the path that will be truncated. - max_width is the length to which path will be truncated. Returns: path truncated to a length of max_width. Notes: If the length of path is less than or equal to max_width, path is returned as is. [Otherwise](#), path is truncated, as necessary, to max_width. **Examples:** `string s[60]` `s = SqueezePath(CurrFilename(), 60)`

StartedFromDosPrompt

Determines whether the editor was started from a DOS or Console prompt, or by some other method (such as from the Windows Start menu or from an icon).

Syntax: `BOOLEAN StartedFromDosPrompt()` Returns: Non-zero if the editor was started from a DOS or Console prompt; [otherwise](#), zero ([FALSE](#)).

Starting an Editing Session

When you start the editor, a selection menu, similar to the following, is

displayed:

```
+-----+
| *Open...***** |
| New File       |
| File Manager...|
| List Recent... |
| Pick File...   |
|               |
```

[Restore State](#) || Where || Exit | +-----+ Select open to display the following prompt box in which to specify the name(s) of the file(s) to select for editing: +-----+ | File(s) to edit: [current path] || | +-----+ Enter the filename (or multiple filenames, each separated by a space). The filename should include its full path designation if it is not located in the current directory. Long filenames and UNC network paths can be specified. If a filename includes leading or embedded spaces, that filename must be enclosed in quotes. Or, press <Enter> on an empty "File(s) to edit:" prompt to obtain a sorted PickList of the files in the current directory. To select a file from this list, move the cursor bar over the desired filename and press <Enter>. You can also use the editor's SpeedSearch feature within a PickList to locate a particular file by typing the first one or more letters of the desired filename. Multiple files can be selected from a PickList. To do this, press the <Spacebar> [key](#) on each desired file, and then press <Enter>. You can bypass the "File(s) to edit:" prompt by typing "e32" on the command line, followed by the desired filename(s). For example, from a command line prompt, type: e32 filename and press <Enter>. As with the previous method, multiple filenames can be specified. If the specified file exists, it is loaded into the editor. If the file does not exist, an empty screen is displayed, and the message, "Editing new file...", is shown on the top line of the screen. You are now ready to begin editing. [Additional Options Available at Startup](#)

----- Additional features and options are available when [starting the editor](#). You can specify filenames using partial names or wildcard characters ("*" [and](#) "?"). You can also specify one or more of the following editing options: -k ("[Load Keyboard Macro](#)") denotes a Keyboard Macro file to load -l ("[Load Macro](#)") denotes a Compiled Macro file to load -e ("[Execute Macro](#)") denotes a Compiled Macro file to load and execute -n ("[Go To Line/Column Number](#)") indicates a starting line and/or column -b ("[Binary Load](#)") signifies a file to load in Binary mode -h ("[Hex Load](#)") signifies a file to load in Hex mode -a ("[Load All](#)") forces all matching files to be selected for loading -a- ("[Pick from List](#)") forces display of a PickList of matching files -s ("[Search SubDirs](#)") searches through subdirectories for matching files -y ("[Browse mode](#)") signifies a file to load in Browse mode -r ("[Restore State](#)") restores state information from previous session -i ("[Augment Load Dir](#)") extends the editor's view of its load directory In addition, you can use the macro autoload file to specify macro files to be automatically loaded each time you start the editor, and you can implement your own startup routine for the editor by using [the TSEStart macro](#) option. You can also indicate the file-selection behavior of the editor that will be in effect when you start the editor without specifying any command-line parameters (that is, when you start the editor without specifying any filenames or editing options on the command line). For example, you can instruct the editor to: invoke the FileManager command, display the Recent Files list, open an "unnamed" file, display a PickList of files in the current directory, or restore environment information and associated files from the previous session. You can enable file-locking (if this option is available) to prevent other applications from modifying the files you are editing in your current session. (For more information on [starting the editor](#) and loading files, see the chapters on "[Starting the Editor](#)" and "[Loading, Processing, and Saving Files](#).")

Starting the Editor

A wide range of features are available to offer flexibility in starting an editing session. You can specify command-line parameters to select one or more files for editing and to initiate a number of editing options. You can

choose to have the environment restored from a previous editing session. In addition, you can use the macro autoload file to automatically load macros each time you start the editor, and you can use

[the TSEstart macro](#) option to implement your own startup routine for the editor. This chapter explains the following about [starting the editor](#): - [Starting the editor from the command line](#) - [Starting the editor from the Desktop](#) - [Starting the editor with no command-line parameters](#) - [Selecting files to edit](#) - [Command-line options](#) ([Load Keyboard Macro](#), [Load Macro](#), [Execute Macro](#), [Go To Line/Column Number](#), [Binary Load](#), [Hex Load](#), [Load All](#), [Pick from List](#), [Search SubDirs](#), [Restore State](#), [Augment Load Dir](#)) - [Editor startup processing](#) +----- Tip
-----+ | To create different "versions" of the editor for your own varied | | needs, you can create multiple ".BAT" files to start the editor | | with various filename specifications and/or [command-line options](#). |
+-----+

Starting the Editor from the Command Line

To start the editor from the command line, the format is:

```
GUI version:
g32 [filename1 filename2 ... filenameN] [
```

[command-line options](#)]* or Console version: e32 [filename1 filename2 ... filenameN] [[command-line options](#)]* Items enclosed in [] are optional. Up to 127 characters can be passed from the command line, allowing one or more filenames (1 to N) to be designated and the desired options to be identified. * Command-line options do not have to occur as the last entries on the command line, and in some cases, must precede the affected filename. The [examples](#) given in this chapter illustrate this. Filenames can be specified by entering the complete filename, by including full path information, or by entering an abbreviated specification. Multiple filenames are specified by separating each filename by a space. You can [include](#) long filenames and UNC network paths. You can also specify filenames with leading or embedded spaces by enclosing the filename within quotes. You can start the editor from the command line without specifying any filename specifications. The behavior of the editor in that case will be determined by the setting of the StartupFlags variable. The section "[Starting the Editor with No Command-Line Parameters](#)," presented later in this chapter, explains various options available for selecting files when the filename specification is not included on the command line.

Starting the Editor from the Desktop

When you start the editor from the Desktop, the editor is generally invoked with no command-line parameters passed to it. For more information on the editor's behavior when you start it without specifying any command-line parameters, see "

[Starting the Editor with No Command-Line Parameters](#)" below.

Starting the Editor With No Command-Line Parameters

If you start the editor without specifying any command-line parameters (that is, you do not include any filenames or editing options on the command line, or you start the editor from the desktop with an empty command line), the editor will behave according to the setting of the StartupFlags variable.

The default behavior for StartupFlags is to display a menu of file-selection options.

The StartupFlags variable provides a number of options from which you can choose the file-selection behavior of the editor that will be in effect when you start the editor with no command-line parameters. The following choices are available:

- You can instruct the editor to issue the "File(s) to edit:" prompt so that a filename specification can be entered. Within the "File(s) to edit:" prompt, you can then specify the desired filenames and

[command-line options](#). (For more information on specifying and selecting files at the "File(s) to edit:" prompt, see "[Adding Files to the Ring: Specifying and Opening Files](#)" in the chapter on "[Loading, Processing, and Saving Files](#)." - You can instruct the editor to execute the FileManager command, passing it a file specification of "*. *". (For more information on the FileManager command, see the chapter on "[Loading, Processing, and Saving Files](#).")) - You can instruct the editor to display the Recent Files list. This option is valid only if the PersistentRecentFiles variable is ON. (For more information on configuring this variable, see "Persistent Recent Files" under "[Command/Format Options](#)" in the chapter on "[Configuring the Editor Interactively](#).")) - You can instruct the editor to open an "unnamed" file, by executing the NewFile command. (For more information on the NewFile command, see the chapter on "[Loading, Processing, and Saving Files](#).")) - You can instruct the editor to display a PickList of files in the current directory. (For more information on selecting a file from a PickList, see "[Obtaining a File PickList Within a Prompt Box](#)" in the chapter on "[Command Prompts and PickLists](#).")) - You can instruct the editor to restore a previous editing environment, provided one was previously saved. (For more information on saving and restoring state information for an editing session, see "[Restore State](#)" earlier in this chapter, and also see "[Saving/Restoring the Editing Environment](#)" in the chapter on "[Loading, Processing, and Saving Files](#).")) - You can instruct the editor to display a selection menu from which you can choose one of the above options, or execute the Where command. This is the default behavior. (For more information on configuring the StartupFlags variable, see "Empty Command-Line Action" under "[Command/Format Options](#)" in the chapter on "[Configuring the Editor Interactively](#)."))

StartPgm

Runs a program using the Windows ShellExecute function.

Syntax: `INTEGER StartPgm(STRING pgm_name [, STRING args [, STRING start_dir [, INTEGER startup_flags]])`
 - pgm_name is the name of the program to run. - args are optional command line arguments that should be passed to pgm_name. - start_dir is an optional starting directory. - startup_flags are optional flags that control how pgm_name is started. [Values](#) can be: _START_HIDDEN_, _START_MINIMIZED_, _START_MAXIMIZED_. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This function is the preferred way to run Win32 GUI programs from the editor. [Examples](#): //Cause the editor to run g32.exe, editing the file "some.file" `StartPgm("g32.exe", "some.file")` See Also: [IDos\(\)](#), [Dos\(\)](#), [Shell\(\)](#)

Startup Processing

Each time the editor is started for an editing session, a number of tasks occur, in a specific order.

First, the editor initializes itself.

Next, if there is a WhenLoaded macro procedure in the installed user-interface macro file, it is invoked.

After that, if there is a TSESTART.MAC macro file found in the current directory, or anywhere along the TSEPath (or in a MAC\ subdirectory of the TSEPath directories), or in the editor load directory, the editor loads and executes that macro file. Then, if there is a TSELOAD.DAT file found in the editor load directory, any [macros](#) listed in that file are loaded. Next, the editor processes the command-line string that is passed to it, and the first file listed on the command line is loaded (if one is specified). If the command line does not contain any options or filenames, the editor checks the setting of the StartupFlags variable to determine the appropriate startup action to take. (For more information on [starting the editor](#) without any options or filenames specified, see "[Starting the Editor with No Command-Line Parameters](#)" in the chapter on "[Starting the Editor](#)" in [the User's Guide](#).) Then, if there is a Main macro procedure in the installed user-interface file, it is called. If any Compiled Macro (.MAC) files were specified using the "-l" or "-e" command-line options, the editor then loads and/or executes those macro files. The _ON_EDITOR_STARTUP_ event is called. Finally, the editor is ready to accept user commands, and the editing session is underway. [The TSEStart Macro](#) [TSEStart Macro](#) ----- The TSESTART.MAC macro file is a user-defined external Compiled Macro file that the editor attempts to execute each time the editor is started. It is especially useful for specifying startup actions such as loading custom [macros](#), and for initially setting [variables](#) and [editing modes](#). [The TSEStart macro](#) feature also provides an alternate method for creating versions of the editor that behave differently depending on the directory from which the editor is started. You can include a unique TSEStart macro file in the desired directories. The editor will execute [the TSEStart macro](#) file located in the directory where you start the editor, and perform the functions specified. [The TSELOAD.DAT File](#) ----- [The TSELOAD.DAT file](#) provides a simple, optional mechanism to automatically load macro files at the beginning of each editing session. If TSELOAD.DAT is used, it is maintained in the editor load directory. On startup, the editor loads any macro files found in this autoloading file, using the [LoadMacro\(\)](#) command. The [EditAutoLoadList\(\)](#) command allows you to add macro files to the autoloading file, so they will be automatically loaded each time you start the editor. You can also use this command to delete macro files from the autoloading file.

StartupFlags

Controls how the editor obtains the file specification, or list of files, to process if the editor is started with an empty command line (that is, a command line on which no files or editor options are specified).

[Values:](#) [INTEGER](#) - _STARTUP_PROMPT_ issues the "File(s) to edit:" prompt. - _STARTUP_FILEMGR_ runs F. MAC (FileManager macro), passing it the string "*.*". - _STARTUP_UNNAMED_ executes the [NewFile\(\)](#) command. - _STARTUP_RECENTFILES_ displays the recently-edited-files list. - _STARTUP_PICKLIST_ displays a PickList of files in the current directory. - _STARTUP_RESTORESTATE_ instructs the editor to restore the editing environment from a previous "saved" session. - _STARTUP_MENU_ displays a selection menu of available startup options. [Default:](#) _STARTUP_MENU_ [Limits:](#) Has effect at startup only. Notes: Only one of the above options can be specified at a time.

StartupVideoMode

The video mode the editor will use when the editor is first invoked.

Values: [INTEGER](#) - [_AUTO_DETECT_](#) - [_25_LINES_](#) - [_28_LINES_](#) - [_30_LINES_](#) - [_33_LINES_](#) - [_36_LINES_](#) - [_40_LINES_](#) - [_43_LINES_](#) - [_44_LINES_](#) - [_50_LINES_](#) - [_MAX_LINES_](#) - [_MAX_COLS_](#) - [_MAX_LINES_COLS_](#) **Default:** [_AUTO_DETECT_](#) **Limits:** Has effect at startup only. Notes: [Values](#) for [StartupVideoMode](#) > 255 are considered as cols x rows. For example: // to get the rows and cols, if the value is >= 256: cols = [HiByte\(StartupVideoMode value\)](#) rows = [LoByte\(StartupVideoMode value\)](#) // to set 100 columns by 36 rows: [StartupVideoMode value](#) = cols * 256 + rows The value for [StartupVideoMode](#), for 100 x 36 (cols 100, rows 36), would be expressed as: [StartupVideoMode value](#) = (100 * 256) + 36 = 25636 - [_AUTO_DETECT_](#) causes the editor to use the current video mode - [_25_LINES_](#) forces 25-line mode - [_28_LINES_](#) forces 28-line mode - [_30_LINES_](#) forces 30-line mode - [_33_LINES_](#) forces 33-line mode - [_36_LINES_](#) forces 36-line mode - [_40_LINES_](#) forces 40-line mode - [_43_LINES_](#) forces 43-line mode - [_44_LINES_](#) forces 44-line mode - [_50_LINES_](#) forces 50-line mode - [_MAX_LINES_](#) forces maximum number of lines for current font - [_MAX_COLS_](#) forces maximum number of columns for current font - [_MAX_LINES_COLS_](#) forces maximum number of lines and columns for current font To change the video mode while in the editor, set CurrVideoMode. See Also: [CurrVideoMode](#)

Statements

Statements are the building blocks of procedures, and hence, macros. SAL features a rich assortment of statements, to make macro writing as productive as possible. Statement types are: variable assignments, control structures, and function calls. Function calls are calls to [menus](#), user-definable commands, and native commands. Variable assignments, control structures, and function calls are composed from [expressions](#). Unless specified otherwise by a control statement, statements are executed sequentially as they appear in the macro. [Variable Assignment Statements](#)

----- Variable assignment statements are used to change, or set, a variable to a specified value. They take the form: variable_name = expression This can be read as: "variable_name is assigned the value of expression", or "set variable_name to expression". where: - variable_name is the name of a previously declared variable. - expression is composed of [variables](#) and optional operators. After being evaluated, expression is assigned to variable_name. The expression must be of the same type as the variable. Integer [expressions](#) are assigned to integers, and [string expressions](#) are assigned to [strings](#). After the assignment statement, variable_name has the value of expression. Some simple [examples](#) of assignment [statements](#) are: [integer](#) line [string](#) word[10] line = 10 // line has the value 10 line = line + 5 // line has the value 15 word = "foo" // word has the value "foo" word = word + "bar" // word has the value "foobar" word = [str](#)(line) + " " + word + "s" // word has the value "15 foobars" word = "1000" // word has the value "1000" line = [val](#)(word) * 2 // line has the value 2000 [Expressions](#) ----- Expressions are the building blocks of statements, and are composed of one or more [variables](#), [constants](#), or functions separated by operators. A function is either a user-defined command, menu, or native command. For a given integer procedure foo() and integers m and n, a simple expression would be: n * foo() / m But an even simpler expression is: foo() or just: n [Expressions](#) themselves may be enclosed within parentheses, and are commonly called sub-[expressions](#). An expression can consist of one or more sub-[expressions](#). For the expression: (n + m) / (2 * foo()) (n + m) and (2 * foo()) are both sub-expressions comprising the full expression (n + m) / (2 * foo()). This expression may be used as a sub-expression within a larger expression. For example: 45 + ((n + m) / (2 * foo())) [mod](#) 89 Thus, [expressions](#) and sub-[expressions](#) are basically interchangeable. Expression evaluation is based on operator precedence. The operators with higher precedence are evaluated before those of lower precedence. For example, multiplication has a higher precedence than subtraction. Given three functions, Alpha(), Beta(), and Gamma(), the expression: Alpha() * Beta() - Gamma() will multiply the results of Alpha() and Beta() before subtracting Gamma(). Note, however, that while the expression is evaluated based on operator precedence, the ORDER of evaluation is not specified. In the example above, there is no guarantee that Alpha() is called first, Beta() second, and Gamma() third. The only guarantee is that the results of Alpha() and Beta() will be multiplied before the result of Gamma() is subtracted. The only exception to this rule is "short-circuit" [boolean expressions](#) discussed in "[Logical Operators](#)" under "[Numeric Expressions](#)" below. To avoid ambiguity in the order of evaluation of expressions, it is recommended that parentheses be used to enclose any sub-

[expressions](#) which should be evaluated before others. For instance, if we rewrite the above example as: Alpha() * (Beta() - Gamma()) then Gamma() is subtracted from Beta() before it is multiplied by Alpha(). There are two classes of [expressions](#): numeric and [string expressions](#). These two types of [expressions](#), along with their associated operators, are discussed below. [String Expressions](#) [String expressions](#) are composed of one or more [string variables](#), [string](#) functions, or [string](#) sub-[expressions](#) separated by zero or more [string](#) operators. A function is either a user-defined command, menu, or native command. [String](#) operators are divided into four categories: concatenation, indexing, [string slices](#), and relational. [Concatenation Operators](#) Concatenation is used to join two or more strings into a single string. The concatenation operator is the "+" symbol. For example, given: [string](#) first[15], last[15], fullname[40], initials[3] then: first = "John" // assigns "John" to first last = "Smith" // assigns "Smith" to last fullname = first + " " + last // assigns "John Smith" to // fullname Concatenation is performed after [string](#) indexing (discussed below), but before [string](#) relational operations (also discussed below). The result of string concatenation is a string. (For more information and a complete list of operators and their precedence, see "[Precedence and Order of Evaluation in Expressions](#)" later in this chapter.) [Indexing Operators](#) Indexing is used to reference a character at a specified offset within a [string](#) variable. To get the nth character in a string variable, follow the variable name with [n], where n is a numeric variable or constant between 1 and 255. From the previous example: initials = first[1] + last[1] // assigns "JS" to initials The result of indexing is a string of length 1. (For more information and a complete list of operators and their precedence, see "[Precedence and Order of Evaluation in Expressions](#)" later in this chapter.) [String Slices](#) String slices allow a string variable to be manipulated by specifying a substring of characters within the string. To reference the desired substring, follow the variable name with [notation], where notation indicates the desired starting and ending positions (as numeric expressions) in the format: s[start_position..end_position] or where notation indicates the desired starting position and length (as numeric expressions) in the format: s[start_position:length] [String slices](#) may be used in [expressions](#) and/or assignments. For example, given: [string](#) s[50] = "StarTrek Next Generation DeepSpace 9 Voyager" [string](#) s1[40], s2[40], s3[40], s4[40] then: s1 = s[1:8] // s1 = "StarTrek" s2 = s[1..24] // s2 = "StarTrek Next Generation" s3 = s[26:11] // s3 = "DeepSpace 9" s4 = s[38:7] // s4 = "Voyager" Or given: [string](#) s[30] = "Captain Kirk" [string](#) s1[10] = "James T." then: s[1:[Length](#)(s1)] = s1 // s = "James T. Kirk" [Relational Operators for String Expressions](#) Relational operators are used to compare the relation between string [expressions](#). The [relational operators for string expressions](#) are: < less than <= less than or equal to > greater than >= greater than or equal to == equal to <> not equal to Note that "==" is used to compare equality, while "=" is used to assign [variables](#). Strings are compared by comparing the individual characters of the strings from left to right. Character comparison continues until the end of either [string](#) is encountered. When comparing individual characters, their ASCII [values](#) are used to determine if any one character is greater than, equal to, or less than any other character. In equality comparisons, [strings](#) are considered different if their lengths differ. For example, given: [string](#) s1[10] = "TEST", s2[10] = "TESTY", s3[10] = "test" then: s1 < s2 // is [TRUE](#) because s2 has extra character. s1 == s3 // is [FALSE](#) since [case](#) of letters is different. s1 == s2 // is [FALSE](#) since the lengths are different. s1 < s3 // is [TRUE](#) since ASCII value of "T" (84) is less // than ASCII value of "t" (116). The relational string operators have the lowest precedence of the string operators. The result of a relational string expression is a boolean (numeric) expression and has a value of TRUE or FALSE. (See "[Boolean Expressions](#)" below.) (For more information and a complete list of operators and their precedence, see "[Precedence and Order of Evaluation in Expressions](#)" later in this chapter.) [Numeric Expressions](#) [Numeric expressions](#) are composed of one or more numeric [variables](#), [constants](#), numeric functions, or numeric sub-[expressions](#) separated by zero or more numeric operators. A function is either a user-defined command, menu, or native command. As noted under "[String Expressions](#)" (above), [string expressions](#) composed with the Relational operator are also numeric expressions. Numeric operators are divided into four categories: arithmetic, relational, logical, and bitwise. [Arithmetic Operators](#) [Arithmetic operators](#) are used to mathematically combine two or more [expressions](#). The [arithmetic operators](#) are: * multiplication / division [mod](#) modulus + addition - subtraction Since integers are whole [numbers](#), [integer](#) division truncates any fractional part. The mod operator yields the remainder of integer division. It is zero when there is no remainder. These operators group left to right, with *, /, and [mod](#) having higher precedence than + and -. If you want to force a different precedence, use parentheses. For example, given: [integer](#) line then: line = 4 * 10 + 5 * 6 // the result is (4 * 10) + (5 * 6) = 70 line = 4 * (10 + 5) * 6 // the result is 360 The result of an arithmetic expression is always an

integer between MININT and [MAXINT](#) (editor [constants](#) defined under "[Numbers](#)" earlier in this chapter). (For more information and a complete list of operators and their precedence see "[Precedence and Order of Evaluation in Expressions](#)" later in this chapter.) [Relational Operators for Numeric Expressions](#) Relational operators are used to compare the relation between numeric [expressions](#). The [relational operators for numeric expressions](#) are: < less than <= less than or equal to > greater than >= greater than or equal to == equal to <> [not](#) equal to Note that "==" is used for equality comparison, while "=" is used for variable assignment. Relational operators have lower precedence than [arithmetic operators](#). Because of this, an expression such as: i < limit - 1 is equivalent to: i < (limit - 1) The result of a relational expression is a boolean expression and has a value of [TRUE](#) or [FALSE](#). (See "[Boolean Expressions](#)" below.) For example, given: [integer](#) m = 5, n = 10 then: m < n // is [TRUE](#) m > n // is [FALSE](#) m <> n // is [TRUE](#) m == n // is [FALSE](#) (For more information on [string expressions](#) involving relational operators, see "[Relational Operators for String Expressions](#)" under "[String Expressions](#)" above. For a complete list of operators and their precedence see "[Precedence and Order of Evaluation in Expressions](#)" later in this chapter.) [Logical Operators](#) [Logical operators](#) are used to test whether [expressions](#) are zero or nonzero. [Logical operators](#) are commonly used to generate [boolean expressions](#). (See "[Boolean Expressions](#)" later in this chapter.) The [logical operators](#) are: and [TRUE](#) if two compared [expressions](#) are both nonzero or [TRUE](#) if either or both of two compared [expressions](#) are nonzero [not TRUE](#) if expression is [FALSE](#); [FALSE](#) if expression is [TRUE](#) in [TRUE](#) if expression is [TRUE](#); [FALSE](#) if expression is [FALSE](#) For example, to determine if [Insert](#) is [ON](#) and [WordWrap](#) is [ON](#): [Query](#)([Insert](#)) and [Query](#)([WordWrap](#)) To determine if either [Insert](#) is [ON](#) or [Wordwrap](#) is [ON](#): [Query](#)([Insert](#)) or [Query](#)([WordWrap](#)) To determine if [Insert](#) is [ON](#) and [Wordwrap](#) is [OFF](#): [Query](#)([Insert](#)) and (not [Query](#)([WordWrap](#))) To determine if a variable-width tab is set: [Query](#)([TabType](#)) in _SMART_, _VARIABLE_ The logical operators "[and](#)," "[or](#)," and "[in](#)" have lower precedence than the arithmetic operators and relational operators. But the logical operator "[not](#)" has higher precedence than arithmetic, relational, and bitwise. As a result, an expression such as: Alpha() and not Beta() or Gamma() is equivalent to: (Alpha() and (not Beta())) or Gamma() Unlike arithmetic, relational, and bitwise expressions, the order of evaluation of logical [expressions](#) IS specified. Logical [expressions](#) are evaluated left-to-right. Therefore, in the expression: Alpha() or Beta() Alpha() is called before Beta(). This is done to employ what is known as "short-circuit evaluation." Short-circuit evaluation allows an expression to be evaluated without executing the entire expression. Once the result of a logical expression or sub-expression is known, the evaluation stops. The expression: Alpha() or Beta() is [TRUE](#) if either Alpha() or Beta() returns [TRUE](#). First, Alpha() is called. If it returns a nonzero value, the expression is [TRUE](#) and Beta() is not called. If Alpha() is zero, then Beta() is called to determine if the expression is [TRUE](#) or [FALSE](#). Similarly, in the expression: Alpha() and Beta() Alpha() is called first. If Alpha() is [FALSE](#), Beta() is [not](#) called since its [return](#) value is inconsequential. But if Alpha() is nonzero, Beta() is called. If Beta() is also [TRUE](#), the entire expression is [TRUE](#); otherwise, the expression is [FALSE](#). Thus, once the value of a logical expression or sub-expression is known, evaluation of that expression or sub-expression stops. The result of a logical expression is a boolean expression and has a value of [TRUE](#) or [FALSE](#). (See "[Boolean Expressions](#)" later in this chapter.) (For more information and a complete list of operators and their precedence, see "[Precedence and Order of Evaluation in Expressions](#)" later in this chapter.) The "[in](#)" operator serves as a convenient tool for checking to see if an expression is within a range of values or within a group of ranges (set inclusion). It can be used in place of the "[and](#)" and "[or](#)" operators to simplify such operations. The format of the "[in](#)" operator is: expression in expression_list In this format, expression_list is a list of expressions, separated by commas, in either the form: expression1 where expression1 is any expression; or the form expression2..expression3 where expression2..expression3 indicates a range extending from expression2 to expression3, inclusively. The type (string or integer) of the expressions in expression_list must match that of expression. For example, without the "[in](#)" operator, you would use code similar to the following to determine if a character, represented as s[1], is alphabetic: [if](#) s[1] >= 'a' and s[1] <= 'z' or s[1] >= 'A' and s[1] <= 'Z' // s[1] is alphabetic [endif](#) The code can be simplified using the "[in](#)" operator as: [if](#) s[1] in 'a'..'z', 'A'..'Z' // s[1] is alphabetic [endif](#) With the "[in](#)" operator, you can easily make an isAlpha function: [integer proc](#) isAlpha([string](#) s) [return](#) (s[1] in 'a'..'z', 'A'..'Z') [end](#) or an isDrawingChar function (the PC graphic drawing characters): [integer proc](#) isDrawingChar([string](#) s) [return](#) ([Asc](#)(s[1]) [in](#) 176..223) [end](#) [Bitwise Logical Operators](#) Bitwise operators are used to manipulate individual bits

[illegible]

[Statements](#) ----- The control statements of a language specify the order in which language statements are executed. Among the features of the macro language is its rich set of statements that control the flow of execution. The macro language includes the following types of control statements: [if](#), [elseif](#), [else](#), [endif](#) [iif](#) (used as an expression) [case](#), [when](#), [otherwise](#), [endcase](#) [while](#), [endwhile](#) [repeat](#), [until](#) [for](#), [endfor](#) [do](#), [enddo](#) [loop](#), [endloop](#) [break](#) [return](#) [halt](#) [goto](#)

[Expressions in Control Statements](#) Control statements are governed by expressions. With the exception of the "[case](#)" statement, all [control statements](#) require [numeric expressions](#). These numeric expressions are evaluated for zero and nonzero values. If the result of the expression is nonzero, the statements within the control structure are executed. If the expression is zero, control is passed to the statement following the end of that control structure; or in the case of "[if](#)," to the following "[else](#)" or "[elseif](#)" statement. Since the results of the [expressions](#) are checked for zero or nonzero, any numeric expression is valid. Also, with the exception of the "[case](#)" statement, strings must be used within relational string expressions which yield a numeric expression. In the following control statement syntax declarations, [numeric_expression](#) is any numeric expression. [statements](#) are any [statements](#) as specified in the "[Statements](#)" section.

[Conditional Execution ... if and iif](#) To execute a group of statements when a condition (expression) is nonzero, use the "[if](#)" statement. The format is: [if](#) [numeric_expression](#) [statements](#) [elseif](#) [numeric_expression](#) [statements](#) [elseif](#) [numeric_expression](#) [statements](#) [else](#) [statements](#) [endif](#) The "[elseif](#)" and the "[else](#)" portions are optional. The "[elseif](#)" statement may be used as many times as desired within an "[if](#)" construct, but only one "[else](#)" statement may be used. The [numeric_expressions](#) are evaluated until one is encountered whose value is nonzero (that is, the condition is TRUE). Once a nonzero numeric expression is found, the statements following the expression are executed [until](#) "[elseif](#)", "[else](#)", or "[endif](#)" is encountered. At that point, control is passed to the statement following "[endif](#)". If a nonzero [numeric_expression](#) is NOT found, control is passed to the first statement following "[else](#)". The [statements](#) following "[else](#)" are executed until "[endif](#)" is encountered. Control is then passed to the statement following "[endif](#)". If a nonzero [numeric_expression](#) is NOT found and no "[else](#)" condition is specified, control immediately passes to the statement following "[endif](#)". Conditional [statements](#) may also be nested. The following example demonstrates "short-circuit boolean evaluation." Given the procedures Alpha() and Beta(): [if](#) Alpha() [and](#) Beta() [statements](#) [endif](#) could also be written: [if](#) Alpha() [if](#) Beta() [statements](#) [endif](#) [endif](#) A special case of the "[if](#)" statement is the "inline if", "[iif](#)". The "inline if" allows an "[if](#)" statement to be nested within an expression. The "[iif](#)" statement [syntax](#) is: [iif](#)([expression1](#), [expression2](#), [expression3](#)) If [expression1](#) is nonzero, "[iif](#)" returns [expression2](#). [Otherwise](#), "[iif](#)" returns [expression3](#). For example: [proc](#) OnOff([integer](#) n) [if](#) n <> 0 [Message](#)("On") [else](#) [Message](#)("Off") [endif](#) [end](#) could also be written more concisely as: [proc](#) OnOff([integer](#) n) [Message](#)([iif](#)(n <> 0, "On", "Off")) [end](#) [expression1](#) must be a numeric expression. [expression2](#) and [expression3](#) may be either [string](#) or [numeric expressions](#). However, they must both be of the same type. Since "[iif](#)" is an expression (whereas "[if](#)", "[else](#)", and "[endif](#)" are a series of statements), it can be used in cases where an expression is allowed, but where [statements](#) are [not](#) allowed. For example: [while](#) [iif](#)([expr1](#), [expr2](#), [expr3](#)) ... [endwhile](#) The "[while](#)" statement requires a [boolean_expression](#) which the "[iif](#)" will satisfy. If [expr1](#) returns nonzero, then [expr2](#) is evaluated. If [expr2](#) is nonzero, then the statements within the "[while](#)" loop will be repeated; otherwise, control passes to the statements after the "[endwhile](#)". If [expr1](#) returns zero, then [expr3](#) is evaluated. If [expr3](#) is nonzero, then the statements within the "[while](#)" loop will be repeated; otherwise, control passes to the statements after the "[endwhile](#)". Continuation of the "[while](#)" loop depends on the outcome of [expr2](#) or [expr3](#) as determined by [expr1](#). The "[iif](#)" statement should only be used to assign a variable, or as part of an expression. It is not intended that "[iif](#)" be used as a stand-alone statement.

[Mega-Conditional Statement ... case](#) The "[case](#)" statement is used as a mega-conditional statement. It compares the results of an expression to many other expressions without the need to re-evaluate the original expression multiple times. The "[case](#)" statement [syntax](#) is: [case](#) [case_expression](#) [when](#) [when_expression1](#) [statements](#) [when](#) [when_expression2](#) [statements](#) [when](#) [when_expression3](#) [statements](#) ... [otherwise](#) [statements](#) [endcase](#) The "[case](#)" statement is used when a single expression is to be tested for multiple values. Note that a "[case](#)" statement can always be expressed as an "[if](#)" statement; however, it is sometimes more convenient and clearer to use the "[case](#)" statement. The [case_expression](#) and the [when_expression\(s\)](#) must be the same type. Thus, the [case_expression](#) may be either an integer or a [string](#). The "[when](#)" and "[otherwise](#)" portions are optional, but it really does [not](#) make much sense to have a case without at least a few "[when](#)" clauses! Within a "[case](#)" statement, once a

when_expression is true and the statements assigned to the "when" are executed, processing proceeds directly to "endcase". The processing does not "fall through" to the next "case" statement. Example: `string fruit_name[10] ... statements ... Upper(fruit_name) case fruit_name when "APPLE" Fruit(1) // After Fruit(1) is executed, control // passes to statements after endcase when "ORANGE" Fruit(2) when "PEAR" Fruit(3) when "MELONS", "HONEYDEW" Fruit(4) otherwise Fruit(0) // Fruit(0) is executed if none of the // previous cases match. endcase` "When" expressions can have a range of values (the beginning and ending values being separated by ".."), or a group of values separated by commas. For example: `integer a case a when 1..10 // if the value of a is in the range // of 1 to 10 statements when 15, 20, 25 // if the value of a is 15, 20, or 25 statements endcase` Iterative Statements ... while, repeat, for, do, and loop Iterative statements are used when a statement needs to be repeatedly executed as long as a condition (expression) is nonzero. SAL offers five types of iterative statements: while/endwhile, repeat/until, for/endfor, do/enddo, and loop/endloop. The "while" statement executes a sequence of statements as long as the specified expression is nonzero. The "while" statement syntax is: `while numeric_expression statements endwhile` Statements are repeatedly executed while numeric_expression is nonzero. If numeric_expression is zero when first encountered, no statements within "while... endwhile" are executed. Thus, to ensure that statements are executed at least once, numeric_expression must be nonzero when first evaluated. Once numeric_expression evaluates to zero, control passes to the statement following "endwhile". The "repeat" statement executes a sequence of statements at least once and continues to repeatedly execute these statements until numeric_expression is nonzero. The "repeat" statement syntax is: `repeat statements until numeric_expression` When "repeat" is encountered, the statements which follow are immediately executed. When "until" is encountered, numeric_expression is evaluated. If nonzero, control passes to the statement following "until". Otherwise, as long as numeric_expression is zero, control is passed back to "repeat" and the statements are executed again until a nonzero numeric_expression is encountered. The "for" and "do" loops offer another method of control when executing a loop, by allowing you to explicitly indicate the number of times a loop is to be repeated. The "for" statement executes a sequence of statements a selected number of times, as indicated by a range specified as numeric_expression through numeric_expression2. The "for" statement syntax is: `for integer_variable = numeric_expression to/downto numeric_expression2 [by numeric_expression3] statements endfor` Specify "to" or "downto" to indicate whether the value of numeric_expression is to be incremented ("to") or decremented ("downto") until it reaches (inclusive) the value of numeric_expression2. The value will be incremented/decremented by 1, unless you specify an alternate "by" numeric_expression3 value. Once the "for" loop terminates, control passes to the statement following "endfor". For example: `integer i for i = 1 to NumFiles() NextFile() endfor` In this example (above), the statements inside the "for" loop are executed the value of NumFiles() times. "i" will range in value from 1 through the value of NumFiles(), inclusive. The above "for" loop may also be written as: `integer i for i = NumFiles() downto 1 NextFile() endfor` If written this way, the statements inside the "for" loop will still be executed the value of NumFiles() times, with "i" ranging in value from the value of NumFiles() through 1, inclusive. Note the use of "downto" instead of "to". "downto" is used whenever the loop index variable should be decremented rather than incremented. In the following example, the statements inside the "for" loop are not executed, because the initial value of "i" (9) is greater than the controlling expression of the "for" loop (8, which is the length of "s"). `integer i string s[20] s = "StarTrek" for i = 9 to Length(s) Warn("Processing char:", s[i]) endfor` "for" loops are actually a specialized form of the "while" loop. Any "for" loop can be rewritten as a "while" loop. (The converse is not necessarily true.) The "for" loop, when used correctly, can help make macros more understandable. For example, the following code, written as a "while" loop: `integer i string s[20] s = "StarTrek" i = 1 while i <= Length(s) Warn("Processing char:", s[i]) i = i + 1 endwhile` can be rewritten as a "for" loop: `integer i string s[20] s = "StarTrek" for i = 1 to Length(s) Warn("Processing char:", s[i]) endfor` Both of these code samples produce the same results. However, the "for" loop version is clearer as to the objective: do something for all the characters in "s". The "do" statement executes a sequence of statements a specified number of times, as indicated by the value of numeric_expression. The "do" statement syntax is: `do numeric_expression times statements enddo` Once the "do" loop terminates, control passes to the statement following "enddo". In the following example, the statements inside the "do" loop are executed 8 times: `string s[20] s = "StarTrek" do Length(s) times // do something here enddo` In the next example, the statements inside the "do" loop are

not executed, because "i" is 0 (zero): [integer](#) i i = 0 [do](#) i [times](#) // do something here [enddo](#) The "[do](#)" loop is a specialized form of the "[for](#)" statement. Any "[do](#)" loop can be rewritten as a "[for](#)" statement. (The converse is not true.) The "[do](#)" loop should be used when a group of statements are to be executed [numeric_expression](#) times, but there is no need to know the current iteration number or count of the loop. The "[loop](#)" statement repeatedly executes [statements](#) until "[break](#)", "[return](#)", or "[halt](#)" is encountered. The "[loop](#)" statement [syntax](#) is: [loop](#) [statements](#) [endloop](#) Because there is no [numeric_expression](#) evaluated to determine when to pass control to the statement after "[endloop](#)", it is imperative that "[break](#)", "[return](#)", or "[halt](#)" be used to terminate the "[loop](#)" instruction. (See "[Termination Statements ... break, return, and halt](#)" below.) In summary, the statements in a "[while](#)-endwhile" loop are repeated 0 or more times, while the controlling condition is nonzero. The statements in a "[repeat](#)-until" loop are repeated 1 or more times, until the controlling condition is nonzero. The statements in a "[for](#)-endfor" loop are repeated a specific number of times, based on a range of values. The statements in a "[do](#)-enddo" loop are repeated a specified number of times, as indicated by the value of a numeric expression. Finally, the statements in a "[loop](#)-endloop" loop are repeated until terminated by either the "[break](#)", "[return](#)", or "[halt](#)" [statements](#). [Termination Statements ... break, return, and halt](#) Termination statements are used to forcefully change the flow control of [statements](#). There are three termination [statements](#): [break](#), [return](#), and [halt](#). The break statement is used to pass control to the statement following the end of the iterative control structure. The "[break](#)" statement [syntax](#) is: [break](#) The "[break](#)" statement immediately terminates execution of looping [statements](#). It is generally intended for use with the "[loop](#)" statement, but can also be used to terminate "[while](#)", "[repeat](#)", "[for](#)", and "[do](#)" loops. The "[return](#)" statement is used to terminate a procedure. The "[return](#)" statement [syntax](#) is: [return](#)() The "[return](#)" statement immediately terminates not only the execution of looping statements, but also the currently executing macro procedure, and returns control to the caller, which could be another macro. If the procedure from which to return is a string procedure, the parameters passed to "[return](#)" must be a string value. Similarly, to return from an "[integer](#)" procedure, pass an integer value to "[return](#)". If no return type is specified for a macro, simply use "[return](#) ()". Example: [integer proc](#) Multiply([integer](#) a, [integer](#) b) [return](#)(a * b) [end](#) ... [string](#) first[] = "Victoria" [string](#) last[] = "Lee" [string proc](#) FullName() [return](#)(first + " " + last) [end](#) ... [proc](#) Test([integer](#) valid) [if not](#) valid [return](#)() [endif](#) [Message](#) (valid, " is valid") [end](#) The "[halt](#)" statement terminates all macros and passes control back to the editor. "[Halt](#)" is intended for use only while [debugging macros](#). The "[halt](#)" statement [syntax](#) is: [halt](#) The "[halt](#)" statement immediately terminates not only the execution of the current macro, but ALL [macros](#). And because all macro control is terminated, the state of the editor is unknown. As a result, it should only be used when [debugging macros](#). Any macro to be given to another user should [not](#) use "[halt](#)" as a method of termination. [Jump Statement ... goto](#) To direct flow which does not easily fit into a construct mentioned above, the "[goto](#)" statement can be used. It is recommended that the "[goto](#)" statement be used sparingly, as it can make [debugging macros](#) more difficult. The syntax of the "[goto](#)" statement is: [goto](#) label where: - label is defined as an identifier that ends with a colon (":"). Control is passed immediately to the statement following the label specified in the "[goto](#)" statement. For example: some_label: [statements](#) [goto](#) some_label

[Function Call Statements](#) ----- The function call statement is one of the most often used statements in the language. The format is: function_name() Control is passed to the named function (which can be a native command, a macro procedure, or a menu). When the called function is complete, control returns to the statement immediately following the function call statement. [The editor features](#) over 400 native commands that can be called to give macros the ability to perform extensive text manipulation. You can also define your own commands or menus within macros, and call them just as the native commands are called. Functions may also [return values](#): variable_name = function_name() Functions that return values may be used in expressions. For example, the native command [CurrLine](#)() returns the line number of the current line. To determine if the cursor is positioned at the first line of the current file, you could use a format such as the following: [if](#) [CurrLine](#)() == 1 [statements](#) [endif](#) The function call statement can occur anywhere that any other statement can occur. (For more information and descriptions of the native editor commands, see the chapter on "[Editor Commands](#).")

StatusLineAtTop

Causes

[the StatusLine](#) to display at the top or bottom of the monitor screen. **Values:** BOOLEAN **Default:** [ON](#) **Notes:** [The HelpLine](#) (if active) appears at the location that is not chosen for [the StatusLine](#). Thus, if [the StatusLine](#) appears at the top of the screen, [the HelpLine](#) appears at the bottom of the screen, and vice versa. See Also: [ShowHelpLine](#), [ShowStatusLine](#), [StatusLineUpdating](#), [StatusLineAttr](#), [StatusLineFillChar](#), [StatusLineRow](#)

StatusLineFillChar

This sets the character that displays in the unused areas of

[the StatusLine](#). **Values:** [INTEGER](#) - [0..255] The [integer](#) value of any ASCII character **Default:** [Asc\(" "\)](#) or 32. **Notes:** Defines the character to use for filling any unused area of the StatusLine. Since the [Asc\(\)](#) command returns the ASCII value for the specified character, the following two assignments are equivalent: [StatusLineFillChar](#) = [Asc\(" "\)](#)
[StatusLineFillChar](#) = 32 See Also: [StatusLineAttr](#)

StatusLineRow

The row coordinate of

[the StatusLine](#), relative to the full display area. **Values:** [INTEGER](#) **Default:** Undefined **Limits:** Can only be queried. Cannot be used within a config/endconfig construct. **Notes:** If StatusLineAtTop is ON, the value of StatusLineRow will always be 1; otherwise, the value of StatusLineRow will be ScreenRows. See Also: [WindowY1](#), [WindowRows](#), [ScreenRows](#), [PopWinY1](#), [StatusLineAtTop](#)

StatusLineUpdating

Determines whether the editor updates

[the StatusLine](#). **Values:** BOOLEAN **Default:** [ON](#) **Notes:** If StatusLineUpdating is OFF, the editor will not update any information on [the StatusLine](#). The `_AFTER_UPDATE_STATUSLINE_` event will continue to be called, however, each time the editor would have updated [the StatusLine](#) (see the [Hook\(\)](#) command). See Also: [ShowHelpLine](#), [ShowStatusLine](#), [StatusLineAttr](#), [StatusLineFillChar](#), [StatusLineRow](#) **Commands:** [Hook\(\)](#)

Str

Converts an integer expression to a string.

Syntax: [STRING](#) [Str](#)([INTEGER](#) n [, [INTEGER](#) base]) - n is the integer expression to be converted to a string. - base is an optional parameter that specifies the number base to use, which defaults to 10 if [not](#) specified. **Returns:** The string equivalent of the passed integer expression. **Notes:** This command converts n to the specified base and returns a string containing the converted number. Leading zeros are not included in the returned string. If the number is negative, a leading minus sign is included in the returned string. The length of the returned string is the minimum length required to contain the converted number; trailing spaces are not added. **Examples:** A macro to prompt the user for a new RightMargin - the old RightMargin value is used as the default. `proc mSetRightMargin() string right_margin[4] = Str (Query(RightMargin)) if Ask("Right Margin:", right_margin) and Length(right_margin) Set(RightMargin, Val`

(right_margin)) [endif](#) [end](#) See Also: [Val\(\)](#), [Format\(\)](#)

StrCount

Returns the number of occurrences of one string within another string.

Syntax: [INTEGER](#) [StrCount](#)([STRING](#) needle, [STRING](#) haystack) - needle is the string to search for. - haystack is the string in which to search. Returns: The number of times needle occurs in haystack. **Examples:** [string](#) s[80] = "Beam me up, Scotty!" [integer](#) n n = [StrCount](#)("e", s) // n will be 2 n = [StrCount](#)("up", s) // n will be 1 n = [StrCount](#)("spock", s) // n will be 0 See Also: [Pos\(\)](#), [StrFind\(\)](#), [StrReplace\(\)](#)

StrFind

Returns the starting position of one string within another one.

Syntax: [INTEGER](#) [StrFind](#)([STRING](#) needle, [STRING](#) haystack [, [STRING](#) options [, [INTEGER](#) start [, [VAR](#) [INTEGER](#) len]]]) - needle is the search for string. - haystack is the string to be searched. - options is optional. If specified, options control the way needle is interpreted and searched for. Valid options are: b i w x ^ \$ See the [Find\(\)](#) command for a description of these options. - start is optional. If specified, start denotes the occurrence of needle at which to start. - len is optional. Len is passed back to the caller as the length of the found text. Returns: The position that needle occurs in haystack, if successful; zero ([FALSE](#)) on failure. Notes: [StrFind\(\)](#) uses the same mechanism as the editors [IFind\(\)](#) command. Therefore, most of the [Find\(\)](#) options are available to [StrFind\(\)](#). See the [Find\(\)](#) command for more information. **Examples:** // last 6 digits could be any number [string](#) haystack[30] = "B0000123456.msg" [if](#) [StrFind](#)(haystack, "B0000.*\n.msg", "x") ... See Also: [StrReplace\(\)](#), [Pos\(\)](#), [StrCount\(\)](#), [Find\(\)](#), [IFind\(\)](#)

Strings

[Constant strings](#) consist of up to 255 characters enclosed within either double quotes (") or single quotes ('). If double quotes begin a constant string, double quotes must be used to end the string. Similarly, if single quotes begin a constant string, single quotes must be used to end the string. **Examples:** "This is a string" 'Hello, world' To include the single quote character in a string, use double quotes at the beginning and end. For example: "Isn't this a good example?" To include the double quote character in a string, use single quotes at the beginning and end. For example: 'The word "misspelled" is misspelled'

StrReplace

Replaces occurrences of one

[string](#) within another [string](#). **Syntax:** [STRING](#) [StrReplace](#)([STRING](#) needle, [STRING](#) haystack, [STRING](#) replace [, [STRING](#) options [, [INTEGER](#) start]]) - needle is the search for string. - haystack is the string to be searched. - replace is the string that replaces needle. - options is optional. If specified, options control the way needle is interpreted and searched for. Valid options are: b i w x ^ \$ and any number, to limit the number of replacements to that number. See the [Replace\(\)](#) command for a description of these options. - start is optional. If specified, start denotes the occurrence of needle at which to start. Returns: The updated [string](#). Notes: [StrReplace\(\)](#) uses the same mechanism as the editors [IReplace\(\)](#) command. Therefore, most of the [Replace\(\)](#) options are available to [StrReplace\(\)](#). See the [Replace\(\)](#) command for more information. **Examples:** [string](#) s1[30] = "abc abc abc" [string](#) s2[30] s2 = [StrReplace](#)(s1, "abc",

"def") // s2 is now: "def def def" See Also: [StrFind\(\)](#), [Pos\(\)](#), [StrCount\(\)](#), [Replace\(\)](#)

SubStr

Returns a portion of a string.

Syntax: [STRING](#) SubStr([STRING](#) s, [INTEGER](#) start, [INTEGER](#) len) - s is the string from which the substring is to be retrieved. - start is the position at which to begin retrieving characters. - len is the number of characters to retrieve. Returns: The [string](#) portion requested. Notes: The [string](#) returned will be shorter than len if start plus len is longer than s. It will be a zero-length string if the combination of start and len make it impossible to retrieve any character. **Examples:** Given: [string](#) abc[11] = "catastrophe" then: [SubStr](#)(abc,1,3) //returns "cat" See Also: [Pos\(\)](#), [DelStr\(\)](#), [InsStr\(\)](#), [LeftStr\(\)](#), [RightStr\(\)](#), [StrCount\(\)](#), [StrFind\(\)](#), [StrReplace\(\)](#)

Summary of Mouse Operations

Mouse Pointer

Block

Marking Mouse Button Location _____ Word Click
<LeftBtn> 2 times Word in any window Single line Click <LeftBtn> 3 times Text in any window Multiple lines Drag
<Alt LeftBtn> Text in any window Column Drag <Ctrl LeftBtn> Text in any window Characters Drag <LeftBtn> Text
in any window _____ [Command Prompts](#)

Mouse Pointer and PickLists Mouse Button Location _____
_____ Cancel/remove any Click <RightBtn> Anywhere on screen Prompt or PickList OR Click
<LeftBtn> Outside Prompt or PickList Accept StatusLine Click <LeftBtn> Anywhere in Prompt or [Prompt Box](#) entry
Select from PickList Click or Drag PickList or [menu](#) item or [Selection Menu](#) <LeftBtn>

_____ Mouse Pointer [Windows](#) Mouse
Button Location _____ Open horizontal Click
<LeftBtn> 2 times Left window border Open vertical Click <LeftBtn> 2 times Top window border Change window
Click <LeftBtn> New window Change window and Hold <LeftBtn> Text of new window position [cursor](#) Resize
horizontal Drag <LeftBtn> Top window border Resize vertical Drag <LeftBtn> Left window border Zoom Click
<LeftBtn> [^] symbol Unzoom Click <LeftBtn> [v] symbol Close Click <LeftBtn> [*] symbol

_____ [Cursor](#) Movement and Scrolling

Mouse Button Mouse Pointer Location _____
Position [cursor](#) Click <LeftBtn> Text in current window Scroll down: 1 line Click <LeftBtn> v symbol multi lines
Hold <LeftBtn> v symbol 1 page Click <LeftBtn> Scrollbar below vertical * multi pages Hold <LeftBtn> Scrollbar
below vertical * Alternatively, by lines/pages Drag <LeftBtn> On vertical * Scroll up: 1 line Click <LeftBtn> ^
symbol multi lines Hold <LeftBtn> ^ symbol 1 page Click <LeftBtn> Scrollbar above vertical * multi pages Hold
<LeftBtn> Scrollbar above vertical * Alternatively, by lines/pages Drag <LeftBtn> On vertical * Scroll left: 1 column
Click <LeftBtn> < symbol multi columns Hold <LeftBtn> < symbol 1 [TabWidth](#) Click <LeftBtn> Scrollbar left of
horiz. * multi TabWidths Hold <LeftBtn> Scrollbar left of horiz. * Alternatively, by columns Drag <LeftBtn> On
horizontal * Scroll right: 1 column Click <LeftBtn> > symbol multi columns Hold <LeftBtn> > symbol 1 [TabWidth](#)
Click <LeftBtn> Scrollbar right of horiz. * multi TabWidths Hold <LeftBtn> Scrollbar right of horiz. * Alternatively,
by columns Drag <LeftBtn> On horizontal *

_____ [Menus](#) Mouse Button Mouse Pointer
Location _____ Display [main Menu](#) Click

<LeftBtn> StatusLine or HelpLine Select [Menu](#) item Click <LeftBtn> [Menu](#)/sub-[menu](#) item Alternatively, Display [main Menu](#) Drag <LeftBtn> StatusLine or HelpLine and Select [Menu](#) item Release [Menu](#) item Exit [Menu](#) Click <RightBtn> Anywhere on screen OR Click <LeftBtn> Outside [Menu](#) Pop up Mouse [menu](#) Click <RightBtn> Anywhere on screen Select Mouse [menu](#) Click <LeftBtn> Mouse [menu](#) item Alternatively, Pop up Mouse [menu](#) Drag <RightBtn> Anywhere on screen and Select Mouse item Release Mouse [menu](#) item Exit Mouse [menu](#) Click <RightBtn> Anywhere on screen OR Click <LeftBtn> Outside Mouse [menu](#)

Summary of New/Enhanced Features in the Semware Editor Professional V4.0

```
+-----+
|   In version 4.0 of The SemWare Editor Professional/32, a number of new   |
|   features have been added, and some existing features have been modified. |
+-----+
```

The online HelpSystem has been updated to provide you with information

[about](#) enhancements. The following summary can help you access much of the updated information in [the HelpSystem](#). This summary information is available from [the HelpSystem](#) "[Table of Contents](#)" under the topic "What's New in Version 4.0". (See the WHATSNEW.DOC file included in The SemWare Editor Professional/32 software for any information [not](#) included in this help.) Also see: What's New in Version 3.0

```
+-----+ | New features |
+-----+ +-----+ | All new GUI
version | +-----+ - All the benefits of a GUI version, including: - - Flexible Window
resizing. - Selection of various fonts, and font sizes. - Functional close button. - Drag and Drop file support. - Built-in
support for the Mouse Wheel. - Flexible startup options, regarding Window size, position, and font. - Enhanced
keyboard handling, especially for non-english language users. - A vertical 'bar' (line) cursor is available, by setting the
appropriate cursor size (InsertCursorSize and/or OverwriteCursorSize) to 9. - An associated Console version. Some
features of the GUI version are not available in the Console version. - Highly compatible with version 3.0. - Edit
longer line lengths (up to 16,000) - Enhanced printing. Printing to the Windows printer driver using the current editing
window display font, or by using a separate, settable printer font. See PrintUseScreenFont and SetPrinterFont\(\).
+-----+ | New user-level commands |
+-----+ Edit Key Macro on the Macro menu (Keyboard Macros).
Allows editing of any currently loaded keyboard macros. SaveAs... on the File Menu Now calls the new
RenameAndSaveFile\(\) command, which is a more "Windows-like" SaveAs command. The editor's SaveAs\(\)
command is still available from the Block menu if "Write to File" is selected, when no block is currently marked. Grep
in Files on the Search Menu Calls the editor's grep macro. Calculator on the Util menu. Calls the editor's expr macro.
ToggleMaximizedWindow (<F11>) This command toggles the main editing window between the maximum possible
screen size, and the previous window size. In the GUI version, the editing window is also maximized.
+-----+ | Miscellaneous enhancements |
+-----+ - New Command-Line Option: -y to load a file (or files) in
Browse mode Configurable "Tab Shifts Block" feature (see TabShiftsBlock). Mark a column block by "right click and
dragging" the mouse. New speedkey macro (available from the Potpourri). Accelerates the rate of the cursor
movement keys. New Whtspc macro (available from the Potpourri). WHTSPC adds a few handy visual elements, such
as whitespace, bookmarks, a cursorline indicator, and a right margin indicator. Each of these can be toggled on/off
from a handy menu. Enhanced cmpfiles macro (available from the Potpourri), which now allows editing and then
resuming the compare. Additional numeric syntax highlighting option added to support Fortran.
+-----+ | EDITOR COMMANDS |
+-----+ - The following new commands are available. - Font
```


commands (related to the editor display): [ChooseFont\(\)](#), [GetCharWidthHeight\(\)](#), [GetFont\(\)](#), [ResizeFont\(\)](#), [SetFont\(\)](#) - Printing commands: [SetPrinterFont\(\)](#) [Macros](#) (Compiled): [isGUI\(\)](#) - File commands: [RenameAndSaveFile\(\)](#) - Miscellaneous commands: [UndoMode\(\)](#) - Mouse commands: [GetWheelScrollLines\(\)](#) - Video commands: [PutOemStrXY\(\)](#) - The following editor command is now obsolete: [NoSound\(\)](#)

+-----+ | [EDITOR VARIABLES](#) |

+-----+ - The following new [editor variables](#) are now available: - [GuiStartupFlags](#) - [FontFlags](#), [FontName](#), [FontSize](#), [SpecialEffects](#) - [PrintUseScreenFont](#), [PrinterFontFlags](#), [PrinterFontName](#), [PrinterFontSize](#) - [TabShiftsBlock](#) - The following [editor variables](#) are now obsolete: [PrintAddFF](#), [PrintDevice](#), [PrintInit](#), [PrintUseFF](#) - Appendices of [the Macro Reference Guide](#) in the on-line help have been revised in accordance with additions and enhancements made in version 4.0 of The SemWare Editor Professional/32. [APPENDIX C: COMMANDS BY CATEGORY](#) [APPENDIX D: VARIABLES BY CATEGORY](#) - Appendices of [the User's Guide](#) in the on-line help have been revised in accordance with additions and enhancements made in version 4.0 of The SemWare Editor Professional/32. [APPENDIX C: STANDARD KEY ASSIGNMENTS](#) [APPENDIX F: STANDARD CONFIGURATION SETTINGS](#) Also see: What's New in Version 3.0

Summary of New/Enhanced Features in the Semware Editor Professional V4.2

+-----+
| In version 4.2 of The SemWare Editor Professional, a number of new |
| features have been added, and some existing features have been modified. |
+-----+

The online HelpSystem has been updated to provide you with information

[about](#) enhancements. The following summary can help you access much of the updated information in [the HelpSystem](#). This summary information is available from [the HelpSystem "Table of Contents"](#) under the topic "What's New in Version 4.2". (See the WHATSNEW.DOC file included in The SemWare Editor Professional software's DOC directory for any information [not](#) included in this help.) Also see: What's New in Version 4.0

+-----+ | [EDITOR COMMANDS](#) |

+-----+ - The following new commands are available. - Block commands (clipboard): [PasteReplace\(\)](#), [PasteReplaceFromWinClip\(\)](#) - [Buffers](#) and Files: [EditThisFile\(\)](#), [CopyFile\(\)](#), [MkDir\(\)](#), [MoveFile\(\)](#), [RmDir\(\)](#) [Syntax](#) Highlighting: [GetSynLanguageType\(\)](#), [GetSynMultiLnDlmt\(\)](#), [GetSynToEOL\(\)](#) - Display Colors: [SetColorTableValue\(\)](#), [GetColorTableValue\(\)](#) - Changes to Commands. - New handling for [Dos\(\)](#) command. The editor now tries [to](#) keep the dos window/command prompt open until <enter> is pressed.

+-----+ | [EDITOR VARIABLES](#) |

+-----+ - The following new [editor variables](#) are now available: -

[BackupPath](#), [CursorTabWidth](#), [DetabOnLoad](#), [EntabOnSave](#) +-----+

| New [Key](#)/Mouse Assignments/[Menus](#) | +-----+ - Changed <shift LeftBtn> to mark a block from the cursor position to the mouse pointer. - If CUAMark (available from the Potpourri) is loaded, <Alt> can be added to the shift-cursor keys to mark a column block. - 'Compile Menu' added to the Macro Menu. +-----+ | Other Changes |

+-----+ - New Color Syntax Highlighting files for the following languages: FORTRAN, Javascript, Latex, PHP, Ruby, SAS, SQL, tcl, Tex. - New "twokey" display on <ctrl k>, <ctrl o>, and <ctrl q>. To use: <leadin-[key](#)><helpline> "[menu](#):string1;string2;stringn;" Example: <Ctrl j><HelpLine> "[menu](#):{B} Up to Equal Indent;" + "{E} Down to Equal Indent;" + "{L} [Goto](#) Line;" + "{C} [Goto](#) Column;{A} [Goto](#) Byte;" - Increased the maximum number of windows from 9 to 20. - CUAMark now optionally handles [windows](#)-like non-persistent mouse [marking](#). - In the picklist: can now remove (empty) directories new [menu](#) commands:

Create/Save File Here Create Directory - FindAndDo changes: Add Find & Keep File option. Will unload files where the find [string](#) is _not_ found. Add Find & Quit File option. Will unload files where the find [string](#) _is_ found. Add Find & Apply Macro. Allows an arbitrary macro to be run against each found [string](#). Add Find & Apply Key. Allows an arbitrary key to be pressed against each found string. Key could be assigned to a keyboard macro, [for](#) instance. - Find and FindReplace allow longer search/replace [strings](#). In regular expression mode, more groups and classes can be used. - Appendices of [the Macro Reference Guide](#) in the on-line help have been revised in accordance with additions and enhancements made in version 4.20 of The SemWare Editor Professional. [APPENDIX C: COMMANDS BY CATEGORY](#) [APPENDIX D: VARIABLES BY CATEGORY](#) Also see: What's New in Version 4.0

Summary of New/Enhanced Features in the Semware Editor Professional V4.4

```
+-----+
|   In version 4.4 of The SemWare Editor Professional, a number of new   |
|   features have been added, and some existing features have been modified. |
+-----+
```

The online HelpSystem has been updated to provide you with information

[about](#) enhancements. The following summary can help you access much of the updated information in [the HelpSystem](#). This summary information is available from [the HelpSystem](#) "[Table of Contents](#)" under the topic "What's New in Version 4.4". (See the WHATSNEW.DOC file included in The SemWare Editor Professional software's DOC directory for any information [not](#) included in this help.) Also see: What's New in Version 4.2

```
+-----+ | EDITOR COMMANDS |
+-----+ - The following new "User Level Built-in Commands" are
available. - Blocks and Deleting Text commands: DelCharOrBlock\(\), DelAnyCharOrBlock\(\), MarkBlockBegin\(\),
MarkBlockEnd\(\) - Editing and Deleting Text commands: DelAnyChar\(\) Movement commands: BegLineTog\(\),
EndLineTog\(\), ScrollToBottom\(\) - The following new "Macro Language commands" are available. - Keyboard
commands: isKeyAssigned\(\), WaitForKeyPressed\(\) - Macros (compiled) commands: isAutoLoaded\(\) - OS commands:
StartPgm\(\) - String commands: LeftStr\(\), StrCount\(\), StrFind\(\), StrReplace\(\), VersionStr\(\) - Strings - Clipboards
commands: GetStrFromWinClip\(\) - Enhancements to the following existing Commands. - CopyAppendToWinClip\(\) -
Optional text can be specified in lieu of using a marked block. - CopyToWinClip\(\) - Optional text can be specified in
lieu of using a marked block. - ExecMacro\(\) - Now accepts an optional "Public" procedure (to be executed) that is
contained in the specified macroname. - MsgBox\(\) - Alternate format that allows you to specify the "String" that will
be displayed, without having to specify a "Title". - WhichOS\(\) - LINUX\_ constant has been added.
```

```
+-----+ | EDITOR VARIABLES |
+-----+ - The following new Editor Variables are now available: -
KeepWinOnTop, LineNumbersAttr, ShowLineNumbers, SingleInstance, Transparency - Enhancements to the
following existing Editor Variable. - SpecialEffects - \_CENTER\_POPUPS\_ flag is now available. If specified, the
editor will attempt to center all popup prompts and menus. - The following editor variables are now obsolete:
AltBlockAttr, AltCurrWinBorderAttr, AltCursorAttr, AltCursorInBlockAttr, AltEOFMarkerAttr, AltHelpBoldAttr,
AltHelpInfoAttr, AltHelpItalicsAttr, AltHelpLinkAttr, AltHelpSelectAttr, AltHelpTextAttr, AltHiLiteAttr,
AltMenuBorderAttr, AltMenuGrayAttr, AltMenuSelectAttr, AltMenuSelectGrayAttr, AltMenuSelectLtrAttr,
AltMenuTextAttr, AltMenuTextLtrAttr, AltMsgAttr, AltOtherWinBorderAttr, AltStatusLineAttr, AltTextAttr, AttrSet,
StartupAttrSet +-----+ | New Key Assignments/Menus/Macro
Enhancements | +-----+ - Single Instance of the Editor can be
adjusted interactively by accessing the editor's Main-Menu then Options->Full-Configuration->System/File Options. -
Center Popups can be adjusted interactively by accessing the editor's Main-Menu then Options->Full-Configuration-
```

>[Display/Color Options](#). - ShowLineNumbers can be adjusted interactively by accessing the editor's [Main-Menu](#) then Options->Full-Configuration->[Display/Color Options](#)-> Display Layout Options. - LineNumbers can be adjusted interactively by accessing the editor's [Main-Menu](#) then Options->Full-Configuration->[Display/Color Options](#)-> [Set Colors](#)->Select Editing Colors. - Transparency can be adjusted interactively by accessing the editor's [Main-Menu](#) then Options->Full-Configuration->[Display/Color Options](#). - KeepWinOnTop can be adjusted interactively by accessing the editor's [Main-Menu](#) then Options->Full-Configuration->[Display/Color Options](#). - <Alt CursorLeft> goes up a directory in the file picker - New key assignments in the win.ui: <Ctrl T> transpose char <AltShift T> transpose line <CtrlShift T> transpose word <Ctrl K \> delete horizontal whitespace <Ctrl K C> comment selection/current line <Ctrl K U> uncomment selection - The Quote macro now uses the .syn file to get comment information. No need to update the quote macro any more if you add a new language - just be sure you have a .syn file with a comment specification. Now, in order to add a new language: 1) create a syntax file (either via the .txt method or the menus) 2) add the 'find the function string' to the .ui 3) add keywords for indenting to the .ui 4) create templates via the template macro 5) create a compile entry via the compile macro 6) modify spellchk.s as appropriate

+-----+ | New User Level Commands implemented as [macros](#) |

+-----+ - Macro Name: Delete-Blank Included commands:

DelBlankLines - deletes blank lines around the cursor line DelWhiteSpace - deletes blank space around the cursor To use (substitute your [key](#) assignment for "[key](#)"): <[key](#)> [ExecMacro](#)("delete-blank", "DelBlankLines") <[key](#)> [ExecMacro](#)("delete-blank", "DelWhiteSpace") - Macro Name: ShellExe (available from the Potpourri) If a command line is passed, it is simply passed along to the Windows ShellExecute function. If a block is marked, the first line of the block is passed to the Windows ShellExecute function. You can use this to go to a URL in the default browser, or to send an email via the default mailer. If you are editing an html file and would like to preview it in a Browser window, simply run ShellExe, and as long as the file extension is .htm or .html, the file will open in the default browser. To go to a web page, mark the following URL and run ShellExe: www.semware.com To send an email, mark any of the following addresses and run ShellExe: james.t.kirk@starfleet.command.org James T. Kirk <james.t.kirk@starfleet.command.org> mailto:James T. Kirk <james.t.kirk@starfleet.command.org> - Macro Name: Transpose Included commands:

TransposeChars TransposeWords TransposeLinesAbove TransposeLinesBelow RotateLineDown RotateLineUp To use (substitute your [key](#) assignment for "[key](#)"): <[key](#)> [ExecMacro](#)("transpose", "TransposeChars") <[key](#)> [ExecMacro](#)("transpose", "TransposeWords") <[key](#)> [ExecMacro](#)("transpose", "TransposeLinesAbove") <[key](#)> [ExecMacro](#)("transpose", "TransposeLinesBelow") <[key](#)> [ExecMacro](#)("transpose", "RotateLineDown") <[key](#)> [ExecMacro](#)("transpose", "RotateLineUp") +-----+ | Miscellaneous

Enhancements/Changes | +-----+ - New Predefined Editor

[Constants](#): _LINUX_, for use with the [WhichOS\(\)](#) editor command. _CENTER_POPUPS_, for use with the

SpecialEffects editor variable. - New Color Syntax Highlighting file for the following language: ColdFusion (cf.syn). -

New [Command-Line Options](#): -1 to start/use a single session of the editor. -1- to start a separate session of the editor. -

Obsolete [Command-Line Options](#): The "-c" (for color attribute set) [and](#) "-m" (for mono attribute set) [command-line](#)

[options](#) are now obsolete. - Current Editor options that are available on the *Windows System Menu: * Right Click the

Windows title bar or press <Alt Spacebar> for the Windows system menu that pertains to the current window Set Font

- Change the editor's current display font, size, and/or type. Same action as is available from the editor's "[Main-Menu](#)"

then "Options"->"Set Font". See the editor's [ChooseFont\(\)](#) command for details. Copy Window to Editor Clipboard -

Copy all the contents of what is currently being displayed in the editor's window to the editor's clipboard. This is

handy for capturing error messages, [menus](#), prompts, etc. - Appendices of [the Macro Reference Guide](#) in the on-line

help have been revised in accordance with additions and enhancements made in version 4.40 of The SemWare Editor

Professional. [APPENDIX C: COMMANDS BY CATEGORY](#) [APPENDIX D: VARIABLES BY CATEGORY](#) -

Appendices of [the User's Guide](#) in the on-line help have been revised in accordance with additions and enhancements

made in version 4.40 of The SemWare Editor Professional. [APPENDIX F: STANDARD CONFIGURATION](#)

[SETTINGS](#) Also see: What's New in Version 4.2

Summary of New/Enhanced Features in the Semware Editor Professional/32 V2.6

In version 2.6 of The SemWare Editor Professional/32, a number of new features have been added, and some existing features have been modified. This version has the same look and feel as the DOS version of the editor, but has been converted to a native 32-bit text (console) mode application that allows you to take advantage of many

[Windows 95/NT](#) features. With this version, you can: - Create and edit files with long filenames, and using UNC network paths - Edit larger files - up to the maximum memory that Windows can allocate on your system - Access the native Windows 95/NT Clipboard for cutting and/or pasting text between the editor and other Windows applications - Manage files within PickLists (sort, delete, load, change drives) - Resize your screen to whatever columns/rows [Windows](#) allows - Edit longer line lengths (up to 8K) - Launch the editor either from the command line without creating another window, or from an icon on the Windows desktop or Start menu. - Call DLLs from the macro programming language (SAL) The documentation has been updated to provide you with detailed information about enhancements. The following summary can help you locate much of the updated information. This summary information is also available from the HelpSystem "[Table of Contents](#)" under the topic "What's New in Version 2.6". This topic summarizes updates to the editor that were made in versions 2.5 and 2.6 (that is, all enhancements since version 2.0). (The WHATSNEW.DOC file included in The SemWare Editor Professional/32 software also provides information [about](#) this release.) The following pages identify key chapters and sections of the manuals that were affected by changes/additions in versions 2.5 [and](#) 2.6. It first summarizes modifications to [the User's Guide](#), and then summarizes modifications to [the Macro Reference Guide](#). USER'S GUIDE SUPPLEMENT [INTRODUCTION: The Editor Features](#) - You can now edit lines up to 8176 characters in length. [STARTING THE EDITOR: Selecting Files to Edit](#) - When specifying files to edit, you can [include](#) long filenames and UNC network paths. You can also specify filenames with leading or embedded spaces by enclosing the filename within quotes. [STARTING THE EDITOR: Command-Line Options](#) - New command-line option: [Load Keyboard Macro](#) You can now load Keyboard macros from the command line or "File(s) to edit:" prompt by specifying "-k". - Changes made to [command-line options: Load Macro, Execute Macro](#) You can now load ("-l") and/or execute ("-e") multiple macros from the command line or "File (s) to edit:" prompt. - New command-line option: [Restore State](#) You can restore the editing environment from the previously-saved editing session by specifying "-r" on the command line. This includes the file(s) that were loaded and the [window](#)(s) and file(s) that were displayed on the screen; the position of the cursor at the time the session was ended; the block (if any) that was marked; the bookmarks that were placed; and the settings of selected editing options (such as those on the "Options" Menu). [STARTING THE EDITOR: Starting the Editor with No Command-Line Parameters](#) - You can configure the file-selection behavior of the editor that will be in effect when you start the editor without specifying any command-line parameters (that is, you do [not include](#) any filenames or editing options on the command line). You can instruct the editor to do one of the following: - Issue the "File(s) to edit:" prompt so that a filename specification can be entered. This is the default behavior. - Execute the new FileManager command, passing it a file specification of "*. *". - Display the Recent Files list (valid only if the new PersistentRecentFiles variable is [ON](#)). - Open an "unnamed" file, by executing the newly-added NewFile command. - Display a PickList of files in the current directory. - Restore a previous editing environment, provided one was previously saved. - Display a selection menu from which you can choose one of the above options, or execute the Where command. [THE PULL-DOWN MENU SYSTEM: Activating the Pull-Down Menu](#) - You can now configure the editor so that the main Menu continually displays at the top of the screen. This can be especially helpful to mouse users, by allowing you to pull down a sub-menu with a single mouse action. [COMMAND PROMPTS AND PICKLISTS: Command Prompts](#) - You can now cancel a command or operation from within a Prompt box by executing the Exit (<Alt X>) command. Also, the following commands are now available in a prompt box: DelLeftWord (<Ctrl Backspace>), DelRightWord (<Ctrl Del>), Copy (<Grey+>), CopyToWinClip (<CtrlShift Grey+>), ListRecentFiles (<Ctrl 0>), PasteFromWinClip (<CtrlShift Grey*>). - The editor gives you the option to maintain History entries across editing sessions, via the PersistentHistory variable. The editor allows you to specify the maximum number of entries (up to 100) that you want to maintain for each History list. And you can remove individual entries from the History Buffer. - You can configure the treatment of directories and the order of the entries in file-related PickLists. The following can be configured: -

Whether or not the editor will change the current directory when a new directory is selected from a file PickList. - Whether the editor will include ALL directories that are in the path containing the files displayed in the current PickList, or whether directory entries will be limited to those that match the filename specification. - Whether the editor will display all included directory entries together at the top of the PickList; or throughout the PickList according to the sort order. - Whether or not the editor adds a slash to the beginning of any included directory entries. - The default sort order (ascending or descending) of the entries in the file PickList, and the default key on which the entries will be sorted (file name, extension, size, date, or attributes). - Once the file PickList is displayed, additional options are available that allow you to sort the PickList entries dynamically, select an alternate drive for which to display the matching list of files, and delete listed files from disk. And in a PickList displayed by the EditFile command, you can select multiple files and/or directories for processing. [GETTING HELP IN THE EDITOR: The Customizable HelpLine](#) - You can configure the editor to display HelpLine bars immediately (with no delay) once the <Shift>, <Alt>, or <Ctrl> key is pressed, or you can lengthen the amount of the delay. [GETTING HELP IN THE EDITOR: The HelpSystem](#) - [The HelpSystem](#) now includes a Search facility, which allows you to search [the HelpSystem](#) for a specified string. Using this option, you can search through the text of all the topics in [the HelpSystem](#) for occurrences of the specified [string](#). [MOUSING AROUND THE EDITOR: Using the Pop-Up Mouse Menu](#) - A special Mouse menu replaces the former mouse-activated ClipBoard menu to provide mouse access (<RightBtn>) to numerous Block-related commands, in addition to ClipBoard-related commands (including the Microsoft Windows Clipboard). [LOADING, PROCESSING, AND SAVING FILES: Adding Files to the Ring: Specifying and Opening Files](#) - The EditFile command allows you to select multiple files and/or directories from a PickList of files displayed by this command. When specifying files to edit, you can [include](#) long filenames and UNC network paths. You can also specify filenames with leading or embedded spaces by enclosing the filename within quotes. - The NewFile command has been added, which creates a new temporary memory buffer with a name in the format <unnamed-#> (where "#" represents an actual number assigned by the editor). This command is useful for creating a temporary file without a name, which can later be discarded, or which can be given a name so that it can be saved. [LOADING, PROCESSING, AND SAVING FILES: Locking Files](#) - The editor includes an option to lock files that have been opened for editing in the current session. This feature allows you to reduce or eliminate the likelihood that another program overwrites a file (on disk) that you are currently editing. Two different methods are provided for [locking files](#): Read-Only and Handle. [LOADING, PROCESSING, AND SAVING FILES: Moving Between Files in the Ring](#) - The ListRecentFiles command can be configured to save the entries in the Recent Files list from one editing session to the next. The maximum number of files (up to 1000) that are maintained in the Recent Files list is controlled by the MaxRecentFiles variable. [LOADING, PROCESSING, AND SAVING FILES: Saving/Restoring the Current Editing Session](#) - You can now save information and settings from your current editing session so that they can be restored in a subsequent editing session. You can take advantage of options for: - [Saving/restoring entries in the History Buffer](#), so that History entries in prompts are available across editing sessions. - [Saving/restoring entries in the Recent Files list](#), so that PickList entries displayed by the ListRecentFiles command are maintained across editing sessions. - [Saving/restoring the editing environment](#). When a session is ended, certain information about that session can be saved, including the file(s) that are loaded and the [window](#)(s) and file(s) that are displayed on the screen; the position of the cursor; the block (if any) that is marked; the bookmarks that are placed; and the settings of selected editing options (such as those on the "Options" Menu). With this feature, you can basically end an editing session, and then return to it at a later time, just as you left it. You can even interactively execute the related State macro, so that you can save the current editing environment, and then later restore it without exiting the editor. [LOADING, PROCESSING, AND SAVING FILES: Alternative Modes for Viewing Files](#) - Hex mode now allows you to edit either the text format of the data OR the hexadecimal format of the data. Press the <Tab> key to toggle editing between the text-display area and the hexadecimal-display area. [LOADING, PROCESSING, AND SAVING FILES: Performing Other File-Processing Operations](#) - The FileManager command allows you to manage and manipulate files located on disk, all from within the editor. Included among the many available features are options to copy, move, delete, or rename files; sort a directory listing; tag multiple files on which to operate; and view/modify compressed files. - The ShowFileInfo command displays selected information about the current file (if the file exists on disk), including: the complete path; the number of lines currently in memory within the editor; the size of the file that is

located on disk; the date and time the file was last written to disk; the disk file attributes; and the current read/write permission status. [SPLITTING THE SCREEN INTO WINDOWS: Opening Windows](#) - The OpenFileAndWindow command provides you with a single command that will open a new window and load a new file into it. [SPLITTING THE SCREEN INTO WINDOWS: Synchronized Scrolling in Windows](#) - Synchronized Scrolling mode allows you to scroll simultaneously in multiple windows, so that cursor movement and scrolling activity in the current window will also occur in each of the non-current windows. [SPLITTING THE SCREEN INTO WINDOWS: Closing Editing Windows](#) - The CloseFileAndWindow command provides you with a single command that will close the current window and also close the file in that window. [MOVING THROUGH TEXT IN A FILE: Changing Position in a File](#) - The BegPara command moves the cursor to the first line of the current paragraph (or current function/procedure). - The EndPara command moves the cursor to the last line of the current paragraph (or current function/procedure). - The NextPara command moves the cursor to the first line of the next paragraph (or next function/procedure). - The PrevPara command moves the cursor to the first line of the current paragraph (or the current function/procedure); or to the first line of the previous paragraph (or previous function/procedure), if the cursor is already on the first line of the paragraph. [USING TABS: Setting Tab Stops](#) - The ZeroBasedTab option allows you to select either column 1 or 0 as the position from which to begin placing tab stops for [Soft and Hard Tabs](#). [MANIPULATING TEXT: Commands to Format Text](#) - The QuoteLine command notates the text on the current line, or in a marked Block, as a quote or comment, depending on the file type (extension) of the current file. You can even "unquote"/"uncomment" text using this command. [EDITING MODES: WordWrap Mode](#) - WordWrap mode now supports an AUTO setting. AUTO offers the same behavior as ON, and additionally causes the editor to automatically reformat text through the end of a paragraph, as characters are inserted or deleted within existing text in the paragraph. **BLOCK OPERATIONS:** [Marking and Manipulating a Block of Text](#) - The MarkToEOL command marks a block that begins at the cursor position, and ends at the last character on the line. - The MarkParagraph command marks the entire paragraph (or function/procedure) in which the cursor is positioned. - The FillBlock command has been enhanced to allow you to specify a string of one or more characters to be used to fill a marked block. - The Capitalize command is an additional case-change command that capitalizes each word in the indicated text; that is, the FIRST character of each word is changed to upper [case](#), and every other letter of each word is changed to lower case. You can use this command to capitalize the current word, or all the words on the current line or in a block. [BLOCK OPERATIONS: Manipulating Blocks Via ClipBoards](#) - The following commands allow you to copy or paste text between the current file and the Microsoft Windows Clipboard. - The CopyToWinClip command copies the block in the current file to the Microsoft [Windows](#) Clipboard. - The PasteFromWinClip command pastes the contents of the Microsoft Windows Clipboard into the current file at the cursor position. [SEARCH FEATURES: FINDING AND REPLACING TEXT: Basic Find and Replace Options](#) - The [A] ("All") option has been added to the Find and Replace commands to allow you to search for or replace a string throughout all files in the Ring. - The [V] ("View") option has been added to the Find command to allow you to display a PickList of all lines that contain a specified search string. - The AgainReverse command, which is a variation of the Again command, allows you to repeat the last search operation, but in the opposite direction. [SEARCH FEATURES: FINDING AND REPLACING TEXT: Listing All Occurrences of a Specified Text String](#) - The CompressView command has been enhanced to allow you to search across all files in the Ring to generate the CompressView PickList. And you can now readily re-display the CompressView PickList, via the RedisplayViewPickList command. - The RedisplayViewPickList command re-displays the PickList that was created by the last search operation in which the "view" option was specified (that is, the last Find operation using the [V] option, or the last execution of CompressView or FunctionList). [CREATING AND USING MACROS: Introduction to External Compiled Macros](#) - [The ExecMacro command](#) has been modified to allow you to pass an optional command-line string to the macro that is executed. - The PurgeMacro command no longer prompts you for the name of the external Compiled Macro file to purge. When you execute PurgeMacro, it immediately displays a PickList of the external macro files that are currently loaded, from which you can select the desired macro file to purge. - The Debug command provides a source-level debugger for macros written in the editor's macro programming language, SAL (SemWare Applications Language). **SPECIAL COMMANDS AND FEATURES:** [Selecting the Video Mode](#) - The following video modes are now supported in the editor: Selection Video Mode _____

25-Line 25 lines by 80 columns 28-Line 28 lines by 80

columns 30-Line 30 lines by 80 columns 33-Line 33 lines by 80 columns 36-Line 36 lines by 80 columns 40-Line 40 lines by 80 columns 43-Line 43 lines by 80 columns 44-Line 44 lines by 80 columns 50-Line 50 lines by 80 columns 80-Column Forces 80 columns Max Lines/Cols Maximum lines/columns supported by current font [SPECIAL COMMANDS AND FEATURES: Executing External Commands from Within the Editor](#) - The CaptureDosOutput command, which is similar to the Dos command, allows you to execute a single external command from the editor, and capture the screen output from the command and load it into a temporary file in the editor. - The Shell command now optionally adds the notation "TSE-Shell" to the operating system prompt [string](#) when Shell is executed. [SPECIAL COMMANDS AND FEATURES: Executing Supplemental Macros from the Potpourri](#) - A number of new macros have been added to the Potpourri, including the following: - The Book macro provides a convenient way to set Bookmarks, or to go to or list the Bookmarks that are currently set. - The CD macro provides a handy facility for changing the current drive and/or directory, using a database of drives and directories on your system. - The CmpFiles macro allows you to compare the current file with another file, by simultaneously displaying and scrolling the two files, and highlighting the differences. - The CUAMark macro (formerly named Shift) enables CUA-style (Common User Access) key assignments in the editor for marking and manipulating character blocks. When you execute this command, the shifted [cursor](#)-movement keys can be used for [marking](#) blocks, and additional block operations can be accomplished using select shifted keys. - The Justify macro is an extension of the WrapPara command. It formats text in paragraph style between left and right margins, and also justifies each line of text at the right margin. - The Overlay macro is a form of synchronized scrolling that allows you to check minor differences between two files. It "overlays" the current file with the next file in the ring, continuously switching back and forth between the two files. - The State macro allows you to interactively save the editing environment for the current session or restore the previously-saved environment. - The Where macro searches one or more drives for files that match an indicated filename specification, and displays a PickList of all matching filenames from which you can select file(s) to edit or to delete (or on which to perform other available operations). [CONFIGURING THE EDITOR INTERACTIVELY: Configuration Options Available Interactively](#) - New/Enhanced [System/File Options](#) - Empty Command-Line Action: Indicates the file-selection behavior of the editor that will be in effect when the editor is started with no command-line parameters specified (that is, with no file specifications or editing options included on the command line). The available options are: Prompt, File Manager, Recent, Unnamed, PickList, [Restore State](#), and [Menu](#). - File Locking: Indicates whether the editor will lock files that are loaded. Two types of file-locking are available: Read-Only and Handle. - File PickList Settings: Allows you to select a number of options for file-related PickLists. The following options control the treatment of directories and the order of the entries in a PickList: - Changes Directory - Force Showing Directories - Show Directories at Top - Precede Directory Entries with Slash - Sort Order - Sort [Key](#) - Line-Termination String: Now includes an "AS LOADED" option, which instructs the editor to use the same termination character as is in the file when it is initially loaded. - Save State on Exit: Indicates whether the editor saves certain information about the current editing environment ("state") when you exit from the editor. - Removed [System/File Options](#) The following System/File Options have been removed from the editor: - Max XMS to Use - Max EMS to Use - Path for Swap File - Renamed System/File Option - Load Wild from DOS has been renamed as Load Wild from Command Line. - Removed [Keyboard/Mouse Options](#) The following Keyboard/Mouse Options have been removed from the editor: - Use Enhanced Keyboard - Use Extended Keyboard Driver - Use Mouse - New/Enhanced [Command/Format Options](#) - Max History Per List: Indicates the maximum number of History entries to be stored in the History Buffer for each History list, ranging from 1 to 100. - Max History Entries: Indicates the maximum number of total History entries to be stored in the History Buffer, ranging from 1 to 1000. - Persistent History: Toggles whether the entries stored in the History Buffer are maintained across editing sessions. - Max Recent Files: Indicates the maximum number of files to be stored in the Recent Files list, ranging from 1 to 1000. - Persistent Recent Files: Toggles whether the entries stored in the Recent Files list are maintained across editing sessions. - Renamed Command/Format Option - Change DOS Prompt in Shell has been renamed as Change Prompt in Shell. - Enhanced Text Entry/Block Option - WordWrap Mode: Indicates the state of WordWrap mode, which affects wrapping of text as characters are inserted or deleted. Three options are available: [OFF](#), [ON](#), or AUTO. - New Tab/Margin Option - Zero-Based Tab Stops: Toggles whether Hard and Soft Tab stops are based on column 1 [or](#) 0. - New/Enhanced [Display/Color Options](#) - Display Main Menu: Toggles whether the editor continually displays the [main](#) pull-down [Menu](#). - HelpLine Delay: Indicates the amount of time a "shift" (<Shift>, <Alt>, or <Ctrl>) key must be held down before the editor updates [the HelpLine](#) with the

associated HelpLine bar. - Startup Video Mode: Indicates the number of lines and columns to be displayed on the screen (that is, the video mode in effect) when the editor is started. Depending on your screen resolution and current font, you can select from the following choices: AutoDetect, 25-Line, 28-Line, 30-Line, 33-Line, 36-Line, 40-Line, 43-Line, 44-Line, 50-Line, and Max Lines/Columns. - Removed Display/Color Option The following Display/Color Option has been removed from the editor: - Eliminate Snow [APPENDIX A: TECHNICAL SPECIFICATIONS](#) [APPENDIX C: STANDARD KEY ASSIGNMENTS](#) [APPENDIX E: COMMAND IMPLEMENTATION](#) [APPENDIX F: STANDARD CONFIGURATION SETTINGS](#) - The appendices of [the User's Guide](#) have been revised in accordance with additions and enhancements made in versions 2.5 [and](#) 2.6 of The SemWare Editor Professional. (Appendix B and Appendix D have been removed.) MACRO REFERENCE GUIDE SUPPLEMENT [MACRO LANGUAGE SYNTAX: Reserved Words](#) - The following words have been added to the list of macro language [reserved words](#): by downto [endfor](#) in to [do enddo](#) for [times](#) [MACRO LANGUAGE SYNTAX: Pre-Defined Constants](#) - Two [pre-defined constants](#) have been added: - WIN32 is intended to be used with the conditional compilation directives to write macros that are portable between the various versions of the editor. - _MAXPATH_ is intended to be used to set the maximum length of a SAL string that deals with files. Its pre-set value is the maximum allowable length for paths/filenames in the version of the editor that you are running. [MACRO LANGUAGE SYNTAX: Statements](#) - String slices have been included as a new string operator. String operators are now divided into four categories: concatenation, indexing, [string slices](#), and relational. String slices allow a string variable to be manipulated by specifying a substring of characters within the string. - Numeric expressions now include the additional logical operator, "[in](#)." The "[in](#)" operator evaluates to TRUE if an expression is TRUE; or to FALSE if an expression is FALSE. It serves as a convenient tool for checking to see if an expression is within a range of values or within a group of ranges (set inclusion), and can be used in place of the "[and](#)" and "[or](#)" operators to simplify such operations. - With the addition of string slices and the "[in](#)" operator, SAL now supports 15 levels of precedence for [expressions](#). [String slices](#) are included in level 1 (the highest level), and the "[in](#)" set-inclusion operator is at level 13 (ahead of the "=" assignment operator and the "," evaluation operator). - The macro language has been expanded to [include](#) two new sets of control [statements](#): for/[endfor](#) and do/[enddo](#). With these, SAL now offers five types of iterative [statements](#): while/[endwhile](#), repeat/until, [for/endfor](#), do/[enddo](#), and [loop/endloop](#). The "[for](#)" and "[do](#)" loops offer an additional method of control when executing a loop, by allowing you to explicitly indicate the number of times a loop is to be repeated. The statements in a "[for](#)-endfor" loop are repeated a specific number of times, based on a range of values. The statements in a "[do](#)-enddo" loop are repeated a specified number of times, as indicated by the value of a numeric expression. - The break statement can be used, if necessary, to terminate the new "[for](#)" and "[do](#)" loops. [MACRO LANGUAGE SYNTAX: Binary](#) - The binary directive is no longer available. See "[Using DLLs](#)" in the chapter on "[Macro Language Syntax](#)" for information [about](#) calling system services. [MACRO LANGUAGE SYNTAX: Using DLLs](#) - The DLL directive allows the user to extend the editor to include additional procedures which cannot be written using the macro language. DLLs are procedures written using another programming language, but which can interface with the editor's calling convention. DLLs replace the binary directive, as well as the Intr() command and associated register [variables](#). [MACRO LANGUAGE SYNTAX: Conditional Compilation Directives](#) - Conditional compilation directives have been included in SAL as an advanced-usage feature. They allow you to selectively compile portions of a macro based on condition(s) supplied at compile time. [MACRO LANGUAGE SYNTAX: Register](#) - Register variables, and the associated Intr() command, are no longer available. See "[Using DLLs](#)" in the chapter on "[Macro Language Syntax](#)" [for](#) information [about](#) calling system services. [USING MACROS: The User-Interface Macro File \(TSE.UI\)](#) - [The user-interface macro file \(TSE.UI\)](#) can now include a WhenPurged macro. If one exists, it is called when the editor is terminating, just before the _ON_ABANDON_EDITOR_ event is called. [USING MACROS: Compiling Macros](#) - The following additional options have been included in the SAL compiler program, SC32.EXE: - -s instructs the compiler to install a new ".UI" or ".CFG" interface file in the editor, by forcing creation of separate automatically-loaded interface files ("TSEUI.DAT" [and](#) "TSECFG.DAT"). - -v instructs the compiler to generate debug information in the resulting object (".MAC") file. - -dsymbolic_constant[=integer_constant] instructs the compiler to define symbolic_constant and optionally assign it the value integer_constant. This option is useful when you want to dynamically assign a value for a conditional compilation directive. - The source_filename parameter accepts

wildcards, and supports List files (specified by using the "@" symbol as the first character in the value specified for source_filename). [USING MACROS: Loading/Executing Macros](#) - In addition to specifying a macro filename when you execute the [ExecMacro\(\)](#) command, you can optionally include a command-line string to pass to the macro(s) included in the macro file. [USING MACROS: Debugging Macros](#) - The [Debug\(\)](#) command provides a source-level debugger for SAL macros. It allows you to examine the execution of a macro to verify proper execution or to help determine the cause of problems while executing. When you invoke [Debug\(\)](#) and execute a macro, control of the macro is passed to you. You can then step through execution of the macro, display the contents of affected files and variables, and even modify the contents of variables as the macro executes. [USING MACROS: Startup Processing](#) - When you start the editor without specifying any options or filenames on the command line, the editor checks the setting of the new [StartupFlags](#) variable to determine the appropriate startup action to take. [USING MACROS: Terminating the Editor](#) - When [terminating the editor](#), additional steps occur to optionally save the editing state, to execute a [WhenPurged](#) macro procedure if included in the user-interface macro file, and to optionally save the Recent Files list and update the History Buffer. [USING MACRO LANGUAGE FEATURES: Command Prompts and Keydefs](#) - The following native commands are now available in a prompt box: [Copy\(\)](#), [CopyToWinClip\(\)](#), [DelLeftWord\(\)](#), [DelRightWord\(\)](#), [PasteFromWinClip\(\)](#), and [Exit\(\)](#). These commands can only be executed within a prompt box if they have been assigned to a key. [USING MACRO LANGUAGE FEATURES: Using Command Lines in Macros](#) - Information can optionally be passed from one macro to another, as [macros](#) are executed, via the [MacroCmdLine](#) editor variable. To pass a command-line string to a macro, specify the desired string when you execute the macro using the [ExecMacro\(\)](#) command. The current macro can also change the value of [MacroCmdLine](#), so that information can be returned to a calling macro. [USING MACRO LANGUAGE FEATURES: Video Output Commands](#) - The following commands and variables have been added to make more of the editor's own low-level video output functions available for advanced macro usage: the commands [PutAttrXY\(\)](#), [PutCharHXY\(\)](#), [PutCharXY\(\)](#), [PutCtrStr\(\)](#), [PutLineXY\(\)](#), [PutNCharAttr\(\)](#), [PutStrAttr\(\)](#), [PutStrAttrXY\(\)](#), [PutStrEOL\(\)](#), [PutStrEOLXY\(\)](#), [PutStrXY\(\)](#), [VWhereX\(\)](#), and [VWhereY\(\)](#); and the [variables](#) [PopWinX1](#), [PopWinY1](#), [PopWinCols](#), and [PopWinRows](#). - Several [video output commands](#) have been added to now allow [reading from the screen](#): [GetStr\(\)](#), [GetStrXY\(\)](#), [GetStrAttr\(\)](#), and [GetStrAttrXY\(\)](#). - To improve screen update performance, the editor allows you to temporarily buffer video output, using the [BufferVideo\(\)](#) and [UnBufferVideo\(\)](#) commands. [USING MACRO LANGUAGE FEATURES: Event Macros](#) - The following additional editor events can be hooked to an event macro: [_ON_FILE_LOAD_](#), [_NONEDIT_IDLE_](#), [_BEFORE_COMMAND_](#), [_BEFORE_NONEDIT_COMMAND_](#), [_AFTER_NONEDIT_COMMAND_](#), [_ON_SELFINSERT_](#), [_ON_DELCHAR_](#), [_ON_UNASSIGNED_KEY_](#), [_ON_NONEDIT_UNASSIGNED_KEY_](#), [_AFTER_GETKEY_](#), [_BEFORE_GETKEY_](#) - The [SetHookState\(\)](#) command allows you to specify whether the hooks for a particular editor event are currently enabled or disabled. By using this command, macros can be temporarily unhooked from an event. - The [GetHookState\(\)](#) command complements the [SetHookState\(\)](#) command by allowing you to check the current state of an editor event to see if the [macros](#) that are hooked to it are currently enabled. This command, used in conjunction with the [SetHookState\(\)](#) command, is useful for saving, setting, and then restoring the hook state of an event. [EDITOR COMMANDS](#) - The editor now offers over 400 native commands that can be called by user-written [macros](#). The following new commands are available. - Block/ClipBoard commands: [CopyAppend\(\)](#), [CopyToWinClip\(\)](#), [CutAppend\(\)](#), [GetClipboardBlockType\(\)](#), [GotoBlockBeginCol\(\)](#), [GotoBlockEndCol\(\)](#), [MarkToEOL\(\)](#), [PasteFromWinClip\(\)](#) - [Bookmark commands](#): [DelAllBookMarks\(\)](#), [DelBookMark\(\)](#), [GetBookMarkInfo\(\)](#), [isBookMarkSet\(\)](#) - [Cursor-movement commands](#): [ScrollToCenter\(\)](#), [ScrollToTop\(\)](#) - [Editing commands](#): [HexEdit\(\)](#), [WrapLine\(\)](#) - [Event macro commands](#): [GetHookState\(\)](#), [QueryEditState\(\)](#), [SetHookState\(\)](#) - [File-handle commands](#): [fClose\(\)](#), [fCreate\(\)](#), [fOpen\(\)](#), [fRead\(\)](#), [fReadFile\(\)](#), [fSeek\(\)](#), [fWrite\(\)](#), [fWriteFile\(\)](#), [fDup\(\)](#), [fDup2\(\)](#) - [File-related commands](#): [AddFileToRing\(\)](#), [CheckDefaultExt\(\)](#), [CurrDir\(\)](#), [CurrExt\(\)](#), [GetDir\(\)](#), [MakeTempName\(\)](#), [NewFile\(\)](#), [ReplaceFile\(\)](#), [SetFileAttr\(\)](#), [EditBuffer\(\)](#), [LoadBuffer\(\)](#), [AddTrailingSlash\(\)](#), [isTrailingSlash\(\)](#), [isWildPath\(\)](#), [QuotePath\(\)](#), [RemoveTrailingSlash\(\)](#), [isCharDevice\(\)](#) - [Integer/string variable manipulation commands](#): [DelGlobalVar\(\)](#), [ExistGlobalVar\(\)](#), [DelStr\(\)](#), [InsStr\(\)](#), [GetFoundText\(\)](#), [GetToken\(\)](#), [GetWord\(\)](#), [NumTokens\(\)](#), [Min\(\)](#), [Max\(\)](#) - [Keyboard commands](#): [isTypeableKey\(\)](#), [KeyCode\(\)](#), [KeyName\(\)](#), [PressKey\(\)](#), [PushKeyStr\(\)](#) - [Low-level commands](#): [Addr\(\)](#), [AdjPtr\(\)](#), [ClearBit\(\)](#), [CurrLinePtr\(\)](#),

[GetBit\(\)](#), [HiByte\(\)](#), [HiWord\(\)](#), [LoByte\(\)](#), [LoWord\(\)](#), [Ofs\(\)](#), [PeekByte\(\)](#), [PeekLong\(\)](#), [PeekWord\(\)](#), [PokeByte\(\)](#), [PokeLong\(\)](#), [PokeWord\(\)](#), [Ptr\(\)](#), [Seg\(\)](#), [SetBit\(\)](#) [BuildPickBufferEx\(\)](#), [PBAttribute\(\)](#), [PBDate\(\)](#), [PBDateStr\(\)](#), [PBName\(\)](#), [PBlockSize\(\)](#), [PBTime\(\)](#), [PBTimeStr\(\)](#) [FFAttribute\(\)](#), [FFDate\(\)](#), [FFDateStr\(\)](#), [FFName\(\)](#), [FFSize\(\)](#), [FFTime\(\)](#), [FFTimeStr\(\)](#), [FindFileClose\(\)](#), [FindFirstFile\(\)](#), [FindNextFile\(\)](#), [FindThisFile\(\)](#) [DosIOResult\(\)](#), [IDos\(\)](#) - Miscellaneous commands: [ChrSet\(\)](#), [ExecLoadedMacro\(\)](#), [LoadStartupMacros\(\)](#), [IShowEntryScreen\(\)](#), [IVersion\(\)](#), [MacroStackAvail\(\)](#), [MatchFilename\(\)](#), [NoOp\(\)](#), [SignOn\(\)](#) [GetWinHandle\(\)](#), [LongestLineInBuffer\(\)](#) - Prompting and History commands: [AskFilename\(\)](#), [AskNumeric\(\)](#), [IReadNumeric\(\)](#), [ReadNumeric\(\)](#) [DelHistoryStr\(\)](#), [FindHistoryStr\(\)](#), [GetHistoryStr\(\)](#), [NumHistoryItems\(\)](#) - Search commands: [SearchHelp\(\)](#), [ViewFinds\(\)](#) - Video output commands: [BufferVideo\(\)](#), [UnBufferVideo\(\)](#) [GetMaxRowsCols\(\)](#), [SetVideoRowsCols\(\)](#) [GetStr\(\)](#), [GetStrAttr\(\)](#), [GetStrAttrXY\(\)](#), [GetStrXY\(\)](#) [GetWindowTitle\(\)](#), [SetWindowTitle\(\)](#) [ListFooter\(\)](#), [ListHeader\(\)](#) [PutCStr\(\)](#), [PutNCharAttr\(\)](#) [PutAttrXY\(\)](#), [PutCharHXY\(\)](#), [PutCharXY\(\)](#), [PutLineXY\(\)](#), [PutStrAttr\(\)](#), [PutStrAttrXY\(\)](#), [PutStrEOL\(\)](#), [PutStrEOLXY\(\)](#), [PutStrXY\(\)](#) [SetCursorOff\(\)](#), [SetCursorOn\(\)](#) [VWhereX\(\)](#), [VWhereY\(\)](#) - The following [editor commands](#) are now obsolete: [BuildPickBuffer\(\)](#), [DecodeDTA\(\)](#), [DecodePickBuffer\(\)](#), [FindFirst\(\)](#), [FindNext\(\)](#), [InByte\(\)](#), [InWord\(\)](#), [Intr\(\)](#), [OutByte\(\)](#), [OutWord\(\)](#), [SetDTA\(\)](#) - The [ChangeCurrFilename\(\)](#) command supports two additional parameter [values](#), [_DONT_EXPAND_](#) and [_OVERWRITE_](#). - The [ChrSet\(\)](#) command now accepts a variable string. - The [EditFile\(\)](#) command supports an additional parameter value, [_FROM_CMDLINE_](#). When this value is passed, the command operates as though it were invoked from the command line. [EDITOR VARIABLES](#) - The following new [editor variables](#) are now available. - Keyboard [variables](#): [KbdMacroRecording](#), [KbdMacroRunning](#) [LastKey](#), [LastMouseKey](#), [LastMouseX](#), [LastMouseY](#) - Miscellaneous [variables](#): [BufferFlags](#), [MacroCmdLine](#), [SaveState](#), [StartupFlags](#), [ViewFindsId](#) [IdleTime](#), [TemplateExpansion](#), [ZeroBasedTab](#) - PickList, History, and Recent Files [variables](#): [PickFileFlags](#), [PickFileSortOrder](#) [CurrHistoryList](#), [MaxHistoryPerList](#), [PersistentHistory](#) [MaxRecentFiles](#), [PersistentRecentFiles](#) - Video output [variables](#): [PopWinCols](#), [PopWinRows](#), [PopWinX1](#), [PopWinY1](#) - Screen-display [variables](#): [HelpLineDelay](#), [ShowMainMenu](#), [StatusLineRow](#) - The following [editor variables](#) are now obsolete: [EliminateSnow](#), [MaxEMS](#), [MaxXMS](#), [SwapPath](#), [UseEnhancedKbd](#), [UseExtendedKbdDriver](#), [UseMouse](#) - The [CurrVideoMode](#) and [StartupVideoMode](#) variables support the following new [values](#): [_30_LINES_](#), [_33_LINES_](#), [_36_LINES_](#), [_40_LINES_](#), [_44_LINES_](#) [_MAX_LINES_](#), [_MAX_COLS_](#), [_MAX_LINES_COLS_](#). - The [EOLType](#) variable now recognizes an additional setting: 0 (AS LOADED) [APPENDIX B: COMMAND IMPLEMENTATION](#) [APPENDIX C: COMMANDS BY CATEGORY](#) [APPENDIX D: VARIABLES BY CATEGORY](#) - The appendices of [the Macro Reference Guide](#) have been revised in accordance with additions and enhancements made in versions 2.5 [and](#) 2.6 of The SemWare Editor Professional. (Appendix A has been removed.)

Summary of New/Enhanced Features in the Semware Editor Professional/32 V2.8

In version 2.8 of The SemWare Editor Professional/32, a number of new features have been added, and some existing features have been modified.

The online HelpSystem has been updated to provide you with information

[about](#) enhancements. The following summary can help you access much of the updated information in [the HelpSystem](#). This summary information is available from [the HelpSystem](#) "[Table of Contents](#)" under the topic "What's New in Version 2.8". (The WHATSNEW.DOC file included in The SemWare Editor Professional/32 software also provides information [about](#) this release.) The following pages identify key chapters and sections of the manuals that were affected by changes/additions in version 2.8. It first summarizes modifications to [the User's Guide](#), and then summarizes modifications to the Macro Reference Guide. Also see: What's New in Version 2.6 USER'S GUIDE SUPPLEMENT [THE PULL-DOWN MENU SYSTEM](#): [Accessing Sub-Menu Commands and Options](#) - Disabled menu items are now "grayed". If you select such an item, no action will occur. [EDITING MODES](#): [SyntaxHilite Mode](#) - SyntaxHilite mode is now available within the editor. This is a special editing display mode that causes certain

portions of the text to be distinctly colorized, based on the file extension. [SPECIAL COMMANDS AND FEATURES: Executing Supplemental Macros from the Potpourri](#) - A number of new macros have been added to the Potpourri, including the following: - The DelBlank macro removes blank lines from a file or block, by either removing all blank lines, or replacing multiple consecutive blank lines with a single blank line. - The SynHiUtl macro allows you to dynamically activate or terminate SyntaxHilite mode for the current file. It also allows you to dynamically load an arbitrary SyntaxHilite mapping (.SYN) file and associate it with the current file. - The Uniq macro deletes duplicate occurrences of a line of text (or portion of a line). [CONFIGURING THE EDITOR INTERACTIVELY: Configuration Options Available Interactively](#) - New [Display/Color Options](#) - Display SyntaxHilite Colors: Toggles whether SyntaxHilite colors are displayed for any files for which [SyntaxHilite mode](#) is active. - Configure SyntaxHilite Mapping Sets: Allows you to configure the mapping sets to be used by [SyntaxHilite mode](#). - [SyntaxHilite mode](#) colors: A number of color attributes have been added to allow you to select the colors to be used by SyntaxHilite mode. - Menu - Grayed: Determines the color attributes for "grayed" menu items. - Menu - Selected Gray: Determines the color attributes for selected "grayed" [menu](#) items. [APPENDIX E: COMMAND IMPLEMENTATION APPENDIX F: STANDARD CONFIGURATION SETTINGS](#) - Appendices of [the User's Guide](#) have been revised in accordance with additions and enhancements made in version 2.8 of The SemWare Editor Professional. [MACRO REFERENCE GUIDE SUPPLEMENT USING MACRO LANGUAGE FEATURES: Menus](#) - CheckBoxes have been added to menus in the editor. - A number of [menu](#) flag [values](#) have been renamed, and others have been added. Also, option [values](#) for this parameter can now be specified using an integer expression, which allows you to dynamically set the desired options. (For those flag values that have been renamed, the previous [values](#) are still supported; however, these previous [values](#) cannot be referenced outside a menu definition, so they cannot be used to dynamically set flag options.) - CloseBefore has been renamed [_MF_CLOSE_BEFORE_](#). - CloseAllBefore has been renamed [_MF_CLOSE_ALL_BEFORE_](#). - CloseAfter has been renamed [_MF_CLOSE_AFTER_](#). - CloseAllAfter has been renamed [_MF_CLOSE_ALL_AFTER_](#). - DontClose has been renamed [_MF_DONT_CLOSE_](#). - Skip has been renamed [_MF_SKIP_](#). - Divide has been renamed [_MF_DIVIDE_](#). - [_MF_ENABLED_](#) is a new flag value that instructs the editor to enable the associated function for this menu item. - [_MF_GRAYED_](#) is a new flag value that instructs the editor to disable the associated function for this menu item, and to display this item using the "gray" menu color attributes. - [_MF_DISABLED_](#) is a new flag value that instructs the editor to disable the associated function for this menu item. - [_MF_CHECKED_](#) is a new flag value that instructs the editor to place a check mark to the left of this menu item. - [_MF_UNCHECKED_](#) is a new flag value that instructs the editor to remove the check mark to the left of this menu item. [EDITOR COMMANDS](#) - The following new [editor commands](#) are now available. - SyntaxHilite commands: [GetSynFilename\(\)](#), [InitSynhiCurrFile\(\)](#), [LinkSynFile\(\)](#), [LoadSynhiAssoc\(\)](#), [PurgeSynhi\(\)](#), [ReplaceSynFile\(\)](#) [EDITOR VARIABLES](#) - The following new [editor variables](#) are now available. - SyntaxHilite variable: [ShowSyntaxHilite](#) - [Color variables](#): [AltMenuGrayAttr](#), [AltMenuSelectGrayAttr](#), [Directive1Attr](#), [Directive2Attr](#), [Directive3Attr](#), [IncompleteQuoteAttr](#), [KeyWords1Attr](#), [KeyWords2Attr](#), [KeyWords3Attr](#), [KeyWords4Attr](#), [KeyWords5Attr](#), [KeyWords6Attr](#), [KeyWords7Attr](#), [KeyWords8Attr](#), [KeyWords9Attr](#), [MenuGrayAttr](#), [MenuSelectGrayAttr](#), [MultiLnDlmt1Attr](#), [MultiLnDlmt2Attr](#), [MultiLnDlmt3Attr](#), [NumberAttr](#), [Quote1Attr](#), [Quote2Attr](#), [Quote3Attr](#), [SingleLnDlmt1Attr](#), [SingleLnDlmt2Attr](#), [SingleLnDlmt3Attr](#), [ToEOL1Attr](#), [ToEOL2Attr](#), [ToEOL3Attr](#) [APPENDIX B: COMMAND IMPLEMENTATION APPENDIX C: COMMANDS BY CATEGORY APPENDIX D: VARIABLES BY CATEGORY](#) - Appendices of [the Macro Reference Guide](#) have been revised in accordance with additions and enhancements made in version 2.8 of The SemWare Editor Professional. Also see: What's New in Version 2.6

Summary of New/Enhanced Features in the Semware Editor Professional/32 V3.0

In version 3.0 of The SemWare Editor Professional/32, a number of new features have been added, and some existing features have been modified.

The online HelpSystem has been updated to provide you with information

[about](#) enhancements. The following summary can help you access much of the updated information in [the HelpSystem](#). This summary information is available from [the HelpSystem "Table of Contents"](#) under the topic "What's New in Version 3.0". (The WHATSNEW.DOC file included in The SemWare Editor Professional/32 software also provides information [about](#) this release.) Also see: What's New in Version 2.8 New features: ----- - New user-level commands: Undo <Alt Backspace> on the Text menu Allows you to undo the last change done to the text. Invoking undo again will undo the next to last change, and so on. Redo <AltShift Backspace> on the Text menu Allows you to cancel the last undo. Invoking redo again will cancel the next to last undo, and so on. The redo information is deleted anytime another change is made to the text, except if the change was made via the undo or redo command. ToggleHorizVertical When there are only two [windows](#), they may be toggled between vertical and horizontal [windows](#). CutToWinClip <CtrlShift Grey-> on the Clip menu Similar to the CopyToWinClip command, this command copies the marked block to the Windows Clipboard, and then deletes the block from the editor. CutAppendToWinClip on the Clip menu Similar to the CutToWinClip command, this command copies the marked block to the Windows Clipboard, appending it to any text already in that clipboard, and then deletes the block from the editor. CopyAppendToWinClip on the Clip menu Similar to the CopyToWinClip command, this command copies the marked block to the Windows Clipboard, appending it to any text already in that clipboard. Empty Clipboard on the Clip menu Deletes the text currently in the editor's clipboard. Browse Mode on the Options menu This option determines if the currently edited file can be changed or not. If Browse Mode is on, the file can not be changed by editing commands, and cannot be saved to disk. If Browse Mode is off, the file can be changed via normal editing commands. Change Dir on the File Menu Allows you to change the editor's current working directory, without typing the entire directory name. New File on the File Menu - [Windows](#) Clipboard enhancements: The editor can now cut and paste column blocks into/out of the Windows Clipboard. If a TSE Pro/32 Column block is pasted into another application, the block will be pasted as a normal windows style block, with each row being a separate line. - Color [Syntax](#) highlighting enhancements: The ToEOL Delimiter tokens support an additional property, making them optionally column based. If the column for a ToEOL Delimiter is 0, then the Delimiter can occur anywhere on a line, otherwise, it can only occur at the specified column. Also, the delimiter can be required to be the first non-whitespace token on a line. To specify this, set the column to -1. This should be helpful for COBOL, FORTRAN and other languages, that have column specific [comments](#). Additionally, a new Keywords WordSet property is settable, for various languages valid wordsets. Finally, there are several new Numeric Formats supported (Pascal Style, Real [Numbers](#), and Customized Set). - New [Command-Line Options](#): -s for recurSe through Subdirectories Will load all matching files in the specified directory, and will also search in subdirectories. -iDirectoryPath to instruct the editor to extend its view of the "load directory" to an additional directory as specified by DirectoryPath. - Additional video modes: 30x90, 30x94, 34x90, 34x94, 40x90, 40x94. Also, a custom number of rows and columns can be set via the Change Video Mode menu that's available from the Util menu. The Full Configuration item Startup Video Mode shares these enhancements. - New way to set the editor's TSEPath variable. If a TSEPATH environment variable exists at editor startup, the editor will use this, in lieu of what the editor's TSEPath variable is set to. - [TSELOADDIR](#) and supporting commandline switch. TSELOADDIR is an environment variable that the editor looks for upon startup. The [TSELOADDIR](#) feature addresses two needs that users have requested: 1) To install the editor binary in a separate directory from the rest of the editor. 2) To install the editor on a read-only network drive, but have per-user configuration files. This is useful with a multi-user license of the editor, and you want to install only one copy of the editor, but you want each user to be able to have their own personal configurations. The TSELOADDIR environment variable can also be overridden using the -i command line switch. - Spelling Checker enhancements: The spelling checker now knows about HTML files, and when checking a file with an extension of .html, .htm, .shtml, or .shtm, it only checks text that is [not](#) in-between HTML tags and [comments](#). Additionally, any words starting with an & and ending with an ';' are not checked. For known programming language filetypes, (SAL, C, C++, Java, JavaScript), only text in [comments](#) and [strings](#) is checked. For other types of files, lines starting with a leading '>' are not checked. - Miscellaneous enhancements: - FileManager and Where both allow you to Start a file by its association. - Added compiler support for Delphi, Java, Perl and Free Pascal. - Additional Java support includes code commenting, listing of routines, language indenting, and color [syntax](#) highlighting. - Listing of routines for .asm files added. - Improved Brief emulation. - The compile command supports Tee'ing. - ListOpen can now sort the list, and toggle between sorted and unsorted. - Grep (available from the Potpourri) - allows you to search for a string in multiple disk files, displaying

the results in a CompressedView like listing. - vMatch (available from the Potpourri) can be used to dynamically show matching ([{ characters. - Templates now allow short-cuts to end with non-[wordset](#) characters. - Cuamarking configures itself on initial loading. - Added "Empty" to [named clipboards menu](#). - Tabutil now supports entabbing of column blocks. - Wrappara can now quote a wrapped area, if called as "wrappara quote" - Match will display the number of characters between " or ' on the statusline. - Recent files list now offers "Cleanup List" option. - Keyboard macros, as well as the Shell and DOS commands are available at editor prompts. - Additional PickFile features: enhanced drive listing, ability to set as current directory. - The Where command now allows the filelist to be converted to a normal text file. - If \n is used in a Regular Expression Replace string, the line is split (as if [SplitLine\(\)](#) was invoked) at the point in the replace string where \n is specified. - Number of Files that can be loaded (or [buffers](#) created) increased from 65,535 to 2,147,483,647. - New/Enhanced Command Options: - [ExpandPath\(\)](#) now supports an optional integer specifying whether this command should respect the case of an existing filename. - [GetWord\(\)](#), [isWord\(\)](#), and [MarkWord\(\)](#) now support an optional "user-defined" wordset that, if specified, will be used in lieu of the current WordSet editor variable setting. - [GetKey\(\)](#) now supports optional [integer](#) flags that [affects](#) certain keycodes that are returned by this command. - [PasteFromWinClip\(\)](#) now supports an optional integer that specifies whether this command should overwrite or [insert](#) column blocks. - [PickFile\(\)](#) now supports optional integer flags that affect the string that is returned by this command. **MACRO LANGUAGE SYNTAX: Pre-Defined Constants** - Two [pre-defined constants](#) have been added: - EDITOR_VERSION is a reserved constant that identifies the version of the editor. For example, a value of 0x3000 represents version 3.00. - MAXSTRINGLEN is the editor's maximum string length, 255. **USING MACRO LANGUAGE FEATURES: Event Macros** - The following additional editor events can be hooked to an event macro: _ON_EDITOR_STARTUP_, _AFTER_FILE_SAVE_, _PRE_UPDATE_ALL_WINDOWS_, and _POST_UPDATE_ALL_WINDOWS_. **EDITOR COMMANDS** - The editor now offers over 500 native commands that can be called by user-written [macros](#). The following new commands are available. - Block/ClipBoard commands: [isCurrLineInBlock\(\)](#), [isWinClipAvailable\(\)](#), [CutToWinClip\(\)](#), [CopyAppendToWinClip\(\)](#), [CutAppendToWinClip\(\)](#), [EmptyWinClip\(\)](#) - Buffer commands: [BrowseMode\(\)](#), [ClearBufferDaTmAttr\(\)](#), [GetBufferDaTmAttr\(\)](#), [UpdateBufferDaTmAttr\(\)](#) - Buffer Global Variable commands: [DelBufferVar\(\)](#), [ExistBufferVar\(\)](#), [GetBufferInt\(\)](#), [GetBufferStr\(\)](#), [SetBufferInt\(\)](#), [SetBufferStr\(\)](#) - Editing commands: [ClearUndoRedoList\(\)](#), [Redo\(\)](#), [Undo\(\)](#) - File commands: [ChangedFilesExist\(\)](#), [LockCurrentFile\(\)](#), [UnLockCurrentFile\(\)](#) - History commands: [LoadHistory\(\)](#), [SaveHistory\(\)](#) - Hook/Event commands: [TerminateEvent\(\)](#) - Initialization File commands: [GetProfileInt\(\)](#), [GetProfileStr\(\)](#), [WriteProfileInt\(\)](#), [WriteProfileStr\(\)](#), [LoadProfileSection\(\)](#), [GetNextProfileItem\(\)](#), [LoadProfileSectionNames\(\)](#), [GetNextProfileSectionName\(\)](#), [RemoveProfileItem\(\)](#), [RemoveProfileSection\(\)](#), [FlushProfile\(\)](#) - Keyboard commands: [MsgBox\(\)](#), [MsgBoxBuff\(\)](#), [MsgBoxEx\(\)](#) - **Menu** commands: [SetMenuBar\(\)](#) - Miscellaneous commands: [isAlpha\(\)](#), [isAlphaNum\(\)](#), [isDigit\(\)](#), [isHexDigit\(\)](#), [isLower\(\)](#), [isUpper\(\)](#) [RedoCount\(\)](#), [SetUndoOff\(\)](#), [SetUndoOn\(\)](#), [UndoCount\(\)](#), [UpdateDisplayFlags\(\)](#) - Position commands: [KillLocation\(\)](#), [PopLocation\(\)](#), [PushLocation\(\)](#), [GetPositionInfo\(\)](#), [SwapPosition\(\)](#) - OS Access commands: [AddFFInfoToBuffer\(\)](#), [is32BitApp\(\)](#), [isFullScreen\(\)](#), [PickDrive\(\)](#), [SqueezePath\(\)](#), [StartedFromDosPrompt\(\)](#), [WhichOS\(\)](#) - **String** variable manipulation commands: [CmpiStr\(\)](#), [EquiStr\(\)](#), [GetFileToken\(\)](#), [NumFileTokens\(\)](#), [RightStr\(\)](#) - Tab manipulation commands: [EntabCurrLine\(\)](#) - Window commands: [SetWindowHeight\(\)](#), [SetWindowWidth\(\)](#) - The following [editor commands](#) are now obsolete: CurrLinePtr(), GetVideoInfo() **EDITOR VARIABLES** - Two new options have been added to [StartupFlags](#): - _STARTUP_RESTORESTATE_ instructs the editor to restore the editing environment from a previous "saved" session. - _STARTUP_MENU_ displays a selection menu of available startup options. - The following new [editor variables](#) are now available: - [UseCommonDialogs](#) If On, then [EditFile\(\)](#), [SaveAs\(\)](#), [SaveBlock\(\)](#), [ReplaceFile\(\)](#), and [ChangeCurrFilename\(\)](#) use the Windows common dialogs to prompt for filenames rather than the TSE Pro prompt boxes. - [KeepUndoBeyondSave](#) If On, Undo information for a particular file is kept until that file is no longer loaded. If Off, Undo information for a particular file is purged [when](#) that file is saved. - The following [editor variables](#) are now obsolete: RestoreDirOnExit, LeftHandedMouse (Adjust via [Windows](#) Control Panel instead) - Changes to operation of [ShowMainMenu](#): If ShowMainMenu is On, the Main Menu occupies the first row of the console window instead of the second row. - Added 4 digit year formats to DateFormat. 4 digit is now the default. - Increased a few maximum

values for History variables: [MaxHistorySize](#): Maximum has been increased from 1,000 to 10,000.

[MaxHistoryPerList](#): Maximum has been increased from 100 to 1,000. [MaxRecentFiles](#): Maximum has been increased from 1,000 to 5,000. - Changes to some editor variable initial ([default](#)) options: [CenterFinds default](#) option changed from On to Off. [DateFormat default](#) option changed from 1 to 4. [MaxHistorySize default](#) option changed from 500 to 1000. [MaxHistoryPerList default](#) option changed from 20 to 50. [MaxRecentFiles default](#) option changed from 20 to 200. [StartupFlags default](#) option changed from _STARTUP_PROMPT_ to _STARTUP_MENU_. [PersistentHistory](#), [PersistentRecentFiles](#), and [ShowMainMenu default](#) option changed from Off to On. [StatusLineAttr default](#) option changed from [Color](#)(Black on White) to [Color](#)(White on Red), so that it will not be the same color as the Main [Menu](#) bar. - Appendices of [the Macro Reference Guide](#) in the on-line help have been revised in accordance with additions and enhancements made in version 3.0 of The SemWare Editor Professional/32. [APPENDIX C: COMMANDS BY CATEGORY](#) [APPENDIX D: VARIABLES BY CATEGORY](#) - Appendices of [the User's Guide](#) in the on-line help have been revised in accordance with additions and enhancements made in version 3.0 of The SemWare Editor Professional/32. [APPENDIX C: STANDARD KEY ASSIGNMENTS](#) Also see: What's New in Version 2.8

SwapPosition

Changes the order of the positions pushed with the

[PushPosition\(\)](#) command. [Syntax](#): BOOLEAN [SwapPosition](#)([[INTEGER](#) n]) - n is the optional index into the position stack. If n is [not](#) specified, 2 is used. Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: If called without specifying n, a value of 2 is used, which causes the last pushed position to be swapped with the next-to-last pushed position. Otherwise, the top-most position is swapped with the nth position, where n is only considered if it is greater than 1. If n is less than 1, no swapping occurs. If there are no pushed positions, or n is greater than the number of pushed positions, or n is less than 1, this command returns [FALSE](#). [Examples](#): [SwapPosition\(\)](#) // swap top two positions - returns false if // less than two positions pushed [SwapPosition\(2\)](#) // same affect as above [SwapPosition\(1\)](#) // no swap occurs - returns [FALSE](#) if no // positions pushed; [otherwise](#) non-zero. [SwapPosition\(0\)](#) // no swap occurs - returns [FALSE](#) [SwapPosition\(-1\)](#) // no swap occurs - returns [FALSE](#) See Also: [GetPositionInfo\(\)](#), [PushPosition\(\)](#), [PopPosition\(\)](#)

Switching Between Windows

Editing takes place only in the current window. To switch to another window for editing in that window, the following commands are available.

* GotoWindow (<Ctrl O G>, or "Go to" under Window

[Menu](#)) This command switches to a window adjacent to the current window. The cursor keys are used to point in the direction of the desired window. With this command you can go directly to any window that shares a common edge with the current window. When you execute GotoWindow, the editor displays [the StatusLine](#) prompt: Go to window (press [cursor key](#)): Press <CursorUp>, <CursorDown>, <CursorLeft>, or <CursorRight>, as appropriate, to immediately move to the desired adjacent window. * GotoWindowNumber (<Alt 1> ... <Alt 9>) This command switches to a specific window number, and is the quickest and most convenient way to change [windows](#). Keys <Alt 1> through <Alt 9> cause the editor to switch to the window whose number corresponds to the number that you press. To invoke this command, press <Alt> and the appropriate number key for the desired window. Use the number keys located above the alphabetic keys, NOT the number keys on the numeric keypad. * NextWindow (<Ctrl O N>) This command moves the cursor to the next window, in numerical order by window number. * PrevWindow (<Ctrl O P>) This command moves the cursor to the previous window, in numerical order by window number.

Synchronized Scrolling in Windows

You can scroll simultaneously in multiple

[windows](#) by setting Synchronized Scrolling mode ON. This causes cursor movement and scrolling activity in the current window to also occur in each of the non-current windows. * ToggleSyncScroll (<Ctrl O Y>, or "Toggle Sync Scrolling" under Window [Menu](#)) This command switches Synchronized Scrolling mode [ON](#) and [OFF](#). When this mode is ON, the editor attempts to synchronize logical cursor movement, as well as scrolling, in all [windows](#), based on [cursor](#) movement and scrolling activity in the current window. However, the logical cursor position and size of non-current [windows](#) may affect scrolling that occurs when the cursor reaches a window edge. Under certain conditions, movement of the cursor may cause scrolling to occur in some non-current windows either before or after it begins in the current window.

SyntaxHilite Mode

[SyntaxHilite mode](#) allows you to distinctly colorize syntactical elements of your program code (or of any text file). When [SyntaxHilite mode](#) is in effect, certain elements of the text are displayed in selected colors to allow you to easily identify and differentiate various portions of your code. The colorization of text in an editing session is governed by file extension. One or more file extensions are associated with an array of SyntaxHilite colors that have been assigned to various classes of text elements. When you edit a file with an extension that has been associated with SyntaxHilite colors, the color-mapped elements of the text will automatically be colorized in accordance with the designated colors (if the editor has been configured to display SyntaxHilite colors). To use SyntaxHilite mode, you will need to do the following: - Select colors for each class of elements (such as keywords, [comments](#), and quoted [strings](#)) to be colorized. (A set of [default](#) colors have been pre-selected. You can modify these according to your preferences.) - Identify specific character strings, or tokens (such as a group of actual function names, or a pair of comment delimiters) within a class, and map each token to one of your pre-selected colors for that class. An array of these color-mapped tokens is placed in a SyntaxHilite mapping file. This mapping file will be used to determine the various portions of text that are to be colorized, and the specified colors to be used. You can create many different mapping files, to achieve different color schemes for different types of files [to](#) be edited. (Several pre-configured mapping files have been provided in the editor, including one for C/C++ and one for SAL. You can modify these, or add your own.) - Associate each SyntaxHilite mapping file with the desired group of files for that color scheme, identified [by](#) extension. ([Default](#) associations have been made for the pre-configured mappings, which you can change or remove, as desired.) - Set ON the option to display SyntaxHilite colors. Each time you edit a file with an extension that has been associated with a SyntaxHilite mapping file, [SyntaxHilite mode](#) will be in effect. Any such file will be colorized according to its associated mapping file. You can make future changes to any of your SyntaxHilite configurations, as desired. This includes your color selections, token mappings, or extension associations. The following sections describe the classes of text elements that can be colorized, and explain how to configure the various SyntaxHilite mode options. [Element](#)

[Classes Recognized by SyntaxHilite Mode](#) ----- Following are the classes of elements that can be colorized. - Multi-line delimited elements A multi-line delimited element is text (typically [comments](#)) that spans multiple lines, and whose beginning and end are marked by a set of tokens. All text between and including the beginning and ending tokens is displayed in the indicated color. - Single-line delimited elements A single-line delimited element is text (typically a comment) that is limited to a single line, and whose beginning and end are marked by a set of tokens. All text between and including the beginning and ending tokens is displayed in the indicated color. - To-EOL delimited elements A to-EOL delimited element is text (typically a comment) that is limited to a single line, whose beginning is marked by a token, and whose end is marked simply by the end of the line. No ending token is required. All text from and including the beginning token, through the end of the line, is displayed in the indicated color. A to-EOL delimited element can be either column-specific or free-floating. If the token must occur in a specific column to be considered a to-EOL delimiter, then the applicable column must be specified for that element. If the token can occur anywhere on the line to be considered a to-EOL delimiter, then the column must be specified as

0 (zero). For languages such as COBOL and FORTRAN, it can be helpful to indicate the appropriate column for to-EOL delimiters. Additionally, the token can be required to be the first non-whitespace token on a line. To specify this, set the column to -1.

- **Quote delimited elements** A quote delimited element is text (typically a "quoted string") that is limited to a single line, and whose beginning and end are marked by the same token value. All text between and including the beginning and ending tokens is displayed in the indicated color.
- **Incomplete quote elements** An incomplete quote element is an unmatched quote token. That is, it is the single occurrence of a quote token on a line, without a second occurrence of that token on the same line. Only the token itself is displayed in the indicated color.
- **Directive elements** A directive element is text (such as a conditional compilation directive) that is limited to a single line, and whose beginning is marked by a token that occurs as the first non-white text on a line, and whose end is marked by the first occurrence of a space character (or the end of the line). All text from and including the beginning token, through the end of the string of non-space characters, is displayed in the indicated color.
- **Keyword elements** A keyword element is text (such as a function name, reserved word, or symbol) that simply matches the characters of a token. The matching text is displayed in the indicated color.
- **Number elements** A number element is text (such as a numeric string, a numeric expression, or a decimal number) that matches a selected number token style. The number text is displayed in the indicated color.

[Selecting Colors for SyntaxHilite Classes](#) ----- To select the colors to be used by SyntaxHilite mode, select "Full Configuration" under the Options Menu and choose "[Display/Color Options](#)," then select "[Set Colors](#)" and choose "Select SyntaxHilite Colors." You then select from the various classes of elements and designate the desired colors. For most of the element classes, you can specify multiple colors. For those that allow multiple colors, each different color attribute is identified by a consecutive number within that class, beginning with the number 1. (This class number is then referenced when mapping a specific token for a class to the desired color.) The following identifies the number of different colors you can select for each class of elements.

Number of Color Class Colors	_____	Multi-line delimited	3	Single-line delimited	3	To-EOL delimited	3	Quote delimited	3	Incomplete quote	1	Directive	3	Keyword	9	Number	1
------------------------------	-------	----------------------	---	-----------------------	---	------------------	---	-----------------	---	------------------	---	-----------	---	---------	---	--------	---

(For more information on setting the SyntaxHilite colors, see "[Set Colors](#)" under "[Display/Color Options](#)" in the chapter on "[Configuring the Editor Interactively](#).")

[Mapping SyntaxHilite Tokens to Colors & Setting Other Options](#) ----- To map specific tokens to your selected colors, select "[Full Configuration](#)" under the Options Menu and choose "[Display/Color Options](#)," then select "[Configure SyntaxHilite Mapping Sets](#)." You then choose whether to update the existing mapping set associated with the current file you are editing (if applicable); select another existing mapping set to update; or create a new mapping set. Mapping sets are maintained in special SyntaxHilite mapping files. You select a directory and a name for the file when you create a new one. The only restriction on the names of these SyntaxHilite mapping files is that they must have an extension of ".SYN". Once you select the mapping set to configure, you can then identify specific tokens for various classes of elements. These tokens are automatically mapped to the pre-selected color associated with that class. For those classes that allow the selection of multiple colors, the specific color is determined by the class element number to which you assign the token(s). Tokens are specified as follows:

- [Multi-line delimiter tokens](#) Multi-line delimiter tokens must be specified in pairs, as a beginning delimiter and an ending delimiter. Each token can be multiple characters. You can assign up to three pairs of multi-line delimiter tokens. The editor allows you to select "[#if 0](#)" / "[#endif](#)" (or "[#else](#)") as a multi-line delimiter token pair. To do this, turn ON the option to highlight "[#if 0](#)" style [comments](#). This token pair will be mapped to the color of the first "open" multi-line delimiter element number in the mapping set (if one is available).
- [Single-line delimiter tokens](#) Single-line delimiter tokens must be specified in pairs, as a beginning delimiter and an ending delimiter. Each token can be multiple characters. You can assign up to three pairs of single-line delimiter tokens.
- [To-EOL delimiter tokens](#) For to-EOL delimiter tokens, you specify the beginning delimiter (and no ending delimiter). However, each delimiter can be multiple characters. You can assign up to three [to-EOL delimiter tokens](#). For each to-EOL delimiter token, you can also specify an optional beginning column number. If the token must occur in a specific column to be considered a to-EOL delimiter, then specify the applicable column for that element. If the token can occur anywhere on the line to be considered a to-EOL delimiter, then the column must be specified as 0 (zero). For languages such as COBOL and FORTRAN, it can be helpful to indicate the appropriate column for to-EOL delimiters. Also, the delimiter can be required to be the first non-whitespace token on a line. To specify this, set the column to -1.
- [Quote delimiter tokens](#) For quote delimiter tokens, you specify a single delimiter that is used as both the beginning and the ending delimiter. However, each delimiter can be multiple characters. You can assign up to three

quote delimiter tokens. For any single-character quote delimiter token, you can also specify an "escape" character. If the editor encounters an occurrence of that single-character quote token, which is immediately preceded by the indicated "escape" character for that quote token, the editor will not treat that occurrence of the quote character as a SyntaxHilite token. - [Directive tokens](#) For each directive token, you specify a single string of one or more characters. You can assign up to three [directive tokens](#). - [Keyword tokens](#) [Keyword tokens](#) are specified in groups, as multiple [strings](#) of one or more characters. You can assign up to nine groups of [keyword tokens](#). - [Keyword Wordset](#) All characters [not](#) in this set are used as keyword delimiters. See the Wordset editor variable for information concerning specification of these characters. - [Number tokens](#) Number tokens are specified by selecting a number token "style." You can choose one of the following styles: Number Style Token Value _____

None No [numbers](#) are specified Integers Any string consisting of one or more of the numeric characters 0 - 9 (without a decimal position) C Style Any string recognized as a number in C: includes integers, 0xf-style hex [numbers](#), real [numbers](#), scientific notation, integers designated as unsigned or long (with a trailing "u" or "l," respectively) Pascal Any string recognized as a number in (Borland) Pascal: Style includes integers, \$0f style hex [numbers](#), real [numbers](#), and scientific notation. SAL Style Any string recognized as a number by SAL Real Integers with a leading or embedded decimal point. [Numbers](#) Comma-sep Any string consisting of one or more of the numeric characters 0 - 9, the decimal point, and the comma Customized Allows one to pick any combination of the following Set styles: Integers 0xf Style Hex 0fh Style Hex \$0f Style Hex %01 Style Binary Real [Numbers](#) Scientific Notation Double Precision Scientific Notation Trailing u/l on Integers, for the C language Comma-separated, for COBOL and other use You can also set other [SyntaxHilite mode](#) options: - Whether or [not](#) to colorize incomplete quotes. - Whether or not to ignore the case of alphabetic characters included in a token. - Whether or [not](#) to apply "transparent" colorization. If you choose transparent, highlighted text will be displayed using the applicable SyntaxHilite foreground color, but against the normal editing text background color. - Precedence of colors for the cursor line. For colorization of highlighted text on the cursor line, you can choose to have the normal cursor-line color override the applicable SyntaxHilite color; or have the applicable SyntaxHilite color override the cursor-line color; or have the applicable SyntaxHilite foreground color applied against the cursor-line background color (which is termed as "transparent"). [Associating a SyntaxHilite Mapping Set with a Group of Files](#)

----- On the same sub-menu you use to configure a particular SyntaxHilite mapping set (described above under "[Mapping SyntaxHilite Tokens to Colors & Setting Other Options](#)"), you can also associate the SyntaxHilite mapping set with a group of files. To do this, select "Associated File Extensions" from the sub-menu. Then enter the extensions of the file(s) that you want to be highlighted according to the indicated SyntaxHilite mapping (.SYN) file. The following conventions apply when entering extensions: - The leading "." is optional. - Multiple extensions must be separated by a space. - Any given extension [string](#) cannot be associated with multiple SyntaxHilite mapping files. - To indicate the set of "all files without an extension," type just a "." (followed immediately by a space, before entering any additional extensions). - The wildcard characters "*" [and](#) "?" can be used. (Precedence in matching will be given to the more explicit specifications. For example, if you associate "c*" with a particular ".SYN" file, and you associate "*" with another ".SYN" file, then files with an extension of "c" or "c01" will match the "c*" association, rather than the "*" association.) To assist you in maintaining all of your SyntaxHilite mapping sets, you can display a modifiable list of the available mapping (.SYN) files, including the extensions associated with each. From this list, you can change the name of any mapping file, or change the group of extensions associated with any mapping file. To access this modifiable list, select "[Full Configuration](#)" under the Options Menu and choose "[Display/Color Options](#)," then select "[Configure SyntaxHilite Mapping Sets](#)," [and](#) choose "Configure Associations." [Setting the Option to Display SyntaxHilite Colors](#) -----

The editor automatically activates SyntaxHilite mode anytime you edit a file whose extension has been associated with an existing SyntaxHilite mapping (.SYN) file. However, you can toggle whether or [not](#) SyntaxHilite colors are actually displayed on the screen during an editing session. To toggle SyntaxHilite color display [ON](#) and [OFF](#), select "[Full Configuration](#)" under the Options Menu and choose "[Display/Color Options](#)," then select "Display SyntaxHilite Colors." For SyntaxHilite colors to be displayed, set this option [ON](#). [Other SyntaxHilite Options](#) -----

The editor's Potpourri facility includes a helpful SyntaxHilite macro that allows you to dynamically activate or terminate SyntaxHilite mode for the current file, and also to dynamically load an arbitrary mapping (.SYN) file and

associate it with the current file. For more information, see the SynHiUtl macro under "[Executing Supplemental Macros from the Potpourri](#)" in the chapter on "[Special Commands and Features](#)." The editor also includes advanced SyntaxHilite options that allow you to access the settings in a SyntaxHilite mapping file, via a text file. To access these options, select "[Full Configuration](#)" under the Options Menu and choose "[Display/Color Options](#)," then select "Configure SyntaxHilite Mapping Sets." The sub-menu that displays includes options that allow you to: - Write the current SyntaxHilite settings to a mapping "text" file - Overlay the settings of the current SyntaxHilite mapping file with the settings from a previously-written mapping "text" file

Table Of Contents

The SemWare Editor HelpSystem

+-----+-----+-----+-----+	
Using Help	SemWare License Agreement
+-----+-----+-----+-----+	
Macros Basic Concepts +-----+ Compiled Macros Key Assignments What's New in Version 4.4 KeyBoard Macros +-----*-----*-----+ Files Blocks Special Features Mouse BookMarks ASCII Chart Windows ClipBoards Compiling +-----+ Cursor Movement Compress View Configuring Deleting/Inserting Text Function List +-----+ Printing Line Drawing Editing Modes Search & Replace Potpourri SyntaxHilite mode Regular Expressions Spell Checking Hex/ Binary mode Incremental Search Templates AutoIndent mode Speed Search WordWrap mode Sorting +-----+ Language-Specific Tabs Appendixes +-----+	
+-----+-----+-----+-----+	

TabLeft

Moves the cursor to the previous tab stop, deleting blanks if Insert is ON.

Syntax: [INTEGER TabLeft\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: When Insert is ON, the editor will not delete blanks if there is a non-blank within the tab interval. For a [TabLeft\(\)](#) that never deletes, but just goes to the previous tab stop, use the following key definition: <Shift Tab> [GotoColumn\(CurrCol\(\)\) - DistanceToTab\(1\)](#) See Also: [TabRight\(\)](#), [DistanceToTab\(\)](#) **Variables:** [CursorTabWidth](#), [TabType](#), [TabWidth](#), [VarTabs](#), [Insert](#)

TabRight

Moves the cursor to the next tab stop, inserting a tab character or spaces (depending on the TabType setting) if Insert is ON.

Syntax: [INTEGER TabRight\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This command returns zero if the cursor column does not change. This command's behavior is affected by several settings, including [Insert](#), [TabWidth](#), [TabType](#), and [VarTabs](#). When Insert is OFF, the cursor goes to the next tab stop as determined by [TabType](#), [TabWidth](#), and [VarTabs](#). No text is changed or inserted. When Insert is ON, the command's behavior is based on the setting of [TabType](#): - If TabType is set to Hard tabs, a tab character is inserted into the text when the [TabRight\(\)](#) command is issued. - If TabType is set to Soft tabs, spaces are inserted to the next tab stop, based on the TabWidth variable. - If TabType is set to Smart tabs, spaces are inserted to the next tab stop, based on the nearest

previous line that has non-white characters past the current cursor column. - If TabType is set to Variable tabs, spaces are inserted to the next tab stop, based on the VarTabs variable. For a [TabRight\(\)](#) that never inserts, but just goes to the next tab stop, use the following key definition: <Tab> [GotoColumn\(CurrCol\(\) + DistanceToTab\(\)\)](#) See Also: [TabLeft\(\)](#), [DistanceToTab\(\)](#) **Variables:** [CursorTabWidth](#), [TabType](#), [TabWidth](#), [VarTabs](#), [Insert](#)

TabSet

Used to create the string to be assigned to the VarTabs variable.

Syntax: [STRING TabSet\(CONSTANT STRING s\)](#) - s is a string of numbers that represent tab stop locations, with each number being separated by a space. s must be a [constant](#) or literal [string](#). Returns: A bit-encoded 32-character string consisting of the specified tab stop locations. Notes: This command is used solely in conjunction with the VarTabs variable. The string passed to [TabSet\(\)](#) must be a constant or literal string. (It can not be a string variable.) To set the VarTabs variable within a config/endconfig construct, you must use the following format: [VarTabs = TabSet\("8 16 24 40"\)](#) **Examples:** In the following procedure, the current VarTabs and TabType values are saved to local variables at the beginning, then restored to their original values at the end of the procedure. [proc](#) Foo() [string](#) saved_tabset[32] // must be 32 chars [integer](#) saved_tabtype // save current tabset and set the new one saved_tabset = [Set\(VarTabs, TabSet\("8 16 24 40"\)\)](#) saved_tabtype = [Set\(TabType, _VARIABLE_\)](#) // do something here // then restore the original values [Set\(VarTabs, saved_tabset\)](#) [Set\(TabType, saved_tabtype\)](#) [end](#) See Also: **Variables:** [VarTabs](#)

TabShiftsBlock

Determines whether the tab key is allowed to shift a block.

Values: BOOLEAN **Default:** [OFF](#) Notes: When TabShiftsBlock is set On, and [<Shift>] <Tab> is pressed, the behavior is as follows: If Insert Mode is On (controlled by the Insert variable), and the cursor is within a block, the block is shifted one TabWidth. The direction is determined by the presence of the <Shift> key. If <Shift> is also pressed, the block is shifted one TabWidth to the left. If <Shift> is not pressed, the block is shifted one TabWidth to the right. This command is implemented in the standard *.ui files. In order for this command to work, the following key assignments should be used: <Tab> mTabRight() <Shift Tab> mTabLeft() If you are using a custom .ui file, you can find the implementation of this command in the mTabRight() and mTabLeft() macro procedures in the tse.ui file. **Affects:** The <Tab> and <Shift Tab> keys. See Also: [Insert](#)

TabType

Determines the editor's treatment of the

[TabRight\(\)](#) and [TabLeft\(\)](#) commands. **Values:** [INTEGER](#) - [_HARD_](#) - [_SOFT_](#) - [_SMART_](#) - [_VARIABLE_](#) **Default:** [_SOFT_](#) Notes: When [Insert](#) is [ON](#) and [TabType](#) is set to [_HARD_](#), actual tab characters (ASCII 9) are placed in the file when the [TabRight\(\)](#) command is invoked. If TabType is set to any other setting, the appropriate number of spaces are placed in the file. When Insert is OFF, only the cursor is moved to the next tab stop location. For more information on the effects of TabType, see the chapter on "[Using Tabs](#)" in the User's Guide. **Affects:** Commands: [TabRight\(\)](#), [TabLeft\(\)](#) See Also: [CursorTabWidth](#), [TabWidth](#), [VarTabs](#)

TabWidth

Determines where TabStops should be set when

[TabType](#) is `_HARD_` or `_SOFT_`. [Values](#): [INTEGER](#) - [1..16] [Default](#): 8 Notes: This setting determines the cursor movement associated with issuing the [TabRight\(\)](#) or [TabLeft\(\)](#) commands. It also affects the screen display when real or hard tab characters exist in the file being edited. See the [TabRight\(\)](#) and [TabLeft\(\)](#) commands for a detailed discussion of tabs. For more information on the effects of TabWidth, see the chapter on "[Using Tabs](#)" in the [User's Guide](#). [Affects](#): [DetabOnLoad](#), [EntabOnSave](#) Commands: [EntabCurrLine\(\)](#), [ExpandTabsToSpaces\(\)](#), [TabRight\(\)](#), [TabLeft\(\)](#) See Also: [CursorTabWidth](#), [DetabOnLoad](#), [EntabOnSave](#), [TabType](#), [VarTabs](#)

Template Mode

Template mode is a versatile feature that allows you to minimize typing when entering repetitive text or generating text in a pre-defined structure. With this feature, an abbreviated text string is identified as a "keyword" and assigned to a user-defined template of expanded text. Template mode operates by expanding the abbreviated text (the keyword) into the text defined in the associated template.

Templates can be useful for entering long words or phrases that you use often, such as your name or your address. They can also be used to create "forms" for guiding the entry of text according to a pre-defined format. Simply assign the expanded text (the template) to the abbreviation (keyword) of your choice. The next time you want to include your name or address in a document, or insert a text-entry form into a file, just type the assigned keyword and press the template-expansion key. The editor replaces the abbreviation with the text defined in the template.

Templates also help save time when entering program code by allowing you to create a template for any regular construct, such as "

[if](#)" or "[while](#)" statements. A number of templates are included in the editor for C, Pascal, xBase, and SemWare's macro language (SAL). A number of Template options are available, allowing you to: - display a PickList of templates in the Template "dictionary" (from which you can expand, add, or delete a template) - expand a keyword into text defined in a template - create your own template and add it to the Template "dictionary" - delete an existing template from the Template "dictionary" Templates that you create are saved by the editor, and are available during each editing session. You can define templates as either local or global. A local template is readily available only for a specified file extension; a global template is readily available for all file extensions. [Basic Operation of Template Mode](#)

----- To toggle Template mode ON and OFF, choose "Template Expansion" under the Util [Menu](#). To use [Template mode](#), you must toggle it [ON](#). When Template mode is ON, the <Tab> and <Ctrl Tab> keys are configured to perform special Template operations. <Tab> expands a template-defined keyword into its pre-assigned word, phrase, or series of lines. (If the cursor is not positioned at the end of a Template-defined keyword when you press <Tab>, the <Tab> key behaves as normal; that is, it executes the TabRight command.) Pressing <Ctrl Tab> pops up the Template menu. The simplest way to invoke Template expansion for a keyword in your text is as follows: - Place the cursor at the end of the keyword; that is, one position past the last character, as it would be if you had just typed the keyword. (If the cursor is positioned within a word, the characters in the word to the immediate left of the cursor will be used as the keyword.) - Press <Tab>. The editor then searches for a (case-insensitive) match in the Template dictionary for the keyword before the cursor. If one is located (and is available to the file extension of the current file), the keyword is replaced with the expanded text from the Template dictionary. Existing text in the file is shifted to accommodate the text inserted from the template. If the keyword is not located (or is not available to the current file extension), the <Tab> key behaves according to its normal action. When the expanded text is inserted in

the current file, the cursor is positioned at the end of the expanded text by default. However, you can specify a particular position in the expanded text on which you want the [cursor](#) to be located instead. You can even temporarily turn on [Insert mode](#) for the line on which the cursor is initially positioned. (These capabilities are described below under "[Creating Your Own Template](#).") To perform other Template operations, use the Template menu. To access the Template [menu](#), press <Ctrl Tab>. The Template [menu](#) provides alternate methods [for](#) expanding keywords. It also allows you [to](#) add, delete, or list templates in the Template dictionary. These options are described in the following sections. [Creating Your Own Template](#) ----- The following describes how to create your own template:

- Type the text as you want it to appear when it is fully expanded.
- If desired, specify the location at which you want the cursor to be positioned by placing "\"c" (without the quotes) at the desired location in the text. By default, if you do not indicate a cursor position, the cursor is placed at the end of the expanded text when the template is inserted.
- If you want to force Insert mode ON for the cursor line when the template is inserted, place "\"C" (uppercase, without the quotes) at the desired location of the cursor. When the template is inserted, the cursor will be positioned at the location indicated by the placement of "\"C". If Insert mode is OFF, it will be set ON until you move the cursor off this cursor line. (Once the cursor moves from this line, Insert mode is set back OFF, if it was OFF when the template was initially expanded.)
- Next mark the expanded text using one of the block-marking commands.
- Press <Ctrl Tab> to display the Template menu.
- Add the currently-marked text to the Template dictionary, either by:
 - Selecting "Add Marked Text as Template" from the Template menu; or
 - Selecting "List Templates (Add/Delete/Expand)" from the Template menu, then pressing <Ins> once the PickList of Templates is displayed.

In either case, the editor prompts you for the keyword to which to assign the template. The case of the keyword is not significant. This keyword should be unique, simple to type, and easy to remember! If you select a keyword that is already in the Template dictionary, you will be given the option to add a duplicate entry, replace the existing entry, or cancel the operation. Duplicate entries can be helpful if you want to use the same keyword for different template expansions, depending on the extension of the file. The editor then prompts you for an extension.

- If you want to limit the immediate availability of the template to a particular file extension (making it a local template), type the desired filename extension, and press <Enter>.
- If you want to make the template readily available for all file extensions (making it a global template), clear the prompt box (press the space bar if the prompt box input area is not empty), and press <Enter>.

Once a template is added, the editor is updated so that the template is available each time you start the editor. [Alternate Methods for Expanding a Template](#) ----- In addition to using the <Tab> key to expand a keyword (as described under "[Basic Operation of Template Mode](#)" above), two other expansion methods are available from the Template menu. The first alternate method is very similar to the <Tab> key method described earlier:

- Place the cursor at the end of the keyword, either on the last character, or one position past the last character as it would be if you had just typed the word.
- Press <Ctrl Tab> to display the Template menu.
- Select "Expand Word Before [Cursor](#)".

The keyword that you typed is replaced with the text in the template defined for that keyword. (If the keyword is not defined, or is not available for the current file extension, no action occurs.) With the second alternate method, you do not have to type the keyword that is to be expanded. This method is useful if you cannot remember a particular keyword, or if you do not want to type the keyword, or if you want to select a particular template for a keyword for which there are duplicates. To use this method:

- Position the cursor where you want the expanded text from the template to be inserted.
- Press <Ctrl Tab> to display the Template menu.
- Select "List Templates (Add/Delete/Expand)".

A PickList is displayed of all template keywords readily available for the current file extension; that is, the keywords for all local templates defined for the current extension, and all global templates.

- If the keyword that you want to expand is not included in this limited PickList, press <Ctrl Enter> to display a PickList of all template keywords that are defined. This PickList includes even those keywords [for](#) templates that are local [to](#) other file extensions.
- Once you have located the desired keyword, move the cursor bar to that keyword and press <Enter>.

The template defined for the selected keyword is inserted into your text at the cursor position. [Deleting an Existing Template](#) ----- To delete a keyword from the Template dictionary:

- Press <Ctrl Tab> to display the Template menu.
- Select "List Templates (Add/Delete/Expand)".

A PickList is displayed of all template keywords readily available for the current file extension.

- If the keyword that you want to delete is not included in this limited PickList, press <Ctrl Enter> to display a PickList of all template keywords that are defined.
- Once you have located the desired keyword, move the cursor bar to that keyword and press .
- The editor displays a prompt box to verify that you want to delete the selected keyword. Place the cursor bar on "Yes" and press <Enter>.

Both the

keyword and its defined template are permanently removed from the Template dictionary. (For more information on making selections from a PickList, see "[Using PickLists](#)" in the chapter on "[Command Prompts and PickLists](#).")

TemplateExpansion

Causes the <Tab> and <Ctrl Tab> keys to perform special Template mode operations.

Values: BOOLEAN **Default:** [OFF](#) **Notes:** When set ON, the <Tab> and <Ctrl Tab> keys are configured to operate in Template mode. <Tab> expands a template-defined keyword into its pre-assigned text. (If the cursor is not positioned at the end of a Template-defined keyword when you press <Tab>, the <Tab> key behaves as it is normally configured.) Pressing <Ctrl Tab> pops up the Template menu. For more information on [Template mode](#), see "[Template Mode](#)" in the chapter on "[Editing Modes](#)" in [the User's Guide](#).

TerminateEvent

Terminates the event for the current event handler, and ends the associated hook chain.

Syntax: [TerminateEvent](#)([INTEGER](#) event_return_code) - event_return_code is an optional integer value specifying the return code to use when the event is terminated. The default value is zero ([FALSE](#)). Returns: Nothing. Notes: [TerminateEvent](#)() only has an effect when called from within a hooked event. Use it when you want to stop the editor event from completing, and you also do [not](#) want any following hooks to be processed. The following events can be terminated with this command: - [_ON_FILE_LOAD_](#) to prevent the file from being loaded. - [_ON_FILE_QUIT_](#) to prevent the file from being abandoned. - [_ON_FILE_SAVE_](#) to prevent the file from being saved. This command is similar to the [BreakHookChain](#)() command; but in addition to stopping the hook chain, it also stops the associated event. See Also: [BreakHookChain](#)(), [Hook](#)(), [UnHook](#)(), [ExecHook](#)(), [SetHookState](#)()

Terminating the Editor

When the editor is terminated, the editor performs a number of processes in a specific order.

First, the editing state is saved, if the editor has been configured for this, and an applicable command is used to exit the editor.

Next, the editor saves any files indicated by the user (depending upon the command used to exit the editor).

After that, if there is a WhenPurged macro procedure in the installed user-interface macro file, it is invoked.

Then, the Recent Files list is saved, if the editor has been configured for this.

Next, if the [_ON_ABANDON_EDITOR_](#) event is hooked, it is called.

Then, entries in the History Buffer are updated, if the editor has been configured for this.

At this point, any open files are closed.

Finally, the editor returns control to the calling program, which is usually the operating system.

(For more information on configuring the editor to save certain information when the editor is terminated, see "

[Saving/Restoring the Current Editing Session](#)" in the chapter on "[Loading, Processing, and Saving Files](#)" [in the User's Guide](#).)

The Customizable Helpline

[The customizable HelpLine](#) displays a single line of key assignments for selected commands. [The HelpLine](#) appears at either the top or bottom of the screen, opposite [the StatusLine](#) location. In the standard configuration, the HelpLine appears at the bottom of the screen. If desired, you can configure the editor so that [the HelpLine](#) is not displayed at all. (For more information on selecting the placement of [the StatusLine](#) and HelpLine, see "StatusLine At Top" [and](#) "Display HelpLine" under "[Display/Color Options](#)" [in](#) the chapter on "[Configuring the Editor Interactively](#)." Four separate HelpLine information bars can be defined. The primary ("unshifted") bar is continually displayed on the screen. Up to three secondary ("shifted") bars can be defined for assignment to the <Shift>, <Alt>, or <Ctrl> keys. To display any one of the secondary bars, press and hold the <Shift>, <Alt>, or <Ctrl> key to which it is assigned. In the standard configuration of the editor, there is a slight delay (less than 1/2 second) before [the HelpLine](#) is updated with the appropriate HelpLine bar once you press one of the "shift" (<Shift>, <Alt>, or <Ctrl>) keys. If you prefer, you can configure the editor to display HelpLine bars immediately (with no delay) once a "shift" key is pressed, or you can lengthen the amount of the delay. (For more information on configuring the length of the delay, see "HelpLine Delay" under "[Display/Color Options](#)" in the chapter on "[Configuring the Editor Interactively](#)." Following are [the HelpLine](#) bars defined in the standard configuration of the editor. They show commands that are assigned to function keys (<F1> ... <F12>). * Primary HelpLine Bar The primary HelpLine bar displays continuously on the screen, and indicates commands assigned to selected non-shifted function keys:

```
+-----+ |F1-Help F2-AddLn F4-DupLn F5-ScrollTop F6-DelToEol
F9-Shell F10-Menu| +-----+ * <Shift> HelpLine Bar The <Shift>
HelpLine bar displays only while the <Shift> key is held down, and indicates commands assigned to selected <Shift>
+ function-key combinations: +-----+ |Shift:F1-Keys F3-
Sort F4-Upper F5-ScrollCtr F7-ShiftLt F8-ShiftRt F10-Potpourri|
```

```
+-----+ * <Alt> HelpLine Bar The <Alt> HelpLine bar
displays only while the <Alt> key is held down, and indicates commands assigned to selected <Alt> + function-key
combinations: +-----+ |Alt: F2-InsLn F3-Match F4-Lower F5-
RollLt F6-RollRt F9-Dos F10-ShowEntry| +-----+ * <Ctrl>
HelpLine Bar The <Ctrl> HelpLine bar displays only while the <Ctrl> key is held down, and indicates commands
assigned to selected <Ctrl> + function-key combinations: +-----+ |Ctrl:
F1-HelpWord F2-SwapLns F7-Shift F9-Compile F10-MacroMenu|
```

```
+-----+ The pull-down Menu sometimes uses the area assigned to the
HelpLine. When it is needed, the HelpLine area is cleared and information about Menu options is displayed. After
exiting the Menu, the HelpLine again displays the selected key assignments. The HelpLine bars can be customized to
display the commands and key assignments (or other text) of your choice. (For more information, see "Changing the
```

[HelpLines](#)" in the chapter on "[Fully Customizing the Editor.](#)")

The Editor Design

Whether you are creating letters, memos, and other documents, or developing program code, the editor is designed for users with a wide variety of feature requirements and operational preferences. To accomplish this, the editor includes an Interactive Configuration facility (IConfig) that allows you to customize much of the behavior of the editor, and can be accessed from within the editor itself.

To provide further configurability, the editor offers the SemWare Applications Language (SAL), a macro programming language that can be used to modify and extend the commands and capabilities of the editor. User-written

[macros](#) can be used to replace or enhance built-in commands and [macros](#), or to add new features to the editor. A number of utility macros are included with the editor software (located in the UI\ and MAC\ editor subdirectories). The editor can also be extended by [using DLLs](#). Additional tasks and procedures, written in another programming language, can be called from the editor.

The Editor Features

The editor allows you to edit any text file with a full set of commands and options. And because of its configurability, you can tailor the editor according to your own editing style.

To access files, you can:

- Invoke the editor from the command-line without creating another window; or, if you prefer, start it from an icon on the Windows desktop or Start

[menu](#) - Simultaneously edit any number of files, including very large files - Select multiple files to edit (using optional wildcard characters) [and](#) load/execute [macros](#) - Lock the files you are editing - Select files using shortcut features like filename completion, PickList of files on disk, PickList of recently-edited files (which can be maintained from one editing session to the next), and SpeedSearch within a PickList - Identify [default](#) file extensions for locating files - Indicate a specific line and/or column number to go to when a file is loaded - Edit lines up to 16,000 characters in length - Search entire drive(s) for specific or wildcarded filename(s) - Manage and manipulate files from within the editor - Open as many as nine resizable [windows](#), horizontally or vertically, in which you can display multiple views of the same file or display different files - Simultaneously scroll the text in multiple windows with synchronized scrolling - Find differences in similar files using file-comparison utilities - Edit files in Hex and/or [Binary mode](#) - Perform search and replace operations across multiple files - Temporarily exit to the operating system, or execute external commands from within the editor - you can even capture the output of an external command and load it into a file To create a custom editing environment, you can: - Reconfigure key assignments, and customize the pull-down Menu, [HelpLines](#), or [Key](#)-Assignment Help screen to suit individual preferences - Choose to have the Menubar display continually at the top of the screen - Extend and customize features of the editor to suit your needs using SAL, the full-featured macro programming language - Make your own macros readily available in the editor from the Potpourri facility - Create your own automatic startup routine for the editor - Specify macros to be loaded each time you start the editor - Call externally-written DLLs to add greater functionality to the editor - Identify the file-selection behavior of the editor that will be in effect when you start the editor from an empty command line - Configure the behavior of file-

related PickLists - Select the line-termination and file-termination character(s) - Adjust the size of the cursor on the display screen - Select a custom number of rows and columns for your editing window - Take advantage of up to 200-column and 100-line screen displays (if available on your system) To accomplish basic editing operations, you can: - Correct editing mistakes with multi-level undo/redo - Restore the previously-saved editing environment in a new editing session - Optionally use your mouse for scrolling, window, block, and [menu](#) operations - Execute commands by pressing a key or by using the customizable pull-down [Menu](#) - Display help information about editor commands and features using the context-sensitive, hypertext HelpSystem; search [the HelpSystem](#) for a specified [string](#) - Manipulate text via line, character, and column blocks; optionally use shifted [cursor](#) keys for [marking](#) blocks - Store and retrieve text using an unlimited number of ClipBoards - Access the Microsoft Windows Clipboard to copy and paste text directly to or from the editor - Perform extensive search and replace operations. Search and replace features [include](#): regular [expressions](#); incremental search; view of all lines with a matching string; and "local" (limited to a block), "global" (throughout the entire current file), and "all" (throughout all files being edited) options - Place [Bookmarks](#) in files to instantly [return](#) to marked positions as needed - In prompt boxes, display a PickList of previous responses to a command, and optionally maintain a history of responses across editing sessions - Use fixed, variable, or "smart" tab settings for entering or changing text - Insert any ASCII character directly into the text from a pop-up ASCII chart - Sort all or a portion of a file, in ascending or descending order, using a sort key that is case sensitive or insensitive - Create Keyboard Macros for executing a series of commands at the touch of a keystroke To accommodate word-processing needs, you can: - Format text using line-centering, automatic wrapping and indentation, and paragraph formatting options - Minimize typing of repetitive text by creating templates for expanding text based on pre-defined abbreviations - Check the spelling of the words in a file using the SpellCheck routine that suggests spellings for possible misspelled words, and allows you [to](#) add words to a user-defined word list - Change the case of characters in a word, on a line, or in a block - Shift a block of text, by one character at a time or by the tab width - Fill an entire block with a string of character(s) - Create tables and diagrams using [LineDrawing](#) mode - Configure the date and time formats - Designate your own set of [WordSet](#) characters - Print directly from the editor, with options like header and footer lines, page numbering, line numbering, assignment of the print device, and adjustable line spacing To tackle programming tasks, you can: - Invoke [SyntaxHilite mode](#) to colorize syntactical elements of your programming language code. - Use automatic indentation modes for various languages - Locate matching bracket characters, such as braces and parentheses - Notate text as a quote or comment - Take advantage of programming templates, or build your own custom templates - List all function or procedure names in a source file; readily locate the next or previous function - Compile from within the editor using multi-language compiler support

The Editor Manuals

The SemWare Editor Professional comes with two manuals to assist you in using the editor: - [The User's Guide](#) - [The Macro Reference Guide](#) [The User's Guide](#) ----- [The User's Guide](#) provides basic information needed to effectively use the editor. It describes editing commands and options available for: - [Starting the editor](#) - [Using the pull-down Menu system](#) - [Getting help in the editor](#) - [Using a mouse](#) - Loading, saving, and closing files - Exiting from the editor - Splitting the editing screen into multiple windows - Navigating through or between files - Performing a wide variety of editing tasks - Printing text - [Creating Keyboard Macros](#) and external Compiled [Macros](#) - Configuring editing options from within the editor - [Customizing the editor](#), including modifying key assignments, Menu entries, and help information The appendices to [the User's Guide](#) provide the following additional information: - Technical specifications of the editor - Key assignments for the standard editor user interface - Native commands or enhanced [macros](#) by which editing commands are implemented - Settings for editing options in the standard user interface The commands presented in [the User's Guide](#) are termed "editing" commands. These are the commands that are executed while you are in an editing session, by pressing the keys to which commands have been assigned, by selecting commands from a menu, or by [using a mouse](#). The commands that are built into the editor, and which form the basis of the editor's functionality, are called "native" commands. These are the commands that are used to implement the available set of editing commands. (Native commands are described in detail in the chapter on "[Editor Commands](#)" in [the Macro Reference Guide](#).) In many cases, editing commands and native commands are synonymous. However, a

number of editing commands are implemented as macros that enhance the behavior of native commands or provide additional functionality not inherent in the native commands. ("[Appendix E: Command Implementation](#)" identifies the native command or enhanced macro by which each editing command is implemented.) Many chapters in [the User's Guide](#) explain how to execute editing commands, either by using the pull-down Menu or by pressing the assigned keystroke(s). Information on using the mouse for editing tasks is provided in a separate chapter. All indicated [Menu](#) selections and [key](#) assignments for commands and options are based on the standard editor configuration settings. For each command that is presented, the standard key assignment (if one is available) is indicated within < >. Also, if available, the Menu selection is listed in quotes followed by the name of the main Menu category under which the option is located. For example, the EditThisFile command is shown as follows: EditThisFile (<Alt E>, or "Open" under File [Menu](#)) This indicates the EditThisFile command can be executed either by pressing <Alt E> (that is, by holding down the <Alt> key, then pressing the <E> key); or by [activating the pull-down Menu](#), choosing the File category, then selecting "Open". The chapters in [the User's Guide](#) begin with a short overview of the features and commands described in that chapter. Because many chapters build on or refer to information covered in other chapters, it may be helpful to first read the introductory portion of each chapter. This will acquaint you with features covered by the documentation and familiarize you with where to find specific information. Note that the instructions in this manual are based on the settings provided by the standard user interface for the editor. Since you can customize the commands in the editor and assign them to different keys, the key assignments and the operation of commands in your working version of the editor may be different from what is described in the manual. Changes you make to option settings may also affect the behavior of particular commands. (For more information on the key assignments for commands in the standard version, see "[Appendix C: Standard Key Assignments](#).") [The Macro Reference Guide](#) ----- [The Macro Reference Guide](#) provides detailed information about the SemWare Applications Language (SAL), including information for creating Compiled Macros using the native commands and variables comprising the editor. SAL is a macro programming language that allows you to further customize and extend the features of the editor beyond what is described in [the User's Guide](#). SAL is fully integrated into the editor; in fact, many of the commands that are built into the editor are actually macros written in SAL. [The Macro Reference Guide](#) provides the following information about SAL and creating Compiled [Macros](#): - Overview of SAL, including creating and executing Compiled [Macros](#) and assigning them to keys, and creating your own [menus](#). - [Syntax](#) for SAL Compiled [Macro procedures](#) - Instructions for effectively developing and using external [macros](#) and user-interface (installed) [macros](#) - Structure of additional features in SAL (such as command/macro [key](#) assignments, [menus](#), and help screens), and methodologies for using SAL features (such as global and local [variables](#), [buffers](#), and [event macros](#)) - Detailed descriptions of the native (built-in) editor commands that are available for use in Compiled [Macros](#) (including [syntax](#), [return values](#), usage notes, [examples](#), and related commands and [variables](#)) - Detailed descriptions of the editor variables that allow you to customize the editing environment (including valid values, default [values](#), usage notes and restrictions, [examples](#), affected commands, and related [variables](#) and commands) The appendices to [the Macro Reference Guide](#) provide the following: - Native commands or enhanced [macros](#) by which editing commands are implemented - Native [editor commands](#) identified by category - [Editor variables](#) identified by category The information presented in [the Macro Reference Guide](#) assumes you have a working knowledge of the editor. Familiarity with [the User's Guide](#) will help you use [the Macro Reference Guide](#).

The End-of-File Marker

To indicate that the end of a file has been reached, an optional End-of-File marker is provided. The following line appears after the last line of a file to signal the end of the file:

```
<*** End of File ***>
```

Since

[the End-of-File marker](#) is optional, the editor can be configured so that it does [not](#) display it. (For more information on displaying this marker, see "Show End-of-File Marker" under "[Display/Color Options](#)" in the chapter on "[Configuring the Editor Interactively](#).")

The Helpline

[The HelpLine](#) is an optional information bar that displays a single line of key assignments for selected commands. [The HelpLine](#) can be customized [to](#) display the desired key assignments. Four separate HelpLine bars can be defined: one that is continually displayed on the screen, and three additional bars that can be displayed by pressing <Shift>, <Alt>, or <Ctrl>. [The HelpLine](#) appears at the top or bottom of the screen, opposite the StatusLine location. For example, if [the StatusLine](#) appears at the top of the screen (as it does in the standard editor setting), [the HelpLine](#) appears at the bottom. You can also configure the editor so that [the HelpLine](#) is not displayed at all on the screen. (For more information on configuring whether [the HelpLine](#) displays, see "Display HelpLine" under "[Display/Color Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") Certain operations in the editor clear [the HelpLine](#) area and use that area to display helpful information. For an example of this, select "Full Configuration" under the Options Menu. (For more information about [the HelpLine](#), see the chapter on "[Getting Help in the Editor](#).")

The Helpsystem

[The HelpSystem](#) is an elaborate, context-sensitive, hypertext system of Help for the editor. It provides online access to detailed information about the editor's commands, options, and macro programming language features. Information in [the HelpSystem](#) is provided through the following features: - Links to related topics and sub-topics - InfoBoxes for selected HelpSystem text keywords - [Table of Contents](#) - Index of selected topics - Keyword access to selected topics - General Search facility to locate a user-specified string - [Key](#)-assignment Help screen for selected commands - Option to re-display the last Help topic viewed - [Context-sensitive Help](#) information access [Obtaining HelpSystem Information](#) ----- [The HelpSystem](#) is a collection of information about features in the editor. This information is composed of descriptions and explanations on a wide variety of topics. Access is provided through pointers, or Links, to specific topic items. These Links are words or phrases that identify a particular topic that can be viewed. Within a topic, there may even be additional Links to related topics and sub-topics. To identify Links in the HelpSystem text, the Link word or phrase is highlighted. When a Link is selected, the associated Link topic is displayed on the screen, and may [include](#) its own Links to other topics. For certain keywords in [the HelpSystem](#) text, supplemental information is provided through InfoBoxes. The keywords that offer InfoBox explanations are highlighted within [the HelpSystem](#) text, in a color different from that of Links. When an InfoBox on a keyword is selected, a pop-up box containing explanatory information is displayed on the screen. Within [the HelpSystem](#), a list of general, high-level HelpSystem topics is maintained in a Table of Contents. To display the Table of Contents from text-editing mode, press <F1> (or select "[Table of Contents](#)" under the Help Menu); to display the Table of Contents from within [the HelpSystem](#), press <Alt C>. When you select a topic from the Table of Contents, information about that topic is displayed on the screen. Included in that information may be Links for sub-topics or other related topics, as well as InfoBox items. As a further aid in using [the HelpSystem](#), an alphabetical Index list of selected topics is available. To display the Index from text-editing mode, select "Index" under the Help Menu; to display the Index from within the HelpSystem, press <Alt I>. When you select a topic from the Index, information about that topic is displayed on the screen, along with any available Links or InfoBox items. An alternative method is provided to allow you to access HelpSystem topics from text-editing mode. By using the KeyWord option, you can display Help for a selected topic with a single keystroke. Just position the cursor in (or to the immediate right of) the keyword in your text for which you want to view Help, then press <Ctrl F1> (or select "Word at Cursor" under the Help Menu). If Help for the indicated keyword is available, you are immediately placed in [the HelpSystem](#) topic for that keyword. If no Help is available, the HelpSystem Index is displayed and you are placed on the Index entry that most closely matches the indicated keyword. [The HelpSystem](#) also offers a Search facility, which allows you to search the HelpSystem for a

specified string. By using this facility, you can search through the text of all the topics in [the HelpSystem](#) for occurrences of the specified string. To access the Search facility from text-editing mode, select "Search" under the Help Menu; to access Search from within the HelpSystem, press <Alt S>. When you access Search, a pop-up menu is displayed, from which you select where you want the search to occur: you can limit the search to the currently-displayed topic ("Current Topic"); or you can search across all topics in [the HelpSystem](#) (beginning with the first topic in [the HelpSystem](#)). When [searching](#) across all topics, you can choose to go immediately to the first occurrence of the string ("All Topics"); or you can choose to generate a PickList of all topics that contain the specified string ("Search & View Topics"), from which you can then select the topic that you want to view. Options are included to allow you to search [forward](#) or backward for the next or previous occurrence within the current topic; search for the next or previous topic containing the specified string; or re-display the topic PickList. The pop-up Search menu includes the available search options, and indicates the key assignments for the options. An additional feature of [the HelpSystem](#) is a customizable key-assignment Help screen for selected commands. In the standard configuration of the editor, the Help screen is composed of commands that are not shown on the pull-down Menu or on any of [the HelpLine](#) bars. To display the key-assignment Help screen, press <Shift F1> (or select "[Key](#) Assignments" under the Help Menu). The Help screen shows commands on the left, and the key assignment for each command (or command set) to the right. (For more information on customizing this Help screen, see "[Modifying the Key-Assignment Help Screen](#)" in the chapter on "[Fully Customizing the Editor](#).") After exiting from [the HelpSystem](#), you can [return](#) to [the HelpSystem](#) and automatically locate and display the text for the last Help topic viewed. To do this, press <Alt F1> (or select "Previous Topic" under the Help Menu). [Navigating through the HelpSystem](#) ----- While in [the HelpSystem](#), you navigate through it by using special HelpSystem [key](#) assignments (or mouse actions). To select a topic Link or InfoBox field, use the <Tab>, <Shift Tab>, and cursor keys to position the cursor bar on the desired field, and press <Enter>. (Or position the mouse pointer on the desired field and click the <LeftBtn>). You can use the editor's SpeedSearch feature within the HelpSystem to quickly locate a desired topic. (For more information on SpeedSearch, see "[SpeedSearch Feature](#)" in the chapter on "[Command Prompts and PickLists](#).") To step back one level at a time, press <Alt B>. To copy [the HelpSystem](#) information for the currently displayed topic to the [System Clipboard](#), press <Grey+>. To display a list of the key and mouse assignments that are active in the HelpSystem, along with the functions performed when an active key or mouse button is pressed, use the Help on Help function. To access it, press <Alt H>. Within [the HelpSystem](#), a "helpline" of certain HelpSystem key assignments is continually displayed at the bottom of [the HelpSystem](#) screen. You can select one of these with the mouse by positioning the mouse pointer on the desired key assignment or function, and clicking the <LeftBtn>. To exit from [the HelpSystem](#) and resume editing, press <Escape> (or click the <RightBtn>). You are returned to the editing state you were in when you invoked [the HelpSystem](#). [Context-Sensitive Help](#) ----- [The HelpSystem](#) includes a convenient feature which places you at an appropriate location or topic within [the HelpSystem](#) based on the context of your current editing state. To invoke [Context-Sensitive Help](#), press <F1>. Depending on where you are when you invoke Help, you are placed in the HelpSystem, as follows: - If you are in [the pull-down Menu system](#), you are placed in the HelpSystem at the topic entry for the current highlighted Menu item. - If you are in a command prompt, you are placed at the topic entry for the command you are executing. - If you are in text-editing mode, you are placed at the Table of Contents. - If you are already in [the HelpSystem](#), you are placed at the Help on Help entry. When you exit from [the HelpSystem](#), you are returned to the editing state from which you invoked Help. [Files Needed by the HelpSystem](#) ----- [The HelpSystem](#) consists of a number of files, each with an extension of ".HLP". In order for you to use [the HelpSystem](#), these files must be available to the editor. When you access [the HelpSystem](#), the editor successively searches the following directories to locate the necessary files: 1. The current directory 2. The directories indicated by the TSEPath variable, as well as the subdirectory HELP\ (if one exists) of each directory indicated by [TSEPath](#) (For more information on the TSEPath variable, see "Path(s) for Supplemental Files" under "[System/File Options](#)" in the chapter on "[Configuring the Editor Interactively](#).") 3. The editor load directory, as well as the subdirectory HELP\ (if one exists) of the editor load directory (For more information on where [the HelpSystem](#) files are placed when you install the editor, see the chapter on "[Installation of The SemWare Editor Professional](#).")

The Pull-Down Menu System

The editor provides two methods for

[executing commands](#) and setting various editing options. The quickest method is to press the keystroke(s) to which the command or option has been assigned. An alternate method is to use the pull-down [Menu](#) system. The [Menu](#) system can be especially useful while learning the editor, and helpful for executing seldom-used commands. This chapter provides the following information on [the pull-down Menu system](#): - [Activating the pull-down Menu](#) - [Accessing sub-menu commands and options](#) - [Exiting from the Menu system](#) - [Key assignments displayed on the Menu](#) - [Pull-down Menu categories](#) The Menu can also be accessed using the mouse. (For more information, see the chapter on "[Mousing Around the Editor](#).") As with many features of the editor, the pull-down Menu is fully customizable. This allows you to change the Menu selections to suit your own editing needs. (For more information on customizing the Menu, see "[Making Changes to the Pull-Down Menu](#)" in the chapter on "[Fully Customizing the Editor](#).")

The Screen Layout

The editor's screen is designed to provide helpful information while you are editing and to allow you to customize the layout of the screen. The basic features of the screen display include:

-

[The StatusLine](#) that displays status information about both the current file and the editing modes in use. - [The End-of-File marker](#) that identifies when the end of a file is reached. - [The HelpLine](#) that offers ready access to selected command [key](#) assignments. - Multi-window capability that allows the screen to be organized as resizable [windows](#) for viewing multiple files or for displaying different views of the same file. - [Mouse symbols and scrollbars](#) that are used for [scrolling text](#) and performing window operations with a mouse. In addition, a main Menubar can be displayed, from which you can access most of the commands and options available in the editor. If desired, you can configure the editor to continually display the main Menu at the top of the screen. The chapter on "[The Pull-Down Menu System](#)" explains how to activate and use the main Menu. The editor also allows you to change the video mode, which governs the number of lines and columns of text displayed on the screen. The chapter on "[Special Commands and Features](#)" tells how to change the video mode being used during the current editing session. The chapter on "[Configuring the Editor Interactively](#)" explains how to select the video mode in effect each time the editor is started.

The Screen Layout With Multiple Windows

A window is a portion of a screen (or a full screen when only one window is open)

[in](#) which you can view and edit text. The screen can be divided into any combination of up to nine horizontal and/or vertical [windows](#). Each window can display a different file, or the same file can be displayed in multiple [windows](#). This can be helpful for comparing, copying, and moving text between files or within the same file. The number of windows that the screen can be divided into does not restrict the number of files that can be opened during an editing session. The number of files that can be opened is limited only by the amount of available virtual memory. The windows created by the editor are tiled. With tiled windows, there is no void or overlap between adjacent windows on the screen. This is similar to tiles that cover a floor. You would not replace a large tile with one half its size, as this would leave a gap in the floor covering. However, you might replace the large tile with two that are each half the size of the larger tile. Window tiles on the screen work much the same way. If windows are added, deleted, or resized, the

other windows are adjusted so that no areas of bare screen are left exposed. Also, tiled [windows](#) do not hide or overlay portions of other [windows](#). Multiple windows are separated by a divider line along adjacent window sides. You can configure the editor to display a full border around each window (including a single window), so that a divider line is placed along each side of every window (including exterior sides on the edges of the screen). To use the scrollbars with a mouse, the editor must be configured to display a full border around each window. Following are illustrations of screens containing one horizontal window and two vertical windows. Note that the same file is being viewed in windows 2 and 3. In the first example, full borders are NOT placed around each window. L 55 C 32 AW 18k *c:\semware\testing.txt ***** [windows.doc](#) *****1**

```
***** testing.txt *****2***** testing.txt *****3** || The cursor is in this window._ |||
||| F1-Help F2-AddLn F4-DupLn F5-ScrollToTop F6-DelToEol F9-Shell F10-Menu In the second example, full
borders ARE placed around each window. L 55 C 32 AW 18k *c:\semware\testing.txt
***** windows.doc *****1*** |||||||||||||||
***** testing.txt *****2***** testing.txt *****3*** ||||| The cursor is in this window._
||||||||||||||||| ***** F1-
Help F2-AddLn F4-DupLn F5-ScrollToTop F6-DelToEol F9-Shell F10-Menu Each window is sequentially numbered,
from 1 to 9, and the window number is displayed on the right side of the top divider line (unless the ZoomWindow
command is executed, as discussed later in this chapter). The filename contained in a given window is also displayed
on the top divider (or "title") line of that window. The current window is the window in which the cursor is located,
and this is the only window in which editing can occur. The StatusLine displays information about the current window,
including the full name and path of the file in that window. In the illustration above, window 3 is the current window.
The window dividers can be customized, allowing you to select the divider line type and color. You can select a
divider line type and color for the current window that is different from that of the non-current window\(s\). (For more
information on configuring window dividers and borders, see "Display/Color Options" in the chapter on "Configuring
the Editor Interactively".)

```

The Statusline

[The StatusLine](#) is an information bar that displays information on the current file. If more than ONE window is displayed on the screen, [the StatusLine](#) gives information on the current window. (Cursor location determines the current window.) In the standard setting, [the StatusLine](#) appears at the top of the screen. However, you can configure [the StatusLine](#) to appear at either the top or bottom of the screen. (For more information on configuring the placement of [the StatusLine](#), see "StatusLine At Top" under "[Display/Color Options](#)" in the chapter on "[Configuring the Editor Interactively](#)".) The following is an illustration of [the StatusLine](#), with a description of the information items that are displayed. L 49 C 21 IAWRL 3k +*c:\semware\screen.doc ^ ^ ^ ^ ^ ^ || || || || || ||__ Current Path and Filename || || || || || ||__ Modified File Indicator || || || || || ||__ File-Locking Indicator || || || || || ||__ Total Virtual Memory in Use || || || || ||__ Active Mode Indicators || I = [Insert ON](#) | A = [AutoIndent ON](#) or STICKY | W = [WordWrap ON](#) or AUTO | R = RecordKeyMacro [ON](#) | L = LineDraw [ON](#) | || ||__ Column Number | ||__ Line Number - Line Number indicates the line on which the cursor is located. - Column Number indicates the column in which the cursor is located. - Active Mode Indicators identify the editing modes that are currently active. The character identifying a mode is displayed only if that mode is [ON](#) (or [otherwise](#) activated). For example, if [Insert mode](#) is OFF, the "I" does not appear on [the StatusLine](#). (On many monitors, the RecordKeyMacro indicator flashes while a Keyboard Macro is being recorded.) - Total Virtual Memory in Use indicates the amount of memory currently in use by the editor. - File-Locking Indicator displays the "+" symbol to indicate that file-locking is in effect for the current file (if this option is available). - Modified File Indicator displays the "*" symbol to indicate that the current file has changed since the last time the file was saved. - Current Path and Filename identifies the full path and filename of the current file. An optional fill character can be displayed on [the StatusLine](#), to fill unused space. In the sample StatusLine below, the fill character is "-". L 49 -----C 21 ---I-W-----3k-----*c:\semware\screen.doc----- In the standard configuration of the editor, the fill character for the StatusLine is a space. (For more information on specifying the fill character [for the StatusLine](#), see "StatusLine Fill Character" under "[Display/Color Options](#)" in the chapter on "[Configuring the Editor Interactively](#)".)

The User-Defined Word List

SpellCheck allows you to keep a list of additional words that are not in the spelling checker's standard word list (SEMWARE.LEX). These words are accepted if encountered by SpellCheck. The user-defined words are stored in the file USER.LEX.

Words are added to this word list while running SpellCheck, using the "Add to word list" option, as described in the previous section.

Words can also be added to

[the user-defined word list](#) by editing the USER.LEX file. When editing USER.LEX, be sure to follow these rules: - [Include](#) only one word per line, in all lower-[case](#) characters. - Do not leave any blank lines in the file. - Do not leave any leading or trailing spaces on any line of the file. Words that you add to [the user-defined word list](#) can later be removed by editing USER.LEX and deleting the lines containing the unwanted words.

TimeFormat

Determines the time string format returned by

[GetTimeStr\(\)](#). [Values](#): [INTEGER](#) For values of 1 and 2, single digit hours are padded with a space. - 1 = 24-hr (for example: 13:03:48 [and](#) 5:19:54) - 2 = 12-hr (for example: 1:03 pm and 5:19 am) For values of 3 and 4, single digit hours are padded with a zero. - 3 = 24-hr (for example: 13:03:48 [and](#) 05:19:54) - 4 = 12-hr (for example: 01:03 pm and 05:19 am) [Default](#): 2 [Notes](#): [GetTimeStr\(\)](#) is used by [PrintFile\(\)](#) [and](#) [PrintBlock\(\)](#) when printing header and footer lines. TimeFormat affects how the time is printed in a header or footer. [Affects](#): [GetTimeStr\(\)](#) See Also: [TimeSeparator](#), [DateFormat](#), [DateSeparator](#), [PrintHeader](#), [PrintFooter](#)

TimeSeparator

The character used to separate the hours, minutes, and seconds of

[GetTimeStr\(\)](#). [Values](#): [INTEGER](#) ASCII value of the desired character [Default](#): [Asc\(":"\)](#) [Notes](#): [GetTimeStr\(\)](#) is used in the [PrintFile\(\)](#) and [PrintBlock\(\)](#) commands [when](#) printing header and footer lines. [TimeSeparator](#) will affect how the time is printed in a header or footer. [Affects](#): [GetTimeStr\(\)](#) See Also: [TimeFormat](#), [DateFormat](#), [DateSeparator](#), [PrintHeader](#), [PrintFooter](#)

Toggle

Toggles a specified editor variable ON or OFF.

[Syntax](#): [Toggle](#)([INTEGER](#) editor_variable) - editor_variable must be a boolean editor variable; that is, one that only accepts [ON](#) or [OFF](#) ([TRUE](#) or [FALSE](#)) as its setting. Returns: Nothing. [Notes](#): For more information on all the variables in the editor, see the chapter on "[Editor Variables](#)." See Also: [Query\(\)](#), [Set\(\)](#), [ToggleInsert\(\)](#)

ToggleInsert

Toggles the Insert variable.

Syntax: [ToggleInsert\(\)](#) Returns: Nothing. Notes: This command is provided so that [Insert](#) can be set in dialog boxes.

See Also: [Toggle\(\)](#), [Set\(\)](#), [Query\(\)](#)

TrackMouseCursor

Causes the text cursor to track the mouse cursor while a mouse key is held.

Syntax: [TrackMouseCursor\(\)](#) Returns: Nothing. Notes: While [MouseKeyHeld\(\)](#) is TRUE, this command continuously moves the text cursor to the mouse cursor. If the mouse is moved outside the current window, the window is scrolled. However, if you do not have the display screen boxed, and you do not have the HelpLines displayed, you will not be able to scroll the display vertically. Internally this command uses [MouseStatus\(\)](#), which updates [MouseX](#), [MouseY](#), and [MouseKey](#). See Also: [MouseStatus\(\)](#), [MouseKeyHeld\(\)](#)

Transparency

Determines if the underlying desktop/windows will "show-through" the editor's window.

Values: [INTEGER](#) - [0..80] **Default:** 0 Notes: When set to 0, the editor's window is completely opaque in relation to the underlying desktop/windows. As the value of this variable is increased toward the maximum of 80%, the desktop/underlying windows will "show-through" the editor's window with increasing intensity. This only has effect under WIN2000/WINXP and greater.

Trim

Trims leading and trailing white space from a string.

Syntax: [STRING Trim\(STRING s\)](#) - s is the input string. Returns: A copy of s, with leading and trailing white space removed. Notes: The passed [string](#) itself is [not](#) modified. **Examples:** Given: [string](#) s[10] = " test ", s2[10] then: s2 = [Trim\(s\)](#) // s2 is "test", // s is still " test " s = [Trim\(s\)](#) // s is "test" s = [Trim\(" abc "\)](#) // s is "abc" See Also: [LTrim\(\)](#), [RTrim\(\)](#)

TSEPath

Used to specify where the editor searches for supplemental files (Compiled Macro files, Help files, the spelling checker word lists, the Potpourri data file, and other data files).

Values: [STRING](#) (maximum of 255 characters) **Default:** "" **Limits:** No individual path within [TSEPath](#) can be greater

than 80 characters. Notes: The format of TSEPath is similar to that of the path environment variable: - "d:\path1;d:\path2;d:\path3" The editor uses TSEPath in conjunction with the built-in command, [SearchPath\(\)](#), to find Compiled Macro files, Help files, the spelling checker word list files, the Potpourri data file, and other data files. When attempting to locate a supplemental file, the editor successively searches the following directories until the indicated file is found: - current directory - [TSEPath](#) directories - editor [load directory](#) In addition to these directories, the editor also checks several additional subdirectories for the following specific files: - "MAC\" when [searching](#) for Compiled Macro files - "SPELL\" when [searching](#) for spelling checker word list files - "HELP\" when [searching](#) for Help files For example, if the current directory is D:\FOO, and TSEPath is set to "C:\PATH1;D:\PATH2", and the editor load directory is C:\TSEPro, the editor would search the following directories for the indicated file type: Compiled Macro File SpellCheck File Help File _____ d:\foo d:\foo d:\foo d:\foo\mac d:\foo\spell d:\foo\help c:\path1 c:\path1 c:\path1 c:\path1\mac c:\path1\spell c:\path1\help d:\path2 d:\path2 d:\path2 d:\path2\mac d:\path2\spell d:\path2\help c:\TSEPro c:\TSEPro c:\TSEPro c:\TSEPro\mac c:\TSEPro\spell c:\TSEPro\help User-written Compiled [Macros](#) can also use [TSEPath](#) in conjunction with the [SearchPath\(\)](#) command to find supplemental files. [Affects:](#) [LoadMacro\(\)](#), [ExecMacro\(\)](#) See Also: Commands: [SearchPath\(\)](#), [LoadMacro\(\)](#), [ExecMacro\(\)](#)

UnBufferVideo

Re-routes all buffered video output from the editor's virtual screen to the display screen.

This command is intended for ADVANCED USAGE.

[Syntax:](#) [UnBufferVideo\(\)](#) Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the editing buffer. It is used to re-route ("unbuffer") video output from the editor's virtual screen to the visible display screen. [UnBufferVideo\(\)](#) works in conjunction with the [BufferVideo\(\)](#) command. The [BufferVideo\(\)](#) command causes video output to be buffered. When [BufferVideo\(\)](#) is executed, all subsequent video output commands operate on the editor's virtual screen (rather than the visible screen). [UnBufferVideo\(\)](#) is used to write any pending output to the screen and to restore normal display of text. To determine when video output is to be buffered (rather than displayed), an internal "buffer mode" counter is maintained. Normally, this counter is set to zero, causing video output to be visibly displayed on the screen. Each time the [BufferVideo\(\)](#) command executes, the buffer mode counter is incremented by 1. Each time the [UnBufferVideo\(\)](#) command executes, the buffer mode counter is decremented by 1. While the buffer mode counter is greater than zero, video output is buffered. To restore normal "displayed" video output, the buffer mode counter must be returned to zero. Thus, each call to [BufferVideo\(\)](#) should be followed, at the appropriate time, by a corresponding call to [UnBufferVideo\(\)](#). For more information on buffering video output, see the [BufferVideo\(\)](#) command and see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." [Examples:](#) See the [BufferVideo\(\)](#) command for an example. See Also: [BufferVideo\(\)](#)

UnDelete

Undeletes (restores) deletions made in the current file by

[DelBlock\(\)](#), [DelLine\(\)](#), [DelRightWord\(\)](#), and [DelToEol\(\)](#). [Syntax:](#) [INTEGER UnDelete\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Deletions are restored in a "last deleted, first restored" order. Unlike [GlobalUnDelete\(\)](#), this command only restores text deleted from the current file. If there are no such entries in the Deletion Buffer, this command returns [FALSE](#). See Also: [GlobalUnDelete\(\)](#), [DelBlock\(\)](#), [DelRightWord\(\)](#), [DelLine\(\)](#), [DelToEol\(\)](#) [Variables:](#) [KillMax](#)

Undo

Cancels the last change made to the text in the current file.

Syntax: [INTEGER Undo\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: When an Undo operation is successful, the information necessary to restore that change, after it has been cancelled, is stored in the Redo list. This allows you to "undo" that Undo request; that is, to redo the change. Normally, when a file is saved, previously-accumulated Undo information for that file is purged. If you want [to](#) be able [to Undo\(\)](#) changes made to a file even after that file is saved, set [KeepUndoBeyondSave ON](#). Undo/Redo information is [NOT](#) recorded for `_SYSTEM_buffers`. See Also: [Redo\(\)](#), [SetUndoOn\(\)](#), [SetUndoOff\(\)](#), [ClearUndoRedoList\(\)](#) [RedoCount\(\)](#), [UndoCount\(\)](#) **Variables:** [KeepUndoBeyondSave](#)

UndoCount

Returns the number of Undo entries available for the current file.

Syntax: [INTEGER UndoCount\(\)](#) Returns: The number of Undo entries available for the current file. Notes: The count returned by this command does [not](#) necessarily reflect the number of outstanding events that can be undone. Many of the events may be grouped together as a single undoable entry. See Also: [Undo\(\)](#), [Redo\(\)](#), [RedoCount\(\)](#), [SetUndoOn\(\)](#), [SetUndoOff\(\)](#), [ClearUndoRedoList\(\)](#), [UndoMode\(\)](#) **Variables:** [KeepUndoBeyondSave](#)

UndoMode

Determines the current buffer's undo/redo status.

Syntax: [BOOLEAN UndoMode\(\[BOOLEAN on_off\]\)](#) - on_off is optional, and can be used to set undo mode for the current buffer either On or Off. Returns: The buffer's current undo status, as of the time this command is invoked. If undo is disabled for this buffer, zero ([FALSE](#)) is returned. [Otherwise](#), non-zero is returned, signifying that undo is enabled for the current buffer. Notes: This command replaces the following commands: [SetUndoOff\(\)](#), [SetUndoOn\(\)](#) **Examples:** Example 1: [if UndoMode\(\)](#) // at this point, undo is On for the current buffer Example 2: [UndoMode \(On\)](#) // turns undo mode on for the current buffer Example 3: [UndoMode\(Off\)](#) // turns undo mode off for the current buffer Example 4: [if UndoMode\(On\)](#) // turns undo mode on for the current buffer // at this point, undo was previously On for the current buffer See Also: [Undo\(\)](#), [Redo\(\)](#), [SetUndoOn\(\)](#), [SetUndoOff\(\)](#), [ClearUndoRedoList\(\)](#), [RedoCount\(\)](#), [UndoCount\(\)](#) **Variables:** [KeepUndoBeyondSave](#)

UnHook

Removes the association of a procedure with any events.

Syntax: [INTEGER UnHook\(PROCEDURE proc_name\)](#) - proc_name is the name of the procedure to be disassociated with any events. Returns: Non-zero if successful, or zero ([FALSE](#)) if the named procedure was [not](#) associated with any events. Notes: If multiple procedures are hooked to the same event, unhooking one of them does not break the chain; the unhooked procedure is simply removed from the chain. See Also: [Hook\(\)](#), [ExecHook\(\)](#), [BreakHookChain\(\)](#),

[GetHookState\(\)](#), [SetHookState\(\)](#), [TerminateEvent\(\)](#)

UnLockCurrentFile

Unlocks the current file, if it had been previously locked.

Syntax: [INTEGER UnLockCurrentFile\(\)](#) Returns: 0 on success, **otherwise:** [_FILE_NOT_LOCKED_](#), [_CANT_UNLOCK_FILE_](#) Notes: It doesn't matter if the file was locked using [_READONLY_LOCKING_](#) or [_HANDLE_LOCKING_](#) prior to calling this command. See [Locking Files](#) for more information. See Also: [LockCurrentFile\(\)](#) **Variables:** [FileLocking](#)

UnMarkAfterPaste

Causes a "pasted" block to be immediately unmarked.

Values: BOOLEAN **Default:** [ON](#) Notes: If set OFF, the [Paste\(\)](#) command inserts the contents of the ClipBoard into the current file as a marked block. If this is set ON, the pasted block is immediately unmarked. **Affects:** [Paste\(\)](#)

UnMarkBlock

Unmarks a marked block.

Syntax: [UnMarkBlock\(\)](#) Returns: Nothing. Notes: This command always unmarks a marked block, if it exists. See Also: [MarkChar\(\)](#), [MarkStream\(\)](#), [MarkColumn\(\)](#), [MarkLine\(\)](#), [MarkWord\(\)](#), [MarkToEOL\(\)](#), [isBlockMarked\(\)](#), [isBlockInCurrFile\(\)](#), [isCursorInBlock\(\)](#) **Variables:** [Marking](#)

Up

Moves the cursor up one line in the current file without changing the cursor column.

Syntax: [INTEGER Up\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Zero is returned when the cursor is at the beginning of the file. If the cursor is on the first line of the current window, the screen will scroll. See Also: [Down\(\)](#), [RollUp\(\)](#), [ScrollUp\(\)](#)

UpdateBufferDaTmAttr

Updates the date, time, size and attribute fields for the current buffer from the associated file on disk.

Syntax: [UpdateBufferDaTmAttr\(\)](#) Returns: Nothing. Notes: For each file loaded into the editor, the editor maintains a record of the file's date, time, size, and file attributes, as of the time the file was loaded into the editor, or the last time it was saved. When a file is edited that has an associated file on disk, this command is invoked. See Also:

[ClearBufferDaTmAttr\(\)](#), [GetBufferDaTmAttr\(\)](#)

UpdateDisplay

Causes the editor to update portions of the current display.

Syntax: [UpdateDisplay](#)([INTEGER](#) refresh_flag) - refresh_flag is an optional integer. refresh_flag values can be OR'ed in order to achieve the desired result. If not specified, `_DEFAULT_` is used. Supported [values](#) for refresh_flag are: - `_DEFAULT_` redraws any areas that are internally marked as requiring redraw. - `_STATUSLINE_REFRESH_` forces a StatusLine redraw. - `_CLINE_REFRESH_` forces the current line to be redrawn. - `_WINDOW_REFRESH_` forces the entire current window less the StatusLine to be redrawn. - `_HELPLINE_REFRESH_` forces [the HelpLine](#) to be redrawn. - `_ALL_WINDOWS_REFRESH_` forces all [windows](#), their title lines, and [the StatusLine](#) to be redrawn. - `_REFRESH_THIS_ONLY_` forces only the specified areas to be redrawn. Any internally set redraw flags are cleared. Returns: Nothing. Notes: UpdateDisplay should ONLY be called when the focus is an editing window. Results are unpredictable if [UpdateDisplay\(\)](#) is called when a menu is active, a [PopWinOpen\(\)](#) has been called, or in any other situation when a text editing window is not the main focus. Multiple refresh_flag values can be combined with the bitwise logical OR (|) operator. For instance, to cause [the StatusLine](#) and the current window to be re-displayed, use: [UpdateDisplay](#)(`_STATUSLINE_REFRESH_ | _WINDOW_REFRESH_`) In addition to forcing a redraw to the specified screen element, all refresh_flag values except `_REFRESH_THIS_ONLY_` cause a redraw of all screen elements flagged internally as needing update. Using the `_REFRESH_THIS_ONLY_` value overrides the [default](#) update. This value can be combined with one or more of the other refresh_flag values to reduce side effects. For example, to update only [the StatusLine](#), regardless of internal flags indicating that other areas need to be redrawn, use: [UpdateDisplay](#)(`_REFRESH_THIS_ONLY_ | _STATUSLINE_REFRESH_`) See Also: [UpdateDisplayFlags\(\)](#)

UpdateDisplayFlags

Sets or queries the

[UpdateDisplay\(\)](#) flags currently in effect. **Syntax:** [UpdateDisplayFlags](#)([INTEGER](#) refresh_flag) - refresh_flag is an optional integer. If not specified, `_DEFAULT_` is used. Supported [values](#) for refresh_flag are: - `_DEFAULT_` redraws any areas that are internally marked as requiring redraw. - `_STATUSLINE_REFRESH_` forces [the StatusLine](#) to be redrawn. - `_CLINE_REFRESH_` forces the current line to be redrawn. The current line is the line that is current at the time the window is actually redrawn (that is, when [UpdateDisplay\(\)](#) is called). - `_THIS_CLINE_REFRESH_` forces the current line to be redrawn. The current line is the line that is current at the time this command is invoked. - `_WINDOW_REFRESH_` forces the entire current window, except for [the StatusLine](#), to be redrawn. - `_HELPLINE_REFRESH_` forces [the HelpLine](#) to be redrawn. - `_ALL_WINDOWS_REFRESH_` forces all [windows](#), their title lines, and [the StatusLine](#) to be redrawn. - `_REFRESH_THIS_ONLY_` forces only the specified areas to be redrawn. Any internally set redraw flags are cleared. Returns: The [UpdateDisplay\(\)](#) flags in effect when this command is invoked. Notes: This command does not cause any immediate changes to the editor's display, but rather, sets flags that will be used in conjunction with internally generated flags, the next time the editor updates the display. Multiple refresh_flag values can be combined with the bitwise logical OR (|) operator. For example, to force [the StatusLine](#) and the current window to be re-displayed, use: [UpdateDisplayFlags](#)(`_STATUSLINE_REFRESH_ | _WINDOW_REFRESH_`) See Also: [UpdateDisplay\(\)](#)

Upper

Changes alphabetic characters to upper

[case](#). [Syntax](#): [STRING Upper](#)([STRING](#) s) - s is an optional string to convert. or: [Upper](#)() Returns: If the first Syntax format is used, the resulting upper-cased [string](#) is returned; [otherwise](#), nothing is returned. Notes: [Upper](#)() "upper-cases" the target characters - all lower-case alphabetic characters [a..z] are converted to upper [case](#) [A..Z]. Other characters are [not](#) changed. If the first Syntax format is used, s is upper-cased. If the second Syntax format is used: if the cursor is in a marked block, the characters in the block are upper-cased; otherwise, the character at the cursor position is upper-cased. [Examples](#): Given: [string](#) s[3], = "aBc", s2[3] then: s2 = [Upper](#)(s) // s2 is "ABC" s2 = [Upper](#)("Def") // s2 is "DEF" s2 = [Upper](#)(s2) // s2 is "DEF" // If the cursor is on a word, upper case it [proc](#) UpperWord() [PushBlock](#)() // save block state if [MarkWord](#)() // if on a word, mark it [Upper](#)() // convert to upper [case](#) [endif](#) [PopBlock](#)() // restore block state [end](#) See Also: [Lower](#)(), [Flip](#)()

UseCommonDialogs

Determines if the editor uses Windows dialogs for certain file related prompts.

[Values](#): BOOLEAN [Default](#): [OFF](#) Notes: By default, the editor uses its own style of prompts to request filenames. If this variable is set ON, the familiar Windows common file dialogs are used instead. [Affects](#): [EditFile](#)(), [EditThisFile](#)(), [ReplaceFile](#)(), [ChangeCurrFilename](#)(), [SaveAs](#)(), [SaveBlock](#)()

UseCurrLineIfNoBlock

Causes the

[Cut](#)() and [Copy](#)() commands to use the current cursor line if there is not a marked block in the current file. [Values](#): BOOLEAN [Default](#): [OFF](#) Notes: Normally [Cut](#)() and [Copy](#)() operate on marked blocks in the current file. If [UseCurrLineIfNoBlock](#) is set [ON](#), and no block is marked in the current file, then the current line will be used for the [Cut](#)() or [Copy](#)() command. [Affects](#): [Cut](#)(), [Copy](#)() See Also: Commands: [isBlockInCurrFile](#)(), [isBlockMarked](#)()

Using Bookmarks

To assist you in moving between particular locations within the files you are editing, the Bookmark feature is available. Bookmarks allow you to temporarily tag specific locations within the files you have loaded, and then

[return](#) to those locations. You can even move to other files and then [return](#) to a location tagged by a Bookmark in a previous file. A Bookmark remains in place until you specifically remove it, or until you close the file or Exit from the editor. (When you close a file, ALL Bookmarks in the file are automatically discarded. If you Exit from the editor, all Bookmarks are discarded, unless the SaveState option is in effect. For more information on the SaveState variable, see "[Saving/Restoring the Editing Environment](#)" in the chapter on "[Loading, Processing, and Saving Files](#).") The editor allows you to place up to 26 Bookmarks within an editing session. Each Bookmark is uniquely identified by one of the alphabetic characters, A through Z. (The case of the letter does not matter.) Though Bookmarks are not visible on the screen, you can return to a Bookmark location by selecting its unique identifier. If you use the same letter to place a Bookmark in two different locations, the second Bookmark location replaces the first. Thus, you lose the first location tagged by that Bookmark. Because [Bookmarks](#) are accessible across files, this is true regardless of whether the locations are in the same or different files. The following two sections describe two different methods for placing and returning to [Bookmarks](#).

Using Command Lines in Macros

Information can optionally be passed from one macro to another, as

[macros](#) are executed, via the MacroCmdLine editor variable. To pass a command-line string to a macro, specify the desired string when you execute the macro using [the ExecMacro command](#), as follows: [ExecMacro](#)([[STRING](#) macro_str]) This command executes (or loads and executes, as necessary) the specified macro (or macro file), and passes it a command-line string, if one is specified. macro_str consists of macro_name and an optional cmd_line_str, in the format "macro_name cmd_line_str", where: - macro_name is the name of a macro (or macro file). - cmd_line_str is an optional command-line string to pass to macro_name. cmd_line_str must be separated from macro_name by one or more spaces. If cmd_line_str is specified when ExecMacro is executed, the MacroCmdLine variable is set to the indicated string; otherwise, MacroCmdLine is set to an empty [string](#). The macro that is executed can retrieve the value of MacroCmdLine using the [Query\(\)](#) command. If macro_name is a macro file containing multiple macros, each of the macros that are executed in the macro file have access to the [MacroCmdLine](#) variable. The current macro can also change the value of MacroCmdLine, using the [Set\(\)](#) command. This can be useful for returning information to a calling macro, once the current macro terminates.

Using Macro Language Features

This chapter describes additional macro features available in SAL for further customization of the editor. These features include the ability to create help information; build menus and menubars; embed data within a macro; and assign commands,

[macros](#), [menus](#), and help to keys. This chapter also provides usage information for other SAL features presented in previous chapters. The following topics are covered: - [Breaking out of run-away macros](#) - [Assigning macros and commands to keys](#) - [Keydef](#) - [Command prompts and keydefs](#) - [Process\(\)](#) and keydefs - [HelpLines](#) - [Include files](#) - [Using command lines in macros](#) - [Session-global variables](#) - [Buffers](#) - [Searching](#) - [Saving and restoring state information](#) - [Getting user input](#) - [Menus](#) - [Datadef](#) - [Helpdef](#) - [Video output commands](#) - [Event macros](#) (For more information on the commands and variables discussed in this chapter, see the chapters on "[Editor Commands](#)" and "[Editor Variables](#)," respectively. For more information on the basics of [customizing the editor](#), see the chapter on "[Fully Customizing the Editor](#)" in [the User's Guide](#).)

Using Macros

This chapter provides information for effectively developing and using external and user-interface Compiled

[Macros](#). The following topics are presented: - [External macros vs. the user-interface macro](#) - [Creating macros](#) - [Compiling macros](#) - [Macro file search order and the TSEPath variable](#) - [Loading/executing macros](#) - [Purging macros](#) - [Debugging macros](#) - [Startup processing](#) - [The TSEStart macro](#) [The TSELOAD.DAT file](#) - [Terminating the editor](#) - [Miscellaneous questions](#)

Using Physical Tabs: Hard

There is a single type of physical tab, called Hard Tabs (often referred to as "real" tabs). Hard Tabs conserve file space by using a single physical tab

character (ASCII 9) to represent a variable number of spaces. The number of spaces that the tab character represents depends on the location of the tab character relative to the next tab stop. For display purposes, the text following a tab character is positioned beginning at the next tab stop on the line.

With Hard Tabs, the setting of the ExpandTabs option affects the treatment of tab characters when text is displayed on the screen. If ExpandTabs is OFF, the tab characters in a file are displayed on the screen as the physical tab character itself. If ExpandTabs is ON, the tab characters are interpreted so that the text is positioned on the screen according to the current TabWidth. (For more information on the effects of the ExpandTabs option, see "

[Effects of ExpandTabs](#)" later in this chapter. For more information on how to set the ExpandTabs option, see "Expand Tabs (Display Only)" under "[Tab/Margin Options](#)" in the chapter on "[Configuring the Editor Interactively](#)." The following describes the behavior of TabRight and TabLeft when used with Hard Tabs. As with positional tabs, the behavior of the tab commands is also affected by the setting of Insert mode. * TabRight (<Tab>) When used with Hard Tabs, this command places the tab character in the text, and/or moves the cursor to the next tab stop on the line, depending on the settings of [Insert mode](#) and [ExpandTabs](#). If Insert is ON, the tab character is inserted. The movement of the cursor and the text is determined by the setting of ExpandTabs, as follows: - If ExpandTabs is ON, then the cursor visibly moves to the next tab stop, along with it any text from and to the right of the cursor position. The tab character itself is not displayed on the screen. (Physical space characters are NOT inserted on the line to the next tab stop. The text only appears to be positioned at the next tab stop.) - If ExpandTabs is OFF, the cursor does not move to the next tab stop. Any text from and to the right of the cursor is shifted right by one character, and the tab character itself is displayed on the screen. The cursor remains on the same character, moving to the right along with the text. If Insert is OFF, the tab character is NOT inserted, and the setting of ExpandTabs has no effect. The cursor moves to the next tab stop on the line, without shifting existing text. * TabLeft (<Shift Tab>) When used with Hard Tabs, this command moves the cursor to the previous tab stop on the line. The tab character is NOT inserted. If Insert is ON, the following behavior occurs: - If there are ONLY white (tab or space) characters on the line from (and including) the previous tab stop and up to (but not including) the cursor position, then when the cursor moves left, any text from and to the right of the cursor position is shifted left to that previous tab stop. The [white characters](#) in between are deleted. - If there are ANY non-white characters on the line from (and including) the previous tab stop and up to (but not including) the cursor position, then the cursor simply moves left to the previous tab stop without shifting existing text. If Insert is OFF, the cursor merely moves left to the previous tab stop without shifting existing text.

+----- Tip -----+ | Many operating system utilities and other programs often assume a | | specific tab width value, and have no knowledge of the value you | | may have used when creating a file with the editor containing tab | | characters. Because of this, it is advisable to use tab characters | | with a degree of caution if the file being edited will later be | | processed by another program or utility. |
+-----+

Using Picklists

The editor offers a special "PickList" feature for selecting an entry from a list of items displayed by a command. You can display a PickList of files from which to choose when executing a file-related command (as described earlier under "

[Obtaining a File PickList within a Prompt Box](#)"). You can obtain a PickList of History entries for previous responses to a command you are executing (as described earlier under "[History Buffer for Prompt Boxes](#)"). Commands like CompressView and AsciiChart display a PickList of entries from which to choose a selection that you want the

command to process. In the pull-down Menu system, the notation "..." follows command entries such as "Compressed View" and "ASCII Chart", which display a PickList. Within a PickList, the keys <CursorUp>, <CursorDown>, <PgUp>, <Ctrl PgUp>, <PgDn>, <Ctrl PgDn>, <Home>, <Ctrl Home>, <End>, <Ctrl End>, and <Spacebar> (which operates the same as <CursorDown>), scroll the cursor bar forward or backward within the list of entries. The editor's SpeedSearch feature, described in the following section, can also be used to move the cursor bar. To select an entry from a PickList, move the cursor bar over the desired entry and press <Enter>. The command you are executing (which issued the PickList) will resume, using the PickList selection you made. (In a file PickList displayed by the EditFile command, you can "tag" multiple entries for processing. For more information, see "[Obtaining a File PickList within a Prompt Box](#)" earlier in this chapter.) To cancel the PickList without making a selection, press <Escape>. If the PickList was displayed from a prompt box, you are returned to the prompt box; otherwise, the command terminates.

Using Positional Tabs: Soft, Variable, and Smart

Positional tabs affect the positioning of the cursor when a tab command is executed. However, these tabs do not cause a physical tab character to be placed into the text.

There are three different types of positional tabs: Soft, Variable, and Smart. These differ from each other only by the manner in which tab stops are identified (as described in the previous section). Each of these tab types causes the cursor to move to the next or previous tab stop each time a tab command is executed. The following describes the behavior of TabRight and TabLeft when used with the positional tabs. Note also the effects of Insert mode on these commands. (For more information on Insert mode, see the chapter on "

[Editing Modes](#).) * TabRight (<Tab>) When used with positional tabs, this command moves the cursor to the next tab stop on the line without placing any tab characters in the text. If Insert is ON, when the cursor moves, any text from and to the right of the cursor is also shifted right to the next tab stop. Spaces are inserted from the cursor position to that next tab stop. If Insert is OFF, the cursor simply moves to the next tab stop without shifting existing text. * TabLeft (<Shift Tab>) When used with positional tabs, this command moves the cursor to the previous tab stop on the line without placing any tab characters in the text. If Insert is ON, the following behavior occurs: - If there are ONLY white (tab or space) characters on the line from (and including) the previous tab stop and up to (but not including) the cursor position, then when the cursor moves left, any text from and to the right of the cursor is also shifted left to that previous tab stop. The [white characters](#) in between are deleted. - If there are ANY non-white characters on the line from (and including) the previous tab stop and up to (but not including) the cursor position, then the cursor simply moves left to the previous tab stop without shifting existing text. If Insert is OFF, the cursor simply moves left to the previous tab stop without shifting existing text.

Using Tabs

To enter text or data at specified columns in a file, you can use the tab commands, TabRight and TabLeft. With these commands, text is automatically positioned in selected columns, based on tab stop settings.

Two tab-handling conventions are available: positional tabs and physical tabs. With positional tabs, the cursor is simply placed at the actual column indicated by the tab stops. With physical tabs, a tab character is actually placed in the text, and the cursor optionally moves to the column position

indicated by the tab stops.

This chapter discusses:

-

[Selecting the tab type \(TabType\)](#) - [Setting tab stops \(TabWidth, ZeroBasedTab, Variable Tab stops\)](#) - [Using positional tabs: Soft, Variable, and Smart \(TabRight, TabLeft\)](#) - [Using physical tabs: Hard \(TabRight, TabLeft\)](#) - [Effects of ExpandTabs](#)

Using the Pop-Up Mouse Menu

A special Mouse menu allows you to use the mouse to display a menu of Block-related and ClipBoard-related commands. To pop-up the Mouse menu with the mouse while in editing mode, click the <RightBtn> (with no menus, prompts, or PickLists active). You can then select one of the displayed commands by placing the mouse pointer on the desired item and clicking the <LeftBtn>.

Alternatively, you can use the dragging feature of the mouse to more easily execute a command from the Mouse menu. Simply press and hold the <RightBtn>, and drag the mouse pointer to the desired command or option, and then release the <RightBtn>. The indicated command or option is invoked.

To remove the Mouse menu, click the <RightBtn> until the Mouse menu is removed from the screen, or move the mouse pointer off the Mouse menu and click the <LeftBtn>.

(For more information on

[marking](#) and manipulating blocks, and using ClipBoards, see the chapter on "[Block Operations](#).")

Using the Pull-Down Menu System

To execute a command or set an option from the pull-down Menu using the mouse, you can follow these steps:

- If the main Menu is not currently displayed on the screen, first display it by placing the mouse pointer on

[the StatusLine](#) or HelpLine and clicking the <LeftBtn>. - Select a sub-menu category from the Menu by placing the mouse pointer on the category and clicking the <LeftBtn>. - Select a command or option from the sub-menu by placing the mouse pointer on the desired item and clicking the <LeftBtn>. Alternatively, you can use the dragging feature of the mouse to more easily access commands or options on the Menu, as follows: - If the main Menu is not currently displayed, place the mouse pointer on [the StatusLine](#) and press and hold the <LeftBtn> to display the main Menu on the screen. Continue holding the <LeftBtn>. If the main Menu is already displayed, place the mouse pointer on the Menubar and press and hold the <LeftBtn>. - Drag the mouse pointer across the Menubar to the desired category. The sub-menu categories are displayed as the pointer passes through each [main Menu](#) category. - Drag the mouse pointer down to the desired command or option, and finally release the <LeftBtn>. The indicated command or option is invoked. Note that if the <LeftBtn> is released with the mouse pointer positioned outside the Menu area, you

immediately terminate the Menu system. To deactivate [the pull-down Menu system](#), either click the <RightBtn> until the main Menu is no longer active on the screen, or move the mouse pointer outside the Menu area and click the <LeftBtn>. (For more information on using the pull-down Menu, see the chapter on "[The Pull-Down Menu System](#).")

Val

Converts a string expression to an integer.

Syntax: [INTEGER](#) [Val](#)([STRING](#) s [, [INTEGER](#) base]) - s is the string expression to be converted to integer. - base is an optional parameter that specifies the number base to use, which defaults to 10 if [not](#) specified. Returns: The [integer](#) value of s. Notes: Leading spaces are ignored, so that [Val](#)("55") and [Val](#)(" 55") both [return](#) 55. If a leading minus or plus sign is present, it is taken into account. If s contains any non-numeric characters, only that portion of s preceding those characters is converted. If a blank, empty, or non-numeric string is encountered, [Val](#)() will [return](#) 0. **Examples:** To convert binary "1000b" to an integer: a = [Val](#)("1000", 2) To convert hex "0x7fff" to an integer: a = [Val](#)("7fff", 16) To convert decimal "1024" to an integer: a = [Val](#)("1024") [Val](#)() stops evaluating at a non-numeric character: a = [Val](#)("12345,6789") // a is 12345 See Also: [Str](#)()

Variables

Unlike [constants](#), [variables](#) are items whose value may be changed. [Variables](#) are either [string](#) or [integer](#) types. All [variables](#) must be declared before use, and may be either global or local in scope. Both types of variables are accessible only within the macro file in which they are declared. Global variables are accessible by all routines following the variable declaration. The values of global variables are retained as long as the macro file in which they are declared remains loaded. Global [variables](#) are declared within the macro file in which they are used, but are declared outside macro procedures in the macro file. If pre-initialized, they must be initialized to a constant expression. Local variables are accessible only within the procedure in which they are declared. Local [variables](#) are temporary and their [values](#) are only valid while the procedure in which they are declared is executing. Once the procedure terminates, the local variables no longer exist and their values are undefined. All local [variables](#) should be initialized within their procedure before use; but unlike global [variables](#), local [variables](#) do [not](#) have to be initialized to a constant expression. Global [variables](#) are declared outside of procedures, while local [variables](#) are declared within procedures. The syntax for declaring a variable is: variable_type identifier [= expression], identifier [= expression], ... where: - variable_type is either [integer](#) or [string](#). - identifier is any valid user-definable identifier. (For more information, see "[Identifiers](#)" earlier in this chapter.) - expression is a combination of one or more constants or identifiers. To declare multiple variables of the same type, separate the identifiers with commas as shown above. To reference a global variable, simply reference the variable name within an expression of the same type as the variable. Local variables may only be referenced within the procedure in which they are declared. Global [variables](#) may be referenced from any procedures that exist after the declaration of the global variable within the same macro file in which the global variable is declared. NOTE: If a local variable is declared with the same name as a global variable, all references to that variable name within the procedure will reference the local variable, NOT the global variable. All references to that variable name outside the procedure will reference the global variable. [Integer Variables](#) ----- Integer variables are whole numbers (... -2, -1, 0, 1, 2, 3, ...) in the range of [MININT](#) (-2,147,483,648) to [MAXINT](#) (+2,147,483,647). (For more information, see "[Numbers](#)" earlier in this chapter.) Integers can be declared globally or locally. [Declaring Global Integers](#) To declare global integers (outside procedures): [integer](#) identifier [= constant_expression] To optionally initialize a global integer, a constant expression must be specified. If a global integer is not specifically initialized, it is automatically initialized to 0. **Examples:** integer a = 10, // set to 10 boolean = [ON](#), // set to 1 ([ON](#)) large = 1000 * 32 + 768 // set to 32768 [integer](#) zero // automatically initialized to 0 [Referencing Global Integers](#) To use a global integer, simply identify the variable name within a numeric expression. A global [integer](#) can be referenced by

any procedure that follows the variable declaration within the same macro source file. To convert a string into an integer use the [Val\(\)](#) command. [Declaring Local Integers](#) To declare local integers (inside procedures): [integer](#) identifier [= numeric_expression] Local integers should always be initialized so their [values](#) will be known when the procedure is executed. Uninitialized local integers are NOT automatically initialized and use of these [variables](#) before initialization will result in undefined [values](#). Unlike global integers, local integers do [not](#) have to be initialized to a constant expression; they may be initialized to any numeric expression. [Examples](#): [integer](#) first_line = 1, last_line = [Query](#)(NumLines()), middle_line = (last_line - first_line) / 2 [integer](#) dangerous // uninitialized local variable has // undefined value [Referencing Local Integers](#) To use a local integer, simply identify the variable name within a numeric expression. A local integer can only be referenced within the procedure in which it is declared. To convert a string into an integer use the [Val\(\)](#) command. [String Variables](#) ----- String variables consist of a group of up to 255 characters. The length allocated to strings is specified in the string's declaration. This length must be a constant. [Strings](#) can be declared globally or locally. [Declaring Global Strings](#) To declare global [strings](#) (outside procedures): [string](#) identifier[[max_length]] [= constant_string_expression] When declaring a global string, the bracket characters, [], following identifier are required. To declare a global string, the maximum length of the string, max_length, must be specified as a constant numeric expression. If, however, a global string is immediately initialized to a constant string expression, the maximum length is automatically computed and set to the length of the initialization string. In this case, the length is not required (though the bracket characters are still required). If a global string is not initialized, it is automatically initialized to an empty string of the specified max_length. (The maximum length of the string is retained. Only the current length is set to zero.) [Examples](#): [string](#) Astring[10], // reserve 10 characters for // Astring Hello[] = "Hello, world", // creates 12-character [string](#) Combo[] = "Hello, " + "world" // creates 12-character [string](#) If a string is initialized with more characters than are allocated by the max_length value, the string will be truncated to the maximum length. [string](#) Truncated[29] = "This [string](#) will be truncated to 29 characters" will be stored as "This [string](#) will be truncated". [Referencing Global Strings](#) To use a global string, simply identify the variable name within a string expression. A global [string](#) can be referenced by any procedure that follows the variable declaration within the same macro source file. NOTE: If a procedure attempts to reference a global string within a procedure which has a local string of the same name, the local string will be referenced instead of the global string. [Declaring Local Strings](#) To declare local [strings](#) (inside procedures): [string](#) identifier[maximum_length] [= string_expression] Like global strings, the bracket characters, [], following identifier are required when declaring a local string. Local string declarations differ from global string declarations in the following respects: maximum_length is REQUIRED for local [strings](#); and string_expression does not have to be a constant string. Example: [string](#) first_name[10] = "Captain", last_name[10] = "Kirk", full_name[21] = first_name + " " + last_name Local [strings](#) should always be initialized so their [values](#) will be known when the procedure is executed. Uninitialized local strings are NOT automatically initialized. Local [strings](#) can be initialized to any [string](#) expression; they are [not](#) limited to [constant strings](#). [Referencing Local Strings](#) To use a local string, simply identify the string name within an expression. A local string can only be referenced within the procedure in which it is declared. To convert an integer or constant into a string, use the [Str\(\)](#) or [Format\(\)](#) command.

VarTabs

Specifies the positions of Variable "Tab" locations.

[Values](#): [STRING](#) 32-character bit-mapped string allowing for the specification of 255 tabstops. [Default](#): [TabSet](#)("4 8 16 24 40") [Limits](#): The maximum settable tab stop is 255. Notes: [VarTabs](#) allows you to specify at which positions to place tab stops when _VARIABLE_ tabs are enabled. Since the string is a bit-encoded string, you should use the [TabSet\(\)](#) command to convert a string of numbers into the bit-encoded VarTabs string. To set variable tab stops at columns 4, 8, 12, 40, 50, set [TabType](#) to _VARIABLE_ and [VarTabs](#) = [TabSet](#)("4 8 12 40 50") A maximum of 255 tab stops are stored in a bitmapped format within a 32-byte string. For purposes of saving/restoring tabs with the macro language, a 32-character string will suffice. If a macro needs to change tab stops, [TabSet\(\)](#) should be used to bit encode

the tab stops. [Affects](#): [TabRight\(\)](#), [TabLeft\(\)](#), [DistanceToTab\(\)](#) See Also: [TabType](#), [TabWidth](#) Commands: [TabSet\(\)](#)

Version

Returns the internal editor version.

Syntax: [INTEGER Version\(\)](#) Returns: The internal editor version number. **Examples:** To create a variable called "version_number" and assign the version number to it: [integer](#) version_number version_number = [Version\(\)](#) See Also: [IVersion\(\)](#)

VersionStr

Returns the displayed version of the editor as a string.

Syntax: [STRING VersionStr\(\)](#) Returns: The editor's version number. Notes: This command is for use in macros. Contrast with the [Version\(\)](#) command, which returns the editor's internal version number. **Examples:** [string](#) s[10] s = [VersionStr\(\)](#) // s is something like "4.20", depending on the // actual version of the editor. See Also: [Version\(\)](#), [IVersion\(\)](#)

VGotoXY

Sets the video output position to the coordinates specified, relative to the current video output window (as defined by

[Window\(\)](#); [PopWinOpen\(\)](#); or the editor's default [FullWindow\(\)](#), which is the entire screen). The visible [cursor](#) position is [not](#) changed. **Syntax:** [VGotoXY\(INTEGER x, INTEGER y\)](#) - x is the column coordinate. - y is the row coordinate. Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. The position specified determines where writing performed by the next video output command will begin. The hardware (visible) [cursor](#) is [NOT](#) set to this position. This command is supplied as an adjunct to the [GotoXY\(\)](#) command, whose sole difference is that it DOES position the hardware (visible) [cursor](#). The [VGotoXY\(\)](#) command can be used in situations where several video output commands are being used, to prevent the visible cursor from flashing on the screen. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [GotoXY\(\)](#), [VGotoXYAbs\(\)](#), [VHomeCursor\(\)](#), [WhereX\(\)](#), [WhereXAbs\(\)](#), [WhereY\(\)](#), [WhereYAbs\(\)](#), [VWhereX\(\)](#), [VWhereY\(\)](#)

VGotoXYAbs

Sets the video output position to the coordinates specified, not relative to any window, but relative to the physical screen. The visible cursor position is

[not](#) changed. **Syntax:** [VGotoXYAbs\(INTEGER x, INTEGER y\)](#) - x is the column coordinate, in the range 1 to the number of columns currently displayed. - y is the row coordinate, in the range 1 to the number of rows currently displayed. Returns: Nothing. Notes: Similar to [VGotoXY\(\)](#), except that this command ignores any video output system [windows](#), if they exist. For more information on [video output commands](#), see "[Video Output Commands](#)" in the

chapter on "[Using Macro Language Features](#)." See Also: [VGotoXY\(\)](#), [GotoXY\(\)](#), [VHomeCursor\(\)](#), [WhereX\(\)](#), [WhereXAbs\(\)](#), [WhereY\(\)](#), [WhereYAbs\(\)](#), [VWhereX\(\)](#), [VWhereY\(\)](#)

VHomeCursor

Sets the video output position to the upper left corner of the current video output window (as defined by

[Window\(\)](#); [PopWinOpen\(\)](#); or the editor's default [FullWindow\(\)](#), which is the entire screen). The visible cursor position is not changed. **Syntax:** [VHomeCursor\(\)](#) Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [VGotoXY\(\)](#), [GotoXY\(\)](#), [VGotoXYAbs\(\)](#), [WhereX\(\)](#), [WhereXAbs\(\)](#), [WhereY\(\)](#), [WhereYAbs\(\)](#), [VWhereX\(\)](#), [VWhereY\(\)](#)

Video Output Commands

The editor makes its own low-level video output functions (commands) available to you within [macros](#); however, they are intended for use [by](#) more advanced macro programmers. These commands can be used to open pop-up windows, write to any area of the display screen, read informational text from the screen, clear portions of the screen, and change the colors used for [writing to the screen](#). It is important to note that these commands DO NOT in any way affect the actual text in the buffer (or file); they are used for display purposes only and [not](#) for editing of text. Their purpose is [to](#) allow you [to](#) display information on the screen for the user, in a format that you design, rather than forcing you to use the pre-defined formats of the editor's native commands. (For an example of a routine in the editor that takes advantage of these video output commands, see the "[Set Colors](#)" option of the "[Display/Color Options](#)" in IConfig. IConfig can be accessed via the "[Full Configuration](#)" option under the Options Menu.) Because these commands are low-level functions within the editor, they operate outside the control of the editor's higher-level screen-refreshing logic. Thus, these commands should be used with careful attention to the effects of the commands. If you use these commands within a macro, it is your responsibility to restore the screen and editing environment once your own macro is complete and before control is returned to the user. [Creating View Port Windows](#) ----- The first group of video output commands allow you to limit the video output window to defined portions of the screen; thus, you can create your own view port [windows](#). These commands are: [Window\(\)](#), [FullWindow\(\)](#), [PopWinOpen\(\)](#), and [PopWinClose\(\)](#). They are used to create [windows](#) that other video output commands operate in. You can use these commands to display special video effects and to build fancy dialog boxes. [Window\(\)](#) and [FullWindow\(\)](#) should be used together to properly maintain the editing screen. If you use the [Window\(\)](#) command, be sure to call [FullWindow\(\)](#) before the macro terminates, or before the editor's screen update routine is called, either directly (via [UpdateDisplay\(\)](#)) [or](#) indirectly. Likewise, [PopWinOpen\(\)](#) and [PopWinClose\(\)](#) should be used together. If a macro calls [PopWinOpen\(\)](#), but does not call [PopWinClose\(\)](#) before the macro terminates, then the editing screen will not be returned to the normal editing state, and the editor will not be able to update the screen properly. Finally, the editor variables, PopWinX1, PopWinY1, PopWinCols, and PopWinRows, may be used to retrieve the coordinates and size of the current video output window. [Manipulating the Video Output Position and the Visible Cursor](#)

----- This second group of video output commands allows you to manipulate the video output position and the visible cursor within the current video output window. These commands are: [GotoXY\(\)](#), [WhereX\(\)](#), [WhereY\(\)](#), [VWhereX\(\)](#), [VWhereY\(\)](#), [VGotoXY\(\)](#), [VGotoXYAbs\(\)](#), [WhereXAbs\(\)](#), [WhereYAbs\(\)](#) and [VHomeCursor\(\)](#). The position where the next video output screen-writing command will display its informational text, is called the "video output position." The commands [GotoXY\(\)](#), [VGotoXY\(\)](#), and [VHomeCursor\(\)](#) are used to set this position. In addition to setting the video output position, [GotoXY\(\)](#) moves the visible cursor, at the same time, to the specified video output position. (This is in contrast to [VGotoXY\(\)](#) and [VHomeCursor\(\)](#), which do not

affect the position of the visible cursor.) To query the position of the visible cursor, use the commands [WhereX\(\)](#) and [WhereY\(\)](#). To query the current video output position, use the commands [VWhereX\(\)](#) and [VWhereY\(\)](#). (Note that these commands do not have any effect on the editing cursor position; that is, the position in the text where actual editing takes place. When the macro terminates, the editor restores the editing cursor position, as appropriate.) For certain video output operations, it is preferable to use [VGotoXY\(\)](#) or [VHomeCursor\(\)](#), instead of [GotoXY\(\)](#), in order to change the video output position without changing the position of the visible cursor. For example, if you are writing text to several different locations on the screen, you would not want to see the cursor moving and flashing about over the screen as you write. [Writing to the Screen](#) ----- The following group of [video output commands](#) are used to display informational text on the screen: [DrawBox\(\)](#), [PutAttr\(\)](#), [PutAttrXY\(\)](#), [PutChar\(\)](#), [PutCharH\(\)](#), [PutCharHXY\(\)](#), [PutCharV\(\)](#), [PutCharXY\(\)](#), [PutCtrStr\(\)](#), [PutLine\(\)](#), [PutLineXY\(\)](#), [PutNCharAttr\(\)](#), [PutStr\(\)](#), [PutStrAttr\(\)](#), [PutStrAttrXY\(\)](#), [PutStrEOL\(\)](#), [PutStrEOLXY\(\)](#), [PutStrXY\(\)](#), [Write\(\)](#), and [WriteLine\(\)](#). These commands display text in the current video window (and have no effect on the text in the current buffer). If no video output window is defined, the full screen is used. All of these commands begin displaying text at the video output position. Only [Write\(\)](#) and [WriteLine\(\)](#) change the position of the visible cursor; all other commands leave the visible cursor position unchanged. If speed is important to you, note that the "Put"() commands are slightly faster than [Write\(\)](#) and [WriteLine\(\)](#). [WriteLine\(\)](#) scrolls text in a video window, if issued from the last line of the window. [Reading from the Screen](#) ----- The following group of [video output commands](#) can be used to read text from the screen into a macro string: [GetStr\(\)](#), [GetStrXY\(\)](#), [GetStrAttr\(\)](#), and [GetStrAttrXY\(\)](#). These commands read text in the current video window (and do not read text in the current buffer). If no video output window is defined, the full screen is used. All of these commands begin reading text (either a string of characters, or a string of character attribute pairs) at the video output position. [Clearing Portions of the Screen](#) ----- This last group of commands is used to clear portions of the screen, and consists of: [ClrEol\(\)](#) and [ClrScr\(\)](#). These commands operate in the current video window. [ClrEol\(\)](#) clears the video window, from the logical cursor position to the end of the line. [ClrScr\(\)](#) clears the entire video window. [Buffering Screen Output](#) ----- Commands that write output to the screen involve significant system overhead. This often results in noticeable slowness and flicker as text is being displayed on the screen. To improve screen update performance, the editor allows you to buffer video output. By buffering the screen output, a series of video output commands can be batched, and the results then displayed on the screen when the informational text is complete. Video output is buffered by temporarily routing the results of video output commands to the editor's non-displayed "virtual" screen, using the [BufferVideo\(\)](#) command. While output is being buffered, any executed video output commands affect only the editor's virtual screen, and are not visible on the display screen. Also, output commands that read from the screen, such as [GetStr\(\)](#), will read from the virtual screen. When the output text is complete, it is "unbuffered" using the [UnBufferVideo\(\)](#) command. The resulting output text is then written to the screen, and is visible to the user. To buffer video output before updating the visible screen, use the following format: [BufferVideo\(\)](#) ... (a series of commands that write to or read from the screen) [UnBufferVideo\(\)](#) Once [UnBufferVideo\(\)](#) is executed, the changes will be reflected on the visible screen.

ViewFinds

Re-displays the list of matching lines found by the last search that used the "v" (view) option.

Syntax: [INTEGER ViewFinds\(\)](#) Returns: Non-zero if terminated by pressing <Enter>. Zero ([FALSE](#)) if terminated by pressing <Escape>; or if a [Find\(\)](#) or [lFind\(\)](#) with the view option, or a CompressView command, had not been previously executed. Notes: Selecting a line in the "view" PickList will take you to that line in the source file. See Also: [Find\(\)](#), [lFind\(\)](#) **Variables:** [ViewFindsId](#)

ViewFindsId

The id of the buffer that contains the matching lines from the last search performed using the "v" (view) option.

Values: [INTEGER](#) **Default:** Undefined **Limits:** Can only be queried. Cannot be used within a config/endconfig construct. **Notes:** ViewFindsId is populated when the [Find\(\)](#) or [lFind\(\)](#) commands are executed using the "v" (view) option, or when CompressView() is executed. ViewFindsId can be used with the [ViewFinds\(\)](#) command to re-display the list of matching lines found by the last view-type search. **See Also:** Commands: [Find\(\)](#), [lFind\(\)](#), [ViewFinds\(\)](#)

VWhereX

Returns the horizontal coordinate of the video output position, relative to the current video output window.

Syntax: [INTEGER](#) [VWhereX\(\)](#) **Returns:** The current "x" coordinate of the video output position, in the range 1 through [Query\(PopWinCols\)](#). **Notes:** The current video output window is defined by the [Window\(\)](#) or [PopWinOpen\(\)](#) commands, or by the editor's default [FullWindow\(\)](#), which is the entire screen. This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." **See Also:** [VWhereY\(\)](#), [WhereX\(\)](#), [WhereY\(\)](#), [GotoXY\(\)](#), [VGotoXY\(\)](#), [WhereYAbs\(\)](#), [WhereXAbs\(\)](#) **Variables:** [PopWinCols](#)

VWhereY

Returns the vertical coordinate of the video output position, relative to the current video output window.

Syntax: [INTEGER](#) [VWhereY\(\)](#) **Returns:** The current "y" coordinate of the video output position, in the range 1 through [Query\(PopWinRows\)](#). **Notes:** The current video output window is defined by the [Window\(\)](#) or [PopWinOpen\(\)](#) commands, or by the editor's default [FullWindow\(\)](#), which is the entire screen. This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." **See Also:** [WhereX\(\)](#), [WhereX\(\)](#), [WhereY\(\)](#), [GotoXY\(\)](#), [VGotoXY\(\)](#), [WhereYAbs\(\)](#), [WhereXAbs\(\)](#) **Variables:** [PopWinRows](#)

VWindow

Creates a new editing window, splitting the current window in half vertically.

Syntax: [INTEGER](#) [VWindow\(\)](#) **Returns:** Non-zero if successful; zero ([FALSE](#)) on failure. **Notes:** If there is not enough room on the screen to create the window, or the maximum number of windows already exists, no action occurs and zero is returned. The new window becomes the current editing window. If only a single file is loaded, it is shown in the new window at the same cursor position. Otherwise, the next file in the Ring is shown in the new window. **Examples:** To force the [VWindow\(\)](#) command to always load the current file into the new window, use the following key assignment: [proc](#) myVWindow() [if](#) [VWindow\(\)](#) [PrevFile\(\)](#) [endif](#) [end](#) <key> myVWindow() To cause VWindow to prompt for a new file: [proc](#) myVWindow() [if](#) [VWindow\(\)](#) [EditFile\(\)](#) [endif](#) [end](#) <key> myVWindow() **See Also:**

[CloseWindow\(\)](#), [HWindow\(\)](#), [ZoomWindow\(\)](#) Variables: [CurrWinBorderAttr](#), [OtherWinBorderAttr](#)

WaitForKeyPressed

Waits a specified time to see if a key was pressed.

Syntax: [BOOLEAN WaitForKeyPressed](#)([INTEGER](#) milliseconds [, [INTEGER](#) allow_idle_hook]) - milliseconds is the number of milliseconds to wait for a key to be pressed. A value of -1 causes the function to not return until a key is pressed. milliseconds is optional; the [default](#) value is 0. - allow_idle_hook determines if the [_IDLE_](#) or [_NONEDIT_IDLE_](#) hooks are called while waiting. allow_idle_hook is optional; the default value is FALSE. Returns: Non-zero if a key was pressed before the timeout value; zero ([FALSE](#)) if no [key](#) was pressed. **Examples:** //Wait up to 100 milliseconds for a key press. Do not allow //idle hooks to be called. [WaitForKeyPressed](#)(100) //Wait up to 1000 milliseconds for a key press. Allow idle //hooks to be called. [WaitForKeyPressed](#)(1000, [TRUE](#)) //Wait indefinitely for a key to be pressed. Allow idle hooks //to be called. //Essentially the same as [GetKey\(\)](#), except the key pressed is //not returned. [WaitForKeyPressed](#)(-1, [TRUE](#)) See Also: [KeyPressed\(\)](#), [GetKey\(\)](#)

WaitForMouseEvent

Waits for a specified mouse event.

Syntax: [INTEGER WaitForMouseEvent](#)([INTEGER](#) event_type) - event_type is any combination of the following (OR'ed together to obtain the desired result): - [_MOUSE_HOLD_TIME_](#) waits until the MouseHoldTime delay has been exceeded. (See the MouseHoldTime variable.) - [_MOUSE_MOVE_](#) waits until the mouse is moved. - [_MOUSE_RELEASE_](#) waits until the mouse key(s) are released, or another mouse [key](#) is pressed. (See [MouseKeyHeld\(\)](#)) Returns: event_type when the specified event occurs. Notes: This command does NOT update the MouseKey, MouseX, and MouseY [variables](#). Use [MouseStatus\(\)](#) to update the current mouse status. **Examples:** Once a mouse key is pressed, to wait until the mouse is moved or released try: [if WaitForMouseEvent](#)([_MOUSE_MOVE_](#) | [_MOUSE_RELEASE_](#)) == [_MOUSE_MOVE_](#) [Message](#)("Mouse Moved") [else Message](#)("Mouse [key](#) Released") [endif](#) To get the mouse location AFTER a [_MOUSE_MOVE_](#), call [MouseStatus\(\)](#) See Also: [MouseStatus\(\)](#), [MouseKeyHeld\(\)](#) Variables: [MouseHoldTime](#)

Warn

Writes the specified expression(s) on the the editor StatusLine and prompts the user to press <Escape>.

Syntax: [INTEGER Warn](#)([[STRING](#)] | [[INTEGER](#)] expr [: [INTEGER](#) width [: [STRING](#) pad_char [: [INTEGER](#) base]]] [, [[STRING](#)] | [[INTEGER](#)] expr [: [INTEGER](#) width [: [STRING](#) pad_char [: [INTEGER](#) base]]]]...) - expr represents each expression. Multiple [expressions](#) must be separated by commas. [Expressions](#) can be of type [integer](#) or [string](#). - width is the optional format width. A negative value will left justify the expression, whereas a positive value will right justify the expression. - pad_char is the optional character that is used to pad the expression to fit the specified width. The space character is used if this character is [not](#) specified. - base is the optional base to use when converting numeric [expressions](#). Base 10 is used if this value is [not](#) specified. Returns: Zero ([FALSE](#)). Notes: The expressions remain on [the StatusLine](#) and the editor operation is suspended until the user presses <Escape>. Specifying

any [optional parameters](#) requires specifying all [optional parameters](#) which precede it. Thus, to specify base requires that width and pad_char be specified. If a numeric expression yields a negative result and the expression is to be right justified (width is a positive value), then the pad_char character is placed between the negative sign "-" (which is left justified) and the actual number. base can only be specified for [numeric expressions](#). If an expression will not fit within the specified width, the expression is truncated to width less 1, and the ">" character is placed in the last position. The [Warn\(\)](#) command supports the same expression format as the [Format\(\)](#) command. [Examples](#): Display the number of lines in the current file on [the StatusLine](#): [Warn](#)("There are ", [NumLines](#)(), " line(s) in the current file.") See Also: [Message\(\)](#), [Format\(\)](#), [Write\(\)](#), [WriteLine\(\)](#), [MsgBox\(\)](#), [MsgBoxEx\(\)](#), [MsgBoxBuff\(\)](#), [YesNo\(\)](#)

WhereX

Returns the horizontal coordinate of the visible cursor, relative to the current video output window.

Syntax: [INTEGER WhereX\(\)](#) Returns: The current "x" coordinate of the visible cursor, in the range 1 through [Query \(PopWinCols\)](#). Notes: The current video output window is defined by the [Window\(\)](#) or [PopWinOpen\(\)](#) commands, or by the editor's default [FullWindow\(\)](#), which is the entire screen. This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [WhereY\(\)](#), [GotoXY\(\)](#), [VGotoXY\(\)](#), [WhereYAbs\(\)](#), [WhereXAbs\(\)](#), [VWhereX\(\)](#), [VWhereY\(\)](#) [Variables](#): [PopWinCols](#)

WhereXAbs

Returns the horizontal coordinate of the visible cursor, relative to the full screen.

Syntax: [INTEGER WhereXAbs\(\)](#) Returns: The current "x" coordinate of the visible cursor, in the range 1 through [Query\(ScreenCols\)](#). Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [WhereYAbs\(\)](#), [WhereX\(\)](#), [WhereY\(\)](#), [GotoXY\(\)](#), [VGotoXY\(\)](#), [VGotoXYAbs\(\)](#), [VWhereX\(\)](#), [VWhereY\(\)](#) [Variables](#): [ScreenCols](#)

WhereY

Returns the vertical coordinate of the visible cursor, relative to the current video output window.

Syntax: [INTEGER WhereY\(\)](#) Returns: The current "y" coordinate of the visible cursor, in the range 1 through [Query \(PopWinRows\)](#). Notes: The current video output window is defined by the [Window\(\)](#) or [PopWinOpen\(\)](#) commands, or by the editor's default [FullWindow\(\)](#), which is the entire screen. This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [WhereX\(\)](#), [GotoXY\(\)](#), [VGotoXY\(\)](#), [WhereYAbs\(\)](#), [WhereXAbs\(\)](#), [VWhereX\(\)](#), [VWhereY\(\)](#) [Variables](#): [PopWinRows](#)

WhereYAbs

Returns the vertical coordinate of the visible cursor, relative to the full screen.

Syntax: [INTEGER WhereYAbs\(\)](#) Returns: The current "y" coordinate of the visible cursor, in the range 1 through [Query\(ScreenRows\)](#). Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [WhereXAbs\(\)](#), [WhereX\(\)](#), [WhereY\(\)](#), [GotoXY\(\)](#), [VGotoXY\(\)](#), [VGotoXYAbs\(\)](#), [VWhereX\(\)](#), [VWhereY\(\)](#) Variables: [ScreenRows](#)

WhichOS

Identifies the operating system under which the editor is currently running.

Syntax: [INTEGER WhichOS\(\)](#) Returns: One of the following: - [_WINDOWS_](#) if the operating system is in the Windows 95 family (including [Windows 98](#) and [Windows Millennium](#)) - [_WINDOWSNT_](#) if the operating system is in the Windows NT family (including [Windows 2000](#) and [Windows XP](#)) - [_LINUX_](#) if the Linux version of the editor is running. **Examples:** [case WhichOS\(\)](#) [when](#) [_WINDOWS_](#) // do Non-WindowsNT specific code here [when](#) [_WINDOWSNT_](#) // do WindowsNT specific code here [when](#) [_LINUX_](#) // do Linux specific code here [endcase](#) See Also: [is32BitApp\(\)](#), [isFullScreen\(\)](#)

White Characters

The macro language is

[not](#) white-character sensitive. This means that anywhere [white characters](#) are required, there is no difference between one white character and multiple [white characters](#). [White characters](#), in this context, consist of spaces, tabs, and end-of-line characters. For example: [integer](#) a = 5 while a > 0 // loop while nonzero a = a - 1 [endwhile](#) Could also be written as: [integer](#) a=5 while a>0 a=a-1 [endwhile](#) Or as: [integer](#) a = 5 [while](#) a > 0 a = a - 1 [endwhile](#) A little judicious use of white characters can go a long way in making macros more readable and understandable.

Window

Creates a new video output window. All screen writes from video output commands will use this window.

Syntax: [Window](#)([INTEGER](#) x1, [INTEGER](#) y1, [INTEGER](#) x2, [INTEGER](#) y2) - x1 is the beginning column coordinate - y1 is the beginning row coordinate - x2 is the ending column coordinate - y2 is the ending row coordinate of the video output window to be established Returns: Nothing. Notes: This is a video output command. It has no effect on the actual text in the buffer. It only affects what is displayed on the screen. The background text is [not](#) saved with this command, as it is with [PopWinOpen\(\)](#). The sole purpose of this command is to limit the output of video output commands to a particular area of the display. Any macros that use the [Window\(\)](#) command should make certain they call [FullWindow\(\)](#) before any non-[video output commands](#) are executed, and especially before the macro is

terminated. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." See Also: [FullWindow\(\)](#)

WindowCols

The number of columns in the current editing window.

Values: [INTEGER](#) **Default:** Undefined **Limits:** Can only be queried. Cannot be used within a config/endconfig construct. **Notes:** WindowCols returns the number of columns in the current editing window. The [Window\(\)](#) and [PopWinOpen\(\)](#) commands do not affect the value of this variable. See Also: [WindowRows](#), [ScreenCols](#), [ScreenRows](#), [WindowX1](#), [WindowY1](#), [PopWinCols](#), [PopWinRows](#)

WindowFooter

Displays a HelpLine string at the bottom of the current editing or pop-up window.

Syntax: [WindowFooter](#)([STRING](#) text) - text is the string (coded in standard HelpLine format) to be displayed. Returns: Nothing. **Notes:** This command is used in conjunction with the [PopWinOpen\(\)](#) and [Window\(\)](#) commands to display a HelpLine at the bottom of a pop-up or editing window. (To display a HelpLine string at the bottom of a List window, via the [List\(\)](#) or [IList\(\)](#) commands, use the [ListFooter\(\)](#) command.) **Examples:** See the [PutCtrStr\(\)](#) command for an example. Also, several supplied macros use this command to place a HelpLine at the bottom of a pop-up window. For an example, see FILEINFO.S (in the MAC\ editor subdirectory). See Also: [ListFooter\(\)](#), [PutHelpLine\(\)](#)

WindowId

Returns the id of the current editing window.

Syntax: [INTEGER](#) [WindowId](#)() Returns: The non-zero id of the current window. **Notes:** The value returned by this command is intended for use with the [GotoWindow\(\)](#) command. See Also: [GotoWindow\(\)](#), [NumWindows\(\)](#), [MouseWindowId\(\)](#)

WindowRows

The number of rows in the current editing window.

Values: [INTEGER](#) **Limits:** Can only be queried. Cannot be used within a config/endconfig construct. **Default:** Undefined **Notes:** WindowRows returns the current number of rows in the current editing window. The [Window\(\)](#) and [PopWinOpen\(\)](#) commands do not affect the value of this variable. See Also: [WindowCols](#), [ScreenCols](#), [ScreenRows](#), [WindowX1](#), [WindowY1](#), [PopWinCols](#), [PopWinRows](#)

Windows

To view multiple files on the screen, or to display multiple views of the same file, the screen can be split into windows. Windows can be created by splitting the screen horizontally and/or vertically, and can be opened, closed, or resized as needed to facilitate editing.

You can configure the editor to separate multiple windows with a divider line only along adjacent sides of windows; or you can specify that a full border be placed around each window (including a single window). To use scrollbars with a mouse, it is necessary to include a full border around each window. In the standard configuration of the editor, full borders are placed around each window when a mouse is enabled; however, if no mouse is in use, borders are

not displayed (so that multiple [windows](#) are separated only by divider lines along adjacent sides). (For more information about creating and using windows, see the chapters on "[Splitting the Screen into Windows](#)" and "[Mousing Around the Editor](#)".)

WindowX1

The top left column coordinate of the current editing window, relative to the full display area.

Values: [INTEGER](#) Default: Undefined Limits: Can only be queried. Cannot be used within a config/endconfig construct. Notes: WindowX1 contains the column coordinate of the top left-hand corner of the current editing window relative to the full screen display. This value can be used in conjunction with [WindowY1](#), [WindowCols](#), and [WindowRows](#) to force video output [windows](#) to be displayed within the bounds of the current editing window. The [Window\(\)](#) and [PopWinOpen\(\)](#) commands do not affect the value of this variable. See Also: [WindowY1](#), [WindowCols](#), [WindowRows](#), [ScreenCols](#), [ScreenRows](#), [PopWinX1](#), [PopWinY1](#)

WindowY1

Contains the top left row coordinate of the current editing window, relative to the full display area.

Values: [INTEGER](#) Default: Undefined Limits: Can only be queried. Cannot be used within a config/endconfig construct. Notes: WindowY1 contains the row coordinate of the top left-hand corner of the current editing window relative to the full screen display. This value can be used in conjunction with [WindowX1](#), [WindowCols](#), and [WindowRows](#) to force video output [windows](#) to be displayed within the bounds of the current editing window. The [Window\(\)](#) and [PopWinOpen\(\)](#) commands do not affect the value of this variable. See Also: [WindowX1](#), [WindowCols](#), [WindowRows](#), [ScreenCols](#), [ScreenRows](#), [PopWinX1](#), [PopWinY1](#)

WordLeft

Moves the cursor to the beginning of the previous word.

Syntax: [INTEGER](#) [WordLeft\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: Returns zero at the beginning of the file. This command moves the cursor to the previous line, if necessary. If the cursor is located within

a word, this command moves it to the beginning of the word. If the cursor is located at the first character of a word, or on a non-word character, this command moves the cursor to the beginning of the previous word. If the cursor is located before or at the first character of the first word on a line, [WordLeft\(\)](#) will place the cursor at the end of the preceding line. The WordSet variable determines what constitutes a word. See Also: [WordRight\(\)](#), [isWord\(\)](#), [MarkWord\(\)](#), [DelLeftWord\(\)](#), [DelRightWord\(\)](#), [BegWord\(\)](#), [EndWord\(\)](#) Variables: [WordSet](#)

WordRight

Moves the cursor to the beginning of the next word.

Syntax: [INTEGER WordRight\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: [FALSE](#) is returned at end of file. This command moves the cursor to the next line, if necessary. If the cursor is already on the last word of a line, [WordRight\(\)](#) moves the cursor to the end of the line. The next [WordRight\(\)](#) will move the cursor to the next line. The WordSet variable determines what constitutes a word. See Also: [WordLeft\(\)](#), [isWord\(\)](#), [MarkWord\(\)](#), [DelLeftWord\(\)](#), [DelRightWord\(\)](#), [BegWord\(\)](#), [EndWord\(\)](#) Variables: [WordSet](#)

WordSet

Defines the characters that make up a "word."

Values: [STRING](#) 32-character bit-mapped string allowing for the specification of 256 characters. **Default:** [ChrSet](#)("0-9A-Z_a-z") Notes: The 256 possible characters of the WordSet are stored in a bit-encoded [string](#). A 32-character [string](#) is necessary to set/query the WordSet. To generate the 32-character bit-encoded string required by WordSet, use the [ChrSet\(\)](#) command. The string passed to [ChrSet\(\)](#) must be a constant or literal string, and can not be a string variable. The slash ("/") character is used as an escape character. In order to include a slash in the set, use two slashes ("\\/"). The \\nnnn notation or \\nnn notation, where nnn is a decimal number, and nn is a hexadecimal number, is allowed in specifications. To include the dash in the set, place it as either the first or last character. The "~" character is used as "not in this set," if it occurs as the first character. This also affects the outcome when using "W" (Words only) in the option field of [Find\(\)](#) and [Replace\(\)](#). **Affects:** [isWord\(\)](#), [WordLeft\(\)](#), [WordRight\(\)](#), [MarkWord\(\)](#), [DelLeftWord\(\)](#), [DelRightWord\(\)](#), [GetWord\(\)](#) See Also: Commands: [ChrSet\(\)](#)

WordWrap

Causes text to wrap to the next line if the cursor position exceeds the current right margin setting and text is typed.

Values: [INTEGER](#) - [OFF](#) turns [WordWrap OFF](#). - [ON](#) turns [WordWrap ON](#). - [_AUTO_](#) enables Automatic WordWrapping. **Default:** [OFF](#) Notes: When WordWrap is ON, and your cursor exceeds the current RightMargin setting as you are entering text, a new line is created and becomes the new cursor line, once a character is typed. If the previous word, before wrapping, exceeded the RightMargin, it is placed on the newly-created line along with the cursor. When WordWrap is set to [_AUTO_](#), the entire paragraph (in which the cursor is located) is reformatted as characters are inserted or deleted. See Also: [LeftMargin](#), [RightMargin](#), [AutoIndent](#) Commands: [WrapLine\(\)](#)

Wordwrap Mode

When you are entering text, it is often desirable to automatically advance, or wrap, the cursor to the next line, once the cursor goes beyond the right margin. Or, you may wish to have the text automatically reformatted through the end of a paragraph, as characters are inserted or deleted within existing text in the paragraph. WordWrap mode allows you to accomplish this.

[WordWrap mode](#) has three settings: [OFF](#), [ON](#), or AUTO. To change [WordWrap mode](#) to one of these settings, select "[WordWrap](#)" under the Options Menu, and then choose the desired setting. When Wordwrap mode is ON or AUTO, a "W" displays on [the StatusLine](#). (You can toggle between [ON](#)/AUTO and [OFF](#) by pressing <Ctrl O W>.) To use [WordWrap mode](#), you must activate it by setting it to [ON](#) or AUTO. You must also specify a right margin by setting the RightMargin option to a value greater than zero. (To set RightMargin, select "Right Margin" under the Options Menu, and enter the desired value.) When WordWrap mode is activated (ON or AUTO), the cursor automatically advances to the next line once you type a non-white character beyond the right margin. The word you are typing also wraps to the next line, and is positioned at [the effective left margin](#). ([The effective left margin](#) is defined in the following section on "[AutoIndent Mode](#).") In addition, when WordWrap mode is set to AUTO, the editor automatically reformats the text from the cursor position, through the end of the paragraph, as you insert or delete characters within the existing text of the paragraph. The text is formatted between [the effective left margin](#) and the right margin. (In this mode, the end of the paragraph is assumed when a blank line, or the end of the file, is encountered.) NOTE: The editor does not break a line in the middle of a word. (In WordWrap mode, a "word" is simply a continuous string of non-white characters.) If you enter a continuous string of characters that extends beyond the right margin, the editor will not wrap that "word" if it is too long to fit on the next line within the left and right margins. In this case, the cursor also does not automatically advance to the next line.

Working in the Editor

[Executing Commands](#) ----- You can execute editor commands either by using the pull-down Menu, or by pressing the key to which that command is assigned. For commands that require additional information, the editor displays a command prompt when you invoke the command. To execute a command from the Menu, first press <F10> (or <Escape>) to activate the main Menubar. Now use the cursor movement keys to position the cursor bar over the desired Menu category. Press <Enter> to select that Menu category and display a sub-menu list of associated commands or options. Then move the cursor bar over the desired command or option and press <Enter>. Alternatively, you can select a Menu item by pressing the highlighted quick-key for the item. For many commands that you execute from the Menu, the editor automatically deactivates the Menu and returns you to your editing session once execution is complete. Otherwise, to deactivate the Menu system, press <Escape> until the Menubar is no longer active. Each time you press <Escape> the editor backs up to the previous Menu selection. When you press <Escape> from the main Menubar, the editor deactivates the Menu system. To execute a command that is assigned to a key, simply press the appropriate keystroke(s). Pre-assigned keystrokes for most commands are displayed on the pull-down [Menu](#), or on [the HelpLine](#) or [Key-Assignment Help](#) screen (described in the section "[Getting Help](#)" later in this chapter). You can also change or add [key](#) assignments for commands [to](#) suit your own preferences. Commands may be assigned to the following types of key assignments: - "Unshifted" keys, such as <K> or <F4>. - "Shifted" keys, in combination with <Ctrl>, <Alt>, or <Shift>, such as <Ctrl K> or <Alt F4>. Shifted keys may be either single-shift combinations, like <Ctrl K>; or multi-shift combinations, like <CtrlAlt F4>. - "Two-[key](#)" [key](#) sequences, which consist of two distinct keystrokes. A two-[key](#) can be either unshifted (such as <F4 A>) or shifted (such as <Ctrl K S>). To activate a two-key, hold down the first unshifted key or shifted [key](#) set that is specified (such as <F4> or <Ctrl K>). The editor then displays "<TwoKey>" on [the StatusLine](#). Release the first key or key set, and enter the second key that is identified (such as <A> or <S>). Note that the combination of <Alt>, <Shift>, or <Ctrl> with another single key (such as <Ctrl K>, <Alt F4>, or <CtrlAlt F4>) is considered a single key, NOT a two-key. Many commands require additional information in order to complete execution. To request this information from the user, the editor uses selection menus, StatusLine prompts, prompt boxes, and PickLists. For commands that issue a prompt box, the editor maintains a

History of previous responses from which you can select an entry. For some commands (such as file-related commands), the editor allows you to display a PickList of entries from which to choose. As you read [the User's Guide](#), notice that some commands are assigned to a key, but are not available on the pull-down Menu. Other commands are available on the pull-down Menu, but are not assigned to a key. A few commands are included that are [not](#) pre-assigned to any [key](#) or placed on any [Menu](#). If desired, you can assign such commands to keys and/or place them on the Menu. (For more information on how to use the pull-down Menu, see the chapter on "[The Pull-Down Menu System](#)." For more information on changing [key](#) assignments or adding key assignments to the Menu, see the chapter on "[Fully Customizing the Editor](#)." For more information on the keys to which commands are assigned in the standard version of the editor, see "[Appendix C: Standard Key Assignments](#).")

[Getting Help](#) ----- A StatusLine is constantly displayed within the editor to provide status information about the file currently being edited. [The StatusLine](#) shows the following: the current line and column number; the setting for various editing modes; the amount of memory currently in use; an indication of whether file-locking is in effect (if this option is available); an indication of whether the current file has been modified; and the path and filename of the current file. In the standard configuration of the editor, [the StatusLine](#) appears at the top of the screen, but it can be configured to appear at the bottom instead. The editor also provides an optional multi-bar HelpLine to display a line of key assignments for selected commands. [The HelpLine](#) appears at the top or bottom of the screen, opposite [the StatusLine](#). Four separate HelpLine "information bars" can be defined: one that is continually displayed on the screen, and three additional bars that can be displayed by pressing <Shift>, <Alt>, or <Ctrl>. A multi-page key-assignment Help screen is available within the editor to display various commands and indicate the key(s) to which those commands have been assigned. The Help screen that is supplied in the standard configuration of the editor displays commands that are not listed on the pull-down Menu or any of [the HelpLine](#) bars. To display the key-assignment Help screen, select "[Key Assignments](#)" under the Help Menu (or press <Shift F1>). Use the <CursorUp>, <CursorDown>, <PgUp>, and <PgDn> keys to scroll through the Help screen entries. Press <Escape> to remove the Help screen and return to the file being edited. Additional help can be obtained through the editor's context-sensitive, hypertext HelpSystem. [The HelpSystem](#) provides online access to information about the editor's commands, options, and macro programming language features. You can access [the HelpSystem](#) from the Help Menu. You can also obtain [context-sensitive Help](#) by pressing <F1>. (For more information on [the StatusLine](#) and other information displayed on the screen, see the chapter on "[The Screen Layout](#)." For more information on [the HelpLine](#), and the key-assignment Help screen and HelpSystem, see the chapter on "[Getting Help in the Editor](#).")

[Using a Mouse](#) ----- If you prefer to use a mouse, the editor's integrated mouse support allows you to use your mouse for scrolling, window, block, and [menu](#) operations. You can even configure the operation of the mouse to suit your own needs. (For more information on using the mouse, see the chapter on "[Mousing Around the Editor](#).")

[Editing Multiple Files and Displaying Multiple Windows](#) ----- The editor allows you to edit an almost unlimited number of files. And by splitting the screen into horizontal and/or vertical windows, you can simultaneously view different files on one screen, or display different views of the same file. You can select multiple files for loading when the editor is started, and you can select additional files from within the editor. File commands are available to: - select files to be loaded, [using PickLists](#), wildcard characters ("*" [or](#) "?"), partial filename specifications, or the editor's Filename Completion option - display a PickList of all selected files or the files used most recently - switch between files and/or [windows](#) - simultaneously scroll through files in multiple [windows](#) - compare two files and identify differences - perform various file operations using the FileManager - search across multiple drives and directories for a file - change the name of a file being edited - insert a copy of one file into another - create a new file from a portion of a file - save files that have been modified - close files that are no longer needed in a session The window commands allow you to open up to nine different [windows](#) on one screen, and move between them. [Windows](#) can be closed or resized as needed. You can temporarily "zoom" one window to fill the entire screen, then automatically restore the individual windows. (For more information on file-processing and window features, see the chapters on "[Loading, Processing, and Saving Files](#)" and "[Splitting the Screen into Windows](#).")

[Moving Around a File](#) ----- Once you are editing a file in the editor, the basic cursor movement keys (<CursorUp>, <CursorDown>, <CursorLeft>, <CursorRight>, <PgUp>, <PgDn>, <Home>, <End>) allow you to move through the text. Commands are available to move to the beginning or end of a line, paragraph, block, window, or file; move forward or backward by words; move to a specific line or column; or

scroll the text on the screen. Tab commands allow you to move the cursor based on fixed or variable tab settings. [Bookmarks](#) allow you to mark positions within files, and then return to a marked position at the touch of a [key](#). (For more information on moving through a file, see the chapter on "[Moving Through Text in a File](#)." For more information on the tab features, see the chapter on "[Using Tabs](#)." For more information on using Bookmarks, see the chapter on "[Bookmarks](#).") [Activating Editing Modes](#) ----- You can activate optional [editing modes](#) to assist you in entering or viewing text. WordWrap mode causes the editor to automatically advance to the next line when you type text beyond a user-defined right margin, and optionally causes text to be automatically reformatted as characters are inserted or deleted within existing text in a paragraph. AutoIndent mode causes the editor to position the cursor on a new line based on text on a previous line. [Special programming-language modes](#) (SmartIndent and C modes) assist you in properly indenting code for various programming languages. [SyntaxHilite mode](#) colorizes syntactical elements of your programming language code. An additional editing mode, Template mode, is a handy shortcut feature for expanding abbreviated text. (For more information on these editing modes, see the chapter on "[Editing Modes](#).") [Performing Editing Tasks](#) ----- Commands are available to allow you to manipulate text by characters, words, lines, or blocks. Text can be entered, copied, moved, shifted, formatted, deleted (and recovered), and much more. Commands are available to allow you to accomplish tasks like: - moving or copying text between different files - using [ClipBoards](#) to temporarily store and retrieve text (and even copy and paste text between the editor and the Microsoft Windows Clipboard) - saving modified files - editing a file in hexadecimal and/or binary mode - formatting text with various options including line-centering, automatic indentation, paragraph formatting, and left and right justification - filling a block with a string of character(s) - changing the case of a single character or the characters in a word, line, or block (including the option to capitalize words) - finding and replacing text within a block, throughout an entire file, or across multiple files (including the use of Regular Expressions) - notating text as a quote or comment - listing all occurrences of a selected string, or listing all functions or procedures for many programming languages - counting the number of occurrences of a particular word, or counting the total number of words, in a block or file - checking the spelling of words in a file - printing all or a portion of a file (with options to include header and footer lines, include page numbers and line numbers, assign the print device, specify line spacing, and more) - [drawing lines in the text](#) to create diagrams and tables - sorting all or a portion of a file, in ascending or descending order - inserting any ASCII character from a pop-up ASCII chart - [inserting the date and time into the text](#) - changing the screen video mode (to whatever rows/columns supported by current screen resolution and font) - [executing external commands from within the editor](#) - compiling a program from within the editor You can even write your own custom commands (Compiled [Macros](#)), and make them readily available within the editor using the Potpourri facility. (For more information on executing specific commands, see the remaining chapters of this manual. For more information on the Potpourri facility, see the chapter on "[Special Commands and Features](#).") [Using Keyboard Macros](#) ----- To automate repetitive editing tasks, you can record [Keyboard Macros](#). A Keyboard Macro is simply a recorded series of commands and keystrokes entered from the keyboard. A Keyboard Macro is created by activating macro-recording mode, then entering the desired commands and characters as you would for normal editing. When complete, you turn macro-recording mode [OFF](#). The Keyboard Macro can then be executed as often as needed to repeat the recorded operations. You can even save [Keyboard Macros](#) for use in future editing sessions. (For more information on recording and running Keyboard Macros, see the chapter on "[Creating and Using Macros](#).")

Working With Windows

This section describes how to use the mouse to create, change, resize, and close

[windows](#). To open a horizontal window: Place the mouse pointer on the left border of the window to be split, at the desired horizontal position, and click the <LeftBtn> twice. To open a vertical window: Place the mouse pointer on the top border of the window to be split, at the desired vertical position, and click the <LeftBtn> twice. To change to another window: Place the mouse pointer in the desired window, and click the <LeftBtn>. To change to another window and also position the editing cursor at the mouse position in that window, press and momentarily hold the

<LeftBtn>. (The <LeftBtn> must be held for about 1/2 second, or for the amount of time indicated by the MouseHoldTime variable). To resize a window horizontally: Place the mouse pointer on the TOP interior border of the window to be resized. Note that the window to be resized must be the same width as the adjacent [window](#)(s) sharing the selected border. Press and hold the <LeftBtn>, so that a resizing bar appears. Then drag the resizing bar to the desired location and release the <LeftBtn>. You cannot select a border for resizing if that border contains a scrollbar. If necessary, change the current window so that the required border does not include a scrollbar. To resize a window vertically: Place the mouse pointer on the LEFT interior border of the window to be resized. Note that the window to be resized must be the same height as the adjacent [window](#)(s) sharing the selected border. Press and hold the <LeftBtn>, so that a resizing bar appears. Then drag the resizing bar to the desired location and release the <LeftBtn>. You cannot select a border for resizing if that border contains a scrollbar. If necessary, change the current window so that the required border does not include a scrollbar. To zoom a window: Place the mouse pointer on the [^] symbol in the title line of the window to be zoomed, and click the <LeftBtn>. While the window is zoomed, the [^] symbol is replaced by the [v] symbol. To "unzoom" a window: Place the mouse pointer on the [v] symbol and click the <LeftBtn>. The previous windows are restored to the screen, along with the [^] symbol. To close a window: Place the mouse pointer on the [*] symbol in the title line of the window to be closed, and click the <LeftBtn>. (For more information on windows, see the chapter on "[Splitting the Screen into Windows](#).")

WrapLine

Wraps the current line at or before the right margin, if the current line length is greater than the RightMargin setting.

Syntax: [INTEGER WrapLine\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: This is the command the editor uses to implement normal WordWrap mode when text is being typed. See Also: [WrapPara\(\)](#) [Variables:](#) [WordWrap](#), [RightMargin](#)

WrapPara

Reformats a paragraph from the current cursor position to the end of the paragraph.

Syntax: [INTEGER WrapPara\(\)](#) Returns: Non-zero if successful; zero ([FALSE](#)) on failure. Notes: [WrapPara\(\)](#) looks forward for a non-blank line, if executed when the cursor is on a blank line. [WrapPara\(\)](#) computes two values to control the reformatting process: Indent and Effective Left Margin. - Indent is set to the column containing the first non-white character on the current line. - If the AutoIndent variable is set to ON or STICKY, the Effective Left Margin is set to the column containing the first non-white character on the next line. - If the AutoIndent variable is set OFF, [the Effective Left Margin](#) is set to the value of the LeftMargin variable or, if the setting is zero, it is set to 1. [WrapPara\(\)](#) reformats such that the current line beginning position is not changed, each subsequent reformatted line begins at the Effective Left Margin, and no characters on each line are in a position greater than the RightMargin variable. However, if a single word, when begun at [the Effective Left Margin](#) or Indent, overruns the RightMargin setting, it is allowed to do so. For instance, if RightMargin is set to 1, a single word is placed on each line. A word's punctuation is always kept with the word. Trailing white characters are always stripped from lines being reformatted. WrapPara continues processing until it reaches a condition specified by the ParaEndStyle variable. If ParaEndStyle is: - 0 (default) - Continue WrapPara process until a blank line or the end of the file is encountered. - 1 - Same as 0 but also stops when a line is encountered whose first non-white character is in the indent column. (This option supports indented and outdented paragraphs.) See Also: [WrapLine\(\)](#) [Variables:](#) [LeftMargin](#), [RightMargin](#), [AutoIndent](#), [ParaEndStyle](#), [WordWrap](#)

Write

Writes the specified expression(s) to the screen at the current video output position. The video output position (and visible cursor) are set to the end of the written text.

Syntax: [Write](#)([[STRING](#)] | [[INTEGER](#)] expr [: [INTEGER](#) width [: [STRING](#) pad_char [: [INTEGER](#) base]]] [, [[STRING](#)] | [[INTEGER](#)] expr [: [INTEGER](#) width [: [STRING](#) pad_char [: [INTEGER](#) base]]] ...) - expr represents each expression. Multiple [expressions](#) must be separated by commas. [Expressions](#) can be of type [integer](#) or [string](#). - width is the optional format width. A negative value will left justify the expression, whereas a positive value will right justify the expression. - pad_char is the optional character that is used to pad the expression to fit the specified width. The space character is used if this character is [not](#) specified. - base is the optional base to use when converting numeric [expressions](#). Base 10 is used if this value is [not](#) specified. Returns: Nothing. Notes: This is a video output command and does not affect the actual text in the file or buffer. Specifying any [optional parameters](#) requires specifying all [optional parameters](#) which precede it. Thus, to specify base requires that width and pad_char be specified. If a numeric expression yields a negative result and the expression is to be right justified (width is a positive value), then the pad_char character is placed between the negative sign "-" (which is left justified) and the actual number. base can only be specified for [numeric expressions](#). If an expression will not fit within the specified width, it is truncated to width less 1, and the ">" character is placed in the last position. The [Write](#)() command supports the same expression format as the [Format](#)() command. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using Macro Language Features](#)." **Examples:** [GotoXY](#)(1, 1) [Write](#)("Hello, World!") See Also: [WriteLine](#)(), [Format](#)(), [PutLine](#)(), [PutStr](#)()

WriteLine

Writes the specified expression(s) to the screen at the current video output position. The video output position (and visible cursor) are set to the beginning of the next screen row.

Syntax: [WriteLine](#)([[STRING](#)] | [[INTEGER](#)] expr [: [INTEGER](#) width [: [STRING](#) pad_char [: [INTEGER](#) base]]] [, [[STRING](#)] | [[INTEGER](#)] expr [: [INTEGER](#) width [: [STRING](#) pad_char [: [INTEGER](#) base]]] ...) - expr represents each expression. Multiple [expressions](#) must be separated by commas. [Expressions](#) can be of type [integer](#) or [string](#). - width is the optional format width. A negative value will left justify the expression, whereas a positive value will right justify the expression. - pad_char is the optional character that is used to pad the expression to fit the specified width. The space character is used if this character is [not](#) specified. - base is the optional base to use when converting numeric [expressions](#). Base 10 is used if this value is [not](#) specified. Returns: Nothing. Notes: This is a video output command and does not affect the actual text in the file or buffer. Specifying any [optional parameters](#) requires specifying all [optional parameters](#) which precede it. Thus, to specify base requires that width and pad_char be specified. If a numeric expression yields a negative result and the expression is to be right justified (width is a positive value), then the pad_char character is placed between the negative sign "-" (which is left justified) and the actual number. base can only be specified for [numeric expressions](#). If an expression will not fit within the specified width, the expression is truncated to width less 1, and the ">" character is placed in the last position. This command is similar to [Write](#)(), except that the video output position is placed at the start of the next screen row after completion. If the video output position is already on the last row of the video output window, the window scrolls. The hardware (visible) cursor is also set to the same position. The [WriteLine](#)() command supports the same expression format as the [Format](#)() command. For more information on [video output commands](#), see "[Video Output Commands](#)" in the chapter on "[Using](#)

[Macro Language Features](#)." [Examples](#): `/***** displays "Hello, World!" on line 1 and "My name is Bobbi and I am 35 years old." on line 2 of the current video output window. *****/ integer age = 35 string name[5] = "Bobbi" GotoXY(1, 1) WriteLine("Hello, World!") WriteLine("My name is ",name," and I am ",age, " years old.") See Also: Write\(\), Format\(\), PutLine\(\), PutStr\(\)`

WriteProfileInt

Assigns a value to an item in an initialization file.

Syntax: `INTEGER WriteProfileInt(STRING section, STRING item, INTEGER value [, STRING profile_name])` - section is the section name within an initialization file. - item is the item name within the section of an initialization file. - value is the INTEGER value that will be assigned to the item. - profile_name is an optional initialization file's name. If not specified, the tse.ini file located in the editor's load directory is searched. If profile_name is specified, and does not include a path, the profile_name that resides in the WINDOWS directory (specified by the windir environment variable) is searched. [Examples](#): See the [FlushProfile\(\)](#) command example. Returns: Non-zero on success; zero ([FALSE](#)) on failure. Notes: The spelling of section and item are [not case](#) sensitive. See Also: [WriteProfileStr\(\)](#), [GetProfileInt\(\)](#), [GetProfileStr\(\)](#), [LoadProfileSection\(\)](#), [GetNextProfileItem\(\)](#), [LoadProfileSectionNames\(\)](#), [GetNextProfileSectionName\(\)](#), [RemoveProfileItem\(\)](#), [RemoveProfileSection\(\)](#), [FlushProfile\(\)](#)

WriteProfileStr

Assigns a value to an item in an initialization file.

Syntax: `INTEGER WriteProfileStr(STRING section, STRING item, STRING value [, STRING profile_name])` - section is the section name within an initialization file. - item is the item name within the section of an initialization file. - value is the STRING value that will be assigned to the item. - profile_name is an optional initialization file's name. If not specified, the tse.ini file located in the editor's load directory is searched. If profile_name is specified, and does not include a path, the profile_name that resides in the WINDOWS directory (specified by the windir environment variable) is searched. Returns: Non-zero on success; zero ([FALSE](#)) on failure. Notes: The spelling of section and item are [not case](#) sensitive. [Examples](#): See the [FlushProfile\(\)](#) command example. See Also: [WriteProfileInt\(\)](#), [GetProfileStr\(\)](#), [GetProfileInt\(\)](#), [LoadProfileSection\(\)](#), [GetNextProfileItem\(\)](#), [LoadProfileSectionNames\(\)](#), [GetNextProfileSectionName\(\)](#), [RemoveProfileItem\(\)](#), [RemoveProfileSection\(\)](#), [FlushProfile\(\)](#)

Writing Configuration Settings to a Settings File

IConfig allows you to create a Settings File containing all your current configuration settings. This file provides a handy listing of your current settings, which can be helpful to have available when calling for technical support regarding the use or operation of the editor.

You can also use the Settings File to create a variable configuration (Config/EndConfig) section for a user-interface macro file. This, in turn, can be used to install your configuration options in the editor, though this is

[not](#) generally recommended. (For more information, see "[Understanding User-Interface \(UI\) Macro Files](#)" and "[Installing a User-Interface \(UI\) Macro File](#)" in the chapter on "[Fully Customizing the Editor](#)".) The Settings File lists all of the IConfig configuration options, using the internal variable names. It also identifies the value for each option, using the internal representation, which may be different from the value as indicated within IConfig. For example, the internal variable name for "Line-Termination [String](#)" is [EOLType](#), with internal [values](#) of 1 (for "CR ONLY"), 2 (for "LF ONLY"), or 3 (for "CR/LF"). (For more information on the internal values for the variables, see the chapter on "[Editor Variables](#)" in [the Macro Reference Guide](#). Or you can press <Ctrl F1> on any of the variable names to access online Help which indicates valid values for a variable.) The following option is available on the IConfig menu: *

Write Settings to ASCII File This option allows you to create a text file containing all of the current settings for the options that are configurable with IConfig. This Settings File consists of each configuration option name (using the editor's internal variable name) followed by the setting for the option (using the editor's internal representation). The options are grouped according to the categories within IConfig (described in the preceding sections). When you select this option, the following prompt is displayed: "Settings filename:" Type the desired filename and press <Enter>. Or press <Escape> to cancel this request.

X1

Contains the current top left column coordinate, relative to the full display area, where the next editor prompt,

[List\(\)](#), menu, or help display will begin. [Values](#): [INTEGER](#) [Default](#): Undefined Notes: When a video output window command is called, the current value of X1 determines the upper left column coordinate for the window. [Affects](#): [List\(\)](#), [MsgBox\(\)](#), [MsgBoxBuff\(\)](#), and [MsgBoxEx\(\)](#); [Menu](#) and Help functions; internal editor prompts See Also: [Y1](#), [ScreenCols](#), [ScreenRows](#), [WindowX1](#), [WindowY1](#), [WindowCols](#), [WindowRows](#)

Y1

Contains the current top left row coordinate, relative to the full display area, where the next editor prompt,

[List\(\)](#), menu, or help display will begin. [Values](#): [INTEGER](#) [Default](#): Undefined Notes: When a video output window command is called, the current value of Y1 determines the upper left row coordinate for the window. [Affects](#): [List\(\)](#), [MsgBox\(\)](#), [MsgBoxBuff\(\)](#), and [MsgBoxEx\(\)](#); [Menu](#) and Help functions; internal editor prompts See Also: [X1](#), [ScreenRows](#), [ScreenCols](#), [WindowY1](#), [WindowX1](#), [WindowRows](#), [WindowCols](#)

YesNo

Displays a pop-up menu on the screen, allowing the user to choose "Yes", "No", or "Cancel".

[Syntax](#): [INTEGER](#) [YesNo](#)([STRING](#) title) - title is displayed along the top border of the menu. Returns: The number of the selected menu item, or zero (FALSE) if <Escape> is pressed. Notes: This is a menu that can be called whenever the user needs to give a choice of "Yes", "No", or "Cancel" to a routine. The following [values](#) are returned by [YesNo\(\)](#): - 0 - <Escape> is pressed; or <c> is pressed or "Cancel" is selected. - 1 - <y> is pressed or "Yes" is selected. - 2 - <n> is pressed or "No" is selected. [Examples](#): Replace the current file with a new file. [proc](#) mReplaceFile() [if](#) [FileChanged\(\)](#) [case](#) [YesNo](#)("Save changes?") [when](#) 0 // <Escape> or Cancel [return](#) () [when](#) 1 // Yes selected [if](#) [not](#) [SaveFile\(\)](#) [return](#) () [endif](#) [endcase](#) [endif](#) [AbandonFile\(\)](#) [EditFile\(\)](#) [end](#) See Also: [MenuKey\(\)](#), [MenuOption\(\)](#), [MsgBox](#)

(), [MsgBoxBuff\(\)](#), [MsgBoxEx\(\)](#), [Warn\(\)](#)

ZeroBasedTab

Determines the offset used to compute `_HARD_` and `_SOFT_` tab stops.

Values: BOOLEAN **Default:** [OFF](#) **Notes:** When set [ON](#), fixed-interval tab stops (`_HARD_` and `_SOFT_`) are computed starting at 0. When set [OFF](#), fixed-interval tab stops are computed starting at 1. For example, if the `TabWidth` is set to 4, and the `TabType` is set to `_SOFT_`, the first 8 tab stops would be as follows. - With [ZeroBasedTab](#) set [ON](#): 4 8 12 16 20 24 28 32 - With [ZeroBasedTab](#) set [OFF](#): 5 9 13 17 21 25 29 33 For more information on Tabs, see the chapter on "[Using Tabs](#)" in [the User's Guide](#). **Affects:** [TabRight\(\)](#), [TabLeft\(\)](#) See Also: [TabType](#), [TabWidth](#), [VarTabs](#)

ZoomWindow

Zooms the current editing window to full screen; or if the window is already zoomed, restores the previous editing windows.

Syntax: [INTEGER ZoomWindow\(\)](#) **Returns:** Non-zero if successful; zero ([FALSE](#)) on failure. **Notes:** If there is only one editing window open on the screen, zero is returned. This command is a toggle, so issuing it twice leaves windows as they were. See Also: [OneWindow\(\)](#), [HWindow\(\)](#), [VWindow\(\)](#), [isZoomed\(\)](#)